

多时间尺度与动态电价下微网购电及储能协同调控模型

摘要

微网是将光伏等分布式电源、小区负载与储能设备组织在同一个电气边界内的小型发电系统。微网购电与储能调度受负载变化、光伏预测偏差及电价波动的共同影响。本文以非孤岛式微网为研究对象，以购电总费用最小化为目标，在满足负载需求和储能运行约束的条件下，结合时序预测、情景随机规划与滚动控制，构建涵盖计划购电、日内调整和紧急补电的购电—储能联合优化模型。

对于问题一，在电价、负载和光伏功率均已知的条件下，统一时段起点口径，采用相邻采样点的平均值近似区间平均功率，并换算为 10 分钟区间电量。在此基础上，建立满足能量平衡、储能状态递推及充放电互斥约束的确定性**混合整数线性规划**模型。典型日最优购电费用为 35205.89 元，较无储能方案下降 26.66%，且满足日初日末储电量相等的要求。

对于问题二，采用扩展窗口逐日滚动预测：负载由 **XGBoost** 融合周期、滞后和滚动统计特征进行预测，光伏采用**自适应加权方法**；再比较点预测、分位数裕量与场景随机规划等四种购电策略。最终方案 D 采用**两阶段场景随机 MILP** 制定日前合同，并以 **MPC** 滚动控制储能，其累计费用较点预测—贪心基准约节省 113.95 万元，降幅为 7.67%。

对于问题三，在第二问模型上加入 6:00、12:00 和 18:00 光伏预报更新和付费调单机制，连续回放 8 种时刻组合。全时刻方案在测试期比仅使用零点预报节省 676741.11 元；**Shapley** 分解表明 12:00 预报的日均贡献最高，最终建议按 12:00 → 18:00 → 6:00 的顺序配置。对于是否引入其他预报时刻，现有数据尚不足以支持无条件增加预报频次；若需进一步扩展，可优先在 6:00—12:00 范围内设置候选时刻，并通过滚动回放比较其边际节费与新增成本后确定。

对于问题四，采用滚动 **XGBoost** 预测 0、6、12 和 18 时发行的动态电价，并比较电价点预测与负载—光伏—价格联合误差场景。允许指定时刻调单的 4-3 方案在 334 个合同窗口中的现金费用为 13870785.24 元，较固定合同的 4-2 方案下降 3.7324%，紧急购电量由 178809.186 kWh 降至 50953.201 kWh。该差额同时包含光伏信息差异，不能完全归因于调单；联合价格场景的移动块 **Bootstrap** 区间跨越零，未显示稳定的额外经济优势。同时，滚动预测、场景规划与 **MPC** 的协同能够降低高价紧急补电风险，但收益仍受预测信息和有限历史窗口影响。

关键词：微网优化；XGBoost；随机混合整数线性规划；模型预测控制；动态电价

一、引言

1.1 问题背景

随着分布式光伏和储能设备在社区能源系统中的广泛应用，微网能够在本地消纳清洁能源，并通过储能设备实现电能在不同时间段之间的转移，从而减少对外部电网的依赖。然而，光伏发电具有明显的间歇性和不确定性，其出力与小区负载之间往往存在时间错配；与此同时，外部电网电价也可能随时段甚至随日期发生变化，使微网面临供电可靠性与运行经济性之间的协调问题。储能设备虽然能够缓解光伏出力和负载需求的不匹配，但其调度受到容量、充放电功率、转换效率以及初始储电状态等多重约束。

题目所提供的数据分别描述了典型日的预测信息、全年实际运行状态、分时光伏预测结果和全年动态电价，为研究微网从日前计划到预测更新再到实时价格响应的多阶段调控过程提供了数据基础。因此，微网购电策略本质上是一个同时考虑能源供需平衡、储能状态演化、预测误差和价格信号的时序优化问题，其核心是在保障小区负载得到满足的前提下，合理协调光伏利用、储能充放电与外部电网购电，以降低系统运行成本并提高能源利用效率。

1.2 问题重述

问题 1：日前计划购电策略 在每日电价和小区负载相同、已知一天光伏发电预测功率的条件下，于每日 0:00 制定当天的计划购电及储能充放电方案，要求日末储电量与日初相同，使全天购电费用最小。

问题 2：考虑全年运行差异的购电策略 在每日电价规律不变、负载和光伏出力随日期变化的条件下，根据每日 0:00 可获得的信息制定当天的计划购电策略。普通购电按计划电量计费，供电不足部分按交易时刻电价的 5 倍紧急购入，要求在满足负载需求的同时降低总购电费用。

问题 3：光伏预测更新下的计划调整 利用每日 0:00、6:00、12:00 和 18:00 发布的未来 24 小时整点光伏预报，制定零点购电计划及后续调整策略。考虑减购部分按相应电价 50% 计收的违约金、增购部分按 1.5 倍电价计收的费用及紧急购电费用，降低总费用，并判断是否有必要利用零点以外时刻的光伏预报。

问题 4：考虑波动电价的购电策略 在外网电价随日期和时刻变化的条件下，结合负载、光伏实际出力、光伏预报及电价数据，分别重新求解问题二和问题三，给出相应的购电与储能调度方案及费用。

二、总体分析

本题研究微网在负荷、光伏出力和外部电价时序变化下的购电与储能调度问题。四个问题由简入繁，沿着“不确定性增强”和“决策结构复杂化”两条主线逐步展开。

问题一将全年数据按 10 分钟时段分组求均值，构造典型日的电价、居民负荷和光伏预测功率，并将其视为确定性参数。其中，购电量与充放电量为连续变量，充放电状态为离散变量，电量关系和费用函数均可线性表达，故采用**混合整数线性规划**建立基础模型。在满足功率平衡和储能运行约束的条件下，制定全天购电及充放电计划，使购电成本最小。因此，问题一属于基于典型日数据的**确定性优化问题**。

问题二的困难在于购电计划先于实际需求确定，预测偏差需要在运行过程中消化。少购与多购的代价不同，偏差的持续时间还会影响电池后续的调节能力，因此需要同时考虑误差造成的购电损失及其在日内的累积影响。本文将日前购电与日内执行分开建模：日前利用历史误差构造供需场景，权衡提前购电与不同场景下的补购支出，形成**两阶段随机规划**；日内根据实际

运行状态，采用**模型预测控制（MPC）**重新安排电池充放电。以点预测规划和贪心执行为基准，先检验滚动控制的作用，再比较**分位数裕量与场景规划**，考察不同误差处理方法对实际购电费用的影响。

问题三在问题二的基础上引入**滚动决策机制**：在指定时刻更新未来 24 小时的光伏预测，并据此调整原有计划。本问在场景规划与 MPC 相结合的模型下，先比较八个预报时刻组合，以费用变化判断不同更新机会是否值得采用；随后固定全部预报时刻启用，补充场景规划与点预测规划、MPC 与贪心控制的对照，分别检验不确定性处理和日内控制的作用。由此，问题三形成了包含信息更新、序贯调整和非对称损失的多阶段动态优化问题。

问题四中，电价由已知的费用系数变为待预测量。供需预测偏差直接影响电量平衡，电价预测偏差则改变各时段购电与储能调节的经济性，二者共同作用于最终调度方案。因此，电价预测的评价需要从预测误差进一步延伸到实际购电费用。本文采用 **XGBoost** 预测未来电价，分别重新求解固定计划与可调整计划两类模型，并按实际价格结算运行费用，分析价格预测对购电和充放电安排的影响。在此基础上，比较电价点预测与联合误差场景方案，结合敏感性分析，判断增加价格风险处理能否带来稳定的费用改善。

总体而言，全文以购电与储能的联合调度为基础逐步展开。问题一建立单日确定性优化模型，作为后续研究的基准；问题二考虑预测偏差，将日前购电规划与日内电池控制相结合；问题三进一步引入购电计划修正，比较不同预报时刻组合下的调整收益与代价；问题四将电价预测纳入决策，分别重新求解固定计划与可调整计划两类模型。四问围绕同一调度问题，依次扩展预测误差的处理、预报信息的利用和波动电价下的决策。

整体思路框架如下：

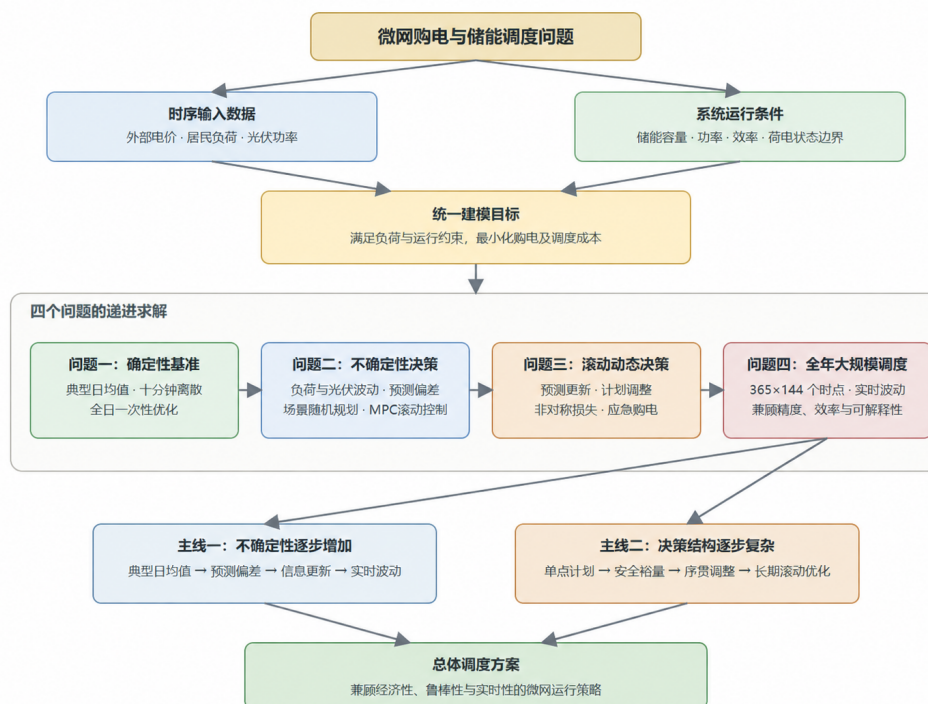


图 1 整体思路框架

三、附件数据画像

附件 1 共包含 144 个时点，每个时点记录时间、电价、小区负载功率和光伏发电预测功率四项信息。经核验，附件 1 中各时点的电价等于附件 4 中全年各日对应时刻数据的算术平均

值，各点时负载功率和光伏发电预测功率均等于附件 2 中全年各日对应时刻数据的算术平均值。附件 2 的负载、光伏与附件 4 的电价均有 365×144 个有效数值，缺失和负值数量均为 0；附件 3 有 $365 \times 4 \times 24$ 个整点预报。分组日期的空白单元格按组内日期继承，夜间光伏零值保留，不作均值插补。年末超出 2025 年的 36 个整点预报没有对应实测值，误差评价将其排除，而非填零。

表 1 附件数据概况

附件	原始结构	使用范围与检查
1	144 行 $\times$ 3 个数值字段	问题 1 使用全部字段；问题 2、问题 3 仅使用电价
2	两张 365×144 数值表	过去值用于预测；当段值用于反馈；全量数据用于事后检验
3	1460 次发布 $\times$ 24 个整点	按发布时间对齐；仅选取当日剩余目标时段
4	365×144 电价	过去值用于预测；当段值用于反馈；实际账本用于结算
5	五个结果模板	保留主体结构，补充 334 天数据及应急事件，并修正区间标签

四、计算精度约定

为统一结果的数值精度，本文各问成本的计算结果统一保留两位小数，其余各量的计算结果均保留四位小数。

五、模型假设及论证

为简化问题，本文做出以下假设：

- **假设 1** 微网不能向外部电网反向售电，当光伏发电在满足小区负载和储能充电功率约束后仍有剩余时，剩余电量可以抛弃。因此约束中保留弃光变量。
- **假设 2** 储能设备的充放电效率在整个运行区间内为常数，不随荷电状态与充放电倍率变化。真实电池的效率会随倍率略有下降，但题目只给出单一效率值，本论文与题意一致的处理。储能设备不计自放电与循环老化。微网内部的线路损耗与变压器损耗忽略不计，所有电量在同一节点上平衡。
- **假设 3** 针对问题一，我们观察到附件 1 中各时刻的电价、负载功率和光伏发电预测功率均由附件 4 中全年各日同一时刻的数据取平均得到。考虑日内数据的周期性，将表中次日 0:00（即 0:00 + 1）的数据视为当日 0:00 的数据；该处理仅用于统一时间索引，不影响模型的计算结果。
- **假设 4** 认为附件 1-4 中的数据均是**瞬时值**。在实际求解中，采用区间两端时刻功率的算术平均值表征该时段的平均功率。
- **假设 5** 在每日制订购电计划时，假设当日 0:00 时刻的电价、负载功率和光伏发电功率均已知，并据此确定当日各时段以及次日 0:00–0:10 时段的购电计划。
- **假设 6** 依据题目描述，认为储能设备的充电和放电效率分别为 90%。对原题描述的另一种理解为充放电效率总体效率为 90%，我们将在敏感性验证中检验其敏感性。

六、符号说明

表 2 全文主要符号

符号	说明
$t, T, \Delta t$	时段编号、规划窗口内时段数及单段时长；通常 $T = 144$, $\Delta t = 1/6 \text{ h}$, 前两者无量纲。
$L_t, \hat{L}_{d,k}, P_{d,k}^{\text{PV}}, \hat{P}_{d,k}^{\text{PV}}$	负载区间平均功率及其预测值、光伏区间平均功率及其预测值, 单位均为 kW。
ℓ_t, E_t^{PV}	单个时段的负载电量、光伏发电量, 单位均为 kWh。
$p_t, \bar{p}_{d,t}, \hat{p}_{d,t}^{(h)}$	模型使用的电价、区间平均电价及在时刻 h 发布的电价预测值, 单位均为元/kWh。
g_t, q_t, e_t	原计划购电量、调整后的普通购电量、紧急购电量, 单位均为 kWh。
$a_t, \rho_t, v_{t,s}$	相对原计划增加的购电量、取消的购电量及场景 s 下未利用的已购普通电量, 单位均为 kWh。
c_t, d_t, r_t	微网侧充电量、放电及弃光量, 单位均为 kWh。
$S_t, S_0, S_{\min}, S_{\max}$	对应时间边界的储电量、运行起点储电量及储电量下限、上限, 单位均为 kWh; S_0 不表示每天重置的储电量。
$P_{\max}, \eta_c, \eta_d$	最大充放电功率、充电效率及放电效率; $P_{\max} = 5000 \text{ kW}$, 主模型 $\eta_c = \eta_d = 0.9$, 效率无量纲。
u_t	充放电方向变量, 取 1 允许充电, 取 0 允许放电, 无量纲。
$\omega, \pi_\omega;$	各问采用的场景编号及对应概率, 均无量纲。
s, π_s	
$Z, J_{\text{cash}}, J_{\text{eval}}$	计划购电费用、实际现金费用及库存价值修正后的评价指标, 单位均为元。

注: d 为日期编号, k 为日内时段编号, h 为预报或决策更新时刻; 帽号表示预测值, 场景下标表示对应场景中的变量。补充符号见附录 A。

七、问题一的模型的建立和求解

7.1 解题思路

问题一是一个确定性的优化问题, 首先将典型日划分为 144 个 10 分钟时段, 并将负荷与光伏功率转换为对应时段的电量。以购电量、充电量、放电量、弃光量和储电量为连续决策变量, 并引入二元变量约束储能的充放电状态。在满足能量平衡、储电量递推、充放电功率、储能容量及日初日末电量相等约束的条件下, 建立以全天购电费用最小为目标的混合整数线性规划模型。随后利用 HiGHS 求解器获得最优购电与储能调度方案, 并逐项检验能量平衡和设备运行约束。最后将优化结果与无储能情形进行比较, 评价储能调度带来的经济效益。

7.2 数据处理与分析

7.2.1 数据预处理

依据假设 3, 将附件 1 次日 0:00 时刻负载功率作当日 0:00 负载功率。在一段 10min 内负载平均功率与光伏发电平均效率均根据假设 4 计算得出:

$$\bar{x}_t = \frac{x_{t-1} + x_t}{2}, \quad t = 1, \dots, 144,$$

其中, x_t 表示第该天第 t 个端点的相应变量值, $\bar{x}_t$ 表示对应区间均值。

7.2.2 数据分析与可视化

基于附录一的典型日数据, 绘制小区负载、光伏预测功率及净负荷 (小区负载减去光伏功率) 的日内变化曲线。如图 2, 光伏功率在 12 时左右达到峰值, 小区负载在 8:00—18:00 期间维持在较高水平。约 10:00—15:00, 净负荷降至 0 以下, 表明光伏出力超过同期负荷, 形成富余光伏电量。该部分电量可优先用于储能充电; 当储能充电能力不足时, 剩余电量则需要通过弃光处理。

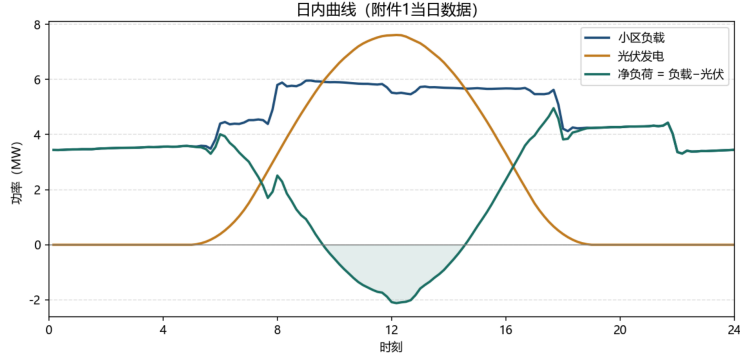


图 2 小区负载、光伏预测功率及净负荷的日内变化曲线

7.3 模型建立

问题一中购电量、充放电量和储电量为连续变量, 充放电互斥状态为二元变量, 目标函数及约束均为线性关系, 因此建立混合整数线性规划模型。令

$$\mathcal{T} = \{1, 2, \dots, 144\}, \quad \Delta t = \frac{1}{6} \text{ h}.$$

各时段的能量平衡约束为

$$g_t + \hat{E}_t^{\text{PV}} - r_t + d_t = L_t \Delta t + c_t, \quad t \in \mathcal{T}. \quad (1)$$

购电量、弃光量及充放电状态满足

$$\begin{cases} g_t \geq 0, & 0 \leq r_t \leq \hat{E}_t^{\text{PV}}, \\ 0 \leq c_t \leq P_{\max} \Delta t u_t, \\ 0 \leq d_t \leq P_{\max} \Delta t (1 - u_t), \\ u_t \in \{0, 1\}, \end{cases} \quad t \in \mathcal{T}. \quad (2)$$

储电量按充放电效率递推，并满足容量及首末状态约束：

$$\begin{cases} S_t = S_{t-1} + \eta_c c_t - \frac{d_t}{\eta_d}, \\ 1200 \leq S_t \leq 10800, \\ S_0 = S_{144} = 6000, \end{cases} \quad \eta_c = \eta_d = 0.9. \quad (3)$$

模型以全天外部电网购电费用最小为目标：

$$\min Z = \sum_{t \in \mathcal{T}} p_t g_t. \quad (4)$$

因此，式 (1)–(4) 共同构成问题一的确定性混合整数线性规划模型。

7.4 模型求解

7.4.1 模型求解方法

问题一建立的模型包含购电量、充放电量、弃光量和储电量等连续变量，以及充放电状态二元变量，因此属于**混合整数线性规划模型**。程序采用 Python 实现，调用 `scipy.optimize.milp` 接口，并使用其内置的 HiGHS 求解器进行优化。

首先，程序读取附录一中的电价、负荷功率和光伏预测功率。由于每个调度时段为 10 分钟，将功率数据统一转换为对应时段的电量：

$$L_t^E = L_t \Delta t, \quad \hat{E}_t^{\text{PV}} = \hat{P}_t^{\text{PV}} \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}.$$

随后，将购电量、充电量、放电量、弃光量、储电量和充放电状态变量组合为决策向量，并利用稀疏矩阵构造能量平衡约束、储电量递推约束及充放电互斥约束。目标函数仅对外部电网购电量计价，求解参数设置为相对最优间隙 10^{-9} ，最大求解时间为 60 秒。

求解完成后，程序重新计算各时段的能量平衡误差、储电量递推误差、容量边界、充放电功率边界、弃光上限和二元状态约束，并检查储能设备是否存在同一时段同时充放电的情况，以验证最优解的可行性。

7.4.2 模型求解结果

根据上述模型得到问题一的最优调度方案。全天外部电网购电量为 59540.1847 kWh，对应的购电费用为 35205.89 元。储能设备日初和日末的储电量均为 6000 kWh，满足周期性约束；全天储电量始终处于 1200 ~ 10800 kWh 的允许范围内。优化结果中的弃光量为 0 kWh，HiGHS 求解器报告的 MILP 相对最优间隙为 0，表明所得结果达到当前模型的最优解。

题目表 1 所要求的指定时段购电量如表 3 所示。

题目表 2 所要求的六个时段充放电量如表 4 所示。

表3 问题一指定时段购电量

时间段	购电量 (kWh)	时间段	购电量 (kWh)
10:00–10:10	0.0000	12:00–12:10	483.4077
14:00–14:10	0.0000	16:00–16:10	420.1816
18:00–18:10	639.0720	20:00–20:10	0.0000

表4 问题一各时段储能充放电量

时间段	充电量 (kWh)	放电量 (kWh)
0:00–4:00	4500.0000	0.0000
4:00–8:00	833.3333	6452.2006
8:00–12:00	4982.7063	1903.4957
12:00–16:00	5393.8493	49.3138
16:00–20:00	0.0000	5066.4749
20:00–24:00	5333.3333	3573.5251

为评价储能调度的经济作用，本论文额外计算不配置储能设备时的购电费用。此时仅利用光伏满足小区负荷，不进行跨时段能量转移，购电费用为 48004.76 元。与无储能情形相比，优化后的购电费用减少 12798.88 元，降幅约为 26.66%。这表明储能设备能够在低价时段充电、在高价时段放电，从而降低微网的全天购电成本。

八、问题二的模型建立和求解

8.1 解题思路

问题二在问题一的基础上，进一步考虑负载与光伏功率的预测不确定性，以及供电不足时按相应时段电价 5 倍计费的紧急购电机理。因此，在沿用储能运行等基本约束的基础上，从时序预测与风险决策两个方面展开求解：

1. **先采用逐日滚动预测负载与光伏**，仅利用预测时刻可得的历史数据，逐日滚动预测目标日 144 个时段的负载与光伏功率。以七日历史均值为基准，比较 XGBoost、LightGBM 的预测性能，并引入日总量 SARIMA 结合日内曲线分配的统计对照，评估各方法的适用性。针对光伏的季节性、日内周期及天气扰动，在缺少气象预报变量的条件下，进一步考察晴空包络短期外推、日总量与归一化日曲线分解、自适应加权等改进方法。

2. **再根据预测结果和缺电风险制定购电计划**。构建“日前购电规划—日内储能控制”两层决策框架：日前确定普通购电量，日内根据运行反馈调整储能动作；围绕预测误差处理和储能控制方式构建 A, B, C, D 四个对照模型，通过连续回放比较实际购电费用，确定优选方案。

8.2 数据处理与分析

8.2.1 数据预处理

根据假设 5，每日 00:00 的电价、负载及光伏功率均已知，可直接作为首个区间的左端点，负载与光伏预测则需覆盖此后 144 个未来端点（包括次日 00:10），因此涉及一日的负载与光伏预测涉及共 145 个端点。

历史数据中，当日 00:00 与前一日 24:00 对应同一时刻，因此采用前一日末端观测补全零点边界。对于缺少前一日记录的 2025 年 1 月 1 日，以当日 24:00 的数值近似补齐零点数据。该近似直接影响首日第一个区间，首日不属于自 2 月 1 日开始的正式规划期。因此在上述处理流程下，该补值不影响正式预测结果。

对于每个 10min 区间的平均功率或平均电价，问题二采用与问题一一致的梯形近似，将每日某时刻的瞬时负载功率、光伏功率及电价统一转换为 10 分钟区间的平均值：

$$\bar{x}_{d,k} = \frac{x_{d,k-1} + x_{d,k}}{2}, \quad k = 1, \dots, 144,$$

其中， $x_{d,k}$ 表示第 d 天第 k 个端点的相应变量值， $\bar{x}_{d,k}$ 表示对应区间均值。

最后，以日期和时段序号统一组织实测数据与预测数据，确保两者的区间起止时刻一致，为预测误差评价及后续购电优化提供统一的数据输入。

8.2.2 数据分析与可视化

本节旨在结合工作日与周末平均日内曲线，识别负载与光伏功率的多时间尺度变化，并进一步绘制二者的全年日内热力图和月度分布箱形图，突出二者年际波动水平。统计图表直接采用附件 2 中的原始端点功率，用于描述数据特征；后续预测评价与购电优化仍沿用前述区间平均口径。

负载与光伏呈现不同的日内变化形态（图3）。负载在白天维持较高水平，工作日平均负载整体高于周末，表明预测需同时考虑日内时段与星期周期波动。光伏功率则呈午间单峰、夜间接近零的变化形态，工作日与周末的平均曲线接近，日内时刻对其变化的刻画更为直接。

全年热力图进一步保留了平均曲线中被弱化的日期差异（图4）。负载的白天高值区域贯穿全年，但不同日期的整体水平存在差异，7 月至 8 月的负载水平相对较高。光伏高值集中于午间，其强度随日期变化，呈现年内变化与逐日波动并存的特征，说明仅用一条固定日曲线难以充分描述全年功率变化。

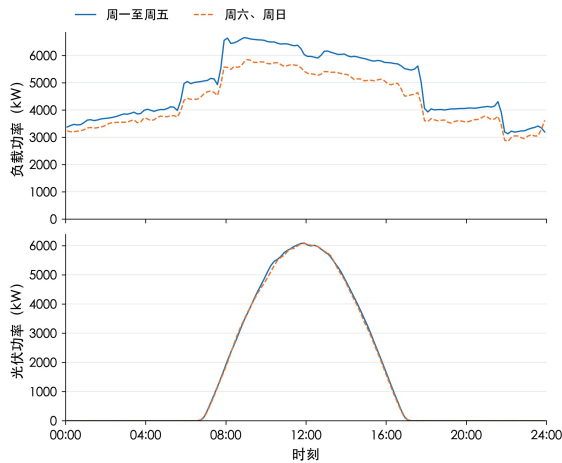


图 3 负载与光伏的工作日、周末平均日内曲线

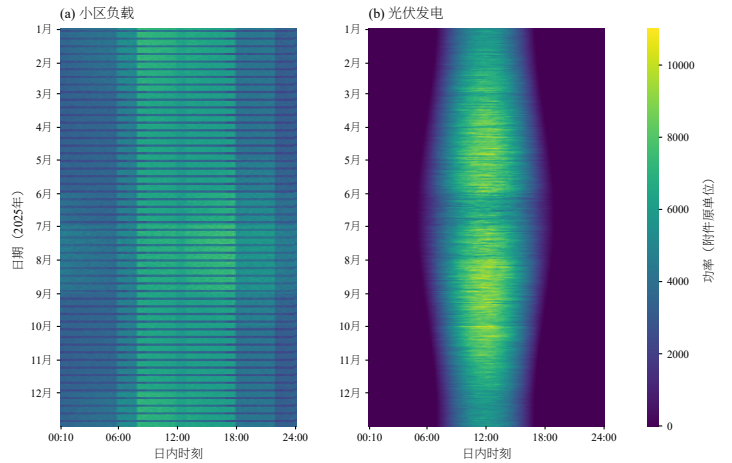


图 4 2025 年负载与光伏功率的全年日内分布

以每日 144 个端点功率的算术平均值作为日均功率，按月份比较其分布（图5）。负载月均水平在 7 月最高，光伏月均水平在 8 月最高、12 月最低，且 6 月低于 5 月，表明光伏的年内变化并非单调。月内散点还显示，负载存在较清晰的高、低值分组，而光伏在同一月份内仍有逐日差异。

据此，负载预测采用时段、星期、年内周期编码、滞后项及滚动统计量等特征，并通过 XGBoost 等模型学习特征间的非线性关系与交互作用。光伏预测在历史同期基准的基础上，进一步考察自适应加权及历史高分位晴空包络短期外推等方法，以兼顾日内形态与近期变化；各方法的实际预测效果通过后续滚动回测比较。

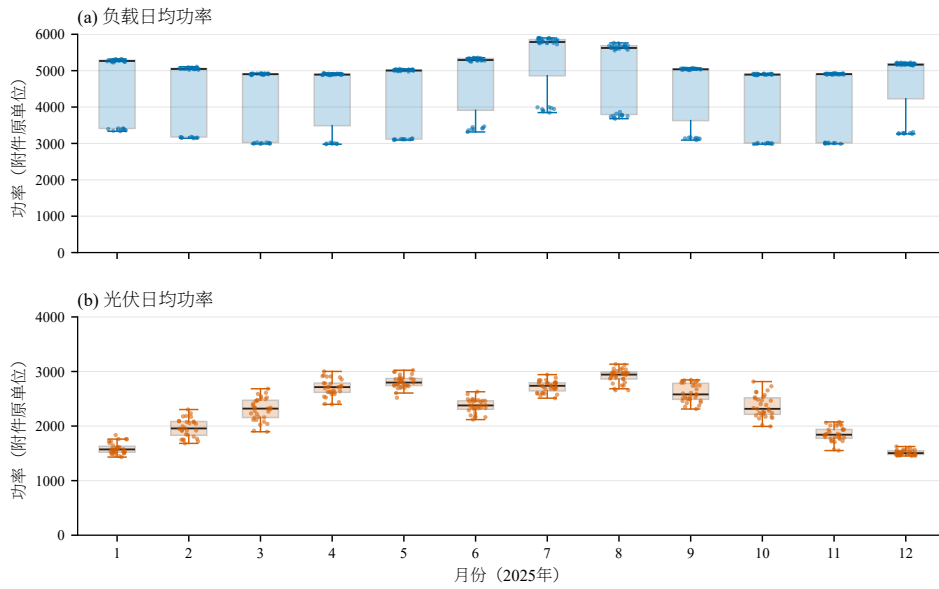


图5 负载与光伏日均功率的月度分布。日均功率为每日 144 个原始端点值的算术平均；箱体表示四分位区间，横线表示中位数，须延伸至 1.5 倍四分位距范围内的最远样本，散点表示全部日期，各月样本数为 28 至 31 天。

8.3 模型建立

8.3.1 跨日边界处理

由于购电计划的时间范围由当日 0:00 延伸至次日 0:10，共涉及 145 个 10 分钟时段。由于当日 0:00 的功率已知，模型还需预测其后 144 个端点，其中次日 0:10 超出了常规日内模型的预测范围。依据常识与光伏数据在 0:00 处稳定性，午夜时段光伏功率稳定为零，因而跨日区间的不确定性主要来自负载功率。为补充次日 0:10 的负载端点，本文进一步建立跨日负载边界预测模型，并将该预测值与当日 24:00 的负载预测值结合，确定 24:00 至次日 0:10 时段的平均负载。

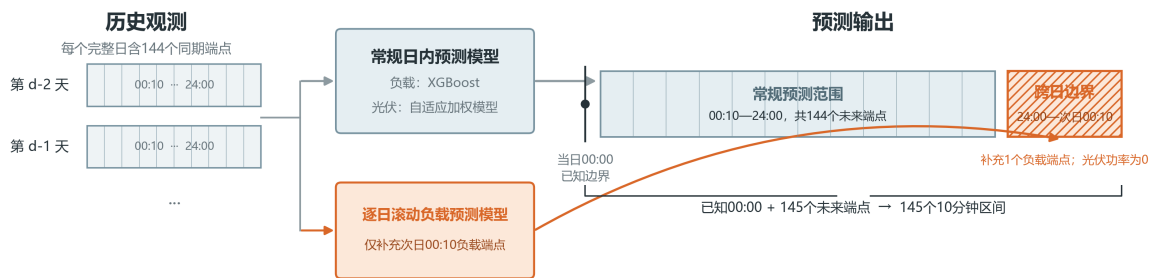


图6

为确定次日 00:10 的负载边界值，本文构建 XGBoost 跨日边界模型、历史跨日比例法和目标时刻上周同期法三种候选方法；分别从次日 00:10 端点预测误差，以及该端点与当日 24:00 预测值共同形成的最后 10 分钟区间预测误差两个层面评价各方法。结果表明，历史跨日比例法在单一端点预测中表现较优；但在最终区间平均值预测中，目标时刻上周同期法能够与当日 24:00 预测误差形成更有效的抵消，综合表现最佳。因此，本文采用目标时刻上周同期法处理跨日负载边界。

8.3.2 负载与光伏功率预测模型

问题二要求微网在每日零点制定购电计划，此时当日未来时段的负载与光伏功率尚未确定。为模拟实际决策过程，本文采用扩展窗口逐日滚动预测方法 [1]：预测第 d 天时，仅使用第 $d-1$ 天及以前已经获得的历史数据。记预测日前的信息集为

$$\mathcal{D}_d = \{(L_{j,k}, P_{j,k}^{\text{PV}}) : j < d, k = 1, 2, \dots, 144\}, \quad (5)$$

其中， $L_{j,k}$ 和 $P_{j,k}^{\text{PV}}$ 分别表示第 j 天第 k 个时刻的负载功率与光伏功率。完成第 d 天的预测后，待当日真实数据获得，再将其加入历史信息集并预测第 $d+1$ 天，从而避免未来信息泄露。

模型首先预测目标日 00:10 至 24:00 的 144 个端点值，再按照前述数据预处理方法转换为相应区间的平均功率。负载与光伏具有不同的变化机理，因此分别建立预测模型。

负载功率预测模型 小区负载同时受到日内用电规律、工作日属性、季节变化及近期负载水平的影响，且不同因素之间存在非线性交互。为此，本文采用 XGBoost[2] 建立负载预测模型，并从历史负载序列中构造以下特征：

$$\mathbf{x}_{d,k}^L = (\mathbf{x}_{d,k}^{\text{time}}, \mathbf{x}_{d,k}^{\text{lag}}, \mathbf{x}_{d,k}^{\text{roll}}), \quad (6)$$

其中， $\mathbf{x}_{d,k}^{\text{time}}$ 包括时段序号、星期属性、是否为周末以及年内日期的正余弦编码； $\mathbf{x}_{d,k}^{\text{lag}}$ 包括前 1、2、7 和 14 天同一时刻的负载功率； $\mathbf{x}_{d,k}^{\text{roll}}$ 包括最近 7 天和 14 天同一时刻负载功率的均值、标准差与中位数。

设模型共包含 M 棵回归树，则第 d 天第 k 个时刻的负载预测值为

$$\hat{L}_{d,k} = \sum_{m=1}^M f_m(\mathbf{x}_{d,k}^L), \quad (7)$$

其中， $f_m(\cdot)$ 表示第 m 棵回归树。模型通过逐步拟合前序树的预测残差，提高对负载非线性变化的刻画能力。其训练目标由预测误差与模型复杂度惩罚两部分构成：

$$\mathcal{L}_L = \sum_{(j,k) \in \mathcal{D}_d} l(L_{j,k}, \hat{L}_{j,k}) + \sum_{m=1}^M \Omega(f_m), \quad (8)$$

其中， $l(\cdot)$ 为预测损失函数， $\Omega(f_m)$ 为第 m 棵树的复杂度惩罚项，用于抑制模型过拟合。

为形成跨日 00:00 至 00:10 时段的负载预测，还需补充预测次日 00:10 的负载端点。根据负载的星期周期，采用目标时刻上周同期值进行预测：

$$\hat{L}_{d+1,00:10} = L_{d-6,00:10}. \quad (9)$$

该预测值与主体模型给出的目标日 24:00 负载预测值共同用于确定跨日区间的平均负载功率。

光伏功率预测模型 光伏功率具有稳定的日内变化轨迹，但同时受到季节变化与短期天气扰动的影响。在未提供目标日云量、辐照度等气象预报的条件下，本文利用近期历史同期功率建立自适应加权预测模型。

针对第 d 天第 k 个时刻，分别构造前 1 天同期、前 2 天同期、前 7 天同期以及此前 7 天同

期均值四个基础预测：

$$\begin{aligned}\hat{P}_{d,k}^{\text{PV},(1)} &= P_{d-1,k}^{\text{PV}}, \\ \hat{P}_{d,k}^{\text{PV},(2)} &= P_{d-2,k}^{\text{PV}}, \\ \hat{P}_{d,k}^{\text{PV},(3)} &= P_{d-7,k}^{\text{PV}}, \\ \hat{P}_{d,k}^{\text{PV},(4)} &= \frac{1}{7} \sum_{r=1}^7 P_{d-r,k}^{\text{PV}}.\end{aligned}\quad (10)$$

为使模型能够根据近期天气状态与预测效果动态调整各历史信息的贡献，选取预测日前最多 28 天的数据构成滚动权重估计窗口 $\mathcal{W}_d$ ，并通过约束非负最小二乘确定权重：

$$\min_{w_d} \sum_{j \in \mathcal{W}_d} \sum_{k=1}^{144} \left[P_{j,k}^{\text{PV}} - \sum_{m=1}^4 w_{m,d} \hat{P}_{j,k}^{\text{PV},(m)} \right]^2, \quad (11)$$

满足

$$w_{m,d} \geq 0, \quad \sum_{m=1}^4 w_{m,d} = 1. \quad (12)$$

由此，第 d 天第 k 个时刻的光伏功率预测值为

$$\hat{P}_{d,k}^{\text{PV}} = \sum_{m=1}^4 w_{m,d} \hat{P}_{d,k}^{\text{PV},(m)}. \quad (13)$$

该模型利用历史同期值保留光伏功率的日内结构，通过 7 天同期均值削弱单日天气扰动，并利用滚动权重反映近期变化。对于跨日 00:00 至 00:10 时段，由于该时段处于无光照条件下，将次日 00:10 的光伏功率预测值设为零。

8.3.3 基于预测的购电方案

本问需要在负载和光伏发电存在预测误差的条件下制定购电策略。普通购电计划在每日零点确定，未使用的计划电量仍需付费；运行中出现的供电缺口则需按当时电价的五倍紧急补购。因此，模型需要同时确定各时段的普通购电量，以及储能在实际运行中的分配方式。

据此，将决策划分为日前购电规划层和日内储能控制层。日前层采用混合整数线性规划 (MILP)，根据预测信息和当前储电量确定普通购电计划；日内层在购电计划固定后，根据运行反馈调整储能动作。为分别考察滚动控制和预测误差处理的作用，构建以下四种模型。

表 5 四种购电方案的模型构成

模型	日前购电规划	日内储能控制	设置目的
A	点预测确定性 MILP	贪心控制	建立比较基准
B	点预测确定性 MILP	LP-MPC	考察滚动控制的作用
C	净负载误差裕量修正后的 MILP	与 B 相同	考察单一路径风险修正的作用
D	两阶段场景随机 MILP	与 B 相同	考察多场景期望成本优化的作用

四类购电策略的构建 A 与 B 的区别在于日内控制方式, B、C、D 的区别在于日前规划对预测误差的处理方式。其中, C 与 D 是两种可替代的不确定性处理方法, 其适用性通过实际运行费用进行比较。

模型 A: 点预测规划与贪心控制 将负载和光伏点预测代入日前 MILP, 以普通购电费用、预测条件下的紧急购电费用及期末储能不足惩罚之和最小为目标:

$$\min \sum_{t=1}^T p_t g_t + \sum_{t=1}^T 5p_t e_t + \kappa \xi, \quad \xi \geq 6000 - S_T, \quad \xi \geq 0. \quad (14)$$

其中, p_t 为时段电价, T 为规划期末, κ 为储能不足惩罚系数。该软惩罚用于抑制有限规划时域内过度消耗库存, 不要求期末储电量恢复至 6000 kWh, 也不计入实际电费。购电计划确定后, 实际运行采用贪心控制: 富余时在容量和功率限制内尽量充电, 出现缺口时尽量放电, 剩余缺口由紧急购电补足, 由此形成基准模型 A。

模型 B: 点预测规划与 LP-MPC 模型 B 沿用 A 的日前规划, 仅将日内贪心控制替换为 LP-MPC。立即放电虽可减少当前缺口, 却可能削弱电池应对后续高价时段的能力, 因此需要结合未来需求和电价分配储能。

在第 t 段, 以已知储电量 S_{t-1} 为初态, 对剩余时段 $\tau = t, \dots, T$ 定义供电富余 U_τ 和供电缺口 V_τ :

$$\begin{aligned} & \min \sum_{\tau=t}^T 5p_\tau e_\tau + \kappa \xi, \\ \text{s.t. } & U_\tau = \max(g_\tau + E_\tau^{\text{PV}} - \ell_\tau, 0), \\ & V_\tau = \max(\ell_\tau - g_\tau - E_\tau^{\text{PV}}, 0), \\ & 0 \leq c_\tau \leq \min\left(U_\tau, \frac{5000}{6}\right), \\ & 0 \leq d_\tau \leq \min\left(V_\tau, \frac{5000}{6}\right), \\ & e_\tau = V_\tau - d_\tau. \end{aligned} \quad (15)$$

同时满足共同的储能状态转移、容量边界及期末软约束。每隔 10 分钟滚动求解剩余时域的充放电方案, 仅执行当前一段, 再根据更新后的储电状态重新优化; 这种滚动时域思想已广泛用于微网电池储能控制 [3]。回放时, 以当前段实际均值近似段内快速反馈, 未来时段仍使用当时已发布的预测。A、B 在相同合同、预测和初态下的对照, 可用于评估滚动控制的作用。

模型 C: 净负载误差裕量规划 模型 B 的日前合同依据单一路径确定, 尚未显式处理预测偏差带来的经济风险。受报童模型中“过量准备与准备不足的损失不对称”思想启发, 考虑到计划电量不足可能引发高额紧急购电费用, 模型 C 利用历史净负载预测误差, 对日前需求进行适度上偏修正。定义历史窗口 j 在时段 t 的误差及相应裕量:

$$\epsilon_{j,t} = (\ell_{j,t}^{\text{act}} - h_{j,t}^{\text{act}}) - (\hat{\ell}_{j,t} - \hat{h}_{j,t}), \quad (16)$$

$$b_t = \max\{Q_q(\epsilon_{\cdot,t}), 0\}, \quad \tilde{\ell}_t = \hat{\ell}_t + b_t. \quad (17)$$

其中, 历史误差由当时发布的预测与后续实际值之差得到。分位数水平 q 根据校准期运行

费用确定后固定，并设置零裕量候选。将修正后的负载 $\tilde{\ell}_t$ 与原光伏预测代入同一日前 MILP，得到模型 C 的购电计划；日内仍采用 B 的 LP-MPC，并使用原始预测，不重复叠加裕量。

模型 D：两阶段场景随机规划 模型 C 以单一路径表示风险，模型 D 采用两阶段随机规划与场景近似的基本思想 [4]，从同一历史误差库中抽取 8 个完整窗口的负载-光伏成对残差，分别叠加至当前预测，并进行非负裁剪。整段抽取有助于保留日内误差形态，成对抽取有助于保留负载与光伏误差的联动关系，各场景概率取 $\pi_\omega = 1/8$ 。类似的概率预测与场景优化方法也已用于不确定条件下的储能调度 [5]。

第一阶段确定各场景共享的普通购电计划 g_t ；第二阶段为各场景配置充放电、紧急购电及储电状态轨迹，目标函数为：

$$\min \sum_{t=1}^T p_t g_t + \sum_{\omega} \pi_{\omega} \left(\sum_{t=1}^T 5p_t e_{\omega,t} + \kappa \xi_{\omega} \right). \quad (18)$$

各场景分别满足共同物理约束及期末软约束。该模型通过最小化期望成本，权衡多购电可能造成的已付费电量闲置与少购电可能引起的紧急补购费用。实际运行仍采用与 B、C 相同的 LP-MPC。

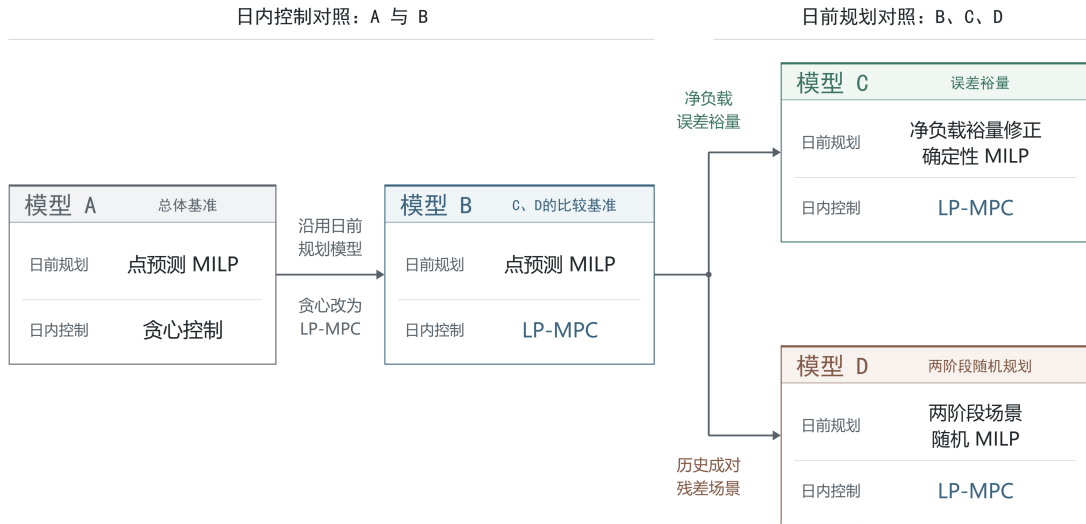


图 7 四种购电模型的构建与选择思路

8.4 模型求解及结果

8.4.1 负载与光伏预测结果

预测模块采用扩展窗口逐日滚动回测，评价期为 2025 年 2 月 1 日至 12 月 31 日，共 334 天。每天零点根据此前历史数据预测未来 144 个端点，并结合已知零点观测，将相邻端点平均为 10 分钟区间功率；预测误差按自然日区间评价。

负载预测采用 XGBoost，综合日内、星期和年内周期、历史滞后项及滚动统计特征。其加权绝对百分比误差（WAPE）为 2.443%，较上周同期基准的 3.435% 相对下降约 28.9%，RMSE 由 225.418 降至 154.669。

光伏预测采用自适应加权历史模型，根据此前最多 28 天的预测表现，动态组合前 1、2、7 天同期值及前 7 天同期均值。其 WAPE 由固定 7 天均值模型的 6.357% 降至 6.303%。这一改善幅度较小，表明在缺少日前气象信息的条件下，增加模型复杂度未必能提高预测精度。后续购电计算采用 XGBoost 负载预测与自适应加权光伏预测作为输入。

8.4.2 购电方案比较与优选

日前模型先通过 MILP 确定购电计划，再固定整数模式进行 LP 精修；日内按各方案对应的控制方式逐段执行。四种方案均从 6000 kWh 开始连续回放，储电状态在窗口间连续传递。购电评价覆盖 334 个窗口，共 48096 个时段；每窗从本日 00:10 延续至次日 00:10，最终执行至 2026 年 1 月 1 日 00:10。

实际购电费用由普通购电费与紧急购电费组成，不包含期末储能软惩罚。连续运行结果见表 6，金额单位为万元，各项独立四舍五入。

表 6 四种购电方案的连续运行结果

方案	普通购电费	紧急购电费	总购电费	期末储电量/kWh
A	1222.13	264.26	1486.40	1200.00
B	1221.79	236.92	1458.71	1200.00
C	1237.63	182.08	1419.72	1200.00
D	1281.94	90.50	1372.45	1603.23

方案 D 的累计购电费用最低，较基准方案 A 节省 113.95 万元，降幅为 7.67%。虽然 D 的普通购电费用较高，但紧急购电费用由 264.26 万元降至 90.50 万元，下降约 65.75%，抵消了普通购电支出的增加。其期末储电量也高于其他方案，按统一库存价值修正后的费用指标仍最低，说明费用优势并非通过额外消耗期末库存取得。

按窗口起点月份统计，D 在 11 个月中的 10 个月费用最低，仅 12 月由 C 取得最低费用。为区分控制方式与日前规划的作用，另在 310 个窗口中逐窗固定初态进行配对比较：同一购电合同下，B 较 A 累计节省 28.05 万元；采用相同 MPC 时，C、D 较 B 分别节省 37.69 万元和 76.43 万元。这些局部实验支持滚动控制与预测误差处理的作用，其费用不并入连续运行账单。

方案 D 的日前平均求解时间为 7.01 秒，高于 A、B、C 的不足 0.02 秒；334 个窗口中有 50 个未证达到预设求解精度，最大相对间隙为 0.017929%，所得可行解经物理约束核验后用于回放。因此，D 是本次预测输入、8 场景及固定随机种子下的优选方案，该结论表示历史运行比较中的费用优势，不代表已证明其在所有预测和参数条件下均为全局最优。

其中，方案中的各指定日期在各时间段的购电量及全天的购电量和购电费、在各时间段的充放电量及 0:00 和 24:00 的储电量和在指定日期的紧急购电量均见论文附录。

九、问题三的模型的建立和求解

9.1 解题思路

问题三在问题二的基础上引入了日内滚动决策机制：除每日 0:00 制定原始购电计划外，还可在 6:00、12:00 和 18:00 获取更新后的光伏预测，并据此付费调整后续购电量。尽管新增预报能够提供更充分的信息，但受预测误差、非对称调单费用及滚动决策误差传递的共同影响，增加预报更新时刻未必能够降低策略的实际购电成本。因此，本文从两个层面展开研究：

1. 在统一初始状态、物理约束和评价口径的条件下，比较由 6:00、12:00 和 18:00 三个可选更新时刻构成的八种策略组合，分析不同更新时刻组合对总购电成本的影响；

2. 是在所选更新时刻组合下，通过场景随机规划与确定性规划、MPC 与贪心控制的对照实验，分别评价不确定性建模和滚动控制方法对购电费用、紧急购电量及储能状态的影响。

基于第一点的分析，我们通过 Shapley 值衡量各更新时刻对购电成本节约的平均边际贡献，以及不同更新时刻的经济价值。

9.2 数据可视化分析

为比较不同发布时间获取最新光伏信息的总体价值，本文分别统计延伸预报与对应时刻最新预报在相同评价口径下的区间平均绝对误差（MAE）。如图 8 所示，灰色点表示延伸预报的 MAE，蓝色点表示最新预报的 MAE，连线表示两种预报之间的误差差异，标注数值为相对 MAE 改善率。

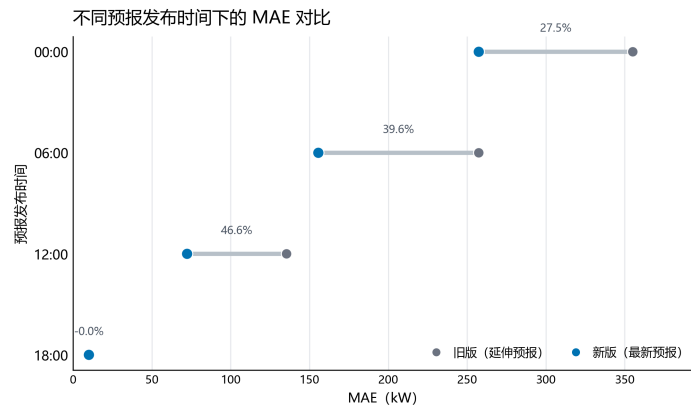


图 8 不同预报发布时间下的总体 MAE 对比

由图 8 可知，12:00 更新后的 MAE 由 135.32 kW 降至 72.28 kW，相对改善率达到 46.6%，高于其余所有发布时间。因此，在本数据集和当前评价口径下，12:00 获取的新预报对后续光伏预测误差的修正最为明显。

相比之下，18:00 时相对改善率接近于零。这主要与 18:00 之后光伏出力较低、绝对误差整体较小有关。因此，18:00 组的结果应解释为最新预报相对于延伸预报的边际改善有限，而不能简单理解为该时刻的预报完全没有作用。

本文将 MAE 改善率展开为“预报发布时间—目标时刻”的二维矩阵，如图 9 所示。每个单元格表示在同一目标时刻上，最新预报相对于延伸预报的 MAE 改善率；灰色单元格表示该发布时间尚未覆盖相应目标时刻，不应视为零改善。

图 9 显示，在白天具有明显光伏出力的目标区间内，较短预报时距通常对应更高的 MAE 改善率。以 06:00 发布的预报为例，07:00 至 17:00 目标时刻的改善率主要处于 15.9% 至 57.6% 之间，而 19:00 以后逐渐下降至接近于零。

这表明在白天有效光伏出力区间内，目标时刻越接近预报发布时间，最新预报通常能够带来更明显的误差修正，体现出短时距预测的准确性优势。

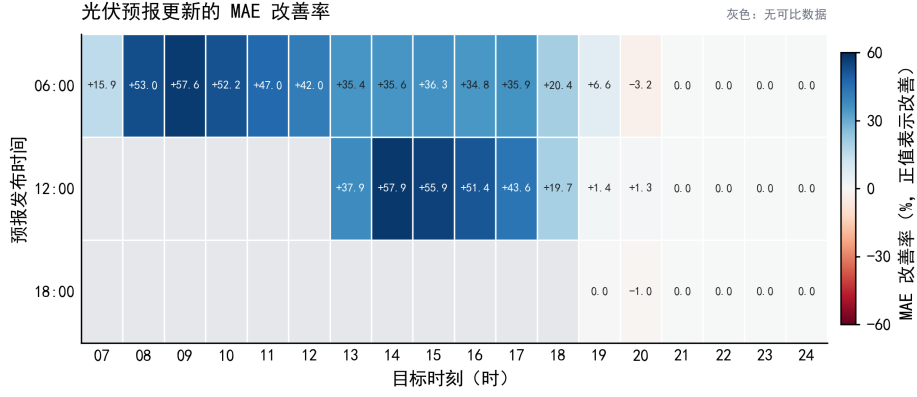


图 9 光伏预报更新的 MAE 改善率

9.3 可选更新时刻的策略组合模型建立与结果

9.3.1 新增约束条件

问题三在问题二储能运行约束的基础上，引入日内光伏预报更新及付费调整机制。对日期 D ，以 00:10 为合同起点，将调度周期划分为 144 个长度为 10 min 的区间：

$$I_{D,t} = [\tau_D + t\Delta t, \tau_D + (t+1)\Delta t), \quad t = 0, \dots, 143, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (19)$$

购电调整约束

设 g_t 为零点确定的原计划购电量， $q_{t,s}$ 为场景 s 下的最终普通购电量， $a_{t,s}$ 和 $b_{t,s}$ 分别表示相对原计划的增购量和取消量，则

$$q_{t,s} = g_t + a_{t,s} - \rho_{t,s}, \quad a_{t,s} \geq 0, \quad 0 \leq \rho_{t,s} \leq g_t. \quad (20)$$

设 $\mathcal{C}_h$ 为时刻 h 需要正式提交的调整区间。在该区间内，各场景必须采用相同的调整决策，即

$$q_{t,s} = q_t, \quad a_{t,s} = a_t, \quad \rho_{t,s} = \rho_t, \quad t \in \mathcal{C}_h, \forall s. \quad (21)$$

未来调整区间 $\mathcal{F}_h$ 中的场景相关订单仅用于评价未来成本，不立即成交。只有当相应预报时刻到达后，模型才根据最新预报和实际储电量重新优化并提交订单。

电量平衡与储能约束

记 $\ell_{t,s}$ 和 $\phi_{t,s}$ 分别为场景 s 下的负荷电量和光伏发电量， $c_{t,s}$ 、 $d_{t,s}$ 分别为交流侧充电量和放电量， $e_{t,s}$ 为紧急购电量， $v_{t,s}$ 为未利用的已购普通电量， $r_{t,s}$ 为弃光量。每个场景均满足

$$q_{t,s} - v_{t,s} + E_{t,s}^{\text{PV}} - r_{t,s} + d_{t,s} + e_{t,s} = \ell_{t,s} + c_{t,s}. \quad (22)$$

储能状态转移及容量约束为

$$S_{t+1,s} = S_{t,s} + 0.9c_{t,s} - \frac{d_{t,s}}{0.9}, \quad 1200 \leq S_{t,s} \leq 10800. \quad (23)$$

令 $u_{t,s} \in \{0, 1\}$ 表示充电方向，则充放电功率及互斥约束为

$$0 \leq c_{t,s} \leq \frac{5000}{6} u_{t,s}, \quad 0 \leq d_{t,s} \leq \frac{5000}{6} (1 - u_{t,s}). \quad (24)$$

同时满足

$$\begin{aligned} 0 \leq v_{t,s} \leq q_{t,s}, & \quad 0 \leq r_{t,s} \leq E_{t,s}^{\text{PV}}, \\ 0 \leq e_{t,s} \leq \ell_{t,s}(1 - u_{t,s}), & \quad v_{t,s} + r_{t,s} \leq (\bar{q}_t + E_{t,s}^{\text{PV}})u_{t,s}. \end{aligned} \quad (25)$$

其中, $\bar{q}_t$ 是线性化所需的充分大上界, 并非新增的外网购电容量限制。上述约束保证充电状态下不使用紧急电充电, 放电状态下不同时弃置普通电或光伏电量。

终端储电量软约束

为避免有限预测时域内过度消耗储能, 引入终端储电量缺口变量 ξ_s :

$$\xi_s \geq 6000 - S_{T,s}, \quad \xi_s \geq 0. \quad (26)$$

目标函数中的 $\kappa\xi_s$ 用于衡量期末库存不足, 但不计入实际现金账单, 也不构成题目规定的硬性末态约束。

9.3.2 负载-光伏联合误差场景构造

为在滚动调单中刻画负载与光伏预测的联合不确定性, 本文根据各决策时刻以前已经完整兑现的历史合同窗口构造经验误差场景。对于历史窗口 j , 负载误差和时刻 h 发布的光伏预报误差分别定义为

$$\varepsilon_{j,t}^L = L_{j,t}^{\text{real}} - \hat{L}_{j,t}, \quad \varepsilon_{j,t}^{\text{PV},(h)} = P_{j,t}^{\text{PV,real}} - \hat{P}_{j,t}^{\text{PV},(h)}. \quad (27)$$

残差库最多保留最近 28 个已完整结束的合同窗口; 历史窗口少于 14 个时, 模型退化为单场景点预测。历史充足时, 从残差库中无放回抽取 8 个历史日, 并在同一场景中使用同一历史日的整段负载误差与相应发行版本的光伏误差, 以保留误差的日内连续性及负载-光伏相关性。场景 s 中的负载和光伏电量为

$$\ell_{s,t} = \frac{\max(0, \hat{L}_t + \varepsilon_{j_s,t}^L)}{6}, \quad E_{t,s}^{\text{PV}} = \frac{\max(0, \hat{G}_t^{(h)} + \varepsilon_{j_s,t}^{G,h})}{6}, \quad (28)$$

各场景等概率赋权。场景生成只使用决策时刻前已经兑现的信息; 尚未结束的合同窗口不进入残差库。本问不再叠加分位数裕量或 CVaR 项, 以避免对同一预测风险重复计量。场景中的未来补救变量可以随整条场景路径变化, 因此该模型属于滚动两阶段随机近似, 而非具有完整非预见性约束的多阶段全局最优模型。

9.3.3 决策变量

实际现金费用

根据题目给定的结算规则, 实际现金购电费用为

$$\begin{aligned} J_{\text{cash}} &= \sum_t (p_t g_t + 1.5p_t a_t - p_t \rho_t + 0.5p_t \rho_t + 5p_t e_t) \\ &= \sum_t (p_t g_t + 1.5p_t a_t - 0.5p_t \rho_t + 5p_t e_t). \end{aligned} \quad (29)$$

其中, 取消电量 ρ_t 先退还原购电费用 $p_t \rho_t$, 再支付该部分 50% 的违约费用, 故其净费用为 $-0.5p_t \rho_t$; 增购电量按 $1.5p_t$ 结算, 紧急购电量按 $5p_t$ 结算。

信息使用与非预见性约束

在启用时刻 h ，模型只能使用截至该时刻已经获得的信息，包括实际储电量、已经执行的负载与光伏数据、已经签订的购电订单、当时已经发布的光伏预测以及已经完整兑现的历史预测误差。若 $z_h = 0$ ，策略在时刻 h 既不读取该版本的光伏预测，也不重新提交普通购电订单。

在正式调整区间内，提交的普通购电量必须在所有场景中保持一致，即

$$q_{t,s} = q_t, \quad t \in \mathcal{C}_h(\mathbf{z}), \quad s = 1, 2, \dots, N_{sc}. \quad (30)$$

该约束保证当前正式决策不依赖尚未观测到的未来场景。

需要指出的是，八种策略同时改变了新版预测信息的获取时刻和订单调整机会。因此，该组策略用于评价日内更新机制的整体价值，而不将策略间差异完全归因于预测精度的变化。

滚动决策目标

设场景总数为 N_{sc} 。零点制定原计划购电策略时，目标函数为

$$\min J^{(0)} = \sum_{t \in \mathcal{T}} p_t g_t + \frac{1}{N_{sc}} \sum_{s=1}^{N_{sc}} \left[\sum_{t \in \mathcal{F}_0} (1.5p_t a_{t,s} - 0.5p_t \rho_{t,s}) + \sum_{t \in \mathcal{T}} 5p_t e_{t,s} + \kappa \xi_s \right]. \quad (31)$$

在 $h \in \{6:00, 12:00, 18:00\}$ 时进行滚动调整时，原计划费用已经确定，对当前决策而言是常数，因此优化目标写为

$$\min J^{(h)} = \sum_{t \in \mathcal{C}_h} (1.5p_t a_t - 0.5p_t \rho_t) + \frac{1}{N_{sc}} \sum_{s=1}^{N_{sc}} \left[\sum_{t \in \mathcal{F}_h} (1.5p_t a_{t,s} - 0.5p_t \rho_{t,s}) + \sum_{t \in \mathcal{R}_h} 5p_t e_{t,s} + \kappa \xi_s \right], \quad (32)$$

其中， $\mathcal{R}_h$ 表示时刻 h 之后的剩余调度区间。

综上，模型以最小化预期购电、调整及紧急购电成本为核心目标，并利用终端库存软惩罚避免短视放电。模型不另外设置“购电量与预测净负荷之差最小”的误差项；负荷和光伏预测通过各场景的电量平衡约束进入模型，使购电策略在经济成本、供电安全和储能状态之间实现协调。

为公平比较不同预报与调单时刻组合，开发期进一步采用库存价值修正指标

$$J_{\text{eval}} = J_{\text{cash}} - \kappa (S_{\text{end}} - S_{\text{start}}). \quad (33)$$

该指标仅用于组合选择，防止某一策略通过消耗更多初始库存获得表面上的低费用，不作为实际购电账单。

9.3.4 模型建立

为分析日内光伏预报更新对购电策略的影响，在每天 0:00 必须制定原计划购电策略的基础上，将 6:00、12:00 和 18:00 三个时刻是否启用作为策略变量。不同策略采用相同的预测数据、物理约束、场景生成方法和储能控制器。

策略空间的构造

记三个可选更新时刻的集合为

$$\mathcal{U} = \{6, 12, 18\}. \quad (34)$$

对任意 $h \in \mathcal{U}$ ，定义二元变量

$$z_h = \begin{cases} 1, & \text{在时刻 } h \text{ 读取新版光伏预报并允许调整购电订单,} \\ 0, & \text{在时刻 } h \text{ 不读取新版预报且不进行订单调整.} \end{cases} \quad (35)$$

由此得到策略向量

$$\mathbf{z} = (z_6, z_{12}, z_{18}) \in \{0, 1\}^3, \quad (36)$$

共形成 $2^3 = 8$ 种候选策略。对给定的策略 $\mathbf{z}$ ，其启用时刻集合定义为

$$\mathcal{H}(\mathbf{z}) = \{0\} \cup \{h \in \mathcal{U} : z_h = 1\}, \quad (37)$$

其中 0:00 为所有策略共有的原计划制定时刻。零点预报为所有策略的基础信息，并从 6:00、12:00 和 18:00 三个补充预报时刻中选择任意组合，共形成 $2^3 = 8$ 种候选策略。记启用时刻集合为

$$\mathcal{H} \subseteq \{6, 12, 18\}.$$

当某一时刻被启用时，系统读取最新光伏预报，并根据当前储电状态调整其后尚未执行的普通购电计划；未启用的时刻不更新预报，也不进行合同调整。若启用多个时刻，则每次仅正式确定当前时刻至下一次更新时刻之间的购电量，最后一次更新负责其后直至次日 00:10 的剩余区间。各策略均从共同初始状态开始连续运行，并根据实际现金费用进行比较。

合同窗口与正式调整区间

日期 d 对应的合同窗口为当日 00:10 至次日 00:10，以 10 分钟为一个调度区间，共包含 144 个区间，即

$$\mathcal{T} = \{0, 1, \dots, 143\}, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (38)$$

四个预报时刻在该合同网格中的起始索引分别为

$$\rho_0 = 0, \quad \rho_6 = 35, \quad \rho_{12} = 71, \quad \rho_{18} = 107, \quad \rho_{\text{end}} = 144. \quad (39)$$

因此，四个连续订单块分别包含 35、36、36 和 37 个十分钟区间。

每天 0:00 根据零点预报确定整个合同窗口的原计划购电量 $\{g_t : t \in \mathcal{T}\}$ 。对于任一启用的日内更新时刻 $h \in \mathcal{H}(\mathbf{z}) \setminus \{0\}$ ，记 $h^+(\mathbf{z})$ 为策略 $\mathbf{z}$ 中位于 h 之后的下一个启用时刻；若不存在下一启用时刻，则令其对应合同终点。时刻 h 的正式调整区间定义为

$$\mathcal{C}_h(\mathbf{z}) = \{t \in \mathcal{T} : \rho_h \leq t < \rho_{h^+(\mathbf{z})}\}. \quad (40)$$

模型在时刻 h 可以对整个剩余合同窗口进行滚动展望，但只将 $\mathcal{C}_h(\mathbf{z})$ 中的普通购电量写入正式订单。已经执行的区间不得回溯修改，每个目标区间至多进行一次正式调整。对更远时段得到的场景订单仅用于评价当前决策的后续影响，不视为已经成交的普通购电量。

统一运行与公平评估

对于每个策略 H_k ，均调用前文建立的滚动优化模型。八种策略采用完全相同的负载预测、光伏预测、实际运行数据、电价、场景数、历史残差窗口、随机种子、储能参数和控制方式，唯一变化因素为启用时刻集合 $\mathcal{H}(\mathbf{z})$ 。

各策略从相同的初始储电量独立开始运行，且在同一策略内部保持储能状态的跨日连续性：

$$S_{d+1,0}^{(k)} = S_{d,144}^{(k)}, \quad k = 1, 2, \dots, 8. \quad (41)$$

因此，储能设备不会在每日零点重新恢复至初始状态，不同策略的期末状态也不会传递给其他策略。在本组更新时间组合评估中，储能执行层统一采用同一种滚动控制方式，从而避免控制器差异干扰更新时刻的评价；MPC 与贪心控制的对照将在选定更新时间组合后另行讨论。

9.3.5 模型结果与调整时刻选择

八种组合的完整期实际现金费用如图 10 所示。

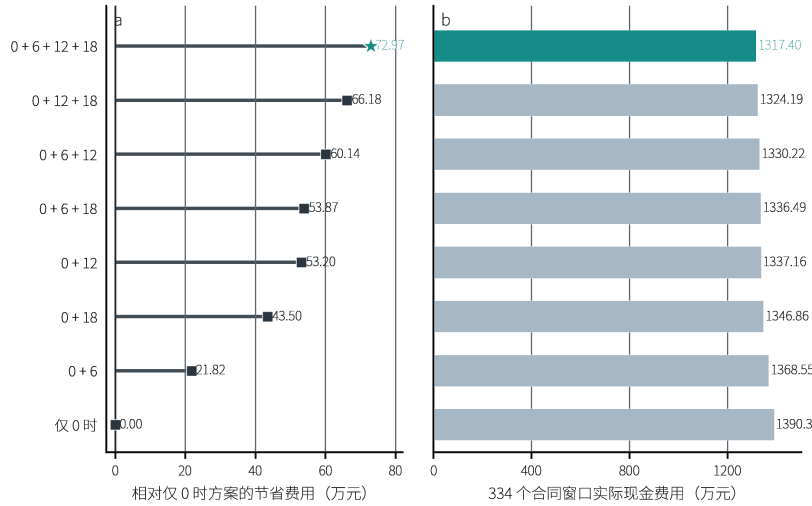


图 10 八种预报组合相对仅 0 时观测节省费用与在 334 个合同窗口中的实际现金费用

由图 10 可知，开发期选定的 0:00 + 6:00 + 12:00 + 18:00 组合在完整回放中的现金费用最低，为 13 173 959.97 元。其费用由原计划购电费、调整净费用和紧急购电费构成，即

$$12\,346\,527.26 + 568\,748.75 + 258\,683.96 = 13\,173\,959.97 \text{ 元。} \quad (42)$$

仅使用零点预报且不进行日内调整时，现金费用为 13 903 624.50 元。因此，选定组合共减少费用 729 664.54 元，降幅为 5.2480%。

9.4 规划方法与控制策略的选择

9.4.1 模型比较

在确定采用 0:00 + 6:00 + 12:00 + 18:00 的预报更新与调单组合后，还需进一步判断场景随机规划和模型预测控制是否均有保留的必要。为此，设置以下三种决策方案：

1. 场景随机规划与 MPC 相结合；
2. 确定性规划与 MPC 相结合；
3. 场景随机规划与贪心控制相结合。

三种方案均采用相同的预报更新时刻、交易规则、储能参数、初始储电量和末端处理方式。场景随机规划与确定性规划的对照用于评价不确定性建模的作用；在场景规划保持不变的条件下，MPC 与贪心控制的对照用于评价滚动控制方法的作用。

从相同初始状态开始的连续 60 个合同窗口上进行对照，结果如表 7 所示。

控制方法同为 MPC 时，场景随机规划相较于确定性规划使现金费用减少 36 311.06 元，降幅为 1.777423%，且观察到确定性规划紧急购电量较大。这说明模型仅根据单条点预测制定订

表 7 不同决策模型在连续 60 个合同窗口上的对照

决策方案	现金费用/元	紧急购电量/kWh	期末储电量/kWh	求解时间/s
场景随机规划 +MPC	2 006 593.25	3 836.537	1 492.128	164.590
确定性规划 +MPC	2 042 904.31	13 653.524	1 251.967	52.824
场景随机规划 + 贪心控制	2 006 577.52	3 818.219	1 492.128	120.776

单时，易在实际供电不足时产生较多紧急购电。而场景随机规划通过同时考虑多条可能的误差轨迹，使调整订单为预测偏差预留了更合理的响应空间。

在场景随机规划保持不变时，MPC 与贪心控制得到了非常接近的结果：贪心控制的现金费用比 MPC 低 15.73 元，紧急购电量低 18.318 kWh，两者期末储电量相同。上述费用差异仅占总费用的 0.000784%，尚不足以说明某一种控制器具有明显的经济优势。

为降低短窗口偶然性对控制器选择的影响，进一步在完整运行周期内比较 MPC 和贪心控制的表现，结果如表 8 所示。

表 8 两种储能控制方法在完整运行周期内的对照

控制方法	现金费用/元	紧急购电量/kWh	期末储电量/kWh	求解时间/s
MPC	13 173 959.97	52 866.062	1 319.758	825.968
贪心控制	13 177 518.55	52 060.019	1 319.758	687.967

完整周期内，MPC 的现金费用比贪心控制低约 3 558.6 元，相对差异仅为 0.027005%；贪心控制的紧急购电量则比 MPC 低 806.043 kWh，求解时间缩短 138.001 s，且两者的期末储电量相同。

基于“场景随机规划 +MPC”的最终决策方案

综合以上结论，可以认为，场景随机规划对降低购电费用和紧急购电风险具有实际作用，应在最终决策模型中予以保留。

对于储能执行层，MPC 相对于贪心控制的全年现金费用优势约 0.027%，但本题以购电总费用最小为主要目标；且 MPC 根据最新观测和储电状态重新优化充放电动作，减轻预测偏差对运行费用的影响，还能够结合后续供需预测，统筹当前放电与后续储能需求，因而更具前瞻性，更适配该场景。因此最终采用“场景随机规划 +MPC”的决策方案。

在 0:00+6:00+12:00+18:00 更新组合下，该方案完整运行周期的现金购电费用为 13 173 959.97 元。

该数值表示综合权衡后所选方案的实际现金费用，而不是所有候选控制器中的最低现金费用（但若仅以现金费用最小为唯一准则，则应选择场景随机规划与 MPC 相结合的方案）。问题三要求的购电计划表见附录。

9.5 基于 Shapley 值的补充预报价值分析

八种方案的费用比较能够确定最优预报组合，但不能直接衡量各预报时刻的独立贡献。为此，将 6:00、12:00 和 18:00 视为合作博弈中的三个参与者，并采用 Shapley 值分解其经济价值。

记补充预报时刻集合为 $N = \{6, 12, 18\}$ ， $C(S)$ 表示在保留零点预报的基础上启用时刻集合 $S \subseteq N$ 时，冻结测试期内的现金费用。定义相对仅使用零点预报的节费价值为

$$v(S) = C(\emptyset) - C(S). \quad (43)$$

Shapley 值通过平均时刻 i 在不同预报组合中的边际节费，衡量其对总收益的贡献；本题仅含三个补充时刻，等价于对三者的 6 种加入顺序取平均。

在 2025 年 2 月 25 日之后的 310 个冻结测试期合同中，全时刻方案相对仅使用零点预报共节省 676 741.11 元。其中，6:00、12:00 和 18:00 的 Shapley 贡献分别为

$$117\,176.39, \quad 322\,217.47, \quad 237\,347.25 \text{ 元},$$

分别占总节省的 17.31%、47.61% 和 35.07%。由图 11 可知，12:00 的经济贡献最高，其次为 18:00 和 6:00。因此，在接口数量受限时，建议按照

$$12:00 \rightarrow 18:00 \rightarrow 6:00$$

的顺序配置补充预报接口。

考虑到增设接口时系统可能已经具备部分预报能力，进一步以

$$\Delta_i(S) = C(S) - C(S \cup \{i\}) \quad (44)$$

表示在已有时刻集合 S 的条件下增加时刻 i 所带来的边际节费。该指标用于判断新增预报在不同信息背景下是否仍具有经济价值。

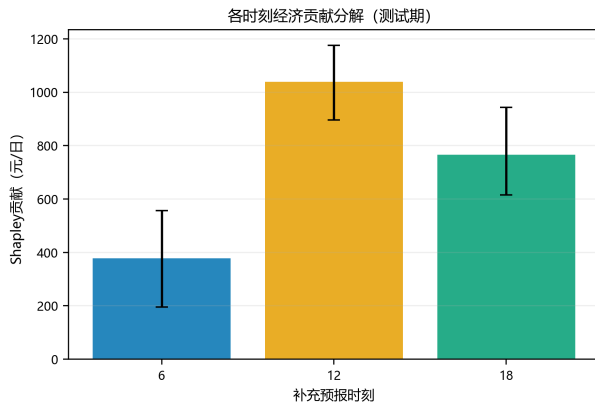


图 11 冻结测试期内各补充预报时刻的 Shapley 日均经济贡献

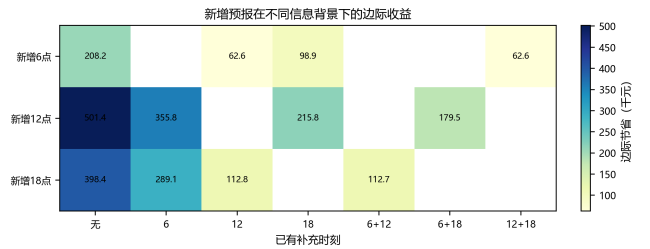


图 12 不同已有信息背景下增加补充预报时刻的边际节省

如图 12 所示，所有边际收益均为正，说明三个补充预报时刻均具有经济价值。随着其他预报时刻逐步启用，各时刻的边际价值总体下降，表明不同时刻提供的信息存在一定程度的覆盖。

预报时刻之间的组合效应 为区分“联合方案是否优于单时刻方案”与“收益是否具有超加性”，对任意两个时刻 $i, j \in N$ ，分别定义

$$G_{ij} = \frac{C(\{i\}) + C(\{j\})}{2} - C(\{i, j\}), \quad (45)$$

$$R_{ij} = v(\{i\}) + v(\{j\}) - v(\{i, j\}), \quad (46)$$

其中， $G_{ij} > 0$ 表示联合方案优于两个单时刻方案费用的平均值； $R_{ij} > 0$ 表示两项单独收益存在加法冗余，即二者具有部分替代关系。

图 13 表明, 3 种联合方案相对相应的单时刻方案费用均值均节省一定数目金额, 说明联合使用多个预报时刻能进一步改善调度结果。然而另一方面, 三组加法冗余值均正且较为可观。因此, 这些预报总体表现为部分替代而非超加性互补, 其中 12:00 与 18:00 的信息重叠最为明显。

时间相关性下的经验稳定性 考虑到相邻日期的负荷、光伏和储能状态具有连续性, 本文采用配对移动区块 Bootstrap[6] 评价结论的时间稳定性。评价结果如图 14 所示其中, 全时刻方案相对仅使用零点预报方案的日均节省为 2173.81 元, 其 95% 经验区间为 [1817.04, 2556.23] 元/日; 在已经启用 12:00 和 18:00 后增加 6:00 预报的日均边际节省为 205.66 元, 区间为 [116.36, 300.87] 元/日; 12:00 与 18:00 的 Shapley 日均贡献之差为 272.78 元, 区间为 [105.69, 426.78] 元/日。上述区间均未跨越零。

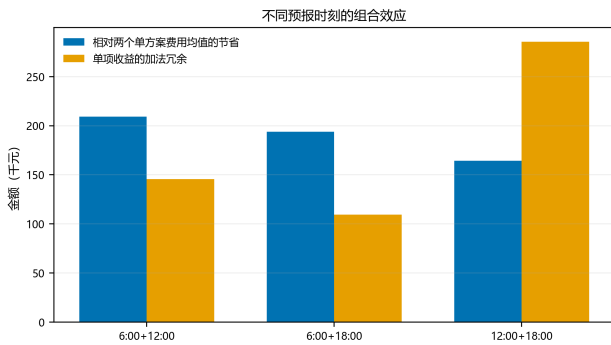


图 13 不同补充预报时刻的二元组合效应

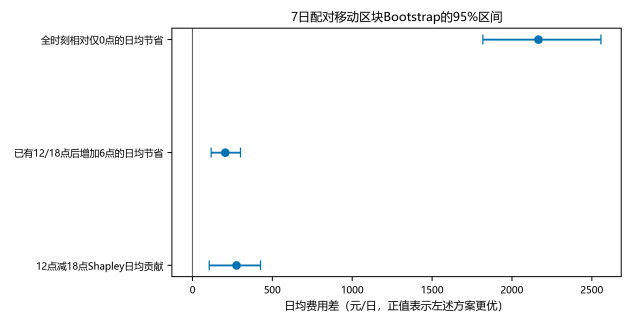


图 14 采用 7 日配对移动区块 Bootstrap 得到的关键费用差 95% 经验区间

将区块长度改为 14 日后, 上述三个区间分别为 [1829.68, 2537.72]、[127.96, 295.98] 和 [84.51, 436.08] 元/日, 结论方向保持不变。因此, 全时刻方案的费用优势、6:00 的正条件边际价值以及 12:00 相对 18:00 的贡献优势在不同连续时间区段的重采样中均较为稳定。

数据获取成本阈值与最终决策

实际运行中, 是否增设补充预报接口还取决于数据购买、通信、维护和计算成本。Shapley 日均贡献可作为不同预报时刻长期平均获取成本的参考上限:

$$\bar{k}_i < \frac{\varphi_i}{310}, \quad (47)$$

其中 $\bar{k}_i$ 为时刻 i 对应数据源的日均增量成本。

由图可得, 6:00、12:00 和 18:00 预报的平均成本参考阈值分别约为 378 元/日、1039 元/日和 766 元/日。该阈值是对全部预报组合取平均后的 Shapley 价值, 并非任意固定建设顺序下都适用的直接投资上限。

若系统尚未配置日内预报接口, 建议优先建设 12:00 接口, 再考虑 18:00, 最后考虑 6:00。若系统已经具备 12:00 和 18:00 接口, 则是否继续增加 6:00 应采用其条件边际收益: 测试期点估计为

$$\frac{62583.36}{310} = 201.88 \text{ 元/日},$$

而不应直接使用其 378 元/日的 Shapley 平均贡献。

综上, 在不考虑补充数据获取成本时, 三个时刻的 Shapley 贡献及条件边际收益均为正, 且全时刻方案在冻结测试期内费用最低, 因此有必要利用 6:00、12:00 和 18:00 的新预报制定调整购电策略。若存在预算或接口数量限制, 则推荐顺序为 12:00 → 18:00 → 6:00; 是否实际

增设某一接口，应将其在当前信息背景下的条件边际收益与对应的全生命周期增量成本进行比较。

9.6 对于是否引入其他预报时刻的回应

进一步地，本文认为没有足够证据支持在 6:00、12:00 和 18:00 之外无条件增加更多预报发布时刻。Shapley 分解和条件边际收益结果表明，补充预报时刻虽然均能带来正收益，但其收益并非线性叠加。随着已有预报时刻增多，新预报的条件边际节省明显下降；同时，二元组合效应中加法冗余均为正，说明不同时刻的预报信息存在一定重叠，（尤其 12:00 与 18:00 之间的信息替代性较强）。因此，继续增加更密集的预报时刻可能导致数据获取、通信和计算成本上升，而实际费用节省有限，整体上不利于盲目扩展预报频次。

若实际系统仍需进一步增加预报时刻，则应优先考虑 6:00 至 12:00 之间的时间段。原因在于，12:00 预报的 Shapley 贡献最高，表明日中附近的信息更新对购电调整最关键；而 6:00 预报贡献相对较低，说明过早发布的预报对后续光伏出力修正有限。因此，在 6:00 和 12:00 之间增加一次预报，有望兼顾较短预报时距和较长可调整窗口，既能更及时修正上午光伏预测误差，又能为中午及下午的储能充放电和购电计划调整保留足够决策空间。综合来看，若无额外成本约束，已有 6:00、12:00 和 18:00 预报已能覆盖主要调整需求；若必须增加新的预报时刻，建议优先评估 9:00 或 10:00 附近的发布方案。

十、问题四的模型的建立和求解

10.1 问题分析与模型继承

问题四将外网电价由固定的已知分时电价扩展为随日期和时刻变化的随机量。由于电价预测误差会改变储能跨时段转移电量的经济价值，不能直接对前三问的购电轨迹替换价格后重新结算，而应根据各决策时刻可获得的负载、光伏和电价信息重新优化。

本文继承问题二、三中的能量守恒、储能状态转移、充放电互斥和滚动控制框架，并增加动态电价预测及价格误差场景。为与题意呼应，分以下两方面求解：

1. 零点签订的普通购电合同在日内保持不变，仅根据更新后的预测滚动控制电池；

2. 在 6:00、12:00 和 18:00 获得新预报后，允许对尚未执行的合同区间付费调单。两类策略均采用实际供电区间价格进行事后结算。

10.2 动态电价预处理与滚动预测

10.2.1 区间价格与信息边界

与前文一致，以相邻价格节点的算术平均值作为 10 分钟区间的等效结算价格：

$$\bar{p}_{d,t} = \frac{p_{d,t}^{\text{node}} + p_{d,t+1}^{\text{node}}}{2}. \quad (48)$$

负载、光伏及其区间电量沿用前文定义；每日决策仅使用当时已经获得的信息，已执行区间不再重新决策。对日期 D ，将 D 日 00:10 至 $D+1$ 日 00:10 划分为 144 个 10 分钟区间。每日 0:00 及后续各调整时刻均只使用该时刻以前已经获得的负载、光伏、电价和储能信息，已执行区间不再重新决策。

10.2.2 XGBoost 电价预测

采用 XGBoost 分别预测 0:00、6:00、12:00 和 18:00 发布版本的未来区间电价。特征包括目标时刻及星期周期、历史同期价格、近期滚动统计量、最新可知价格和近期价格水平，预测值表示为：

$$\hat{p}_{d,t}^{(h)} = \sum_{m=1}^{M_{\text{tree}}} f_m(x_{d,t}^{(h)}). \quad (49)$$

其中， f_m 为第 m 棵回归树。模型最多使用最近 28 个满足滞后条件的历史发行日，每日重新拟合；树数为 350，最大深度为 5，学习率为 0.04。

零点版本预测从 D 日 00:10 到 $D+1$ 日 00:10 的 145 个价格节点；在 6、12、18 时版本分别补充剩余 109、73 和 37 个未来节点。价格更新、光伏预报更新和普通购电计划调整并不总是在同一时间发生。问题 4-2 和问题 4-3 均在 0、6、12、18 时更新价格；问题 4-2 的普通购电计划只在零点确定，之后保持不变。

10.3 联合误差场景构造

问题四沿用问题三中的负载-光伏历史配对场景，并进一步引入动态电价误差。负载和光伏采用加法残差，电价采用对数比例残差：

$$\varepsilon_{j,t}^L = L_{j,t}^{\text{real}} - \hat{L}_{j,t}, \quad \varepsilon_{j,t}^{\text{PV}} = P_{j,t}^{\text{PV,real}} - \hat{P}_{j,t}^{\text{PV}}, \quad z_{j,t}^p = \ln \frac{\bar{p}_{j,t}^{\text{real}}}{\hat{p}_{j,t}}. \quad (50)$$

对每个场景，负载、光伏和电价使用同一历史日索引 j_s ，以保留三者的日内连续性及相关性：

$$\ell_{s,t} = \frac{\max(0, \hat{L}_t + \varepsilon_{j_s,t}^L)}{6}, \quad E_{s,t}^{\text{PV}} = \max(0, \hat{P}_t^{\text{PV}} + \varepsilon_{j_s,t}^{\text{PV}}) \Delta t, \quad p_{s,t} = \hat{p}_t \exp(z_{j_s,t}^p). \quad (51)$$

当有效历史窗口少于 14 个时，价格场景退化为点预测。本节不再额外叠加分位数裕量或 CVaR 项，以避免对同一预测风险重复计量。

10.4 动态电价下的购电与储能模型

10.4.1 决策变量与现金费用

记 g_t 为零点签订的原普通购电量， q_t 为正式调整后的普通购电量， a_t 和 ρ_t 分别为增购量和取消量； c_t, d_t, e_t, v_t, r_t 分别为充电量、放电量、紧急购电量、未利用的已付普通电量和弃光量， S_t 为储电量， u_t 为充放电方向变量。各电量变量单位均为 kWh。合同调整关系为

$$q_t = g_t + a_t - \rho_t, \quad a_t, \rho_t \geq 0, \quad 0 \leq \rho_t \leq g_t. \quad (52)$$

增购部分按普通价格的 1.5 倍收费；取消部分退还原费用后再收取取消量 50% 的违约金；紧急购电按普通价格的 5 倍收费。因此，策略执行后的实际现金费用为

$$C = \sum_t \bar{p}_t^{\text{real}} (g_t + 1.5a_t - 0.5\rho_t + 5e_t). \quad (53)$$

未使用但已经购入的普通电 v_t 不退款，未来场景中的暂定订单也不计入实际账单。

10.4.2 两阶段随机 MILP

场景期末储电量软约束沿用问题三式 (32)，不再重复列写。
以场景期望费用和软末端库存惩罚为目标：

$$\min \sum_s \pi_s \left[\sum_t p_{s,t} (g_t + 1.5a_{s,t} - 0.5\rho_{s,t} + 5e_{s,t}) + \kappa\xi_s \right]. \quad (54)$$

其中，现在需要签订的 g_t 或当前正式调整块的 q_t 跨场景共享，未来调单和电池补救变量允许随场景变化。故该模型是滚动两阶段随机近似，而不是具有完整非预见性约束的多阶段全局最优模型。软末端项用于避免有限时域过度消耗库存，不计入式 (53) 的实际现金费用。

场景中的合同调整量和储能补救变量允许随场景变化，而当前已确定的普通购电量在各场景间保持一致。因此，该模型属于滚动两阶段随机近似。

场景供需平衡、储能状态递推、容量边界、充放电功率、充放电互斥、弃光及紧急购电等物理约束均沿用问题三式 (28)–(32)，本节不再重复列写。式 (32) 中的 ξ_s 继续用于惩罚场景末端储能不足，但不计入实际现金费用。

模型由 SciPy 调用 HiGHS 求解，目标相对间隙设置为 0.01%，限时后固定整数模式进行 LP 精修，并重新核对约束和现金费用。

10.5 滚动执行策略

每个 10 分钟区间开始时，控制器根据最新可得的价格、负载、光伏信息和当前储能状态，重新优化剩余时域，但仅执行当前区间动作。问题 4-2 在 0:00 确定普通购电计划，后续仅更新储能运行；问题 4-3 在 0:00、6:00、12:00 和 18:00 更新价格与光伏预报，并对尚未执行的合同区间进行付费调单。已执行区间不回溯修改，未来场景订单仅用于评价。

10.6 模型求解与结果分析

10.6.1 主策略费用及组成

策略选择仅使用 2 月 17 日至 23 日的 7 个完整开发窗口，并在模拟 2 月 25 日零点冻结；其后 310 个窗口仅作留出检验，不用于反向改选。最终问题 4-2 采用电价点预测加负载/光伏场景，问题 4-3 采用相同价格模式并在 0、6、12、18 时启用光伏预报和正式调单。

表 9 问题四两类主策略的 334 个合同窗口运行结果

策略	现金费用/元	紧急购电量/kWh	期末 SOC/kWh
4-2 固定普通计划	14,408,569.00	178,809.186	5,596.131
4-3 指定时刻调单	13,870,785.24	50,953.201	5,715.626

由表 9 可知，在 334 个合同窗口内，问题 4-3 较问题 4-2 减少现金费用 537,783.76 元，降幅为 3.732450,953.201 kWh。由于两种方案使用的光伏预报信息不同，该差异体现的是信息更新与调单机制的联合效果，不能全部归因于调单权限。

图 15 显示，问题 4-3 虽然产生 576,440.99 元的净调整费用，但其紧急购电费用由 941,472.54 元下降至 273,720.38 元，从而取得更低的总费用。表中统计范围是 334 个 $D\ 00:10$ 至 $D+1\ 00:10$ 合同窗口，并不等同于自然日口径的全年账单。

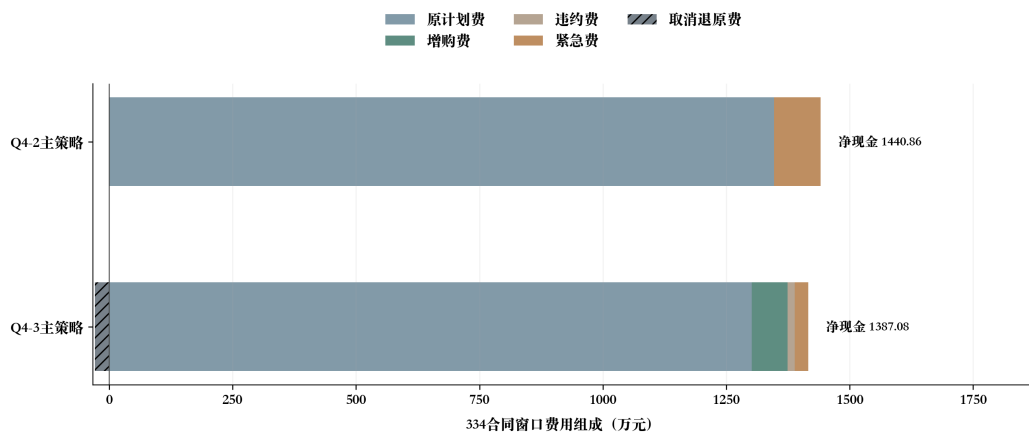


图 15 问题四两类主策略的现金费用组成：取消退还原费用记为负项；库存评价项和启动段费用不计入图中现金组成

10.6.2 预报与调单时刻的选择

在点价格模式下，问题 4-3 的 8 种预报启用组合中（结果见图 16），0:00、6:00、12:00 和 18:00 全部启用的方案费用最低。该结论仅适用于本文比较的 8 种组合及当前预测、场景和调单规则，不能外推为所有连续调单策略下的全局最优。

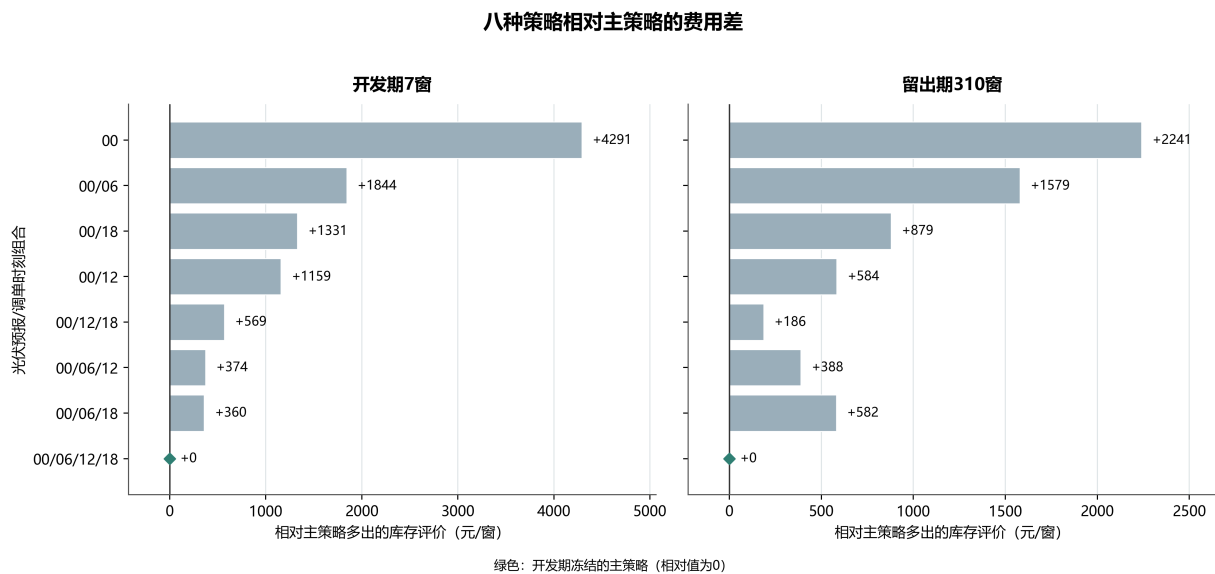


图 16 八种光伏预报与调单时刻组合在开发期和留出期的费用表现

因此，在当前预测质量和费用规则下，使用全部日内更新时刻可以显著减少紧急购电及总费用；这一结论只针对本文比较的 8 种组合，不能外推为所有连续调单政策中的全局最优。

10.6.3 点价格与联合价格场景比较

表 10 中正值表示联合价格场景费用更高。两分支在留出期的现金费用均略有增加，且采用 7 日配对移动区块 Bootstrap 得到的日均费用差区间均跨越零。图 17 进一步展示了累计配对差值。因此，在当前历史窗口、场景数和单年数据下，联合刻画价格误差尚未表现出稳定的经济优势；点价格方案配合负载、光伏场景已经具有较好的费用表现。

表 10 联合价格场景相对点价格方案的费用差

分支	334 窗现金差/元	310 测试窗现金差/元	日均差 95% 区间/(元/日)
4-2	9,014.86	8,537.82	[−24.323, 77.985]
4-3	7,526.66	5,754.44	[−15.606, 47.035]

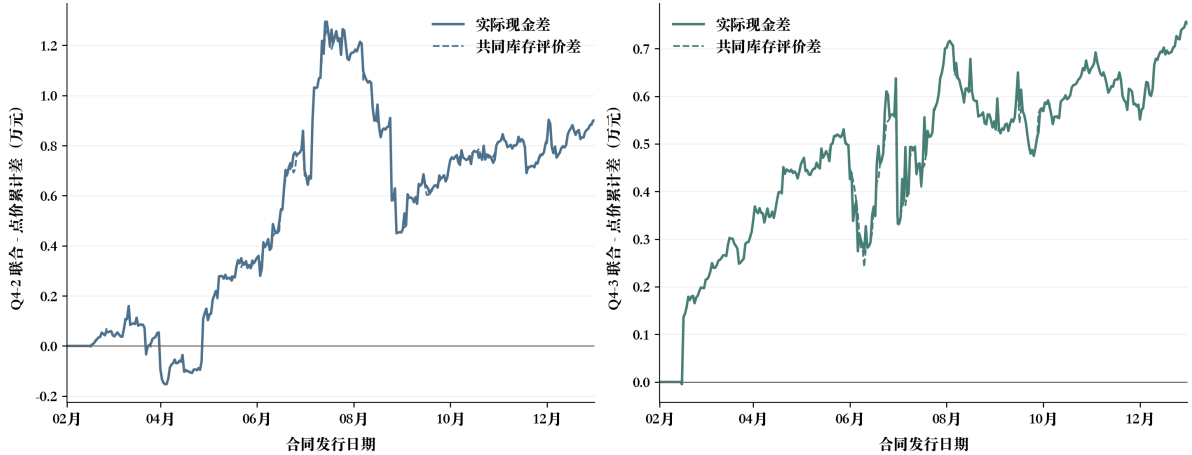


图 17 联合价格场景减去点价格方案的累计实际费用差

十一、模型的分析与检验

11.1 灵敏度分析

本文从参数扰动、策略消融和执行假设三个层面检验模型稳健性。正文仅列出直接影响主要结论的代表性结果，完整检验项目见附录 G。代表性结果汇总于表 11。其中，题目“充放电效率为 90%”存在单向效率与往返效率两种解释。主模型取 $\eta_c = \eta_d = 0.9$ ；替代口径取 $\eta_c = \eta_d = \sqrt{0.9} = 0.948683$ ，使往返效率恰为 90%。

表 11 关键因素的灵敏度与稳健性检验

检验因素	基准与扰动设置	费用变化或比较结果	主要结论
储能效率	0.9 与 $\sqrt{0.9}$	问题一费用下降 3.79%	题意解释影响费用数值，但不影响可行性
问题二效率	A-D 四方案重跑	分别下降 2.52%、2.78%、2.88%、3.10%	两种口径下均为方案 D 费用最低
问题三规划与控制	场景/确定性规划，MPC/贪心	新口径 334 窗中场景 MPC 比确定性 MPC 少 184826.75 元；与场景贪心仅差 2094.86 元	随机规划优势保留；MPC 与贪心差距较小
问题四效率	4-2 与 4-3 冻结策略重跑	分别下降 2.93% 与 3.19%	两种口径下 4-3 均优于 4-2
场景数与初态	场景数 4/8/16；SOC 为 1200/6000/10800 kWh	8 与 16 场景结果接近；库存修正后初态差异缩小	8 场景兼顾效果与计算量，核心判断不依赖单一初态

效率口径重跑表明，更高的单向效率减少了储能循环损耗，使问题一费用由 35205.89 元降至 33871.94 元；问题二 A-D 的 334 窗费用均下降，而 D 仍为最低费用方案；问题四 4-2 和 4-3 费用分别由 14408569.00 元、13870785.24 元降至 13985925.76 元、13427995.13 元，4-3 的相对优势保持。因此，核心方案选择不依赖于效率歧义。问题三需谨慎解释：原效率三算法的共同对照为 60 窗，新效率重跑为 334 窗，不能直接作严格的跨效率数值差；已有证据只支持随机规划相对确定性仍占优，而 MPC 与贪心费用接近，其细微排序可能随评价窗口改变。

此外，本文已检验裕量分位数、历史残差窗口、场景数、随机种子、末端库存惩罚、初始SOC、价格场景构造、预报与调单时刻、边界补值及执行粒度。多数参数检验采用7、14、28或60窗的共同初态局部回放，用于识别敏感方向，不将其外推为全年最优保证。综合而言，储能效率和场景抽样会改变费用水平，但没有推翻本文的主要方案选择。

11.2 误差分析

误差主要来自三方面。其一，附件中的时刻值通过相邻端点平均近似为区间均值，10分钟均值反馈在段初并非完全可知；分钟级因果回放显示该近似可能使费用偏低。其二，负载、光伏与电价预测均存在误差，场景法仅由有限历史残差构造，不能覆盖所有极端天气与价格事件。其三，随机MILP受场景样本、时间限制和相对最优间隙影响。本文对全部执行轨迹重新核验能量平衡、SOC递推、容量和功率边界；保存结果的最大物理残差均低于 10^{-5} kWh。同时通过移动块Bootstrap、边界端点多种补值和不同执行粒度检查结论方向，从而避免把单个日期、单个随机种子或末端估计造成的波动误认为稳定收益。

十二、模型的评价、改进与推广

12.1 模型的优点

- 优点1：模型通过储电状态衔接日前购电、日内调单与电池运行，保留了跨日库存对费用的影响。同时通过惩罚项抑制模型放空电池以节省单日电费的短视行为，有较强的合理性。
- 优点2：采用负载与光伏成对的历史误差轨迹构造场景，使购电计划兼顾预测偏差造成的补救成本。方案选择同时考察费用、紧急购电量和计算时间，为随机规划及控制方法的取舍提供了依据。

12.2 模型的缺点

- 缺点1：场景构造依赖有限历史样本，对极端天气和用电规律突变的覆盖不足。
- 缺点2：滚动两阶段模型对未来补救安排存在一定乐观估计。

12.3 模型的改进

可按季节、天气和预测时长完善误差场景，补充极端偏差下的运行测试。

12.4 模型的推广

调整设备参数与交易规则后，该框架可用于校园、园区等分布式能源系统。若进一步加入车辆充电和可转移负荷变量，以及用能时限等约束，还可研究储能与需求侧共同参与的调度问题。

十三、AI 工具使用声明

本参赛队在竞赛过程中使用了AI工具，主要用于理解题目、提供思路、代码调试等工作，详细使用情况见支撑材料。

参考文献

- [1] HYNDMAN R J, ATHANASOPOULOS G. Forecasting: Principles and practice[M/OL]. 3rd ed. Melbourne, Australia: OTexts, 2021. <https://otexts.com/fpp3/>.
- [2] CHEN T, GUESTRIN C. Xgboost: A scalable tree boosting system[C/OL]//Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. 2016: 785-794. DOI: 10.1145/2939672.2939785.
- [3] MORSTYN T, HREDZAK B, AGUILERA R P, et al. Model predictive control for distributed microgrid battery energy storage systems[J/OL]. IEEE Transactions on Control Systems Technology, 2018, 26(3):1107-1114. DOI: 10.1109/TCST.2017.2699159.
- [4] SHAPIRO A. Stochastic programming approach to optimization under uncertainty[J/OL]. Mathematical Programming, 2008, 112(1):183-220. DOI: 10.1007/s10107-006-0090-4.
- [5] APPINO R R, GONZÁLEZ ORDIANO J Á, MIKUT R, et al. Storage scheduling with stochastic uncertainties: Feasibility and cost of imbalances[C/OL]//2018 Power Systems Computation Conference (PSCC). 2018: 1-7. DOI: 10.23919/PSCC.2018.8442529.
- [6] KÜNSCH H R. The jackknife and the bootstrap for general stationary observations[J/OL]. The Annals of Statistics, 1989, 17(3):1217-1241. DOI: 10.1214/aos/1176347265.

附录 A 补充符号说明

表 2 列出正文核心符号，其余数据处理、预测、滚动决策与评价符号汇总如下。

表 12 补充符号说明

符号	说明
$\hat{P}_t^{\text{PV}}$	第 t 个时段的光伏预测功率，是省略日期下标的记法，单位为 kW。
$E_{\max}$	单个 10 分钟时段的最大充放电量， $E_{\max} = P_{\max} \Delta t = 5000/6 \text{ kWh}$ ，约为 833.33 kWh。
z_t	与 u_t 同义的充放电状态记号，取 1 允许充电、取 0 允许放电，无量纲；区别于预报启用变量 z_h 。
$d, k, x_{d,k}, \bar{x}_{d,k}$	日期编号、日内时段编号、原始数据及区间平均值；编号无量纲，数据单位随负载、光伏或电价变量确定。
$\mathcal{D}_d$	第 d 天以前可获得的历史训练数据集。
$x_{d,k}^L, f_m(\cdot), M$	负载预测特征向量、第 m 棵回归树及树的总数；特征单位随定义确定，回归输出为 kW， M 为无量纲计数，在场景模型中也表示场景数。
$\mathcal{L}_L, l(\cdot), \Omega(f_m)$	负载预测的目标函数、损失函数及正则项，量纲由所采用的损失与正则项定义确定。
$W_d, w_{m,d}$	光伏组合预测的历史窗口及第 m 个基预测器的权重；窗口长度按天计，权重无量纲。

符号	说明
U_τ, V_τ	时段 τ 普通供电的剩余电量与缺口电量，单位均为 kWh。
κ, ξ	终端储电不足惩罚系数及松弛变量，单位分别为元/kWh、kWh；惩罚项不计入实际电费。
$\epsilon_{j,t}, Q_q(\cdot),$ $b_t, \tilde{\ell}_t$	历史净负载电量误差、 q 分位数算子、安全裕量及修正后的负载电量；误差、分位数结果、裕量和电量均以 kWh 计， q 无量纲。
$D, \tau_D, I_{D,t}$	日期、日期 D 的规划起始时刻，以及该规划窗口内第 t 个 10 分钟区间。
$q_{t,s}, a_{t,s}, \rho_{t,s}$	场景 s 下调整后的普通购电量、相对原计划的增购量及取消量，单位均为 kWh。
$E_{t,s}^{\text{PV}}, \bar{q}_t$	场景 s 下的光伏发电量及普通购电量上界，单位均为 kWh。
C_h, F_h, R_h	时刻 h 对应的正式调整区间、未来待评估区间及剩余时段集合，正文也以花体记号表示。
$\mathcal{U}, z_h, z,$ $H(z), h^+(z)$	可选更新时刻集合、预报调整启用变量、策略向量、启用时刻集合及下一次调整时刻； $\mathcal{U} = \{6, 12, 18\}$ ，时刻按小时表示，启用变量无量纲。
$\beta_0, \beta_6, \beta_{12},$ $\beta_{18}, \beta_{\text{end}}$	0:00、6:00、12:00、18:00 及结束时刻对应的时段边界编号，无量纲。
$J(h)$	时刻 h 的滚动优化目标，单位为元，与实际现金账单区分。
$S_{\text{start}}, S_{\text{end}}$	评价区间开始、结束时的储电量，单位均为 kWh。
$N, S, C(S), v(S)$	Shapley 分析中的预报时刻集合、其子集、子集对应费用及经济贡献；集合无量纲，费用和贡献以元计。
$\phi_i, \Delta_i(S),$ $G_{ij}, R_{ij}, \bar{k}_i$	时刻 i 的 Shapley 贡献（正文记作 φ_i ）、边际贡献、两时刻组合收益、冗余收益及日均新增数据获取成本上限；前四项单位为元，最后一项为元/日。
$p_{d,t}^{\text{node}}$	第 d 天第 t 个节点的电价，单位为元/kWh。
$x_{d,t}^{(h)}$	时刻 h 发布的电价预测所用特征向量，各分量单位随特征定义确定。
$\epsilon_{j,t}^L, \epsilon_{j,t}^G,$ $z_{j,t}^p, j_s$	负载、光伏历史预测误差（单位为 kW）、实际价格与预测价格之比的对数误差（无量纲），以及场景 s 抽取的历史日期编号；正文也用 ϵ^{PV} 表示光伏误差。
$\ell_{s,t}, E_{s,t}^{\text{PV}}, p_{s,t}$	场景 s 下第 t 个时段的负载电量、光伏发电量及电价；前两项单位为 kWh，电价单位为元/kWh。
C, A_t, D_t	动态电价下的实际现金费用（元），以及滚动执行时第 t 个时段的剩余电量和缺口电量（kWh）。

附录 B 问题一主要计算结果

表 13 和表 14 给出问题一在端点均值预处理口径下的指定时段结果。完整 144 时段轨迹由程序结果文件保存，此处列出题目要求的购电量、分时充放电量及关键汇总指标。

表 13 问题一指定时段购电量及费用汇总

时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.0000	12:00–12:10	483.4077
14:00–14:10	0.0000	16:00–16:10	420.1816
18:00–18:10	639.0720	20:00–20:10	0.0000
全天购电量		59540.1847 kWh	
最优购电费用		35205.89 元	
无储能购电费用		48004.76 元	

表 14 问题一分时充放电量

时间段	充电量/kWh	放电量/kWh
00:00–04:00	4500.0000	0.0000
04:00–08:00	833.3333	6452.2006
08:00–12:00	4982.7063	1903.4957
12:00–16:00	5393.8493	49.3138
16:00–20:00	0.0000	5066.4749
20:00–24:00	5333.3333	3573.5251
全天合计	21043.2223	17045.0101

日初与日末储电量均为 6000 kWh，运行范围为 1200–10800 kWh；弃光量为 0 kWh，MILP 相对最优间隙为 0，最大能量平衡残差为 1.14×10^{-13} kWh。

附录 C 第二问指定图表

四个指定日期的购电量与费用分别见表 15、表 16、表 17和表 18；对应充放电量与储电状态分别见表 19、表 20、表 21和表 22，连续紧急购电区间见表 23。

3.1 微网在指定时间段的计划购电量及全天购电量和购电费

表 15 2025.3.20 微网在指定时间段的计划购电量及全天购电量和购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	523.41	14:00–14:10	0.00
16:00–16:10	462.52	18:00–18:10	671.12	20:00–20:10	0.00
全天购电量/kWh	65023.72	全天购电费/元	43249.88		

表 16 2025.6.21 微网在指定时间段的计划购电量及全天购电量和购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	212.57	18:00–18:10	386.00	20:00–20:10	0.00
全天购电量/kWh	31273.88	全天购电费/元	18558.44		

表 17 2025.9.23 微网在指定时间段的计划购电量及全天购电量和购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	520.40	14:00–14:10	0.00
16:00–16:10	495.66	18:00–18:10	759.18	20:00–20:10	0.00
全天购电量/kWh	71580.84	全天购电费/元	45418.34		

表 18 2025.12.21 微网在指定时间段的计划购电量及全天购电量和购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	970.17	14:00–14:10	0.00
16:00–16:10	829.07	18:00–18:10	684.35	20:00–20:10	0.00
全天购电量/kWh	98659.99	全天购电费/元	63987.49		

3.2 储能设备在指定时间段的充放电量及储电量

表 19 2025.3.20 储能设备在指定时间段的充放电量及储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	8294.83	0.00	4:00–8:00	954.80	3407.99
8:00–12:00	4319.33	3013.36	12:00–16:00	4869.59	457.10
16:00–20:00	94.61	5190.06	20:00–24:00	447.03	3389.05
0:00 储电量/kWh		1615.32	24:00 储电量/kWh		1522.41

注：表中充电量和放电量均为储能设备端口处的原始电量，未乘以充放电效率 0.9；储电量递推时已计入效率损耗。下表同。

表 20 2025.6.21 储能设备在指定时间段的充放电量及储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	198.86	407.29	4:00–8:00	266.26	3011.80
8:00–12:00	10035.01	0.00	12:00–16:00	0.00	0.00
16:00–20:00	0.00	4823.49	20:00–24:00	936.55	3139.81
0:00 储电量/kWh		5148.89	24:00 储电量/kWh		2794.79

表 21 2025.9.23 储能设备在指定时间段的充放电量及储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	9442.53	0.00	4:00–8:00	1220.69	4336.08
8:00–12:00	4577.41	2136.24	12:00–16:00	3604.50	357.84
16:00–20:00	90.15	5182.28	20:00–24:00	930.17	3350.53
0:00 储电量/kWh		1428.44	24:00 储电量/kWh		2237.38

表 22 2025.12.21 储能设备在指定时间段的充放电量及储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	9886.73	54.00	4:00–8:00	1155.42	1457.15
8:00–12:00	5445.49	6854.04	12:00–16:00	6535.29	2131.03
16:00–20:00	15.19	5322.25	20:00–24:00	475.38	3079.06
0:00 储电量/kWh		1741.82	24:00 储电量/kWh		1906.73

3.3 指定日期的紧急购电量

表 23 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
01:40–02:00	17.63	—	—	20:10–20:20	2.04	—	—
02:40–04:00	148.48						
04:50–05:30	78.41						
05:40–06:00	78.91						
15:30–16:30	328.35						
16:40–17:30	80.16						
20:40–20:50	23.68						
21:20–22:00	70.28						

注：— 代表该日无紧急购电计划；后文同类表格含义相同。

附录 D 第三问指定图表

表 24 汇总指定日期的储能运行结果，普通购电量与费用见表 25，连续紧急购电区间见表 26。

4.1 指定日期储能设备充放电量

表 24 指定日期储能设备充放电量及储电量

日期	时间段	充电量/kWh	放电量/kWh	0:00 储电量/kWh	24:00 储电量/kWh
2025.3.20	0:00–4:00	8680.98	28.68	1689.85	1542.66
	4:00–8:00	1016.03	5726.68		
	8:00–12:00	3000.65	2540.09		
	12:00–16:00	6715.92	182.27		
	16:00–20:00	633.71	5130.46		
	20:00–24:00	405.36	3090.96		
2025.6.21	0:00–4:00	645.00	161.84	3188.58	3443.63
	4:00–8:00	374.54	1826.57		
	8:00–12:00	9892.42	0.00		
	12:00–16:00	0.00	0.00		
	16:00–20:00	4.84	5072.93		
	20:00–24:00	1920.63	3107.45		
2025.9.23	0:00–4:00	8790.06	0.00	1805.31	1795.04
	4:00–8:00	1208.07	5786.80		
	8:00–12:00	3918.69	2136.24		
	12:00–16:00	6213.49	287.28		
	16:00–20:00	119.01	5419.15		
	20:00–24:00	704.19	3352.11		
2025.12.21	0:00–4:00	8878.11	66.63	1621.82	1515.51
	4:00–8:00	2350.44	1638.37		
	8:00–12:00	4622.07	7721.55		
	12:00–16:00	8407.48	1962.15		
	16:00–20:00	0.00	5268.77		
	20:00–24:00	387.74	3401.34		

4.2 指定日期微网购电量及全天购电量和购电费

表 25 指定日期微网购电量及全天购电量和购电费

日期	10:00–10:10	12:00–12:10	14:00–14:10	16:00–16:10	18:00–18:10	20:00–20:10	全天购电量/kWh	全天购电费/元
2025.3.20	0.00	505.67	0.00	600.91	714.09	0.00	65733.82	44074.40
2025.6.21	0.00	0.00	0.00	84.40	401.18	0.00	33516.35	19343.34
2025.9.23	0.00	366.96	0.00	515.48	754.60	0.00	66273.33	43272.37
2025.12.21	0.00	966.04	0.00	863.41	699.74	0.00	94781.01	62907.43

4.3 指定日期微网紧急购电量

表 26 指定日期微网紧急购电量

日期	时间段	购电量/kWh	日期	时间段	购电量/kWh
2025.3.20	09:00–10:10	473.28	2025.9.23	20:10–20:20	1.23
2025.6.21	–	–		20:50–21:10	29.59
				21:30–21:40	0.85
2025.12.21	18:30–18:50	30.20			
	20:40–20:50	44.16			

附录 E 问题二补充比较结果

为区分日内控制算法与日前风险规划的贡献，在相同初态和相同控制条件下进行配对实验，结果见表 27。表中“累计减少”为基线费用减候选费用；Bootstrap 区间对应候选减基线，负值表示候选更省。

表 27 问题二控制与规划模块的配对消融结果

对照	共同条件	窗口数	现金减少/元	库存修正减少/元	胜出窗口数	平均每窗差及 95% 区间/元
B 相对 A	同一合同 g 、同 6000 初态	310	280504.43	283556.69	162	−904.85 [−1380.84, −508.05]
C 相对 B	同 MPC、同 6000 初态	310	376949.24	381619.59	283	−1215.97 [−1571.58, −889.06]
D 相对 B	同 MPC、同 6000 初态	310	764335.76	844577.53	216	−2465.60 [−3158.72, −1771.50]
D 相对 C	同 MPC、同 6000 初态	310	387386.52	462957.94	172	−1249.63 [−1790.45, −718.72]

参数在前 24 个窗口完成热身与校准，并于 2025 年 2 月 25 日前冻结。冻结后的 310 个测试窗口结果见表 28。

表 28 问题二冻结参数后的时间顺序测试结果

方案	测试期现金/元	起始 SOC/kWh	末端 SOC/kWh	库存修正费用/元
A	13810977.00	1239.63	1200.00	13810994.64
B	13530040.72	1242.42	1200.00	13530059.61
C	13145843.35	1544.10	1200.00	13145996.55
D	12674848.55	1517.09	1603.23	12674810.20

模型 C 的误差裕量分位数仅使用 2025 年 2 月 17 日至 23 日的 7 个已完成窗口校准。候选参数结果见表 29，最终选取 $q = 0.5$ ，未使用测试期结果反向挑选参数。

表 29 问题二净负载误差裕量参数校准结果

分位数 q	校准现金/元	库存修正费用/元	紧急购电费/元
0.0	271707.81	286376.88	11437.42
0.5	268556.92	283126.39	7307.26
0.8	271377.82	285010.36	237.64
0.9	277776.82	290744.18	68.50

不同初始储电量下的 28 窗口独立敏感性结果见表 30。在三个初态内部，D 的现金费用均最低；跨初态的费用不直接比较，因为初始可用能量不同。

表 30 问题二初始 SOC 敏感性分析

初态/kWh	A 费用/元	B 费用/元	C 费用/元	D 费用/元	最低方案
1200	1214697.70	1220138.06	1213153.67	1210276.33	D
6000	1212402.50	1217842.86	1210858.47	1207981.13	D
10800	1210148.61	1215636.23	1208651.84	1205774.50	D

最后一个跨年区间缺少右端实测节点。固定此前合同、动作与入段 SOC，采用持平、末两点线性外推和末六点回归外推，只重新计算末段。A、B、C 三种估计下总费用跨度均为 42.30 元，D 为 0 元，远小于方案间费用差，因而不改变排序。

附录 F 问题四指定日期计算结果

问题四分别采用固定普通购电计划的 4-2 方案（Q42_point_H0）和允许在 0、6、12、18 时更新预报并调单的 4-3 方案（Q43_point_H0_6_12_18）。4-2 和 4-3 的普通购电结果分别见表 31 和表 32，分时储能结果见表 33，连续紧急购电区间见表 34。各表均按 00:00–24:00 自然日重组，首个区间继承前一合同，储能状态不按日重置。

表 31 问题四 4-2 方案指定日期普通购电量及费用

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天普通购电量/kWh	全天现金费/元	紧急电量/kWh
2025.3.20	0.0000	623.4737	0.0000	511.6361	0.0000	685.7413	67866.1110	43531.94	0.0000
2025.6.21	0.0000	0.0000	0.0000	302.0390	402.8941	0.0000	36040.3555	19741.39	0.0000
2025.9.23	0.0000	520.3981	0.0000	601.0830	0.0000	754.2057	69645.6656	46288.13	181.9561
2025.12.21	0.0000	941.4449	0.0000	858.9718	699.7449	0.0000	94414.2102	73009.48	0.0000

表 32 问题四 4-3 方案指定日期普通购电量及费用

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天普通购电量/kWh	全天现金费/元	紧急电量/kWh
2025.3.20	0.0000	499.9121	0.0000	644.0707	0.0000	685.7413	65270.2372	44383.24	452.6103
2025.6.21	0.0000	0.0000	0.0000	75.8765	401.1816	0.0000	34236.9661	17180.56	0.0000
2025.9.23	0.0000	366.9624	0.0000	516.5722	94.0452	754.2057	66962.4039	45369.25	39.5829
2025.12.21	0.0000	966.0375	0.0000	978.4199	693.8813	0.0000	94847.1199	73672.07	96.0791

表中 10:00 等列分别表示 10:00–10:10 等六个题目指定区间。全天现金费按实际区间价格结算并包含紧急购电费用。

表 33 问题四指定日期分时充放电量及日初、日末储电量

方案	日期	时间段	充电量/kWh	放电量/kWh	0:00 SOC/kWh	24:00 SOC/kWh
4-2	2025.3.20	00:00–04:00	5337.0814	88.7519	6016.0388	4935.9929
		04:00–08:00	226.8834	4278.4021		
		08:00–12:00	4813.7448	3013.3626		
		12:00–16:00	4274.5723	251.6703		
		16:00–20:00	67.2248	6760.9219		
		20:00–24:00	3788.9906	1570.8153		
4-2	2025.6.21	00:00–04:00	1044.9116	4170.1320	6567.4867	6514.5361
		04:00–08:00	1099.1757	1659.9957		
		08:00–12:00	9756.3938	0.0000		
		12:00–16:00	0.0000	0.0000		
		16:00–20:00	1.3158	3923.0487		
		20:00–24:00	3082.8318	2432.0283		
4-2	2025.9.23	00:00–04:00	5825.9715	1.3012	4849.1012	5636.5899
		04:00–08:00	787.7447	5824.7534		
		08:00–12:00	5527.8015	2136.2388		
		12:00–16:00	4480.9552	455.8727		
		16:00–20:00	173.5846	6750.4099		
		20:00–24:00	4929.5444	1720.4216		
4-2	2025.12.21	00:00–04:00	5409.3307	133.2575	5943.3635	5874.2682
		04:00–08:00	1104.7111	772.1435		
		08:00–12:00	4697.1871	7498.5572		
		12:00–16:00	7154.7454	2131.0286		
		16:00–20:00	42.2275	4232.8437		
		20:00–24:00	3337.1257	2908.0706		
4-3	2025.3.20	00:00–04:00	5033.2257	34.9302	5942.0960	5302.5705
		04:00–08:00	182.0174	5863.6730		
		08:00–12:00	2688.6240	2593.6301		
		12:00–16:00	6873.6104	182.2666		
		16:00–20:00	662.2153	6773.7967		
		20:00–24:00	4617.4135	1373.5325		
4-3	2025.6.21	00:00–04:00	966.0258	4335.1952	6567.1371	6597.4120
		04:00–08:00	1114.6911	1601.1643		
		08:00–12:00	9951.3031	0.0000		
		12:00–16:00	0.0000	0.0000		
		16:00–20:00	4.8445	5077.1373		
		20:00–24:00	5430.0371	3107.4460		

续下页

续表 33

方案	日期	时间段	充电量/kWh	放电量/kWh	0:00 SOC/kWh	24:00 SOC/kWh
4-3	2025.9.23	00:00–04:00	4830.5907	0.0000	5534.8797	6018.4553
		04:00–08:00	1037.3997	5800.6335		
		08:00–12:00	4258.9433	2136.2388		
		12:00–16:00	5874.6127	287.2811		
		16:00–20:00	107.7446	6943.4628		
		20:00–24:00	5384.6702	1807.2743		
4-3	2025.12.21	00:00–04:00	5133.4915	61.0326	6072.3019	6084.6546
		04:00–08:00	1162.4898	858.0157		
		08:00–12:00	3943.4800	8565.7682		
		12:00–16:00	9343.3865	2122.3619		
		16:00–20:00	0.0000	5063.1111		
		20:00–24:00	5456.4378	3600.4144		

表 34 问题四指定日期连续紧急购电区间

方案	日期	紧急购电时段	区间电量/kWh	当日合计/kWh
4-2	2025.3.20	无	0.0000	0.0000
4-2	2025.6.21	无	0.0000	0.0000
4-2	2025.9.23	19:00–19:20	15.4513	181.9561
		20:10–21:10	166.5048	
4-2	2025.12.21	无	0.0000	0.0000
4-3	2025.3.20	09:10–10:10	419.7326	452.6103
		20:20–20:30	23.1158	
		21:40–21:50	9.7619	
4-3	2025.6.21	无	0.0000	0.0000
4-3	2025.9.23	20:10–20:40	17.6724	39.5829
		21:00–21:10	21.9104	
4-3	2025.12.21	10:50–11:00	8.2332	96.0791
		18:20–19:00	70.6748	
		20:40–20:50	17.1711	

注：各日期均按 00:00–24:00 自然日统计。当日合计由未舍入电量求和，可能与表中区间电量之和存在 0.0001 kWh 的舍入差。

附录 G 敏感性与稳健性补充结果

表 35和表 36汇总全部已实施检验。表中“局部”表示从共同初态独立回放指定窗口，不与全年连续轨迹直接拼接；Bootstrap 区间用于描述历史样本中的费用差稳定性，不是未来收益或价格预测区间。

表 35 问题一、二已实施的敏感性与稳健性检验

问题	检验项目	设置	结果摘要
一	储能效率口径	$\eta_c = \eta_d = 0.9$ 与 $\sqrt{0.9}$	费用下降 3.79%，约束仍全部满足
二	储能效率口径	对 A-D 进行 334 窗重跑	费用下降 2.52% ~ 3.10%，D 仍最低
二	裕量分位数	$q = 0, 0.5, 0.8, 0.9$ ，开发窗校准	选择 $q = 0.5$ ；裕量过大可能增加未用合同电
二	初始 SOC	1200、6000、10800 kWh，独立 28 窗	各初态下 D 均为组内最低
二	控制器消融	同合同下比较贪心与 MPC	MPC 整体降低费用，但并非每个窗口均占优
二	规划方法消融	点预测、误差裕量、随机场景，共同 MPC	场景随机规划减少紧急电并取得最低总费用
二	末端补值	持平、两点线性、六点 OLS，只重放末段	仅影响最后区间，主要排序不变
二	块 Bootstrap	7 窗圆形移动块，1000 次	D 相对 B、C 的平均费用优势方向稳定

表 36 问题三、四已实施的敏感性与稳健性检验

问题	检验项目	设置	结果摘要
三	储能效率口径	三种指定策略按 $\sqrt{0.9}$ 重跑 334 窗	场景 MPC 最低；随机规划优于确定性，MPC 与贪心接近
三	预报/调单时刻	0/6/12/18 构成的 8 种组合	全时刻启用方案最低
三	场景数与种子	4/8/16 场景；3 个种子，8 场景为折中；抽样会造成有限费用波动 7 个开发窗	
三	初始 SOC	1200/6000/10800 kWh，现金受初始库存影响，库存修正后差异缩小独立 60 窗	
三	算法消融	场景 MPC、确定性 MPC、场景贪心，60 窗	场景法优于确定性；MPC 与贪心差异很小
三	光伏输入来源	队伍区间预测与直接线性还原	费用差约 0.0046%
三	边界与执行粒度	多种首尾补值；10 分钟与分钟因果回放	组合排名不变；10 分钟均值反馈存在乐观风险
四	储能效率口径	4-2、4-3 最终冻结策略重跑 334 窗	分别下降 2.93%、3.19%，4-3 仍优
四	历史窗口与场景数	14/28/56 历史窗；4/8/16 场景，局部 14 窗	参数影响费用，但未推翻主要比较结论
四	随机种子与末端惩罚	3 个种子； κ 取基准值的 0.5、1、2 倍	4-2 对种子较敏感，末端价值影响库存使用
四	初始 SOC 与控制器	1200/6000/10800 kWh；MPC/贪心	库存修正后初态差异缩小，控制器优势依分支而异
四	价格场景构造	点价格、联合、均值修正、打乱配对	联合价格场景未显示稳定额外收益
四	更新时刻与 Bootstrap	8 种调单组合；3/7/14 日块各 2000 次	全时刻组合最低；调单优势较稳定
四	执行粒度与尾段	10 分钟/1 分钟/0.1 分钟；3 × 3 尾段估计	分钟执行费用较高；尾段全年费用跨度约 11.5 元

附录 H 程序文件清单

附录 I 支撑材料文件清单

本清单依据支撑材料目录中的文件清单.csv 生成，共列出 206 个交付文件。为便于复核，表 37 保留相对路径和文件大小。

表 37 支撑材料完整文件清单

相对路径	文件大小/字节
原项目 README.MD	10047
源代码附录.md	1128508
common\run_efficiency_sensitivity_sqrt09.py	1790
common\solve.py	14929
generate_code_appendix.py	4938
question1\结果说明.md	1942
question1\independent_audit.md	2110

续下页

相对路径	文件大小/字节
question1\manifest.json	2328
question1\q1.py	17078
question1\README.md	5083
question1\requirements.txt	46
question2\planning\code\audit_window_results.py	24834
question2\planning\code\build_conclusion.py	15398
question2\planning\code\build_window_report.py	36849
question2\planning\code\complete_initial_soc_baseline.py	7366
question2\planning\code\control_refined.py	14679
question2\planning\code\control.py	15112
question2\planning\code\evaluate_reports.py	5180
question2\planning\code\input_io.py	10435
question2\planning\code\planners.py	22361
question2\planning\code\risk.py	2907
question2\planning\code\run_all.py	1292
question2\planning\code\study.py	14880
question2\planning\code\synthetic.py	2271
question2\planning\code\test_calibration_cutoff.py	1447
question2\planning\code\test_control_v2.py	8893
question2\planning\code\test_input_risk.py	12278
question2\planning\code\test_planner_polish.py	5849
question2\planning\code\test_planner_prefix.py	5225
question2\planning\code\test_planners.py	8767
question2\planning\code\test_window_control.py	6716
question2\planning\code\test_window_io.py	7107
question2\planning\code\test_window_study.py	10345
question2\planning\code>window_control.py	6351
question2\planning\code>window_io.py	11842
question2\planning\code>window_study.py	17567
question2\planning\explain.md	7684
question2\planning\README.md	10873
question2\planning\requirements.txt	62
question2\planning\support_manifest.json	24496
question2\planning\support\独立审计说明.md	1180
question2\planning\support\计算结论.md	8982
question2\planning\support\交付副本测试记录.txt	9783
question2\planning\support\交付说明.md	1291
question2\planning\support\模型与执行步骤.md	7387
question2\planning\support\阅读说明.md	2603
question2\planning\support\运行环境.json	444
question2\planning\support\资料来源.md	1937
question2\prediction\光伏改进实验报告.md	3643
question2\prediction\交付.md	23842
question2\prediction\跨日负载边界预测报告.md	3388
question2\prediction\baseline_forecast.py	7296
question2\prediction\cross_day_load_boundary.py	12727
question2\prediction\export_planning_forecast.py	10913
question2\prediction\export_standardized_interval_forecast.py	8592
question2\prediction\export_standardized_intervals.py	7257
question2\prediction\forecast_models.py	16097
question2\prediction\improved_pv_forecast.py	14666
question2\prediction\interval_forecast_postprocess.py	8111
question2\prediction\interval_preprocessing.py	10171
question2\prediction\README.md	4324
question2\prediction\requirements.txt	114
question3\judge\问题三_补充预报价值分析.md	5232
question3\judge\analyze_forecast_value.py	23810
question3\judge\README.md	613
question3\planning\交付文件校验.json	46698
question3\planning\code\build_q3_method_doc.py	10678
question3\planning\code\q3_audit_batch.py	5837
question3\planning\code\q3_audit.py	25299
question3\planning\code\q3_boundaries.py	9418
question3\planning\code\q3_causal_replay.py	15831
question3\planning\code\q3_conclusions.py	29414
question3\planning\code\q3_figures.py	7860

续下页

相对路径	文件大小/字节
question3\planning\code\q3_guard_review.py	11176
question3\planning\code\q3_io.py	19312
question3\planning\code\q3_optimizer.py	17945
question3\planning\code\q3_reports.py	29982
question3\planning\code\q3_risk.py	3355
question3\planning\code\q3_runner.py	16740
question3\planning\code\q3_sensitivity.py	2101
question3\planning\code\run_all.py	2788
question3\planning\code\test_optimizer.py	6474
question3\planning\code\test_q3_audit.py	5190
question3\planning\code\test_q3_cache.py	1631
question3\planning\code\test_q3_chronology.py	2079
question3\planning\code\test_q3_conclusions.py	1547
question3\planning\code\test_q3_io.py	4485
question3\planning\code\test_q3_reports.py	7316
question3\planning\notes\报表字段复核.md	1474
question3\planning\notes\代码版本与复现.md	3416
question3\planning\notes\方法审查结论.md	13550
question3\planning\notes\控制器差异审查.md	7688
question3\planning\notes\输入审计与时间接口.md	6030
question3\planning\notes\问题三_结果与评价.md	15556
question3\planning\notes\问题三_模型与操作说明_验证.json	4319
question3\planning\notes\问题三_模型与操作说明.conversion.json	2316
question3\planning\notes\问题三_模型与复现说明.md	24265
question3\planning\notes\阅读说明.md	853
question3\planning\notes\audit_main_validation.json	8715
question3\planning\notes\audit_sensitivity_validation.json	1764
question3\planning\notes\control_difference_counterfactuals.json	6394
question3\planning\notes\control_difference_evidence.json	15920
question3\planning\notes\forecast_comparison.json	592
question3\planning\notes\reference_review.md	16205
question3\planning\notes\test_log_delivery.txt	7246
question3\planning\notes\test_log_final.txt	7246
question3\planning\notes\workbook_formula_audit.json	208
question3\planning\README.md	10359
question3\planning\requirements.txt	80
question3\prediction\交付.md	5807
question3\prediction\build_pv_interval_forecasts.py	23015
question3\prediction\compare_forecast_versions.py	21632
question4\第四问_结果与结论.md	11733
question4\第四问_模型与操作方法.md	11257
question4\交付文件清单.json	108546
question4\code\audit_price_inputs.py	4149
question4\code\build_document.py	11290
question4\code\package_delivery.py	5956
question4\code\physical_io.py	19312
question4\code\plot_q4_relative_schedule.py	3904
question4\code\price_inputs.py	13118
question4\code\q4_analysis.py	28843
question4\code\q4_audit.py	25785
question4\code\q4_causal_audit.py	15204
question4\code\q4_causal_replay.py	21927
question4\code\q4_causal_study.py	11090
question4\code\q4_conclusions.py	13251
question4\code\q4_figures.py	25236
question4\code\q4_four_day_review.py	6162
question4\code\q4_history_check.py	3232
question4\code\q4_io.py	3362
question4\code\q4_optimizer.py	20767
question4\code\q4_price_reports.py	11502
question4\code\q4_reports.py	37160
question4\code\q4_risk.py	3653
question4\code\q4_runner.py	16558
question4\code\q4_solver_check.py	14278
question4\code\q4_solver_zero_screen.py	3982
question4\code\q4_study.py	6396

续下页

相对路径	文件大小/字节
question4\code\README.md	792
question4\code\run_all.py	4922
question4\code\test_price_inputs.py	4821
question4\code\test_q4_analysis.py	7574
question4\code\test_q4_audit.py	3808
question4\code\test_q4_causal_replay.py	12324
question4\code\test_q4_figures.py	5712
question4\code\test_q4_independent.py	8982
question4\code\test_q4_reports.py	7993
question4\code\test_q4_solver_check.py	2781
question4\code\verify_document.py	4931
question4\code\xgb_intraday.py	6778
question4\notes\代码版本与复现.md	3026
question4\notes\电价输入与日内更新.md	5323
question4\notes\独立审查来源.json	2170
question4\notes\方案独立审查.md	19208
question4\notes\分钟因果执行敏感性.md	5069
question4\notes\交付副本测试.txt	249
question4\notes\交付副本独立审计.txt	71
question4\notes\交付副本分钟审计.txt	1744
question4\notes\交付副本复核.json	506
question4\notes\交付副本文稿验证.json	710
question4\notes\求解精度解释.md	5149
question4\notes\首次完整测试.txt	194
question4\notes\输出范围与技能适配.md	972
question4\notes\填表契约.md	12477
question4\notes\通用扫描兼容性说明.md	997
question4\notes\通用数据扫描日志.txt	3957
question4\notes\统一入口执行日志.txt	3920
question4\notes\完整测试_计算中.txt	238
question4\notes\文稿生成日志.json	2068
question4\notes\文稿自动验证.json	710
question4\notes\用户提供的参考思路.txt	12899
question4\notes\原模板审计.json	13889
question4\notes\ai-usage-log.md	973
question4\notes\calibration_log.txt	2417
question4\notes\causal_replay_tests.txt	1470
question4\notes\data_profile.md	161303
question4\notes\data-audit.md	1361
question4\notes\DOCX公式校验.json	226
question4\notes\DOCX结构校验.txt	366
question4\notes\history_check_log.txt	707
question4\notes\main_log.txt	14185
question4\notes\model-registry.md	1134
question4\notes\price_generation_log.txt	4405
question4\notes\price_input_tests.txt	1130
question4\notes\problem-contract.md	1269
question4\notes\result-ledger.md	1173
question4\notes\selection_log.txt	6942
question4\notes\sensitivity_log.txt	3221
question4\notes\source-ledger.md	1112
question4\notes\test_q4_figures_log.txt	1231
question4\notes\test_q4_reports_log.txt	1407
question4\notes\test_q4_solver_check_log.txt	989
question4\notes\validation-plan.md	2269
question4\README.md	5762
question4\requirements-documents.txt	80
question4\requirements.txt	130
question4\support\第四问_结果与结论.md	11733
question4\support\阅读说明.md	844
question4\support\阅读与验收说明.md	902
README.md	3338
result\result1.xlsx	11362
result\result2.xlsx	816266
result\result3.xlsx	1205215
result\result4-2.xlsx	732931

续下页

相对路径	文件大小/字节
result\result4-3.xlsx	1217761

附录 J 源程序代码

后续各页为本文数据预处理及问题一至问题四求解程序的带高亮代码清单。

本附录共收录 **97** 个 Python 源文件。文件按相对路径排序；代码保持文本形式，可复制、搜索和选择。

文件目录

1. common/run_efficiency_sensitivity_sqrt09.py
2. common/solve.py
3. question1/q1.py
4. question2/planning/code/audit_window_results.py
5. question2/planning/code/build_conclusion.py
6. question2/planning/code/build_window_report.py
7. question2/planning/code/complete_initial_soc_baseline.py
8. question2/planning/code/control.py
9. question2/planning/code/control_refined.py
10. question2/planning/code/evaluate_reports.py
11. question2/planning/code/input_io.py
12. question2/planning/code/planners.py
13. question2/planning/code/risk.py
14. question2/planning/code/run_all.py
15. question2/planning/code/study.py
16. question2/planning/code/synthetic.py
17. question2/planning/code/test_calibration_cutoff.py
18. question2/planning/code/test_control_v2.py
19. question2/planning/code/test_input_risk.py
20. question2/planning/code/test_planner_polish.py
21. question2/planning/code/test_planner_prefix.py
22. question2/planning/code/test_planners.py
23. question2/planning/code/test_window_control.py
24. question2/planning/code/test_window_io.py
25. question2/planning/code/test_window_study.py
26. question2/planning/code/window_control.py
27. question2/planning/code/window_io.py
28. question2/planning/code/window_study.py
29. question2/prediction/baseline_forecast.py
30. question2/prediction/cross_day_load_boundary.py
31. question2/prediction/export_planning_forecast.py
32. question2/prediction/export_standardized_interval_forecast.py
33. question2/prediction/export_standardized_intervals.py
34. question2/prediction/forecast_models.py
35. question2/prediction/improved_pv_forecast.py
36. question2/prediction/interval_forecast_postprocess.py
37. question2/prediction/interval_preprocessing.py
38. question3/judge/analyze_forecast_value.py
39. question3/planning/code/build_q3_method_doc.py
40. question3/planning/code/q3_audit.py
41. question3/planning/code/q3_audit_batch.py
42. question3/planning/code/q3_boundaries.py

- 43. question3/planning/code/q3_causal_replay.py
- 44. question3/planning/code/q3_conclusions.py
- 45. question3/planning/code/q3_figures.py
- 46. question3/planning/code/q3_guard_review.py
- 47. question3/planning/code/q3_io.py
- 48. question3/planning/code/q3_optimizer.py
- 49. question3/planning/code/q3_reports.py
- 50. question3/planning/code/q3_risk.py
- 51. question3/planning/code/q3_runner.py
- 52. question3/planning/code/q3_sensitivity.py
- 53. question3/planning/code/run_all.py
- 54. question3/planning/code/test_optimizer.py
- 55. question3/planning/code/test_q3_audit.py
- 56. question3/planning/code/test_q3_cache.py
- 57. question3/planning/code/test_q3_chronology.py
- 58. question3/planning/code/test_q3_conclusions.py
- 59. question3/planning/code/test_q3_io.py
- 60. question3/planning/code/test_q3_reports.py
- 61. question3/prediction/build_pv_interval_forecasts.py
- 62. question3/prediction/compare_forecast_versions.py
- 63. question4/code/audit_price_inputs.py
- 64. question4/code/build_document.py
- 65. question4/code/package_delivery.py
- 66. question4/code/physical_io.py
- 67. question4/code/plot_q4_relative_schedule.py
- 68. question4/code/price_inputs.py
- 69. question4/code/q4_analysis.py
- 70. question4/code/q4_audit.py
- 71. question4/code/q4_causal_audit.py
- 72. question4/code/q4_causal_replay.py
- 73. question4/code/q4_causal_study.py
- 74. question4/code/q4_conclusions.py
- 75. question4/code/q4_figures.py
- 76. question4/code/q4_four_day_review.py
- 77. question4/code/q4_history_check.py
- 78. question4/code/q4_io.py
- 79. question4/code/q4_optimizer.py
- 80. question4/code/q4_price_reports.py
- 81. question4/code/q4_reports.py
- 82. question4/code/q4_risk.py
- 83. question4/code/q4_runner.py
- 84. question4/code/q4_solver_check.py
- 85. question4/code/q4_solver_zero_screen.py
- 86. question4/code/q4_study.py
- 87. question4/code/run_all.py
- 88. question4/code/test_price_inputs.py
- 89. question4/code/test_q4_analysis.py
- 90. question4/code/test_q4_audit.py

91. question4/code/test_q4_causal_replay.py
92. question4/code/test_q4_figures.py
93. question4/code/test_q4_independent.py
94. question4/code/test_q4_reports.py
95. question4/code/test_q4_solver_check.py
96. question4/code/verify_document.py
97. question4/code/xgb_intraday.py

1. common/run_efficiency_sensitivity_sqrt09.py

源文件 SHA-256: 9d8f240828fc59c7a22ed1e423cdc69ae6e11873495d6e08f4b4784bb665cca9 ; 代码行数: 39。

```
import os, sys, json
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path

ETA = 0.9 ** 0.5
os.environ['MCM_BATTERY_ETA'] = str(ETA)
ROOT = Path(__file__).resolve().parent

def run_q3_one(spec):
    d = ROOT / '附件' / 'question3' / '第三问_完整计算' / '第三问_完整计算'
    sys.path.insert(0, str(d / 'code'))
    import q3_runner
    out = d / 'results_sqrt09'
    controller, scenarios, name = spec
    return q3_runner.run_strategy(d/'data', out, (0,6,12,18), config={'controller': controller,
'scenarios': scenarios}, name=name)

def run_q3():
    specs=[('mpc',8,'H0_6_12_18_scenario_mpc_sqrt09'),('mpc',1,'H0_6_12_18_deterministic_mpc_sqrt09'),
('greedy',8,'H0_6_12_18_scenario_greedy_sqrt09')]
    with ProcessPoolExecutor(max_workers=3) as pool:
        return list(pool.map(run_q3_one,specs))

def run_q4_one(spec):
    d = ROOT / '附件' / 'question4' / '第四问_完整计算' / '第四问_完整计算'
    sys.path.insert(0, str(d / 'code'))
    import q4_runner
    out = d / 'results_sqrt09'
    branch, schedule, name = spec
    cfg={'controller':'mpc','scenarios':8,'price_mode':'point','price_hours':[0,6,12,18]}
    return q4_runner.run_strategy(d/'data', out, schedule, branch=branch, config=cfg, name=name)

def run_q4():
    specs=[('4-2',(0),'Q42_point_H0_sqrt09'),('4-3',(0,6,12,18),'Q43_point_H0_6_12_18_sqrt09')]
    with ProcessPoolExecutor(max_workers=2) as pool:
        return list(pool.map(run_q4_one,specs))

if __name__ == '__main__':
    all_results = {'eta': ETA, 'q3': run_q3(), 'q4': run_q4()}
    (ROOT/'efficiency_sensitivity_sqrt09_raw.json').write_text(json.dumps(all_results,
ensure_ascii=False, indent=2, default=str), encoding='utf-8')
    print(json.dumps(all_results, ensure_ascii=False, default=str))
```

2. common/solve.py

源文件 SHA-256: c4aaae770c229f977fad3bf424a44b2cc27d101f39a0e5bf09241c66278eca21 ; 代码行数: 270。

```

"""CUMCM C: causal empirical-scenario procurement and storage dispatch.
Run from project root: python solve.py --smoke; python solve.py --all
Input XLSX files are read only. All power samples are right-end samples.
"""

from pathlib import Path
import argparse, json, time, sys, hashlib
from copy import copy
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from scipy.optimize import linprog, milp, Bounds, LinearConstraint
from scipy.sparse import lil_matrix, vstack, hstack, csr_matrix

ROOT = Path(__file__).resolve().parent
SEED = 20260910
DT, EMIN, EMAX, EINIT, B = 1/6, 1200., 10800., 6000., 5000/6
SKILL = Path('C:/Users/admin/.codex/skills/math-modeling-skill')
np.random.seed(SEED)

def read_data():
    def rows(name, sheet=0):
        w=load_workbook(ROOT/'附件'/name, read_only=True, data_only=True)
        s=w.worksheets[sheet] if isinstance(sheet,int) else w[sheet]
        out=list(s.values); w.close(); return out
    a=np.array(rows('附件1.xlsx')[1:],object)[:,:1:].astype(float)
    lrows=rows('附件2.xlsx',0); srows=rows('附件2.xlsx',1)
    prows=rows('附件4.xlsx'); frows=rows('附件3.xlsx')
    dates=pd.DatetimeIndex([r[0] for r in lrows[1:]])
    assert list(dates)==list(pd.date_range('2025-01-01','2025-12-31'))
    for rr in [srows,prows]:
        assert [r[0] for r in rr[1:]]==list(dates.to_pydatetime())
        assert rr[0][1:]==lrows[0][1:]
    L=np.array([r[1:] for r in lrows[1:]],float)*DT
    S=np.array([r[1:] for r in srows[1:]],float)*DT
    P=np.array([r[1:] for r in prows[1:]],float)
    F=np.array([r[2:] for r in frows[1:]],float).reshape(365,4,24)*DT
    date=None
    for i,r in enumerate(frows[1:]):
        if r[0]: date=pd.Timestamp(r[0])
        assert date==dates[i//4] and r[1]==f'{(i%4)*6}:00'
    assert a.shape==(144,3) and L.shape==S.shape==P.shape==(365,144)
    assert F.shape==(365,4,24)
    for v in [a,L,S,P,F]: assert np.isfinite(v).all() and v.min()≥0
    return dict(a=a, L=L,S=S,P=P,F=F,dates=dates)

def forecast(data, day, release, step=False):
    """Only release data and already-observed PV at the release boundary."""
    start=release*36
    anchor=data['S'][day,start-1] if start else (data['S'][day-1,-1] if day else 0.)
    hourly=data['F'][day,release]
    if step: return np.repeat(hourly,6)
    return np.maximum(0,np.interp(np.arange(1,145)/6,np.arange(25),np.r_[anchor,hourly]))

def scenarios(data,day,release,pv,K=14,step=False):
    slots=(release*36+np.arange(144))%144
    if day==0:
        load=data['a'][:,:1]*DT
        sol=forecast(data,day,release,step) if pv else data['a'][slots,2]*DT
        return (load[slots]-sol)[None,: ]
    if not pv:
        hist=np.arange(max(0,day-K),day)
        return (data['L'][hist]-data['S'][hist])[: ,slots]
    latest=forecast(data,day,release,step)
    # Historical 24-hour forecasts have finished before the current day.

```

```

hist=np.arange(max(0,day-1-K),day-1)
if not len(hist):
    hist=np.arange(max(0,day-K),day)
    return data['L'][hist][:,slots]-latest
err=[]
flat=data['S'].ravel()
for h in hist:
    k=h*144+release*36
    err.append(flat[k:k+144]-forecast(data,h,release,step))
return data['L'][hist][:,slots]-np.maximum(0,latest+np.array(err))

def prices(data,day,release,variable,oracle=False):
    slots=(release*36+np.arange(144))%144
    if not variable: return data['a'][slots,0]
    if oracle:
        arr=data['P'].ravel(); ix=day*144+release*36+np.arange(144)
        # Beyond the supplied year no oracle is available; repeat last day.
        return arr[np.minimum(ix,len(arr)-1)] if ix[-1]<len(arr) else
np.r_[arr[ix[ix<len(arr)]]],data['P'][-1,slots[ix≥len(arr)]]]
    return (data['P'][max(0,day-7):day].mean(0) if day else data['a'][:,0])[slots]

def optimize(net,price,E0,eta=.9,base=None,known=None,terminal=True,no_battery=False):
    """LP: g,c,d,E,scenario r, purchase cost epigraph q, terminal abs h."""
    m,T=net.shape
    G,C,D,E,R,Q,H=0,T,2*T,3*T,4*T,4*T+m*T,5*T+m*T
    nv=H+1; obj=np.zeros(nv)
    obj[R:Q]=np.tile(5*price/m,m); obj[Q:H]=1
    obj[H]=np.median(price)/eta if terminal else 0
    # Small lexicographic regularizer only breaks near-degenerate cycling.
    obj[C:D]=1e-9; obj[D:E]=1e-9
    bounds=[(0,None)]*nv
    for t in range(T):
        bounds[C+t]=(0,0 if no_battery else B)
        bounds[D+t]=(0,0 if no_battery else (min(B,known[t]) if known is not None else B))
        bounds[E+t]=(EMIN,EMAX)
    if not terminal: bounds[E+T-1]=(E0,E0)
    if known is not None:
        for j in range(R,Q): bounds[j]=(0,0)
    eq=lil_matrix((T,nv)); beq=np.zeros(T); beq[0]=E0
    for t in range(T):
        eq[t,E+t]=1; eq[t,C+t]=-eta; eq[t,D+t]=1/eta
        if t: eq[t,E+t-1]=-1
    ub=lil_matrix((m*T+2*T+2,nv)); bu=np.zeros(ub.shape[0])
    for s in range(m):
        rr=np.arange(T)+s*T
        for t,j in enumerate(rr):
            ub[j,C+t]=1;ub[j,D+t]=-1;ub[j,G+t]=-1;ub[j,R+j]=-1
            bu[rr]=-net[s]
    for t in range(T):
        rr=m*T+2*t
        if base is not None and np.isfinite(base[t]):
            # f(g)=max(.5 p(g+g0), 1.5pg-.5pg0).
            ub[rr,G+t]=.5*price[t];ub[rr,Q+t]=-1;bu[rr]=-1.5*price[t]*base[t]
            ub[rr+1,G+t]=1.5*price[t];ub[rr+1,Q+t]=-1;bu[rr+1]=.5*price[t]*base[t]
        else:
            ub[rr,G+t]=price[t];ub[rr,Q+t]=-1
            ub[rr+1,G+t]=price[t];ub[rr+1,Q+t]=-1
    ub[-2,E+T-1]=1;ub[-2,H]=-1;bu[-2]=EINIT
    ub[-1,E+T-1]=-1;ub[-1,H]=-1;bu[-1]=-EINIT
    ans=linprog(obj,A_ub=ub.tocsr(),b_ub=bu,A_eq=eq.tocsr(),b_eq=beq,bounds=bounds,method='highs')
    if not ans.success: raise RuntimeError(ans.message)
    g=np.maximum(ans.x[G:C],0);c=np.maximum(ans.x[C:D],0);d=np.maximum(ans.x[D:E],0)
    remove=np.minimum(c,d/eta**2);c-=remove;d-=eta**2*remove
    energy=np.r_[E0,E0+np.cumsum(eta*c-d/eta)]
    assert np.min(energy)≥EMIN-1e-6 and np.max(energy)≤EMAX+1e-6
    assert np.max(np.minimum(c,d))<1e-6

```

```

    return dict(g=g, c=c, d=d, E=energy, objective=float(ans.fun), nit=ans.nit,
                residual=float(np.max(np.abs(eq@ans.x-beq))))

def execute(g, cplan, dplan, L, S, E0, eta):
    T=len(g); c=np.zeros(T); d=np.zeros(T); E=np.zeros(T+1); E[0]=E0
    for t in range(T):
        c[t]=min(cplan[t], max(0, (EMAX-E[t])/eta))
        d[t]=min(dplan[t], max(0, eta*(E[t]-EMIN)), L[t])
        E[t+1]=E[t]+eta*c[t]-d[t]/eta
    x=g+S+d-L-c; r=np.maximum(-x, 0); w=np.minimum(S, np.maximum(x, 0)); z=np.maximum(x, 0)-w
    balance=g-z+S+d+r-L-c-w
    assert max(np.abs(balance))<1e-6 and max(z-g)<1e-6
    assert min(E)≥EMIN-1e-6 and max(E)≤EMAX+1e-6
    assert max(np.minimum(c, d))<1e-6 and max(c)≤B+1e-6 and max(d)≤B+1e-6
    return dict(c=c, d=d, E=E, r=r, w=w, z=z, projection=np.abs(c-cplan)+np.abs(d-dplan), balance=balance)

def q1(data, eta=.9):
    a=data['a']; L=a[:, 1]*DT; S=a[:, 2]*DT; p=a[:, 0]
    sol=optimize((L-S)[None, :], p, EINIT, eta, known=L, terminal=False)
    actual=execute(sol['g'], sol['c'], sol['d'], L, S, EINIT, eta)
    assert max(actual['projection'])<1e-6 and max(actual['r'])<1e-6

out=pd.DataFrame(dict(t=np.arange(144), L=L, S=S, p=p, g=sol['g'], c=actual['c'], d=actual['d'], E0=actual['E']
[: -1], E1=actual['E'][1:], w=actual['w'], z=actual['z']))
out['cost']=out.p*out.g
return
out, dict(cost=float(out.cost.sum()), purchase=float(out.g.sum()), baseline=float(np.dot(p, np.maximum(L-
S, 0))), lp_objective=sol['objective'], max_residual=sol['residual'])

def
simulate(data, name, days=365, K=14, eta=.9, pv=False, update=False, variable=False, oracle=False, step=False, no_
battery=False, verbose=True):
    frames=[]; E0=EINIT; starttime=time.perf_counter()
    for day in range(days):
        original=None; records=[]; previous=np.zeros(144); seq_extra=np.zeros(144)
        for release in (range(4) if update else [0]):
            start=36*release; size=36 if update else 144
            n=scenarios(data, day, release, pv, K, step)
            pp=prices(data, day, release, variable, oracle)
            base=None if release==0 else np.r_[original[start:], np.full(start, np.nan)]
            sol=optimize(n, pp, E0, eta, base=base, no_battery=no_battery)
            if release==0:
                original=sol['g'].copy(); previous=original.copy()
            else:
                new=sol['g'][:144-start]; old=previous[start:]
                # Increment to normal-price actual quantity cost for successive changes.
                seq_extra[start:] +=.5*np.abs(new-old)
                previous[start:]=new
            g=sol['g'][:size]; L=data['L'][day, start:start+size]; S=data['S'][day, start:start+size]
            act=execute(g, sol['c'][:size], sol['d'][:size], L, S, E0, eta)
            realp=data['P'][day, start:start+size] if variable else data['a'][start:start+size, 0]
            baseg=original[start:start+size]; up=np.maximum(g-baseg, 0); down=np.maximum(baseg-g, 0)
            ordinary=realp*baseg; refund=-
            realp*down; cancel=.5*realp*down; add=1.5*realp*up; emergency=5*realp*act['r']
            cost=ordinary+refund+cancel+add+emergency
            assert np.max(np.abs(cost-(realp*np.minimum(baseg, g)+.5*realp*down+1.5*realp*up+emergency)))
<1e-6
            records.append(pd.DataFrame(dict(scheme=name, date=str(data['dates']
[day].date()), t=np.arange(start, start+size),
                L=L, S=S, p=realp, p_hat=pp[:size], g0=baseg, g=g, c_plan=sol['c'][:size], d_plan=sol['d']
[:size],
                c=act['c'], d=act['d'], E0=act['E'][: -1], E1=act['E'][1:], r=act['r'], w=act['w'], z=act['z'],
                projection=act['projection'], ordinary=ordinary, refund=refund, cancel=cancel, add=add, emergency=emergency, c
ost=cost,
                no_refund_cost=cost-refund, seq_cost=realp*(g+seq_extra[start:start+size])+emergency,

```

```

        net_mean=n.mean(0)[:size],net_q80=np.quantile(n,.8,axis=0)
[:size],balance=act['balance'],solver_residual=sol['residual']))
        E0=act['E'][-1]
        frames.extend(records)
        if verbose and (day+1)%60==0: print(name,day+1,'days',round(time.perf_counter()-
starttime,1),'s',flush=True)
        return pd.concat(frames,ignore_index=True)

def interval(t):
    def fmt(k): return f'{k//6:02d}:{(k%6)*10:02d}'
    return fmt(t)+'-'+fmt(t+1)

def events(frame):
    records=[]
    for date,day in frame.groupby('date',sort=False):
        r=day.r.to_numpy();mask=r>1e-6;t=0;found=False
        while t<144:
            if not mask[t]:t+=1;continue
            a=t
            while t<144 and mask[t]:t+=1
            records.append(dict(date=date,start=a,end=t,period=interval(a).split('-')[0]+'-'+interval(t-
1).split('-')[1],energy=float(r[a:t].sum()))
            found=True
        if not found:records.append(dict(date=date,start=-1,end=-1,period='无',energy=0.))
    return pd.DataFrame(records)

def summarize(frame):
    sums=
    ['g0','g','r','c','d','w','z','ordinary','refund','cancel','add','emergency','cost','no_refund_cost','se
q_cost','projection']
    daily=frame.groupby(['scheme','date'])[sums].sum().reset_index()
    en=frame.groupby(['scheme','date']).agg(E_start=('E0','first'),E_end=('E1','last'))
    daily=daily.merge(en.reset_index())
    return daily

def write_template(name,frame,one=None):
    wb=load_workbook(ROOT/'附件'/'附件5'/f'{name}.xlsx')
    if name=='result1':
        sh=wb['计划购电量']
        for t in range(144):sh.cell(t+2,1,interval(t));sh.cell(t+2,2,float(one.g.iloc[t]))
        sh=wb['充放电量']
        for b in range(6):
            sh.cell(b+2,2,float(one.c.iloc[b*24:(b+1)*24].sum()));sh.cell(b+2,3,float(one.d.iloc[b*24:
(b+1)*24].sum()))
            sh.cell(2,5,float(one.E0.iloc[0]));sh.cell(3,5,float(one.E1.iloc[-1]))
        else:
            daily=summarize(frame)
            for sname,col in [('计划购电量','g0'),('调整购电量','g')]:
                if sname not in wb:continue
                sh=wb[sname]
                for t in range(144):sh.cell(1,t+2,interval(t))
                for i,(date,day) in enumerate(frame.groupby('date',sort=False),2):
                    sh.cell(i,1,pd.Timestamp(date).to_pydatetime())
                    for t,v in enumerate(day[col],2):sh.cell(i,t,float(v))
                    sh.cell(i,146,float(day[col].sum()))
                    # Original order sheet bills original normal purchases; adjusted sheet total final
                    settlement.
                    sh.cell(i,147,float(day.ordinary.sum() if col=='g0' else day.cost.sum()))
                sh=wb['充放电量'];sh.delete_rows(2,sh.max_row)
                for date,day in frame.groupby('date',sort=False):
                    for b in range(6):
                        chunk=day.iloc[b*24:(b+1)*24]
                        sh.append([pd.Timestamp(date).to_pydatetime() if b==0 else None,f'{4*b}:00-
{4*b+4}:00',float(chunk.c.sum()),float(chunk.d.sum()),'0:00' if b==0 else ('24:00' if b==1 else
None),float(day.E0.iloc[0]) if b==0 else (float(day.E1.iloc[-1]) if b==1 else None)])
                sh=wb['紧急购电量'];sh.delete_rows(2,sh.max_row)

```



```

        for row in
events(frame).itertuples():sh.append([pd.Timestamp(row.date).to_pydatetime(),row.period,row.energy])
    for sh in wb:
        for row in sh:
            for c in row:
                if isinstance(c.value,float):c.number_format='0.000000'
            assert all(c.data_type!='e' and c.data_type!='f' for row in sh for c in row)
wb.save(ROOT/'results'/f'{name}.xlsx')

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--smoke',action='store_true');ap.add_argument('--
all',action='store_true');ap.add_argument('--plots',action='store_true');ap.add_argument('--
output',default='results')
    args=ap.parse_args();data=read_data();out=ROOT/args.output;out.mkdir(parents=True,exist_ok=True)
    one,stat=q1(data);one.to_csv(out/'q1_dispatch.csv',index=False)
    if args.smoke:
        reports={'q1':stat}
        for name,pv,upd,var in [('q2',False,False,False),('q3',True,True,False),
('q4_2',False,False,True),('q4_3',True,True,True)]:
            f=simulate(data,name,days=3,pv=pv,update=upd,variable=var,verbose=False)
            f.to_csv(out/f'{name}_smoke.csv',index=False)
            # Future actual perturbation, while release forecasts and historical info stay fixed.
            altered={**data,'L':data['L'].copy(),'S':data['S'].copy(),'P':data['P'].copy()}
            altered['L'][1:]*=1.15;altered['S'][1:]*=.8;altered['P'][1:]*=1.2
            before=scenarios(data,1,0,pv);after=scenarios(altered,1,0,pv)
            assert np.array_equal(before,after)
            assert np.array_equal(prices(data,1,0,var),prices(altered,1,0,var))
            reports[name]=
{'days':3,'cost':float(f.cost.sum()),'emergency':float(f.r.sum()),'max_balance':float(f.balance.abs().ma
x()),'E_min':float(f.E1.min()),'E_max':float(f.E1.max()),'projection':float(f.projection.sum()),'causal_
check':True}
            (out/'smoke.json').write_text(json.dumps(reports,ensure_ascii=False,indent=2),encoding='utf-
8');print(json.dumps(reports,ensure_ascii=False,indent=2));return
    if not args.all:return
    # Additional experiments and figures are introduced only after P1 approval.
    raise RuntimeError('Full run awaits P1 gate')

if __name__=='__main__':main()

```

3. question1/q1.py

源文件 SHA-256: 18ad690b0cfe49a8ae3923e41097c41d5055563d077ef9898eddc978db6d7c83 ; 代码行数: 331。

"""问题一：严格按用户2026-09-10截图中的MILP模型求解。

运行: `python q1.py`

依赖: `numpy`、`scipy`、`openpyxl`; 不依赖其余问题的代码或商业求解器。

单位: 附件数据为每10分钟的瞬时端点值; 模型使用两端点平均值。

功率为 `kW`, 决策变量为每10分钟的电量 `kWh`, 费用为元。

"""

```
from __future__ import annotations
```

```
import argparse
```

```
import csv
```

```
import datetime as dt
```

```
import hashlib
```

```
import json
```

```
import os
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import openpyxl
```

```
import scipy
```

```
from openpyxl.styles import Alignment, Font, PatternFill
```

```
from scipy.optimize import Bounds, LinearConstraint, milp
```

```
from scipy.sparse import coo_matrix
```

```
ROOT = Path(__file__).resolve().parent
```

```
STEP_HOURS = 1 / 6
```

```
CAPACITY = 12000.0
```

```
SOC_MIN, SOC_MAX = 0.1 * CAPACITY, 0.9 * CAPACITY
```

```
SOC_START = SOC_END = 0.5 * CAPACITY
```

```
POWER_MAX = 5000.0
```

```
ENERGY_MAX = POWER_MAX * STEP_HOURS
```

```
# 主解释: 充电效率、放电效率分别为90%, 往返效率为81%。
```

```
ETA_C = ETA_D = float(os.environ.get("MCM_BATTERY_ETA", "0.9"))
```

```
def interval_label(slot: int, length: int = 1) → str:
```

```
    """slot 从0起; 最后一个时段是23:50-24:00。"""
```

```
    start, end = slot * 10, (slot + length) * 10
```

```
    return f"{start // 60:02d}:{start % 60:02d}-{end // 60:02d}:{end % 60:02d}"
```

```
def right_end_minute(value) → int:
```

```
    if isinstance(value, dt.time):
```

```
        return 60 * value.hour + value.minute
```

```
    text = str(value).strip()
```

```
    if text in ("0:00+1", "00:00+1", "24:00"):
```

```
        return 1440
```

```
    hour, minute = text.split(":")[:2]
```

```
    return 60 * int(hour) + int(minute)
```

```
def endpoint_label(minute: int) → str:
```

```
    """生成瞬时端点标签, 末点使用24:00以避免与当日00:00混淆。"""
```

```
    if minute == 1440:
```

```
        return "24:00"
```

```
    return f"{minute // 60:02d}:{minute % 60:02d}"
```

```
def write_preprocessed_data(path: Path, endpoint_data: np.ndarray,
```

```
                           interval_data: np.ndarray) → None:
```

```
    """暂存补齐后的瞬时端点和供模型使用的区间平均值。"""
```

```
    path.parent.mkdir(parents=True, exist_ok=True)
```

```
    book = openpyxl.Workbook()
```

```
    endpoints = book.active
```

```

endpoints.title = "瞬时端点"
endpoints.append(["时间", "瞬时电价_元每kWh", "瞬时小区负载_kW", "瞬时光伏功率_kW", "备注"])
for i, values in enumerate(endpoint_data):
    note = "复制原始24:00数值" if i == 0 else "来自附件1"
    endpoints.append([endpoint_label(i * 10), *map(float, values), note])

intervals = book.create_sheet("区间平均")
intervals.append(["时间段", "平均电价_元每kWh", "平均小区负载_kW",
                  "平均光伏功率_kW", "左端点", "右端点"])
for i, values in enumerate(interval_data):
    intervals.append([interval_label(i), *map(float, values),
                      endpoint_label(i * 10), endpoint_label((i + 1) * 10)])

for sheet in book:
    sheet.freeze_panes = "A2"
    sheet.auto_filter.ref = sheet.dimensions
    for cell in sheet[1]:
        cell.font = Font(name="宋体", size=11, bold=True)
        cell.fill = PatternFill("solid", fgColor="DCE6F1")
    for row in sheet.iter_rows(min_row=2):
        for cell in row:
            cell.font = Font(name="宋体", size=11)
            if isinstance(cell.value, (int, float)):
                cell.number_format = "#,##0.0000"
    for column in range(1, sheet.max_column + 1):
        sheet.column_dimensions[openpyxl.utils.get_column_letter(column)].width = 24
book.save(path)
book.close()

def read_data(path: Path, preprocessed_path: Path) → tuple[np.ndarray, np.ndarray, np.ndarray]:
    """读取144个瞬时端点，补入00:00后计算144个区间的端点平均值。"""
    book = openpyxl.load_workbook(path, read_only=True, data_only=True)
    try:
        rows = list(book.worksheets[0].values)
    finally:
        book.close()
    if len(rows) != 145 or any(len(row) != 4 for row in rows):
        raise ValueError("附件1应包含1行表头、144行数据、4列。")
    if list(rows[0]) != ["时间", "电价", "小区负载", "光伏发电预测功率"]:
        raise ValueError("附件1列名或列顺序与题目原文件不同，请先核对。")
    minutes = [right_end_minute(row[0]) for row in rows[1:]]
    if minutes != list(range(10, 1441, 10)):
        raise ValueError("时间应为00:10至次日00:00，每10分钟一条。")
    raw_data = np.asarray([row[1:] for row in rows[1:]], dtype=float)
    if not np.isfinite(raw_data).all() or (raw_data < 0).any() or (raw_data[:, 0] ≤ 0).any():
        raise ValueError("数据必须完整、有限、非负，且电价严格为正。")
    # 原始24:00数值复制为当日00:00，再对每个10分钟区间取左右端点算术平均。
    endpoint_data = np.vstack([raw_data[-1], raw_data])
    interval_data = (endpoint_data[:-1] + endpoint_data[1:]) / 2
    write_preprocessed_data(preprocessed_path, endpoint_data, interval_data)
    price = interval_data[:, 0]
    load = interval_data[:, 1] * STEP_HOURS
    pv = interval_data[:, 2] * STEP_HOURS
    return load, pv, price

def solve(load, pv, price):
    """g、c、d、r是微网侧kWh；s是电池内部kWh；u为充放电开关。"""
    n = len(load)
    # 决策向量x=[g, c, d, r, s, u]，各块有n项；s块对应S_1至S_144。
    # 图中r仅代表弃光，不包含已购电弃用。
    rows, cols, values = [], [], []

    def add(row, col, value):
        rows.append(row)

```

```

cols.append(col)
values.append(value)

for t in range(n):
    # 图中供需平衡:  $g + G\Delta t - r + d = L\Delta t + c$ 。
    for block, coefficient in [(0, 1), (1, -1), (2, 1), (3, -1)]:
        add(t, block * n + t, coefficient)
    # 储电递推:  $s[t+1] = s[t] + \text{ETA\_C} * c[t] - d[t] / \text{ETA\_D}$ 。
    add(n + t, n + t, -ETA_C)
    add(n + t, 2 * n + t, 1 / ETA_D)
    add(n + t, 4 * n + t, 1)
    if t > 0:
        add(n + t, 4 * n + t - 1, -1)
a_eq = coo_matrix((values, (rows, cols)), shape=(2 * n, 6 * n)).tocsc()
b_eq = np.r_[load - pv, SOC_START, np.zeros(n - 1)]

idx = np.arange(n)
# 目标严格为 $\sum(p * g)$ , 不添加充放电平局成本或终端奖励。
objective = np.zeros(6 * n)
objective[:n] = price
lower = np.zeros(6 * n)
upper = np.full(6 * n, np.inf)
upper[n:2 * n] = ENERGY_MAX
upper[2 * n:3 * n] = ENERGY_MAX
upper[3 * n:4 * n] = pv
lower[4 * n:5 * n] = SOC_MIN
upper[4 * n:5 * n] = SOC_MAX
lower[5 * n - 1] = upper[5 * n - 1] = SOC_END
upper[5 * n:6 * n] = 1
# u=1允许充电, u=0允许放电; 允许两者都为0。
charge_direction = coo_matrix(
    (np.r_[np.ones(n), -np.full(n, ENERGY_MAX)],
     (np.r_[idx, idx], np.r_[n + idx, 5 * n + idx])),
    shape=(n, 6 * n),
).tocsc()
discharge_direction = coo_matrix(
    (np.r_[np.ones(n), np.full(n, ENERGY_MAX)],
     (np.r_[idx, idx], np.r_[2 * n + idx, 5 * n + idx])),
    shape=(n, 6 * n),
).tocsc()
result = milp(
    objective, integrality=np.r_[np.zeros(5 * n), np.ones(n)],
    bounds=Bounds(lower, upper),
    constraints=[LinearConstraint(a_eq, b_eq, b_eq),
                 LinearConstraint(charge_direction, -np.inf, 0),
                 LinearConstraint(discharge_direction, -np.inf, ENERGY_MAX)],
    options={"mip_rel_gap": 1e-9, "time_limit": 60},
)
if not result.success:
    raise RuntimeError(f"求解失败: {result.message}")
# 保留原始求解器数值; 不做改变调度的后处理。
x = result.x.copy()
solution = {key: x[k * n:(k + 1) * n] for k, key in enumerate(("g", "c", "d", "r", "s", "u"))}
solution["s"] = np.r_[SOC_START, solution["s"]]
solution.update(cost=float(price @ solution["g"]), mip_gap=float(result.mip_gap),
                 mip_dual_bound=float(result.mip_dual_bound), solver_message=result.message)
return solution

def validate(solution, load, pv):
    """从返回轨迹重新核对物理约束; 有违反时直接报错。"""
    g, c, d, r, s, u = [solution[k] for k in ("g", "c", "d", "r", "s", "u")]
    checks = {
        "balance_error_kwh": float(np.max(abs(g + pv - r + d - load - c))),
        "state_error_kwh": float(np.max(abs(np.diff(s) - ETA_C * c + d / ETA_D))),
        "negative_flow_violation_kwh": float(max(0, -min(g.min(), c.min(), d.min(), r.min()))),
    }

```

```

"soc_lower_violation_kwh": float(max(0, SOC_MIN - s.min())),
"soc_upper_violation_kwh": float(max(0, s.max() - SOC_MAX)),
"charge_limit_violation_kwh": float(max(0, c.max() - ENERGY_MAX)),
"discharge_limit_violation_kwh": float(max(0, d.max() - ENERGY_MAX)),
"pv_curtailment_violation_kwh": float(max(0, np.max(r - pv))),
"charge_binary_violation_kwh": float(max(0, np.max(c - ENERGY_MAX * u))),
"discharge_binary_violation_kwh": float(max(0, np.max(d - ENERGY_MAX * (1 - u)))),
"binary_integrality_error": float(np.max(abs(u - np.round(u)))),
"binary_bound_violation": float(max(0, -u.min(), u.max() - 1)),
"start_end_error_kwh": float(max(abs(s[0] - SOC_START), abs(s[-1] - SOC_END))),
"simultaneous_slots": int(np.sum((c > 1e-7) & (d > 1e-7))),
}
if max(checks.values()) > 1e-5:
    raise RuntimeError(f"物理约束检查失败: {checks}")
return checks

def write_outputs(output, template, solution, load, pv, price, summary):
    output.mkdir(parents=True, exist_ok=True)
    g, c, d, r, s, u = [solution[k] for k in ("g", "c", "d", "r", "s", "u")]
    with (output / "q1_detail.csv").open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["时间段", "电价_元每kWh", "负载_kWh", "光伏_kWh", "购电_kWh",
                        "充电_kWh", "放电_kWh", "弃光_kWh", "段初储电_kWh", "段末储电_kWh", "费用_元", "充放电开关_u"])
        for t in range(len(g)):
            writer.writerow([interval_label(t), price[t], load[t], pv[t], g[t], c[t],
                            d[t], r[t], s[t], s[t + 1], price[t] * g[t], u[t]])
    (output / "q1_summary.json").write_text(
        json.dumps(summary, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")

# 在题目模板的副本填写，源模板保持不变；总计写数值，无需公式重算。
book = openpyxl.load_workbook(template)
plan_sheet, storage_sheet = book["计划购电量"], book["充放电量"]
for t in range(len(g)):
    plan_sheet.cell(t + 2, 1, interval_label(t))
    plan_sheet.cell(t + 2, 2, float(g[t]))
for block in range(6):
    lo, hi = block * 24, (block + 1) * 24
    storage_sheet.cell(block + 2, 1, interval_label(lo, 24))
    storage_sheet.cell(block + 2, 2, float(c[lo:hi].sum()))
    storage_sheet.cell(block + 2, 3, float(d[lo:hi].sum()))
storage_sheet["D2"], storage_sheet["D3"] = "00:00", "24:00"
storage_sheet["E2"], storage_sheet["E3"] = float(s[0]), float(s[-1])
totals = book.create_sheet("结果汇总")
totals.append(["指标", "数值"])
for name, value in [
    ("全天购电量 (kWh)", float(g.sum())), ("全天购电费用 (元)", solution["cost"]),
    ("无储能费用 (元)", summary["no_storage_cost_yuan"]),
    ("费用节约比例", summary["saving_fraction"]),
    ("全天充电量 (kWh, 微网侧)", float(c.sum())),
    ("全天放电量 (kWh, 微网侧)", float(d.sum())),
    ("全天弃光量 (kWh)", float(r.sum())), ("MILP相对最优间隙", solution["mip_gap"]),
]:
    totals.append([name, value])
totals["B3"].number_format = totals["B4"].number_format = "#,##0.00"
totals["B5"].number_format = "0.00%"
notes = book.create_sheet("口径说明")
for row in [
    ["项目", "说明"],
    ["数据预处理", "附件数据视为瞬时端点值；00:00复制24:00数值，每个10分钟区间取两端点算术平均。"],
    ["时间对应", "区间平均数据依次对应00:00-00:10至23:50-24:00；源附件未修改。"],
    ["供电路径", "外网和光伏可直接供负载，也可充电；容量只约束当时电池储电。"],
    ["效率", "(f"充电、放电单向效率分别为{ETA_C:.6%}、{ETA_D:.6%}；"
        f"往返效率为{ETA_C * ETA_D:.6%}；充放电量为微网侧电量。")],
    ["储能约束", "额定12000kWh，运行范围1200至10800kWh，日初日末均6000kWh。"],

```

```

["功率约束", "充放电各不超过5000kW, 即每10分钟各不超过833.3333kWh。"],
["弃光约束", "严格按截图:  $0 \leq r \leq G\Delta t$ , 仅允许弃光; 没有已购电弃用变量, 不售电。"],
["求解模型", "纯购电费用目标的MILP; u为二元开关; 没有额外平局成本或附加放电负载上限。"],
["显示精度", "电量显示4位小数、费用2位小数; 底层数值不截断, 总计直接保存数值。"],
]:
    notes.append(row)
for sheet in book:
    sheet.freeze_panes = "A2"
    for row in sheet:
        for cell in row:
            cell.font = Font(name="宋体", size=11, bold=cell.row == 1)
            cell.alignment = Alignment(vertical="center", wrap_text=True)
            if cell.row == 1:
                cell.fill = PatternFill("solid", fgColor="DCE6F1")
            if isinstance(cell.value, (int, float)) and cell.number_format == "General":
                cell.number_format = "#,##0.0000"
    for column in range(1, sheet.max_column + 1):
        sheet.column_dimensions[openpyxl.utils.get_column_letter(column)].width = 23
    if sheet.title in ("结果汇总", "口径说明"):
        sheet.column_dimensions["A"].width = 36
    if sheet.title == "口径说明":
        sheet.column_dimensions["B"].width = 80
book.save(output / "result1.xlsx")
book.close()

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=ROOT / "input/附件1.xlsx")
    parser.add_argument("--template", type=Path, default=ROOT / "input/result1模板.xlsx")
    parser.add_argument("--output", type=Path, default=ROOT / "results")
    args = parser.parse_args()
    # 防止用户把输出目录指向输入目录而意外覆盖任何输入文件。
    output_files = [args.output / name for name in
                     ("result1.xlsx", "q1_detail.csv", "q1_summary.json", "preprocessed_data.xlsx")]
    if any(p.resolve() in (args.input.resolve(), args.template.resolve()) for p in output_files):
        raise ValueError("输出文件与输入或模板重合, 请指定其他输出目录。")
    preprocessed_path = args.output / "preprocessed_data.xlsx"
    load, pv, price = read_data(args.input, preprocessed_path)
    solution = solve(load, pv, price)
    checks = validate(solution, load, pv)
    baseline = float(price @ np.maximum(load - pv, 0))
    summary = {
        "model": "用户截图中的纯购电费用目标MILP",
        "cost_yuan": solution["cost"], "purchase_kwh": float(solution["g"].sum()),
        "no_storage_cost_yuan": baseline, "saving_fraction": 1 - solution["cost"] / baseline,
        "charge_kwh": float(solution["c"].sum()), "discharge_kwh": float(solution["d"].sum()),
        "initial_soc_kwh": float(solution["s"][0]), "final_soc_kwh": float(solution["s"][-1]),
        "min_soc_kwh": float(solution["s"].min()), "max_soc_kwh": float(solution["s"].max()),
        "pv_curtailment_kwh": float(solution["r"].sum()),
        "mip_gap": solution["mip_gap"], "mip_dual_bound_yuan": solution["mip_dual_bound"],
        "solver_message": solution["solver_message"],
        "eta_charge": ETA_C, "eta_discharge": ETA_D, "checks": checks,
        "data_preprocessing": "00:00复制24:00瞬时值, 每10分钟取两端点算术平均",
        "preprocessed_file": str(preprocessed_path.resolve()),
        "numpy_version": np.__version__, "scipy_version": scipy.__version__,
        "input_sha256": hashlib.sha256(args.input.read_bytes()).hexdigest(),
    }
    write_outputs(args.output, args.template, solution, load, pv, price, summary)
    print(f"问题一购电费用: {solution['cost']:.2f} 元")
    print(f"全天购电量: {summary['purchase_kwh']:.4f} kWh")
    print(f"无储能费用: {baseline:.2f} 元; 节约: {summary['saving_fraction']:.2%}")
    print("物理约束检查: 通过")
    print(f"预处理数据: {preprocessed_path.resolve()}")
    print(f"弃光量: {summary['pv_curtailment_kwh']:.4f} kWh; MILP相对最优间隙: {summary['mip_gap']:.2e}")
    print(f"结果目录: {args.output.resolve()}")

```

```
if __name__ == "__main__":  
    main()
```

4. question2/planning/code/audit_window_results.py

源文件 SHA-256: 43e7ed462b717da52d1ee66d5e6b154805ebee5f052a9f6c3e704fa06d448384 ; 代码行数: 404。

```

"""Independent audit of frozen shifted-window Q2 results; never waits for a run.

The accounting group is (method, plan_date), not the calendar date of a slot.
Generate optional standard scientific figures only after all checks pass.
"""
from __future__ import annotations

import argparse
from collections import defaultdict
import csv
import datetime as dt
import hashlib
import json
import math
from pathlib import Path
from unittest.mock import patch

import numpy as np

from study import json_write
from window_control import execute_prefix

ROOT = Path(__file__).resolve().parents[1]
METHODS = ("A", "B", "C", "D")
TOL = 1e-4
PHYSICAL_TOL = 1e-5

def csv_rows(path):
    with Path(path).open(encoding="utf-8-sig", newline="") as f:
        yield from csv.DictReader(f)

def read_json(path):
    return json.loads(Path(path).read_text(encoding="utf-8"))

def boolean(x):
    if isinstance(x, bool):
        return x
    if str(x).lower() in ("true", "1"):
        return True
    if str(x).lower() in ("false", "0"):
        return False
    raise ValueError(f"Invalid boolean {x!r}")

def file_sha(path):
    h = hashlib.sha256()
    with Path(path).open("rb") as f:
        for block in iter(lambda: f.read(1024 * 1024), b""):
            h.update(block)
    return h.hexdigest()

def assignment_probe():
    """Runtime structural check, distinguished from stored vector evidence."""
    import window_study as ws
    g = np.arange(144, dtype=float) + .125
    data = {k: np.zeros((1, 144)) for k in ("load_a", "pv_a", "load_f", "pv_f")}
    pl = {"g": g, "checks": {"passed": True}, "mip_gap": 0., "status": "test_only"}
    captures = []

```



```

def capture(method, contract, *args):
    captures.append((method, contract is g, np.asarray(contract).copy()))
    return {}

with patch.object(ws, "DATA", data), patch.object(ws, "PRICE", np.ones(144)), \
    patch.object(ws, "CONFIG", {"initial_soc": 6000.}), \
    patch.object(ws, "plan", return_value=(pl, {})) as planner, \
    patch.object(ws, "execute", side_effect=capture), \
    patch.object(ws, "record", side_effect=lambda *args: {}):
    ws.local_job((0, "AB", 0))
return {"passed": planner.call_count == 1 and len(captures) == 2
        and [x[0] for x in captures] == ["A", "B"]
        and all(x[1] for x in captures)
        and np.array_equal(captures[0][2], captures[1][2]),
        "scope": "current local_job code passes the same g object/vector to A and B; not a
reconstruction of saved per-day vectors"}

def make_figures(out, daily, summary):
    """Each plot has one task: cost decomposition; window-month variation."""
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.rcParams.update({"font.size": 10, "axes.titlesize": 12,
                        "axes.labelsize": 10, "savefig.dpi": 220,
                        "axes.spines.top": False, "axes.spines.right": False})

    figdir = out / "figures"
    figdir.mkdir(exist_ok=True)
    planned = [sum(float(r["planned_cost_yuan"]) for r in daily if r["method"] == m) / 1e6 for m in
METHODS]
    emergency = [sum(float(r["emergency_cost_yuan"]) for r in daily if r["method"] == m) / 1e6 for m in
METHODS]
    fig, ax = plt.subplots(figsize=(7.6, 4.7), layout="constrained")
    x = np.arange(4)
    ax.bar(x, planned, color="#557F9E", width=.58, label="Ordinary fixed contract")
    ax.bar(x, emergency, bottom=planned, color="#CE9B68", width=.58, label="Emergency purchase")
    for i, value in enumerate(np.add(planned, emergency)):
        ax.annotate(f"{value:.3f}", (i, value), xytext=(0, 5), textcoords="offset points", ha="center")
    ax.set_xticks(x, METHODS)
    ax.set_ylabel("Cash cost (million CNY)")
    ax.set_title("Continuous operation: cost by component")
    ax.grid(axis="y", alpha=.2)
    ax.set_axisbelow(True)
    ax.margins(y=.16)
    ax.legend(frameon=False, loc="upper center", ncols=2)
    for suffix in ("png", "pdf"):
        fig.savefig(figdir / f"cost_components.{suffix}", bbox_inches="tight")
    plt.close(fig)
    months = sorted({r["window_start"][:7] for r in daily})
    colors = ("#557F9E", "#508B84", "#CE9B68", "#8A7894")
    fig, ax = plt.subplots(figsize=(8.2, 4.7), layout="constrained")
    for method, color in zip(METHODS, colors):
        values = [sum(float(r["cash_cost_yuan"]) for r in daily
                        if r["method"] == method and r["window_start"][:7] == month) / 1e6 for month in
months]
        ax.plot(np.arange(len(months)), values, marker="o", markersize=4,
                linewidth=1.6, color=color, label=method)
    ax.set_xticks(np.arange(len(months)), months, rotation=35, ha="right")
    ax.set_xlabel("Month of window start (00:10)")
    ax.set_ylabel("Window-month cash cost (million CNY)")
    ax.set_title("Monthly cash cost grouped by window start")
    ax.grid(alpha=.2)
    ax.legend(title="Method", ncols=4, frameon=False)
    for suffix in ("png", "pdf"):
        fig.savefig(figdir / f"monthly_window_cash.{suffix}", bbox_inches="tight")
    plt.close(fig)

```

```

    return {"files": [f"figures/{name}.{suffix}" for name in ("cost_components", "monthly_window_cash")
for suffix in ("png", "pdf")],
        "scope": "conditional continuous-run cash bills; first SOC prescribed; final interval partly
estimated; not calendar-month meter totals",
        "figure_tasks": ["Separate planned cost from emergency cost", "Show variation using the
month of each 00:10-start window"]}

def audit(out, figures=False):
    out = Path(out)
    if not (out / "summary.json").exists():
        raise SystemExit("结果尚未冻结: summary.json不存在。脚本不等待，也不审计未完成输出。")
    summary = read_json(out / "summary.json")
    frozen = read_json(out / "frozen_parameters.json")
    config = frozen["config"]
    errors, counts, maxima = [], defaultdict(int), defaultdict(float)

    def require(ok, detail):
        counts["assertions_checked"] += 1
        if not ok:
            counts["assertions_failed"] += 1
            if len(errors) < 80:
                errors.append(detail)

    def residual(name, value):
        require(math.isfinite(float(value)), f"Nonfinite residual {name}")
        maxima[name] = max(maxima[name], abs(float(value)))

    canonical = {}
    windows = {}
    estimated_keys = set()
    for r in csv_rows(out / "canonical_intervals.csv"):
        key = (r["plan_date"], r["interval_start"])
        require(key not in canonical, f"Duplicate canonical interval {key}")
        canonical[key] = (float(r["load_actual_kw"]), float(r["pv_actual_kw"]),
boolean(r["actual_estimated"]), r["interval_end"])
        if canonical[key][2]:
            estimated_keys.add(key)
        w = windows.setdefault(r["plan_date"], {"issued": r["issued_at"], "starts": []})
        w["starts"].append(r["interval_start"])
        require(w["issued"] == r["issued_at"], f"Mixed forecast issue times {r['plan_date']}")
    dates = sorted(windows)
    for day in dates:
        w = windows[day]
        start = dt.datetime.combine(dt.date.fromisoformat(day), dt.time(0, 10))
        w["start"], w["end"] = start, start + dt.timedelta(days=1)
        require(w["starts"] == [(start + dt.timedelta(minutes=10 * t)).isoformat() for t in range(144)],
            f"Canonical window is not 144 ordered 00:10-start slots: {day}")
        require(dt.datetime.fromisoformat(w["issued"]) ≤ start - dt.timedelta(minutes=10), f"Late issue
{day}")
        require(len(estimated_keys) == 1, f"Full-source expected exactly one estimated interval, got
{len(estimated_keys)}")
        if estimated_keys:
            expected_last_key = (dates[-1], (windows[dates[-1]]["end"] -
dt.timedelta(minutes=10)).isoformat())
            require(estimated_keys == {expected_last_key}, "Estimated mask is not restricted to the final
interval")
        daily = list(csv_rows(out / "stage3_continuous_daily.csv"))
        daily_map = {(r["method"], r["date"]): r for r in daily}
        require(len(daily_map) == len(daily) == len(dates) * 4, "Duplicate or missing continuous daily
rows")
        ledger = {}
        last, final_rows, estimate_counts = {}, {}, defaultdict(int)
        for r in csv_rows(out / "stage3_interval_trace.csv"):
            counts["trace_intervals"] += 1
            method, day = r["method"], r["plan_date"]

```

```

        require(method in METHODS and day in windows, f"Unknown method/window {method} {day}")
        key = (method, day)
        z = ledger.setdefault(key, {"g": [], "p": [], "cash": 0., "planned": 0., "emergency": 0.,
                                     "emergency_kwh": 0., "estimated_emergency": 0., "s0":
float(r["soc_start_kwh"])}))
        t = len(z["g"])
        a, end = dt.datetime.fromisoformat(r["interval_start"]),
dt.datetime.fromisoformat(r["interval_end"])
        require(t < 144 and a == windows[day]["start"] + dt.timedelta(minutes=10 * t)
               and end - a == dt.timedelta(minutes=10), f"Wrong interval order/window grouping {key} t={t}")

        source_key = (day, r["interval_start"])
        require(source_key in canonical, f"Missing canonical source {source_key}")
        load_kw, pv_kw = float(r["load_actual_kw"]), float(r["pv_actual_kw"])
        ca = canonical[source_key]
        residual("source_load_kw", load_kw - ca[0]); residual("source_pv_kw", pv_kw - ca[1])
        estimated = boolean(r["actual_estimated"])
        require(estimated == ca[2], f"Source estimate mask changed {key} t={t}")
        estimate_counts[method] += int(estimated)
        g, c, d, e, v, dump, p, s0, s1, cash = [float(r[f]) for f in (
            "g_kwh", "charge_kwh", "discharge_kwh", "emergency_kwh", "unused_paid_kwh",
            "curtailed_pv_kwh", "price_yuan_per_kwh", "soc_start_kwh", "soc_end_kwh", "cash_cost_yuan")]
        require(all(math.isfinite(x) for x in (g, c, d, e, v, dump, p, s0, s1, cash)), f"Nonfinite trace
values {key} t={t}")
        load, pv = load_kw / 6., pv_kw / 6.
        surplus, deficit = max(g + pv - load, 0.), max(load - g - pv, 0.)
        residual("energy_balance_kwh", g - v + pv - dump + d + e - load - c)
        residual("soc_transition_kwh", s1 - s0 - .9 * c + d / .9)
        residual("slot_cash_yuan", cash - p * g - 5. * p * e)
        residual("physical_bound_violation", max(0., 1200. - s0, 1200. - s1, s0 - 10800., s1 - 10800.,
        c - 5000. / 6., d - 5000. / 6., c - surplus, d - deficit, abs(e - deficit + d),
        v - g, dump - pv, -min(g, c, d, e, v, dump), min(c, d), min(c, e)))
        if method in last:
            residual("cross_slot_and_window_soc_kwh", s0 - last[method][1])
            require(a == last[method][0], f"Time gap/overlap across windows for {method}: {a}")
        last[method] = (end, s1)
        z["g"].append(g); z["p"].append(p)
        z["cash"] += cash; z["planned"] += p * g; z["emergency"] += 5. * p * e
        z["emergency_kwh"] += e; z["estimated_emergency"] += 5. * p * e * estimated
        z["s1"] = s1
        if t == 143:
            z["tail_s0"] = s0
        final_rows[method] = r
        require(set(ledger) == set(daily_map), "Trace windows differ from daily-table windows")
        for key, z in ledger.items():
            r = daily_map[key]
            require(len(z["g"]) == 144, f"Window {key} does not contain 144 slots")
            for expected, actual, label in ((z["cash"], r["cash_cost_yuan"], "window_cash_yuan"),
            (z["planned"], r["planned_cost_yuan"], "window_planned_yuan"),
            (z["emergency"], r["emergency_cost_yuan"], "window_emergency_yuan"),
            (z["emergency_kwh"], r["emergency_kwh"], "window_emergency_kwh"),
            (z["s0"], r["initial_soc_kwh"], "window_initial_soc_kwh"),
            (z["s1"], r["final_soc_kwh"], "window_final_soc_kwh"),
            (z["estimated_emergency"], r["estimated_interval_emergency_yuan"],
"estimated_emergency_yuan")):
                residual(label, expected - float(actual))
            adjusted = z["cash"] - config["inventory_value"] * (z["s1"] - z["s0"])
            residual("inventory_adjusted_cost_yuan", adjusted - float(r["inventory_adjusted_cost_yuan"]))
            residual("known_bill_decomposition_yuan", z["cash"] - z["estimated_emergency"] -
float(r["known_contract_plus_observed_emergency_yuan"]))
            for method in METHODS:
                require(estimate_counts[method] == 1, f"{method} does not have exactly one estimated interval")
                logs = read_json(out / "continuous" / f"{method}_solver.json")
                require(len(logs) == len(dates), f"Wrong contract-log count for {method}")
                for i, log in enumerate(logs):
                    day = dates[i]; z = ledger[(method, day)]

```

```

require(log["date"] == day, f"Wrong contract-log date {method} {i}")
residual("new_contract_total_kwh", float(log["new_contract_kwh"]) - sum(z["g"]))
residual("new_contract_cost_yuan", float(log["new_contract_cost"]) - z["planned"])
residual("executed_window_start_soc_kwh", float(log["executed_soc_at_window_start"]) -
z["s0"])
    if i:
        previous = ledger[(method, dates[i - 1])]
        residual("fixed_prefix_contract_kwh", float(log["fixed_prefix_kwh"]) - previous["g"]
[-1])
        residual("midnight_before_old_tail_soc_kwh", float(log["midnight_observed_soc"]) -
float(previous["tail_s0"]))
    else:
        require(log["fixed_prefix_kwh"] is None, f"Unexpected prefix for first {method}
contract")
    for report in summary["periods"]:
        if report["phase"] not in ("continuous_all", "continuous_test"):
            continue
        chosen = [r for r in daily if r["method"] == report["method"]
and (report["phase"] == "continuous_all" or r["stage"] == "test")]
        require(len(chosen) == report["days"], "Summary day count differs from daily table")
        for field in ("cash_cost_yuan", "planned_cost_yuan", "emergency_cost_yuan",
"inventory_adjusted_cost_yuan"):
            residual("summary_" + field, sum(float(r[field]) for r in chosen) - float(report[field]))
# A/B: inspect all persisted comparable quantities and any contract hashes.
ab = list(csv_rows(out / "stage1_same_contract_daily.csv"))
abmap = {(r["method"], r["date"]): r for r in ab}
abdates = sorted({r["date"] for r in ab})
hash_pairs = 0
for day in abdates:
    a, b = abmap[("A", day)], abmap[("B", day)]
    for field in ("initial_soc_kwh", "planned_kwh", "planned_cost_yuan"):
        residual("AB_same_contract_" + field, float(a[field]) - float(b[field]))
    hashfield = next((f for f in ("fixed_contract_sha256", "contract_sha256", "g_sha256",
"contract_vector_sha256") if a.get(f) and b.get(f)), None)
    if hashfield:
        require(a[hashfield] == b[hashfield], f"A/B contract digest mismatch {day}")
    hash_pairs += 1
probe = assignment_probe()
require(probe["passed"], "A/B same-vector assignment probe failed")
# Every recorded history set and scenario draw must end by its issue time.
history_sets = 0
solver_status_counts = defaultdict(int)
gaps = []
def walk(value):
    nonlocal history_sets
    if isinstance(value, dict):
        if "history_indices" in value:
            history_sets += 1
            issued = dt.datetime.fromisoformat(value["issued_at"])
            history = value["history_indices"]
            expected = [j for j, day in enumerate(dates) if windows[day]["end"] ≤ issued][-
config["window"]:]
            require(history == expected, f"History differs from completed trailing-window rule at
{issued}")
            for j in history:
                require(0 ≤ j < len(dates) and windows[dates[j]]["end"] ≤ issued,
f"History leaks incomplete window {j} at {issued}")
            for j in value.get("scenario_history_indices", []):
                require(j in history, f"Scenario uses unapproved history {j} at {issued}")
        if "status" in value and "gap" in value and "checks" in value:
            solver_status_counts[value["status"]] += 1
            require(bool(value["checks"]["passed"]), "Recorded planner checks failed")
            if value["gap"] is not None:
                gaps.append(float(value["gap"]))
        for child in value.values():
            walk(child)

```

```

        elif isinstance(value, list):
            for child in value:
                walk(child)
    solver_files = sorted(out.rglob("*_solver.json")) + [out / "local_solver_audit.json"]
    for path in solver_files:
        walk(read_json(path))
    effective = dt.datetime.combine(dt.date.fromisoformat(frozen["effective_from"]), dt.time())
    for day in frozen["calibration_dates"]:
        require(day in windows and windows[day]["end"] ≤ effective,
            f"Calibration target not completed before frozen parameters become effective: {day}")
    calibration = list(csv_rows(out / "calibration_quantiles.csv"))
    for r in calibration:
        require(r["date"] in frozen["calibration_dates"], "Unexpected calibration date")
        issued = dt.datetime.fromisoformat(windows[r["date"]]["issued"])
        expected = [j for j, day in enumerate(dates) if windows[day]["end"] ≤ issued][-1]
    config["window"]:]
        require(int(r["historical_residual_days"]) == len(expected), "Calibration history-count mismatch")
        scores = {q: sum(float(r["inventory_adjusted_cost_yuan"]) for r in calibration if float(r["quantile"]) == q)
            for q in sorted({float(r["quantile"]) for r in calibration})}
        require(bool(scores), "Empty calibration records")
    if scores:
        require(float(frozen["selected_quantile"]) == min(scores, key=lambda q: (scores[q], q)), "Frozen quantile does not reproduce calibration-only choice")
    # Recompute each final-boundary alternative with identical g and prefix SOC.
    boundary_rows = list(csv_rows(out / "final_interval_sensitivity.csv"))
    source_candidates = read_json(out / "input_audit.json")["last_boundary_candidates"]
    candidate_names = set(source_candidates)
    require(len(boundary_rows) == 4 * len(candidate_names), "Missing final-boundary candidate rows")
    require({(r["method"], r["boundary_method"]) for r in boundary_rows}
        == {(method, name) for method in METHODS for name in candidate_names},
        "Missing/duplicate final-boundary method combination")
    boundary_checks = []
    for r in boundary_rows:
        method = r["method"]; lastrow = final_rows[method]
        for field in ("load_mean_kw", "pv_mean_kw", "load_right_endpoint_kw", "pv_right_endpoint_kw"):
            residual("boundary_source_" + field, float(r[field]) -
                float(source_candidates[r["boundary_method"]][field]))
        g = float(lastrow["g_kwh"]); p = float(lastrow["price_yuan_per_kwh"])
        s0 = float(lastrow["soc_start_kwh"])
        op = execute_prefix("greedy" if method == "A" else "mpc", [g,
            [float(r["load_mean_kw"]) / 6., [float(r["pv_mean_kw"]) / 6.],
            [float(lastrow["load_forecast_kw"]) / 6.], [float(lastrow["pv_forecast_kw"]) / 6.],
            [p], s0, 1, terminal_weight=config["terminal_weight"], s_ref=6000.)
        require(op["checks"]["passed"] and op["g"][0] == g, "Boundary replay changed contract or violated physics")
        base_total = sum(z["cash"] for (m, _), z in ledger.items() if m == method)
        expected_total = base_total - float(lastrow["cash_cost_yuan"]) + op["fees_total"]
        for expected, stored, field in ((op["fees_total"], r["last_interval_cash_yuan"],
            "boundary_cash_yuan"),
            (op["fees_emergency"], r["last_interval_emergency_yuan"], "boundary_emergency_yuan"),
            (op["s"][-1], r["final_soc_kwh"], "boundary_final_soc_kwh"),
            (expected_total, r["total_cash_yuan"], "boundary_total_cost_yuan")):
            residual(field, expected - float(stored))
        boundary_checks.append({"method": method, "candidate": r["boundary_method"], "fixed_g_kwh": g,
            "unchanged_last_step_initial_soc_kwh": s0,
            "total_cost_change_yuan": expected_total - base_total})
    for field, value in maxima.items():
        tolerance = TOL if field.endswith("_yuan") else PHYSICAL_TOL
        require(value ≤ tolerance, f"Residual {field}={value} exceeds {tolerance}")
    result = {
        "audit_passed": not errors and counts["assertions_failed"] == 0,
        "scope": "frozen shifted-window outputs grouped by plan_date; independent cash, SOC, timing and terminal-boundary audit",
        "cash_tolerance_yuan": TOL, "physical_tolerance_kwh": PHYSICAL_TOL,

```



```

        "counts": dict(counts), "windows_per_method": len(dates),
        "max_absolute_residuals": dict(maxima), "estimated_intervals_per_method": dict(estimate_counts),
        "AB_evidence": {"paired_days": len(abdates), "persisted_digest_pairs": hash_pairs,
            "same_assignment_probe": probe,
            "limitation": None if hash_pairs == len(abdates) else "Saved stage1 rows lack per-slot
contract vectors/digests; equality of totals is supplemented by a runtime same-vector code probe, not
misrepresented as persisted vector verification."},
        "recorded_history_sets_checked": history_sets,
        "calibration_history_scope": "recorded date/count plus reconstruction of completed-window rule;
per-calibration scenario arrays not stored",
        "solver_status_counts": dict(solver_status_counts), "maximum_recorded_gap": max(gaps) if gaps
else None,
        "boundary_recomputation": boundary_checks,
        "limitations": ["Forecast-generation causality remains conditional on provided forecast
provenance.",
            "Current interval means are a fast-feedback approximation.",
            "Instantaneous-node trapezoidal means are approximations; final right endpoint
is estimated.",
            "Initial SOC is a user-prescribed experiment condition; this is not an
unconditional competition optimum."],
        "errors": errors,
        "source_hashes": {name: file_sha(out / name) for name in ("summary.json",
"canonical_intervals.csv", "stage3_interval_trace.csv", "frozen_parameters.json")},
        "audit_script_sha256": file_sha(Path(__file__)),
    }
    if result["audit_passed"] and figures:
        result["figures"] = make_figures(out, daily, summary)
    json_write(out / "window_result_audit.json", result)
    lines = ["# 第二窗口结果独立审计", "", f"审计状态: {'通过' if result['audit_passed'] else '未通过'}。",
        f"按计划日期分组, 核对每方案{len(dates)}个窗口、每窗口144段; 共核对{counts['trace_intervals']:,}条执行记
录。",
        "", "已核对: 区间能量守恒、充放电与储电边界、跨午夜SOC连接、固定旧末段合同、新窗口计划费用与逐段账本、历史误差截
止时刻、校准日期、最后边界估计。",
        f"每方案估计实际量仅{len(estimated_keys)}段; 边界替代法以相同固定购电量和末段初始SOC重算, 完整回放费用变化恰
为该末段变化。",
        f"全部数值检查的最大绝对残差为{max(maxima.values(), default=0.):.6g}; 混合单位的各项残差详见JSON。",
        "", "A/B同合同证据: 所有配对初态、总购电量与计划费一致; 执行函数同向量传参测试通过。" + ("并已核对每个日期的合
同摘要。" if hash_pairs == len(abdates) else "现有阶段1表未保存逐段合同或摘要, 不能仅凭总量相等宣称已独立逐段核
对。"),
        "", "这份审计证明所列结果与给定口径、物理约束和账本一致, 不证明预测来源完全无信息泄漏, 也不把时限内可行解称为已
证明最优。"]
    if errors:
        lines += ["", "发现的问题: "] + ["- " + x for x in errors]
    if "figures" in result:
        lines += ["", "图1拆分计划费与紧急费; 图2按窗口起点月份汇总费用。跨年最后一段归属12月31日计划窗口, 不按其自然
日重新分组。"]
    (out / "独立审计说明.md").write_text("\n".join(lines) + "\n", encoding="utf-8")
    if not result["audit_passed"]:
        raise AssertionError(f"Audit failed; inspect window_result_audit.json: {errors[:5]}")
    print(json.dumps({"audit_passed": True, "windows_per_method": len(dates),
        "trace_intervals": counts["trace_intervals"], "figures": bool(figures)},
ensure_ascii=False), flush=True)
    return result

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("result_directory", type=Path, nargs="?", default=ROOT / "results/final_team")
    parser.add_argument("--figures", action="store_true")
    args = parser.parse_args()
    audit(args.result_directory, args.figures)

```

5. question2/planning/code/build_conclusion.py

源文件 SHA-256: 6760927fb96994d8cf84dc6fe3160f4c63f57905b20d787c9951f879792c293b ; 代码行数: 151。

```

"""Generate a traceable Chinese interpretation from the frozen Q2 ledgers."""
import argparse
import csv
import json
from pathlib import Path
import numpy as np
from evaluate_reports import paired_block_interval

ROOT = Path(__file__).resolve().parents[1]
NAMES = {'A': '点预测MILP+贪心', 'B': '点预测MILP+MPC',
         'C': '误差裕量MILP+MPC', 'D': '场景随机MILP+MPC'}

def read_rows(path):
    with path.open(encoding='utf-8-sig', newline='') as f:
        return list(csv.DictReader(f))

def build(out):
    summary = json.loads((out / 'summary.json').read_text())
    frozen = json.loads((out / 'frozen_parameters.json').read_text())
    config = frozen['config']
    audit = json.loads((out / 'window_result_audit.json').read_text())
    ledger = json.loads((out / 'independent_evaluation.json').read_text())
    report = json.loads((out / '中文报表/报表核验.json').read_text())
    assert audit['audit_passed'] and ledger['independent_checks_passed'] and report['all_checks_passed']
    all_rows = {x['method']: x for x in summary['periods'] if x['phase'] == 'continuous_all'}
    test_rows = {x['method']: x for x in summary['periods'] if x['phase'] == 'continuous_test'}
    daily = read_rows(out / 'stage3_continuous_daily.csv')
    best = min(all_rows, key=lambda m: all_rows[m]['cash_cost_yuan'])
    inventory_best = min(all_rows, key=lambda m: all_rows[m]['inventory_adjusted_cost_yuan'])
    saving = all_rows['A']['cash_cost_yuan'] - all_rows[best]['cash_cost_yuan']
    text = ['## 第二问: 新144段输入的计算结论', '',
           f'在本次队伍预测、共同6000 kWh实验初态和统一物理条件下, 334窗口连续回放现金费用最低的是**方案{best} ({NAMES[best]}) **.',
           f'相对A累计少支出{saving:,.2f}元 ({100*saving/all_rows["A"]["cash_cost_yuan"]:.2f}%)。这是当前输入与预设参数下的历史比较, 不代表所有预测版本或参数下都最优。', '',
           '输入为2025年2月1日至12月31日的334行预测; 每行覆盖本日00:10至次日00:10, 共48,096段。完整执行结束于2026年1月1日00:10, 不是全年自然日账单。', '',
           '## 1. 连续运行总费用', '',
           '|方案|普通购电费/元|紧急购电费/元|总现金费用/元|最终储电/kWh|库存修正指标/元|',
           '|-----|-----|-----|-----|-----|',
           for m, r in all_rows.items():
               text.append(f'|{m}| {NAMES[m]}| {r["planned_cost_yuan"]:.2f}| {r["emergency_cost_yuan"]:.2f}| {r["cash_cost_yuan"]:.2f}| {r["final_soc_kwh"]:.2f}| {r["inventory_adjusted_cost_yuan"]:.2f}|')
           months = sorted({r['window_start'][:7] for r in daily})
           monthly = {month: {m: sum(float(r['cash_cost_yuan']) for r in daily
                                   if r['method'] == m and r['window_start'].startswith(month)) for m in NAMES}
                      for month in months}
           monthly_winners = {month: min(costs, key=costs.get) for month, costs in monthly.items()}
           best_months = sum(m == best for m in monthly_winners.values())
           exceptions = ','.join(f'{month}为{method}' for month, method in monthly_winners.items() if method != best)
           text += ['', f'按窗口起点月份分组, 方案{best}在{len(months)}个月中的{best_months}个月现金费用最低。' + (f'其他月份: {exceptions}。' if exceptions else ''),
                   '各月比较承接各自既有电池状态, 不代表每个月重新从相同SOC开始; 逐月图见独立审计的figures目录。',
                   f'库存修正为现金费用-{config["inventory_value"]:.10f}×(末储电-初储电), 只用于比较库存消耗, 绝不当作退款。实际费用包含已签普通电与五倍价格紧急电; 未用普通电不退款, 软末态惩罚不计费。',
                   f'按库存修正指标最低的是方案{inventory_best}; 现金排名和库存修正排名需结合末态储电量一起判断。', '',
                   '最后一段的右端瞬时节点缺失, 主结果以持平估计补齐; 其余48,095段均有左右实测节点。所有实际区间均值仍采用端点平均近似, 最后一段的估计标记和替代估计结果完整保留。', '',
                   '## 2. 按顺序区分控制作用与日前规划作用', '',
                   '|对照|相同条件|比较窗口数|累计现金减少/元|累计库存修正减少/元|候选现金更低窗口|平均每窗现金差/元及95%块Bootstrap区间|',

```



```

'|———:——:——:——:|']
comparisons = [('stage1_same_contract_daily.csv', 'A', 'B', '同一合同g、同6000初态'),
                ('stage2_dayahead_daily.csv', 'B', 'C', '同MPC、同6000初态'),
                ('stage2_dayahead_daily.csv', 'B', 'D', '同MPC、同6000初态'),
                ('stage2_dayahead_daily.csv', 'C', 'D', '同MPC、同6000初态')]
for file, base, candidate, condition in comparisons:
    rows = read_rows(out / file)
    ref = {x['date']: x for x in rows if x['method'] == base}
    cand = sorted([x for x in rows if x['method'] == candidate], key=lambda x: x['date'])
    if len(cand) != len({x['date'] for x in cand}) or set(ref) != {x['date'] for x in cand}:
        raise AssertionError('Paired comparison dates are missing or duplicated')
    for x in cand:
        baseline = ref[x['date']]
        if abs(float(x['initial_soc_kwh']) - float(baseline['initial_soc_kwh'])) > 1e-6:
            raise AssertionError('Local comparison does not share the same initial SOC')
        if file.startswith('stage1'):
            for key in ('planned_cost_yuan', 'planned_kwh'):
                if abs(float(x[key]) - float(baseline[key])) > 1e-5:
                    raise AssertionError('A/B declared same-contract totals differ')
        diff = np.array([float(x['cash_cost_yuan']) - float(ref[x['date']]['cash_cost_yuan']) for x in
cand])
        inventory_diff = np.array([float(x['inventory_adjusted_cost_yuan']) - float(ref[x['date']]
['inventory_adjusted_cost_yuan']) for x in cand])
        ci = paired_block_interval(diff)
        ci_label = '样本不足' if ci is None else f'[{ci[0]:, .2f}, {ci[1]:, .2f}]'
        text.append(f'|{candidate}相对{base}|{condition}|{len(diff)}|{-diff.sum():, .2f}|{-
inventory_diff.sum():, .2f}|{(diff < -1e-5).sum()}/{len(diff)}|{diff.mean():, .2f}; {ci_label}|')
        text += ['|', '上表“累计减少”为基线-候选，正值表示候选更省；“平均每窗差”为候选-基线，负值表示候选更省。库存修正列用
于检查现金改善是否部分来自消耗更多末态库存。',
                'A/B的逐段同合同由主程序把同一次日前求解的g分别交给两种控制器保证；购电总量与费用相同是汇总防错检查，不能单独代
替时段级同合同条件。',
                'Bootstrap使用7窗口圆形移动块、1000次重采样和固定种子，给出配对平均现金差的条件性95%区间；它是描述性检查，未作
多重比较校正，不能消除季节变化或保证未来收益。逐窗实验每窗重置初态，只用来区分升级的作用，其费用之和不能冒充连续运行账
单。',
                'MPC根据后续预测和电价尝试把部分电量留给后面更贵的缺口；裕量和场景方案可能增加或重新分配普通购电，以改变紧急购电
风险。是否值得，要用执行后的两种费用总和判断。即使D累计更省，也不能据此推断每个窗口都更省，具体窗口胜出数见表。',
                '## 3. 参数冻结后的时间顺序测试期',
                f'前{config["calibration_days"]}窗是热身/校准；测试合同从{summary["test_dates"][0]}开始，每方案
{test_rows["A"]["days"]}窗。四方案测试起点SOC来自各自此前运行；该表衡量各自历史运行后继续执行的条件表现，不能单独作为
同起始资源的控制器对照，应结合上面的同初态局部实验。',
                '|方案|测试期现金/元|测试起始SOC/kWh|测试末SOC/kWh|测试期库存修正指标/元|', '|———:——:——:——:|']
        for m, r in test_rows.items():
            text.append(f'|{m}|{r["cash_cost_yuan"]: ,.2f}|{r["initial_soc_kwh"]: ,.2f}|
{r["final_soc_kwh"]: ,.2f}|{r["inventory_adjusted_cost_yuan"]: ,.2f}|')
            calibration_dates = frozen['calibration_dates']
            calibration_scope = f'{calibration_dates[0]}至{calibration_dates[-1]}的{len(calibration_dates)}个已完
成窗口' if calibration_dates else '没有足够的已完成窗口，采用预设值'
            text += ['|', f'裕量分位数校准范围为{calibration_scope}，使用q={summary["selected_quantile"]}；自
{frozen["effective_from"]}起冻结，代码没有使用测试期实际费用选择该参数。上一窗口到决策日00:10才结束，故不能用于决策日
零点的参数选择。',
                    '这只保证本规划实验的时间切分与参数筛选顺序，不等于已独立审计原预测器的训练截止时间；最终比较与方法建议也属于看完
这批回放后的历史评价。',
                    '|裕量q|校准现金/元|校准库存修正指标/元|校准紧急费/元|', '|———:——:——:——:|']
            calibration = read_rows(out / 'calibration_quantiles.csv')
            for q in (0, .5, .8, .9):
                rows = [r for r in calibration if float(r['quantile']) == q]
                sums = [sum(float(r[k]) for r in rows) for k in
('cash_cost_yuan', 'inventory_adjusted_cost_yuan', 'emergency_cost_yuan')]
                text.append(f'|{q:g}|{sums[0]: ,.2f}|{sums[1]: ,.2f}|{sums[2]: ,.2f}|')
                text += ['|', f'更大裕量不一定更好：它可能降低紧急费，也可能买下更多用不掉且不退款的合同电。本次只有
{len(calibration_dates)}个有效校准窗口，所选分位数是当前实验候选，不能称为跨季节最优参数。',
                        '## 4. 初态与末端敏感性',
                        '三种初态均独立运行相同的前28窗口，使用相同预测、冻结参数与最后一段结束规则。6000基准没有从334窗主回放直接截
取，以避免其多看到第29窗合同造成不公平。',
                        '|初态/kWh|A现金/元|B现金/元|C现金/元|D现金/元|本组最低|', '|——:——:——:——:——:|']
            sensitivity = read_rows(out / 'initial_soc_sensitivity_daily.csv')

```

```

for s0 in (1200., 6000., 10800.):
    costs = {m: sum(float(r['cash_cost_yuan']) for r in sensitivity if r['method']==m and
float(r['experiment_initial_soc'])==s0) for m in NAMES}
    text.append(f'|{s0:,.0f}|' + '|'.join(f'{costs[m]:,.2f}' for m in NAMES) + f'|
{min(costs, key=costs.get)}|')
    text += [' ', '不同初态拥有不同数量的初始电，不能把跨初态的现金差直接当作算法优劣；应在同一初态内比较方案。这项检验
覆盖28窗，不声称已经做三种初态的334窗全程实验。', ' ',
    '最后缺失端点用持平、末两点线性外推、末六点OLS外推三种方法，只重放末段，之前合同与动作完全固定。', ' ',
    '|方案|三种估计下总现金最小/元|最大/元|跨度/元|', '|———:|——:|——:|']
    boundary = read_rows(out / 'final_interval_sensitivity.csv')
    for m in NAMES:
        values = [float(r['total_cash_yuan']) for r in boundary if r['method']==m]
        text.append(f'|{m}|{min(values):,.2f}|{max(values):,.2f}|{max(values)-min(values):,.2f}|')
        text += [' ', '这些端点估计用于说明最后一段对结论的影响，不能称为已获得缺失的实测数据，也没有据结果反选最便宜的填充
值。', ' ',
        '## 5. 求解精度与运行代价', ' ',
        '|方案|日前平均求解/秒|日内全部LP求解/秒|未证达到目标Gap的窗口|最大相对Gap|', '|———:|——:|——:|——:|
-:|']
    for m in NAMES:
        rows = [r for r in daily if r['method']==m]
        secs = np.mean([float(r['dayahead_solve_seconds']) for r in rows])
        control = sum(float(r['controller_solve_seconds']) for r in rows)
        limited = sum(r['dayahead_solver_status'] != 'optimal_within_requested_gap' for r in rows)
        reported_gaps = [float(r['dayahead_mip_gap']) for r in rows if r['dayahead_mip_gap'] not in ('',
None)]
        gap_label = f'{100*max(reported_gaps):.6f}%' if reported_gaps else '未报告'
        if len(reported_gaps) < len(rows):
            gap_label += f': {len(rows)-len(reported_gaps)}窗未报告'
        text.append(f'|{m}|{secs:.4f}|{control:.2f}|{limited}/{len(rows)}|{gap_label}|')
        text += [' ', '上表统计连续回放。局部规划对照也可能含限时可行解，其状态另外列出：', ' ',
        '|局部实验|方案|未证达到请求Gap窗口数|总窗口数|', '|———:|——:|']
        for filename, label in [('stage1_same_contract_daily.csv', '同g控制对照'),
('stage2_dayahead_daily.csv', '同MPC规划对照')]:
            local_rows = read_rows(out / filename)
            for method in sorted({r['method'] for r in local_rows}):
                group = [r for r in local_rows if r['method'] == method]
                text.append(f'|{label}|{method}|{sum(r["dayahead_solver_status"] !=
"optimal_within_requested_gap" for r in group)}|{len(group)}|')
                text += [' ', f'求解时间为本机记录，受并行负载影响，不是普遍速度承诺。日前先MILP、再固定整数模式进行LP精修，两个阶
段各限{config["milp_seconds"]}:g秒；请求相对Gap为{100*config["mip_gap"]:.6f}%，最终Gap使用原MILP的全局下界。限时
可行解通过物理检验后用于回放，但不能称为已证明全局最优；LP精修也不等于重新完成了全局整数搜索。', ' ',
                '独立程序核对四方案共192,384条逐段记录的能量守恒、SOC转移、功率和容量边界、费用、跨零点合同衔接及历史截止时刻。
工作簿另核对逐段-逐窗-汇总一致性。机器可读证据见独立审计JSON和中文报表核验文件。', ' ',
                '## 6. 建议与适用边界', ' ',
                f'当前预测版本若以累计购电现金费用为主要目标，可将{best}作为本轮优先候选，但这是本批结果的事后筛选。应保留B作为
点预测控制基线；C避免D的多场景整数模型，可作为相对D较简洁的风险处理候选，是否取用仍看本表费用、库存和求解记录。', ' ',
                'D使用预设8场景和单个固定种子，两阶段场景补救具有整条路径预见性，实际收益必须看MPC回放。尚不能声称已完成场景数、
随机种子、风险偏好和所有预测版本的全面优化。', ' ',
                '本次只评估规划模块。预测工作簿没有训练截止或版本日志，其零点前可用基于队伍说明；今后更换预测模型，应重建对应历史
误差库，在相同窗口、物理条件、实验初态及切分下重新比较A-D，不能把当前费用或胜负直接沿用。', ' ',
                '操作步骤、完整模型和标准格式见README及support/模型与执行步骤.md。中文汇总位于results/final_team/中文报表/
问题二_计算结果.xlsx，四份策略工作簿及逐段CSV在同一目录。', ' ']
            target = ROOT / 'support/计算结论.md'
            target.parent.mkdir(exist_ok=True)
            target.write_text('\n'.join(text), encoding='utf-8')
            print(target)
            return target

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('results', type=Path, nargs='?', default=ROOT/'results/final_team')
    build(parser.parse_args().results)

```

6. question2/planning/code/build_window_report.py

源文件 SHA-256: dd8a2788d3596c0661f9230d15b173e23cd8052318c9f9eb74526323b7f77a95 ; 代码行数: 629。

```
"""Build checked Chinese workbooks from a completed supplied-data window run.
```

```
Run only after window_study.py has finished and summary.json exists. This
builder never fits a model, changes a contract, or fabricates missing outputs.
Large interval traces are streamed one method at a time. They are copied as CSV
companions; purchase workbooks use openpyxl's write-only mode.
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import csv
```

```
import datetime as dt
```

```
import hashlib
```

```
import json
```

```
import math
```

```
from pathlib import Path
```

```
import shutil
```

```
from time import perf_counter
```

```
from openpyxl import Workbook, load_workbook
```

```
from openpyxl.cell import WriteOnlyCell
```

```
from openpyxl.styles import Alignment, Font, PatternFill
```

```
from openpyxl.utils import get_column_letter
```

```
ROOT = Path(__file__).resolve().parents[1]
```

```
MONEY_TOL = 1e-4
```

```
ENERGY_TOL = 1e-5
```

```
NAVY = "17365D"
```

```
TEAL = "176B87"
```

```
PALE = "EEF5FA"
```

```
AMBER = "FFF2CC"
```

```
WHITE = "FFFFFF"
```

```
ERROR_VALUES = {"#VALUE!", "#DIV/0!", "#REF!", "#NAME?", "#NULL!", "#NUM!", "#N/A"}
```

```
METHODS = {
```

```
    "A": "点预测MILP+贪心电池控制",
```

```
    "B": "点预测MILP+改进滚动LP控制",
```

```
    "C": "净负载分位数裕量MILP+同一滚动控制",
```

```
    "D": "成对历史残差场景随机MILP+同一滚动控制",
```

```
}
```

```
PHASES = {
```

```
    "same_contract_local": "同购电计划A/B: 各窗独立同初态",
```

```
    "dayahead_local": "同滚动控制B/C/D: 各窗独立同初态",
```

```
    "continuous_all": "连续执行: 全部334个窗口",
```

```
    "continuous_test": "连续执行: 参数冻结后的测试窗口",
```

```
}
```

```
SUM_FIELDS = (
```

```
    "cash_cost_yuan", "planned_cost_yuan", "emergency_cost_yuan",
```

```
    "inventory_adjusted_cost_yuan", "emergency_kwh", "unused_paid_kwh", "curtailed_pv_kwh",
```

```
)
```

```
DAILY_COLUMNS = [
```

```
    ("date", "窗口归属日期"), ("method", "方案"), ("stage", "阶段"),
```

```
    ("window_start", "窗口开始"), ("window_end", "窗口结束"),
```

```
    ("planned_cost_yuan", "计划购电费(元)", ("emergency_cost_yuan", "紧急购电费(元)"),
```

```
    ("cash_cost_yuan", "现金总费用(元)", ("inventory_adjusted_cost_yuan", "库存修正比较费用(元)"),
```

```
    ("planned_kwh", "计划购电量(kWh)", ("emergency_kwh", "紧急购电量(kWh)"),
```

```
    ("initial_soc_kwh", "窗口开始储电量(kWh)", ("final_soc_kwh", "窗口结束储电量(kWh)"),
```

```
    ("unused_paid_kwh", "未用已付合同电量(kWh)", ("curtailed_pv_kwh", "弃光量(kWh)"),
```

```
    ("emergency_intervals", "紧急购电时段数"), ("historical_residual_days", "可用完整残差窗口数"),
```

```
    ("fallback_to_point", "是否回退点预测"), ("dayahead_solver_status", "日前求解状态"),
```

```
    ("dayahead_mip_gap", "日前相对Gap"), ("actual_estimated_intervals", "含估计端点时段数"),
```

```
    ("estimated_interval_emergency_yuan", "估计端点时段紧急费(元)"),
```

```
]
```

```

SUMMARY_COLUMNS = [
    ("phase_label", "比较范围"), ("method", "方案"), ("method_name", "模型与控制器"), ("days", "窗口数"),
    ("planned_cost_yuan", "计划购电费(元)", ("emergency_cost_yuan", "紧急购电费(元)",
    ("cash_cost_yuan", "现金总费用(元)", ("inventory_adjusted_cost_yuan", "库存修正比较费用(元)",
    ("emergency_kwh", "紧急购电量(kWh)", ("unused_paid_kwh", "未用合同电量(kWh)",
    ("curtailed_pv_kwh", "弃光量(kWh)", ("p95_daily_cash_cost_yuan", "窗口费用95%分位数(元)",
    ("maximum_daily_cash_cost_yuan", "最高单窗费用(元)", ("initial_soc_kwh", "首窗初态(kWh)",
    ("final_soc_kwh", "末窗终态(kWh)", ("fallback_days", "回退点预测窗口数"),
    ("unproven_plan_days", "未证达到请求Gap窗口数"), ("all_constraints_passed", "约束均通过"),
]

METRICS = [
    ("g_kwh", "计划购电量", "kWh/时段", "零点确定的新窗口普通购电合同：日内固定，未用也付费。"),
    ("charge_kwh", "充电量", "kWh/时段", "微网侧进入电池的电量：内部增加0.9倍，不再除以6。"),
    ("discharge_kwh", "放电量", "kWh/时段", "电池对微网提供的电量：内部减少放电量÷0.9。"),
    ("emergency_kwh", "紧急购电量", "kWh/时段", "普通供应与电池仍不能满足的负载缺口，价格为5倍。"),
    ("unused_paid_kwh", "未用已付合同电量", "kWh/时段", "合同仍按原计划量计费，不退款。"),
    ("curtailed_pv_kwh", "弃光量", "kWh/时段", "未被负载或电池利用的光伏电量：执行分摊遵循光伏优先消纳。"),
    ("planned_cost_yuan", "计划购电费", "元", "各时段区间电价×固定合同量之和；旧前缀合同不重复计费。"),
    ("emergency_cost_yuan", "紧急购电费", "元", "各时段5×区间电价×紧急购电量之和。"),
    ("cash_cost_yuan", "现金总费用", "元", "计划购电费+紧急购电费；包括明确标记的最后估计时段。"),
    ("inventory_adjusted_cost_yuan", "库存修正比较费用", "元", "现金费用+固定库存估值×(起始储电量-结束储电量)，只作策略比较。"),
    ("initial_soc_kwh / final_soc_kwh", "窗口首末储电量", "kWh", "对应D日00:10和D+1日00:10，不是自然日00:00/24:00。"),
    ("actual_estimated", "端点估计标记", "布尔", "True: 本段右端点缺失并估计。False: 两端瞬时值已观测，但区间均值仍是端点平均近似。"),
    ("dayahead_mip_gap", "日前相对最优间隙", "比例", "求解器上下界的相对差。可行超时解不能称精确最优；按原状态保留。"),
    ("dayahead_objective_not_real_bill", "日前规划目标", "元", "含场景预见回补及软末态惩罚的规划值，不是真实MPC电费。"),
    ("historical_residual_days", "完整历史残差窗口数", "窗口", "只用零点前已经完整结束的预测窗口，上一未完窗口被排除。"),
]

def typed(value):
    if value is None or value == "":
        return None
    if value in ("True", "False"):
        return value == "True"
    try:
        number = float(value)
    except (TypeError, ValueError):
        return value
    if not math.isfinite(number):
        raise ValueError(f"Nonfinite CSV value: {value}")
    return number

def read_csv(path):
    with Path(path).open(encoding="utf-8-sig", newline="") as handle:
        return [{key: typed(value) for key, value in row.items()} for row in csv.DictReader(handle)]

def read_json(path):
    with Path(path).open(encoding="utf-8") as handle:
        return json.load(handle)

def close(actual, expected, description, tol=MONEY_TOL):
    error = abs(float(actual) - float(expected))
    if error > tol:
        raise AssertionError(f"{description}: {actual} ≠ {expected}; error={error}")
    return error

```

```

def digest(path):
    h = hashlib.sha256()
    with Path(path).open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024*1024), b''):
            h.update(chunk)
    return h.hexdigest()

def clean_sheet(workbook, name, columns, rows):
    sheet = workbook.create_sheet(name)
    sheet.append([label for _, label in columns])
    for row in rows:
        sheet.append([row.get(key) for key, _ in columns])
    sheet.freeze_panes = "A2"
    sheet.auto_filter.ref = sheet.dimensions
    sheet.sheet_view.showGridLines = False
    sheet.row_dimensions[1].height = 34
    for cell in sheet[1]:
        cell.fill = PatternFill("solid", fgColor=NAVY)
        cell.font = Font(name="微软雅黑", color=WHITE, bold=True)
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
    for col, (key, label) in enumerate(columns, start=1):
        width = 14 if key in ("method", "date") else 22
        if key in ("method_name", "phase_label", "dayahead_solver_status", "status"):
            width = 32
        if key in ("window_start", "window_end", "issued_at", "interval_start", "interval_end"):
            width = 23
        sheet.column_dimensions[get_column_letter(col)].width = width
    for cells in sheet.iter_rows(min_row=2, min_col=col, max_col=col):
        cell = cells[0]
        cell.font = Font(name="微软雅黑", size=10)
        cell.alignment = Alignment(vertical="center", wrap_text=isinstance(cell.value, str))
        if cell.row % 2 == 0:
            cell.fill = PatternFill("solid", fgColor=PALE)
        if isinstance(cell.value, (float, int)) and not isinstance(cell.value, bool):
            if "gap" in key:
                cell.number_format = "0.000000%"
            elif "yuan" in key or key == "new_contract_cost":
                cell.number_format = '#,##0.00'
            elif "kwh" in key or key.endswith("_kw"):
                cell.number_format = '#,##0.0000'
            elif "seconds" in key:
                cell.number_format = '0.000'
            else:
                cell.number_format = '0'
        if ("status" in key and cell.value not in (None, "optimal_within_requested_gap")) or (key ==
        "all_constraints_passed" and cell.value is False):
            cell.fill = PatternFill("solid", fgColor=AMBER)
    return sheet

def notes_sheet(workbook, notes):
    sheet = workbook.create_sheet("阅读说明")
    sheet.append(["事项", "说明"])
    for label, text in notes:
        sheet.append([label, text])
    sheet.column_dimensions["A"].width = 26
    sheet.column_dimensions["B"].width = 115
    sheet.freeze_panes = "B2"
    sheet.sheet_view.showGridLines = False
    for row in sheet:
        for cell in row:
            cell.font = Font(name="微软雅黑", size=11, bold=cell.row == 1, color=WHITE if cell.row == 1
            else "25354A")
            cell.alignment = Alignment(vertical="top", wrap_text=True)
            cell.fill = PatternFill("solid", fgColor=NAVY if cell.row == 1 else (PALE if cell.row % 2 ==

```



```

0 else WHITE))
    sheet.row_dimensions[row[0].row].height = 48 if row[0].row > 1 else 28
    return sheet

def verify_summaries(summaries, stage1, stage2, stage3):
    source = {
        "same_contract_local": stage1,
        "dayahead_local": stage2,
        "continuous_all": stage3,
        "continuous_test": [r for r in stage3 if r["stage"] == "test"],
    }
    max_error = 0.0
    for row in summaries:
        group = [r for r in source[row["phase"]] if r["method"] == row["method"]]
        if not group or int(row["days"]) != len(group):
            raise AssertionError("Summary row count does not agree with daily source")
        for key in SUM_FIELDS:
            max_error = max(max_error, close(math.fsum(r[key] for r in group), row[key],
f'{row["phase"]}/{row["method"]}/{key}'))
            expected_unproven = sum(r["dayahead_solver_status"] != "optimal_within_requested_gap" for r in
group)
            if int(row["unproven_plan_days"]) != expected_unproven:
                raise AssertionError("Unproven solver count differs from daily source")
            if not all(r["planner_checks_passed"] and r["controller_checks_passed"] for r in group):
                raise AssertionError("A daily trajectory has failed recorded checks")
    return max_error

def aggregate_initial(rows, expected_windows=28):
    groups = {}
    for row in rows:
        groups.setdefault((row["method"], float(row["experiment_initial_soc"])), []).append(row)
    output = []
    for (method, initial), group in sorted(groups.items()):
        group.sort(key=lambda r: r["date"])
        if len(group) != expected_windows:
            raise AssertionError("Initial-state sensitivity is not exactly the declared 28 windows")
        close(group[0]["initial_soc_kwh"], initial, "Sensitivity first state", ENERGY_TOL)
        for a, b in zip(group, group[1:]):
            close(a["final_soc_kwh"], b["initial_soc_kwh"], "Sensitivity state continuity", ENERGY_TOL)
        output.append({
            "method": method, "experiment_initial_soc": initial, "windows": len(group),
            "window_start": group[0]["window_start"], "window_end": group[-1]["window_end"],
            **{key: math.fsum(r[key] for r in group) for key in SUM_FIELDS},
            "final_soc_kwh": group[-1]["final_soc_kwh"],
            "unproven_plan_days": sum(r["dayahead_solver_status"] != "optimal_within_requested_gap" for
r in group),
        })
        reference = {r["method"]: r for r in output if r["experiment_initial_soc"] == 6000}
        for row in output:
            base = reference[row["method"]]
            row["cash_change_vs_6000_yuan"] = row["cash_cost_yuan"] - base["cash_cost_yuan"]
            row["inventory_adjusted_change_vs_6000_yuan"] = row["inventory_adjusted_cost_yuan"] -
base["inventory_adjusted_cost_yuan"]
        return output

def collect_solver_logs(results, daily_index):
    rows = []
    for method in METHODS:
        logs = read_json(results / "continuous" / f"{method}_solver.json")
        for log in logs:
            day = daily_index[(method, log["date"])]
            close(log["new_contract_cost"], day["planned_cost_yuan"], "New-contract solver/daily fee")
            if not log["checks"]["passed"]:

```

```

        raise AssertionError("A solver log has failed checks")
    predicted = log.get("forecast_soc_at_window_start")
    if predicted is None:
        predicted = []
    rows.append({
        "method": method, "date": log["date"], "issued_at": day["issued_at"],
        "decision_soc_kwh": log["midnight_observed_soc"],
        "forecast_start_min_kwh": min(predicted) if predicted else None,
        "forecast_start_max_kwh": max(predicted) if predicted else None,
        "actual_start_kwh": log.get("executed_soc_at_window_start"),
        "old_prefix_g_kwh": log.get("fixed_prefix_kwh"),
        "new_contract_kwh": log["new_contract_kwh"], "new_contract_cost":
log["new_contract_cost"],
        "status": log["status"], "gap": log["gap"],
        "checks_passed": log["checks"]["passed"],
        "max_violation_kwh": log["checks"]["max_violation_kwh"],
        "history_days": log["risk"]["history_days"],
        "fallback": log["risk"].get("fallback", False),
        "solve_seconds": day["dayahead_solve_seconds"],
    })
    return rows

def stream_trace(path, method, daily_rows, expected_windows):
    """Keep just one 144-row day in memory, not the full interval dataframe."""
    day_index = {r["date"]: r for r in daily_rows}
    plans, stats, episodes = [], [], []
    current_date, current_rows, episode = None, [], None
    previous_end = None
    previous_soc = None
    estimated_locations = []
    count = 0
    max_money_error = 0.0

    def finish_day(date, rows):
        nonlocal max_money_error
        if len(rows) != 144 or date not in day_index:
            raise AssertionError(f"{method}/{date}: expected exactly 144 trace rows")
        daily = day_index[date]
        start = dt.datetime.fromisoformat(rows[0]["interval_start"])
        end = dt.datetime.fromisoformat(rows[-1]["interval_end"])
        if start.time() != dt.time(0, 10) or end - start != dt.timedelta(days=1):
            raise AssertionError("Trace is not a 00:10 to next 00:10 window")
        amounts = {
            "planned_cost_yuan": math.fsum(r["price_yuan_per_kwh"] * r["g_kwh"] for r in rows),
            "emergency_cost_yuan": math.fsum(5*r["price_yuan_per_kwh"] * r["emergency_kwh"] for r in
rows),
            "cash_cost_yuan": math.fsum(r["cash_cost_yuan"] for r in rows),
            "planned_kwh": math.fsum(r["g_kwh"] for r in rows),
            "emergency_kwh": math.fsum(r["emergency_kwh"] for r in rows),
            "unused_paid_kwh": math.fsum(r["unused_paid_kwh"] for r in rows),
            "curtailed_pv_kwh": math.fsum(r["curtailed_pv_kwh"] for r in rows),
        }
        for key, value in amounts.items():
            max_money_error = max(max_money_error, close(value, daily[key], f"{method}/{date}/{key}"))
        close(rows[0]["soc_start_kwh"], daily["initial_soc_kwh"], "Window start SOC", ENERGY_TOL)
        close(rows[-1]["soc_end_kwh"], daily["final_soc_kwh"], "Window end SOC", ENERGY_TOL)
        plans.append([date] + [r["g_kwh"] for r in rows])
        stats.append({
            "date": date, "window_start": start.isoformat(sep=" "), "window_end": end.isoformat(sep="
"),
            **amounts, "charge_kwh": math.fsum(r["charge_kwh"] for r in rows),
            "discharge_kwh": math.fsum(r["discharge_kwh"] for r in rows),
            "initial_soc_kwh": rows[0]["soc_start_kwh"], "final_soc_kwh": rows[-1]["soc_end_kwh"],
            "actual_estimated_intervals": sum(r["actual_estimated"] for r in rows),
            "dayahead_solver_status": daily["dayahead_solver_status"], "dayahead_mip_gap":

```



```

daily["dayahead_mip_gap"],
})

with Path(path).open(encoding="utf-8-sig", newline="") as handle:
    for source in csv.DictReader(handle):
        row = {k: typed(v) for k, v in source.items()}
        if row["method"] != method:
            raise AssertionError("Method label differs from trace filename")
        a = dt.datetime.fromisoformat(row["interval_start"])
        b = dt.datetime.fromisoformat(row["interval_end"])
        if b-a != dt.timedelta(minutes=10):
            raise AssertionError("Trace interval has wrong duration")
        if previous_end is not None:
            if a != previous_end:
                raise AssertionError("Trace timestamps have a gap/overlap")
            close(row["soc_start_kwh"], previous_soc, "Continuous trace SOC", ENERGY_TOL)
            close(row["cash_cost_yuan"], row["price_yuan_per_kwh"]*
(row["g_kwh"])+5*row["emergency_kwh"]), "Trace single-slot fee")
        if row["actual_estimated"]:
            estimated_locations.append(row["interval_start"])
        if current_date is not None and row["plan_date"] != current_date:
            finish_day(current_date, current_rows)
            current_rows = []
            current_date = row["plan_date"]
            current_rows.append(row)
        if row["emergency_kwh"] > ENERGY_TOL:
            if episode is None:
                episode = {"interval_start": a.isoformat(sep=" "), "interval_end": b.isoformat(sep="
"),
                        "intervals": 0, "duration_minutes": 0, "emergency_kwh": 0.,
                        "emergency_cost_yuan": 0., "peak_equivalent_kw": 0.,
"estimated_intervals": 0}
                episode["interval_end"] = b.isoformat(sep=" ")
                episode["intervals"] += 1
                episode["duration_minutes"] += 10
                episode["emergency_kwh"] += row["emergency_kwh"]
                episode["emergency_cost_yuan"] += 5*row["price_yuan_per_kwh"]*row["emergency_kwh"]
                episode["peak_equivalent_kw"] = max(episode["peak_equivalent_kw"],
row["emergency_kwh"]*6)
                episode["estimated_intervals"] += int(row["actual_estimated"])
            elif episode is not None:
                episodes.append(episode)
                episode = None
            previous_end, previous_soc = b, row["soc_end_kwh"]
            count += 1
        if current_rows:
            finish_day(current_date, current_rows)
        if episode is not None:
            episodes.append(episode)
        if len(plans) != expected_windows or count != expected_windows*144:
            raise AssertionError("Incomplete full-period trace")
        close(stats[0]["initial_soc_kwh"], 6000, "User-prescribed experiment initial SOC", ENERGY_TOL)
        final_interval_start = (dt.datetime.fromisoformat(stats[-1]["window_end"]) -
dt.timedelta(minutes=10)).isoformat()
        if estimated_locations != [final_interval_start]:
            raise AssertionError("Exactly the final interval must be marked estimated")
    return plans, stats, episodes, {
        "method": method, "windows": len(plans), "trace_rows": count,
        "estimated_intervals": len(estimated_locations), "max_daily_reconciliation_error":
max_money_error,
        "planned_kwh": math.fsum(r["planned_kwh"] for r in stats),
        "cash_cost_yuan": math.fsum(r["cash_cost_yuan"] for r in stats),
        "source_sha256": digest(path),
    }

```

```

def write_only_sheet(book, name, headers, rows, widths=None):
    sheet = book.create_sheet(name)
    sheet.freeze_panes = "B2" if name == "购电计划_kWh" else "A2"
    sheet.sheet_view.showGridLines = False
    sheet.row_dimensions[1].height = 34
    for i in range(1, len(headers)+1):
        sheet.column_dimensions[get_column_letter(i)].width = widths[i-1] if widths else (15 if i == 1
else 21)
    cells = []
    for value in headers:
        cell = WriteOnlyCell(sheet, value)
        cell.font = Font(name="微软雅黑", bold=True, color=WHITE)
        cell.fill = PatternFill("solid", fgColor=NAVY)
        cell.alignment = Alignment(wrap_text=True, vertical="center", horizontal="center")
        cells.append(cell)
    sheet.append(cells)
    count = 0
    for row in rows:
        cells = []
        for column_index, value in enumerate(row):
            cell = WriteOnlyCell(sheet, value)
            cell.font = Font(name="微软雅黑", size=10)
            if isinstance(value, (float, int)) and not isinstance(value, bool):
                cell.number_format = '#,##0.0000'
            cell.alignment = Alignment(vertical="center", wrap_text=isinstance(value, str))
            if count % 2 == 0:
                cell.fill = PatternFill("solid", fgColor=PALE)
            if name == "阅读说明" and row[0] == "完整逐段策略" and column_index == 1:
                cell.hyperlink = str(value)
                cell.font = Font(name="微软雅黑", size=10, color="0563C1", underline="single")
            cells.append(cell)
            if name == "阅读说明":
                sheet.row_dimensions[count+2].height = 48
        sheet.append(cells)
        count += 1
    sheet.auto_filter.ref = f"A1:{get_column_letter(len(headers))}{count+1}"
    return sheet

def plan_headers():
    def clock(minutes):
        prefix = "次日" if minutes ≥ 1440 else ""
        minutes %= 1440
        return f"{prefix}{minutes//60:02d}:{minutes%60:02d}"
    return ["窗口归属日期"] + [f"{clock(t*10)}-{clock(t*10+10)}" for t in range(1,145)]

def write_strategy(path, method, plans, stats, episodes, companion, notes):
    book = Workbook(write_only=True)
    write_only_sheet(book, "阅读说明", ["事项", "说明"], notes + [("方案", f"{method}: {METHODS[method]}"),
        ("完整逐段策略", str(companion)), ("完整CSV字段", "每10分钟购电、充放电、应急、弃光、未用合同、SOC、费用和
估计标记。")], widths=[25,115])
    write_only_sheet(book, "购电计划_kWh", plan_headers(), plans)
    columns = [x for x in DAILY_COLUMNS if x[0] in stats[0]] + [("charge_kWh", "实际充电量(kWh)",
("discharge_kWh", "实际放电量(kWh)")]
    write_only_sheet(book, "逐日执行与SOC", [v for _,v in columns], [[r.get(k) for k,_ in columns] for r
in stats])
    episode_columns = [
        ("interval_start", "连续应急开始"), ("interval_end", "连续应急结束"),
        ("intervals", "连续十分钟段数"), ("duration_minutes", "持续分钟"),
        ("emergency_kWh", "紧急购电量(kWh)", ("emergency_cost_yuan", "紧急购电费(元)",
        ("peak_equivalent_kW", "最高等效功率(kW)", ("estimated_intervals", "含估计端点段数"),
    ]
    write_only_sheet(book, "连续应急时段", [v for _,v in episode_columns], [[r[k] for k,_ in
episode_columns] for r in episodes])
    book.save(path)

```

```

def inspect_workbook(path, expected_plan_windows=None, expected_g=None):
    book = load_workbook(path, read_only=True, data_only=False)
    counts = {}
    plan_sum = 0.0
    try:
        for sheet in book:
            count = 0
            for row in sheet.iter_rows(values_only=True):
                for value in row:
                    if isinstance(value, str) and (value in ERROR_VALUES or value.startswith("=")):
                        raise AssertionError(f"Unexpected formula/error in {path.name}/{sheet.title}")
                    if sheet.title == "购电计划_kWh" and count > 0:
                        if len(row) != 145:
                            raise AssertionError("Purchase wide table must have one date and 144 energy
columns")
                        plan_sum += math.fsum(float(x) for x in row[1:])
                        count += 1
            counts[sheet.title] = count
        if expected_plan_windows is not None:
            if counts.get("购电计划_kWh") != expected_plan_windows+1:
                raise AssertionError("Purchase workbook has wrong window count")
            close(plan_sum, expected_g, "Workbook purchase total", 1e-3)
    finally:
        book.close()
    return counts

def build(results, out, expected_windows=334):
    started = perf_counter()
    results, out = Path(results), Path(out)
    required = ["summary.json", "comparison_summary.csv", "stage1_same_contract_daily.csv",
                "stage2_dayahead_daily.csv", "stage3_continuous_daily.csv",
                "initial_soc_sensitivity_daily.csv", "final_interval_sensitivity.csv",
"frozen_parameters.json"]
    for name in required:
        if not (results/name).exists():
            raise FileNotFoundError(f"Completed run required; missing {results/name}")
    summary = read_json(results/"summary.json")
    if summary.get("data_role") != "supplied_forecasts_with_endpoint_averaged_actuals":
        raise ValueError("This report requires supplied forecasts, not synthetic experiment outputs")
    start, end = map(dt.datetime.fromisoformat, summary["dates"])
    if end-start != dt.timedelta(days=expected_windows):
        raise ValueError("Summary does not cover the expected complete window horizon")
    if summary["estimated_actual_intervals"] != 1:
        raise AssertionError("Expected only the final missing-endpoint interval to be estimated")
    stage1 = read_csv(results/"stage1_same_contract_daily.csv")
    stage2 = read_csv(results/"stage2_dayahead_daily.csv")
    stage3 = read_csv(results/"stage3_continuous_daily.csv")
    summaries = read_csv(results/"comparison_summary.csv")
    json_periods = {(r["phase"], r["method"]): r for r in summary["periods"]}
    for row in summaries:
        counterpart = json_periods[(row["phase"], row["method"])]
        for field in SUM_FIELDS:
            close(row[field], counterpart[field], f"CSV/JSON summary agreement: {field}")
    for row in summaries:
        row["phase_label"] = PHASES[row["phase"]]
        row["method_name"] = METHODS[row["method"]]
    reconciliation = verify_summaries(summaries, stage1, stage2, stage3)
    daily_index = {(r["method"], r["date"]): r for r in stage3}
    if len(daily_index) != 4*expected_windows or len(stage3) != len(daily_index):
        raise AssertionError("Continuous daily rows are incomplete or duplicated")
    solver_rows = collect_solver_logs(results, daily_index)
    if len(solver_rows) != 4*expected_windows:
        raise AssertionError("Continuous solver logs are incomplete")

```

```

initial = aggregate_initial(read_csv(results/"initial_soc_sensitivity_daily.csv"))
boundary = read_csv(results/"final_interval_sensitivity.csv")
frozen = read_json(results/"frozen_parameters.json")
for method in METHODS:
    candidates = [r for r in boundary if r["method"] == method]
    if {r["boundary_method"] for r in candidates} != {"hold", "linear2", "ols6"}:
        raise AssertionError("Boundary sensitivity must contain all three declared methods")
    hold = next(r for r in candidates if r["boundary_method"] == "hold")
    method_summary = next(r for r in summaries if r["phase"] == "continuous_all" and r["method"] ==
method)
    close(hold["total_cash_yuan"], method_summary["cash_cost_yuan"], "Hold boundary/full-period
bill")
    for row in candidates:
        row["cash_change_vs_hold_yuan"] = row["total_cash_yuan"] - hold["total_cash_yuan"]
    notes = [
        ("数据性质", "本次使用用户提供的负载与光伏预测、附件2原始数据；不是合成数据。预测生成过程仍以团队来源声明为条
件。"),
        ("覆盖范围", f"{start:%Y-%m-%d %H:%M} 至 {end:%Y-%m-%d %H:%M}，共{expected_windows}个00:10至次日
00:10窗口。"),
        ("预测口径", "本日00:00-00:10预测列已删除；每窗保留144个区间平均功率，包含次日00:00-00:10。预测均值不再相邻
平均。"),
        ("实际口径", "附件2为瞬时节点：本行前143个区间取相邻端点均值，最后区间跨到下一日首00:10节点；不是把原单点当区间
均值。"),
        ("最后一段", "2026-01-01 00:10右端节点缺失，主结果用末节点保持估计；仅最后10分钟标estimated，另列2点线性与6
点OLS敏感性。"),
        ("初始储电量", "6000 kWh是用户指定的首个00:10窗口统一实验初态，不冒称题目给定2月1日00:10观测值；另做前28窗初
态敏感性。"),
        ("零点决策", "继续在00:00定新合同。规划联算旧合同00:00-00:10前缀与新144段，只用该前缀预测；先固定新合同再执行
前缀，旧合同仅付一次。"),
        ("功率与费用", "区间平均功率×1/6得到kWh；g、充放电量已是kWh不再除以6。费用=计划购电费+5倍电价紧急购电费。"),
        ("电价近似", "附件1瞬时电价先周期补00:00节点，再取相邻端点均价；采用区间均价结算近似。"),
        ("日内反馈", "当前区间均值用于模拟段内快速反馈，不等于零点或段初已知未来十分钟实值。"),
        ("比较范围", "同g A/B和同MPC B/C/D为各窗独立同初态实验；连续执行才把SOC逐窗传递。不能把前者累计当连续年运
行。"),
        ("参数冻结", f"裕量分位数={frozen['selected_quantile']}；生效日={frozen['effective_from']}。场景数=
{frozen['config']['scenarios']}为预设值，不声称已优化。"),
        ("优化目标与现金", "库存修正费用只作可比性评价；随机MILP的整日场景回补具有预见性近似，规划目标不能替代实际滚动
现金费用。"),
        ("提交模板", "本工作簿是00:10窗口策略与研究结果，不是自然日0:00/24:00的result2提交模板。请勿把窗口SOC/账目直
接贴成自然日指标。"),
        ("数值校验", "金额从逐段CSV重新累加，逐日对账并核对阶段汇总；保留求解Gap和未证最优状态，未加入隐藏平局成本。"),
    ]
    out.mkdir(parents=True, exist_ok=True)
    companions = out/"逐段策略CSV"
    companions.mkdir(exist_ok=True)
    trace_audits = []
    workbook_checks = {}
    for method in METHODS:
        source = results/"continuous"/f"{method}_trace.csv"
        days = sorted([r for r in stage3 if r["method"] == method], key=lambda r:r["date"])
        plans, stats, episodes, audit = stream_trace(source, method, days, expected_windows)
        companion = companions/f"方案{method}_完整逐段策略.csv"
        shutil.copy2(source, companion)
        path = out/f"方案{method}_完整策略.xlsx"
        write_strategy(path, method, plans, stats, episodes, companion.relative_to(out), notes)
        workbook_checks[path.name] = inspect_workbook(path, expected_windows, audit["planned_kwh"])
        audit["emergency_episode_count"] = len(episodes)
        trace_audits.append(audit)
    print(f"Checked strategy workbook: {method}; {len(plans)} windows", flush=True)

book = Workbook()
book.remove(book.active)
notes_sheet(book, notes)
clean_sheet(book, "方法汇总", SUMMARY_COLUMNS, summaries)
clean_sheet(book, "连续逐日", DAILY_COLUMNS, stage3)
clean_sheet(book, "同g_AB", DAILY_COLUMNS, stage1)

```

```

clean_sheet(book, "同MPC_BCD", DAILY_COLUMNS, stage2)
initial_columns = [
    ("method", "方案"), ("experiment_initial_soc", "实验初态(kWh)", ("windows", "敏感性连续窗口数"),
    ("window_start", "开始"), ("window_end", "结束"),
    ("planned_cost_yuan", "计划费(元)", ("emergency_cost_yuan", "紧急费(元)",
    ("cash_cost_yuan", "28窗现金费用(元)", ("inventory_adjusted_cost_yuan", "库存修正费用(元)",
    ("final_soc_kwh", "第28窗末SOC(kWh)",
    ("cash_change_vs_6000_yuan", "相对6000初态现金差(元)",
    ("inventory_adjusted_change_vs_6000_yuan", "相对6000库存修正差(元)",
    ("unproven_plan_days", "未证达到请求Gap窗口数"),
]
clean_sheet(book, "初SOC敏感性_前28窗", initial_columns, initial)
boundary_columns = [
    ("method", "方案"), ("boundary_method", "边界估计方法"),
    ("interval_start", "末段开始"), ("interval_end", "末段结束"),
    ("load_right_endpoint_kw", "估计右端负载(kW)", ("pv_right_endpoint_kw", "估计右端光伏(kW)",
    ("load_mean_kw", "末段平均负载(kW)", ("pv_mean_kw", "末段平均光伏(kW)",
    ("last_interval_cash_yuan", "最后一段现金费(元)",
    ("last_interval_emergency_yuan", "最后一段紧急费(元)",
    ("total_cash_yuan", "全周期现金费(元)", ("cash_change_vs_hold_yuan", "相对hold费用差(元)",
    ("final_soc_kwh", "末段后SOC(kWh)", ("estimated", "该段使用估计端点"),
]
clean_sheet(book, "末段估计敏感性", boundary_columns, boundary)
gap_columns = [
    ("method", "方案"), ("date", "新窗口归属日期"), ("issued_at", "决策时间"),
    ("decision_soc_kwh", "决策所用SOC(kWh)",
    ("forecast_start_min_kwh", "预测00:10SOC最小值(kWh)", ("forecast_start_max_kwh", "预测00:10SOC最大
值(kWh)",
    ("actual_start_kwh", "执行到00:10实际SOC(kWh)", ("old_prefix_g_kwh", "前缀已付合同(kWh)",
    ("new_contract_kwh", "新合同电量(kWh)", ("new_contract_cost", "新合同费用(元)",
    ("status", "日前求解状态"), ("gap", "相对Gap"), ("checks_passed", "物理检查通过"),
    ("max_violation_kwh", "最大约束误差(kWh)", ("history_days", "完整历史窗口数"),
    ("fallback", "回退点预测"), ("solve_seconds", "日前求解秒数"),
]
clean_sheet(book, "求解Gap", gap_columns, solver_rows)
metric_rows = [dict(zip(("key", "name", "unit", "meaning"), row)) for row in METRICS]
dictionary = clean_sheet(book, "指标字典", [("key", "原始字段"), ("name", "中文名称"), ("unit", "单位"),
("meaning", "含义与边界")], metric_rows)
dictionary.column_dimensions["A"].width = 45
dictionary.column_dimensions["B"].width = 25
dictionary.column_dimensions["D"].width = 85
summary_path = out/"问题二_计算结果.xlsx"
book.save(summary_path)
workbook_checks[summary_path.name] = inspect_workbook(summary_path)
with (out/"中文指标字典.csv").open("w", encoding="utf-8-sig", newline="") as handle:
    writer = csv.writer(handle)
    writer.writerow(["原始字段", "中文名称", "单位", "含义与边界"])
    writer.writerows(METRICS)
with (out/"阅读说明.md").open("w", encoding="utf-8") as handle:
    handle.write("# 第二问计算结果与完整策略\n\n")
    for label, text in notes:
        handle.write(f"#{label}: {text}\n\n")
    handle.write("`问题二_计算结果.xlsx`汇总方法、逐日执行、两类公平对照、敏感性Gap.\n\n")
    handle.write("`每份`方案A/B/C/D_完整策略.xlsx`包含144段购电宽表、实际逐日SOC与连续应急段; 同名完整逐段CSV位
于`逐段策略CSV`文件夹.\n")
audit = {
    "source_role": summary["data_role"], "synthetic": False,
    "source_summary_sha256": digest(results/"summary.json"),
    "window_start": start.isoformat(), "window_end": end.isoformat(),
    "expected_windows": expected_windows, "cash_reconciliation_tolerance_yuan": MONEY_TOL,
    "max_summary_daily_reconciliation_error": reconciliation,
    "trace_checks": trace_audits, "workbook_row_counts": workbook_checks,
    "all_checks_passed": True,
    "expense_basis": "window accounting; final estimated interval retained and labelled; not a
natural-day result2 template",
    "elapsed_seconds": perf_counter()-started,

```

```
}  
with (out/"报表核验.json").open("w", encoding="utf-8") as handle:  
    json.dump(audit, handle, ensure_ascii=False, indent=2)  
print(json.dumps(audit, ensure_ascii=False, indent=2))  
return audit  
  
def main():  
    parser = argparse.ArgumentParser(description=__doc__)  
    parser.add_argument("--results", type=Path, default=ROOT/"results/final_team")  
    parser.add_argument("--out", type=Path, default=ROOT/"results/final_team/中文报表")  
    parser.add_argument("--expected-windows", type=int, default=334)  
    args = parser.parse_args()  
    build(args.results, args.out, args.expected_windows)  
  
if __name__ == "__main__":  
    main()
```

7. question2/planning/code/complete_initial_soc_baseline.py

源文件 SHA-256: f40313eef6dce2ee110eaabb1d8391954f2b3044245486d876c8572d17df7deb ; 代码行数: 136。


```

"""Supplement the SOC sensitivity with an independent 6000-kWh, 28-window run.

The long main replay and its sensitivity CSV are never overwritten. All four
methods run sequentially with the frozen main-experiment parameters. Their last
interval uses the same one-step ending as the existing 1200/10800 experiments.
"""
from __future__ import annotations

import argparse
import csv
import datetime as dt
import hashlib
import json
from pathlib import Path
import time

import window_study as study
from input_io import write_csv
from window_io import attach_actuals, prices, read_team_forecasts

ROOT = Path(__file__).resolve().parents[1]
WINDOWS = 28
INITIAL_SOC = 6000.

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def verify_output(out, method, daily, data):
    """Re-read actual output boundaries, cash and SOC continuity independently."""
    with (out / f"{method}_trace.csv").open(encoding="utf-8-sig", newline="") as f:
        rows = list(csv.DictReader(f))
        if len(daily) != WINDOWS or len(rows) != WINDOWS * 144:
            raise AssertionError("The baseline must contain exactly 28 complete windows")
        if float(rows[0]["soc_start_kwh"]) != INITIAL_SOC:
            raise AssertionError("Wrong prescribed first state")
        if dt.datetime.fromisoformat(rows[0]["interval_start"]) != data["window_start"][0]:
            raise AssertionError("Wrong first interval")
        if dt.datetime.fromisoformat(rows[-1]["interval_end"]) != data["window_end"][WINDOWS - 1]:
            raise AssertionError("Wrong final interval")
        for left, right in zip(rows, rows[1:]):
            if left["interval_end"] != right["interval_start"]:
                raise AssertionError("Time discontinuity")
            if abs(float(left["soc_end_kwh"]) - float(right["soc_start_kwh"])) > 1e-5:
                raise AssertionError("State discontinuity")
        cash = sum(float(r["price_yuan_per_kwh"]) *
                    (float(r["g_kwh"]) + 5 * float(r["emergency_kwh"]))) for r in rows)
        stored_cash = sum(float(r["cash_cost_yuan"]) for r in rows)
        daily_cash = sum(float(r["cash_cost_yuan"]) for r in daily)
        if max(abs(cash - stored_cash), abs(cash - daily_cash)) > 1e-5:
            raise AssertionError("Trace/daily cash mismatch")
        if not all(r["planner_checks_passed"] and r["controller_checks_passed"] for r in daily):
            raise AssertionError("Physical check failed")
        logs = json.loads((out / f"{method}_solver.json").read_text(encoding="utf-8"))
        if len(logs) != WINDOWS or logs[-1]["date"] != str(data["dates"][WINDOWS - 1]):
            raise AssertionError("A 29th-window contract must not enter this experiment")
        return {"method": method, "windows": WINDOWS, "intervals": len(rows),
                "cash_cost_yuan": cash, "final_soc_kwh": float(rows[-1]["soc_end_kwh"]),
                "time_state_and_cash_checks_passed": True,
                "all_planner_and_controller_checks_passed": True}

```

```

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--results", type=Path, default=ROOT / "results/final_team")
    parser.add_argument("--forecasts", type=Path, default=ROOT / "data/team_forecasts.xlsx")
    parser.add_argument("--actuals", type=Path, default=ROOT / "data/attachment2_original.xlsx")
    args = parser.parse_args()
    frozen_path = args.results / "frozen_parameters.json"
    frozen = json.loads(frozen_path.read_text(encoding="utf-8"))
    config = frozen["config"].copy()
    if float(config["initial_soc"]) != INITIAL_SOC:
        raise ValueError("This correction is specifically for the prescribed 6000-kWh baseline")
    q = float(frozen["selected_quantile"])
    data = attach_actuals(read_team_forecasts(args.forecasts), args.actuals,
                        confirmed_instantaneous=True)
    if len(data["dates"]) < WINDOWS:
        raise ValueError("At least 28 forecast windows are required")
    p, pm = prices(ROOT / "data/attachment1.xlsx")
    audit = json.loads((args.results / "input_audit.json").read_text(encoding="utf-8"))
    for key in ("forecast_sha256", "actual_sha256"):
        if data["metadata"][key] != audit[key]:
            raise ValueError(f"Input differs from main experiment: {key}")
    if pm["sha256"] != audit["price"]["sha256"]:
        raise ValueError("Price source differs from the main experiment")
    out = args.results / "initial_soc_6000_common_horizon"
    out.mkdir(parents=True, exist_ok=True)
    manifest = {
        "purpose": "Replace the long-replay slice with an independent 28-window 6000-kWh sensitivity
baseline",
        "status": "running", "config": config, "selected_quantile": q,
        "windows": WINDOWS, "initial_soc_kwh": INITIAL_SOC,
        "window_start": str(data["window_start"][0]),
        "window_end": str(data["window_end"][WINDOWS - 1]),
        "terminal_rule": "Last interval is one-step execution, without a 29th-window contract or
forecast horizon",
        "parameter_rule": "Use frozen main parameters; retain preset q before its original effective
date; no retuning",
        "execution": "A, B, C, D sequentially in a single process",
        "main_334_window_results_modified": False,
        "main_initial_soc_sensitivity_csv_modified": False,
        "frozen_parameters_sha256": sha(frozen_path),
        "inputs": {"forecast_sha256": data["metadata"]["forecast_sha256"],
                    "actual_sha256": data["metadata"]["actual_sha256"], "price_sha256": pm["sha256"]},
        "code_sha256": {name: sha(ROOT / "code" / name) for name in
                        ("window_study.py", "window_io.py", "window_control.py",
                         "planners.py", "control.py", "control_refined.py")},
        "completed_methods": [], "verification": [],
    }
    started = time.perf_counter()
    study.json_write(out / "correction_manifest.json", manifest)
    study.initialize(data, p, config)
    supplemental = []
    for method in ("A", "B", "C", "D"):
        manifest["current_method"] = method
        study.json_write(out / "correction_manifest.json", manifest)
        print(f"Independent common-horizon baseline: {method}, S0=6000, 28 windows", flush=True)
        result = study.continuous_job((method, q, str(out), WINDOWS, INITIAL_SOC))
        manifest["verification"].append(verify_output(out, method, result["daily"], data))
        manifest["completed_methods"].append(method)
        supplemental.extend(dict(row, experiment_initial_soc=INITIAL_SOC) for row in result["daily"])
        study.json_write(out / "correction_manifest.json", manifest)
    supplemental.sort(key=lambda r: (r["date"], r["method"]))
    supplement_path = args.results / "initial_soc_6000_common_horizon_daily.csv"
    write_csv(supplement_path, supplemental)
    manifest.update(status="complete", current_method=None,
                    elapsed_seconds=time.perf_counter() - started,
                    supplemental_csv=str(supplement_path),

```



```
        supplemental_rows=len(supplemental),
        merge_status="Not merged: wait for the main run to finish before replacing its 6000-
kWh sensitivity rows")
    study.json_write(out / "correction_manifest.json", manifest)
    print(json.dumps({"status": "complete", "rows": len(supplemental),
                     "supplement": str(supplement_path)}, ensure_ascii=False), flush=True)

if __name__ == "__main__":
    main()
```

8. question2/planning/code/control.py

源文件 SHA-256: ca96f1b8f58b16f52cf119f2b4daa8394861e65dae7566830b675781a4cea2c0 ; 代码行数: 390。

```
"""Fixed-contract battery dispatch, with causal receding-horizon control.
```

```
All time-slot quantities (g, load, pv, c, d, e, v, r, w) are kWh.
```

```
Prices are yuan/kWh; battery state is internal stored energy in kWh.
```

```
An input slot represents ten minutes. This module never infers timestamps,  
generates a contract, adds a forecast margin, or learns parameters from actuals.
```

```
`simulate_day` reveals only the current actual observation to the controller.
```

```
Future inputs remain the forecast supplied at midnight. Real-time observation  
within each slot is an explicit simulation assumption. `offline_oracle` is a  
separate, noncausal benchmark that may access the entire actual trajectory.
```

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
import os
```

```
from time import perf_counter
```

```
from typing import Literal
```

```
import numpy as np
```

```
from scipy.optimize import linprog
```

```
from scipy.sparse import coo_matrix
```

```
Terminal = Literal["shortfall", "value", "none"]
```

```
Method = Literal["greedy", "mpc_soft", "mpc_value", "mpc_none"]
```

```
@dataclass(frozen=True)
```

```
class Battery:
```

```
    """Battery parameters. The power rating applies on the microgrid side."""
```

```
    low: float = 1200.0
```

```
    high: float = 10800.0
```

```
    power_kw: float = 5000.0
```

```
    eta_c: float = float(os.environ.get("MCM_BATTERY_ETA", "0.9"))
```

```
    eta_d: float = float(os.environ.get("MCM_BATTERY_ETA", "0.9"))
```

```
    def __post_init__(self) → None:
```

```
        values = (self.low, self.high, self.power_kw, self.eta_c, self.eta_d)
```

```
        if not all(np.isfinite(v) for v in values):
```

```
            raise ValueError("Battery parameters must be finite.")
```

```
        if not 0 ≤ self.low < self.high:
```

```
            raise ValueError("Require 0 ≤ low < high.")
```

```
        if self.power_kw ≤ 0:
```

```
            raise ValueError("power_kw must be positive.")
```

```
        if not 0 < self.eta_c ≤ 1 or not 0 < self.eta_d ≤ 1:
```

```
            raise ValueError("Charge/discharge efficiency must be in (0, 1].")
```

```
    @property
```

```
    def rate(self) → float:
```

```
        """Maximum external charge or discharge energy per ten-minute slot."""
```

```
        return self.power_kw / 6.0
```

```
DEFAULT_BATTERY = Battery()
```

```
PHYSICAL_TOL = 1e-5
```

```
LP_TOL = 1e-8
```

```
def _vector(value, name: str, n: int | None = None) → np.ndarray:
```

```
    a = np.asarray(value, dtype=float)
```

```
    if a.ndim ≠ 1 or a.size == 0:
```

```

        raise ValueError(f"{name} must be a nonempty one-dimensional array.")
    if n is not None and a.size != n:
        raise ValueError(f"{name} has {a.size} entries, expected {n}.")
    if not np.isfinite(a).all() or np.any(a < 0):
        raise ValueError(f"{name} must contain only finite nonnegative values.")
    return a.copy()

def _initial_state(s0: float, b: Battery) → float:
    s0 = float(s0)
    if not np.isfinite(s0) or not b.low - PHYSICAL_TOL ≤ s0 ≤ b.high + PHYSICAL_TOL:
        raise ValueError("Initial battery state is outside operating bounds.")
    # Repeated solves can leave boundary deviations at machine precision.
    return float(np.clip(s0, b.low, b.high))

def _terminal_parameters(terminal: str, weight: float, s_ref: float) → None:
    if terminal not in ("shortfall", "value", "none"):
        raise ValueError(f"Unknown terminal mode: {terminal}")
    if not np.isfinite(weight) or weight < 0:
        raise ValueError("Terminal weight must be finite and nonnegative.")
    if not np.isfinite(s_ref):
        raise ValueError("s_ref must be finite.")

def _account_surplus(g, pv, load, c, d):
    """PV-first bookkeeping: unused paid energy first, then PV curtailment."""
    surplus = np.maximum(g + pv - load, 0.0)
    deficit = np.maximum(load - g - pv, 0.0)
    e = np.maximum(deficit - d, 0.0)
    w = np.maximum(surplus - c, 0.0)
    v = np.minimum(w, g)
    r = w - v
    return e, v, r, w

def solve_horizon(
    g,
    load,
    pv,
    price,
    s0: float,
    b: Battery = DEFAULT_BATTERY,
    terminal: Terminal = "shortfall",
    weight: float = 0.0,
    s_ref: float = 6000.0,
) → dict:
    """Solve a remaining-horizon dispatch LP for a fixed purchase contract.

    There are n charges, n discharges, n end-of-slot states, and optionally one
    terminal shortfall variable. Emergency purchase satisfies  $e = D - d$  and is
    eliminated algebraically. Simultaneous charge/discharge is prevented by the
    disjoint surplus/deficit bounds, rather than binary variables.

    ``shortfall`` objective:  $\text{sum}(5 * \text{price} * e) + \text{weight} * \max(s\_ref - s[-1], 0)$ .
    ``value`` objective:  $\text{sum}(5 * \text{price} * e) - \text{weight} * s[-1]$ .
    ``none`` objective:  $\text{sum}(5 * \text{price} * e)$ .
    The returned objective includes the emergency-cost constant eliminated in
    the LP. No tie-breaking cost is added. An unsuccessful solver raises an
    exception instead of silently using an unverified or altered policy.
    """
    g = _vector(g, "g")
    n = len(g)
    load = _vector(load, "load", n)
    pv = _vector(pv, "pv", n)
    price = _vector(price, "price", n)

```

```

s0 = _initial_state(s0, b)
_terminal_parameters(terminal, weight, s_ref)

surplus = np.maximum(g + pv - load, 0.0)
deficit = np.maximum(load - g - pv, 0.0)
c_upper = np.minimum(surplus, b.rate)
d_upper = np.minimum(deficit, b.rate)
has_shortfall = terminal == "shortfall"
nvar = 3 * n + int(has_shortfall)
cost = np.zeros(nvar)
cost[n : 2 * n] = -5.0 * price
if terminal == "value":
    cost[3 * n - 1] = -weight
elif has_shortfall:
    cost[-1] = weight

# -eta_c*c_t + d_t/eta_d + S_(t+1) - S_t = 0.
# The initial S_0 is a known RHS rather than an optimization variable.
slots = np.arange(n)
rows = np.concatenate((slots, slots, slots, slots[1:]))
cols = np.concatenate((slots, n + slots, 2 * n + slots, 2 * n + slots[:-1]))
vals = np.concatenate((
    np.full(n, -b.eta_c), np.full(n, 1.0 / b.eta_d),
    np.ones(n), -np.ones(max(n - 1, 0)),
))
a_eq = coo_matrix((vals, (rows, cols)), shape=(n, nvar)).tocsc()
rhs = np.zeros(n)
rhs[0] = s0
bounds = (
    [(0.0, float(cap)) for cap in c_upper]
    + [(0.0, float(cap)) for cap in d_upper]
    + [(b.low, b.high)] * n
)
a_ub = None
rhs_ub = None
if has_shortfall:
    #  $x_i \geq s_{\text{ref}} - S_N$ .
    bounds.append((0.0, None))
    a_ub = coo_matrix(
        ([-1.0, -1.0], ([0, 0], [3 * n - 1, nvar - 1])),
        shape=(1, nvar),
    ).tocsc()
    rhs_ub = np.array([-s_ref])

started = perf_counter()
result = linprog(
    cost,
    A_eq=a_eq,
    b_eq=rhs,
    A_ub=a_ub,
    b_ub=rhs_ub,
    bounds=bounds,
    method="highs",
    options={
        "primal_feasibility_tolerance": LP_TOL,
        "dual_feasibility_tolerance": LP_TOL,
    },
)
seconds = perf_counter() - started
if not result.success:
    raise RuntimeError(f"Dispatch LP failed ({result.status}): {result.message}")

# Remove only numerical excursions beyond the explicit per-slot bounds.
c = np.clip(result.x[:n], 0.0, c_upper)
d = np.clip(result.x[n : 2 * n], 0.0, d_upper)
s = np.concatenate([s0, s0 + np.cumsum(b.eta_c * c - d / b.eta_d)])

```

```

if s.min() < b.low - PHYSICAL_TOL or s.max() > b.high + PHYSICAL_TOL:
    raise RuntimeError("LP actions fail the independently accumulated SOC check.")
e, v, r, w = _account_surplus(g, pv, load, c, d)
emergency_cost = float(5.0 * price @ e)
terminal_shortfall = max(float(s_ref - s[-1]), 0.0)
terminal_objective = (
    weight * terminal_shortfall if terminal == "shortfall"
    else -weight * float(s[-1]) if terminal == "value"
    else 0.0
)
objective = emergency_cost + terminal_objective
solver_objective = float(result.fun + 5.0 * price @ deficit)
if abs(objective - solver_objective) > 1e-5 * max(1.0, abs(objective)):
    raise RuntimeError("Recomputed objective does not agree with the LP objective.")
return {
    "c": c, "d": d, "e": e, "v": v, "r": r, "w": w, "s": s,
    "objective": objective,
    "fees_emergency": emergency_cost,
    "terminal": terminal,
    "weight": float(weight),
    "s_ref": float(s_ref),
    "terminal_shortfall": terminal_shortfall,
    "terminal_objective": float(terminal_objective),
    "success": True,
    "status_code": int(result.status),
    "message": str(result.message),
    "solve_seconds": seconds,
    "iterations": int(result.nit),
}

def greedy_step(
    g: float,
    load: float,
    pv: float,
    s0: float,
    b: Battery = DEFAULT_BATTERY,
) → dict:
    """Charge maximum available surplus or discharge maximum feasible deficit."""
    for name, value in (("g", g), ("load", load), ("pv", pv)):
        if not np.isfinite(value) or value < 0:
            raise ValueError(f"{name} must be finite and nonnegative.")
    s0 = _initial_state(s0, b)
    surplus = max(g + pv - load, 0.0)
    deficit = max(load - g - pv, 0.0)
    c = min(surplus, b.rate, max((b.high - s0) / b.eta_c, 0.0))
    d = min(deficit, b.rate, max((s0 - b.low) * b.eta_d, 0.0))
    s_next = s0 + b.eta_c * c - d / b.eta_d
    e, v, r, w = _account_surplus(g, pv, load, c, d)
    return {
        "c": float(c), "d": float(d), "e": float(e),
        "v": float(v), "r": float(r), "w": float(w),
        "s_next": float(s_next),
    }

def simulate_day(
    g,
    load_actual,
    pv_actual,
    load_forecast,
    pv_forecast,
    price,
    s0: float,
    method: Method = "greedy",
    weight: float = 0.0,

```

```

s_ref: float = 6000.0,
b: Battery = DEFAULT_BATTERY,
) → dict:
    """Execute fixed purchases under greedy or receding-horizon control.

    In every solve, only position zero of a fresh forecast suffix is replaced
    with the current actual observation. The optimizer receives no future actual
    values. The contract array is copied and never changed. A general length is
    accepted to support small verification examples; a full day has 144 slots.
    """
    terminal_for_method = {
        "greedy": "none", "mpc_soft": "shortfall",
        "mpc_value": "value", "mpc_none": "none",
    }
    if method not in terminal_for_method:
        raise ValueError(f"Unknown control method: {method}")
    terminal = terminal_for_method[method]
    _terminal_parameters(terminal, weight, s_ref)
    g = _vector(g, "g")
    n = len(g)
    load_actual = _vector(load_actual, "load_actual", n)
    pv_actual = _vector(pv_actual, "pv_actual", n)
    load_forecast = _vector(load_forecast, "load_forecast", n)
    pv_forecast = _vector(pv_forecast, "pv_forecast", n)
    price = _vector(price, "price", n)
    s = np.empty(n + 1)
    s[0] = _initial_state(s0, b)
    actions = {key: np.empty(n) for key in ("c", "d", "e", "v", "r", "w")}
    solve_seconds = 0.0
    solve_count = 0
    for t in range(n):
        if method == "greedy":
            step = greedy_step(g[t], load_actual[t], pv_actual[t], s[t], b)
            for key in actions:
                actions[key][t] = step[key]
            s[t + 1] = step["s_next"]
        else:
            remaining_load = load_forecast[t:].copy()
            remaining_pv = pv_forecast[t:].copy()
            remaining_load[0] = load_actual[t]
            remaining_pv[0] = pv_actual[t]
            horizon = solve_horizon(
                g[t:], remaining_load, remaining_pv, price[t:], s[t],
                b=b, terminal=terminal, weight=weight, s_ref=s_ref,
            )
            for key in actions:
                actions[key][t] = horizon[key][0]
            s[t + 1] = horizon["s"][1]
            solve_seconds += horizon["solve_seconds"]
            solve_count += 1

    fees_plan = float(price @ g)
    fees_emergency = float(5.0 * price @ actions["e"])
    out = {
        **actions, "s": s, "g": g, "price": price,
        "load": load_actual, "pv": pv_actual,
        "load_forecast": load_forecast, "pv_forecast": pv_forecast,
        "method": method, "terminal": terminal, "weight": float(weight),
        "s_ref": float(s_ref), "fees_plan": fees_plan,
        "fees_emergency": fees_emergency, "fees_total": fees_plan + fees_emergency,
        "solve_count": solve_count, "solve_seconds": solve_seconds,
        "success": True, "status_code": 0,
    }
    out["checks"] = check_trajectory(out, b)
    return out

```

```

def offline_oracle(
    g,
    load_actual,
    pv_actual,
    price,
    s0: float,
    b: Battery = DEFAULT_BATTERY,
    terminal: Terminal = "none",
    weight: float = 0.0,
    s_ref: float = 6000.0,
) → dict:
    """Noncausal benchmark using all supplied future actuals in one LP.

    Compare its objective only with the same initial state, horizon, and terminal
    valuation/constraint. With terminal='none', its emergency cash cost is a
    lower bound for causal controllers on exactly this fixed-contract horizon.
    It must never be presented as an implementable online control policy.
    """
    g = _vector(g, "g")
    n = len(g)
    load_actual = _vector(load_actual, "load_actual", n)
    pv_actual = _vector(pv_actual, "pv_actual", n)
    price = _vector(price, "price", n)
    out = solve_horizon(
        g, load_actual, pv_actual, price, s0, b, terminal, weight, s_ref,
    )
    fees_plan = float(price @ g)
    out.update({
        "g": g, "price": price, "load": load_actual, "pv": pv_actual,
        "fees_plan": fees_plan,
        "fees_total": fees_plan + out["fees_emergency"],
        "method": "offline_oracle", "noncausal": True, "solve_count": 1,
    })
    out["checks"] = check_trajectory(out, b)
    return out


def check_trajectory(out: dict, b: Battery = DEFAULT_BATTERY) → dict:
    """Independently check slot energy accounting and battery constraints."""
    g, load, pv, s = (np.asarray(out[key]) for key in ("g", "load", "pv", "s"))
    c, d, e, v, r, w = (np.asarray(out[key]) for key in ("c", "d", "e", "v", "r", "w"))
    surplus = np.maximum(g + pv - load, 0.0)
    deficit = np.maximum(load - g - pv, 0.0)
    violations = {
        "energy_balance_kwh": float(np.max(np.abs(g - v + pv - r + d + e - load - c))),
        "soc_recursion_kwh": float(np.max(np.abs(np.diff(s) - b.eta_c * c + d / b.eta_d))),
        "soc_lower_kwh": float(max(b.low - np.min(s), 0.0)),
        "soc_upper_kwh": float(max(np.max(s) - b.high, 0.0)),
        "charge_power_kwh": float(max(np.max(c) - b.rate, 0.0)),
        "discharge_power_kwh": float(max(np.max(d) - b.rate, 0.0)),
        "charge_surplus_kwh": float(max(np.max(c - surplus), 0.0)),
        "discharge_deficit_kwh": float(max(np.max(d - deficit), 0.0)),
        "simultaneous_charge_discharge_kwh": float(np.max(np.minimum(c, d))),
        "unused_contract_bound_kwh": float(max(np.max(v - g), 0.0)),
        "curtailment_bound_kwh": float(max(np.max(r - pv), 0.0)),
        "w_split_kwh": float(np.max(np.abs(w - v - r))),
        "emergency_deficit_kwh": float(np.max(np.abs(e - deficit + d))),
        "negative_energy_kwh": float(max(-np.min(np.concatenate((c, d, e, v, r, w))), 0.0)),
    }
    worst = max(violations.values())
    return {**violations, "max_violation_kwh": worst, "tolerance_kwh": PHYSICAL_TOL, "passed":
bool(worst ≤ PHYSICAL_TOL)}

```

9. question2/planning/code/control_refined.py

源文件 SHA-256: 2fecb69c4c392f7c2d4b32532564388786d5954b1037181ff171c714c74066ae ; 代码行数: 323。


```
"""Fixed-contract greedy/MPC execution for the Q2 A/B/C/D comparison.
```

```
The public interface is ``run_controller``. Every flow input is already kWh per
ten-minute interval; this module does not parse timestamps or average samples.
All methods share the same physical model and fixed contract. Only current
actuals replace forecasts, under the explicit fast intra-interval feedback
approximation; future actuals are never passed to the optimization solver.
```

```
The MPC primary objective is emergency cash cost plus a soft terminal shortfall.
No small weighted tie-break term is hidden in that objective: stage two imposes
an explicit primary-cost upper bound and maximizes the current discharge.
Positive current surplus is charged as much as physically possible; free
disposal and a nonincreasing terminal penalty make higher stored energy weakly
preferable in this particular model. This rule assumes no degradation cost.
"""
```

```
from __future__ import annotations
```

```
from time import perf_counter
```

```
import numpy as np
```

```
from scipy.optimize import linprog
```

```
from scipy.sparse import coo_matrix, vstack
```

```
from control import (
    Battery, DEFAULT_BATTERY, LP_TOL, PHYSICAL_TOL,
    _initial_state, _vector, check_trajectory, greedy_step, solve_horizon,
    simulate_day,
)
```

```
PRIMARY_ALLOWANCE_YUAN = 1e-7
```

```
PRIMARY_VERIFICATION_TOL_YUAN = 1e-5
```

```
def _dispatch_lp(g, load, pv, price, s0, b, terminal, weight, s_ref):
    """Build the same physical model independently for explicit stage two."""
    g = _vector(g, "g")
    n = len(g)
    load = _vector(load, "load", n)
    pv = _vector(pv, "pv", n)
    price = _vector(price, "price", n)
    s0 = _initial_state(s0, b)
    if terminal not in ("shortfall", "value"):
        raise ValueError("Refined controller supports shortfall or value terminal modes.")
    if not np.isfinite(weight) or weight < 0 or not np.isfinite(s_ref):
        raise ValueError("Invalid terminal weight or reference.")
    a = np.maximum(g + pv - load, 0.0)
    deficit = np.maximum(load - g - pv, 0.0)
    nvar = 3 * n + int(terminal == "shortfall")
    f = np.zeros(nvar)
    f[n : 2 * n] = -5 * price
    if terminal == "shortfall":
        f[-1] = weight
    else:
        f[3 * n - 1] = -weight
    indices = np.arange(n)
    row = np.concatenate((indices, indices, indices, indices[1:]))
    col = np.concatenate((indices, n + indices, 2 * n + indices, 2 * n + indices[:-1]))
    val = np.concatenate((
        np.full(n, -b.eta_c), np.full(n, 1 / b.eta_d),
        np.ones(n), -np.ones(max(n - 1, 0)),
    ))
    equality = coo_matrix((val, (row, col)), shape=(n, nvar)).tocsc()
```

```

equality_rhs = np.zeros(n)
equality_rhs[0] = s0
bounds = (
    [(0., float(z)) for z in np.minimum(a, b.rate)]
    + [(0., float(z)) for z in np.minimum(deficit, b.rate)]
    + [(b.low, b.high)] * n
)
upper = None
upper_rhs = None
if terminal == "shortfall":
    bounds.append((0., None))
    upper = coo_matrix(
        ([[-1., -1.], ([0, 0], [3 * n - 1, nvar - 1])]),
        shape=(1, nvar),
    ).tocsc()
    upper_rhs = np.array([-s_ref])
return {
    "g": g, "load": load, "pv": pv, "price": price,
    "s0": s0, "n": n, "nvar": nvar, "surplus": a, "deficit": deficit,
    "primary_linear": f, "primary_constant": float(5 * price @ deficit),
    "A_eq": equality, "b_eq": equality_rhs,
    "A_ub": upper, "b_ub": upper_rhs, "bounds": bounds,
}

def _solve_built(problem, objective, b, terminal, weight, s_ref,
    primary_ceiling=None, forced_first_charge=None):
    n = problem["n"]
    bounds = list(problem["bounds"])
    if forced_first_charge is not None:
        forced_first_charge = float(forced_first_charge)
        if not bounds[0][0] - PHYSICAL_TOL ≤ forced_first_charge ≤ bounds[0][1] + PHYSICAL_TOL:
            raise ValueError("Forced first charge lies outside the surplus/power bounds.")
        bounds[0] = (forced_first_charge, forced_first_charge)
    upper, rhs = problem["A_ub"], problem["b_ub"]
    if primary_ceiling is not None:
        # This is an explicit epsilon-constraint, not a weighted extra objective.
        ceiling_row = coo_matrix(problem["primary_linear"].reshape(1, -1)).tocsc()
        ceiling_rhs = float(primary_ceiling - problem["primary_constant"])
        upper = ceiling_row if upper is None else vstack((upper, ceiling_row), format="csc")
        rhs = np.array([ceiling_rhs]) if rhs is None else np.append(rhs, ceiling_rhs)
    started = perf_counter()
    result = linprog(
        objective, A_eq=problem["A_eq"], b_eq=problem["b_eq"],
        A_ub=upper, b_ub=rhs, bounds=bounds, method="highs",
        options={"primal_feasibility_tolerance": LP_TOL,
            "dual_feasibility_tolerance": LP_TOL},
    )
    seconds = perf_counter() - started
    if not result.success:
        raise RuntimeError(f"Refinement LP failed ({result.status}): {result.message}")
    c = np.clip(result.x[:n], 0, np.minimum(problem["surplus"], b.rate))
    d = np.clip(result.x[n : 2 * n], 0, np.minimum(problem["deficit"], b.rate))
    s = np.r_[problem["s0"], problem["s0"] + np.cumsum(b.eta_c * c - d / b.eta_d)]
    e = np.maximum(problem["deficit"] - d, 0)
    w = np.maximum(problem["surplus"] - c, 0)
    v = np.minimum(w, problem["g"])
    r = w - v
    terminal_cost = (
        weight * max(s_ref - float(s[-1]), 0.) if terminal == "shortfall"
        else -weight * float(s[-1])
    )
    primary = float(5 * problem["price"] @ e) + terminal_cost
    out = {
        "g": problem["g"], "load": problem["load"], "pv": problem["pv"],
        "c": c, "d": d, "e": e, "v": v, "r": r, "w": w, "s": s,

```

```

        "primary_objective": primary, "solve_seconds": seconds,
        "status_code": int(result.status), "message": str(result.message),
    }
    checks = check_trajectory(out, b)
    if not checks["passed"]:
        raise RuntimeError(f"Refinement physical checks failed: {checks}")
    out["checks"] = checks
    return out

def solve_refined_horizon(
    g, load, pv, price, s0,
    b: Battery = DEFAULT_BATTERY, terminal="shortfall", weight=None,
    s_ref=6000.,
) → dict:
    """Primary optimum, then maximize current d within 1e-7 yuan of that cost."""
    price = _vector(price, "price")
    weight = float(np.min(price) / b.eta_c if weight is None else weight)
    primary = solve_horizon(g, load, pv, price, s0, b, terminal, weight, s_ref)
    problem = _dispatch_lp(g, load, pv, price, s0, b, terminal, weight, s_ref)
    secondary_objective = np.zeros(problem["nvar"])
    secondary_objective[problem["n"]] = -1.0
    out = _solve_built(
        problem, secondary_objective, b, terminal, weight, s_ref,
        primary_ceiling=primary["objective"] + PRIMARY_ALLOWANCE_YUAN,
    )
    error = out["primary_objective"] - primary["objective"]
    if abs(error) > PRIMARY_VERIFICATION_TOL_YUAN:
        raise RuntimeError(f"Secondary stage changed primary objective by {error} yuan.")
    out.update(
        primary_optimum=primary["objective"], primary_cost_error_yuan=float(error),
        solve_count=2,
        solve_seconds=out["solve_seconds"] + primary["solve_seconds"],
        success=True,
    )
    return out

def forced_charge_primary_objective(
    g, load, pv, price, s0, forced_first_charge,
    b: Battery = DEFAULT_BATTERY, terminal="shortfall", weight=None, s_ref=6000.,
) → dict:
    """Verification helper: primary LP with a prescribed first charge amount.

    It is not used by the online controller or for policy selection.
    """
    price = _vector(price, "price")
    weight = float(np.min(price) / b.eta_c if weight is None else weight)
    problem = _dispatch_lp(g, load, pv, price, s0, b, terminal, weight, s_ref)
    return _solve_built(
        problem, problem["primary_linear"], b, terminal, weight, s_ref,
        forced_first_charge=forced_first_charge,
    )

def simulate_refined_day(
    g, load_actual, pv_actual, load_forecast, pv_forecast, price, s0,
    terminal="shortfall", weight=None, s_ref=6000., b: Battery = DEFAULT_BATTERY,
) → dict:
    """Causal replay using maximal surplus charging and explicit LP refinement.

    Current actual values replace the first element of a future forecast suffix.
    No future actual observation is passed to either LP. Neutral/surplus slots
    need no optimization; deficit slots each use two LP solves.
    """
    g = _vector(g, "g")

```

```

n = len(g)
load_actual = _vector(load_actual, "load_actual", n)
pv_actual = _vector(pv_actual, "pv_actual", n)
load_forecast = _vector(load_forecast, "load_forecast", n)
pv_forecast = _vector(pv_forecast, "pv_forecast", n)
price = _vector(price, "price", n)
if terminal not in ("shortfall", "value"):
    raise ValueError("terminal must be shortfall or value.")
weight = float(np.min(price) / b.eta_c if weight is None else weight)
if not np.isfinite(weight) or weight < 0 or not np.isfinite(s_ref):
    raise ValueError("Invalid terminal weight or reference.")
s = np.empty(n + 1)
s[0] = _initial_state(s0, b)
actions = {key: np.empty(n) for key in ("c", "d", "e", "v", "r", "w")}
solve_count = 0
solve_seconds = 0.0
primary_errors = []
forced_charge_steps = 0
neutral_steps = 0
for t in range(n):
    net = g[t] + pv_actual[t] - load_actual[t]
    if net ≥ 0:
        step = greedy_step(g[t], load_actual[t], pv_actual[t], s[t], b)
        for key in actions:
            actions[key][t] = step[key]
        s[t + 1] = step["s_next"]
        if net > 0:
            forced_charge_steps += 1
        else:
            neutral_steps += 1
        continue
    remaining_load = load_forecast[t:].copy()
    remaining_pv = pv_forecast[t:].copy()
    remaining_load[0] = load_actual[t]
    remaining_pv[0] = pv_actual[t]
    horizon = solve_refined_horizon(
        g[t:], remaining_load, remaining_pv, price[t:], s[t],
        b=b, terminal=terminal, weight=weight, s_ref=s_ref,
    )
    for key in actions:
        actions[key][t] = horizon[key][0]
    s[t + 1] = horizon["s"][1]
    solve_count += horizon["solve_count"]
    solve_seconds += horizon["solve_seconds"]
    primary_errors.append(horizon["primary_cost_error_yuan"])
fees_plan = float(price @ g)
fees_emergency = float(5 * price @ actions["e"])
out = {
    **actions, "g": g, "s": s, "price": price,
    "load": load_actual, "pv": pv_actual,
    "load_forecast": load_forecast, "pv_forecast": pv_forecast,
    "method": "refined-soft" if terminal == "shortfall" else "refined-value",
    "terminal": terminal, "weight": weight, "s_ref": float(s_ref),
    "fees_plan": fees_plan, "fees_emergency": fees_emergency,
    "fees_total": fees_plan + fees_emergency,
    "solve_count": solve_count, "solve_seconds": solve_seconds,
    "max_primary_cost_error_yuan": max([abs(x) for x in primary_errors] or [0.]),
    "max_primary_cost_increase_yuan": max(primary_errors + [0.]),
    "primary_allowance_yuan": PRIMARY_ALLOWANCE_YUAN,
    "primary_verification_tolerance_yuan": PRIMARY_VERIFICATION_TOL_YUAN,
    "forced_surplus_charge_steps": forced_charge_steps,
    "neutral_steps": neutral_steps, "two_stage_steps": len(primary_errors),
    "success": True, "status_code": 0,
}
out["checks"] = check_trajectory(out, b)
if not out["checks"]["passed"]:

```

```

        raise RuntimeError(f"Refined replay trajectory checks failed: {out['checks']}")
    return out

def run_controller(
    method, g, load_actual, pv_actual, load_forecast, pv_forecast, price, s0,
    terminal_weight=None, s_ref=6000., b: Battery = DEFAULT_BATTERY,
) → dict:
    """Run one supplied horizon using a fixed ordinary purchase contract.

    ``method`` is ``greedy`` or ``mpc``. Inputs g/load/pv are interval energies
    (kWh), price is yuan/kWh, and s0 is measured internal stored energy (kWh).
    A full production day contains exactly 144 slots; shorter horizons are
    accepted here for independent verification. Input/output boundary handling
    and selection of a day's 144 intervals belong to the data interface.

    The default MPC kappa is min(full_day_price) / eta_c and stays fixed in every
    shrinking suffix solve. It is a candidate terminal-valuation setting, not
    a competition rule or a parameter selected from future outcomes. A caller
    should pass one calibrated value explicitly when conducting sensitivity
    studies. The terminal penalty is never included in fees_total.

    Actual state is carried across days by passing the previous result's s[-1]
    as the next s0. This function never resets state to 6000 automatically.
    Returned arrays c/d/e/v/r/w have n elements and s has n+1 boundaries.
    """
    if method not in ("greedy", "mpc"):
        raise ValueError("method must be 'greedy' or 'mpc'.")
    p = _vector(price, "price")
    weight = float(np.min(p) / b.eta_c if terminal_weight is None else terminal_weight)
    if not np.isfinite(weight) or weight < 0 or not np.isfinite(s_ref):
        raise ValueError("Terminal weight and reference must be finite; weight ≥ 0.")
    contract_before = _vector(g, "g", len(p))
    if method == "greedy":
        out = simulate_day(
            contract_before, load_actual, pv_actual, load_forecast, pv_forecast,
            p, s0, method="greedy", weight=0., s_ref=s_ref, b=b,
        )
    else:
        out = simulate_refined_day(
            contract_before, load_actual, pv_actual, load_forecast, pv_forecast,
            p, s0, terminal="shortfall", weight=weight, s_ref=s_ref, b=b,
        )
    contract_unchanged = bool(np.array_equal(out["g"], contract_before))
    out["checks"]["fixed_contract_unchanged"] = contract_unchanged
    out["checks"]["passed"] = bool(out["checks"]["passed"] and contract_unchanged)
    out.update(
        method=method,
        terminal_weight=weight if method == "mpc" else 0.,
        terminal_shortfall_kwh=max(float(s_ref - out["s"][-1]), 0.),
        terminal_penalty_yuan=(weight * max(float(s_ref - out["s"][-1]), 0.)
                               if method == "mpc" else 0.),
        observation_assumption="current_interval_mean_as_fast_feedback_proxy",
        future_inputs="day_ahead_forecast_only",
        execution_rule="execute_first_step_then_reobserve",
        surplus_bookkeeping="use_PV_first_then_paid_grid; unused_paid_grid_has_no_refund",
        input_flow_units="kWh_per_10min_interval",
    )
    if not out["checks"]["passed"]:
        raise RuntimeError(f"Controller physical checks failed: {out['checks']}")
    return out

```

10. question2/planning/code/evaluate_reports.py

源文件 SHA-256: 04ffa7c73ab3c3866cccf2cf93648e259baa41dca06e11d843a72858dc2d133f ; 代码行数: 95。

```

"""Independent accounting checks and paired day-block cost diagnostics."""
import argparse
import csv
import json
from pathlib import Path
import numpy as np

def rows(path):
    with Path(path).open(encoding='utf-8-sig', newline='') as f:
        return list(csv.DictReader(f))

def paired_block_interval(values, block=7, draws=1000, seed=20260911):
    """Circular moving-block bootstrap; paired daily differences, not raw slots.

    This is a conditional finite-sample diagnostic, not a guarantee of future
    cost or a correction for all policy-state dependence/nonstationarity.
    """
    x = np.asarray(values, float)
    if len(x) < 2:
        return None
    block = min(block, len(x)); rng = np.random.default_rng(seed)
    means = []
    for _ in range(draws):
        starts = rng.integers(0, len(x), size=int(np.ceil(len(x)/block)))
        indices = ((starts[:, None] + np.arange(block)) % len(x)).ravel()[:len(x)]
        means.append(float(x[indices].mean()))
    return np.quantile(means, [.025, .975]).tolist()

def audit(out):
    out = Path(out); summary = json.loads((out/'summary.json').read_text())
    daily = rows(out/'stage3_continuous_daily.csv')
    trace = rows(out/'stage3_interval_trace.csv')
    maxima = {'energy_balance_kwh': 0., 'soc_transition_kwh': 0., 'soc_link_kwh': 0.,
              'daily_bill_yuan': 0., 'interval_bill_yuan': 0., 'bound_violation': 0.}
    ledger = {}; last = {}
    for r in trace:
        k = r['method']; date = r.get('plan_date', r['interval_start'][:10]); key = (k, date)
        g, c, d, e, v, pv_dump = [float(r[f]) for f in
('g_kwh', 'charge_kwh', 'discharge_kwh', 'emergency_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh')]
        load, pv = float(r['load_actual_kw'])/6, float(r['pv_actual_kw'])/6
        s0, s1 = float(r['soc_start_kwh']), float(r['soc_end_kwh'])
        p = float(r['price_yuan_per_kwh']); cost = float(r['cash_cost_yuan'])
        maxima['energy_balance_kwh'] = max(maxima['energy_balance_kwh'], abs(g-v+pv-pv_dump+d+e-load-c))
        maxima['soc_transition_kwh'] = max(maxima['soc_transition_kwh'], abs(s1-s0-.9*c+d/.9))
        maxima['interval_bill_yuan'] = max(maxima['interval_bill_yuan'], abs(cost-p*g-5*p*e))
        a = max(g+pv-load, 0); deficit = max(load-g-pv, 0)
        maxima['bound_violation'] = max(maxima['bound_violation'],
1200-s0, 1200-s1, s0-10800, s1-10800, c-5000/6, d-5000/6, c-a, d-deficit,
abs(e-deficit+d), v-g, pv_dump-pv, -min(g, c, d, e, v, pv_dump))
        if k in last:
            maxima['soc_link_kwh'] = max(maxima['soc_link_kwh'], abs(s0-last[k]))
        last[k] = s1
        ledger.setdefault(key, []).append(cost)
    for r in daily:
        values = ledger[(r['method'], r['date'])]
        if len(values) != 144:
            raise AssertionError('Executed day must have exactly 144 intervals')
        maxima['daily_bill_yuan'] = max(maxima['daily_bill_yuan'], abs(sum(values)-
float(r['cash_cost_yuan'])))
        if not all(v < 1e-4 for v in maxima.values()):
            raise AssertionError(f'Independent ledger audit failed: {maxima}')

```

```

comparisons=[]
for filename,base,candidates in [('stage1_same_contract_daily.csv','A',['B']),
                                  ('stage2_dayahead_daily.csv','B',['C','D'])]:
    src=rows(out/filename);ref={r['date']:r for r in src if r['method']==base}
    for candidate in candidates:
        cand=sorted([r for r in src if r['method']==candidate],key=lambda r:r['date'])
        for metric in ['cash_cost_yuan','inventory_adjusted_cost_yuan']:
            diff=[]
            for r in cand:
                b=ref[r['date']]
                assert abs(float(r['initial_soc_kwh'])-float(b['initial_soc_kwh']))<1e-6
                if filename.startswith('stage1'):
                    assert abs(float(r['planned_cost_yuan'])-float(b['planned_cost_yuan']))<1e-5
                    assert abs(float(r['planned_kwh'])-float(b['planned_kwh']))<1e-5
                diff.append(float(r[metric])-float(b[metric]))
            comparisons.append({'comparison':candidate+' minus '+base,'metric':metric,
                              'days':len(diff),'mean_daily_difference_yuan':float(np.mean(diff)),
                              'candidate_lower_fraction':float(np.mean(np.asarray(diff)<-1e-5)),
                              'block_length_days':min(7,len(diff)),'bootstrap_draws':1000,
                              'conditional_95pct_mean_interval_yuan':paired_block_interval(diff),
                              'small_sample_caution':len(diff)<30})
    result={'data_role':summary['data_role'],'independent_checks_passed':True,
           'max_absolute_residuals':maxima,'paired_local_comparisons':comparisons,
           'interpretation':'Negative difference favors candidate only for these supplied inputs and
declared data assumptions. Bootstrap is descriptive, not a guarantee across seasons or forecast
versions.'}

(out/'independent_evaluation.json').write_text(json.dumps(result,ensure_ascii=False,indent=2,allow_nan=False),encoding='utf-8')
print(json.dumps({'independent_checks_passed':True,'max_residuals':maxima}))
return result

if __name__=='__main__':
    p=argparse.ArgumentParser(description=__doc__)
    p.add_argument('result_directory',type=Path)
    audit(p.parse_args().result_directory)

```

11. question2/planning/code/input_io.py

源文件 SHA-256: c3555f7ba362b17ce712fef8bd6f5dd74459c76a773dad4cdc624e66aa019fc5 ; 代码行数: 200。


```

"""Explicit interval interfaces. No inference of point/mean from column count."""
from __future__ import annotations
import csv
import datetime as dt
import hashlib
import json
from pathlib import Path
import re
import numpy as np
from openpyxl import load_workbook

FORECAST_FIELDS = ['plan_date', 'issued_at', 'interval_start', 'interval_end',
                   'load_forecast_kw', 'pv_forecast_kw']
ACTUAL_FIELDS = ['interval_start', 'interval_end', 'load_actual_kw', 'pv_actual_kw']

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def timestamp(value):
    x = value if isinstance(value, dt.datetime) else dt.datetime.fromisoformat(str(value))
    if x.tzinfo is not None:
        x = x.astimezone(dt.timezone(dt.timedelta(hours=8))).replace(tzinfo=None)
    if x.second or x.microsecond:
        raise ValueError('Timestamp must be on a minute boundary')
    return x

def clock_minute(value):
    if isinstance(value, dt.time):
        return value.hour*60+value.minute
    s = str(value).strip().replace(':', '')
    tomorrow = '次日' in s or '+1' in s
    s = s.replace('次日', '').replace('+1', '').strip()
    match = re.fullmatch(r'(\d{1,2}):(\d{2})', s)
    if not match:
        raise ValueError(f'Unrecognized clock {value!r}')
    h, m = map(int, match.groups())
    if h > 24 or m > 59 or (h == 24 and m):
        raise ValueError('Invalid clock')
    return h*60+m+(1440 if tomorrow and h < 24 else 0)

def interval_header(value):
    # Specific alias explicitly confirmed by the user for the supplied format.
    if str(value).strip() == '次日00:00':
        return 1430, 1440
    parts = re.split(r'[-~\~\~\~\~\~\~]', str(value).strip())
    if len(parts) != 2:
        raise ValueError(f'Explicit interval required, got {value!r}')
    a, b = map(clock_minute, parts)
    if b == 0 and a == 1430:
        b = 1440
    if b-a != 10:
        raise ValueError(f'Not a ten-minute interval: {value!r}')
    return a, b

def read_wide_means(path, sheet=None):
    """User format: date + 145 interval means; ambiguous midnight alias disclosed."""
    w = load_workbook(path, read_only=True, data_only=True)
    try:
        if sheet is None and len(w.sheetnames) != 1:
            raise ValueError('Workbook has multiple sheets; explicitly choose one')
        s = w[sheet] if sheet else w.worksheets[0]
        rows = list(s.values)
        bounds = [interval_header(h) for h in rows[0][1:]]
        if bounds != [(i*10, i*10+10) for i in range(145)]:

```

```

        raise ValueError('Expected 144 current-day intervals plus next-day first interval')
    dates = [timestamp(r[0]).date() for r in rows[1:]]
    if dates != sorted(set(dates)):
        raise ValueError('Dates must be unique and sorted')
    values = np.asarray([r[1:] for r in rows[1:]], dtype=float)
    if values.shape != (len(dates), 145) or not np.isfinite(values).all() or (values < 0).any():
        raise ValueError('Invalid or missing interval-average powers')
    return dates, values, {'filename': Path(path).name, 'sha256': sha(path),
                           'sheet': s.title, 'days': len(dates), 'intervals_per_issue': 145,
                           'alias': {'次日00:00': '23:50-24:00 (user confirmed)'},
                           'value_kind': 'interval_mean_kw', 'no_averaging_performed': True}
    finally:
        w.close()

def read_price(path):
    w = load_workbook(path, read_only=True, data_only=True)
    try:
        rows = list(w.worksheets[0].values)
    finally:
        w.close()
    if len(rows) != 145 or tuple(rows[0][2:]) != ('时间', '电价'):
        raise ValueError('Attachment1 headers/row count differ')
    if [clock_minute(r[0]) for r in rows[1:]] != list(range(10, 1441, 10)):
        raise ValueError('Attachment1 instantaneous nodes must be 00:10 through 24:00')
    p = np.asarray([r[1] for r in rows[1:]], float)
    if not np.isfinite(p).all() or (p <= 0).any():
        raise ValueError('Prices must be positive')
    nodes = np.r_[p[-1], p]
    mean = (nodes[:-1] + nodes[1:])/2
    return mean, {'filename': Path(path).name, 'sha256': sha(path),
                  'source_kind': 'instantaneous_price', 'zero_boundary': 'periodic p(00:00)=p(24:00)',
                  'price_nodes': 145, 'mean_intervals': 144, 'price_first': float(mean[0]),
                  'price_last': float(mean[-1]), 'settlement': 'interval mean price approximation'}

def write_csv(path, records, fields=None):
    records = list(records)
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    with Path(path).open('w', newline='', encoding='utf-8-sig') as f:
        w = csv.DictWriter(f, fieldnames=fields or list(records[0]))
        w.writeheader()
        w.writerows(records)

def pair_forecast_workbooks(load_path, pv_path, out, load_sheet=None, pv_sheet=None):
    """Conversion assumes user-declared midnight issue. It does not prove provenance."""
    if Path(load_path).resolve() == Path(pv_path).resolve() and load_sheet == pv_sheet:
        raise ValueError('Load and PV must be distinct explicitly selected sheets')
    dl, l, lm = read_wide_means(load_path, load_sheet)
    dg, g, gm = read_wide_means(pv_path, pv_sheet)
    if Path(load_path).resolve() == Path(pv_path).resolve() and lm['sheet'] == gm['sheet']:
        raise ValueError('Load and PV resolved to the same sheet')
    if dl != dg:
        raise ValueError('Load/PV forecast dates do not match')
    records = []
    for i, day in enumerate(dl):
        zero = dt.datetime.combine(day, dt.time())
        for t in range(145):
            start = zero + dt.timedelta(minutes=10*t)
            records.append(dict(plan_date=day.isoformat(), issued_at=zero.isoformat(),
                                interval_start=start.isoformat(), interval_end=
                                (start + dt.timedelta(minutes=10)).isoformat(),
                                load_forecast_kw=l[i, t], pv_forecast_kw=g[i, t]))
        write_csv(out, records, FORECAST_FIELDS)
    meta = {'load': lm, 'pv': gm, 'issuance_basis': 'user declaration: issued by plan-date midnight; not
    verified from generator',
            'extra_interval': 'next-day first interval retained; excluded from daily optimization'}

```

```

Path(out).with_suffix('.metadata.json').write_text(json.dumps(meta,ensure_ascii=False,indent=2),encoding
='utf-8')

def load_bundle(forecasts_csv,actual_csv):
    """Strict gate: explicit interval targets, complete 145 forecasts / 144 actuals."""
    provenance={}
    for label,path in [('forecast',Path(forecasts_csv)),('actual',Path(actual_csv))]:
        sidecar=path.with_suffix('.metadata.json')
        if sidecar.exists():
            provenance[label]=json.loads(sidecar.read_text(encoding='utf-8'))
        manifest=path.parent/'synthetic_manifest.json'
        if manifest.exists():
            provenance[label+'-_directory_manifest']=json.loads(manifest.read_text(encoding='utf-8'))
    known_synthetic=any(isinstance(m,dict) and
        (str(m.get('role','')).startswith('synthetic') or m.get('data_role')=='synthetic')
        for m in provenance.values())
    role='synthetic' if known_synthetic else 'supplied_unverified'
    with Path(forecasts_csv).open(encoding='utf-8-sig',newline='') as f:
        rows=list(csv.DictReader(f))
    if not rows or not set(FORECAST_FIELDS).issubset(rows[0]):
        raise ValueError('Forecast CSV must contain paired load/PV, plan_date, issued_at,
interval_start/end')
    groups={}
    for r in rows:
        day=dt.date.fromisoformat(r['plan_date'])
        zero=dt.datetime.combine(day,dt.time())
        issue=timestamp(r['issued_at'])
        a,b=timestamp(r['interval_start']),timestamp(r['interval_end'])
        if issue>zero or b-a≠dt.timedelta(minutes=10):
            raise ValueError('Forecast was issued after the decision or has invalid duration')
        index=int((a-zero).total_seconds()/600)
        if a-zero≠dt.timedelta(minutes=index*10) or not 0≤index<145:
            raise ValueError('Forecast interval outside its 145-slot target')
        cells=groups.setdefault(day,{})
        if index in cells:
            raise ValueError('Duplicate forecast target/version; select the actual midnight version
explicitly')
        cells[index]=(float(r['load_forecast_kw']),float(r['pv_forecast_kw']))
    days=sorted(groups)
    if any(days[i]-days[i-1]≠dt.timedelta(days=1) for i in range(1,len(days))):
        raise ValueError('Forecast days must be consecutive for stateful experiments')
    for day in days:
        if set(groups[day])≠set(range(145)):
            raise ValueError(f'{day}: incomplete 145 forecast intervals')
    with Path(actual_csv).open(encoding='utf-8-sig',newline='') as f:
        raw=list(csv.DictReader(f))
    if not raw or not set(ACTUAL_FIELDS).issubset(raw[0]):
        raise ValueError('Actual CSV must use explicit interval_start/end and paired load/PV')
    actual={}
    for r in raw:
        a,b=timestamp(r['interval_start']),timestamp(r['interval_end'])
        if b-a≠dt.timedelta(minutes=10) or a.minute%10:
            raise ValueError('Invalid actual interval')
        if a in actual:
            raise ValueError('Duplicate actual interval')
        actual[a]=(float(r['load_actual_kw']),float(r['pv_actual_kw']))
    fdata=np.asarray([[groups[day][i] for i in range(145)] for day in days])
    adata=[]
    for day in days:
        zero=dt.datetime.combine(day,dt.time())
        keys=[zero+dt.timedelta(minutes=10*i) for i in range(144)]
        if any(k not in actual for k in keys):
            raise ValueError(f'{day}: actual data missing a current-day interval')
        adata.append([actual[k] for k in keys])
    adata=np.asarray(adata)
    if not np.isfinite(fdata).all() or not np.isfinite(adata).all() or (fdata<0).any() or

```

```
(adata<0).any():
    raise ValueError('Missing, negative or nonfinite power value')
return {'dates':days,'load_f':fdata[:,144,0],'pv_f':fdata[:,144,1],
        'extra_load_f':fdata[:,144,0],'extra_pv_f':fdata[:,144,1],
        'load_a':adata[:,0],'pv_a':adata[:,1],
        'metadata':{'forecast_sha256':sha(forecasts_csv),'actual_sha256':sha(actual_csv),
                    'source_role':role,'source_provenance':provenance,
                    'time_zone':'Asia/Shanghai','value_kind':'interval_mean_kw','daily_intervals':144,
                    'forecast_intervals_per_issue':145,'extra_interval_used_for_dispatch':False,
                    'actual_feedback':'current interval mean used as fast within-interval response
approximation',
                    'issuance_check':'declared issued_at ≤ plan_date midnight; generator causality must be
audited separately'}}
```

12. question2/planning/code/planners.py

源文件 SHA-256: b7dbef04fb514f00f89b9c08bcd963e9ae33202179e7364bb024ebce19850aa ; 代码行数: 410。

```
"""Unified day-ahead MILP for point forecasts, margins and finite scenarios.
```

All supplied and returned flows are interval ENERGY (kWh), not power. The caller must align actual intervals and convert interval-average kW to kWh once. This module does not handle timestamps, estimate forecast residuals, choose margins, or access actual future data. It accepts an already constructed scenario set.

The ordinary purchase vector `g` is shared by every scenario. Scenario recourse can foresee that scenario's entire day, so its expected cost is a two-stage approximation, not the cash expense of a causal intraday controller. Ordinary energy may be unused without refund; emergency energy may only fill a deficit.

```
from __future__ import annotations
```

```
from time import perf_counter
import warnings
```

```
import numpy as np
from scipy.optimize import Bounds, LinearConstraint, linprog, milp
from scipy.sparse import coo_matrix, vstack
```

```
from control import Battery, DEFAULT_BATTERY
```

```
PHYSICAL_TOL_KWH = 1e-5
INTEGRALITY_TOL = 1e-5
OBJECTIVE_TOL_YUAN = 1e-4
MIP_FEASIBILITY_TOL = 1e-9
LP_FEASIBILITY_TOL = 1e-9
```

```
def _fixed_mode_polish(objective, matrix, row_lower, row_upper, lower, upper,
                      integer, incumbent, time_limit):
    """Reoptimize continuous variables with exact rounded incumbent modes.

    The original objective, rows and continuous-variable bounds are retained.
    In particular g stays optimizable except for an inherited fixed prefix.
    This removes big-M leakage caused by almost-integral binary values; it does
    not weaken physical tolerances or round charge/discharge flows by hand.
    """
    mask = integer.astype(bool)
    rounded = np.rint(incumbent[mask])
    integer_error = float(np.max(np.abs(incumbent[mask] - rounded)))
    if (integer_error > INTEGRALITY_TOL or np.any(rounded < lower[mask])
        or np.any(rounded > upper[mask])):
        raise RuntimeError(f"MILP incumbent has unacceptable integrality error: {integer_error}")
    fixed_lower, fixed_upper = lower.copy(), upper.copy()
    fixed_lower[mask] = fixed_upper[mask] = rounded
    equal = np.isfinite(row_lower) & np.isfinite(row_upper) & (row_lower == row_upper)
    hi = np.isfinite(row_upper) & ~equal
    lo = np.isfinite(row_lower) & ~equal
    a_ub = vstack((matrix[hi], -matrix[lo]), format="csc")
    b_ub = np.r_[row_upper[hi], -row_lower[lo]]
    started = perf_counter()
    polished = linprog(
        c=objective, A_ub=a_ub, b_ub=b_ub,
        A_eq=matrix[equal], b_eq=row_lower[equal],
        bounds=np.column_stack((fixed_lower, fixed_upper)), method="highs-ds",
        options={"time_limit": float(time_limit), "disp": False,
                 "primal_feasibility_tolerance": LP_FEASIBILITY_TOL,
                 "dual_feasibility_tolerance": LP_FEASIBILITY_TOL},
    )
    seconds = perf_counter() - started
```

```

if (polished.status != 0 or not polished.success or polished.x is None
    or not np.isfinite(polished.x).all() or polished.fun is None
    or not np.isfinite(polished.fun)):
    raise RuntimeError(
        "Fixed-mode LP polishing failed; no unpolished plan is accepted: "
        f"status={polished.status}, message={polished.message}, "
        f"solve_seconds={seconds}, original_integrality_error={integer_error}"
    )
return polished, seconds, integer_error

def _vector(value, name, n=None):
    out = np.asarray(value, dtype=float).copy()
    if out.ndim != 1 or len(out) == 0 or (n is not None and len(out) != n):
        raise ValueError(f"{name} must be a nonempty vector with the expected length.")
    if not np.isfinite(out).all() or np.any(out < 0):
        raise ValueError(f"{name} must contain only finite nonnegative values.")
    return out

def _scenario_array(value, name, n):
    out = np.asarray(value, dtype=float).copy()
    if out.ndim != 2 or out.shape[0] == 0 or out.shape[1] != n:
        raise ValueError(f"{name} must have shape (scenario_count, {n}).")
    if not np.isfinite(out).all() or np.any(out < 0):
        raise ValueError(f"{name} must contain finite nonnegative interval energies.")
    return out

def solve_plan(
    load_kwh,
    pv_kwh,
    price,
    s0,
    *,
    scenario_load_kwh=None,
    scenario_pv_kwh=None,
    probabilities=None,
    fixed_prefix_g=None,
    b: Battery | None = None,
    terminal_weight=None,
    s_ref=6000.0,
    time_limit=30.0,
    mip_rel_gap=1e-5,
) → dict:
    """Solve a shared-contract point/scenario MILP with soft terminal inventory.

```

Point forecasts (or caller-supplied risk-adjusted point forecasts) become one scenario when both scenario arguments are absent. Otherwise load and PV scenarios must both be supplied with matching (m, n) shapes. Supplied probabilities must sum to one; omitted probabilities are uniform.

fixed_prefix_g optionally fixes the first k ordinary-purchase entries to an already paid contract. The supplied horizon can therefore contain that prefix followed by a new 144-slot window (145 slots for a one-slot prefix). The input s0 is the observed state at the beginning of the full horizon; prefix load/PV must still be forecasts/scenarios, not future actuals. Prefix end states remain scenario-dependent planned states, not observations.

plan_cost and objective include the whole horizon's ordinary purchase cost, including the fixed-prefix constant. fixed_prefix_plan_cost and new_plan_cost expose the exact split so the caller can avoid billing an already paid prefix a second time. All flow/state arrays retain the prefix.

Objective = price@g + E[5*price@emergency + kappa*max(s_ref-S_end, 0)]. kappa defaults to min(price)/charge_efficiency. The terminal penalty is not

cash electricity expense. States are continuous across days through `s0`; no daily terminal equality is imposed.

A feasible time-limited solution may be returned only after all physical constraints have been checked. Read `status/optimality_proven/mip_gap` before describing its optimization accuracy. Failure without a usable incumbent raises `RuntimeError`.

Every accepted MILP incumbent is followed by an LP with its rounded binary modes fixed exactly. Each solver gets `time_limit` separately; `solve_seconds` includes both stages. Final gap uses the polished objective and the original MILP global dual bound, never the restricted LP's dual objective.

```

"""
battery = DEFAULT_BATTERY if b is None else b
load = _vector(load_kwh, "load_kwh")
n = len(load)
pv = _vector(pv_kwh, "pv_kwh", n)
p = _vector(price, "price", n)
prefix = np.asarray([] if fixed_prefix_g is None else fixed_prefix_g, dtype=float).copy()
if prefix.ndim != 1 or len(prefix) > n or not np.isfinite(prefix).all() or np.any(prefix < 0):
    raise ValueError("fixed_prefix_g must be a finite nonnegative vector no longer than the
horizon.")
prefix_length = len(prefix)
if (scenario_load_kwh is None) != (scenario_pv_kwh is None):
    raise ValueError("Supply both scenario_load_kwh and scenario_pv_kwh, or neither.")
if scenario_load_kwh is None:
    scenario_load, scenario_pv = load[None, :], pv[None, :]
else:
    scenario_load = _scenario_array(scenario_load_kwh, "scenario_load_kwh", n)
    scenario_pv = _scenario_array(scenario_pv_kwh, "scenario_pv_kwh", n)
    if scenario_load.shape != scenario_pv.shape:
        raise ValueError("Load and PV scenarios must have identical shapes.")
m = scenario_load.shape[0]
if probabilities is None:
    probability = np.full(m, 1.0 / m)
else:
    probability = _vector(probabilities, "probabilities", m)
    if abs(float(probability.sum()) - 1.0) > 1e-8:
        raise ValueError("Scenario probabilities must sum to one.")
    probability /= probability.sum()
s0 = float(s0)
if not np.isfinite(s0) or not battery.low - PHYSICAL_TOL_KWH <= s0 <= battery.high +
PHYSICAL_TOL_KWH:
    raise ValueError("s0 is outside the battery operating range.")
s0 = float(np.clip(s0, battery.low, battery.high))
s_ref = float(s_ref)
kappa = float(np.min(p) / battery.eta_c if terminal_weight is None else terminal_weight)
if not np.isfinite(s_ref) or s_ref < 0 or not np.isfinite(kappa) or kappa < 0:
    raise ValueError("Terminal reference and terminal weight must be finite and nonnegative.")
if not np.isfinite(time_limit) or time_limit <= 0:
    raise ValueError("time_limit must be finite and positive.")
if not np.isfinite(mip_rel_gap) or not 0 <= mip_rel_gap < 1:
    raise ValueError("mip_rel_gap must be in [0, 1).")

rate = float(battery.rate)
# Proven sufficient cap: buying max scenario load plus full possible charge
# can serve every scenario; additional ordinary energy would only be wasted.
# Nonnegative purchase prices and no reward for wasting energy justify this
# finite cap. It is a modelling bound, not an assumed grid-connection limit.
g_upper = np.max(scenario_load, axis=0) + rate
# An inherited contract may exceed today's sufficient optimization bound.
# Honor it in full, and enlarge disposal bounds/mode coefficients as well.
# Unused contracted energy is allowed, so excess is feasible and not refunded.
if prefix_length:
    g_upper[:prefix_length] = np.maximum(g_upper[:prefix_length], prefix)
scene_width = 7 * n + 1

```



```

nvar = n + m * scene_width
objective = np.zeros(nvar)
objective[:n] = p
lower = np.zeros(nvar)
upper = np.full(nvar, np.inf)
upper[:n] = g_upper
lower[:prefix_length] = prefix
upper[:prefix_length] = prefix
integer = np.zeros(nvar, dtype=np.uint8)
row_indices, column_indices, coefficients = [], [], []
constraint_lower, constraint_upper = [], []

def add_row(entries, lo=-np.inf, hi=np.inf):
    row = len(constraint_lower)
    for column, value in entries:
        if value:
            row_indices.append(row)
            column_indices.append(column)
            coefficients.append(value)
    constraint_lower.append(lo)
    constraint_upper.append(hi)

for scenario in range(m):
    base = n + scenario * scene_width
    # Scenario blocks: c, d, e, v, r, S_1..S_n, u, then xi.
    upper[base : base + 2 * n] = rate
    upper[base + 2 * n : base + 3 * n] = scenario_load[scenario]
    upper[base + 3 * n : base + 4 * n] = g_upper
    upper[base + 4 * n : base + 5 * n] = scenario_pv[scenario]
    lower[base + 5 * n : base + 6 * n] = battery.low
    upper[base + 5 * n : base + 6 * n] = battery.high
    upper[base + 6 * n : base + 7 * n] = 1.0
    integer[base + 6 * n : base + 7 * n] = 1
    xi = base + 7 * n
    upper[xi] = max(s_ref - battery.low, 0.0)
    objective[base + 2 * n : base + 3 * n] = probability[scenario] * 5.0 * p
    objective[xi] = probability[scenario] * kappa
    for t in range(n):
        c, d, e, v, r, s, u = (base + block * n + t for block in range(7))
        L, G = float(scenario_load[scenario, t]), float(scenario_pv[scenario, t])
        # g + PV - v - r + d + e = load + c.
        add_row([(t, 1.), (c, -1.), (d, 1.), (e, 1.), (v, -1.), (r, -1.)], L - G, L - G)
        soc_entries = [(c, -battery.eta_c), (d, 1.0 / battery.eta_d), (s, 1.)]
        if t:
            soc_entries.append((s - 1, -1.))
        soc_rhs = s0 if t == 0 else 0.0
        add_row(soc_entries, soc_rhs, soc_rhs)
        add_row([(c, 1.), (u, -rate)], hi=0.)
        add_row([(d, 1.), (u, rate)], hi=rate)
        # e ≤ load*(1-u); no emergency energy in charging mode.
        add_row([(e, 1.), (u, L)], hi=L)
        # v+r ≤ (g_upper+PV)*u; no disposal in deficit mode.
        # Together with balance this makes charging require an ordinary
        # surplus, and discharging require an ordinary supply deficit.
        add_row([(v, 1.), (r, 1.), (u, -(g_upper[t] + G))], hi=0.)
        add_row([(v, 1.), (t, -1.)], hi=0.)
    last_state = base + 6 * n - 1
    add_row([(last_state, -1.), (xi, -1.)], hi=-s_ref)

matrix = coo_matrix(
    (coefficients, (row_indices, column_indices)),
    shape=(len(constraint_lower), nvar),
).tocsc()
row_lower, row_upper = np.asarray(constraint_lower), np.asarray(constraint_upper)
started = perf_counter()
with warnings.catch_warnings():

```



```

# SciPy forwards this supported HiGHS option. Suppress only its exact
# forwarding notice; retain every other solver warning.
warnings.filterwarnings(
    "ignore", category=RuntimeWarning,
    message=r"Unrecognized options detected: \{'mip_feasibility_tolerance'\}\. These will be
passed to HiGHS verbatim\.",
)
result = milp(
    c=objective, integrality=integer, bounds=Bounds(lower, upper),
    constraints=LinearConstraint(matrix, row_lower, row_upper),
    options={"time_limit": float(time_limit), "mip_rel_gap": float(mip_rel_gap),
            "mip_feasibility_tolerance": MIP_FEASIBILITY_TOL, "disp": False},
)
mip_seconds = perf_counter() - started
if result.x is None or not np.isfinite(result.x).all():
    raise RuntimeError(f"No usable MILP incumbent: status={result.status}, message=
{result.message}")
if result.status not in (0, 1):
    raise RuntimeError(f"MILP terminated without accepted success/limit status: {result.status},
{result.message}")
if result.fun is None or not np.isfinite(result.fun):
    raise RuntimeError("MILP incumbent has no finite objective.")

polished, polish_seconds, original_integer_error = _fixed_mode_polish(
    objective, matrix, row_lower, row_upper, lower, upper, integer,
    np.asarray(result.x, dtype=float), time_limit,
)
seconds = perf_counter() - started
raw = polished.x
g = np.maximum(raw[:n], 0.)
flows = {name: np.empty((m, n)) for name in ("c", "d", "e", "v", "r")}
modes_raw = np.empty((m, n))
raw_s = np.empty((m, n))
raw_xi = np.empty(m)
for scenario in range(m):
    base = n + scenario * scene_width
    for block, name in enumerate(flows):
        flows[name][scenario] = np.maximum(raw[base + block * n : base + (block + 1) * n], 0.)
    raw_s[scenario] = raw[base + 5 * n : base + 6 * n]
    modes_raw[scenario] = raw[base + 6 * n : base + 7 * n]
    raw_xi[scenario] = raw[base + 7 * n]
modes = np rint(modes_raw).astype(int)
c, d, e, v, r = (flows[name] for name in ("c", "d", "e", "v", "r"))
s = np.concatenate((np.full((m, 1), s0), s0 + np.cumsum(battery.eta_c * c - d / battery.eta_d,
axis=1)), axis=1)
surplus = np.maximum(g[None, :] + scenario_pv - scenario_load, 0.)
deficit = np.maximum(scenario_load - g[None, :] - scenario_pv, 0.)
violations = {
    "energy_balance_kwh": float(np.max(np.abs(g + scenario_pv - v - r + d + e - scenario_load -
c))),
    "soc_recursion_kwh": float(np.max(np.abs(np.diff(s, axis=1) - battery.eta_c * c + d /
battery.eta_d))),
    "solver_soc_agreement_kwh": float(np.max(np.abs(s[:, 1:] - raw_s))),
    "soc_initial_error_kwh": float(np.max(np.abs(s[:, 0] - s0))),
    "soc_lower_kwh": float(max(battery.low - np.min(s), 0.)),
    "soc_upper_kwh": float(max(np.max(s) - battery.high, 0.)),
    "ordinary_purchase_cap_kwh": float(max(np.max(g - g_upper), 0.)),
    "fixed_prefix_error_kwh": float(np.max(np.abs(g[:prefix_length] - prefix))) if prefix_length
else 0.,
    "charge_rate_kwh": float(max(np.max(c) - rate, 0.)),
    "discharge_rate_kwh": float(max(np.max(d) - rate, 0.)),
    "charge_mode_kwh": float(max(np.max(c - rate * modes), 0.)),
    "discharge_mode_kwh": float(max(np.max(d - rate * (1 - modes)), 0.)),
    "charge_exceeds_ordinary_surplus_kwh": float(max(np.max(c - surplus), 0.)),
    "discharge_exceeds_ordinary_deficit_kwh": float(max(np.max(d - deficit), 0.)),
    "emergency_deficit_equation_kwh": float(np.max(np.abs(e - deficit + d))),
}

```

```

        "unused_surplus_equation_kwh": float(np.max(np.abs(v + r - surplus + c))),
        "emergency_bound_kwh": float(max(np.max(e - scenario_load * (1 - modes)), 0.)),
        "emergency_during_charging_kwh": float(np.max(np.minimum(c, e))),
        "simultaneous_charge_discharge_kwh": float(np.max(np.minimum(c, d))),
        "unused_contract_bound_kwh": float(max(np.max(v - g), 0.)),
        "curtailment_bound_kwh": float(max(np.max(r - scenario_pv), 0.)),
        "terminal_shortfall_constraint_kwh": float(max(np.max(s_ref - s[:, -1] - raw_xi), 0.)),
        "nonnegative_energy_kwh": float(max(-np.min(np.r_[g, c.ravel(), d.ravel(), e.ravel(), v.ravel(),
r.ravel()]), 0.)),
    }
    plan_cost = float(p @ g)
    fixed_prefix_plan_cost = float(p[:prefix_length] @ g[:prefix_length])
    new_plan_cost = float(p[prefix_length:] @ g[prefix_length:])
    scenario_emergency_cost = 5.0 * (e @ p)
    shortfall = np.maximum(s_ref - s[:, -1], 0.)
    scenario_terminal_cost = kappa * shortfall
    expected_emergency_cost = float(probability @ scenario_emergency_cost)
    terminal_cost = float(probability @ scenario_terminal_cost)
    recomputed_objective = plan_cost + expected_emergency_cost + terminal_cost
    objective_difference = abs(recomputed_objective - float(polished.fun))
    integrality_error = float(np.max(np.abs(modes_raw - modes)))
    worst = max(violations.values())
    checks = {
        **violations, "max_violation_kwh": worst,
        "integrality_error": integrality_error,
        "original_mip_integrality_error": original_integer_error,
        "objective_difference_yuan": objective_difference,
        "physical_tolerance_kwh": PHYSICAL_TOL_KWH,
        "objective_tolerance_yuan": OBJECTIVE_TOL_YUAN,
        "passed": bool(worst ≤ PHYSICAL_TOL_KWH and integrality_error ≤ INTEGRALITY_TOL and
objective_difference ≤ OBJECTIVE_TOL_YUAN),
    }
    if not checks["passed"]:
        raise RuntimeError(f"Polished MILP plan failed physical/objective verification: {checks}")
    raw_gap = getattr(result, "mip_gap", None)
    original_gap = None if raw_gap is None or not np.isfinite(raw_gap) else float(raw_gap)
    dual = getattr(result, "mip_dual_bound", None)
    dual = None if dual is None or not np.isfinite(dual) else float(dual)
    bound_error = None if dual is None else max(dual - recomputed_objective, 0.)
    bound_consistent = dual is not None and bound_error ≤ OBJECTIVE_TOL_YUAN
    absolute_gap = max(recomputed_objective - dual, 0.) if bound_consistent else None
    gap = absolute_gap / max(abs(recomputed_objective), 1e-10) if bound_consistent else None
    # Conservatively retain an unproven label for an originally time-limited
    # solve, even if its global lower bound happens to meet the polished value.
    proven = bool(result.status == 0 and gap is not None and gap ≤ float(mip_rel_gap) + 1e-10)
    status = "optimal_within_requested_gap" if proven else "feasible_unproven"
    mip_diagnostics = {
        "status_code": int(result.status), "success": bool(result.success),
        "message": str(result.message), "objective_yuan": float(result.fun),
        "reported_gap": original_gap, "dual_bound_yuan": dual,
        "solve_seconds": mip_seconds, "time_limit_seconds": float(time_limit),
        "integrality_error": original_integer_error,
        "mip_feasibility_tolerance": MIP_FEASIBILITY_TOL,
    }
    }
    polish_diagnostics = {
        "method": "LP highs-ds with rounded MILP modes fixed exactly",
        "status_code": int(polished.status), "success": bool(polished.success),
        "message": str(polished.message), "objective_yuan": float(polished.fun),
        "recomputed_objective_yuan": recomputed_objective,
        "objective_change_yuan": recomputed_objective - float(result.fun),
        "solve_seconds": polish_seconds, "time_limit_seconds": float(time_limit),
        "feasibility_tolerance": LP_FEASIBILITY_TOL,
        "fixed_binary_variables": int(integer.sum()),
        "ordinary_g_reoptimized_except_fixed_prefix": True,
        "gap_against_original_global_dual_bound": gap,
        "absolute_gap_yuan": absolute_gap, "global_bound_consistent": bound_consistent,
    }

```

```

        "dual_bound_above_primal_yuan": bound_error,
    }
    return {
        "g": g, **flows, "w": v + r, "s": s, "u": modes,
        "objective": recomputed_objective, "plan_cost": plan_cost,
        "plan_cost_scope": "Full supplied horizon, including the already paid fixed-prefix constant; use
new_plan_cost for new-contract billing.",
        "fixed_prefix_length": prefix_length, "fixed_prefix_g": prefix,
        "fixed_prefix_plan_cost": fixed_prefix_plan_cost, "new_plan_cost": new_plan_cost,
        "state_at_new_window_start_kwh": s[:, prefix_length].copy(),
        "expected_emergency_cost": expected_emergency_cost, "terminal_cost": terminal_cost,
        "scenario_emergency_cost": scenario_emergency_cost,
        "scenario_terminal_cost": scenario_terminal_cost,
        "terminal_shortfall_kwh": shortfall,
        "probabilities": probability, "scenario_count": m,
        "scenario_load_kwh": scenario_load, "scenario_pv_kwh": scenario_pv,
        "terminal_weight": kappa, "s_ref": s_ref,
        "status": status, "status_code": int(result.status), "message": str(result.message),
        "success": True, "optimality_proven": proven,
        "mip_gap": gap, "gap": gap, "requested_mip_rel_gap": float(mip_rel_gap),
        "dual_bound": dual, "absolute_gap_yuan": absolute_gap,
        "gap_basis": "Polished primal objective versus original MILP global dual bound; not the fixed-
mode LP bound.",
        "mip_diagnostics": mip_diagnostics, "polish_diagnostics": polish_diagnostics,
        "solve_seconds": seconds, "checks": checks,
        "ordinary_purchase_upper_kwh": g_upper,
        "recourse_information": "Each scenario's recourse sees the whole supplied horizon, including a
forecast-only fixed-contract prefix: a two-stage approximation, not causal MPC execution cost.",
        "bounds_explanation": "New  $g \leq \max_s(\text{load}_s) + \text{rate}$ ; fixed-prefix bounds are enlarged to honor
inherited contracts;  $e_s \leq \text{load}_s$ ; discarded energy  $\leq g_{\text{upper}} + \text{pv}_s$ . These are sufficient optimization
bounds, not a grid power limit.",
    }

```

13. question2/planning/code/risk.py

源文件 SHA-256: a13a5bdc25268f796c1daf1d257ae3d85136d576485805228e1ab6cf10efbb64 ; 代码行数: 55.

```

"""Historical same-origin residuals only; full-day paired bootstrap scenarios."""
import numpy as np

def historical_residuals(data,i>window=28):
    if not isinstance(window,int) or window<1 or not 0<=i<len(data['dates']):
        raise ValueError('History window must be positive and day index in range')
    first=max(0,i>window)
    # Each earlier target day has ended by this day's decision time.
    if any(day>=data['dates'][i] for day in data['dates'][first:i]):
        raise ValueError('Current/future day in residual history')
    return (data['load_a'][first:i]-data['load_f'][first:i],
            data['pv_a'][first:i]-data['pv_f'][first:i])

def build_risk_inputs(data,i,method,*,quantile=.8>window=28,min_history=14,
                    scenarios=8,seed=20260911):
    if method not in ('A','B','C','D'):
        raise ValueError('Unknown method')
    if not isinstance(min_history,int) or not 1<=min_history<=window:
        raise ValueError('Require 1 <= min_history <= window')
    if not 0<=quantile<1 or not isinstance(scenarios,int) or scenarios<1:
        raise ValueError('Invalid quantile or scenario count')
    lf,pf=data['load_f'][i].copy(),data['pv_f'][i].copy()
    lr,pr=historical_residuals(data,i>window)
    count=len(lr)
    meta={'method':method,'history_days':count,'window_days':window,'fallback':False,
          'residual_last_day':str(data['dates'][i-1]) if i else None}
    if method in ('A','B'):
        return lf,pf,None,None,meta
    if count<min_history:
        meta.update(fallback=True,fallback_reason='insufficient completed paired forecast days')
        return lf,pf,None,None,meta
    if method=='C':
        if not 0<=quantile<1:
            raise ValueError('quantile must be in [0,1)')
        margin=np.maximum(np.quantile(lr-pr,quantile,axis=0),0) if quantile else np.zeros_like(lf)
        meta.update(quantile=quantile,margin_kwh=float(margin.sum())/6)
        return lf+margin,pf,None,None,meta
    if method=='D':
        if scenarios<1:
            raise ValueError('Need at least one scenario')
        rng=np.random.default_rng(seed+i*1009)
        draw=rng.integers(0,count,size=scenarios)
        sl=np.maximum(lf[None,:]+lr[draw],0)
        sp=np.maximum(pf[None,:]+pr[draw],0)
        # Only impose a nighttime zero at slots which every available PV actual
        # and forecast marks zero. Do not infer a seasonal sunrise from the future.
        first=max(0,i>window)
        dark=(pf==0)&np.all(data['pv_a'][first:i]==0,axis=0)&np.all(data['pv_f'][first:i]==0,axis=0)
        sp[:,dark]=0
        meta.update(scenarios=scenarios,seed=seed+i*1009,
                    historical_day_indices=(draw+first).tolist(),
                    generation='paired whole-day residual bootstrap; clipped nonnegative',
                    oracle_recourse_approximation=True)
        return lf,pf,sl,sp,meta
    raise ValueError('Unknown method')

```

14. question2/planning/code/run_all.py

源文件 SHA-256: 5259412e9d9e5d8e76cf0846eb958ce533a11d36f468e8ac786e270182bd8608 ; 代码行数: 36.

```
"""Reproduce Q2 computation, independent checks, Chinese workbooks and figures."""
import argparse
import os
from pathlib import Path
import subprocess
import sys
import tempfile

ROOT = Path(__file__).resolve().parents[1]

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--out', type=Path, default=ROOT / 'results/final_team')
    parser.add_argument('--reports-only', action='store_true',
                        help='Verify and render an existing completed calculation.')
    args = parser.parse_args()
    out = args.out.resolve()
    env = dict(os.environ)
    env.setdefault('MPLCONFIGDIR', str(Path(tempfile.gettempdir()) / 'q2-matplotlib'))
    commands = []
    if not args.reports_only:
        commands.append(['window_study.py', '--out', str(out)])
    commands.extend([
        ['evaluate_reports.py', str(out)],
        ['audit_window_results.py', str(out), '--figures'],
        ['build_window_report.py', '--results', str(out), '--out', str(out / '中文报表')],
        ['build_conclusion.py', str(out)],
    ])
    for name, *arguments in commands:
        subprocess.run([sys.executable, str(ROOT / 'code' / name), *arguments],
                        cwd=ROOT, env=env, check=True)

if __name__ == '__main__':
    main()
```

15. question2/planning/code/study.py

源文件 SHA-256: 66b0085d24ed289982bf2ce281862de5dfcbdb9647addfef7e99ca72a350bd908 ; 代码行数: 232。

```

"""Ordered A--D comparison. Report synthetic and supplied-data studies distinctly."""
from __future__ import annotations
import argparse
import datetime as dt
import json
from pathlib import Path
import time
import numpy as np
from input_io import load_bundle, read_price, write_csv, pair_forecast_workbooks
from synthetic import generate
from risk import build_risk_inputs
from planners import solve_plan
from control_refined import run_controller

ROOT=Path(__file__).resolve().parents[1]

def json_write(path,data):
    def convert(x):
        if isinstance(x,np.ndarray):return x.tolist()
        if isinstance(x,np.generic):return x.item()
        if isinstance(x,(dt.datetime,dt.date,Path)):return str(x)
        raise TypeError(type(x).__name__)

    Path(path).write_text(json.dumps(data,ensure_ascii=False,indent=2,default=convert,allow_nan=False),encoding='utf-8')

def plan_day(data,i,method,p,s0,config,q):
    lf,pf,sl,sp,meta=build_risk_inputs(data,i,method,quantile=q,
        window=config['window'],min_history=config['min_history'],
        scenarios=config['scenarios'],seed=config['seed'])
    result=solve_plan(lf/6,pf/6,p,s0,
        scenario_load_kwh=None if sl is None else sl/6,
        scenario_pv_kwh=None if sp is None else sp/6,
        terminal_weight=config['terminal_weight'],s_ref=config['s_ref'],
        time_limit=config['milp_seconds'],mip_rel_gap=config['mip_gap'])
    return result,meta

def operate(data,i,p,g,s0,method,config):
    return run_controller('greedy' if method=='A' else 'mpc',g,
        data['load_a'][i]/6,data['pv_a'][i]/6,data['load_f'][i]/6,data['pv_f'][i]/6,p,s0,
        terminal_weight=config['terminal_weight'],s_ref=config['s_ref'])

def daily_record(data,i,method,stage,plan,op,meta,config):
    return {'date':str(data['dates'][i]),'method':method,'stage':stage,
        'cash_cost_yuan':op['fees_total'],'planned_cost_yuan':op['fees_plan'],
        'emergency_cost_yuan':op['fees_emergency'],
        'inventory_adjusted_cost_yuan':op['fees_total']-config['inventory_value']*(op['s'][-1]-op['s']
[0]),
        'initial_soc_kwh':op['s'][0],'final_soc_kwh':op['s'][-1],
        'planned_kwh':float(op['g'].sum()),'emergency_kwh':float(op['e'].sum()),
        'emergency_intervals':int((op['e']>1e-5).sum()),
        'unused_paid_kwh':float(op['v'].sum()),'curtailed_pv_kwh':float(op['r'].sum()),

        'fallback_to_point':meta.get('fallback',False),'historical_residual_days':meta.get('history_days',0),
        'dayahead_solver_status':plan['status'],'dayahead_mip_gap':plan['mip_gap'],
        'dayahead_solve_seconds':plan['solve_seconds'],'controller_solve_seconds':op['solve_seconds'],
        'planner_checks_passed':bool(plan['checks']['passed']),
        'controller_checks_passed':bool(op['checks']['passed']),
        'dayahead_expected_emergency_yuan':plan['expected_emergency_cost'],
        'dayahead_objective_not_real_bill':plan['objective']}

def trajectory_rows(data,i,method,op,p):
    zero=dt.datetime.combine(data['dates'][i],dt.time())
    for t in range(144):

```



```

a=zero+dt.timedelta(minutes=10*t)
yield {'method':method,'interval_start':a.isoformat(),'interval_end':
(a+dt.timedelta(minutes=10)).isoformat(),
      'price_yuan_per_kwh':p[t], 'load_forecast_kw':data['load_f'][i,t],
      'pv_forecast_kw':data['pv_f'][i,t], 'load_actual_kw':data['load_a'][i,t],
      'pv_actual_kw':data['pv_a'][i,t], 'g_kwh':op['g'][t], 'charge_kwh':op['c'][t],
      'discharge_kwh':op['d'][t], 'emergency_kwh':op['e'][t],
      'unused_paid_kwh':op['v'][t], 'curtailed_pv_kwh':op['r'][t],
      'soc_start_kwh':op['s'][t], 'soc_end_kwh':op['s'][t+1],
      'cash_cost_yuan':p[t]*op['g'][t]+5*p[t]*op['e'][t]}

def summarize(rows,phase):
    result=[]
    for method in ('A','B','C','D'):
        part=[r for r in rows if r['method']==method]
        if not part:continue
        cost=np.array([r['cash_cost_yuan'] for r in part])
        result.append({'phase':phase,'method':method,'days':len(part),
                      'cash_cost_yuan':float(cost.sum()),
                      'inventory_adjusted_cost_yuan':float(sum(r['inventory_adjusted_cost_yuan'] for r in part)),
                      'planned_cost_yuan':float(sum(r['planned_cost_yuan'] for r in part)),
                      'emergency_cost_yuan':float(sum(r['emergency_cost_yuan'] for r in part)),
                      'emergency_kwh':float(sum(r['emergency_kwh'] for r in part)),
                      'unused_paid_kwh':float(sum(r['unused_paid_kwh'] for r in part)),
                      'curtailed_pv_kwh':float(sum(r['curtailed_pv_kwh'] for r in part)),
                      'p95_daily_cash_cost_yuan':float(np.quantile(cost,.95)),
                      'maximum_daily_cash_cost_yuan':float(cost.max()),
                      'initial_soc_kwh':part[0]['initial_soc_kwh'],'final_soc_kwh':part[-1]['final_soc_kwh'],
                      'fallback_days':sum(r['fallback_to_point'] for r in part),
                      'unproven_plan_days':sum(r['dayahead_solver_status']!='optimal_within_requested_gap' for r
in part),
                      'all_constraints_passed':all(r['planner_checks_passed'] and r['controller_checks_passed']
for r in part)})
    return result

def run_study(data,p,config,out,role):
    n=len(data['dates']);cut=config['calibration_days']
    if not 1≤cut≤n:
        raise ValueError('Need nonempty chronological calibration and test periods')
    if not 1≤config['min_history']≤config['window'] or config['scenarios']<1:
        raise ValueError('Require 1 ≤ min_history ≤ window and positive scenario count')
    if config['calibration_tail']<1 or not config['quantile_candidates']:
        raise ValueError('Need a positive calibration tail and at least one quantile candidate')
    numbers=[config[k] for k in
('initial_soc','terminal_weight','inventory_value','s_ref','milp_seconds','mip_gap')]
    if not np.isfinite(numbers).all() or not 1200≤config['initial_soc']≤10800:
        raise ValueError('Invalid numeric parameter or initial SOC outside physical bounds')
    if min(config['terminal_weight'],config['inventory_value'],config['mip_gap'])<0 or
config['milp_seconds']≤0:
        raise ValueError('Invalid cost weight, gap or solve time limit')
    out=Path(out);out.mkdir(parents=True,exist_ok=True)
    s0=config['initial_soc'];test=list(range(cut,n));started=time.perf_counter()
    events=[];all_rows=[];solver_log=[]
    def event(name,details):
        events.append({'order':len(events)+1,'stage':name,**details})
        json_write(out/'experiment_events.json',events)
        print(name,details,flush=True)
    # Stage 1: contract held fixed for control-only attribution.
    event('1_same_contract_control_comparison',{'test_days':len(test),'local_initial_soc':s0})
    same=[];reference={}
    for i in test:
        pl,meta=plan_day(data,i,'B',p,s0,config,0)
        reference[i]=(pl,meta)
        for method in ('A','B'):
            op=operate(data,i,p,pl['g'],s0,method,config)
            row=daily_record(data,i,method,'same_contract',pl,op,meta,config)

```

```

        same.append(row)
        if (i-cut+1)%4==0:print('same-contract days',i-cut+1,flush=True)
    write_csv(out/'stage1_same_contract_daily.csv',same)
    # Stage 2: all tuning uses calibration days, never the test days above.
    eligible=list(range(config['min_history'],cut))[-config['calibration_tail']:]
    tuning=
[ ];q_selected=config['default_quantile'];selection_status='preset_no_eligible_calibration_days'
    for q in config['quantile_candidates']:
        for i in eligible:
            pl,meta=plan_day(data,i,'C',p,s0,config,q)
            op=operate(data,i,p,pl['g'],s0,'C',config)

tuning.append({'quantile':q,**daily_record(data,i,'C','calibration_only',pl,op,meta,config)})
    if tuning:
        scores={q:sum(r['inventory_adjusted_cost_yuan'] for r in tuning if r['quantile']==q) for q in
config['quantile_candidates']}
        q_selected=min(scores,key=lambda q:(scores[q],q))
        selection_status='selected_on_calibration_only'
        write_csv(out/'calibration_quantiles.csv',tuning)
        json_write(out/'frozen_parameters.json',{'config':config,'selected_quantile':q_selected,
'selection_status':selection_status,'selected_from_dates':[str(data['dates'][i]) for i in
eligible],
'effective_from':str(data['dates'][cut]),'scenario_count_status':'preset_not_claimed_optimal',
'test_was_not_used_for_tuning':True})
        event('2_dayahead_planning_comparison',{'methods':
['B','C','D'],'same_controller':'mpc','selected_quantile':q_selected})
        local=[]
        for i in test:
            for method in ('B','C','D'):
                pl,meta=reference[i] if method=='B' else plan_day(data,i,method,p,s0,config,q_selected)
                op=operate(data,i,p,pl['g'],s0,method,config)
                local.append(daily_record(data,i,method,'local_dayahead',pl,op,meta,config))
                solver_log.append({'stage':'local','date':str(data['dates'][i]),'method':method,
'status':pl['status'],'gap':pl['mip_gap'],'checks':pl['checks'],'risk':meta})
                if (i-cut+1)%4==0:print('dayahead comparison days',i-cut+1,flush=True)
        write_csv(out/'stage2_dayahead_daily.csv',local)
        # Stage 3: whole supplied period, including an honest preset warm-up phase.
        # Each system has its own measured state and hence its own next-day contract.
        event('3_continuous_system_replay',{'all_days':n,'initial_soc':s0,'calibration_days':cut})
        continuous=[];traces=[]
        states={m:s0 for m in ('A','B','C','D')}
        for i in range(n):
            q=q_selected if i≥cut else config['default_quantile']
            for method in ('A','B','C','D'):
                initial=states[method]
                pl,meta=plan_day(data,i,method,p,initial,config,q)
                op=operate(data,i,p,pl['g'],initial,method,config)
                states[method]=float(op['s'][-1])
                phase='test' if i≥cut else 'warmup_calibration'
                row=daily_record(data,i,method,phase,pl,op,meta,config)
                continuous.append(row);traces.extend(trajecory_rows(data,i,method,op,p))
                solver_log.append({'stage':'continuous','date':str(data['dates'][i]),'method':method,
'status':pl['status'],'gap':pl['mip_gap'],'checks':pl['checks'],'risk':meta})
                if (i+1)%4==0:print('continuous days',i+1,flush=True)
        write_csv(out/'stage3_continuous_daily.csv',continuous)
        write_csv(out/'stage3_interval_trace.csv',traces)
        summaries=(summarize(same,'same_contract_local')+summarize(local,'dayahead_local')+
summarize(continuous,'continuous_all')+
summarize([r for r in continuous if r['stage']=='test'],'continuous_test'))
        write_csv(out/'comparison_summary.csv',summaries)
        json_write(out/'solver_audit.json',solver_log)
        summary={'data_role':role,'real_competition_result':False,
'provenance_status':'synthetic_engineering_only' if role=='synthetic' else
'conditional_on_supplied_forecast_provenance_not_independently_verified',
'elapsed_seconds':time.perf_counter()-started,'dates':[str(data['dates'][0]),str(data['dates']
[-1])],

```



```

        'test_dates':[str(data['dates'][cut]),str(data['dates'][-1])],
        'all_physical_checks_passed':all(r['all_constraints_passed'] for r in summaries),
        'selected_quantile':q_selected, 'scenarios':config['scenarios'],
        'periods':summaries, 'input_metadata':data['metadata'],
        'interpretation':'Synthetic fixture outputs only validate execution. Do not use them to rank
real A-D policies.' if role=='synthetic' else
        'Historical evaluation conditional on input forecasts; two-stage scenario objectives are not
realized bills.',
        'terminal_inventory_comparison':'cash minus common value times inventory change; not a refund;
report cash and inventories separately',
        'test_subset_initial_states':'After policy-specific warmup, test-boundary SOC may differ;
inventory adjustment is reported separately.}
    json_write(out/'summary.json',summary)
    event('4_reports_complete',{ 'all_checks_passed':summary['all_physical_checks_passed']})
    return summary

def main():
    a=argparse.ArgumentParser(description=__doc__);sp=a.add_subparsers(dest='command',required=True)
    cv=sp.add_parser('convert-forecast');cv.add_argument('--workbook',type=Path,required=True)
    cv.add_argument('--load-sheet',default='负载预测');cv.add_argument('--pv-sheet',default='光伏预测')
    cv.add_argument('--output',type=Path,required=True)
    for name in ('synthetic','evaluate'):
        x=sp.add_parser(name);x.add_argument('--out',type=Path,default=ROOT/'results'/name)
        x.add_argument('--price',type=Path,default=ROOT/'data/attachment1.xlsx')
        x.add_argument('--calibration-days',type=int,default=24)
        x.add_argument('--min-history',type=int,default=14);x.add_argument('--
window',type=int,default=28)
        x.add_argument('--scenarios',type=int,default=8);x.add_argument('--
seed',type=int,default=20260911)
        x.add_argument('--milp-seconds',type=float,default=30);x.add_argument('--mip-
gap',type=float,default=1e-5)
        if name=='evaluate':
            x.add_argument('--forecasts',type=Path,required=True);x.add_argument('--
actuals',type=Path,required=True)
            x.add_argument('--s0',type=float,required=True,help='State at first supplied date, not an
assumed February 6000')
            else:x.add_argument('--days',type=int,default=40);x.add_argument('--s0',type=float,default=6000)
    args=a.parse_args()
    if args.command=='convert-forecast':
        pair_forecast_workbooks(args.workbook,args.workbook,args.output,args.load_sheet,args.pv_sheet)
        print(args.output);return
    if args.command=='synthetic':
        fp,ap=generate(ROOT/'data/synthetic_fixture',args.days,args.seed)
    else:
        fp,ap=args.forecasts,args.actuals
        # The supplied format workbook is never selected as an actual forecast by this command.
        data=load_bundle(fp,ap)
        if args.command=='evaluate' and data['metadata']['source_role']=='synthetic':
            raise ValueError('Known synthetic data cannot enter supplied-data evaluation; use synthetic
command')
        p,pm=read_price(args.price)
        config={'initial_soc':args.s0, 'calibration_days':args.calibration_days, 'calibration_tail':7,
        'window':args.window, 'min_history':args.min_history, 'scenarios':args.scenarios, 'seed':args.seed,
        'terminal_weight':float(p.min()/0.9), 'inventory_value':float(p.min()/0.9), 's_ref':6000,
        'default_quantile':.8, 'quantile_candidates':[0,.5,.8,.9],
        'milp_seconds':args.milp_seconds, 'mip_gap':args.mip_gap}
        data['metadata']['price']=pm
        result=run_study(data,p,config,args.out,'synthetic' if args.command=='synthetic' else
'supplied_forecasts')
        print(json.dumps({'out':str(args.out), 'all_checks_passed':result['all_physical_checks_passed'],
        'data_role':result['data_role']},ensure_ascii=False))

if __name__=='__main__':main()

```

16. question2/planning/code/synthetic.py

源文件 SHA-256: 6aa05c90d3069dce5f4aac3de21fe883c33c8769643377c55c980d2b67ef9b12 ; 代码行数: 41。

```

"""Explicitly synthetic engineering fixture; never a competition cost result."""
import datetime as dt
from pathlib import Path
import json
import numpy as np
from input_io import write_csv, FORECAST_FIELDS, ACTUAL_FIELDS

def generate(out, days=40, seed=20260911):
    out=Path(out); out.mkdir(parents=True, exist_ok=True)
    rng=np.random.default_rng(seed)
    hours=np.arange(145)/6
    solar=np.maximum(np.sin(np.pi*(hours-6)/12), 0)
    solar[hours ≥ 18]=0
    forecast, actual=[], []
    for i in range(days):
        zero=dt.datetime(2030, 1, 1)+dt.timedelta(days=i)
        load_f=4400+450*np.sin(2*np.pi*(hours-8)/24)+700*np.exp(-((hours-19)/2.5)**2)
        load_f+=250*np.cos(2*np.pi*i/7)
        pv_f=(8200+500*np.sin(i/3))*solar
        # Forecast is defined before generating the realized disturbances.
        load_bias=rng.normal(70, 230)
        cloud_error=rng.normal(-.025, .20)
        ar=np.zeros(145)
        for t in range(1, 145): ar[t]=.85*ar[t-1]+rng.normal(0, 80)
        load_a=np.maximum(load_f+load_bias+ar, 0)
        pv_a=np.maximum(pv_f*(1+cloud_error)+solar*rng.normal(0, 80, 145), 0)
        for t in range(145):
            a=zero+dt.timedelta(minutes=10*t); b=a+dt.timedelta(minutes=10)
            forecast.append(dict(plan_date=zero.date().isoformat(), issued_at=zero.isoformat(),
                                interval_start=a.isoformat(), interval_end=b.isoformat(),
                                load_forecast_kw=float(load_f[t]), pv_forecast_kw=float(pv_f[t])))
            if t<144:
                actual.append(dict(interval_start=a.isoformat(), interval_end=b.isoformat(),
                                   load_actual_kw=float(load_a[t]), pv_actual_kw=float(pv_a[t])))
    write_csv(out/'synthetic_forecasts.csv', forecast, FORECAST_FIELDS)
    write_csv(out/'synthetic_actuals.csv', actual, ACTUAL_FIELDS)
    (out/'synthetic_manifest.json').write_text(json.dumps({
        'role': 'synthetic_engineering_fixture_not_competition_results', 'seed': seed, 'days': days,
        'dates': 'fictional 2030 dates', 'data_generation': 'fixed known mean profiles, then sampled
        correlated disturbances',
        'disclaimer': 'Tests code and experiment order only; no real model ranking or monetary savings
        claim.'}, indent=2), encoding='utf-8')
    return out/'synthetic_forecasts.csv', out/'synthetic_actuals.csv'

```

17. question2/planning/code/test_calibration_cutoff.py

源文件 SHA-256: 9b34616749412dc9e3a81d20256429392cfadef3db3bbeaed0690cfa1be7aa74 ; 代码行数: 37。

```

"""Calibration scores become available only after their 00:10 window end."""
import datetime as dt
import unittest

from window_study import calibration_indices

def timeline(days=30):
    origin = dt.datetime(2025, 2, 1)
    issues = [origin + dt.timedelta(days=i) for i in range(days)]
    return {
        "issued_at": issues,
        "window_end": [x + dt.timedelta(days=1, minutes=10) for x in issues],
    }

class CalibrationCutoffTests(unittest.TestCase):
    def test_incomplete_previous_window_cannot_select_test_parameter(self):
        data = timeline()
        selected = calibration_indices(data, cut=24, minimum=14, tail=7)
        self.assertEqual(selected, [16, 17, 18, 19, 20, 21, 22])
        self.assertNotIn(23, selected)
        self.assertEqual(data["window_end"][23] - data["issued_at"][24],
                          dt.timedelta(minutes=10))
        self.assertTrue(all(data["window_end"][i] ≤ data["issued_at"][24]
                             for i in selected))

    def test_first_risk_enabled_day_is_not_yet_a_complete_score(self):
        data = timeline()
        # Window 15 can use 14 completed historical windows, but its own
        # score is unavailable at window 16's midnight decision.
        self.assertEqual(calibration_indices(data, cut=16, minimum=14, tail=7), [])
        self.assertEqual(calibration_indices(data, cut=17, minimum=14, tail=7), [15])

if __name__ == "__main__":
    unittest.main()

```

18. question2/planning/code/test_control_v2.py

源文件 SHA-256: 245e1cb7cb15ab8d09a75bd7d693fe8c716142f90450ff8b59f23eb358c5303a ; 代码行数: 183。

```

"""Synthetic engineering checks; these tests are not a real-data policy ranking.

Run from this directory with: python -m unittest test_control_v2 -v
"""

from __future__ import annotations

import unittest
from unittest.mock import patch

import numpy as np

import control_refined
from control import Battery, DEFAULT_BATTERY, solve_horizon
from control_refined import run_controller, solve_refined_horizon

class ControlV2Tests(unittest.TestCase):
    def test_two_slot_90_kwh_example_same_contract(self):
        args = dict(g=[0., 0.], load_actual=[100., 100.], pv_actual=[0., 0.],
                    load_forecast=[100., 100.], pv_forecast=[0., 0.],
                    price=[.4, 1.], s0=1300.)
        greedy = run_controller("greedy", **args)
        mpc = run_controller("mpc", **args)
        self.assertAlmostEqual(greedy["fees_total"], 520., places=5)
        self.assertAlmostEqual(mpc["fees_total"], 250., places=5)
        np.testing.assert_allclose(greedy["d"], [90., 0.], atol=1e-6)
        np.testing.assert_allclose(mpc["d"], [0., 90.], atol=1e-6)
        self.assertAlmostEqual(mpc["s"][-1], 1200., places=5)
        # The soft terminal penalty is deliberately excluded from cash fees.
        self.assertGreater(mpc["terminal_penalty_yuan"], 0.)
        self.assertAlmostEqual(mpc["fees_total"], mpc["fees_plan"] + mpc["fees_emergency"])

    def test_units_and_efficiencies_independently(self):
        b = DEFAULT_BATTERY
        self.assertAlmostEqual(b.rate, 5000. / 6.)
        discharge = run_controller("greedy", [0.], [1000.], [0.], [1000.], [0.], [.5], 6000.)
        self.assertAlmostEqual(discharge["d"][0], 5000. / 6.)
        self.assertAlmostEqual(discharge["e"][0], 1000. - 5000. / 6.)
        self.assertAlmostEqual(discharge["s"][1], 6000. - (5000. / 6.) / .9)
        self.assertAlmostEqual(discharge["fees_emergency"], 5. * .5 * (1000. - 5000. / 6.))
        charge = run_controller("greedy", [1000.], [0.], [0.], [0.], [0.], [.5], 6000.)
        self.assertAlmostEqual(charge["c"][0], 5000. / 6.)
        self.assertAlmostEqual(charge["s"][1], 6750.)
        self.assertAlmostEqual(charge["fees_plan"], 500.)
        # Already-converted 100 kWh must not be divided by six again.
        neutral = run_controller("mpc", [100.], [100.], [0.], [100.], [0.], [.5], 6000.)
        self.assertAlmostEqual(neutral["fees_plan"], 50.)
        self.assertAlmostEqual(neutral["s"][1], 6000.)

    def test_future_actual_perturbation_cannot_change_past_actions(self):
        n = 12
        g = np.full(n, 100.)
        lf = np.full(n, 200.)
        pf = np.zeros(n)
        actual1 = np.full(n, 220.)
        actual2 = actual1.copy()
        actual2[4:] += 500.
        pv2 = pf.copy()
        pv2[5:] = 1000.
        price = np.linspace(.3, 1., n)
        first = run_controller("mpc", g, actual1, pf, lf, pf, price, 2100.)
        second = run_controller("mpc", g, actual2, pv2, lf, pf, price, 2100.)
        for key in ("c", "d", "e", "v", "r"):

```

```

        np.testing.assert_allclose(first[key][:4], second[key][:4], atol=1e-8)
        np.testing.assert_allclose(first["s"][:5], second["s"][:5], atol=1e-8)

def test_solver_receives_forecast_suffix_and_current_actual_only(self):
    n = 6
    lf = np.arange(n, dtype=float) + 100.
    pf = np.arange(n, dtype=float)
    actual = lf + 30.
    pva = pf + 2.
    p = np.array([.1, .5, .6, .7, .8, .9])
    captured = []
    original = control_refined.solve_refined_horizon

    def record(g, load, pv, price, s0, **kwargs):
        t = n - len(g)
        self.assertAlmostEqual(load[0], actual[t])
        self.assertAlmostEqual(pv[0], pva[t])
        np.testing.assert_array_equal(load[1:], lf[t + 1:])
        np.testing.assert_array_equal(pv[1:], pf[t + 1:])
        captured.append((t, kwargs["weight"]))
        return original(g, load, pv, price, s0, **kwargs)

    with patch.object(control_refined, "solve_refined_horizon", side_effect=record):
        out = run_controller("mpc", np.zeros(n), actual, pva, lf, pf, p, 1300.)
    self.assertEqual([t for t, _ in captured], list(range(n)))
    np.testing.assert_allclose([weight for _, weight in captured], [.1 / .9])
    self.assertTrue(out["checks"]["passed"])

def test_only_first_horizon_action_is_executed(self):
    starts = []

    def fake_horizon(g, load, pv, price, s0, **kwargs):
        starts.append(s0)
        n = len(g)
        # Deliberately unusable future actions must never be executed.
        out = {key: np.r_[0., np.full(n - 1, 99999.)]
               for key in ("c", "d", "v", "r", "w")}
        out["e"] = np.r_[100., np.full(n - 1, 99999.)]
        out["s"] = np.r_[s0, s0, np.full(n - 1, -99999.)]
        out.update(solve_count=2, solve_seconds=0., primary_cost_error_yuan=0.)
        return out

    with patch.object(control_refined, "solve_refined_horizon", side_effect=fake_horizon):
        out = run_controller("mpc", [0.] * 3, [100.] * 3, [0.] * 3,
                               [100.] * 3, [0.] * 3, [.5] * 3, 6000.)
    np.testing.assert_array_equal(out["d"], [0., 0., 0.])
    np.testing.assert_array_equal(out["e"], [100., 100., 100.])
    np.testing.assert_array_equal(out["s"], [6000.] * 4)
    self.assertEqual(starts, [6000.] * 3)
    self.assertTrue(out["checks"]["passed"])

def test_zero_error_144_slot_feasible_cycle(self):
    load = np.full(144, 500.)
    g = np.tile(np.r_[np.full(24, 600.), np.full(24, 419.)], 3)
    pv = np.zeros(144)
    price = np.full(144, .5)
    original_g = g.copy()
    for method in ("greedy", "mpc"):
        out = run_controller(method, g, load, pv, load, pv, price, 6000.)
        self.assertTrue(out["checks"]["passed"])
        self.assertLess(out["fees_emergency"], 1e-5)
        np.testing.assert_allclose(out["s"][-1], 6000., atol=1e-5)
        self.assertEqual(len(out["s"]), 145)
        self.assertEqual(len(out["c"]), 144)
        np.testing.assert_array_equal(out["g"], original_g)
    np.testing.assert_array_equal(g, original_g)

```

```

def test_bounds_and_emergency_never_charges(self):
    for method in ("greedy", "mpc"):
        empty = run_controller(method, [0.], [500.], [0.], [500.], [0.], [.5], 1200.)
        self.assertEqual(empty["c"][0], 0.)
        self.assertEqual(empty["d"][0], 0.)
        self.assertEqual(empty["e"][0], 500.)
        self.assertEqual(empty["s"][-1], 1200.)
        full = run_controller(method, [400.], [100.], [200.], [100.], [200.], [.5], 10800.)
        self.assertEqual(full["c"][0], 0.)
        self.assertEqual(full["v"][0], 400.)
        self.assertEqual(full["r"][0], 100.)
        self.assertEqual(full["fees_plan"], 200.)
        self.assertEqual(full["s"][-1], 10800.)
        self.assertTrue(full["checks"]["passed"])

def test_state_carries_between_calls(self):
    day1 = run_controller("greedy", [100.], [0.], [0.], [0.], [0.], [.5], 6000.)
    day2 = run_controller("greedy", [0.], [100.], [0.], [100.], [0.], [.5], day1["s"][-1])
    self.assertEqual(day1["s"][-1], 6090.)
    self.assertEqual(day2["s"][0], 6090.)
    self.assertEqual(day2["s"][-1], 6090. - 100. / .9)

def test_refinement_preserves_primary_objective(self):
    rng = np.random.default_rng(20260911)
    for _ in range(5):
        n = 12
        g, load, pv = (rng.uniform(0, 500, n) for _ in range(3))
        price = rng.uniform(.3, 1., n)
        s0 = float(rng.uniform(1200, 10800))
        weight = float(np.min(price) / .9)
        basic = solve_horizon(g, load, pv, price, s0, weight=weight)
        refined = solve_refined_horizon(g, load, pv, price, s0, weight=weight)
        self.assertLess(abs(basic["objective"] - refined["primary_objective"]), 1e-5)
        self.assertTrue(refined["checks"]["passed"])

def test_invalid_inputs_fail_loudly(self):
    valid = dict(g=[0.], load_actual=[100.], pv_actual=[0.], load_forecast=[100.],
                pv_forecast=[0.], price=[.5], s0=6000.)
    for key, value in (("g", [-1.]), ("price", [np.nan]), ("s0", 1190.),
                      ("load_forecast", [100., 100.])):
        with self.subTest(key=key), self.assertRaises(ValueError):
            run_controller("mpc", **{**valid, key: value})
    with self.assertRaises(ValueError):
        run_controller("unknown", **valid)
    with self.assertRaises(ValueError):
        run_controller("mpc", **valid, terminal_weight=-1.)
    with self.assertRaises(ValueError):
        Battery(eta_c=0.)

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

19. question2/planning/code/test_input_risk.py

源文件 SHA-256: 1fcc6c745b0a3f9adcde4ae3d74791bd1385b8db6b9f18eb1fe504f84707c772 ; 代码行数: 268。

```

"""Data-interface and causal risk checks using synthetic temporary fixtures."""

from __future__ import annotations

import copy
import csv
import datetime as dt
from pathlib import Path
import tempfile
import unittest

import numpy as np
from openpyxl import Workbook

from input_io import (
    clock_minute, interval_header, load_bundle, pair_forecast_workbooks,
    read_price, read_wide_means,
)
from risk import build_risk_inputs, historical_residuals

FORECAST_COLUMNS = [
    "plan_date", "issued_at", "interval_start", "interval_end",
    "load_forecast_kw", "pv_forecast_kw",
]
ACTUAL_COLUMNS = ["interval_start", "interval_end", "load_actual_kw", "pv_actual_kw"]

def write_rows(path, rows, columns):
    with Path(path).open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=columns)
        writer.writeheader()
        writer.writerows(rows)

def interval_headers():
    def clock(minutes):
        if minutes ≥ 1440:
            return f"次日{(minutes - 1440)//60:02d}:{(minutes - 1440)%60:02d}"
        return f"{minutes//60:02d}:{minutes%60:02d}"
    return [f"{clock(10*t)}-{clock(10*t+10)}" for t in range(145)]

def fixture_rows(days):
    forecast, actual = [], []
    for day_index, day in enumerate(days):
        zero = dt.datetime.combine(day, dt.time())
        for t in range(145):
            start = zero + dt.timedelta(minutes=10*t)
            forecast.append({
                "plan_date": day.isoformat(), "issued_at": zero.isoformat(),
                "interval_start": start.isoformat(),
                "interval_end": (start + dt.timedelta(minutes=10)).isoformat(),
                "load_forecast_kw": 100 + 10 * day_index + (100 if t % 2 else 0),
                "pv_forecast_kw": 10 + t / 100,
            })
            if t < 144:
                actual.append({
                    "interval_start": start.isoformat(),
                    "interval_end": (start + dt.timedelta(minutes=10)).isoformat(),
                    "load_actual_kw": 200 + day_index + t / 100,
                    "pv_actual_kw": 15 + t / 100,
                })
    return forecast, actual

```



```

def risk_fixture(days=35):
    dates = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(days)]
    slots = np.arange(144)[None, :]
    day_id = np.arange(days)[: , None]
    load_forecast = np.full((days, 144), 1000.)
    pv_forecast = np.full((days, 144), 400.)
    # Each day has a unique full-path residual, and the two paths are paired.
    load_residual = 10 * (day_id + 1) + .01 * slots
    pv_residual = 2 * (day_id + 1) + .003 * slots
    return {
        "dates": dates, "load_f": load_forecast, "pv_f": pv_forecast,
        "load_a": load_forecast + load_residual,
        "pv_a": pv_forecast + pv_residual,
    }

class InputRiskTests(unittest.TestCase):
    def setUp(self):
        self.temp = tempfile.TemporaryDirectory()
        self.root = Path(self.temp.name)
        self.days = [dt.date(2025, 2, 1), dt.date(2025, 2, 2)]

    def tearDown(self):
        self.temp.cleanup()

    def bundle(self, forecast, actual, forecast_columns=FORECAST_COLUMNS):
        fp, ap = self.root / "forecast.csv", self.root / "actual.csv"
        write_rows(fp, forecast, forecast_columns)
        write_rows(ap, actual, ACTUAL_COLUMNS)
        return load_bundle(fp, ap)

    def test_145_interval_means_survive_xlsx_conversion_without_averaging(self):
        book = Workbook()
        load_sheet = book.active
        load_sheet.title = "负载预测"
        pv_sheet = book.create_sheet("光伏预测")
        headers = interval_headers()
        # The supplied format's specifically confirmed alias must also work.
        headers[143] = "次日00:00"
        for sheet in (load_sheet, pv_sheet):
            sheet.append(["日期"] + headers)
        expected_load, expected_pv = [], []
        for j, day in enumerate(self.days):
            load = 100 + j * 1000 + 300 * (np.arange(145) % 2)
            pv = 10 + j * 10 + np.arange(145) / 10
            load[-1] = 999999 + j
            pv[-1] = 77777 + j
            expected_load.append(load)
            expected_pv.append(pv)
            load_sheet.append([dt.datetime.combine(day, dt.time())] + load.tolist())
            pv_sheet.append([dt.datetime.combine(day, dt.time())] + pv.tolist())
        path = self.root / "paired_forecasts.xlsx"
        book.save(path)
        dates, values, metadata = read_wide_means(path, "负载预测")
        self.assertEqual(dates, self.days)
        np.testing.assert_array_equal(values, np.array(expected_load))
        self.assertTrue(metadata["no_averaging_performed"])
        forecast_path = self.root / "converted.csv"
        with self.assertRaises(ValueError):
            pair_forecast_workbooks(path, path, forecast_path, "负载预测", "负载预测")
        pair_forecast_workbooks(path, path, forecast_path, "负载预测", "光伏预测")
        _, actual = fixture_rows(self.days)
        actual_path = self.root / "actual.csv"
        write_rows(actual_path, actual, ACTUAL_COLUMNS)

```



```

bundle = load_bundle(forecast_path, actual_path)
np.testing.assert_array_equal(bundle["load_f"], np.array(expected_load)[: , :144])
np.testing.assert_array_equal(bundle["pv_f"], np.array(expected_pv)[: , :144])
np.testing.assert_array_equal(bundle["extra_load_f"], np.array(expected_load)[: , 144])
self.assertEqual(bundle["load_f"].shape, (2, 144))
self.assertFalse(bundle["metadata"]["extra_interval_used_for_dispatch"])
self.assertEqual(bundle["load_f"][1, 0], expected_load[1][0])
self.assertNotEqual(bundle["load_f"][1, 0], bundle["extra_load_f"][0])

def test_instantaneous_prices_use_periodic_zero_and_both_interval_endpoints(self):
    book = Workbook()
    sheet = book.active
    sheet.append(["时间", "电价"])
    nodes_given = .1 + np.arange(1, 145) / 100
    for t, price in enumerate(nodes_given, start=1):
        minutes = 10 * t
        label = "00:00+1" if t == 144 else f"{minutes//60:02d}:{minutes%60:02d}"
        sheet.append([label, float(price)])
    path = self.root / "attachment1.xlsx"
    book.save(path)
    interval_price, metadata = read_price(path)
    self.assertEqual(interval_price.shape, (144,))
    self.assertAlmostEqual(interval_price[0], (nodes_given[-1] + nodes_given[0]) / 2)
    self.assertAlmostEqual(interval_price[1], (nodes_given[0] + nodes_given[1]) / 2)
    self.assertAlmostEqual(interval_price[-1], (nodes_given[-2] + nodes_given[-1]) / 2)
    self.assertEqual(metadata["price_nodes"], 145)
    self.assertEqual(metadata["mean_intervals"], 144)

def test_illegal_clocks_and_non_ten_minute_intervals_are_rejected(self):
    for value in ("25:50", "24:10", "12:60"):
        with self.subTest(clock=value), self.assertRaises(ValueError):
            clock_minute(value)
    for value in ("25:50-26:00", "00:00-00:20", "00:05-00:10"):
        with self.subTest(interval=value), self.assertRaises(ValueError):
            interval_header(value)

def test_missing_pv_and_late_forecast_release_are_rejected(self):
    forecast, actual = fixture_rows(self.days)
    missing = [{k: v for k, v in row.items() if k != "pv_forecast_kw"} for row in forecast]
    columns = [x for x in FORECAST_COLUMNS if x != "pv_forecast_kw"]
    with self.assertRaises(ValueError):
        self.bundle(missing, actual, columns)
    forecast[0]["issued_at"] = "2025-02-01T00:01:00"
    with self.assertRaises(ValueError):
        self.bundle(forecast, actual)

def test_missing_actual_slot_and_nonconsecutive_dates_are_rejected(self):
    forecast, actual = fixture_rows(self.days)
    with self.assertRaises(ValueError):
        self.bundle(forecast, actual[:70] + actual[71:])
    discontinuous = [self.days[0], self.days[1] + dt.timedelta(days=1)]
    forecast, actual = fixture_rows(discontinuous)
    with self.assertRaises(ValueError):
        self.bundle(forecast, actual)

def test_current_and_future_actual_changes_do_not_change_risk_inputs(self):
    data = risk_fixture()
    i = 30
    altered = copy.deepcopy(data)
    altered["load_a"][i:] += 100000
    altered["pv_a"][i:] += 50000
    altered["load_f"][i + 1:] += 5000
    altered["pv_f"][i + 1:] += 2000
    # Records outside the declared past window must also be irrelevant.
    altered["load_a"][:i - 7] += 80000
    for method in ("C", "D"):

```

```

a = build_risk_inputs(data, i, method, window=7, min_history=5, scenarios=12, seed=31)
z = build_risk_inputs(altered, i, method, window=7, min_history=5, scenarios=12, seed=31)
for left, right in zip(a[:4], z[:4]):
    if left is None:
        self.assertIsNone(right)
    else:
        np.testing.assert_array_equal(left, right)
self.assertEqual(a[4], z[4])
self.assertEqual(a[4]["history_days"], 7)

def test_scenarios_sample_whole_day_load_pv_residual_pairs(self):
    data = risk_fixture()
    i = 20
    lf, pf, sl, sp, meta = build_risk_inputs(data, i, "D", window=8, min_history=5, scenarios=20,
seed=902)
    historical_indices = meta["historical_day_indices"]
    self.assertEqual(len(historical_indices), 20)
    self.assertTrue(all(i - 8 ≤ j < i for j in historical_indices))
    for scenario, day in enumerate(historical_indices):
        np.testing.assert_allclose(sl[scenario] - lf, data["load_a"][day] - data["load_f"][day])
        np.testing.assert_allclose(sp[scenario] - pf, data["pv_a"][day] - data["pv_f"][day])
    # A repeat must reproduce both paths and the selected historical days.
    again = build_risk_inputs(data, i, "D", window=8, min_history=5, scenarios=20, seed=902)
    np.testing.assert_array_equal(sl, again[2])
    np.testing.assert_array_equal(sp, again[3])

def test_margin_uses_one_sided_net_residual_and_zero_means_no_margin(self):
    data = risk_fixture(days=5)
    data["load_f"][:] = 100
    data["pv_f"][:] = 50
    data["pv_a"][:] = 55
    data["load_a"][:] = 90
    data["load_a"][:4, 0] = 100 + np.array([-5, 5, 15, 25])
    lf, pf, _, _, meta = build_risk_inputs(data, 4, "C", window=4, min_history=4, quantile=.5)
    # Net residuals at slot zero are -10,0,10,20; median is 5 kW.
    self.assertAlmostEqual(lf[0], 105)
    np.testing.assert_array_equal(lf[1:], data["load_f"][4, 1:])
    np.testing.assert_array_equal(pf, data["pv_f"][4])
    self.assertAlmostEqual(meta["margin_kwh"], 5 / 6)
    no_margin = build_risk_inputs(data, 4, "C", window=4, min_history=4, quantile=0)
    np.testing.assert_array_equal(no_margin[0], data["load_f"][4])

def test_short_history_falls_back_and_future_dated_history_is_rejected(self):
    data = risk_fixture(days=5)
    for method in ("C", "D"):
        lf, pf, sl, sp, meta = build_risk_inputs(data, 2, method, min_history=3)
        np.testing.assert_array_equal(lf, data["load_f"][2])
        np.testing.assert_array_equal(pf, data["pv_f"][2])
        self.assertIsNone(sl)
        self.assertIsNone(sp)
        self.assertTrue(meta["fallback"])
        self.assertEqual(meta["history_days"], 2)
    data["dates"][0] = data["dates"][3]
    with self.assertRaises(ValueError):
        historical_residuals(data, 2)

def test_invalid_risk_configuration_is_rejected_before_short_history_fallback(self):
    data = risk_fixture(days=5)
    configurations = (
        {"method": "unknown"},
        {"method": "C", "quantile": 1.0},
        {"method": "C", "quantile": -0.1},
        {"method": "D", "scenarios": 0},
        {"method": "D", "window": 0},
        {"method": "D", "window": 28, "min_history": 29},
    )

```

```
        for configuration in configurations:
            with self.subTest(configuration=configuration), self.assertRaises(ValueError):
                build_risk_inputs(data, 0, **configuration)

if __name__ == "__main__":
    unittest.main()
```

20. question2/planning/code/test_planner_polish.py

源文件 SHA-256: 52968bd7642dc0a3d2e2bd4171201a78bbe03ca4d9cb839ecda36e369ae03999 ; 代码行数: 108。

```

"""Numerical regressions for integer-tolerance leakage and LP polishing."""
import unittest
from unittest.mock import patch

import numpy as np
from scipy.optimize import OptimizeResult

from control import Battery
from planners import PHYSICAL_TOL_KWH, solve_plan

class PlannerPolishTests(unittest.TestCase):
    def setUp(self):
        self.small = Battery(low=0, high=100, power_kw=600, eta_c=1, eta_d=1)

    def test_epsilon_binary_big_m_leak_is_removed_without_looser_physics(self):
        epsilon, leaked = 1e-7, 3e-4
        battery = Battery(low=0, high=1000, power_kw=6000, eta_c=1, eta_d=1)

    def almost_integer_incumbent(**kwargs):
        # One-slot order: g,c,d,e,v,r,S,u,xi. With load=Pv=1000,
        # M=g_upper+Pv=3000. Tiny u permits d=r=3000*u although
        # true ordinary supply has no deficit at all.
        x = np.array([0., 0., leaked, 0., 0., leaked,
                      100. - leaked, epsilon, 0.])
        self.assertEqual(kwargs["options"]["mip_feasibility_tolerance"], 1e-9)
        rows = kwargs["constraints"]
        lhs = rows.A @ x
        self.assertTrue(np.all(lhs ≥ rows.lb - 1e-9))
        self.assertTrue(np.all(lhs ≤ rows.ub + 1e-9))
        self.assertGreater(leaked, PHYSICAL_TOL_KWH)
        return OptimizeResult(x=x, fun=0., status=0, success=True,
                               message="Injected near-integer incumbent",
                               mip_gap=0., mip_dual_bound=0.)

    with patch("planners.milp", side_effect=almost_integer_incumbent):
        result = solve_plan([1000], [1000], [1], 100, b=battery,
                             terminal_weight=0, s_ref=0)
        self.assertEqual(result["checks"]["physical_tolerance_kwh"], 1e-5)
        self.assertAlmostEqual(result["checks"]["original_mip_integrality_error"], epsilon)
        self.assertEqual(result["checks"]["integrality_error"], 0.)
        self.assertLess(result["checks"]["max_violation_kwh"], 1e-8)
        np.testing.assert_allclose(result["d"], 0., atol=1e-9)
        np.testing.assert_allclose(result["r"], 0., atol=1e-9)
        np.testing.assert_allclose(result["s"], [[100., 100.]], atol=1e-9)
        self.assertTrue(result["polish_diagnostics"]["success"])
        self.assertEqual(result["polish_diagnostics"]["fixed_binary_variables"], 1)

    def test_g_objective_and_gap_are_recomputed_after_fixed_mode_lp(self):
        def limited_incumbent(**kwargs):
            # A feasible but unnecessarily expensive charging plan. Fixed u=1
            # still allows the LP to drop ten kWh of unnecessary purchase.
            x = np.array([110., 10., 0., 0., 0., 0., 10., 1., 0.])
            return OptimizeResult(x=x, fun=110., status=1, success=False,
                                   message="Injected time-limited incumbent",
                                   mip_gap=20. / 110., mip_dual_bound=90.)

        with patch("planners.milp", side_effect=limited_incumbent):
            result = solve_plan([100], [0], [1], 0, b=self.small,
                                terminal_weight=0, s_ref=0)
            self.assertAlmostEqual(result["g"][0], 100.)
            self.assertAlmostEqual(result["objective"], 100.)
            self.assertAlmostEqual(result["mip_diagnostics"]["objective_yuan"], 110.)
            self.assertAlmostEqual(result["polish_diagnostics"]["objective_change_yuan"], -10.)

```

```

self.assertAlmostEqual(result["mip_gap"], .1)
self.assertAlmostEqual(result["absolute_gap_yuan"], 10.)
self.assertNotAlmostEqual(result["mip_gap"], result["mip_diagnostics"]["reported_gap"])
self.assertEqual(result["status"], "feasible_unproven")
self.assertFalse(result["optimality_proven"])
self.assertEqual(result["mip_diagnostics"]["status_code"], 1)
self.assertEqual(result["polish_diagnostics"]["status_code"], 0)
self.assertGreaterEqual(result["solve_seconds"],
                        result["mip_diagnostics"]["solve_seconds"] +
                        result["polish_diagnostics"]["solve_seconds"])

def test_successful_restricted_lp_is_not_a_global_bound(self):
    incumbent = OptimizeResult(
        x=np.array([100., 0., 0., 0., 0., 0., 0., 0.]),
        fun=100., status=0, success=True, message="No global bound supplied",
        mip_gap=0., mip_dual_bound=None,
    )
    with patch("planners.milp", return_value=incumbent):
        result = solve_plan([100], [0], [1], 0, b=self.small,
                            terminal_weight=0, s_ref=0)
    self.assertTrue(result["polish_diagnostics"]["success"])
    self.assertIsNone(result["mip_gap"])
    self.assertFalse(result["optimality_proven"])

def test_failed_lp_is_explicit_and_never_returns_raw_incumbent(self):
    failure = OptimizeResult(status=2, success=False, x=None, fun=None,
                             message="Synthetic fixed-mode infeasibility")
    with patch("planners.linprog", return_value=failure):
        with self.assertRaisesRegex(RuntimeError, "Fixed-mode LP polishing failed"):
            solve_plan([10], [0], [1], 0, b=self.small,
                       terminal_weight=0, s_ref=0)

def test_limit_without_incumbent_does_not_start_polishing(self):
    failure = OptimizeResult(status=1, success=False, x=None, fun=None,
                             message="Synthetic time limit without incumbent")
    with patch("planners.milp", return_value=failure), patch("planners.linprog") as lp:
        with self.assertRaisesRegex(RuntimeError, "No usable MILP incumbent"):
            solve_plan([10], [0], [1], 0, b=self.small,
                       terminal_weight=0, s_ref=0)
    lp.assert_not_called()

if __name__ == "__main__":
    unittest.main()

```

21. question2/planning/code/test_planner_prefix.py

源文件 SHA-256: 9b59dd581ec6bb11af34531cccc86152e4f1fe6e93b514d776a1bae63c4d43d9 ; 代码行数: 106.

```

"""Synthetic checks of inherited contracts in a 00:00 planning horizon."""

import unittest

import numpy as np

from control import Battery
from planners import solve_plan

class PlannerPrefixTests(unittest.TestCase):
    def setUp(self):
        self.b = Battery(low=0, high=100, power_kw=600, eta_c=1, eta_d=1)

    def test_inherited_contract_above_optimization_cap_is_honored_and_paid(self):
        # max(load)+rate=110, but an inherited 500-kWh contract must be honored.
        # The battery is already full. Its 540-kWh supply surplus can be unused.
        out = solve_plan(
            [10], [50], [2], 100, fixed_prefix_g=[500],
            b=self.b, terminal_weight=0, s_ref=0,
        )
        self.assertAlmostEqual(out["g"][0], 500)
        self.assertGreaterEqual(out["ordinary_purchase_upper_kwh"][0], 500)
        self.assertAlmostEqual(out["v"][0, 0] + out["r"][0, 0], 540)
        self.assertAlmostEqual(out["plan_cost"], 1000)
        self.assertAlmostEqual(out["fixed_prefix_plan_cost"], 1000)
        self.assertAlmostEqual(out["new_plan_cost"], 0)
        self.assertTrue(out["checks"]["passed"])

    def test_145_slots_keep_prefix_and_exact_new_contract_cost_split(self):
        # One inherited slot and 144 new slots, each needing one kWh. The paid
        # prefix contains 5 kWh, of which 4 can be carried into the new window.
        out = solve_plan(
            np.ones(145), np.zeros(145), np.ones(145), 0,
            fixed_prefix_g=[5], b=self.b, terminal_weight=0, s_ref=0,
        )
        self.assertEqual(out["g"].shape, (145,))
        self.assertEqual(out["s"].shape, (1, 146))
        self.assertEqual(out["fixed_prefix_length"], 1)
        self.assertAlmostEqual(out["g"][0], 5)
        self.assertAlmostEqual(out["state_at_new_window_start_kwh"][0], 4)
        self.assertAlmostEqual(out["fixed_prefix_plan_cost"], 5)
        self.assertAlmostEqual(out["new_plan_cost"], 140)
        self.assertAlmostEqual(out["plan_cost"], 145)
        self.assertAlmostEqual(out["objective"], 145)
        self.assertAlmostEqual(out["plan_cost"], out["fixed_prefix_plan_cost"] + out["new_plan_cost"])

    def test_prefix_end_state_is_a_forecast_scenario_not_an_observed_state(self):
        # Midnight SOC is known (50). The unfinished prefix load is uncertain.
        # Its two forecast scenarios produce two different planned 00:10 SOC's.
        out = solve_plan(
            [30, 0], [0, 0], [1, 1], 50,
            scenario_load_kwh=[[20, 0], [40, 0]],
            scenario_pv_kwh=[[0, 0], [0, 0]], probabilities=[.5, .5],
            fixed_prefix_g=[0], b=self.b, terminal_weight=0, s_ref=0,
        )
        np.testing.assert_allclose(out["s"][:, 0], [50, 50])
        np.testing.assert_allclose(out["state_at_new_window_start_kwh"], [30, 10])
        np.testing.assert_allclose(out["g"], [0, 0], atol=1e-6)
        self.assertAlmostEqual(out["expected_emergency_cost"], 0)
        # Prefix endpoint observations are not accepted as a planning input.
        with self.assertRaises(TypeError):
            solve_plan([30, 0], [0, 0], [1, 1], 50,
                fixed_prefix_g=[0], observed_prefix_end_soc=99)

```

```

def test_fixed_zero_contract_cannot_be_replaced_with_cheaper_ordinary_purchase(self):
    # With no battery energy and no ordinary prefix purchase, 5 kWh must be
    # bought at 5 times the 2-yuan rate. The planner cannot reopen that slot.
    out = solve_plan(
        [5, 0], [0, 0], [2, 1], 0, fixed_prefix_g=[0],
        b=self.b, terminal_weight=0, s_ref=0,
    )
    self.assertAlmostEqual(out["g"][0], 0)
    self.assertAlmostEqual(out["e"][0, 0], 5)
    self.assertAlmostEqual(out["plan_cost"], 0)
    self.assertAlmostEqual(out["expected_emergency_cost"], 50)
    self.assertAlmostEqual(out["objective"], 50)

def test_multi_slot_prefix_is_unchanged(self):
    original = np.array([0., 80.])
    out = solve_plan(
        [10, 50, 20], [0, 0, 0], [1, 2, 3], 20,
        fixed_prefix_g=original, b=self.b, terminal_weight=0, s_ref=0,
    )
    np.testing.assert_array_equal(original, [0, 80])
    np.testing.assert_allclose(out["g"][:2], original)
    self.assertEqual(out["fixed_prefix_length"], 2)
    self.assertAlmostEqual(out["fixed_prefix_plan_cost"], 160)
    self.assertAlmostEqual(out["plan_cost"], out["fixed_prefix_plan_cost"] + out["new_plan_cost"])
    self.assertTrue(out["checks"]["passed"])

def test_empty_prefix_is_compatible_and_invalid_prefixes_fail(self):
    args = dict(load_kwh=[10, 20], pv_kwh=[0, 0], price=[1, 2], s0=0,
        b=self.b, terminal_weight=0, s_ref=0)
    unprefixed = solve_plan(**args)
    empty = solve_plan(**args, fixed_prefix_g=[])
    self.assertAlmostEqual(unprefixed["objective"], empty["objective"])
    self.assertEqual(empty["fixed_prefix_length"], 0)
    self.assertAlmostEqual(empty["new_plan_cost"], empty["plan_cost"])
    for bad in ([1], [1, 2, 3], [-1], [np.nan], [np.inf]):
        with self.subTest(prefix=bad), self.assertRaises(ValueError):
            solve_plan(**args, fixed_prefix_g=bad)

if __name__ == "__main__":
    unittest.main()

```

22. question2/planning/code/test_planners.py

源文件 SHA-256: cb99fdccd71a8c71d7f59a8cb62ba927cf5c8525a51460d9a14c116308b7a526 ; 代码行数: 180。

```

"""Synthetic-only planner verification; no claimed contest-data performance."""

import unittest
from unittest.mock import patch

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, milp as scipy_milp

from control import Battery
from planners import solve_plan

def direct_deterministic_model(load, pv, price, initial, battery, kappa, target):
    """Independent Q1-style deterministic MILP, with soft rather than fixed end.

    It has no emergency/unused-contract variables and no scenario formulation.
    Positive prices should make its objective match the one-scenario planner.
    """
    n = len(load)
    size = 6 * n + 1
    a = np.zeros((4 * n + 1, size))
    lower = np.zeros(size)
    upper = np.full(size, np.inf)
    integer = np.zeros(size)
    cost = np.zeros(size)
    row_lower = np.full(4 * n + 1, -np.inf)
    row_upper = np.zeros(4 * n + 1)
    for t in range(n):
        g, c, d, r, s, u = range(6 * t, 6 * t + 6)
        cost[g] = price[t]
        upper[c] = upper[d] = battery.rate
        upper[r] = pv[t]
        lower[s], upper[s] = battery.low, battery.high
        upper[u], integer[u] = 1, 1
        a[4 * t, [g, c, d, r]] = [1, -1, 1, -1]
        row_lower[4 * t] = row_upper[4 * t] = load[t] - pv[t]
        a[4 * t + 1, [c, d, s]] = [-battery.eta_c, 1 / battery.eta_d, 1]
        if t:
            a[4 * t + 1, s - 6] = -1
        row_lower[4 * t + 1] = row_upper[4 * t + 1] = initial if t == 0 else 0
        a[4 * t + 2, [c, u]] = [1, -battery.rate]
        a[4 * t + 3, [d, u]] = [1, battery.rate]
        row_upper[4 * t + 3] = battery.rate
    a[-1, [-3, -1]] = [-1, -1]
    row_upper[-1] = -target
    cost[-1] = kappa
    result = scipy_milp(
        cost, integrality=integer, bounds=Bounds(lower, upper),
        constraints=LinearConstraint(a, row_lower, row_upper),
        options={"mip_rel_gap": 1e-9, "time_limit": 30},
    )
    if not result.success:
        raise AssertionError(result.message)
    return result

class PlannerTests(unittest.TestCase):
    def setUp(self):
        self.b = Battery(low=0, high=100, power_kw=600, eta_c=1, eta_d=1)

    def test_interval_energy_is_not_divided_twice(self):
        # A caller converts 600 kW over ten minutes to 100 kWh exactly once.
        result = solve_plan([600 / 6], [0], [2], 0, b=self.b, terminal_weight=0, s_ref=0)
        self.assertAlmostEqual(result["g"][0], 100)

```



```

self.assertAlmostEqual(result["plan_cost"], 200)
self.assertAlmostEqual(self.b.rate, 600 / 6)

def test_emergency_cost_is_five_times_price(self):
    # High load occurs with probability .1. Each extra ordinary kWh costs 1,
    # but saves only .1*5=.5 expected yuan, so the optimal contract is zero.
    result = solve_plan(
        [10], [0], [1], 0, b=self.b, terminal_weight=0, s_ref=0,
        scenario_load_kwh=[[0], [100]], scenario_pv_kwh=[[0], [0]],
        probabilities=[.9, .1], mip_rel_gap=1e-9,
    )
    self.assertAlmostEqual(result["g"][0], 0)
    np.testing.assert_allclose(result["e"].ravel(), [0, 100], atol=1e-6)
    self.assertAlmostEqual(result["expected_emergency_cost"], 50)
    self.assertAlmostEqual(result["objective"], 50)

def test_initial_state_is_shared_and_terminal_is_not_a_hard_reset(self):
    result = solve_plan([50], [0], [1], 50, b=self.b, terminal_weight=0, s_ref=0)
    self.assertEqual(result["s"].shape, (1, 2))
    self.assertAlmostEqual(result["s"][0, 0], 50)
    self.assertAlmostEqual(result["s"][0, -1], 0)
    self.assertAlmostEqual(result["plan_cost"], 0)

def test_soft_terminal_penalty_is_separate_from_cash(self):
    # A target above the physical maximum makes unavoidable penalty obvious.
    result = solve_plan([0], [0], [1], 100, b=self.b, terminal_weight=2, s_ref=150)
    self.assertAlmostEqual(result["plan_cost"], 0)
    self.assertAlmostEqual(result["expected_emergency_cost"], 0)
    self.assertAlmostEqual(result["terminal_cost"], 100)
    self.assertAlmostEqual(result["objective"], 100)

def test_single_scenario_matches_independent_direct_model(self):
    rng = np.random.default_rng(12091)
    for seed in range(10):
        with self.subTest(seed=seed):
            n = 8
            load = rng.uniform(10, 170, n)
            pv = rng.uniform(0, 140, n)
            price = rng.uniform(.2, 2, n)
            battery = Battery(low=0, high=220, power_kw=600, eta_c=.9, eta_d=.9)
            initial = 80
            kappa = .7
            unified = solve_plan(load, pv, price, initial, b=battery, terminal_weight=kappa,
s_ref=100, mip_rel_gap=1e-9)
            independent = direct_deterministic_model(load, pv, price, initial, battery, kappa, 100)
            self.assertLess(abs(unified["objective"] - independent.fun), 1e-5)
            self.assertLess(np.max(unified["e"]), 1e-5)
            self.assertTrue(unified["checks"]["passed"])

def test_shared_contract_beats_average_of_per_scenario_optima(self):
    demands = np.array([0., 100.])
    probabilities = np.array([.5, .5])
    combined = solve_plan(
        [50], [0], [1], 0, b=self.b, terminal_weight=0, s_ref=0,
        scenario_load_kwh=demands[:, None], scenario_pv_kwh=np.zeros((2, 1)),
        probabilities=probabilities, mip_rel_gap=1e-9,
    )
    # Enumerate all integer contracts; this one-slot example has no initial
    # stored energy, so its expected expense has this independent formula.
    grid = np.arange(101.)
    costs = grid + 5 * np.maximum(demands[:, None] - grid[None, :], 0).T @ probabilities
    self.assertAlmostEqual(combined["objective"], float(np.min(costs)))
    separate = [solve_plan([x], [0], [1], 0, b=self.b, terminal_weight=0, s_ref=0) for x in demands]
    mean_contract = float(sum(p * r["g"][0] for p, r in zip(probabilities, separate)))
    averaged_cost = mean_contract + 5 * float(probabilities @ np.maximum(demands - mean_contract,
0))

```

```

self.assertAlmostEqual(mean_contract, 50)
self.assertAlmostEqual(averaged_cost, 175)
self.assertAlmostEqual(combined["objective"], 100)
self.assertLess(combined["objective"], averaged_cost)

def test_scenarios_obey_execution_rules_and_all_balances(self):
    rng = np.random.default_rng(38217)
    scenarios_load = rng.uniform(0, 180, (3, 8))
    scenarios_pv = rng.uniform(0, 140, (3, 8))
    result = solve_plan(
        scenarios_load.mean(axis=0), scenarios_pv.mean(axis=0),
        rng.uniform(.2, 2, 8), 55, b=self.b, terminal_weight=5, s_ref=100,
        scenario_load_kwh=scenarios_load, scenario_pv_kwh=scenarios_pv,
        probabilities=[.2, .3, .5], mip_rel_gap=1e-8,
    )
    self.assertEqual(result["g"].shape, (8,))
    self.assertEqual(result["c"].shape, (3, 8))
    np.testing.assert_allclose(result["s"][:, 0], 55)
    surplus = np.maximum(result["g"] + scenarios_pv - scenarios_load, 0)
    deficit = np.maximum(scenarios_load - result["g"] - scenarios_pv, 0)
    self.assertLessEqual(np.max(result["c"] - surplus), 1e-5)
    self.assertLessEqual(np.max(result["d"] - deficit), 1e-5)
    self.assertLess(np.max(np.minimum(result["e"], result["c"])), 1e-5)
    np.testing.assert_allclose(result["e"], deficit - result["d"], atol=1e-5)
    np.testing.assert_allclose(result["v"] + result["r"], surplus - result["c"], atol=1e-5)
    self.assertTrue(result["checks"]["passed"])

def test_feasible_limit_solution_is_not_reported_as_proven(self):
    def limited(*args, **kwargs):
        result = scipy_milp(*args, **kwargs)
        result.status = 1
        result.success = False
        result.message = "Synthetic limit-status fixture with a valid incumbent"
        result.mip_gap = .25
        return result
    with patch("planners.milp", side_effect=limited):
        result = solve_plan([100], [0], [1], 0, b=self.b, terminal_weight=0, s_ref=0)
    self.assertEqual(result["status"], "feasible_unproven")
    self.assertFalse(result["optimality_proven"])
    self.assertTrue(result["checks"]["passed"])

def test_invalid_scenarios_and_probabilities_fail_explicitly(self):
    with self.assertRaises(ValueError):
        solve_plan([1], [0], [1], 0, b=self.b, scenario_load_kwh=[[1]])
    with self.assertRaises(ValueError):
        solve_plan([1], [0], [1], 0, b=self.b, scenario_load_kwh=[[1], [2]], scenario_pv_kwh=[[0],
[0]], probabilities=[.2, .5])
    with self.assertRaises(ValueError):
        solve_plan([1], [0], [1], 101, b=self.b)

if __name__ == "__main__":
    unittest.main()

```

23. question2/planning/code/test_window_control.py

源文件 SHA-256: a251b4d1575e2e654309402cf4743fffd06283236820c6174a71cfa728c44f498 ; 代码行数: 133。

```

"""Synthetic causality, accounting and handoff tests for prefix execution."""
from __future__ import annotations

import unittest
from unittest.mock import patch

import numpy as np

import window_control
from control_refined import run_controller
from window_control import execute_prefix

class WindowControlTests(unittest.TestCase):
    def test_143_and_144_prefixes_have_correct_boundaries_and_balances(self):
        n = 145
        g = np.r_[np.tile(np.r_[np.full(24, 600.), np.full(24, 419.)], 3), 600.]
        load = np.full(n, 500.)
        pv = np.zeros(n)
        p = np.full(n, .5)
        for method in ("greedy", "mpc"):
            for steps in (143, 144):
                with self.subTest(method=method, steps=steps):
                    out = execute_prefix(method, g, load[:steps], pv[:steps], load, pv, p, 6000., steps)
                    self.assertEqual(len(out["g"]), steps)
                    self.assertEqual(len(out["s"]), steps + 1)
                    self.assertEqual(out["horizon_steps"], 145)
                    self.assertTrue(out["checks"]["passed"])
                    self.assertAlmostEqual(out["fees_plan"], float(p[:steps] @ g[:steps]))
                    np.testing.assert_allclose(out["g"] - out["v"] + out["pv"] - out["r"] + out["d"] +
                                                out["load"] + out["c"], atol=1e-6)

        out["e"],

    def test_one_step_considers_full_145_forecast_horizon(self):
        n = 145
        g, pv = np.zeros(n), np.zeros(n)
        forecast = np.r_[999., 100., np.zeros(n - 2)]
        price = np.r_[.4, 1., np.full(n - 2, .4)]
        mpc = execute_prefix("mpc", g, [100.], [0.], forecast, pv, price, 1300., 1)
        greedy = execute_prefix("greedy", g, [100.], [0.], forecast, pv, price, 1300., 1)
        self.assertAlmostEqual(mpc["d"][0], 0., places=5)
        self.assertAlmostEqual(greedy["d"][0], 90., places=5)
        self.assertAlmostEqual(mpc["fees_emergency"], 200., places=5)
        self.assertAlmostEqual(mpc["s"][-1], 1300., places=5)
        self.assertEqual(mpc["executed_steps"], 1)
        self.assertEqual(mpc["unexecuted_steps"], 144)

    def test_missing_or_extra_actuals_are_rejected(self):
        n = 145
        args = dict(method="mpc", g=np.zeros(n), load_actual=[100.], pv_actual=[0.],
                    load_forecast=np.full(n, 100.), pv_forecast=np.zeros(n),
                    price=np.full(n, .5), s0=6000., steps=2)
        with self.assertRaises(ValueError):
            execute_prefix(**args)
        with self.assertRaises(ValueError):
            execute_prefix(**args, "steps": 1, "load_actual": [100., 100.])
        for steps in (0, 146, 1.5, True):
            with self.subTest(steps=steps), self.assertRaises(ValueError):
                execute_prefix(**args, "steps": steps)

    def test_lp_only_receives_current_actual_then_forecasts(self):
        n, steps = 145, 2
        forecast = np.arange(n, dtype=float) + 100.
        pv_forecast = np.arange(n, dtype=float)

```

```

seen = []
original = window_control.solve_refined_horizon

def record(g, load, pv, price, s0, **kwargs):
    t = n - len(g)
    self.assertEqual(load[0], [150., 180.][t])
    self.assertEqual(pv[0], [0., 1.][t])
    np.testing.assert_array_equal(load[1:], forecast[t + 1:])
    np.testing.assert_array_equal(pv[1:], pv_forecast[t + 1:])
    seen.append(len(g))
    return original(g, load, pv, price, s0, **kwargs)

with patch.object(window_control, "solve_refined_horizon", side_effect=record):
    out = execute_prefix("mpc", np.zeros(n), [150., 180.], [0., 1.],
                        forecast, pv_forecast, np.full(n, .5), 1300., steps)
    self.assertEqual(seen, [145, 144])
    self.assertTrue(out["checks"]["passed"])

def test_later_actuals_do_not_change_first_action(self):
    n = 145
    forecast = np.full(n, 100.)
    price = np.linspace(.4, 1., n)
    first = execute_prefix("mpc", np.zeros(n), [110., 120.], [0., 0.],
                        forecast, np.zeros(n), price, 1300., 2)
    second = execute_prefix("mpc", np.zeros(n), [110., 9000.], [0., 8000.],
                        forecast, np.zeros(n), price, 1300., 2)
    for key in ("c", "d", "e", "v", "r"):
        self.assertAlmostEqual(first[key][0], second[key][0], places=8)
    self.assertAlmostEqual(first["s"][1], second["s"][1], places=8)

def test_split_horizon_state_and_fees_connect_without_reset(self):
    g = np.array([600., 419., 600., 419.])
    la, pv = np.full(4, 500.), np.zeros(4)
    price = np.array([.4, .7, .5, 1.])
    for method in ("greedy", "mpc"):
        full = execute_prefix(method, g, la, pv, la, pv, price, 6000., 4, terminal_weight=.4)
        first = execute_prefix(method, g, la[:3], pv[:3], la, pv, price, 6000., 3,
terminal_weight=.4)
        tail = execute_prefix(method, g[3:], la[3:], pv[3:], la[3:], pv[3:], price[3:],
                        first["s"][-1], 1, terminal_weight=.4)
        for key in ("g", "c", "d", "e", "v", "r"):
            np.testing.assert_allclose(np.r_[first[key], tail[key]], full[key], atol=1e-6)
        np.testing.assert_allclose(np.r_[first["s"], tail["s"]], full["s"], atol=1e-6)
        self.assertAlmostEqual(first["fees_total"] + tail["fees_total"], full["fees_total"],
places=5)
        self.assertAlmostEqual(tail["s"][0], first["s"][-1])

def test_full_execution_matches_existing_controller(self):
    g = np.array([600., 400., 300., 650.])
    actual = np.array([500., 600., 500., 500.])
    forecast = np.full(4, 500.)
    pv = np.zeros(4)
    price = np.array([.4, .5, 1., .3])
    original = g.copy()
    for method in ("greedy", "mpc"):
        out = execute_prefix(method, g, actual, pv, forecast, pv, price, 1500., 4)
        existing = run_controller(method, g, actual, pv, forecast, pv, price, 1500.)
        for key in ("c", "d", "e", "s"):
            np.testing.assert_allclose(out[key], existing[key], atol=1e-6)
        self.assertAlmostEqual(out["fees_total"], existing["fees_total"], places=5)
    np.testing.assert_array_equal(g, original)

def test_unexecuted_contract_is_not_billed(self):
    out = execute_prefix("greedy", [100., 9999.], [100.], [0.], [100., 0.],
                        [0., 0.], [.4, 1.], 6000., 1)
    self.assertAlmostEqual(out["fees_plan"], 40.)

```

```
self.assertEqual(out["fees_total"], 40.)
self.assertEqual(out["s"][-1], 6000.)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

24. question2/planning/code/test_window_io.py

源文件 SHA-256: e25300b9cbc5a1d3b03aeea47f5adb983a430817b7dc6b2082a89162c86c0e02 ; 代码行数: 140。

```

"""Synthetic tests of instantaneous actuals and shifted-window history timing."""

import copy
import datetime as dt
from pathlib import Path
import tempfile
import unittest

import numpy as np
from openpyxl import Workbook

from window_io import attach_actuals, risk_inputs


class WindowActualTests(unittest.TestCase):
    def setUp(self):
        self.temp=tempfile.TemporaryDirectory()
        self.root=Path(self.temp.name)
        self.days=[dt.date(2025,12,29)+dt.timedelta(days=i) for i in range(3)]

    def tearDown(self):
        self.temp.cleanup()

    def data(self,days=None):
        days=self.days if days is None else days
        starts=[dt.datetime.combine(d,dt.time(0,10)) for d in days]
        return {
            'dates':list(days),'window_start':starts,
            'window_end':[a+dt.timedelta(days=1) for a in starts],
            'issued_at':[dt.datetime.combine(d,dt.time()) for d in days],
            'load_f':np.full((len(days),144),100.),
            'pv_f':np.full((len(days),144),10.),
            'metadata':{},
        }

    def workbook(self,days=None,filename='actual.xlsx'):
        days=self.days if days is None else days
        load=np.asarray([10+1000*i+np.arange(144) for i in range(len(days))],float)
        pv=np.asarray([1+i+np.arange(144)/100 for i in range(len(days))],float)
        # Distinct extrapolation candidates with exact analytical values.
        load[-1,-6:]=[0,0,0,0,0,60]
        pv[-1,-6:]=[5,4,3,2,1,0]
        header=[f'{(t*10)//60:02d}:{(t*10)%60:02d}' for t in range(1,144)]+['00:00+1']
        book=Workbook()
        book.remove(book.active)
        for name,values in [('小区负载',load),('光伏发电实际功率',pv)]:
            sheet=book.create_sheet(name)
            sheet.append(['日期']+header)
            for d,values_row in zip(days,values):
                sheet.append([dt.datetime.combine(d,dt.time())]+values_row.tolist())
        path=self.root/filename
        book.save(path)
        return path,load,pv

    def test_first_143_means_use_adjacent_nodes_and_last_uses_next_calendar_row(self):
        path,load,pv=self.workbook()
        out=attach_actuals(self.data(),path,confirmed_instantaneous=True)
        np.testing.assert_allclose(out['load_a'][0,:143],(load[0,:143]+load[0,1:])/2)
        np.testing.assert_allclose(out['pv_a'][0,:143],(pv[0,:143]+pv[0,1:])/2)
        self.assertAlmostEqual(out['load_a'][0,143],(load[0,-1]+load[1,0])/2)
        self.assertAlmostEqual(out['load_a'][1,143],(load[1,-1]+load[2,0])/2)
        self.assertNotAlmostEqual(out['load_a'][0,143],(load[0,-1]+load[0,0])/2)
        self.assertEqual(out['metadata']['actual_source_kind'],'instantaneous_power_kw')

```

```

def test_exactly_one_final_interval_is_estimated_and_default_is_hold_last(self):
    path, load, pv=self.workbook()
    out=attach_actuals(self.data(), path, confirmed_instantaneous=True)
    self.assertEqual(out['actual_estimated'].dtype, np.dtype(bool))
    self.assertEqual(out['actual_estimated'].shape, (3, 144))
    np.testing.assert_array_equal(np.argwhere(out['actual_estimated']), [[2, 143]])
    np.testing.assert_array_equal(out['actual_observed_endpoints'], ~out['actual_estimated'])
    self.assertAlmostEqual(out['load_a'][-1, -1], load[-1, -1])
    self.assertAlmostEqual(out['pv_a'][-1, -1], pv[-1, -1])
    self.assertEqual(out['metadata']['estimated_actual_interval_count'], 1)
    interval=out['metadata']['estimated_actual_intervals'][0]
    self.assertEqual(interval['missing_node_at'], '2026-01-01T00:10:00')
    self.assertIn('not observed', interval['status'])

def test_boundary_candidates_are_endpoint_extrapolations_then_means(self):
    path, _, _=self.workbook()
    out=attach_actuals(self.data(), path, confirmed_instantaneous=True)
    candidates=out['last_boundary_candidates']
    self.assertEqual(set(candidates), {'hold', 'linear2', 'ols6'})
    self.assertAlmostEqual(candidates['hold']['load_right_endpoint_kw'], 60)
    self.assertAlmostEqual(candidates['hold']['load_mean_kw'], 60)
    self.assertAlmostEqual(candidates['linear2']['load_right_endpoint_kw'], 120)
    self.assertAlmostEqual(candidates['linear2']['load_mean_kw'], 90)
    self.assertAlmostEqual(candidates['ols6']['load_right_endpoint_kw'], 40)
    self.assertAlmostEqual(candidates['ols6']['load_mean_kw'], 50)
    for method in candidates:
        self.assertAlmostEqual(candidates[method]['pv_right_endpoint_kw'], 0)
        self.assertAlmostEqual(candidates[method]['pv_mean_kw'], 0)
        self.assertEqual(candidates[method]['interval_start'], '2026-01-01T00:00:00')
        self.assertEqual(candidates[method]['interval_end'], '2026-01-01T00:10:00')
        self.assertTrue(candidates[method]['estimated'])

def test_available_later_source_row_means_no_estimation_for_shorter_requested_range(self):
    path, load, _=self.workbook()
    out=attach_actuals(self.data(self.days[:2]), path, confirmed_instantaneous=True)
    self.assertEqual(int(out['actual_estimated'].sum()), 0)
    self.assertEqual(out['last_boundary_candidates'], {})
    self.assertAlmostEqual(out['load_a'][-1, -1], (load[1, -1]+load[2, 0])/2)

def test_instantaneous_confirmation_required_and_interior_missing_day_rejected(self):
    path, _, _=self.workbook()
    with self.assertRaises(ValueError):
        attach_actuals(self.data(), path)
    with self.assertRaises(TypeError):
        attach_actuals(self.data(), path, confirmed_start_means=True)
    gap_path, _, _=self.workbook([self.days[0], self.days[2]], 'gap.xlsx')
    with self.assertRaises(ValueError):
        attach_actuals(self.data([self.days[0], self.days[2]]), gap_path, confirmed_instantaneous=True)

def test_incomplete_previous_window_and_future_actuals_are_excluded_from_risk(self):
    days=[dt.date(2025, 12, 28)+dt.timedelta(days=i) for i in range(4)]
    path, _, _=self.workbook(days, 'four_days.xlsx')
    data=attach_actuals(self.data(days), path, confirmed_instantaneous=True)
    altered=copy.deepcopy(data)
    # At Dec31 00:00, the Dec30 forecast window is not complete until
    # Dec31 00:10. Changing it and all later actuals must not change risk.
    altered['load_a'][2:]+=100000
    altered['pv_a'][2:]+=50000
    config={'window':28, 'min_history':2, 'seed':1984, 'scenarios':6}
    for method in ('C', 'D'):
        before=risk_inputs(data, 3, method, config, .8)
        after=risk_inputs(altered, 3, method, config, .8)
        self.assertEqual(before[4]['history_indices'], [0, 1])
        self.assertTrue(before[4]['previous_incomplete_window_excluded'])
        for a, b in zip(before[:4], after[:4]):
            if a is None:

```

```
        self.assertIsNone(b)
    else:
        np.testing.assert_array_equal(a,b)
    self.assertEqual(before[4],after[4])
    if method=='D':
        self.assertTrue(all(i≤1 for i in before[4]['scenario_history_indices']))

if __name__=='__main__':
    unittest.main()
```

25. question2/planning/code/test_window_study.py

源文件 SHA-256: 4656eee2ebd243818510367a0c43b8b62049c2c00da55b1d519d30b232cf44d2 ; 代码行数: 200。


```

"""Midnight-bridge integration tests with real physical prefix execution.

The inexpensive test planner is deliberately state-dependent and is not a
candidate dispatch model. Three complete 144-slot windows isolate scheduling
order, causal handoff, contract preservation and cash bookkeeping.
"""
from __future__ import annotations

import csv
import datetime as dt
import json
from pathlib import Path
import tempfile
import unittest
from unittest.mock import patch

import numpy as np

import window_study as ws

def fixture(days=3):
    dates = [dt.date(2030, 2, 1) + dt.timedelta(days=i) for i in range(days)]
    origins = [dt.datetime.combine(day, dt.time()) for day in dates]
    starts = [origin + dt.timedelta(minutes=10) for origin in origins]
    return {
        "dates": dates, "issued_at": origins, "window_start": starts,
        "window_end": [start + dt.timedelta(days=1) for start in starts],
        "load_f": np.full((days, 144), 600.), "pv_f": np.zeros((days, 144)),
        "load_a": np.full((days, 144), 600.), "pv_a": np.zeros((days, 144)),
        "actual_estimated": np.zeros((days, 144), dtype=bool),
        "metadata": {"role": "synthetic_midnight_bridge_test"},
    }

def config():
    return dict(initial_soc=6000., calibration_days=1, default_quantile=.8,
                inventory_value=.5 / .9, terminal_weight=.5 / .9,
                milp_seconds=30., mip_gap=1e-5, s_ref=6000.)

class MidnightBridgeTests(unittest.TestCase):
    def replay(self, changed_final_actual=False, method="B"):
        data = fixture()
        if changed_final_actual:
            # The interval [next midnight, next 00:10] is not observed yet when
            # the next midnight contract is signed.
            data["load_a"][0, -1] = 1200.
        p = np.full(144, .5)
        ws.initialize(data, p, config())
        events, plan_records, execution_records = [], [], []
        original_execute = ws.execute

        def cheap_state_dependent_plan(i, mode, s0, q, prefix=None):
            events.append(("plan", i))
            # Restore any inherited deficit gradually over the *new* window.
            new_g = np.full(144, 100. + (6000. - s0) / (.9 * 144.))
            contract = new_g if prefix is None else np.r_[prefix[0], new_g]
            plan_records.append({"i": i, "s0": float(s0), "prefix": prefix,
                                "new_g": new_g.copy(), "full_g": contract.copy()})

            pl = {
                "g": contract, "status": "optimal_within_requested_gap", "mip_gap": 0.,
                "checks": {"passed": True, "scope": "mock planner; physical execution independently
checked"}},

```

```

        "solve_seconds": 0., "expected_emergency_cost": 0.,
        "objective": float(.5 * contract.sum()),
        "state_at_new_window_start_kwh": float(s0),
    }
    return pl, {"history_days": 0, "fallback": False, "test_planner": True}

def record_execution(mode, g, la, pa, lf, pf, price, s0, steps):
    if steps == 143:
        name = ("body", sum(e[0] == "body" for e in events))
    else:
        name = ("tail", sum(e[0] == "tail" for e in events))
    events.append(name)
    out = original_execute(mode, g, la, pa, lf, pf, price, s0, steps)
    execution_records.append({"event": name, "s0": float(s0),
        "g_horizon": np.asarray(g).copy(), "forecast_horizon": np.asarray(lf).copy(),
        "actual": np.asarray(la).copy(), "steps": steps, "out": out})
    return out

with tempfile.TemporaryDirectory() as tmp:
    with patch.object(ws, "plan", side_effect=cheap_state_dependent_plan), \
        patch.object(ws, "execute", side_effect=record_execution):
        result = ws.continuous_job((method, .8, tmp, 3, 6000.))
    with (Path(tmp) / f"{method}_trace.csv").open(encoding="utf-8-sig", newline="") as f:
        trace = list(csv.DictReader(f))
    logs = json.loads((Path(tmp) / f"{method}_solver.json").read_text(encoding="utf-8"))
    return result, trace, events, plan_records, execution_records, logs

def test_new_plan_precedes_unobserved_old_tail_feedback(self):
    _, _, events, plans, executions, logs = self.replay(True)
    self.assertEqual(events, [("plan", 0), ("body", 0), ("plan", 1), ("tail", 0),
        ("body", 1), ("plan", 2), ("tail", 1), ("body", 2), ("tail", 2)])
    self.assertEqual(plans[1]["prefix"], (100., 600., 0.))
    self.assertEqual(plans[1]["s0"], 6000.)
    bridge = next(x for x in executions if x["event"] == ("tail", 0))
    self.assertEqual(len(bridge["g_horizon"]), 145)
    self.assertEqual(len(bridge["actual"]), 1)
    self.assertEqual(bridge["g_horizon"][0], plans[0]["new_g"][-1])
    self.assertEqual(bridge["forecast_horizon"][0], 600.)
    self.assertEqual(bridge["actual"][0], 1200.)
    self.assertAlmostEqual(logs[1]["executed_soc_at_window_start"], 6000. - 100. / .9)

def test_old_tail_perturbation_cannot_change_just_signed_new_contract(self):
    for method in ("A", "B"):
        with self.subTest(method=method):
            base = self.replay(False, method)
            changed = self.replay(True, method)
            bp, cp = base[3], changed[3]
            np.testing.assert_array_equal(bp[0]["new_g"], cp[0]["new_g"])
            np.testing.assert_array_equal(bp[1]["new_g"], cp[1]["new_g"])
            self.assertEqual(bp[1]["s0"], cp[1]["s0"])
            self.assertGreater(np.max(np.abs(bp[2]["new_g"] - cp[2]["new_g"])), .1)
            self.assertNotAlmostEqual(base[5][1]["executed_soc_at_window_start"],
                changed[5][1]["executed_soc_at_window_start"])

def test_window_states_and_slot_times_are_contiguous(self):
    result, trace, _, _, _ = self.replay(True)
    days = result["daily"]
    self.assertEqual(len(days), 3)
    self.assertEqual(len(trace), 3 * 144)
    self.assertEqual(trace[0]["interval_start"], "2030-02-01T00:10:00")
    self.assertEqual(trace[-1]["interval_end"], "2030-02-04T00:10:00")
    for i in (1, 2):
        self.assertAlmostEqual(days[i - 1]["final_soc_kwh"], days[i]["initial_soc_kwh"])
    for a, b in zip(trace, trace[1:]):
        self.assertEqual(a["interval_end"], b["interval_start"])
        self.assertAlmostEqual(float(a["soc_end_kwh"]), float(b["soc_start_kwh"]))

```

```

self.assertEqual(len({r["interval_start"] for r in trace}), len(trace))
self.assertTrue(all(r["planner_checks_passed"] and r["controller_checks_passed"] for r in days))

def test_prefix_cost_is_counted_once_in_its_own_window(self):
    result, trace, _, plans, _, logs = self.replay(True)
    days = result["daily"]
    for i, day in enumerate(days):
        rows = trace[i * 144:(i + 1) * 144]
        self.assertEqual(len(rows), 144)
        np.testing.assert_allclose([float(r["g_kwh"]) for r in rows], plans[i]["new_g"])
        cash = sum(float(r["cash_cost_yuan"]) for r in rows)
        planned = sum(float(r["price_yuan_per_kwh"]) * float(r["g_kwh"]) for r in rows)
        emergency = sum(5. * float(r["price_yuan_per_kwh"]) * float(r["emergency_kwh"]) for r in
rows)

        self.assertAlmostEqual(cash, day["cash_cost_yuan"], places=6)
        self.assertAlmostEqual(planned, day["planned_cost_yuan"], places=6)
        self.assertAlmostEqual(emergency, day["emergency_cost_yuan"], places=6)
        self.assertAlmostEqual(planned, logs[i]["new_contract_cost"], places=6)
        expected_contract_cash = sum(.5 * r["new_g"].sum() for r in plans)
        self.assertAlmostEqual(sum(r["planned_cost_yuan"] for r in days), expected_contract_cash,
places=6)

def test_plan_passes_fixed_prefix_to_milp_and_preserves_new_forecasts(self):
    data = fixture()
    p = np.linspace(.4, .8, 144)
    ws.initialize(data, p, config())
    lf = np.arange(144, dtype=float) + 600.
    pf = np.arange(144, dtype=float)
    sl = np.stack((lf, lf + 6.))
    sp = np.stack((pf, pf + 3.))
    seen = {}

    def fake_solve(load_kwh, pv_kwh, price, s0, **kwargs):
        seen.update(load=np.asarray(load_kwh), pv=np.asarray(pv_kwh), price=np.asarray(price),
s0=s0, **kwargs)
        return {"g": np.r_[123., np.full(144, 100.)]}

    with patch.object(ws, "risk_inputs", return_value=(lf, pf, sl, sp, {})), \
        patch.object(ws, "solve_plan", side_effect=fake_solve):
        _, meta = ws.plan(1, "D", 5432., .8, prefix=(123., 900., 60.))
    self.assertEqual(seen["fixed_prefix_g"], [123.])
    self.assertEqual(seen["s0"], 5432.)
    self.assertEqual(len(seen["load"]), 145)
    self.assertEqual(seen["load"][0], 150.)
    self.assertEqual(seen["pv"][0], 10.)
    self.assertEqual(seen["price"][0], p[-1])
    np.testing.assert_array_equal(seen["load"][1:], lf / 6.)
    np.testing.assert_array_equal(seen["pv"][1:], pf / 6.)
    np.testing.assert_array_equal(seen["scenario_load_kwh"][:, 0], [150., 150.])
    np.testing.assert_array_equal(seen["scenario_pv_kwh"][:, 0], [10., 10.])
    np.testing.assert_array_equal(seen["scenario_load_kwh"][:, 1:], sl / 6.)
    self.assertIn("not future actual", meta["bridge_prefix"])

def test_incomplete_old_window_is_not_used_for_margin_or_scenarios(self):
    data = fixture()
    cfg = {**config(), "window": 28, "min_history": 1, "scenarios": 2, "seed": 20260911}
    ws.initialize(data, np.full(144, .5), cfg)
    for method in ("C", "D"):
        data["load_a"][1, -1] = 600.
        before = ws.risk_inputs(data, 2, method, cfg, .8)
        data["load_a"][1, -1] = 90000.
        after = ws.risk_inputs(data, 2, method, cfg, .8)
        self.assertEqual(before[-1]["history_indices"], [0])
        self.assertEqual(after[-1]["history_indices"], [0])
        for a, b in zip(before[:4], after[:4]):
            if a is None:

```

```
        self.assertIsNone(b)
    else:
        np.testing.assert_array_equal(a, b)

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

26. question2/planning/code/window_control.py

源文件 SHA-256: cb2fb5ecdf7d33ce6636e74542083b915d5176941a6a3838c8aedd711372617c ; 代码行数: 129。

"""Execute an observed prefix while retaining the full fixed-contract horizon.

All flows are kWh per ten-minute interval. Actual arrays contain exactly the executed prefix, never padded future observations. At each step only its current actual mean enters the LP; all later inputs remain the supplied forecast. This is the explicitly declared fast within-interval feedback approximation.

The module has no timestamp rules. In the midnight-issue Q2 workflow a caller can execute 143 old-window slots, formulate the next contract at midnight using the measured state and old final contract slot, then execute that one old slot against a 145-slot horizon consisting of old prefix plus the new 144 slots.

```

"""
from __future__ import annotations

from numbers import Integral
from time import perf_counter

import numpy as np

from control import (
    Battery, DEFAULT_BATTERY, PHYSICAL_TOL, _initial_state, _vector,
    check_trajectory, greedy_step,
)
from control_refined import solve_refined_horizon

def execute_prefix(
    method, g, load_actual, pv_actual, load_forecast, pv_forecast, price, s0,
    steps, terminal_weight=None, s_ref=6000., b: Battery = DEFAULT_BATTERY,
) → dict:
    """Execute ``steps`` slots, considering all ``n`` forecast/contract slots.

    Require  $1 \leq \text{steps} \leq n$ .  $g/\text{forecasts}/\text{price}$  have length  $n$ ; each actual vector
    has exactly  $\text{steps}$  entries. ``mpc`` optimizes the full remaining suffix and
    implements only its first action. ``greedy`` maximizes feasible current
    charging/discharging. Both keep every supplied ordinary purchase fixed.

    The default  $\kappa$  is  $\min(\text{full supplied price})/\eta_{\text{c}}$ , held constant during
    the call. Its soft SOC target belongs to the *full* horizon end, which may
    lie after the executed prefix. Reported fees include only executed slots
    and never include terminal penalties or the unexecuted ordinary contract.

    Return  $g, c, d, e, v, r, w, \text{price}/\text{load}/\text{pv}$  with  $\text{steps}$  entries,  $s$  with  $\text{steps}+1$  entries,
    the prefix cash fees, independent physical checks and solver timing.
    The caller connects subsequent execution by passing result['s'][-1].
    """
    if method not in ("greedy", "mpc"):
        raise ValueError("method must be 'greedy' or 'mpc'.")
    g = _vector(g, "g")
    n = len(g)
    if isinstance(steps, bool) or not isinstance(steps, Integral) or not  $1 \leq \text{steps} \leq n$ :
        raise ValueError("steps must be an integer with  $1 \leq \text{steps} \leq \text{horizon length}$ .")
    steps = int(steps)
    lf = _vector(load_forecast, "load_forecast", n)
    pf = _vector(pv_forecast, "pv_forecast", n)
    p = _vector(price, "price", n)
    # Exact prefix length is intentional: no future actuals are needed or used.
    la = _vector(load_actual, "load_actual", steps)
    pa = _vector(pv_actual, "pv_actual", steps)
    weight = float(np.min(p) / b.eta_c if terminal_weight is None else terminal_weight)
    if not np.isfinite(weight) or weight < 0 or not np.isfinite(s_ref):
        raise ValueError("Terminal weight/reference must be finite; weight  $\geq 0$ .")
    fixed_contract = g.copy()
    s = np.empty(steps + 1)

```

```

s[0] = _initial_state(s0, b)
actions = {key: np.empty(steps) for key in ("c", "d", "e", "v", "r", "w")}
solve_seconds = 0.
solve_count = 0
surplus_steps = 0
primary_errors = []
start = perf_counter()
for t in range(steps):
    net = g[t] + pa[t] - la[t]
    if method == "greedy" or net ≥ 0:
        step = greedy_step(g[t], la[t], pa[t], s[t], b)
        for key in actions:
            actions[key][t] = step[key]
        if method == "mpc" and net > 0:
            surplus_steps += 1
    else:
        remaining_load = lf[t:].copy()
        remaining_pv = pf[t:].copy()
        remaining_load[0] = la[t]
        remaining_pv[0] = pa[t]
        horizon = solve_refined_horizon(
            g[t:], remaining_load, remaining_pv, p[t:], s[t],
            b=b, terminal="shortfall", weight=weight, s_ref=s_ref,
        )
        for key in actions:
            actions[key][t] = horizon[key][0]
        solve_seconds += horizon["solve_seconds"]
        solve_count += horizon["solve_count"]
        primary_errors.append(horizon["primary_cost_error_yuan"])
    # Independent accumulation from the actually executed first action.
    s[t + 1] = s[t] + b.eta_c * actions["c"][t] - actions["d"][t] / b.eta_d
    if s[t + 1] < b.low - PHYSICAL_TOL or s[t + 1] > b.high + PHYSICAL_TOL:
        raise RuntimeError("Executed prefix action violates battery state bounds.")
    prefix_price, prefix_g = p[:steps].copy(), g[:steps].copy()
    fees_plan = float(prefix_price @ prefix_g)
    fees_emergency = float(5. * prefix_price @ actions["e"])
    out = {
        **actions, "g": prefix_g, "s": s, "price": prefix_price,
        "load": la, "pv": pa, "load_forecast": lf[:steps].copy(),
        "pv_forecast": pf[:steps].copy(), "method": method,
        "fees_plan": fees_plan, "fees_emergency": fees_emergency,
        "fees_total": fees_plan + fees_emergency,
        "solve_seconds": solve_seconds, "solve_count": solve_count,
        "execution_seconds": perf_counter() - start,
        "horizon_steps": n, "executed_steps": steps,
        "unexecuted_steps": n - steps,
        "terminal": "shortfall" if method == "mpc" else "none",
        "terminal_weight": weight if method == "mpc" else 0.,
        "weight": weight if method == "mpc" else 0.,
        "s_ref": float(s_ref), "terminal_reference_boundary": n,
        "forced_surplus_charge_steps": surplus_steps,
        "max_primary_cost_error_yuan": max([abs(e) for e in primary_errors] or [0.]),
        "input_flow_units": "kWh_per_10min_interval",
        "observation_assumption": "current_interval_mean_as_fast_feedback_proxy",
        "future_inputs": "forecast_only_no_padded_actuals",
        "fee_scope": "executed_prefix_only_no_terminal_penalty",
        "success": True, "status_code": 0,
    }
}
out["checks"] = check_trajectory(out, b)
unchanged = bool(np.array_equal(g, fixed_contract))
out["checks"]["full_fixed_contract_unchanged"] = unchanged
out["checks"]["passed"] = bool(out["checks"]["passed"] and unchanged)
if not out["checks"]["passed"]:
    raise RuntimeError(f"Prefix execution checks failed: {out['checks']}")
return out

```

27. question2/planning/code/window_io.py

源文件 SHA-256: f77b158c94ffa09a28c2e5bbf40451d9a10022d6b75e012510f3f3e36886a0e7 ; 代码行数: 212。

```

"""Confirmed shifted windows: issue at 00:00, cover 00:10 to next 00:10."""
import datetime as dt
import json
from pathlib import Path
import numpy as np
from openpyxl import load_workbook
from input_io import interval_header, clock_minute, timestamp, read_price, sha, write_csv

ROOT=Path(__file__).resolve().parents[1]

def read_team_forecasts(path):
    w=load_workbook(path, read_only=True, data_only=True)
    out={}; meta=[]; dates_ref=None
    try:
        for sheet, key in [('XGBoost_负载', 'load_f'), ('自适应加权_光伏', 'pv_f')]:
            s=w[sheet]; rows=list(s.values)
            bounds=[interval_header(x) for x in rows[0][1:]]
            if bounds!=[(i*10, i*10+10) for i in range(1, 145)]:
                raise ValueError('Expected 144 intervals from 00:10 through next-day 00:10')
            dates=[timestamp(r[0]).date() for r in rows[1:]]
            if dates!=sorted(set(dates)) or any(b-a!=dt.timedelta(days=1) for a, b in
zip(dates, dates[1:])):
                raise ValueError('Forecast dates must be unique and consecutive')
            if dates_ref is not None and dates!=dates_ref: raise ValueError('Load/PV dates differ')
            dates_ref=dates
            v=np.asarray([r[1:] for r in rows[1:]], float)
            if v.shape!=(len(dates), 144) or not np.isfinite(v).all() or (v<0).any():
                raise ValueError('Missing/invalid forecast power')
            out[key]=v; meta.append({'sheet': sheet, 'days': len(dates), 'intervals': 144})
    finally: w.close()
    out['dates']=dates_ref
    out['window_start']=[dt.datetime.combine(d, dt.time(0, 10)) for d in dates_ref]
    out['window_end']=[a+dt.timedelta(days=1) for a in out['window_start']]
    out['issued_at']=[a-dt.timedelta(minutes=10) for a in out['window_start']]
    out['metadata']={'forecast_sha256': sha(path), 'sheets': meta, 'value_kind': 'interval_mean_kw',
        'issue_basis': 'user-declared midnight issuance; forecast generator not independently audited',
        'window': 'D 00:10 to D+1 00:10, 144 intervals',
        'current_day_00_00_column_deleted': True, 'next_day_first_interval_retained': True,
        'actual_feedback': 'current interval mean is fast intra-interval feedback approximation'}
    return out

def _last_endpoint_candidates(last_nodes):
    """Estimate the one unavailable next ten-minute instantaneous endpoint.

    The returned endpoint is clipped nonnegative before averaging. Every method
    uses only the final observed nodes: hold-last, two-node linear extrapolation,
    or an OLS line through the final six equally spaced instantaneous samples.
    These estimates are not measured data or a claim of midnight periodicity.
    """
    y=np.asarray(last_nodes, float)
    if y.shape!=(144,) or not np.isfinite(y).all() or (y<0).any():
        raise ValueError('Boundary extrapolation requires 144 valid instantaneous nodes')
    x=np.arange(6, dtype=float)
    recent=y[-6:]
    slope=float(np.sum((x-x.mean())*(recent-recent.mean()))/np.sum((x-x.mean())**2))
    return {
        'hold': float(y[-1]),
        'linear2': float(max(2*y[-1]-y[-2], 0.)),
        'ols6': float(max(recent.mean()+slope*(6-x.mean()), 0.)),
    }

```



```

def attach_actuals(data, path, *, confirmed_instantaneous=False):
    """Convert confirmed instantaneous nodes to shifted-window interval means.

    Source row D contains 144 instantaneous nodes from D 00:10 through D+1
    00:00. The first 143 interval means use adjacent nodes in that row. The
    final interval is (D+1 00:00 + next row's D+1 00:10)/2. The next row is
    selected by its true date; the first row of a year is never recycled.

    Only the source file's final missing right endpoint may be estimated. Its
    default is hold-last, with linear2/ols6 retained for a separately labelled
    boundary sensitivity. actual_estimated marks intervals containing an
    estimated endpoint. Even unmarked interval means use the declared linear
    interpolation approximation between measured instantaneous endpoints.
    """
    if not confirmed_instantaneous:
        raise ValueError('Explicit confirmed_instantaneous=True is required for instantaneous actual
nodes')
    wanted=list(data['dates'])
    if not wanted:
        raise ValueError('No forecast windows supplied for actual alignment')
    w=load_workbook(path, read_only=True, data_only=True)
    source={}
    source_dates=None
    try:
        for sheet, key in [('小区负载', 'load_a'), ('光伏发电实际功率', 'pv_a')]:
            rows=list(w[sheet].values)
            if len(rows)<2 or [clock_minute(v) for v in rows[0][1:]]!=list(range(10, 1441, 10)):
                raise ValueError('Actual source must contain 00:10 through next 00:00 instantaneous
nodes')
            dates=[timestamp(r[0]).date() for r in rows[1:]]
            if dates!=sorted(set(dates)):
                raise ValueError('Actual source dates must be unique and sorted')
            if any(b-a!=dt.timedelta(days=1) for a, b in zip(dates, dates[1:])):
                raise ValueError('Missing interior day in instantaneous actual source')
            if source_dates is not None and dates!=source_dates:
                raise ValueError('Load and PV instantaneous source dates differ')
            source_dates=dates
            values=np.asarray([r[1:] for r in rows[1:]], float)
            if values.shape!=(len(dates), 144) or not np.isfinite(values).all() or (values<0).any():
                raise ValueError('Missing or invalid instantaneous actual powers')
            source[key]={day: values[i] for i, day in enumerate(dates)}
    finally:
        w.close()
    if any(day not in source['load_a'] for day in wanted):
        raise ValueError('Forecast window date missing from actual instantaneous source')
    estimated=np.zeros((len(wanted), 144), dtype=bool)
    converted={key: np.empty((len(wanted), 144)) for key in ('load_a', 'pv_a')}
    boundary_candidates={}
    estimated_intervals=[]
    last_source_day=source_dates[-1]
    for i, day in enumerate(wanted):
        following_day=day+dt.timedelta(days=1)
        endpoint_missing=following_day not in source['load_a']
        if endpoint_missing and day!=last_source_day:
            raise ValueError('A missing interior right endpoint cannot be replaced by a year-boundary
estimate')
        for key in ('load_a', 'pv_a'):
            nodes=source[key][day]
            converted[key][i, :143]=(nodes[:-1]+nodes[1:])/2
            if not endpoint_missing:
                right_endpoint=float(source[key][following_day][0])
            else:
                estimates=_last_endpoint_candidates(nodes)
                right_endpoint=estimates['hold']
                label='load' if key=='load_a' else 'pv'
                for method, endpoint in estimates.items():

```

```

        candidate=boundary_candidates.setdefault(method, {})
        candidate[f'{label}_right_endpoint_kw']=float(endpoint)
        candidate[f'{label}_mean_kw']=float((nodes[-1]+endpoint)/2)
        candidate['interval_start']=dt.datetime.combine(following_day, dt.time()).isoformat()

candidate['interval_end']=dt.datetime.combine(following_day, dt.time(0,10)).isoformat()
        candidate['estimated']=True

candidate['value_kind']='interval_mean_kw_from_observed_left_and_estimated_right_endpoint'
        converted[key][i,143]=(nodes[-1]+right_endpoint)/2
    if endpoint_missing:
        estimated[i,143]=True
        estimated_intervals.append({
            'window_date':str(day), 'window_index':i, 'interval_index':143,
            'interval_start':dt.datetime.combine(following_day, dt.time()).isoformat(),
            'interval_end':dt.datetime.combine(following_day, dt.time(0,10)).isoformat(),
            'missing_node_at':dt.datetime.combine(following_day, dt.time(0,10)).isoformat(),
            'default_method':'hold', 'status':'estimated, not observed',
        })
    data.update(converted)
    data['actual_estimated']=estimated
    data['actual_observed_endpoints']=~estimated
    data['last_boundary_candidates']=boundary_candidates
    metadata=data.setdefault('metadata', {})
    metadata.update({
        'actual_sha256':sha(path),
        'actual_source_kind':'instantaneous_power_kw',
        'actual_value_kind':'interval_mean_kw_from_adjacent_instantaneous_nodes',
        'actual_mapping':'first 143 adjacent-node means; last mean crosses to next calendar row first
00:10 node',
        'actual_mean_assumption':'linear interpolation/trapezoidal mean between ten-minute instantaneous
endpoints',
        'actual_available_at':'interval_end when both endpoint observations exist (no reporting latency
assumed)',
        'actual_estimated_mask_field':'actual_estimated: bool[windows,144]; true means a right endpoint
was estimated',
        'actual_observed_mask_field':'actual_observed_endpoints: inverse mask; both endpoints observed,
interval mean remains an interpolation approximation',
        'estimated_actual_interval_count':int(estimated.sum()),
        'estimated_actual_intervals':estimated_intervals,
        'last_boundary_default':'hold last observed instantaneous node; not a periodic copy from year
start',
        'last_boundary_candidates':boundary_candidates,
    })
    return data

def prices(path):
    natural,meta=read_price(path)
    # Price endpoints already averaged; rotate once, never rotate values alone.
    return np.r_[natural[1:],natural[0]],meta

def write_canonical(data,out):
    out=Path(out);out.mkdir(exist_ok=True,parents=True)
    records=[]
    for i,date in enumerate(data['dates']):
        for t in range(144):
            a=data['window_start'][i]+dt.timedelta(minutes=10*t)
            r={'plan_date':str(date), 'issued_at':data['issued_at'][i].isoformat(),
                'interval_start':a.isoformat(), 'interval_end':(a+dt.timedelta(minutes=10)).isoformat(),
                'load_forecast_kw':data['load_f'][i,t], 'pv_forecast_kw':data['pv_f'][i,t]}
            if 'load_a' in data:r.update(load_actual_kw=data['load_a'][i,t],pv_actual_kw=data['pv_a']
[i,t],
                actual_estimated=bool(data['actual_estimated'][i,t]))
            records.append(r)

```

```

write_csv(out/'canonical_intervals.csv',records)

(out/'input_audit.json').write_text(json.dumps(data['metadata'],ensure_ascii=False,indent=2),encoding='utf-8')

def risk_inputs(data,i,method,config,q):
    issued=data['issued_at'][i]
    history=[j for j in range(i) if data['window_end'][j]≤issued][-config['window']:]
    lf,pf=data['load_f'][i].copy(),data['pv_f'][i].copy()
    meta={'method':method,'history_indices':history,'history_days':len(history),
          'issued_at':str(issued),'fallback':False,'previous_incomplete_window_excluded':i>0}
    if method in ('A','B'):return lf,pf,None,None,meta
    if len(history)<config['min_history']:
        meta.update(fallback=True,fallback_reason='insufficient fully completed forecast windows')
        return lf,pf,None,None,meta
    lr=data['load_a'][history]-data['load_f'][history]
    pr=data['pv_a'][history]-data['pv_f'][history]
    if method=='C':
        margin=np.zeros(144) if q==0 else np.maximum(np.quantile(lr-pr,q,axis=0),0)
        meta.update(quantile=q,margin_kwh=float(margin.sum()/6))
        return lf+margin,pf,None,None,meta
    if method!='D':raise ValueError('Unknown method')
    rng=np.random.default_rng(config['seed']+1009*i)
    draw=rng.integers(len(history),size=config['scenarios'])
    sl=np.maximum(lf+lr[draw],0);sp=np.maximum(pf+pr[draw],0)
    dark=(pf==0)&np.all(data['pv_f'][history]==0,axis=0)&np.all(data['pv_a'][history]==0,axis=0)
    sp[:,dark]=0
    meta.update(scenario_history_indices=np.asarray(history)[draw].tolist(),
                scenarios=config['scenarios'],oracle_recourse_approximation=True)
    return lf,pf,sl,sp,meta

```

28. question2/planning/code/window_study.py

源文件 SHA-256: 0dffdb52daf074ad69fe8cfac3c84e1b5df3560d2da29596da9127161cd57cb2 ; 代码行数: 282。

```

"""Ordered supplied-data evaluation with causal midnight contract bridging."""
import argparse, csv, datetime as dt, hashlib, json, multiprocessing as mp, time
from concurrent.futures import ProcessPoolExecutor, as_completed
from pathlib import Path
import numpy as np
from window_io import read_team_forecasts, attach_actuals, prices, write_canonical, risk_inputs
from window_control import execute_prefix
from control import check_trajectory, DEFAULT_BATTERY
from planners import solve_plan
from input_io import write_csv
from study import json_write, daily_record, summarize

ROOT=Path(__file__).resolve().parents[1]
DATA=None; PRICE=None; CONFIG=None

class ExperimentPool(ProcessPoolExecutor):
    """Stop this experiment's own workers promptly after an error/interruption."""
    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            for process in list((self._processes or {}).values()):
                process.terminate()
            return super().__exit__(exc_type, exc_val, exc_tb)

def calibration_indices(data, cut, minimum=14, tail=7):
    return [i for i in range(cut) if data['window_end'][i] ≤ data['issued_at'][cut]
            and sum(data['window_end'][j] ≤ data['issued_at'][i] for j in range(i)) ≥ minimum][-tail:]

def initialize(data, p, config):
    global DATA, PRICE, CONFIG
    DATA, PRICE, CONFIG = data, p, config

def plan(i, method, s0, q, prefix=None):
    lf, pf, sl, sp, meta = risk_inputs(DATA, i, method, CONFIG, q)
    p = PRICE; fixed = None
    if prefix is not None:
        old_g, old_l, old_pv = prefix
        lf = np.r_[old_l, lf]; pf = np.r_[old_pv, pf]; p = np.r_[PRICE[-1], PRICE]
        if sl is not None:
            sl = np.column_stack((np.full(len(sl), old_l), sl))
            sp = np.column_stack((np.full(len(sp), old_pv), sp))
        fixed = [old_g]
        meta['bridge_prefix'] = 'old forecast and fixed contract, not future actual'
    pl = solve_plan(lf/6, pf/6, p, s0, scenario_load_kwh=None if sl is None else sl/6,
                    scenario_pv_kwh=None if sp is None else sp/6, fixed_prefix_g=fixed,
                    terminal_weight=CONFIG['terminal_weight'], s_ref=6000,
                    time_limit=CONFIG['milp_seconds'], mip_rel_gap=CONFIG['mip_gap'])
    return pl, meta

def execute(method, g, la, pa, lf, pf, p, s0, steps):
    return execute_prefix('greedy' if method=='A' else 'mpc', g, np.asarray(la)/6, np.asarray(pa)/6,
                          np.asarray(lf)/6, np.asarray(pf)/6, p, s0, steps,
                          terminal_weight=CONFIG['terminal_weight'], s_ref=6000)

def record(i, method, stage, pl, op, meta):
    r = daily_record(DATA, i, method, stage, pl, op, meta, CONFIG)
    r.update(window_start=str(DATA['window_start'][i]), window_end=str(DATA['window_end'][i]),
            issued_at=str(DATA['issued_at'][i]), actual_estimated_intervals=int(DATA['actual_estimated']
[i].sum()),

```

```

        dayahead_objective_scope='full optimization horizon including old fixed prefix when present',
        initial_state_basis='experiment prescribed' if stage!='continuous' or i==0 else 'actual
execution from preceding window')
        est=DATA['actual_estimated'][i]
        r['estimated_interval_emergency_yuan']=float(np.sum(5*PRICE[est]*op['e'][est]))
        r['known_contract_plus_observed_emergency_yuan']=r['cash_cost_yuan']-
r['estimated_interval_emergency_yuan']
        return r

def local_job(job):
    i,mode,q=job;s0=CONFIG['initial_soc'];m='B' if mode=='AB' else mode
    try:pl,meta=plan(i,m,s0,q)
    except Exception as e:raise RuntimeError(f'Window {DATA["dates"][i]}, method {mode}: {e}') from e
    output=[]
    for method in ('A','B') if mode=='AB' else (mode,):
        op=execute(method,pl['g'],DATA['load_a'][i],DATA['pv_a'][i],DATA['load_f'][i],DATA['pv_f']
[i],PRICE,s0,144)
        output.append(record(i,method,'same_contract' if mode=='AB' else 'local_dayahead',pl,op,meta))
    return {'rows':output,'risk':meta,'checks':pl['checks'],'gap':pl['mip_gap'],'status':pl['status'],
        'mip_diagnostics':pl.get('mip_diagnostics'),'polish_diagnostics':pl.get('polish_diagnostics')}

def merge(prefix,tail):
    out={k:np.r_[prefix[k],tail[k]] for k in ('g','c','d','e','v','r','w','price','load','pv')}
    if abs(prefix['s'][-1]-tail['s'][0])>1e-6:raise AssertionError('SOC broken at midnight')
    out['s']=np.r_[prefix['s'],tail['s'][1:]]
    for k in ('fees_plan','fees_emergency','fees_total','solve_seconds'):out[k]=prefix[k]+tail[k]
    out['checks']=check_trajectory(out)
    if not out['checks']['passed']:raise AssertionError(out['checks'])
    return out

def trace_rows(i,method,op):
    for t in range(144):
        a=DATA['window_start'][i]+dt.timedelta(minutes=10*t)
        yield {'method':method,'plan_date':str(DATA['dates'][i]),'interval_start':a.isoformat(),
            'interval_end':(a+dt.timedelta(minutes=10)).isoformat(),'price_yuan_per_kwh':PRICE[t],
            'load_forecast_kw':DATA['load_f'][i,t],'pv_forecast_kw':DATA['pv_f'][i,t],
            'load_actual_kw':DATA['load_a'][i,t],'pv_actual_kw':DATA['pv_a'][i,t],
            'actual_estimated':bool(DATA['actual_estimated'][i,t]),
            'g_kwh':op['g'][t],'charge_kwh':op['c'][t],'discharge_kwh':op['d'][t],
            'emergency_kwh':op['e'][t],'unused_paid_kwh':op['v'][t],'curtailed_pv_kwh':op['r'][t],
            'soc_start_kwh':op['s'][t],'soc_end_kwh':op['s'][t+1],
            'cash_cost_yuan':PRICE[t]*op['g'][t]+5*PRICE[t]*op['e'][t]}

def continuous_job(job):
    method,q_selected,outname,n,s0=job
    out=Path(outname);out.mkdir(parents=True,exist_ok=True)
    daily=[];logs=[];boundary=[]
    fh=(out/f'{method}_trace.csv').open('w',newline='',encoding='utf-8-sig');writer=None
    def complete(i,pl,meta,op):
        nonlocal writer
        stage='test' if i≥CONFIG['calibration_days'] else 'warmup_calibration'
        r=record(i,method,'continuous',pl,op,meta);r['stage']=stage
        daily.append(r)
        for row in trace_rows(i,method,op):
            if writer is None:
                writer=csv.DictWriter(fh,fieldnames=list(row));writer.writeheader()
                writer.writerow(row)
            if (i+1)%28==0:
                fh.flush();write_csv(out/f'{method}_daily.csv',daily)
                print(f'continuous {method}: {i+1}/{n}',flush=True)
    try:
        for i in range(n):

```

```

q=q_selected if i≥CONFIG['calibration_days'] else CONFIG['default_quantile']
prefix=None if i==0 else (g_prev[-1],DATA['load_f'][i-1,-1],DATA['pv_f'][i-1,-1])
# No current bridge actual is used before this new contract is fixed.
pl,meta=plan(i,method,s0,q,prefix)
g=pl['g'] if i==0 else pl['g'][1:]
logs.append({'method':method,'date':str(DATA['dates'][i]),'midnight_observed_soc':s0,
'forecast_soc_at_window_start':pl.get('state_at_new_window_start_kwh'),
'new_contract_kwh':float(g.sum()),'new_contract_cost':float(PRICE@g),
'fixed_prefix_kwh':None if prefix is None else prefix[0],
'status':pl['status'],'gap':pl['mip_gap'],'checks':pl['checks'],'risk':meta,

'mip_diagnostics':pl.get('mip_diagnostics'),'polish_diagnostics':pl.get('polish_diagnostics')})
if i>0:
    aug_g=np.r_[g_prev[-1],g]
    aug_l=np.r_[DATA['load_f'][i-1,-1],DATA['load_f'][i]]
    aug_pv=np.r_[DATA['pv_f'][i-1,-1],DATA['pv_f'][i]]
    tail=execute(method,aug_g,DATA['load_a'][i-1,-1:],DATA['pv_a'][i-1,-1:],
        aug_l,aug_pv,np.r_[PRICE[-1],PRICE],s0,1)
    complete(i-1,pl_prev,meta_prev,merge(part_prev,tail))
    s0=float(tail['s'][-1])
    logs[-1]['executed_soc_at_window_start']=s0
else:logs[-1]['executed_soc_at_window_start']=s0
part=execute(method,g,DATA['load_a'][i,:143],DATA['pv_a'][i,:143],
    DATA['load_f'][i],DATA['pv_f'][i],PRICE,s0,143)
s0=float(part['s'][-1]);g_prev=g;part_prev=part;pl_prev=pl;meta_prev=meta
i=n-1
tail=execute(method,g_prev[-1:],DATA['load_a'][i,-1:],DATA['pv_a'][i,-1:],
    DATA['load_f'][i,-1:],DATA['pv_f'][i,-1:],PRICE[-1:],s0,1)
complete(i,pl_prev,meta_prev,merge(part_prev,tail))
if DATA['actual_estimated'][i,-1]:
    for name,candidate in DATA.get('last_boundary_candidates',{}).items():
        op=execute(method,g_prev[-1:],[candidate['load_mean_kw']], [candidate['pv_mean_kw']],
            DATA['load_f'][i,-1:],DATA['pv_f'][i,-1:],PRICE[-1:],s0,1)
        boundary.append({'method':method,'boundary_method':name,**candidate,

'last_interval_cash_yuan':op['fees_total'],'last_interval_emergency_yuan':op['fees_emergency'],
'total_cash_yuan':sum(r['cash_cost_yuan'] for r in daily)-
tail['fees_total']+op['fees_total'],
'final_soc_kwh':op['s'][-1]})
finally:fh.close()
write_csv(out/f'{method}_daily.csv',daily)
json_write(out/f'{method}_solver.json',logs)
if boundary:write_csv(out/f'{method}_boundary.csv',boundary)
return {'method':method,'daily':daily,'boundary':boundary}

def parallel(pool,jobs,worker,name,cache=None):
    out=[];pending=[]
    if cache is not None:Path(cache).mkdir(parents=True,exist_ok=True)
    for j in jobs:
        key=hashlib.sha256(json.dumps(j).encode()).hexdigest()[0:20]
        path=None if cache is None else Path(cache)/(key+'.json')
        if path is not None and path.exists():out.append(json.loads(path.read_text()))
        else:pending.append((j,path))
    if out:print(f'{name}: reused {len(out)} completed checked jobs',flush=True)
    futures={pool.submit(worker,j):path for j,path in pending}
    for k,f in enumerate(as_completed(futures),1):
        value=f.result();out.append(value)
        if futures[f] is not None:json_write(futures[f],value)
        if k%28==0 or k==len(futures):print(f'{name}: {k}/{len(futures)}',flush=True)
    return out

def run(data,p,config,out,workers=3):
    out=Path(out);out.mkdir(parents=True,exist_ok=True);n=len(data['dates']);cut=config['calibration_days']

```



```

    if not 15<cut<n:raise ValueError('Need calibration history and a held-out test period')
    if not 1≤config['min_history']≤config['window'] or config['scenarios']<1:raise ValueError('Invalid
history/scenarios')
    initialize(data,p,config);t0=time.perf_counter();events=[]
    source_digest=hashlib.sha256()
    for name in
('planners.py','window_study.py','window_io.py','window_control.py','control.py','control_refined.py'):
        source_digest.update((ROOT/'code'/name).read_bytes())
    signature=
{'code_sha256':source_digest.hexdigest(),'inputs':data['metadata'],'config':config,'windows':n}
    run_hash=hashlib.sha256(json.dumps(signature,sort_keys=True,default=str).encode()).hexdigest()
    cache=out/'job_cache'/run_hash[:16];json_write(out/'run_signature.json',signature)
    def event(s):

events.append({'order':len(events)+1,'stage':s});json_write(out/'events.json',events);print(s,flush=True
)

    with
ExperimentPool(max_workers=workers,mp_context=mp.get_context('spawn'),initializer=initialize,initargs=
(data,p,config)) as pool:
        event('1 same-contract A/B local comparison')
        first=parallel(pool,[(i,'AB',0) for i in range(cut,n)],local_job,'A/B',cache/'ab')
        same=sorted([r for x in first for r in x['rows']],key=lambda r:(r['date'],r['method']))
        write_csv(out/'stage1_same_contract_daily.csv',same)
        event('calibration only; test data not used for parameter choice')
        eligible=calibration_indices(data,cut,config['min_history'])
        calibration=[]
        for q in [0,.5,.8,.9]:
            for x in parallel(pool,[(i,'C',q) for i in eligible],local_job,f'calibrate q=
{q}',cache/'calibration'):
                calibration.extend([{'quantile':q,**r} for r in x['rows']])
            q=min([0,.5,.8,.9],key=lambda a:(sum(r['inventory_adjusted_cost_yuan'] for r in calibration if
r['quantile']==a),a)) if calibration else config['default_quantile']
            if calibration:write_csv(out/'calibration_quantiles.csv',calibration)
            frozen={'config':config,'selected_quantile':q,'calibration_dates':[str(data['dates'][i]) for i
in eligible],
                    'effective_from':str(data['dates'][cut]),'scenario_count_status':'preset 8, not optimized',
                    'selection_status':'calibration_only' if calibration else
'preset_insufficient_eligible_windows',
                    'test_data_used_for_selection':False,'initial_soc_basis':'user-prescribed experiment state
at first window start'}
            json_write(out/'frozen_parameters.json',frozen)
            event('2 B/C/D same-MPC local planning comparison')
            second=parallel(pool,[(i,m,q) for i in range(cut,n) for m in
('C','D')],local_job,'C/D',cache/'cd')
            local=[dict(r,stage='local_dayahead') for r in same if r['method']=='B']+ [r for x in second for
r in x['rows']]
            local.sort(key=lambda r:
(r['date'],r['method']));write_csv(out/'stage2_dayahead_daily.csv',local)
            json_write(out/'local_solver_audit.json',first+second)
            event('3 continuous midnight decisions, bridged to 00:10 window starts')
            continuous=parallel(pool,[(m,q,str(out/'continuous'),n,config['initial_soc']) for m in
('A','B','C','D')],continuous_job,'systems')
            daily=sorted([r for x in continuous for r in x['daily']],key=lambda r:(r['date'],r['method']))
            write_csv(out/'stage3_continuous_daily.csv',daily)
            with (out/'stage3_interval_trace.csv').open('w',encoding='utf-8-sig',newline='') as f:
                writer=None
                for m in ('A','B','C','D'):
                    with (out/f'continuous/{m}_trace.csv').open(encoding='utf-8-sig',newline='') as source:
                        for r in csv.DictReader(source):
                            if writer is
None:writer=csv.DictWriter(f,fieldnames=list(r));writer.writeheader()
                            writer.writerow(r)
                boundary=[r for x in continuous for r in x['boundary']]
                if boundary:write_csv(out/'final_interval_sensitivity.csv',boundary)
            event('4 initial SOC sensitivity, independent equal-length 28-window experiments')
            sensitivity=[]

```

```

# The configured reference state (6000 in this experiment) must also
# end after 28 windows. A slice of the 334-window run would let its last
# interval use the 29th contract, unlike the two bounded-state runs.
for s0 in (1200.,float(config['initial_soc']),10800.):
    results=parallel(pool,[(m,q,str(out/f'initial_soc_{int(s0)}'),min(28,n),s0) for m in
('A','B','C','D')],continuous_job,f'S0={s0}')
    for x in results:
        sensitivity.extend([dict(r,experiment_initial_soc=s0) for r in x['daily']])
    write_csv(out/'initial_soc_sensitivity_daily.csv',sensitivity)

summaries=summarize(same,'same_contract_local')+summarize(local,'dayahead_local')+summarize(daily,'conti
nuous_all')+summarize([r for r in daily if r['stage']=='test'],'continuous_test')
    write_csv(out/'comparison_summary.csv',summaries)
    summary=
{'data_role':'supplied_forecasts_with_endpoint_averaged_actuals','real_competition_result':False,
 'interpretation':'Conditional on user-prescribed first SOC and forecast provenance; final
interval actual uses an explicitly estimated endpoint.',
 'dates':[str(data['window_start'][0]),str(data['window_end'][-1])],
 'test_dates':[str(data['dates'][cut]),str(data['dates']
[-1])], 'elapsed_seconds':time.perf_counter()-t0,
 'selected_quantile':q,'all_physical_checks_passed':all(r['all_constraints_passed'] for r in
summaries),

'periods':summaries,'input_metadata':data['metadata'],'estimated_actual_intervals':int(data['actual_esti
mated'].sum()),
 'midnight_bridge':'plan with observed 00:00 state before using 00:00-00:10 actual; fixed prefix
paid only once',
 'initial_soc_sensitivity_scope':'independent first 28 windows for each initial state, identical
one-step final interval; no full-period claim'}
    json_write(out/'summary.json',summary);event('complete')
    return summary

def main():
    a=argparse.ArgumentParser(description=__doc__)
    a.add_argument('--out',type=Path,default=ROOT/'results/team')
    a.add_argument('--forecasts',type=Path,default=ROOT/'data/team_forecasts.xlsx')
    a.add_argument('--actuals',type=Path,default=ROOT/'data/attachment2_original.xlsx')
    a.add_argument('--workers',type=int,default=3);a.add_argument('--calibration-
days',type=int,default=24)
    a.add_argument('--scenarios',type=int,default=8);a.add_argument('--milp-
seconds',type=float,default=30)
    a.add_argument('--s0',type=float,default=6000);a.add_argument('--limit-days',type=int)
    args=a.parse_args()
    data=attach_actuals(read_team_forecasts(args.forecasts),args.actuals,confirmed_instantaneous=True)
    p,pm=prices(ROOT/'data/attachment1.xlsx');data['metadata']['price']=pm
    data['metadata']['initial_soc_experiment_user_confirmed']=args.s0
    if args.limit_days:
        for key in
('dates','window_start','window_end','issued_at','load_f','pv_f','load_a','pv_a','actual_estimated'):dat
a[key]=data[key][:args.limit_days]
    args.out.mkdir(exist_ok=True,parents=True);write_canonical(data,args.out)
    config={'initial_soc':args.s0,'calibration_days':args.calibration_days,'window':28,'min_history':14,

'scenarios':args.scenarios,'seed':20260911,'terminal_weight':float(p.min())/DEFAULT_BATTERY.eta_c),
 'inventory_value':float(p.min())/DEFAULT_BATTERY.eta_c),'s_ref':6000,'default_quantile':.8,
 'milp_seconds':args.milp_seconds,'mip_gap':1e-5}
    run(data,p,config,args.out,args.workers)

if __name__=='__main__':main()

```


29. question2/prediction/baseline_forecast.py

源文件 SHA-256: 4b2332973f5b57e1087a95e14feb7b4321d6a28b77a091f7b61f8f32b7f52116 ; 代码行数: 184。

```

"""问题二基准预测：负载取上周同期，光伏取此前 7 天同期均值。"""

from __future__ import annotations

import argparse
from datetime import datetime
from pathlib import Path

import numpy as np
from openpyxl import Workbook, load_workbook
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter

LOAD_SHEET = "小区负载"
PV_SHEET = "光伏发电实际功率"
FORECAST_START = datetime(2025, 2, 1)

def read_sheet(path: Path, sheet_name: str) → tuple[list[datetime], list[object], np.ndarray]:
    """读取一个 365×144 的功率矩阵，并严格核对表结构。"""
    workbook = load_workbook(path, read_only=True, data_only=True)
    worksheet = workbook[sheet_name]
    rows = list(worksheet.iter_rows(values_only=True))
    workbook.close()

    if len(rows) ≠ 366 or len(rows[0]) ≠ 145:
        raise ValueError(
            f"{sheet_name} 结构异常：期望 366 行×145 列，实际 {len(rows)} 行×{len(rows[0])} 列"
        )

    times = list(rows[0][1:])
    dates = [row[0] for row in rows[1:]]
    values = np.asarray([row[1:] for row in rows[1:]], dtype=float)

    if len(set(dates)) ≠ len(dates):
        raise ValueError(f"{sheet_name} 存在重复日期")
    if np.isnan(values).any():
        raise ValueError(f"{sheet_name} 存在缺失或非数值功率")
    if (values < 0).any():
        raise ValueError(f"{sheet_name} 存在负功率")
    return dates, times, values

def make_baseline(
    dates: list[datetime], load: np.ndarray, pv: np.ndarray
) → tuple[list[datetime], np.ndarray, np.ndarray]:
    """仅用预测日 0:00 前的数据生成逐日预测。"""
    first = next(i for i, date in enumerate(dates) if date ≥ FORECAST_START)
    forecast_dates = dates[first:]

    # 负载：严格采用前 7 天同一时段。
    load_prediction = load[first - 7 : len(dates) - 7].copy()

    # 光伏：对预测日前 7 天的同一时段取算术平均。
    pv_prediction = np.vstack([pv[i - 7 : i].mean(axis=0) for i in range(first, len(dates))])
    pv_prediction = np.clip(pv_prediction, 0.0, None)

    expected_shape = (len(forecast_dates), 144)
    if load_prediction.shape ≠ expected_shape or pv_prediction.shape ≠ expected_shape:
        raise AssertionError("基准预测矩阵维度不正确")
    return forecast_dates, load_prediction, pv_prediction

```

```

def style_header(worksheet) → None:
    fill = PatternFill("solid", fgColor="D9EAF7")
    for cell in worksheet[1]:
        cell.font = Font(bold=True)
        cell.fill = fill
        cell.alignment = Alignment(horizontal="center", vertical="center")
    worksheet.freeze_panes = "B2"
    worksheet.auto_filter.ref = worksheet.dimensions

def write_wide_sheet(workbook: Workbook, name: str, dates, times, values) → None:
    worksheet = workbook.create_sheet(name)
    worksheet.append(["日期\\时间", *times])
    for date, row in zip(dates, values, strict=True):
        worksheet.append([date, *[float(value) for value in row]])
    style_header(worksheet)
    worksheet.column_dimensions["A"].width = 13
    for column in range(2, 146):
        worksheet.column_dimensions[get_column_letter(column)].width = 10
    for cell in worksheet["A"][1:]:
        cell.number_format = "yyyy-mm-dd"
    for row in worksheet.iter_rows(min_row=2, min_col=2):
        for cell in row:
            cell.number_format = "0.0000"

def write_output(path: Path, dates, times, load_prediction, pv_prediction) → None:
    workbook = Workbook()
    workbook.remove(workbook.active)
    write_wide_sheet(workbook, "小区负载预测", dates, times, load_prediction)
    write_wide_sheet(workbook, "光伏发电预测", dates, times, pv_prediction)

    worksheet = workbook.create_sheet("预测长表")
    worksheet.append(["日期", "时段序号", "时刻", "负载预测功率", "光伏预测功率"])
    for day_index, date in enumerate(dates):
        for period_index, time_label in enumerate(times):
            worksheet.append(
                [
                    date,
                    period_index + 1,
                    time_label,
                    float(load_prediction[day_index, period_index]),
                    float(pv_prediction[day_index, period_index]),
                ]
            )
    style_header(worksheet)
    worksheet.column_dimensions["A"].width = 13
    worksheet.column_dimensions["B"].width = 10
    worksheet.column_dimensions["C"].width = 12
    worksheet.column_dimensions["D"].width = 18
    worksheet.column_dimensions["E"].width = 18
    for cell in worksheet["A"][1:]:
        cell.number_format = "yyyy-mm-dd"
    for cell in worksheet["C"][1:]:
        cell.number_format = "hh:mm"
    for column in ("D", "E"):
        for cell in worksheet[column][1:]:
            cell.number_format = "0.0000"

    path.parent.mkdir(parents=True, exist_ok=True)
    workbook.save(path)

def validate_output(path: Path, expected_dates: int) → None:
    workbook = load_workbook(path, read_only=True, data_only=True)
    expected = {

```

```

        "小区负载预测": (expected_dates + 1, 145),
        "光伏发电预测": (expected_dates + 1, 145),
        "预测长表": (expected_dates * 144 + 1, 5),
    }
    for sheet_name, shape in expected.items():
        worksheet = workbook[sheet_name]
        actual = (worksheet.max_row, worksheet.max_column)
        if actual != shape:
            raise AssertionError(f"{sheet_name} 维度错误: 期望 {shape}, 实际 {actual}")
    first_date = workbook["预测长表"]["A2"].value
    last_date = workbook["预测长表"][f"A{expected_dates * 144 + 1}"].value
    if first_date != datetime(2025, 2, 1) or last_date != datetime(2025, 12, 31):
        raise AssertionError(f"日期范围错误: {first_date} 至 {last_date}")
    workbook.close()

def parse_args() → argparse.Namespace:
    script_dir = Path(__file__).resolve().parent
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--input",
        type=Path,
        default=script_dir.parent / "附件2.xlsx",
        help="附件2.xlsx 路径",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=script_dir / "baseline_predictions.xlsx",
        help="输出 Excel 路径",
    )
    return parser.parse_args()

def main() → None:
    args = parse_args()
    load_dates, times, load = read_sheet(args.input, LOAD_SHEET)
    pv_dates, pv_times, pv = read_sheet(args.input, PV_SHEET)
    if load_dates != pv_dates or times != pv_times:
        raise ValueError("负载与光伏工作表的日期或时段不一致")

    dates, load_prediction, pv_prediction = make_baseline(load_dates, load, pv)
    write_output(args.output, dates, times, load_prediction, pv_prediction)
    validate_output(args.output, len(dates))

    print(f"已生成: {args.output}")
    print(f"预测日期: {dates[0]:%Y-%m-%d} 至 {dates[-1]:%Y-%m-%d}, 共 {len(dates)} 天")
    print(f"每日电力时段: {len(times)}; 长表记录: {len(dates) * len(times)}")
    print(f"负载预测范围: {load_prediction.min():.4f} 至 {load_prediction.max():.4f}")
    print(f"光伏预测范围: {pv_prediction.min():.4f} 至 {pv_prediction.max():.4f}")

if __name__ == "__main__":
    main()

```

30. question2/prediction/cross_day_load_boundary.py

源文件 SHA-256: 46885e9a4328a06a33dda3b29febfa5e85930a54c7513c4e6cd5000aeb644ca4 ; 代码行数: 274。

```

"""评价次日 00:10 负载的两种补点方法及最后一个区间平均负载。"""

from __future__ import annotations

import argparse
import json
import math
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
from xgboost import XGBRegressor

from forecast_models import LOAD_SHEET, read_power_sheet

BASE_MODELS = ("Baseline", "XGBoost", "LightGBM", "SARIMA")
BOUNDARY_METHODS = ("XGBoost跨日边界", "历史跨日比例", "目标时刻上周同期")
XGB_FEATURE_NAMES = (
    "前1日00:10", "前2日00:10", "前7日00:10", "前14日00:10",
    "近7日00:10均值", "近7日00:10标准差", "近14日00:10均值", "近14日00:10标准差",
    "前日23:20", "前日23:30", "前日23:40", "前日23:50", "前日24:00",
    "前日全天均值", "前日全天标准差", "近7日全天均值", "近7日全天标准差",
    "目标日星期正弦", "目标日星期余弦", "目标日年周期正弦", "目标日年周期余弦",
)

def error_metrics(actual: np.ndarray, predicted: np.ndarray) → dict[str, float]:
    error = predicted - actual
    return {
        "样本数": int(len(actual)),
        "MAE": float(np.abs(error).mean()),
        "RMSE": float(math.sqrt(np.square(error).mean())),
        "WAPE_percent": float(np.abs(error).sum() / np.abs(actual).sum() * 100),
        "Bias": float(error.mean()),
        "MAPE_percent": float(np.mean(np.abs(error) / np.maximum(np.abs(actual), 1e-8)) * 100),
        "R2": float(1 - np.square(error).sum() / np.square(actual - actual.mean()).sum()),
    }

def rolling_ratio(values: np.ndarray, planning_day: int, window: int) → tuple[float, int]:
    """只用规划日前已完整观测的跨日比例。"""
    # r_j = L_{j+1,00:10} / L_{j,24:00}; 在 planning_day 的 00:00, 最大可用 j 为 day-2。
    stop = planning_day - 1
    start = max(0, stop - window)
    denominators = values[start:stop, -1]
    numerators = values[start + 1 : stop + 1, 0]
    valid = denominators > 1e-8
    ratios = numerators[valid] / denominators[valid]
    if len(ratios) == 0:
        raise ValueError(f"规划日索引 {planning_day} 没有可用跨日比例")
    return float(np.median(ratios)), int(len(ratios))

def xgb_boundary_features(values: np.ndarray, dates: list[datetime], planning_day: int) → np.ndarray:
    """构造规划日 00:00 已知的特征, 目标为次日 00:10。"""
    first_slot_7 = values[planning_day - 7 : planning_day, 0]
    first_slot_14 = values[planning_day - 14 : planning_day, 0]
    prior_days_7 = values[planning_day - 7 : planning_day]
    target_date = dates[planning_day + 1]
    dow = target_date.weekday()
    doy = target_date.timetuple().tm_yday
    return np.asarray(

```

```

[
    values[planning_day - 1, 0], values[planning_day - 2, 0],
    values[planning_day - 7, 0], values[planning_day - 14, 0],
    first_slot_7.mean(), first_slot_7.std(), first_slot_14.mean(), first_slot_14.std(),
    *values[planning_day - 1, -5:],
    values[planning_day - 1].mean(), values[planning_day - 1].std(),
    prior_days_7.mean(), prior_days_7.std(),
    math.sin(2 * math.pi * dow / 7), math.cos(2 * math.pi * dow / 7),
    math.sin(2 * math.pi * doy / 365), math.cos(2 * math.pi * doy / 365),
],
dtype=float,
)

def train_xgb_boundary(
    values: np.ndarray,
    dates: list[datetime],
    planning_indices: np.ndarray,
    n_estimators: int,
) → tuple[dict[int, float], pd.DataFrame]:
    """逐日扩展窗口训练：最后一个训练标签必须在规划日前已经观测。"""
    predictions: dict[int, float] = {}
    importance_rows = []
    for planning_day in planning_indices:
        # 样本 j 的标签是 j+1 日 00:10。规划日 i 的最后可用样本为 j=i-2。
        train_days = np.arange(14, planning_day - 1)
        x_train = np.vstack([xgb_boundary_features(values, dates, j) for j in train_days])
        y_train = values[train_days + 1, 0]
        model = XGBRegressor(
            objective="reg:squarederror",
            n_estimators=n_estimators,
            max_depth=2,
            learning_rate=0.05,
            min_child_weight=3,
            subsample=0.9,
            colsample_bytree=0.9,
            reg_lambda=8.0,
            random_state=2026,
            n_jobs=-1,
            tree_method="hist",
        )
        model.fit(x_train, y_train)
        prediction = float(model.predict(xgb_boundary_features(values, dates, planning_day)[None, :])
    [0])
    predictions[planning_day] = max(0.0, prediction)
    importance_rows.append(model.feature_importances_)
    importance = pd.DataFrame(
        {
            "特征": XGB_FEATURE_NAMES,
            "全年日模型平均重要性": np.vstack(importance_rows).mean(axis=0),
        }
    ).sort_values("全年日模型平均重要性", ascending=False)
    return predictions, importance.reset_index(drop=True)

def build_predictions(
    values: np.ndarray,
    dates: list[datetime],
    old_results: Path,
    ratio_window: int,
    n_estimators: int,
) → tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    # 2025-12-31 的次日 00:10 不在附件中，故最后可评价的规划日为 12 月 30 日。
    planning_indices = np.arange(31, len(dates) - 1)
    ratio_rows = []
    ratio_by_day = {}

```

```

for day_index in planning_indices:
    ratio, sample_count = rolling_ratio(values, day_index, ratio_window)
    ratio_by_day[day_index] = ratio
    ratio_rows.append(
        {
            "规划日期": dates[day_index],
            "比例窗口": ratio_window,
            "有效历史比例数": sample_count,
            "跨日比例中位数": ratio,
        }
    )

xgb_by_day, xgb_importance = train_xgb_boundary(values, dates, planning_indices, n_estimators)

rows = []
for base_model in BASE_MODELS:
    full_prediction = np.load(old_results / f"{base_model}_负载_predictions.npy")
    if full_prediction.shape != (334, 144):
        raise ValueError(f"{base_model} 负载预测矩阵维度异常: {full_prediction.shape}")
    for day_index in planning_indices:
        prediction_row = day_index - 31
        predicted_2400 = float(full_prediction[prediction_row, -1])
        actual_2400 = float(values[day_index, -1])
        actual_next_0010 = float(values[day_index + 1, 0])
        actual_interval = (actual_2400 + actual_next_0010) / 2

        boundary_predictions = {
            "XGBoost跨日边界": xgb_by_day[day_index],
            "历史跨日比例": predicted_2400 * ratio_by_day[day_index],
            # 目标为 d+1 日 00:10, 其 7 天前为 d-6 日 00:10。
            "目标时刻上周同期": float(values[day_index - 6, 0]),
        }
        for boundary_method, predicted_next_0010 in boundary_predictions.items():
            rows.append(
                {
                    "规划日期": dates[day_index],
                    "目标日期": dates[day_index + 1],
                    "24:00基础模型": base_model,
                    "边界预测方法": boundary_method,
                    "预测_当日24:00": predicted_2400,
                    "实际_当日24:00": actual_2400,
                    "预测_次日00:10": predicted_next_0010,
                    "实际_次日00:10": actual_next_0010,
                    "预测_最后区间平均负载": (predicted_2400 + predicted_next_0010) / 2,
                    "实际_最后区间平均负载": actual_interval,
                }
            )
    )
return pd.DataFrame(rows), pd.DataFrame(ratio_rows), xgb_importance

def evaluate(predictions: pd.DataFrame) → tuple[pd.DataFrame, pd.DataFrame]:
    overall_rows = []
    monthly_rows = []
    for (base_model, method), group in predictions.groupby(["24:00基础模型", "边界预测方法"], sort=False):
        endpoint_metrics = error_metrics(
            group["实际_次日00:10"].to_numpy(), group["预测_次日00:10"].to_numpy()
        )
        interval_metrics = error_metrics(
            group["实际_最后区间平均负载"].to_numpy(),
            group["预测_最后区间平均负载"].to_numpy(),
        )
        overall_rows.extend(
            [
                {"24:00基础模型": base_model, "边界预测方法": method, "评价对象": "次日00:10端点",
                **endpoint_metrics},
                {"24:00基础模型": base_model, "边界预测方法": method, "评价对象": "最后10分钟区间平均",

```

```

**interval_metrics},
    ]
)
for month, monthly_group in group.groupby(group["规划日期"].dt.month):
    for label, actual_column, prediction_column in (
        ("次日00:10端点", "实际_次日00:10", "预测_次日00:10"),
        ("最后10分钟区间平均", "实际_最后区间平均负载", "预测_最后区间平均负载"),
    ):
        monthly_rows.append(
            {
                "24:00基础模型": base_model,
                "边界预测方法": method,
                "评价对象": label,
                "月份": int(month),
                **error_metrics(
                    monthly_group[actual_column].to_numpy(),
                    monthly_group[prediction_column].to_numpy(),
                ),
            }
        )
overall = pd.DataFrame(overall_rows).sort_values(
    ["评价对象", "WAPE_percent", "24:00基础模型", "边界预测方法"]
)
return overall.reset_index(drop=True), pd.DataFrame(monthly_rows)

def save_excel(path: Path, predictions, ratios, importance, overall, monthly) → None:
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        predictions.to_excel(writer, sheet_name="逐日预测", index=False)
        overall.to_excel(writer, sheet_name="综合指标", index=False)
        monthly.to_excel(writer, sheet_name="逐月指标", index=False)
        ratios.to_excel(writer, sheet_name="滚动跨日比例", index=False)
        importance.to_excel(writer, sheet_name="XGBoost特征重要性", index=False)

def main() → None:
    script_dir = Path(__file__).resolve().parent
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=script_dir.parent / "附件2.xlsx")
    parser.add_argument("--old-results", type=Path, default=script_dir / "results")
    parser.add_argument("--output-dir", type=Path, default=script_dir / "cross_day_load_results")
    parser.add_argument("--ratio-window", type=int, default=28)
    parser.add_argument("--xgb-n-estimators", type=int, default=100)
    args = parser.parse_args()
    if args.ratio_window ≤ 0 or args.xgb_n_estimators ≤ 0:
        raise ValueError("比例窗口和 XGBoost 树数量必须为正整数")

    dates, _, values = read_power_sheet(args.input, LOAD_SHEET)
    predictions, ratios, importance = build_predictions(
        values, dates, args.old_results, args.ratio_window, args.xgb_n_estimators
    )
    overall, monthly = evaluate(predictions)
    args.output_dir.mkdir(parents=True, exist_ok=True)
    predictions.to_csv(args.output_dir / "跨日负载逐日预测.csv", index=False, encoding="utf-8-sig")
    overall.to_csv(args.output_dir / "跨日负载综合指标.csv", index=False, encoding="utf-8-sig")
    monthly.to_csv(args.output_dir / "跨日负载逐月指标.csv", index=False, encoding="utf-8-sig")
    ratios.to_csv(args.output_dir / "滚动跨日比例.csv", index=False, encoding="utf-8-sig")
    importance.to_csv(args.output_dir / "XGBoost特征重要性.csv", index=False, encoding="utf-8-sig")
    save_excel(args.output_dir / "跨日负载预测与评价.xlsx", predictions, ratios, importance, overall,
monthly)
    (args.output_dir / "实验配置.json").write_text(
        json.dumps(
            {
                "planning_period": ["2025-02-01", "2025-12-30"],
                "sample_days": 333,
                "ratio_window": args.ratio_window,
            }
        )
    )

```



```
        "ratio_statistic": "median",
        "xgb_n_estimators": args.xgb_n_estimators,
        "information_boundary": "规划日00:00时仅使用前一日及更早的完整观测",
        "base_models": list(BASE_MODELS),
        "boundary_methods": list(BOUNDARY_METHODS),
    },
    ensure_ascii=False,
    indent=2,
),
encoding="utf-8",
)
print(overall.to_string(index=False))

if __name__ == "__main__":
    main()
```

31. question2/prediction/export_planning_forecast.py

源文件 SHA-256: 41fb14ec98530e5b94d500a3d1ed163c570472f4656c561881b91d70c2be9a1a ; 代码行数: 277。

"""导出最终选定预测模型的数据，供问题二规划模型直接读取。

最终模型：负载使用 **XGBoost**，光伏使用自适应加权 **Baseline**。
所有功率均为 **10** 分钟区间平均功率，而非瞬时端点值。

"""

```
from __future__ import annotations

import argparse
import hashlib
import json
from datetime import datetime, timedelta
from pathlib import Path

import numpy as np
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Alignment, Font, PatternFill

from interval_preprocessing import attachment1_tables, period_mapping

FORECAST_START = datetime(2025, 2, 1)
FORECAST_END = datetime(2025, 12, 31)
EXPECTED_DAYS = 334
PERIODS_PER_DAY = 144
LOAD_MODEL = "XGBoost"
PV_MODEL = "自适应加权Baseline"

def sha256(path: Path) → str:
    digest = hashlib.sha256()
    with path.open("rb") as file:
        for block in iter(lambda: file.read(1024 * 1024), b''):
            digest.update(block)
    return digest.hexdigest()

def forecast_dates() → list[datetime]:
    dates = []
    current = FORECAST_START
    while current ≤ FORECAST_END:
        dates.append(current)
        current += timedelta(days=1)
    if len(dates) ≠ EXPECTED_DAYS:
        raise AssertionError(f"预测日期数量异常: {len(dates)}")
    return dates

def read_prediction(path: Path, label: str) → np.ndarray:
    values = np.load(path)
    expected = (EXPECTED_DAYS, PERIODS_PER_DAY)
    if values.shape ≠ expected:
        raise ValueError(f"{label}维度异常: 期望{expected}, 实际{values.shape}")
    if not np.isfinite(values).all():
        raise ValueError(f"{label}存在NaN或无穷值")
    if np.any(values < 0):
        raise ValueError(f"{label}存在负功率")
    return values.astype(float, copy=False)

def read_interval_price(attachment1: Path) → np.ndarray:
    _, interval_table = attachment1_tables(attachment1)
    # attachment1_tables 的第7列为区间平均电价。
```

```

prices = interval_table.iloc[:, 6].to_numpy(dtype=float)
if prices.shape != (PERIODS_PER_DAY,) or not np.isfinite(prices).all():
    raise ValueError("附件1区间平均电价异常")
if np.any(prices < 0):
    raise ValueError("附件1区间平均电价存在负值")
return prices

def build_long_table(
    dates: list[datetime],
    mapping: pd.DataFrame,
    prices: np.ndarray,
    load_prediction: np.ndarray,
    pv_prediction: np.ndarray,
) → pd.DataFrame:
    net_load = load_prediction - pv_prediction
    rows = EXPECTED_DAYS * PERIODS_PER_DAY
    table = pd.DataFrame(
        {
            "日期": np.repeat(dates, PERIODS_PER_DAY),
            "时段序号": np.tile(np.arange(1, PERIODS_PER_DAY + 1), EXPECTED_DAYS),
            "开始时刻": np.tile(mapping["开始时刻"].to_numpy(), EXPECTED_DAYS),
            "结束时刻": np.tile(mapping["结束时刻"].to_numpy(), EXPECTED_DAYS),
            "时间区间": np.tile(mapping["时间区间"].to_numpy(), EXPECTED_DAYS),
            "时长_h": np.full(rows, 1 / 6),
            "区间平均电价": np.tile(prices, EXPECTED_DAYS),
            "预测负载功率": load_prediction.ravel(),
            "预测光伏功率": pv_prediction.ravel(),
            "预测净负荷功率": net_load.ravel(),
        }
    )
    return table

def wide_table(dates: list[datetime], labels: list[str], values: np.ndarray) → pd.DataFrame:
    frame = pd.DataFrame(values, columns=labels)
    frame.insert(0, "日期\\区间", dates)
    return frame

def style_workbook(path: Path) → None:
    workbook = load_workbook(path)
    fill = PatternFill("solid", fgColor="D9EAF7")
    for worksheet in workbook.worksheets:
        worksheet.freeze_panes = "A2"
        worksheet.auto_filter.ref = worksheet.dimensions
        for cell in worksheet[1]:
            cell.font = Font(bold=True)
            cell.fill = fill
            cell.alignment = Alignment(horizontal="center", vertical="center")
        for column in worksheet.columns:
            sample = list(column)[:200]
            width = min(24, max(10, max(len(str(cell.value or "")) for cell in sample) + 2))
            worksheet.column_dimensions[column[0].column_letter].width = width
        for row in worksheet.iter_rows(min_row=2):
            for cell in row:
                if isinstance(cell.value, float):
                    cell.number_format = "0.0000"
    workbook.save(path)

def export(
    load_path: Path,
    pv_path: Path,
    attachment1: Path,
    output_dir: Path,

```

```

) → None:
    dates = forecast_dates()
    mapping = period_mapping()
    labels = mapping["时间区间"].tolist()
    load_prediction_data = read_prediction(load_path, "XGBoost负载预测")
    pv_prediction = read_prediction(pv_path, "自适应加权光伏预测")
    prices = read_interval_price(attachment1)
    net_load = load_prediction_data - pv_prediction
    long_table = build_long_table(
        dates, mapping, prices, load_prediction_data, pv_prediction
    )

    output_dir.mkdir(parents=True, exist_ok=True)
    xlsx_path = output_dir / "规划模型预测输入.xlsx"
    csv_path = output_dir / "规划模型预测输入.csv"
    npz_path = output_dir / "规划模型预测输入.npz"

    notes = pd.DataFrame(
        [
            ["预测日期", "2025-02-01至2025-12-31, 共334天"],
            ["时段数量", "每天144个10分钟区间"],
            ["负载模型", LOAD_MODEL],
            ["光伏模型", PV_MODEL],
            ["功率口径", "相邻瞬时端点平均得到的10分钟区间平均功率"],
            ["00:00边界", "规划时刻真实值已在端点转区间过程中使用"],
            ["净负荷定义", "预测负载功率-预测光伏功率; 允许为负, 表示预测光伏盈余"],
            ["电价口径", "附件1相邻端点平均; 按题意每天采用同一日内价格序列"],
            ["规划主输入", "工作表“规划模型输入”或同名CSV"],
        ],
        columns=["项目", "说明"],
    )

    price_table = mapping[["时段序号", "开始时刻", "结束时刻", "时间区间", "时长/h"]].copy()
    price_table["区间平均电价"] = prices

    with pd.ExcelWriter(xlsx_path, engine="openpyxl") as writer:
        notes.to_excel(writer, sheet_name="使用说明", index=False)
        long_table.to_excel(writer, sheet_name="规划模型输入", index=False)
        wide_table(dates, labels, load_prediction_data).to_excel(
            writer, sheet_name="负载预测_宽表", index=False
        )
        wide_table(dates, labels, pv_prediction).to_excel(
            writer, sheet_name="光伏预测_宽表", index=False
        )
        wide_table(dates, labels, net_load).to_excel(
            writer, sheet_name="净负荷预测_宽表", index=False
        )
        price_table.to_excel(writer, sheet_name="日内区间电价", index=False)
    style_workbook(xlsx_path)

    long_table.to_csv(csv_path, index=False, encoding="utf-8-sig", float_format="%.6f")
    np.savez_compressed(
        npz_path,
        dates=np.asarray([date.strftime("%Y-%m-%d") for date in dates]),
        period=np.arange(1, PERIODS_PER_DAY + 1),
        interval_labels=np.asarray(labels),
        price=prices,
        load_prediction=load_prediction_data,
        pv_prediction=pv_prediction,
        net_load_prediction=net_load,
        duration_hours=np.asarray(1 / 6),
    )

    manifest = {
        "load_model": LOAD_MODEL,
        "pv_model": PV_MODEL,
        "prediction_period": ["2025-02-01", "2025-12-31"],
    }

```

```

        "days": EXPECTED_DAYS,
        "periods_per_day": PERIODS_PER_DAY,
        "records": len(long_table),
        "load_input": {"path": str(load_path.resolve()), "sha256": sha256(load_path)},
        "pv_input": {"path": str(pv_path.resolve()), "sha256": sha256(pv_path)},
        "price_input": {"path": str(attachment1.resolve()), "sha256": sha256(attachment1)},
        "outputs": [xlsx_path.name, csv_path.name, npz_path.name],
    }
    (output_dir / "导出清单.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    validate_outputs(xlsx_path, csv_path, npz_path, load_prediction_data, pv_prediction)

def validate_outputs(
    xlsx_path: Path,
    csv_path: Path,
    npz_path: Path,
    expected_load: np.ndarray,
    expected_pv: np.ndarray,
) → None:
    workbook = load_workbook(xlsx_path, read_only=True, data_only=True)
    expected_sheets = [
        "使用说明",
        "规划模型输入",
        "负载预测_宽表",
        "光伏预测_宽表",
        "净负荷预测_宽表",
        "日内区间电价",
    ]
    if workbook.sheetnames != expected_sheets:
        raise AssertionError(f"工作表结构异常: {workbook.sheetnames}")
    if workbook["规划模型输入"].max_row != EXPECTED_DAYS * PERIODS_PER_DAY + 1:
        raise AssertionError("规划模型输入行数异常")
    if workbook["规划模型输入"].max_column != 10:
        raise AssertionError("规划模型输入列数异常")
    workbook.close()

    csv_table = pd.read_csv(csv_path, header=0)
    if len(csv_table) != EXPECTED_DAYS * PERIODS_PER_DAY:
        raise AssertionError("CSV记录数异常")
    if not np.allclose(csv_table["预测负载功率"].to_numpy(), expected_load.ravel(), atol=5e-7):
        raise AssertionError("CSV负载预测与源数组不一致")
    if not np.allclose(csv_table["预测光伏功率"].to_numpy(), expected_pv.ravel(), atol=5e-7):
        raise AssertionError("CSV光伏预测与源数组不一致")

    data = np.load(npz_path)
    if not np.array_equal(data["load_prediction"], expected_load):
        raise AssertionError("NPZ负载预测与源数组不一致")
    if not np.array_equal(data["pv_prediction"], expected_pv):
        raise AssertionError("NPZ光伏预测与源数组不一致")

def parse_args() → argparse.Namespace:
    script_dir = Path(__file__).resolve().parent
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--load",
        type=Path,
        default=script_dir / "interval_results" / "XGBoost_负载_区间预测.npy",
    )
    parser.add_argument(
        "--pv",
        type=Path,
        default=script_dir / "interval_results" / "自适应加权Baseline_光伏_区间预测.npy",
    )

```

```
parser.add_argument("--attachment1", type=Path, default=script_dir.parent / "附件1.xlsx")
parser.add_argument("--output-dir", type=Path, default=script_dir / "planning_input")
return parser.parse_args()

def main() → None:
    args = parse_args()
    export(args.load, args.pv, args.attachment1, args.output_dir)
    print(f"已生成: {args.output_dir / '规划模型预测输入.xlsx'}")
    print(f"规划记录数: {EXPECTED_DAYS * PERIODS_PER_DAY}")
    print(f"负载模型: {LOAD_MODEL}; 光伏模型: {PV_MODEL}")

if __name__ == "__main__":
    main()
```

32.

question2/prediction/export_standardized_interval_forecast.py

源文件 SHA-256: 86cca0a279d567f08e8efeaba682809d3c1a319868d8544d72d71c1655fd7e70 ; 代码行数: 187。

```

"""输出问题二标准化区间预测: XGBoost负载与自适应加权光伏。"""

from __future__ import annotations

import argparse
from copy import copy
from datetime import datetime
from pathlib import Path

import numpy as np
from openpyxl import Workbook, load_workbook
from openpyxl.utils import get_column_letter

from forecast_models import PV_SHEET, read_power_sheet

START_DATE = datetime(2025, 2, 1)
EXPECTED_DAYS = 334
EXPECTED_ENDPOINTS = 144

def format_endpoint(total_minutes: int) → str:
    if total_minutes < 1440:
        hour, minute = divmod(total_minutes, 60)
        return f"{hour:02d}:{minute:02d}"
    hour, minute = divmod(total_minutes - 1440, 60)
    return f"次日{hour:02d}:{minute:02d}"

def interval_headers() → list[str]:
    """00:10-00:20 至 次日00:00-次日00:10, 共144个区间。"""
    return [
        f"{format_endpoint(10 + 10 * i)}-{format_endpoint(20 + 10 * i)}"
        for i in range(144)
    ]

def load_dates(reference_path: Path) → list[datetime]:
    workbook = load_workbook(reference_path, read_only=True, data_only=True)
    worksheet = workbook[workbook.sheetnames[0]]
    dates = [worksheet.cell(row, 1).value for row in range(2, worksheet.max_row + 1)]
    workbook.close()
    if len(dates) ≠ EXPECTED_DAYS or dates[0] ≠ START_DATE or dates[-1] ≠ datetime(2025, 12, 31):
        raise ValueError("参考文件日期范围不是 2025-02-01 至 2025-12-31 的334天")
    return dates

def load_predictions(
    question2_dir: Path, attachment2: Path, dates: list[datetime]
) → tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    load_endpoints = np.load(question2_dir / "results" / "XGBoost_负载_predictions.npy")
    pv_endpoints = np.load(
        question2_dir / "improved_pv_results" / "自适应加权Baseline_predictions.npy"
    )
    if load_endpoints.shape ≠ (EXPECTED_DAYS, EXPECTED_ENDPOINTS):
        raise ValueError(f"XGBoost负载预测维度异常: {load_endpoints.shape}")
    if pv_endpoints.shape ≠ (EXPECTED_DAYS, EXPECTED_ENDPOINTS):
        raise ValueError(f"自适应光伏预测维度异常: {pv_endpoints.shape}")

    historical_dates, _, historical_pv = read_power_sheet(attachment2, PV_SHEET)
    load_dates, _, historical_load = read_power_sheet(attachment2, "小区负载")
    if load_dates ≠ historical_dates:
        raise ValueError("附件2负载与光伏日期不一致")
    date_to_index = {date: i for i, date in enumerate(historical_dates)}

```

```

planning_indices = np.asarray([date_to_index[date] for date in dates])

# 目标为规划日次日00:10, 其上周同期为规划日往前6天的00:10。
load_extra_0010 = historical_load[planning_indices - 6, 0].astype(float, copy=True)

# 规划时当天00:10尚不可得, 采用规划日前7个已观测00:10的均值。
pv_extra_0010 = np.asarray(
    [historical_pv[i - 7 : i, 0].mean() for i in planning_indices], dtype=float
)
return load_endpoints, load_extra_0010, pv_endpoints, pv_extra_0010

def to_interval_average(endpoints: np.ndarray, extra_0010: np.ndarray) → np.ndarray:
    """144个日内端点加1个跨日端点, 转换为144个区间平均值。"""
    within_day = (endpoints[:, :-1] + endpoints[:, 1:]) / 2
    final_interval = ((endpoints[:, -1] + extra_0010) / 2)[: , None]
    result = np.hstack([within_day, final_interval])
    if result.shape != (EXPECTED_DAYS, 144):
        raise AssertionError(f"区间矩阵维度异常: {result.shape}")
    if not np.isfinite(result).all() or np.any(result < 0):
        raise AssertionError("区间预测含缺失、无穷或负值")
    return result

def copy_reference_style(reference_path: Path, worksheet, rows: int, columns: int) → None:
    reference_book = load_workbook(reference_path, read_only=False, data_only=False)
    reference_sheet = reference_book[reference_book.sheetnames[0]]
    # 参考文件的数值区使用默认 General 样式; 只复制非默认的表头和日期列即可。
    def apply_style(source, target) → None:
        target.font = copy(source.font)
        target.fill = copy(source.fill)
        target.border = copy(source.border)
        target.alignment = copy(source.alignment)
        target.protection = copy(source.protection)
        target.number_format = source.number_format

    for column in range(1, columns + 1):
        source_column = min(column + 1, reference_sheet.max_column) if column > 1 else 1
        apply_style(reference_sheet.cell(1, source_column), worksheet.cell(1, column))
        source_letter = get_column_letter(source_column)
        target_letter = get_column_letter(column)
        worksheet.column_dimensions[target_letter].width =
reference_sheet.column_dimensions[source_letter].width
    for row in range(2, rows + 1):
        apply_style(reference_sheet.cell(2, 1), worksheet.cell(row, 1))
    worksheet.row_dimensions[1].height = reference_sheet.row_dimensions[1].height
    reference_book.close()

def write_sheet(workbook: Workbook, name: str, dates, headers, values, reference_path: Path) → None:
    worksheet = workbook.create_sheet(name)
    worksheet.append(["日期\\区间", *headers])
    for date, row in zip(dates, values, strict=True):
        worksheet.append([date, *[float(value) for value in row]])
    copy_reference_style(reference_path, worksheet, len(dates) + 1, len(headers) + 1)

def write_output(path: Path, reference_path: Path, dates, load_intervals, pv_intervals) → None:
    workbook = Workbook()
    workbook.remove(workbook.active)
    headers = interval_headers()
    write_sheet(workbook, "负载", dates, headers, load_intervals, reference_path)
    write_sheet(workbook, "光伏", dates, headers, pv_intervals, reference_path)
    path.parent.mkdir(parents=True, exist_ok=True)
    workbook.save(path)

```



```

def validate_output(path: Path, load_expected: np.ndarray, pv_expected: np.ndarray) → None:
    workbook = load_workbook(path, read_only=True, data_only=True)
    if workbook.sheetnames ≠ ["负载", "光伏"]:
        raise AssertionError(f"工作表异常: {workbook.sheetnames}")
    headers = interval_headers()
    for sheet_name, expected in (("负载", load_expected), ("光伏", pv_expected)):
        worksheet = workbook[sheet_name]
        if (worksheet.max_row, worksheet.max_column) ≠ (335, 145):
            raise AssertionError(f"{sheet_name} 尺寸异常")
        row_iterator = worksheet.iter_rows(values_only=True)
        header_row = next(row_iterator)
        actual_headers = list(header_row[1:])
        if actual_headers ≠ headers:
            raise AssertionError(f"{sheet_name} 区间表头异常")
        data_rows = list(row_iterator)
        actual_dates = [row[0] for row in data_rows]
        if actual_dates[0] ≠ START_DATE or actual_dates[-1] ≠ datetime(2025, 12, 31):
            raise AssertionError(f"{sheet_name} 日期范围异常")
        actual = np.asarray([row[1:] for row in data_rows], dtype=float)
        if not np.allclose(actual, expected, rtol=0, atol=1e-10):
            raise AssertionError(f"{sheet_name} 写入数值与计算结果不一致")
    workbook.close()

def main() → None:
    script_dir = Path(__file__).resolve().parent
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--reference",
        type=Path,
        default=script_dir.parent.parent / "演示数据用于参考格式.xlsx",
    )
    parser.add_argument("--attachment2", type=Path, default=script_dir.parent / "附件2.xlsx")
    parser.add_argument(
        "--output",
        type=Path,
        default=script_dir / "标准化负载与光伏区间预测.xlsx",
    )
    args = parser.parse_args()

    dates = load_dates(args.reference)
    load_endpoints, load_extra, pv_endpoints, pv_extra = load_predictions(
        script_dir, args.attachment2, dates
    )
    load_intervals = to_interval_average(load_endpoints, load_extra)
    pv_intervals = to_interval_average(pv_endpoints, pv_extra)
    write_output(args.output, args.reference, dates, load_intervals, pv_intervals)
    validate_output(args.output, load_intervals, pv_intervals)
    print(f"输出文件: {args.output}")
    print(f"工作表: 负载、光伏; 日期: {dates[0]:%Y-%m-%d} 至 {dates[-1]:%Y-%m-%d}")
    print(f"每个工作表: {len(dates)}天 × {load_intervals.shape[1]}个区间")
    print(f"负载区间范围: {load_intervals.min():.6f} 至 {load_intervals.max():.6f}")
    print(f"光伏区间范围: {pv_intervals.min():.6f} 至 {pv_intervals.max():.6f}")

if __name__ == "__main__":
    main()

```

33. question2/prediction/export_standardized_intervals.py

源文件 SHA-256: 113a862ebb7151e309ac4e9b148acfcba737ca69f0ee35ee76145ceef3f3f44f ; 代码行数: 168。

```

"""输出问题二最终采用模型的 144 个十分钟区间平均预测。"""

from __future__ import annotations

import argparse
from datetime import datetime
from pathlib import Path

import numpy as np
from openpyxl import Workbook
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter

from forecast_models import LOAD_SHEET, PV_SHEET, read_power_sheet

START_INDEX = 31 # 2025-02-01
EXPECTED_DAYS = 334

def interval_headers() → list[str]:
    """生成 00:10-00:20 至次日 00:00-00:10 的 144 个区间。"""
    headers = []
    for interval_index in range(144):
        start_minutes = 10 + interval_index * 10
        end_minutes = start_minutes + 10

        def label(total_minutes: int) → str:
            if total_minutes < 1440:
                return f"{total_minutes // 60:02d}:{total_minutes % 60:02d}"
            shifted = total_minutes - 1440
            return f"次日{shifted // 60:02d}:{shifted % 60:02d}"

        headers.append(f"{label(start_minutes)}-{label(end_minutes)}")
    if headers[0] ≠ "00:10-00:20" or headers[-1] ≠ "次日00:00-次日00:10":
        raise AssertionError("区间表头生成错误")
    return headers

def next_0010_load_weekly(load_actual: np.ndarray, day_indices: np.ndarray) → np.ndarray:
    """目标 d+1 日 00:10 的上周同期为 d-6 日 00:10。"""
    return np.asarray([load_actual[day_index - 6, 0] for day_index in day_indices], dtype=float)

def next_0010_pv_history_mean(pv_actual: np.ndarray, day_indices: np.ndarray) → np.ndarray:
    """使用规划时刻已观测的 d-7 至 d-1 共 7 个 00:10 功率。"""
    return np.asarray(
        [pv_actual[day_index - 7 : day_index, 0].mean() for day_index in day_indices],
        dtype=float,
    )

def endpoints_to_intervals(main_prediction: np.ndarray, final_endpoint: np.ndarray) → np.ndarray:
    if main_prediction.shape ≠ (EXPECTED_DAYS, 144) or final_endpoint.shape ≠ (EXPECTED_DAYS,):
        raise ValueError(
            f"预测维度异常: 主预测 {main_prediction.shape}, 跨日端点 {final_endpoint.shape}"
        )
    endpoints = np.column_stack([main_prediction, final_endpoint])
    intervals = (endpoints[:, :-1] + endpoints[:, 1:]) / 2.0
    if intervals.shape ≠ (EXPECTED_DAYS, 144):
        raise AssertionError("区间平均矩阵维度错误")
    if not np.isfinite(intervals).all() or np.any(intervals < 0):
        raise ValueError("区间平均预测含缺失、无穷或负值")
    return intervals

```

```

def write_sheet(workbook: Workbook, name: str, dates, headers, values) → None:
    worksheet = workbook.create_sheet(name)
    worksheet.append(["日期\\区间", *headers])
    for date, row in zip(dates, values, strict=True):
        worksheet.append([date, *[float(value) for value in row]])

    header_fill = PatternFill("solid", fgColor="D9EAF7")
    for cell in worksheet[1]:
        cell.font = Font(bold=True)
        cell.fill = header_fill
        cell.alignment = Alignment(horizontal="center", vertical="center")
    worksheet.freeze_panes = "B2"
    worksheet.auto_filter.ref = worksheet.dimensions
    worksheet.column_dimensions["A"].width = 13
    for column_index in range(2, 146):
        worksheet.column_dimensions[get_column_letter(column_index)].width = 19
    for cell in worksheet["A"][1:]:
        cell.number_format = "yyyy-mm-dd"
        cell.alignment = Alignment(horizontal="center")
    for row in worksheet.iter_rows(min_row=2, min_col=2):
        for cell in row:
            cell.number_format = "0.0000"

def validate_output(path: Path, dates: list[datetime], headers: list[str]) → None:
    from openpyxl import load_workbook

    workbook = load_workbook(path, read_only=True, data_only=True)
    if workbook.sheetnames != ["XGBoost_负载", "自适应加权_光伏"]:
        raise AssertionError(f"工作表名称错误: {workbook.sheetnames}")
    for sheet_name in workbook.sheetnames:
        worksheet = workbook[sheet_name]
        if (worksheet.max_row, worksheet.max_column) != (335, 145):
            raise AssertionError(
                f"{sheet_name} 结构错误: {worksheet.max_row} 行×{worksheet.max_column} 列"
            )
        actual_headers = [cell.value for cell in next(worksheet.iter_rows(min_row=1, max_row=1))]
        if actual_headers != ["日期\\区间", *headers]:
            raise AssertionError(f"{sheet_name} 表头不一致")
        if worksheet.cell(2, 1).value != dates[0] or worksheet.cell(335, 1).value != dates[-1]:
            raise AssertionError(f"{sheet_name} 日期范围错误")
        values = np.asarray(
            list(worksheet.iter_rows(min_row=2, min_col=2, values_only=True)), dtype=float
        )
        if values.shape != (334, 144) or not np.isfinite(values).all() or np.any(values < 0):
            raise AssertionError(f"{sheet_name} 数值检查失败")
    workbook.close()

def main() → None:
    script_dir = Path(__file__).resolve().parent
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=script_dir.parent / "附件2.xlsx")
    parser.add_argument("--load-prediction", type=Path, default=script_dir / "results" / "XGBoost_负载_predictions.npy")
    parser.add_argument(
        "--pv-prediction",
        type=Path,
        default=script_dir / "improved_pv_results" / "自适应加权Baseline_predictions.npy",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=script_dir / "问题二_标准化区间预测.xlsx",
    )

```

```

)
args = parser.parse_args()

load_dates, _, load_actual = read_power_sheet(args.input, LOAD_SHEET)
pv_dates, _, pv_actual = read_power_sheet(args.input, PV_SHEET)
if load_dates != pv_dates:
    raise ValueError("负载和光伏日期不一致")
dates = load_dates[START_INDEX:]
day_indices = np.arange(START_INDEX, len(load_dates))
if len(dates) != EXPECTED_DAYS or dates[0] != datetime(2025, 2, 1) or dates[-1] != datetime(2025,
12, 31):
    raise ValueError("输出日期范围异常")

load_main = np.load(args.load_prediction)
pv_main = np.load(args.pv_prediction)
load_intervals = endpoints_to_intervals(
    load_main, next_0010_load_weekly(load_actual, day_indices)
)
pv_intervals = endpoints_to_intervals(
    pv_main, next_0010_pv_history_mean(pv_actual, day_indices)
)

headers = interval_headers()
workbook = Workbook()
workbook.remove(workbook.active)
write_sheet(workbook, "XGBoost_负载", dates, headers, load_intervals)
write_sheet(workbook, "自适应加权_光伏", dates, headers, pv_intervals)
args.output.parent.mkdir(parents=True, exist_ok=True)
workbook.save(args.output)
validate_output(args.output, dates, headers)

print(f"已生成: {args.output}")
print(f"日期: {dates[0]:%Y-%m-%d} 至 {dates[-1]:%Y-%m-%d}, 共 {len(dates)} 天")
print(f"每个工作表: {len(dates)} 行数据×{len(headers)} 个区间")
print(f"负载区间范围: {load_intervals.min():.4f} 至 {load_intervals.max():.4f}")
print(f"光伏区间范围: {pv_intervals.min():.4f} 至 {pv_intervals.max():.4f}")

if __name__ == "__main__":
    main()

```

34. question2/prediction/forecast_models.py

源文件 SHA-256: 3e086e7d0e3a55f7a8eb6a2eacc0644262fec26a1d9e4cbacc2ffbcf6b1d9b41 ; 代码行数: 402。

"""问题二：按日滚动预测负载与光伏（Baseline/XGBoost/LightGBM/SARIMA）。

所有预测都模拟当天 0:00 的信息边界：预测某日时只允许使用此前日期的真实值。

"""

```
from __future__ import annotations
```

```
import argparse
import json
import math
import time
import warnings
from dataclasses import asdict, dataclass
from datetime import datetime
from pathlib import Path
from typing import Callable

import numpy as np
import pandas as pd
from lightgbm import LGBMRegressor
from openpyxl import load_workbook
from sklearn.metrics import mean_absolute_error, mean_squared_error
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tools.sm_exceptions import ConvergenceWarning
from xgboost import XGBRegressor
```

```
SEED = 2026
LOAD_SHEET = "小区负载"
PV_SHEET = "光伏发电实际功率"
TARGETS = ("负载", "光伏")
DEFAULT_START = datetime(2025, 2, 1)
DEFAULT_END = datetime(2025, 12, 31)
```

```
@dataclass(frozen=True)
```

```
class Config:
    start_date: str
    end_date: str
    tree_refit_every: int = 1
    sarima_refit_every: int = 30
    n_estimators: int = 120
    max_depth: int = 5
    learning_rate: float = 0.05
    sarima_maxiter: int = 30
    seed: int = SEED
```

```
def read_power_sheet(path: Path, sheet_name: str) → tuple[list[datetime], list[object], np.ndarray]:
    workbook = load_workbook(path, read_only=True, data_only=True)
    worksheet = workbook[sheet_name]
    rows = list(worksheet.iter_rows(values_only=True))
    workbook.close()
    if len(rows) ≠ 366 or len(rows[0]) ≠ 145:
        raise ValueError(f"{sheet_name} 应为 366×145, 实际为 {len(rows)}×{len(rows[0])}")
    dates = [pd.Timestamp(row[0]).to_pydatetime() for row in rows[1:]]
    times = list(rows[0][1:])
    values = np.asarray([row[1:] for row in rows[1:]], dtype=float)
    if values.shape ≠ (365, 144) or not np.isfinite(values).all():
        raise ValueError(f"{sheet_name} 数值矩阵异常")
    if np.any(values < 0):
        raise ValueError(f"{sheet_name} 含负功率")
    return dates, times, values
```

```

def load_data(path: Path):
    dates, times, load = read_power_sheet(path, LOAD_SHEET)
    pv_dates, pv_times, pv = read_power_sheet(path, PV_SHEET)
    if dates != pv_dates or times != pv_times:
        raise ValueError("两个工作表的日期或时段不一致")
    if dates[0] != datetime(2025, 1, 1) or dates[-1] != datetime(2025, 12, 31):
        raise ValueError(f"日期范围异常: {dates[0]} 至 {dates[-1]}")
    return dates, times, {"负载": load, "光伏": pv}

def date_indices(dates, start: datetime, end: datetime) → np.ndarray:
    indices = np.asarray([i for i, date in enumerate(dates) if start ≤ date ≤ end], dtype=int)
    if len(indices) == 0:
        raise ValueError("指定预测日期范围没有数据")
    if indices[0] < 31:
        raise ValueError("预测起点必须不早于 2025-02-01, 以保证至少一个月历史数据")
    return indices

def baseline_predict(values: np.ndarray, indices: np.ndarray, target: str) → np.ndarray:
    if target == "负载":
        return np.vstack([values[i - 7] for i in indices])
    return np.vstack([values[i - 7 : i].mean(axis=0) for i in indices])

def calendar_features(date: datetime, slots: int = 144) → np.ndarray:
    slot = np.arange(slots)
    dow = date.weekday()
    doy = date.timetuple().tm_yday
    return np.column_stack(
        [
            slot / (slots - 1),
            np.sin(2 * np.pi * slot / slots),
            np.cos(2 * np.pi * slot / slots),
            np.full(slots, dow / 6),
            np.full(slots, float(dow ≥ 5)),
            np.full(slots, np.sin(2 * np.pi * dow / 7)),
            np.full(slots, np.cos(2 * np.pi * dow / 7)),
            np.full(slots, np.sin(2 * np.pi * doy / 365)),
            np.full(slots, np.cos(2 * np.pi * doy / 365)),
        ]
    )

def day_features(values: np.ndarray, dates: list[datetime], day_index: int) → np.ndarray:
    """构造在 day_index 当天 0:00 已知的特征, 不读取当天真实值。"""
    if day_index < 14:
        raise ValueError("树模型特征至少需要 14 天历史")
    history_7 = values[day_index - 7 : day_index]
    history_14 = values[day_index - 14 : day_index]
    return np.column_stack(
        [
            calendar_features(dates[day_index], values.shape[1]),
            values[day_index - 1],
            values[day_index - 2],
            values[day_index - 7],
            values[day_index - 14],
            history_7.mean(axis=0),
            np.median(history_7, axis=0),
            history_7.std(axis=0),
            history_14.mean(axis=0),
            history_14.std(axis=0),
        ]
    )

```

```

def training_matrix(values: np.ndarray, dates: list[datetime], forecast_day: int):
    days = range(14, forecast_day)
    x = np.vstack([day_features(values, dates, i) for i in days])
    y = np.concatenate([values[i] for i in days])
    return x, y

def make_xgboost(config: Config):
    return XGBRegressor(
        objective="reg:squarederror",
        n_estimators=config.n_estimators,
        max_depth=config.max_depth,
        learning_rate=config.learning_rate,
        min_child_weight=8,
        subsample=0.85,
        colsample_bytree=0.85,
        reg_lambda=2.0,
        random_state=config.seed,
        n_jobs=-1,
        tree_method="hist",
    )

def make_lightgbm(config: Config):
    return LGBMRegressor(
        objective="regression",
        n_estimators=config.n_estimators,
        max_depth=config.max_depth,
        num_leaves=min(31, 2**config.max_depth),
        learning_rate=config.learning_rate,
        min_child_samples=30,
        subsample=0.85,
        colsample_bytree=0.85,
        reg_lambda=2.0,
        random_state=config.seed,
        n_jobs=-1,
        verbosity=-1,
    )

def tree_predict(
    values: np.ndarray,
    dates: list[datetime],
    indices: np.ndarray,
    model_factory: Callable[[], object],
    refit_every: int,
) → np.ndarray:
    predictions = []
    model = None
    for offset, day_index in enumerate(indices):
        if model is None or offset % refit_every == 0:
            x_train, y_train = training_matrix(values, dates, day_index)
            model = model_factory()
            model.fit(x_train, y_train)
        pred = np.asarray(model.predict(day_features(values, dates, day_index)), dtype=float)
        predictions.append(np.clip(pred, 0.0, None))
    return np.vstack(predictions)

def fit_sarima(endog: np.ndarray, maxiter: int):
    """对每日总功率和建立带 7 日季节周期的 SARIMA。"""
    model = SARIMAX(
        endog,
        order=(1, 0, 1),
        seasonal_order=(1, 0, 0, 7),
    )

```

```

        trend="c",
        enforce_stationarity=False,
        enforce_invertibility=False,
        simple_differencing=False,
    )
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", ConvergenceWarning)
        result = model.fit(dis= False, maxiter=maxiter, method="lbfgs")
    if not result.mle_retvals.get("converged", False):
        # 早期样本较少时 L-BFGS 偶尔停在迭代上限, 改用无梯度方法复核。
        with warnings.catch_warnings():
            warnings.simplefilter("ignore", ConvergenceWarning)
            result = model.fit(dis= False, maxiter=max(100, maxiter), method="powell")
    if not result.mle_retvals.get("converged", False):
        raise RuntimeError("SARIMA 参数估计未收敛")
    return result

def normalized_profile(values: np.ndarray, day_index: int, target: str) → np.ndarray:
    """只用历史日曲线, 将 SARIMA 的日总量分配到 144 个时段。"""
    if target == "负载":
        # 优先使用过去四个同星期日; 数据不足时退回最近七天。
        candidates = [i for i in range(day_index - 7, -1, -7)][0:4]
        history = values[candidates] if candidates else values[day_index - 7 : day_index]
    else:
        history = values[day_index - 7 : day_index]
    totals = history.sum(axis=1)
    valid = totals > 1e-8
    if not valid.any():
        return np.full(values.shape[1], 1.0 / values.shape[1])
    profiles = history[valid] / totals[valid, None]
    profile = profiles.mean(axis=0)
    return profile / profile.sum()

def sarima_predict(
    values: np.ndarray,
    indices: np.ndarray,
    refit_every: int,
    maxiter: int,
    target: str,
) → np.ndarray:
    daily_totals = values.sum(axis=1)
    predictions = []
    result = None
    state_end_day = None
    for offset, day_index in enumerate(indices):
        if result is None or offset % refit_every == 0:
            result = fit_sarima(daily_totals[:day_index], maxiter)
            state_end_day = day_index
        elif state_end_day != day_index:
            raise AssertionError("SARIMA 状态更新时间不连续")

        predicted_total = max(0.0, float(np.asarray(result.forecast(steps=1))[0]))
        profile = normalized_profile(values, day_index, target)
        predictions.append(predicted_total * profile)

        # 当天结束后才将当天真实值加入状态, 供下一日 0:00 使用。
        result = result.append([daily_totals[day_index]], refit=False)
        state_end_day = day_index + 1
    return np.vstack(predictions)

def metrics(actual: np.ndarray, predicted: np.ndarray, target: str) → dict[str, float]:
    error = predicted - actual
    denominator = np.abs(actual).sum()

```



```

result = {
    "MAE": float(mean_absolute_error(actual.ravel(), predicted.ravel())),
    "RMSE": float(math.sqrt(mean_squared_error(actual.ravel(), predicted.ravel()))),
    "WAPE_percent": float(np.abs(error).sum() / denominator * 100),
    "Bias": float(error.mean()),
}
if target == "光伏":
    daylight = actual > 1.0
    result["Daylight_MAE"] = float(np.abs(error[daylight]).mean())
    result["Daylight_WAPE_percent"] = float(
        np.abs(error[daylight]).sum() / np.abs(actual[daylight]).sum() * 100
    )
return result

def evaluate(predictions, actuals, dates, indices) → tuple[pd.DataFrame, pd.DataFrame]:
    overall_rows = []
    monthly_rows = []
    for model_name, target_predictions in predictions.items():
        for target in TARGETS:
            actual = actuals[target][indices]
            pred = target_predictions[target]
            overall_rows.append({"模型": model_name, "对象": target, **metrics(actual, pred, target)})
            months = sorted({dates[i].month for i in indices})
            for month in months:
                mask = np.asarray([dates[i].month == month for i in indices])
                monthly_rows.append(
                    {"模型": model_name, "对象": target, "月份": month, **metrics(actual[mask],
pred[mask], target)})
    )
    return pd.DataFrame(overall_rows), pd.DataFrame(monthly_rows)

def save_predictions(path: Path, dates, times, indices, predictions, actuals) → None:
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        rows = []
        for model_name, target_predictions in predictions.items():
            for local_day, day_index in enumerate(indices):
                for slot in range(144):
                    rows.append(
                        {
                            "模型": model_name,
                            "日期": dates[day_index],
                            "时段序号": slot + 1,
                            "时刻": times[slot],
                            "负载预测功率": target_predictions["负载"][local_day, slot],
                            "光伏预测功率": target_predictions["光伏"][local_day, slot],
                            "负载实际功率": actuals["负载"][day_index, slot],
                            "光伏实际功率": actuals["光伏"][day_index, slot],
                        }
                    )
        pd.DataFrame(rows).to_excel(writer, sheet_name="全部预测长表", index=False)
        for model_name, target_predictions in predictions.items():
            for target in TARGETS:
                frame = pd.DataFrame(target_predictions[target], columns=[str(t) for t in times])
                frame.insert(0, "日期\\时间", [dates[i] for i in indices])
                sheet = f"{model_name}_{target}":31
                frame.to_excel(writer, sheet_name=sheet, index=False)

def parse_methods(text: str) → list[str]:
    allowed = {"Baseline", "XGBoost", "LightGBM", "SARIMA"}
    methods = [item.strip() for item in text.split(",") if item.strip()]
    unknown = set(methods) - allowed
    if unknown:
        raise ValueError(f"未知模型: {sorted(unknown)}")

```

```

return methods

def main() → None:
    script_dir = Path(__file__).resolve().parent
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=script_dir.parent / "附件2.xlsx")
    parser.add_argument("--output-dir", type=Path, default=script_dir / "results")
    parser.add_argument("--start-date", default="2025-02-01")
    parser.add_argument("--end-date", default="2025-12-31")
    parser.add_argument("--methods", default="Baseline,XGBoost,LightGBM,SARIMA")
    parser.add_argument("--tree-refit-every", type=int, default=1)
    parser.add_argument("--sarima-refit-every", type=int, default=30)
    parser.add_argument("--n-estimators", type=int, default=120)
    parser.add_argument("--sarima-maxiter", type=int, default=30)
    args = parser.parse_args()

    config = Config(
        start_date=args.start_date,
        end_date=args.end_date,
        tree_refit_every=args.tree_refit_every,
        sarima_refit_every=args.sarima_refit_every,
        n_estimators=args.n_estimators,
        max_depth=5,
        learning_rate=0.05,
        sarima_maxiter=args.sarima_maxiter,
    )
    if min(config.tree_refit_every, config.sarima_refit_every, config.n_estimators) ≤ 0:
        raise ValueError("重训练间隔和树数量必须为正整数")

    dates, times, actuals = load_data(args.input)
    indices = date_indices(dates, datetime.fromisoformat(args.start_date),
datetime.fromisoformat(args.end_date))
    methods = parse_methods(args.methods)
    args.output_dir.mkdir(parents=True, exist_ok=True)
    predictions: Dict[str, Dict[str, np.ndarray]] = {}

    for method in methods:
        started = time.perf_counter()
        predictions[method] = {}
        for target in TARGETS:
            values = actuals[target]
            if method == "Baseline":
                pred = baseline_predict(values, indices, target)
            elif method == "XGBoost":
                pred = tree_predict(values, dates, indices, lambda: make_xgboost(config),
config.tree_refit_every)
            elif method == "LightGBM":
                pred = tree_predict(values, dates, indices, lambda: make_lightgbm(config),
config.tree_refit_every)
            else:
                pred = sarima_predict(
                    values, indices, config.sarima_refit_every, config.sarima_maxiter, target
                )
            predictions[method][target] = pred
            np.save(args.output_dir / f"{method}_{target}_predictions.npy", pred)
        print(f"{method} 完成, 耗时 {time.perf_counter() - started:.1f} 秒")

    overall, monthly = evaluate(predictions, actuals, dates, indices)
    overall.to_csv(args.output_dir / "overall_metrics.csv", index=False, encoding="utf-8-sig")
    monthly.to_csv(args.output_dir / "monthly_metrics.csv", index=False, encoding="utf-8-sig")
    save_predictions(args.output_dir / "all_model_predictions.xlsx", dates, times, indices, predictions,
actuals)
    manifest = {
        "input": str(args.input.resolve()),
        "config": asdict(config),

```

```
        "methods": methods,
        "prediction_days": len(indices),
        "periods_per_day": 144,
        "information_boundary": "预测某日时仅使用此前日期的真实数据",
    }
    (args.output_dir / "run_config.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(overall.to_string(index=False))

if __name__ == "__main__":
    main()
```

35. question2/prediction/improved_pv_forecast.py

源文件 SHA-256: a759333c9f0ddc088e4975d203e362ffb1021285af794801e71744a6bda2163f ; 代码行数: 338。

```

"""问题二光伏预测改进实验，并与原四模型进行统一滚动回测比较。"""

from __future__ import annotations

import argparse
import json
import math
import time
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import nnls
from sklearn.metrics import r2_score
from xgboost import XGBRegressor

from forecast_models import PV_SHEET, date_indices, read_power_sheet

SEED = 2026
OLD_MODELS = ("Baseline", "XGBoost", "LightGBM", "SARIMA")
NEW_MODELS = ("日总量+归一化曲线", "晴空包络短期外推", "自适应加权Baseline")

def daily_total_features(totals: np.ndarray, date: datetime, day_index: int) → np.ndarray:
    """当天 0:00 已知的日总量滞后与日历特征。"""
    history_7 = totals[day_index - 7 : day_index]
    history_14 = totals[day_index - 14 : day_index]
    doy = date.timetuple().tm_yday
    dow = date.weekday()
    return np.asarray(
        [
            totals[day_index - 1],
            totals[day_index - 2],
            totals[day_index - 3],
            totals[day_index - 7],
            totals[day_index - 14],
            history_7.mean(),
            history_7.std(),
            history_14.mean(),
            history_14.std(),
            (totals[day_index - 1] - totals[day_index - 3]) / 2,
            math.sin(2 * math.pi * doy / 365),
            math.cos(2 * math.pi * doy / 365),
            math.sin(2 * math.pi * dow / 7),
            math.cos(2 * math.pi * dow / 7),
        ],
        dtype=float,
    )

def normalized_mean_profile(values: np.ndarray, day_index: int, days: int = 7) → np.ndarray:
    history = values[day_index - days : day_index]
    totals = history.sum(axis=1)
    valid = totals > 1e-8
    profiles = history[valid] / totals[valid, None]
    # 越近的日期权重越高，同时保留 7 天平滑能力。
    weights = np.geomspace(0.65, 1.0, len(profiles))
    profile = np.average(profiles, axis=0, weights=weights)
    profile = np.clip(profile, 0.0, None)
    return profile / profile.sum()

```

```

def energy_profile_predict(
    values: np.ndarray, dates: list[datetime], indices: np.ndarray, n_estimators: int
) → np.ndarray:
    """XGBoost 预测日总量，近期归一化曲线预测日内形状。"""
    totals = values.sum(axis=1)
    predictions = []
    for day_index in indices:
        train_days = np.arange(14, day_index)
        x_train = np.vstack([daily_total_features(totals, dates[j], j) for j in train_days])
        y_train = totals[train_days]
        model = XGBRegressor(
            objective="reg:squarederror",
            n_estimators=n_estimators,
            max_depth=3,
            learning_rate=0.05,
            min_child_weight=4,
            subsample=0.9,
            colsample_bytree=0.9,
            reg_lambda=5.0,
            random_state=SEED,
            n_jobs=-1,
            tree_method="hist",
        )
        model.fit(x_train, y_train)
        predicted_total = max(
            0.0,
            float(model.predict(daily_total_features(totals, dates[day_index], day_index)[None, :])[0]),
        )
        predictions.append(predicted_total * normalized_mean_profile(values, day_index))
    return np.vstack(predictions)

def smooth_curve(curve: np.ndarray) → np.ndarray:
    kernel = np.asarray([1, 2, 3, 2, 1], dtype=float)
    kernel /= kernel.sum()
    return np.convolve(curve, kernel, mode="same")

def extrapolated_clear_sky_envelope(values: np.ndarray, day_index: int) → np.ndarray:
    """由两个历史窗口的高分位包络估计季节趋势，并短期外推至目标日。"""
    history_start = max(0, day_index - 42)
    history = values[history_start:day_index]
    split = max(7, len(history) // 2)
    old = history[:-split]
    recent = history[-split:]
    if len(old) < 5:
        old = history[: max(5, len(history) // 2)]
    old_envelope = np.quantile(old, 0.90, axis=0)
    recent_envelope = np.quantile(recent, 0.90, axis=0)

    center_gap = max(1.0, (len(old) + len(recent)) / 2)
    horizon_from_recent_center = (len(recent) + 1) / 2
    slope = (recent_envelope - old_envelope) / center_gap
    # 限制短期趋势，防止云量异常导致高分位包络爆炸。
    slope_limit = np.maximum(2.0, 0.02 * np.maximum(recent_envelope, 1.0))
    slope = np.clip(slope, -slope_limit, slope_limit)
    envelope = recent_envelope + slope * horizon_from_recent_center
    envelope = smooth_curve(np.clip(envelope, 0.0, None))

    historical_cap = np.max(history, axis=0)
    envelope = np.minimum(envelope, np.maximum(1.15 * historical_cap, recent_envelope))
    # 历史持续无发电的夜间强制为零。
    envelope[historical_cap < 1.0] = 0.0
    return np.clip(envelope, 0.0, None)

```

```

def clear_sky_predict(values: np.ndarray, indices: np.ndarray) → np.ndarray:
    envelopes: dict[int, np.ndarray] = {}

    def envelope(day_index: int) → np.ndarray:
        if day_index not in envelopes:
            envelopes[day_index] = extrapolated_clear_sky_envelope(values, day_index)
        return envelopes[day_index]

    predictions = []
    for day_index in indices:
        target_envelope = envelope(day_index)
        ratios = []
        for historical_day in range(day_index - 7, day_index):
            historical_envelope = envelope(historical_day)
            ratio = np.divide(
                values[historical_day],
                historical_envelope,
                out=np.zeros(144, dtype=float),
                where=historical_envelope > 1.0,
            )
            ratios.append(np.clip(ratio, 0.0, 1.25))
        # 最近日权重更大; 晴空指数承载短期天气持续性。
        weights = np.geomspace(0.55, 1.0, 7)
        predicted_ratio = np.average(np.vstack(ratios), axis=0, weights=weights)
        prediction = target_envelope * predicted_ratio
        predictions.append(np.minimum(np.clip(prediction, 0.0, None), 1.10 * target_envelope))
    return np.vstack(predictions)

def baseline_candidates(values: np.ndarray, day_index: int) → np.ndarray:
    return np.column_stack(
        [
            values[day_index - 1],
            values[day_index - 2],
            values[day_index - 7],
            values[day_index - 7 : day_index].mean(axis=0),
        ]
    )

def adaptive_baseline_predict(
    values: np.ndarray, indices: np.ndarray, calibration_days: int = 28
) → tuple[np.ndarray, np.ndarray]:
    """滚动 NNLS 学习四个历史基准的非负、和为 1 的自适应权重。"""
    predictions = []
    all_weights = []
    for day_index in indices:
        first_train_day = max(7, day_index - calibration_days)
        train_days = range(first_train_day, day_index)
        x_train = np.vstack([baseline_candidates(values, j) for j in train_days])
        y_train = np.concatenate([values[j] for j in train_days])
        daylight = np.logical_or(y_train > 1.0, x_train.max(axis=1) > 1.0)
        weights, _ = nnls(x_train[daylight], y_train[daylight])
        if weights.sum() ≤ 1e-12:
            weights = np.full(4, 0.25)
        else:
            weights /= weights.sum()
        prediction = baseline_candidates(values, day_index) @ weights
        predictions.append(np.clip(prediction, 0.0, None))
        all_weights.append(weights)
    return np.vstack(predictions), np.vstack(all_weights)

def extended_metrics(actual: np.ndarray, predicted: np.ndarray) → dict[str, float]:
    error = predicted - actual
    # 所有模型共用真实功率阈值, 避免模型相关的“日间”样本集合造成不公平比较。

```

```

daylight = actual > 1.0
# 每个样本代表 10 分钟，功率和乘 1/6 h 后得到日电量。
actual_energy = actual.sum(axis=1) / 6.0
predicted_energy = predicted.sum(axis=1) / 6.0
actual_peak = actual.max(axis=1)
predicted_peak = predicted.max(axis=1)
return {
    "MAE": float(np.abs(error).mean()),
    "RMSE": float(np.sqrt(np.square(error).mean())),
    "WAPE_percent": float(np.abs(error).sum() / actual.sum() * 100),
    "Bias": float(error.mean()),
    "Daylight_MAE": float(np.abs(error[daylight]).mean()),
    "Daylight_WAPE_percent": float(np.abs(error[daylight]).sum() / actual[daylight].sum() * 100),
    "R2": float(r2_score(actual.ravel(), predicted.ravel())),
    "Daily_energy_WAPE_percent": float(
        np.mean(np.abs(predicted_energy - actual_energy) / actual_energy) * 100
    ),
    "Daily_energy_MAE": float(np.abs(predicted_energy - actual_energy).mean()),
    "Daily_peak_MAE": float(np.abs(predicted_peak - actual_peak).mean()),
    "Overforecast_rate_percent": float((predicted[daylight] > actual[daylight]).mean() * 100),
}

def comparison_tables(predictions, actual, dates, indices):
    overall_rows = []
    monthly_rows = []
    for model_name, predicted in predictions.items():
        overall_rows.append({"模型": model_name, **extended_metrics(actual, predicted)})
        for month in sorted({dates[i].month for i in indices}):
            mask = np.asarray([dates[i].month == month for i in indices])
            monthly_rows.append({"模型": model_name, "月份": month, **extended_metrics(actual[mask],
predicted[mask])})
    overall = pd.DataFrame(overall_rows).sort_values("WAPE_percent").reset_index(drop=True)
    monthly = pd.DataFrame(monthly_rows)
    return overall, monthly

def save_excel(path: Path, predictions, actual, dates, times, indices, weights) → None:
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        long_parts = []
        for model_name, predicted in predictions.items():
            frame = pd.DataFrame(predicted, columns=[str(item) for item in times])
            frame.insert(0, "日期\\时间", [dates[i] for i in indices])
            frame.to_excel(writer, sheet_name=model_name[:31], index=False)
            long_parts.append(
                pd.DataFrame(
                    {
                        "模型": model_name,
                        "日期": np.repeat([dates[i] for i in indices], 144),
                        "时段序号": np.tile(np.arange(1, 145), len(indices)),
                        "时刻": np.tile(times, len(indices)),
                        "预测功率": predicted.ravel(),
                        "实际功率": np.tile(np.nan, predicted.size),
                    }
                )
            )
        long_parts[-1]["实际功率"] = actual.ravel()
        pd.concat(long_parts, ignore_index=True).to_excel(writer, sheet_name="全部模型长表", index=False)
        pd.DataFrame(
            weights,
            columns=["前1天", "前2天", "前7天", "过去7天均值"],
            index=[dates[i] for i in indices],
        ).rename_axis("日期").to_excel(writer, sheet_name="自适应权重")

def main() → None:

```

```

script_dir = Path(__file__).resolve().parent
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("--input", type=Path, default=script_dir.parent / "附件2.xlsx")
parser.add_argument("--old-results", type=Path, default=script_dir / "results")
parser.add_argument("--output-dir", type=Path, default=script_dir / "improved_pv_results")
parser.add_argument("--start-date", default="2025-02-01")
parser.add_argument("--end-date", default="2025-12-31")
parser.add_argument("--n-estimators", type=int, default=100)
args = parser.parse_args()

dates, times, values = read_power_sheet(args.input, PV_SHEET)
indices = date_indices(dates, datetime.fromisoformat(args.start_date),
datetime.fromisoformat(args.end_date))
actual = values[indices]
args.output_dir.mkdir(parents=True, exist_ok=True)

predictions = {}
for model_name in OLD_MODELS:
    path = args.old_results / f"{model_name}_光伏_predictions.npy"
    full_prediction = np.load(path)
    positions = indices - 31 # 旧结果固定从 2025-02-01 开始。
    if positions.min() < 0 or positions.max() ≥ len(full_prediction):
        raise ValueError(f"旧模型 {model_name} 不覆盖本次日期范围")
    predicted = full_prediction[positions]
    if predicted.shape ≠ actual.shape:
        raise ValueError(f"旧模型 {model_name} 截取后维度异常: {predicted.shape}")
    predictions[model_name] = predicted

started = time.perf_counter()
predictions[NEW_MODELS[0]] = energy_profile_predict(values, dates, indices, args.n_estimators)
print(f"{NEW_MODELS[0]} 完成, 耗时 {time.perf_counter() - started:.1f} 秒", flush=True)

started = time.perf_counter()
predictions[NEW_MODELS[1]] = clear_sky_predict(values, indices)
print(f"{NEW_MODELS[1]} 完成, 耗时 {time.perf_counter() - started:.1f} 秒", flush=True)

started = time.perf_counter()
predictions[NEW_MODELS[2]], adaptive_weights = adaptive_baseline_predict(values, indices)
print(f"{NEW_MODELS[2]} 完成, 耗时 {time.perf_counter() - started:.1f} 秒", flush=True)
if np.any(adaptive_weights < -1e-12) or not np.allclose(adaptive_weights.sum(axis=1), 1.0):
    raise AssertionError("自适应组合权重必须非负且逐日和为 1")

for model_name in NEW_MODELS:
    predicted = predictions[model_name]
    if predicted.shape ≠ actual.shape or not np.isfinite(predicted).all() or np.any(predicted < 0):
        raise AssertionError(f"{model_name} 输出存在维度、有限性或非负性错误")
    np.save(args.output_dir / f"{model_name}_predictions.npy", predicted)

overall, monthly = comparison_tables(predictions, actual, dates, indices)
overall.to_csv(args.output_dir / "光伏模型综合指标.csv", index=False, encoding="utf-8-sig")
monthly.to_csv(args.output_dir / "光伏模型逐月指标.csv", index=False, encoding="utf-8-sig")
save_excel(
    args.output_dir / "光伏改进模型预测与对比.xlsx",
    predictions,
    actual,
    dates,
    times,
    indices,
    adaptive_weights,
)
config = {
    "prediction_period": [args.start_date, args.end_date],
    "information_boundary": "每个预测日仅使用该日0:00以前的历史数据",
    "new_models": list(NEW_MODELS),
    "energy_profile_n_estimators": args.n_estimators,
    "adaptive_calibration_days": 28,

```



```
        "clear_sky_history_days": 42,  
        "clear_sky_quantile": 0.90,  
        "seed": SEED,  
    }  
    (args.output_dir / "改进实验配置.json").write_text(  
        json.dumps(config, ensure_ascii=False, indent=2), encoding="utf-8"  
    )  
    print(overall.to_string(index=False), flush=True)  
  
if __name__ == "__main__":  
    main()
```

36. question2/prediction/interval_forecast_postprocess.py

源文件 SHA-256: f2ee4976a9ee45e5723164b2b8675cefda1b1659a11bbf304746f6d76d0ad399 ; 代码行数: 180。

```

"""把现有端点预测转换为144个区间平均预测，并重新计算统一指标。"""

from __future__ import annotations

import argparse
import math
from pathlib import Path

import numpy as np
import pandas as pd
from openpyxl import load_workbook
from sklearn.metrics import r2_score

from forecast_models import LOAD_SHEET, PV_SHEET, read_power_sheet
from interval_preprocessing import interval_average_actual, period_mapping

ORIGINAL_MODELS = ("Baseline", "XGBoost", "LightGBM", "SARIMA")
IMPROVED_PV_MODELS = ("日总量+归一化曲线", "晴空包络短期外推", "自适应加权Baseline")
FIRST_FORECAST_INDEX = 31

def endpoint_to_interval(predicted_endpoints: np.ndarray, observed_00: np.ndarray) → np.ndarray:
    """真实00:00与预测00:10-次日00:00组成145端点，再取相邻均值。"""
    if predicted_endpoints.ndim != 2 or predicted_endpoints.shape[1] != 144:
        raise ValueError("端点预测必须为 n×144 矩阵")
    if len(observed_00) != len(predicted_endpoints):
        raise ValueError("00:00真实边界长度与预测天数不一致")
    endpoints = np.column_stack([observed_00, predicted_endpoints])
    return (endpoints[:, :-1] + endpoints[:, 1:]) / 2

def basic_metrics(actual: np.ndarray, predicted: np.ndarray, target: str) → dict[str, float]:
    error = predicted - actual
    result = {
        "MAE": float(np.abs(error).mean()),
        "RMSE": float(np.sqrt(np.square(error).mean())),
        "WAPE_percent": float(np.abs(error).sum() / np.abs(actual).sum() * 100),
        "Bias": float(error.mean()),
        "R2": float(r2_score(actual.ravel(), predicted.ravel())),
    }
    actual_energy = actual.sum(axis=1) / 6
    predicted_energy = predicted.sum(axis=1) / 6
    result["Daily_energy_WAPE_percent"] = float(
        np.mean(np.abs(predicted_energy - actual_energy) / np.maximum(actual_energy, 1e-8)) * 100
    )
    result["Daily_energy_MAE"] = float(np.abs(predicted_energy - actual_energy).mean())
    result["Daily_peak_MAE"] = float(np.abs(predicted.max(axis=1) - actual.max(axis=1)).mean())
    if target == "光伏":
        daylight = actual > 1.0
        result["Daylight_MAE"] = float(np.abs(error[daylight]).mean())
        result["Daylight_WAPE_percent"] = float(
            np.abs(error[daylight]).sum() / actual[daylight].sum() * 100
        )
        result["Overforecast_rate_percent"] = float(
            (predicted[daylight] > actual[daylight]).mean() * 100
        )
    return result

def wide_frame(dates, labels, values):
    frame = pd.DataFrame(values, columns=labels)
    frame.insert(0, "日期\\区间", dates)
    return frame

```

```

def save_baseline_pipeline(path, dates, labels, load_prediction, pv_prediction):
    long = pd.DataFrame(
        {
            "日期": np.repeat(dates, 144),
            "时段序号": np.tile(np.arange(1, 145), len(dates)),
            "时间区间": np.tile(labels, len(dates)),
            "负载区间平均预测功率": load_prediction.ravel(),
            "光伏区间平均预测功率": pv_prediction.ravel(),
        }
    )
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        wide_frame(dates, labels, load_prediction).to_excel(writer, sheet_name="负载区间预测",
index=False)
        wide_frame(dates, labels, pv_prediction).to_excel(writer, sheet_name="光伏区间预测", index=False)
        long.to_excel(writer, sheet_name="预测长表", index=False)

def save_all_predictions(path, dates, labels, predictions, metrics):
    with pd.ExcelWriter(path, engine="openpyxl") as writer:
        metrics.to_excel(writer, sheet_name="区间预测指标", index=False)
        for (model, target), values in predictions.items():
            wide_frame(dates, labels, values).to_excel(
                writer, sheet_name=f"{model}_{target}[:31]", index=False
            )

def validate_excel(path: Path, expected_sheets: int) → None:
    workbook = load_workbook(path, read_only=True, data_only=True)
    if len(workbook.sheetnames) ≠ expected_sheets:
        raise AssertionError(f"工作表数量异常: {len(workbook.sheetnames)}")
    for sheet_name in workbook.sheetnames[1:]:
        worksheet = workbook[sheet_name]
        if worksheet.max_row ≠ 335 or worksheet.max_column ≠ 145:
            raise AssertionError(f"{sheet_name}维度异常")
    workbook.close()

def main() → None:
    script_dir = Path(__file__).resolve().parent
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--attachment2", type=Path, default=script_dir.parent / "附件2.xlsx")
    parser.add_argument("--original-results", type=Path, default=script_dir / "results")
    parser.add_argument("--improved-results", type=Path, default=script_dir / "improved_pv_results")
    parser.add_argument("--output-dir", type=Path, default=script_dir / "interval_results")
    args = parser.parse_args()

    load_dates, _, load_actual_endpoints = read_power_sheet(args.attachment2, LOAD_SHEET)
    pv_dates, _, pv_actual_endpoints = read_power_sheet(args.attachment2, PV_SHEET)
    if load_dates ≠ pv_dates:
        raise ValueError("附件2负载和光伏日期不一致")
    dates = load_dates[FIRST_FORECAST_INDEX:]
    labels = period_mapping()["时间区间"].tolist()

    load_00, load_actual_all = interval_average_actual(load_actual_endpoints)
    pv_00, pv_actual_all = interval_average_actual(pv_actual_endpoints)
    actuals = {
        "负载": load_actual_all[FIRST_FORECAST_INDEX:],
        "光伏": pv_actual_all[FIRST_FORECAST_INDEX:],
    }
    boundaries = {
        "负载": load_00[FIRST_FORECAST_INDEX:],
        "光伏": pv_00[FIRST_FORECAST_INDEX:],
    }

```

```

predictions: dict[tuple[str, str], np.ndarray] = {}
for model in ORIGINAL_MODELS:
    for target in ("负载", "光伏"):
        endpoint_path = args.original_results / f"{model}_{target}_predictions.npy"
        endpoint_prediction = np.load(endpoint_path)
        predictions[(model, target)] = endpoint_to_interval(
            endpoint_prediction, boundaries[target]
        )
for model in IMPROVED_PV_MODELS:
    endpoint_path = args.improved_results / f"{model}_predictions.npy"
    predictions[(model, "光伏")] = endpoint_to_interval(
        np.load(endpoint_path), boundaries["光伏"]
    )

rows = []
monthly_rows = []
for (model, target), predicted in predictions.items():
    actual = actuals[target]
    if predicted.shape != (334, 144) or np.any(predicted < 0) or not np.isfinite(predicted).all():
        raise AssertionError(f"{model}-{target}区间预测异常")
    rows.append({"模型": model, "对象": target, **basic_metrics(actual, predicted, target)})
    for month in range(2, 13):
        mask = np.asarray([date.month == month for date in dates])
        monthly_rows.append(
            {"模型": model, "对象": target, "月份": month, **basic_metrics(actual[mask],
predicted[mask], target)}
        )

metrics = pd.DataFrame(rows).sort_values(["对象", "WAPE_percent"]).reset_index(drop=True)
monthly_metrics = pd.DataFrame(monthly_rows)
args.output_dir.mkdir(parents=True, exist_ok=True)
metrics.to_csv(args.output_dir / "区间预测综合指标.csv", index=False, encoding="utf-8-sig")
monthly_metrics.to_csv(args.output_dir / "区间预测逐月指标.csv", index=False, encoding="utf-8-sig")
for (model, target), values in predictions.items():
    np.save(args.output_dir / f"{model}_{target}_区间预测.npy", values)

save_baseline_pipeline(
    args.output_dir / "baseline_interval_predictions.xlsx",
    dates,
    labels,
    predictions[("Baseline", "负载")],
    predictions[("Baseline", "光伏")],
)
all_path = args.output_dir / "全部模型区间预测与指标.xlsx"
save_all_predictions(all_path, dates, labels, predictions, metrics)
validate_excel(all_path, 1 + len(predictions))
print(metrics.to_string(index=False))
print(f"已生成: {all_path}")

if __name__ == "__main__":
    main()

```

37. question2/prediction/interval_preprocessing.py

源文件 SHA-256: 59bc15c083e3ce390fbb598c3cc38d4f6ebf3eb1911c5a58499fffc745a78535 ; 代码行数: 232。

```

"""将附件中的瞬时端点数据转换为 10 分钟区间平均数据并保存。"""

from __future__ import annotations

import argparse
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Alignment, Font, PatternFill

PERIODS = 144
STEP_MINUTES = 10

def minute_label(minutes: int, next_day: bool = False) → str:
    if minutes == 1440 or next_day:
        return "次日00:00"
    return f"{minutes // 60:02d}:{minutes % 60:02d}"

def period_mapping() → pd.DataFrame:
    rows = []
    for k in range(PERIODS):
        start = k * STEP_MINUTES
        end = (k + 1) * STEP_MINUTES
        midpoint = start + STEP_MINUTES // 2
        rows.append(
            {
                "时段序号": k + 1,
                "开始时刻": minute_label(start),
                "结束时刻": minute_label(end),
                "中点时刻": minute_label(midpoint),
                "时间区间": f"{minute_label(start)}-{minute_label(end)}",
                "时长/h": 1 / 6,
            }
        )
    return pd.DataFrame(rows)

def read_attachment1(path: Path) → tuple[list[str], np.ndarray]:
    workbook = load_workbook(path, read_only=True, data_only=True)
    worksheet = workbook[workbook.sheetnames[0]]
    rows = list(worksheet.iter_rows(values_only=True))
    workbook.close()
    if len(rows) != 145 or len(rows[0]) != 4:
        raise ValueError(f"附件1应为145行×4列，实际为{len(rows)}行×{len(rows[0])}列")
    headers = [str(item) for item in rows[0]]
    values = np.asarray([[row[j] for j in range(1, 4)] for row in rows[1:]], dtype=float)
    if values.shape != (144, 3) or not np.isfinite(values).all():
        raise ValueError("附件1数值区域异常")
    return headers, values

def attachment1_tables(path: Path) → tuple[pd.DataFrame, pd.DataFrame]:
    headers, endpoints_10_to_24 = read_attachment1(path)
    # 按题设修订约定：单日附件缺少 00:00，使用该表 24:00 数值补齐。
    endpoints = np.vstack([endpoints_10_to_24[-1], endpoints_10_to_24])
    endpoint_labels = ["00:00"] + [minute_label((k + 1) * 10) for k in range(144)]
    endpoint_table = pd.DataFrame(endpoints, columns=headers[1:])
    endpoint_table.insert(0, headers[0], endpoint_labels)

```

```

averages = (endpoints[:-1] + endpoints[1:]) / 2
mapping = period_mapping()
interval_table = mapping.copy()
for column_index, column_name in enumerate(headers[1:]):
    interval_table[f"区间平均{column_name}"] = averages[:, column_index]
return endpoint_table, interval_table

def read_attachment2_sheet(path: Path, sheet_name: str):
    workbook = load_workbook(path, read_only=True, data_only=True)
    worksheet = workbook[sheet_name]
    rows = list(worksheet.iter_rows(values_only=True))
    workbook.close()
    if len(rows) != 366 or len(rows[0]) != 145:
        raise ValueError(f"{sheet_name}应为366行×145列")
    dates = [pd.Timestamp(row[0]).to_pydatetime() for row in rows[1:]]
    values = np.asarray([row[1:] for row in rows[1:]], dtype=float)
    if values.shape != (365, 144) or not np.isfinite(values).all():
        raise ValueError(f"{sheet_name}数值区域异常")
    return dates, values

def interval_average_actual(values: np.ndarray) → tuple[np.ndarray, np.ndarray]:
    """返回每天真实 00:00 边界和 144 个区间平均值。"""
    starts = np.empty(values.shape[0], dtype=float)
    starts[0] = values[0, -1] # 仅首日缺少前一日边界, 按约定近似。
    starts[1:] = values[:-1, -1]
    endpoints = np.column_stack([starts, values])
    averages = (endpoints[:, :-1] + endpoints[:, 1:]) / 2
    return starts, averages

def wide_table(dates, averages: np.ndarray, interval_labels: list[str]) → pd.DataFrame:
    frame = pd.DataFrame(averages, columns=interval_labels)
    frame.insert(0, "日期\\区间", dates)
    return frame

def build_output(attachment1: Path, attachment2: Path, output: Path) → None:
    endpoint_table, attachment1_intervals = attachment1_tables(attachment1)
    mapping = period_mapping()
    interval_labels = mapping["时间区间"].tolist()

    load_dates, load_values = read_attachment2_sheet(attachment2, "小区负载")
    pv_dates, pv_values = read_attachment2_sheet(attachment2, "光伏发电实际功率")
    if load_dates != pv_dates:
        raise ValueError("附件2两个工作表的日期不一致")
    load_00, load_intervals = interval_average_actual(load_values)
    pv_00, pv_intervals = interval_average_actual(pv_values)

    long_table = pd.DataFrame(
        {
            "日期": np.repeat(load_dates, PERIODS),
            "时段序号": np.tile(np.arange(1, PERIODS + 1), len(load_dates)),
            "时间区间": np.tile(interval_labels, len(load_dates)),
            "区间平均负载功率": load_intervals.ravel(),
            "区间平均光伏功率": pv_intervals.ravel(),
        }
    )
    boundaries = pd.DataFrame(
        {
            "日期": load_dates,
            "00:00真实负载功率": load_00,
            "00:00真实光伏功率": pv_00,
            "边界来源": ["首日以本日24:00近似"] + ["前一日24:00真实值"] * (len(load_dates) - 1),
        }
    )

```

```

    }
)

# 论文展示表对应附件1的144个规划区间，列名简化以便直接复制到论文。
paper_table = attachment1_intervals.rename(
    columns={
        attachment1_intervals.columns[6]: "平均电价",
        attachment1_intervals.columns[7]: "平均负载功率",
        attachment1_intervals.columns[8]: "平均光伏功率",
    }
)[["时段序号", "时间区间", "平均电价", "平均负载功率", "平均光伏功率"]]

output.parent.mkdir(parents=True, exist_ok=True)
notes = pd.DataFrame(
    [
        ["数据含义", "附件原值视为每10分钟时刻的瞬时端点值"],
        ["规划时刻", "每天00:00制定计划，并获得该时刻真实电价、负载和光伏功率"],
        ["预测范围", "模型预测00:10至次日00:00共144个未来端点"],
        ["区间转换", "第k个区间平均值=(区间左端点+区间右端点)/2"],
        ["附件1的00:00", "按约定使用附件1中24:00的数值补齐"],
        ["附件2的00:00", "1月2日起使用前一日24:00真实值；1月1日以本日24:00近似"],
        ["区间数量", "145个端点形成144个10分钟区间"],
        ["电量换算", "区间平均功率乘1/6小时得到该区间电量"],
    ],
    columns=["项目", "处理规则"],
)

with pd.ExcelWriter(output, engine="openpyxl") as writer:
    notes.to_excel(writer, sheet_name="处理说明", index=False)
    mapping.to_excel(writer, sheet_name="时段映射", index=False)
    endpoint_table.to_excel(writer, sheet_name="附件1_145个端点", index=False)
    attachment1_intervals.to_excel(writer, sheet_name="附件1_区间平均", index=False)
    paper_table.to_excel(writer, sheet_name="论文展示表", index=False)
    boundaries.to_excel(writer, sheet_name="每日00点真实边界", index=False)
    wide_table(load_dates, load_intervals, interval_labels).to_excel(
        writer, sheet_name="负载区间平均_宽表", index=False
    )
    wide_table(pv_dates, pv_intervals, interval_labels).to_excel(
        writer, sheet_name="光伏区间平均_宽表", index=False
    )
    long_table.to_excel(writer, sheet_name="全年区间平均_长表", index=False)

format_workbook(output)
validate_output(output)

def format_workbook(path: Path) → None:
    workbook = load_workbook(path)
    header_fill = PatternFill("solid", fgColor="D9EAF7")
    for worksheet in workbook.worksheets:
        worksheet.freeze_panes = "A2"
        worksheet.auto_filter.ref = worksheet.dimensions
        for cell in worksheet[1]:
            cell.font = Font(bold=True)
            cell.fill = header_fill
            cell.alignment = Alignment(horizontal="center", vertical="center")
        for column_cells in worksheet.columns:
            sample = list(column_cells)[:200]
            width = min(24, max(10, max(len(str(cell.value or "")) for cell in sample) + 2))
            worksheet.column_dimensions[column_cells[0].column_letter].width = width
        for row in worksheet.iter_rows(min_row=2):
            for cell in row:
                if isinstance(cell.value, float):
                    cell.number_format = "0.0000"
    workbook.save(path)

```

```

def validate_output(path: Path) → None:
    workbook = load_workbook(path, read_only=True, data_only=True)
    expected = {
        "处理说明": (9, 2),
        "时段映射": (145, 6),
        "附件1_145个端点": (146, 4),
        "附件1_区间平均": (145, 9),
        "论文展示表": (145, 5),
        "每日00点真实边界": (366, 4),
        "负载区间平均_宽表": (366, 145),
        "光伏区间平均_宽表": (366, 145),
        "全年区间平均_长表": (365 * 144 + 1, 5),
    }
    for sheet_name, shape in expected.items():
        worksheet = workbook[sheet_name]
        actual = (worksheet.max_row, worksheet.max_column)
        if actual != shape:
            raise AssertionError(f"{sheet_name}维度异常: 期望{shape}, 实际{actual}")
    workbook.close()

def main() → None:
    script_dir = Path(__file__).resolve().parent
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--attachment1", type=Path, default=script_dir.parent / "附件1.xlsx")
    parser.add_argument("--attachment2", type=Path, default=script_dir.parent / "附件2.xlsx")
    parser.add_argument("--output", type=Path, default=script_dir / "区间平均预处理数据.xlsx")
    args = parser.parse_args()
    build_output(args.attachment1, args.attachment2, args.output)
    print(f"已生成: {args.output}")
    print("附件1: 145个瞬时端点 → 144个区间平均值")
    print("附件2: 365天 × 144个区间; 2月以后00:00边界均来自前一日24:00真实值")

if __name__ == "__main__":
    main()

```

38. question3/judge/analyze_forecast_value.py

源文件 SHA-256: 0d16cf4d828c67b362b8ed3455e147149d5ab7f4627275501fa82c39e055d483 ; 代码行数: 400。


```
#!/usr/bin/env python3
"""问题三：8种预报/调单组合的经济价值分析。

唯一正式复现命令（在 C题 根目录执行）：
F:\miniconda3\envs\fond\python.exe 附件\question3\judge\analyze_forecast_value.py
"""

from __future__ import annotations

import argparse
import hashlib
import json
import math
import platform
from itertools import combinations, permutations
from pathlib import Path

import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from PIL import Image, ImageOps

SEED = 20260912
PLAYERS = ("6", "12", "18")
STRATEGIES = {
    frozenset(): "H0",
    frozenset({"6"}): "H0_6",
    frozenset({"12"}): "H0_12",
    frozenset({"18"}): "H0_18",
    frozenset({"6", "12"}): "H0_6_12",
    frozenset({"6", "18"}): "H0_6_18",
    frozenset({"12", "18"}): "H0_12_18",
    frozenset({"6", "12", "18"}): "H0_6_12_18",
}
LABELS = {
    "H0": "仅0点", "H0_6": "0+6点", "H0_12": "0+12点", "H0_18": "0+18点",
    "H0_6_12": "0+6+12点", "H0_6_18": "0+6+18点",
    "H0_12_18": "0+12+18点", "H0_6_12_18": "0+6+12+18点",
}
COLORS = {"6": "#0072B2", "12": "#E69F00", "18": "#009E73"}

def parse_args() → argparse.Namespace:
    here = Path(__file__).resolve().parent
    default_main = here.parent / "第三问_完整计算" / "第三问_完整计算" / "results" / "main"
    p = argparse.ArgumentParser()
    p.add_argument("--input-dir", type=Path, default=default_main)
    p.add_argument("--output-dir", type=Path, default=here)
    p.add_argument("--test-start", default="2025-02-25")
    p.add_argument("--bootstrap", type=int, default=5000)
    p.add_argument("--blocks", type=int, nargs="+", default=[7, 14])
    p.add_argument("--seed", type=int, default=SEED)
    p.add_argument("--smoke", action="store_true", help="只验证读取、配对和核心公式，不正式出图")
    return p.parse_args()

def sha256(path: Path) → str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b''):
            h.update(chunk)
    return h.hexdigest()
```

```

def load_daily(input_dir: Path) → tuple[pd.DataFrame, dict[str, str]]:
    frames, hashes = [], {}
    for coalition, strategy in STRATEGIES.items():
        path = input_dir / strategy / "daily.csv"
        if not path.exists():
            raise FileNotFoundError(path)
        df = pd.read_csv(path)
        required = {"strategy", "window_date", "cash_cost_yuan", "emergency_kwh"}
        if not required.issubset(df.columns):
            raise ValueError(f"{path} 缺列: {sorted(required - set(df.columns))}")
        df["window_date"] = pd.to_datetime(df["window_date"])
        df["strategy"] = strategy
        df["coalition"] = "+".join(sorted(coalition, key=int)) or "none"
        frames.append(df)
        hashes[str(path.resolve())] = sha256(path)
    all_df = pd.concat(frames, ignore_index=True)
    counts = all_df.groupby("strategy")["window_date"].nunique()
    if counts.nunique() ≠ 1:
        raise ValueError(f"八种方案日期数不一致: {counts.to_dict()}")
    pivot = all_df.pivot(index="window_date", columns="strategy", values="cash_cost_yuan")
    if pivot.isna().any().any() or len(pivot.columns) ≠ 8:
        raise ValueError("八种方案不能按日期完整配对")
    return all_df.sort_values(["window_date", "strategy"]), hashes

def cost_map(pivot: pd.DataFrame, indices: np.ndarray | None = None) → dict[frozenset[str], float]:
    x = pivot if indices is None else pivot.iloc[indices]
    return {coalition: float(x[strategy].sum()) for coalition, strategy in STRATEGIES.items()}

def shapley(costs: dict[frozenset[str], float]) → dict[str, float]:
    """以费用下降为收益; 返回各更新时刻对总节省的Shapley分摊。"""
    out = {p: 0.0 for p in PLAYERS}
    perms = list(permutations(PLAYERS))
    for order in perms:
        joined = frozenset()
        for p in order:
            new = joined | {p}
            out[p] += costs[joined] - costs[new]
            joined = new
    return {p: v / len(perms) for p, v in out.items()}

def marginal_table(costs: dict[frozenset[str], float]) → pd.DataFrame:
    rows = []
    for p in PLAYERS:
        others = [x for x in PLAYERS if x ≠ p]
        for n in range(3):
            for context_tuple in combinations(others, n):
                context = frozenset(context_tuple)
                rows.append({
                    "新增时刻": f"{p}:00", "已有时刻": "+".join(sorted(context, key=int)) or "无",
                    "边际节省_元": costs[context] - costs[context | {p}],
                })
    return pd.DataFrame(rows)

def interaction_table(costs: dict[frozenset[str], float]) → pd.DataFrame:
    rows = []
    base = costs[frozenset()]
    for a, b in combinations(PLAYERS, 2):
        ca, cb, cab = costs[frozenset({a})], costs[frozenset({b})], costs[frozenset({a, b})]
        avg_benchmark_gain = (ca + cb) / 2 - cab
        redundancy = (base - ca) + (base - cb) - (base - cab)

```

```

        rows.append({
            "组合": f"{a}:{00}+{b}:{00}", "单时刻费用均值_元": (ca + cb) / 2,
            "联合方案费用_元": cab, "联合相对单方案均值节省_元": avg_benchmark_gain,
            "加法冗余_元": redundancy,
            "解释": "边际收益重叠（替代）" if redundancy > 0 else "正协同（互补）",
        })
    return pd.DataFrame(rows)

def moving_block_indices(n: int, block: int, rng: np.random.Generator) → np.ndarray:
    if block < 1 or block > n:
        raise ValueError(f"非法区块长度 {block}, n={n}")
    starts = rng.integers(0, n - block + 1, size=math.ceil(n / block))
    return np.concatenate([np.arange(s, s + block) for s in starts]):n]

def bootstrap(pivot: pd.DataFrame, reps: int, block: int, seed: int) → pd.DataFrame:
    rng = np.random.default_rng(seed + block * 1009)
    rows = []
    for r in range(reps):
        idx = moving_block_indices(len(pivot), block, rng)
        costs = cost_map(pivot, idx)
        phi = shapley(costs)
        row = {
            "replicate": r, "block_days": block,
            "all_vs_h0_saving_yuan_per_day": (costs[frozenset()] - costs[frozenset(PLAYERS)]) /
len(idx),
            "add6_given_12_18_yuan_per_day": (costs[frozenset({"12", "18"})] -
costs[frozenset(PLAYERS)]) / len(idx),
            "shapley_6_yuan_per_day": phi["6"] / len(idx),
            "shapley_12_yuan_per_day": phi["12"] / len(idx),
            "shapley_18_yuan_per_day": phi["18"] / len(idx),
            "shapley_12_minus_18_yuan_per_day": (phi["12"] - phi["18"]) / len(idx),
        }
        rows.append(row)
    return pd.DataFrame(rows)

def ci_table(boot: pd.DataFrame) → pd.DataFrame:
    names = {
        "all_vs_h0_saving_yuan_per_day": "全时刻相对仅0点的日均节省",
        "add6_given_12_18_yuan_per_day": "已有12/18点后增加6点的日均节省",
        "shapley_6_yuan_per_day": "6点Shapley日均贡献",
        "shapley_12_yuan_per_day": "12点Shapley日均贡献",
        "shapley_18_yuan_per_day": "18点Shapley日均贡献",
        "shapley_12_minus_18_yuan_per_day": "12点减18点Shapley日均贡献",
    }
    rows = []
    for block, g in boot.groupby("block_days"):
        for col, label in names.items():
            q = g[col].quantile([0.025, 0.5, 0.975])
            rows.append({"区块长度_日": block, "指标": label, "均值": g[col].mean(),
                "中位数": q.loc[0.5], "CI下限": q.loc[0.025], "CI上限": q.loc[0.975],
                "区间跨0": bool(q.loc[0.025] ≤ 0 ≤ q.loc[0.975])})
    return pd.DataFrame(rows)

def setup_plot_style() → None:
    candidates = ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC", "Arial Unicode MS"]
    available = {f.name for f in mpl.font_manager.fontManager.ttflist}
    mpl.rcParams.update({
        "font.family": "sans-serif", "font.sans-serif": [x for x in candidates if x in available] +
["DejaVu Sans"],
        "axes.unicode_minus": False, "font.size": 9, "axes.labelsize": 9, "axes.titlesize": 10,
        "xtick.labelsize": 8, "ytick.labelsize": 8, "legend.fontsize": 8,
        "svg.fonttype": "none", "pdf.fonttype": 42, "figure.dpi": 120, "savefig.dpi": 320,
    })

```

```

})

def savefig(fig: plt.Figure, figures: Path, stem: str) → None:
    figures.mkdir(parents=True, exist_ok=True)
    for ext in ("png", "svg"):
        fig.savefig(figures / f"{stem}.{ext}", bbox_inches="tight", dpi=320 if ext == "png" else None)
    preview_dir = figures / "previews"
    preview_dir.mkdir(parents=True, exist_ok=True)
    gray = preview_dir / f"{stem}_gray.png"
    color_preview = preview_dir / f"{stem}_preview_tmp.png"
    fig.savefig(color_preview, bbox_inches="tight", dpi=160)
    with Image.open(color_preview) as im:
        ImageOps.grayscale(im).save(gray, dpi=(160, 160))
    color_preview.unlink()
    plt.close(fig)

def make_figures(all_df: pd.DataFrame, pivot_test: pd.DataFrame, costs: dict, marg: pd.DataFrame,
    inter: pd.DataFrame, shap: dict, boot: pd.DataFrame, ci: pd.DataFrame, figures: Path) → None:
    setup_plot_style()
    order = list(STRATEGIES.values())
    labels = [LABELS[x] for x in order]

    # raw 1: 八方案日费用分布
    fig, ax = plt.subplots(figsize=(7.2, 4.2), constrained_layout=True)
    vals = [all_df.loc[all_df.strategy == s, "cash_cost_yuan"].to_numpy() / 1000 for s in order]
    ax.boxplot(vals, tick_labels=labels, showfliers=False, patch_artist=True,
        boxprops={"facecolor": "#D9EAF7", "edgecolor": "#3B6D8C"}, medianprops={"color":
"#D55E00"})
    ax.set_ylabel("单合同现金费用 (千元)"); ax.set_title("八种预报组合的日费用分布")
    ax.tick_params(axis="x", rotation=25); ax.grid(axis="y", alpha=.22)
    savefig(fig, figures, "raw_q3_01_daily_cost_distribution")

    # raw 2: 月度相对H0节省热力图
    tmp = all_df.copy(); tmp["month"] = tmp.window_date.dt.to_period("M").astype(str)
    mon = tmp.groupby(["month", "strategy"]).cash_cost_yuan.sum().unstack()
    saving = mon["H0"].to_numpy()[ :, None] - mon[order].to_numpy()
    fig, ax = plt.subplots(figsize=(7.2, 4.0), constrained_layout=True)
    im = ax.imshow(saving / 1000, aspect="auto", cmap="viridis")
    ax.set_xticks(range(8), labels, rotation=25, ha="right"); ax.set_yticks(range(len(mon)), mon.index)
    ax.set_title("各月相对仅0点方案的费用节省"); ax.set_ylabel("合同月份")
    cb = fig.colorbar(im, ax=ax); cb.set_label("节省 (千元)")
    savefig(fig, figures, "raw_q3_02_monthly_saving_heatmap")

    # raw 3: 全时刻相对H0的逐日差异及7日均线
    d = (pivot_test["H0"] - pivot_test["H0_6_12_18"]) / 1000
    fig, ax = plt.subplots(figsize=(7.2, 3.6), constrained_layout=True)
    ax.axhline(0, color="#666666", lw=.8); ax.plot(d.index, d, color="#9ECAE1", lw=.6, label="逐日")
    ax.plot(d.index, d.rolling(7, min_periods=1).mean(), color="#0072B2", lw=1.5, label="7日移动平均")
    ax.set_ylabel("相对H0节省 (千元/日)"); ax.set_title("全时刻方案逐日经济收益"); ax.legend(frameon=False)
    savefig(fig, figures, "raw_q3_03_daily_saving_timeseries")

    # process 1: 联盟价值
    coalition_savings = [(LABELS[STRATEGIES[c]], (costs[frozenset()] - costs[c]) / 1000) for c in STRATEGIES]
    coalition_savings.sort(key=lambda x: x[1])
    fig, ax = plt.subplots(figsize=(6.6, 4.0), constrained_layout=True)
    ax.barh([x[0] for x in coalition_savings], [x[1] for x in coalition_savings], color="#56B4E9")
    ax.set_xlabel("相对仅0点方案节省 (千元)"); ax.set_title("不同预报联盟的经济价值"); ax.grid(axis="x",
alpha=.2)
    savefig(fig, figures, "process_q3_01_coalition_value")

    # process 2: 边际收益矩阵
    contexts = ["无", "6", "12", "18", "6+12", "6+18", "12+18"]

```

```

mat = np.full((3, len(contexts)), np.nan)
for i, p in enumerate(PLAYERS):
    for _, row in marg[marg["新增时刻"] == f"{p}:00"].iterrows():
        mat[i, contexts.index(row["已有时刻"])] = row["边际节省_元"] / 1000
fig, ax = plt.subplots(figsize=(7.2, 2.8), constrained_layout=True)
masked = np.ma.masked_invalid(mat); im = ax.imshow(masked, cmap="YlGnBu", aspect="auto")
ax.set_xticks(range(len(contexts)), contexts); ax.set_yticks(range(3), [f"新增{p}点" for p in PLAYERS])
for i in range(3):
    for j in range(len(contexts)):
        if np.isfinite(mat[i, j]): ax.text(j, i, f"{mat[i,j]:.1f}", ha="center", va="center",
fontsize=7)
cb = fig.colorbar(im, ax=ax); cb.set_label("边际节省 (千元)")
ax.set_xlabel("已有补充时刻"); ax.set_title("新增预报在不同信息背景下的边际收益")
savefig(fig, figures, "process_q3_02_marginal_benefit_heatmap")

# process 3: 用户指定的均值基准与标准加法冗余
fig, ax = plt.subplots(figsize=(6.5, 3.6), constrained_layout=True)
x = np.arange(len(inter)); w = .36
ax.bar(x-w/2, inter["联合相对单方案均值节省_元"]/1000, w, label="相对两个单方案费用均值的节省",
color="#0072B2")
ax.bar(x+w/2, inter["加法冗余_元"]/1000, w, label="单项收益的加法冗余", color="#E69F00")
ax.set_xticks(x, inter["组合"]); ax.set_ylabel("金额 (千元)"); ax.set_title("不同预报时刻的组合效应")
ax.legend(frameon=False); ax.grid(axis="y", alpha=.2)
savefig(fig, figures, "process_q3_03_combination_effect")

# result 1: Shapley点估计 + 7日CI
ci7 = ci[(ci["区块长度_日"] == 7) & ci["指标"].str.contains("Shapley日均贡献")].copy()
point = np.array([shap[p] / len(pivot_test) for p in PLAYERS])
lo = ci7.set_index("指标").loc[[f"{p}点Shapley日均贡献" for p in PLAYERS], "CI下限"].to_numpy()
hi = ci7.set_index("指标").loc[[f"{p}点Shapley日均贡献" for p in PLAYERS], "CI上限"].to_numpy()
fig, ax = plt.subplots(figsize=(5.5, 3.7), constrained_layout=True)
ax.bar(PLAYERS, point, color=[COLORS[p] for p in PLAYERS], alpha=.85)
ax.errorbar(range(3), point, yerr=np.vstack([point-lo, hi-point]), fmt="none", color="black",
capsize=4)
ax.set_xlabel("补充预报时刻"); ax.set_ylabel("Shapley贡献 (元/日)"); ax.set_title("各时刻经济贡献分解 (测试期)")
ax.grid(axis="y", alpha=.2)
savefig(fig, figures, "result_q3_01_shapley_contribution")

# result 2: Bootstrap森林图
metrics = ["全时刻相对仅0点的日均节省", "已有12/18点后增加6点的日均节省", "12点减18点Shapley日均贡献"]
c7 = ci[(ci["区块长度_日"] == 7) & ci["指标"].isin(metrics)].set_index("指标").loc[metrics]
y = np.arange(len(metrics))[:-1]
fig, ax = plt.subplots(figsize=(7.0, 3.5), constrained_layout=True)
ax.axvline(0, color="#555555", lw=.8)
ax.errorbar(c7["中位数"], y, xerr=np.vstack([c7["中位数"]-c7["CI下限"], c7["CI上限"]-c7["中位数"]]),
fmt="o", color="#0072B2", capsize=4)
ax.set_yticks(y, metrics); ax.set_xlabel("日均费用差 (元/日, 正值表示左述方案更优)")
ax.set_title("7日配对移动区块Bootstrap的95%区间")
savefig(fig, figures, "result_q3_02_bootstrap_forest")

# result 3: 数据获取成本阈值
vals = point; orderp = np.argsort(vals)
fig, ax = plt.subplots(figsize=(5.8, 3.6), constrained_layout=True)
ax.barh(np.array(PLAYERS)[orderp], vals[orderp], color=[COLORS[PLAYERS[i]] for i in orderp])
for j, i in enumerate(orderp): ax.text(vals[i] + max(vals)*.015, j, f"{vals[i]:.0f}元/日",
va="center", fontsize=8)
ax.set_xlabel("补充数据的平均经济价值上限 (元/日)"); ax.set_ylabel("预报时刻")
ax.set_title("基于Shapley价值的数据获取成本阈值"); ax.grid(axis="x", alpha=.2)
savefig(fig, figures, "result_q3_03_data_cost_threshold")

def fmt_money(x: float) -> str:
    return f"{x:,.2f}"

```

```

def dataframe_to_markdown(df: pd.DataFrame) → str:
    """不依赖可选 tabulate 包的轻量 Markdown 表格导出。"""
    def cell(v: object) → str:
        if isinstance(v, (float, np.floating)):
            return f"{float(v):.2f}"
        return str(v).replace("|", "\\|")
    header = "|" + "|".join(map(str, df.columns)) + "|"
    separator = "|" + "|".join("----" for _ in df.columns) + "|"
    rows = ["|" + "|".join(cell(v) for v in row) + "|" for row in df.itertuples(index=False, name=None)]
    return "\n".join([header, separator, *rows])

def write_report(out: Path, costs_full: dict, costs_test: dict, shap_full: dict, shap_test: dict,
                marg: pd.DataFrame, inter: pd.DataFrame, ci: pd.DataFrame, n_test: int) → None:
    total = costs_test[frozenset()] - costs_test[frozenset(PLAYERS)]
    lines = [
        "# 问题三补充预报价值研究", "",
        "## 1. 研究口径", "",
        f"以6:00、12:00和18:00是否启用为三个二元因素，八种方案构成完整的 $2^3$ 组合。主结论采用2025-02-25起的冻结测试期（{n_test}个合同），全年334个合同仅作回溯稳健性对照。每日费用按合同起始日配对。",
        "移动区块Bootstrap对已经连续运行得到的逐日费用差重采样，并未重新求解被抽中的SOC轨迹，因此它用于衡量时间区段上的经验稳定性，不等同于重新模拟新的年度路径。", "",
        "## 2. Shapley经济贡献", "",
        f"全时刻方案在测试期相对仅0点方案共节省 **{fmt_money(total)}元**。Shapley值将该总收益按每个时刻在6种加入顺序中的平均边际贡献进行唯一分摊：", "",
        "|时刻|测试期贡献（元）|占比|全年贡献（元）|", "|---|---|---|---|",
    ]
    for p in PLAYERS:
        lines.append(f"|{p}:00|{fmt_money(shap_test[p])}|{shap_test[p]/total:.2%}|{fmt_money(shap_full[p])}|")
    lines += [
        "", "12点贡献最大，18点次之，6点最小；三者贡献均为正。若只能补充一个时刻，现有证据优先支持12点。", "",
        "![Shapley贡献](figures/result_q3_01_shapley_contribution.png)", "", "## 3. 增加预报的边际效益", "",
        "边际效益取决于已经启用了哪些时刻，不能只报告单独加入的结果。", "",
        dataframe_to_markdown(marg), "",
        "所有情境下新增时刻的边际节省均为正，但随着其他时刻已经启用，新增价值普遍下降，说明信息之间存在覆盖。",
    ],
    [
        "![边际效益](figures/process_q3_02_marginal_benefit_heatmap.png)", "", "## 4. 组合效应", "",
        "按题意同时报告两种口径：第一种比较两个单时刻方案费用的平均值与联合方案；第二种使用标准的加法收益分解。‘加法冗余’为正表示两条单项收益存在重叠，即替代关系，而不是负面作用。", "",
        dataframe_to_markdown(inter), "",
        "三个二时刻联合方案均优于对应两个单时刻方案的费用均值；与此同时，加法冗余均为正，表明这些时刻总体属于部分替代而非超加性互补。其中12点与18点的收益重叠最大。", "",
        "![组合效应](figures/process_q3_03_combination_effect.png)", "", "## 5. 配对移动区块Bootstrap", "",
        "同一次重采样对八种方案使用完全相同的连续日期区块，保留同日可比性和一部分跨日相关。主结果使用7日区块以保留周周期，14日区块用于稳健性复核；每种长度重复5000次。", "",
        dataframe_to_markdown(ci), "",
        "若95%区间不跨0，可表述为该方向在不同连续时间区段的重采样中较稳定；本文不把它夸大为独立同分布假设下的传统显著性检验。", "",
        "![Bootstrap区间](figures/result_q3_02_bootstrap_forest.png)", "", "## 6. 对开放性问题的回答",
    ],
    [
        "",
        "在不计补充数据获取成本时，三个时刻全部启用的测试期总费用最低，且三个时刻Shapley贡献均为正，因此补充日内光伏预报具有经济价值。优先级为12点、18点、6点。",
        "是否实际增设某个数据源还应比较其获取、通信和维护成本与相应边际收益。Shapley日均贡献可以作为长期平均成本阈值；若系统已有12点和18点接口，是否再增加6点则应使用“已有12/18点后增加6点”的条件边际收益，而不是6点的独立贡献。", "",
        "![成本阈值](figures/result_q3_03_data_cost_threshold.png)", "", "## 7. 图表索引", "",
        "其余数据分布、月度稳定性、逐日差异、联盟价值图均位于 `figures/`。PNG用于论文预览，SVG保留可编辑矢量文字，`*_gray.png`用于灰度可读性检查。", "",
    ]
    (out / "问题三_补充预报价值分析.md").write_text("\n".join(lines), encoding="utf-8")

def main() → None:
    args = parse_args(); out = args.output_dir.resolve(); results = out / "results"; figures = out / "figures"

```



```

results.mkdir(parents=True, exist_ok=True)
all_df, hashes = load_daily(args.input_dir.resolve())
pivot_full = all_df.pivot(index="window_date", columns="strategy",
values="cash_cost_yuan").sort_index()
pivot_test = pivot_full.loc[pd.Timestamp(args.test_start):]
if len(pivot_test) == 0: raise ValueError("测试期为空")
costs_full, costs_test = cost_map(pivot_full), cost_map(pivot_test)
shap_full, shap_test = shapley(costs_full), shapley(costs_test)
if not np.isclose(sum(shap_test.values()), costs_test[frozenset()] - costs_test[frozenset(PLAYERS)]),
atol=1e-6):
    raise AssertionError("Shapley效率公理校验失败")
marg = marginal_table(costs_test); inter = interaction_table(costs_test)
if args.smoke:
    smoke = {"status": "PASS", "n_full": len(pivot_full), "n_test": len(pivot_test), "shapley_test":
shap_test,
            "shapley_sum": sum(shap_test.values()), "total_saving": costs_test[frozenset()] -
costs_test[frozenset(PLAYERS)]}
    (results / "smoke_result.json").write_text(json.dumps(smoke, ensure_ascii=False, indent=2),
encoding="utf-8")
    print(json.dumps(smoke, ensure_ascii=False, indent=2)); return
boots = [bootstrap(pivot_test, args.bootstrap, b, args.seed) for b in args.blocks]
boot = pd.concat(boots, ignore_index=True); ci = ci_table(boot)
pd.DataFrame([{"时刻": f"{p}:00", "测试期Shapley_元": shap_test[p], "全年Shapley_元": shap_full[p],
               "测试期占比": shap_test[p]/sum(shap_test.values())} for p in PLAYERS]).to_csv(results /
"shapley_value.csv", index=False, encoding="utf-8-sig")
marg.to_csv(results / "marginal_benefit.csv", index=False, encoding="utf-8-sig")
inter.to_csv(results / "combination_effect.csv", index=False, encoding="utf-8-sig")
boot.to_csv(results / "bootstrap_replicates.csv", index=False, encoding="utf-8-sig")
ci.to_csv(results / "bootstrap_ci.csv", index=False, encoding="utf-8-sig")
make_figures(all_df, pivot_test, costs_test, marg, inter, shap_test, boot, ci, figures)
write_report(out, costs_full, costs_test, shap_full, shap_test, marg, inter, ci, len(pivot_test))
manifest = {"python": platform.python_version(), "numpy": np.__version__, "pandas": pd.__version__,
            "matplotlib": mpl.__version__, "seed": args.seed, "bootstrap_repetitions":
args.bootstrap,
            "block_lengths_days": args.blocks, "test_start": args.test_start, "input_sha256":
hashes,
            "command": "F:\\miniconda3\\envs\\fond\\python.exe 附件
\\question3\\judge\\analyze_forecast_value.py"}
(results / "复现清单.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2),
encoding="utf-8")
print(f"完成: {out}")

if __name__ == "__main__":
    main()

```

39. question3/planning/code/build_q3_method_doc.py

源文件 SHA-256: fb951bf80b70cd58cee6a7be27479c4215c595f65a4cb842ac1926fb4b98ea67 ; 代码行数: 165。

```

"""Prepare the Chinese reference DOCX and presentation-only Markdown copy.

The source method text stays intact. Pandoc/equations.py performs the final
conversion so all mathematical expressions become native editable OMML.
"""

from pathlib import Path
import re
import json
import hashlib

from docx import Document
from docx.enum.style import WD_STYLE_TYPE
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.shared import Cm, Pt, RGBColor
from docx.oxml import OxmlElement
from docx.oxml.ns import qn

ROOT=Path(__file__).resolve().parents[1]
OUT=ROOT/'notes/docx_build'

def font(style, east='Songti SC', western='Times New Roman', size=11, bold=False):
    style.font.name=western; style.font.size=Pt(size); style.font.bold=bold
    rpr=style.element.get_or_add_rPr(); rf=rpr.find(qn('w:rFonts'))
    if rf is None: rf=OxmlElement('w:rFonts'); rpr.insert(0, rf)
    for attr, value in [('ascii', western), ('hAnsi', western), ('eastAsia', east),
                        ('cs', western)]: rf.set(qn('w:'+attr), value)

def ensure_style(doc, name, typ=WD_STYLE_TYPE.PARAGRAPH):
    return doc.styles[name] if name in doc.styles else doc.styles.add_style(name, typ)

def reference(path):
    doc=Document(); section=doc.sections[0]
    section.page_width=Cm(21); section.page_height=Cm(29.7)
    section.top_margin=Cm(1.9); section.bottom_margin=Cm(1.8)
    section.left_margin=section.right_margin=Cm(2.0)
    section.header_distance=Cm(.75); section.footer_distance=Cm(.8)
    for name in ('Normal', 'Body Text', 'First Paragraph', 'Compact'):
        s=ensure_style(doc, name); font(s)
        s.paragraph_format.line_spacing=1.25; s.paragraph_format.space_after=Pt(5)
        s.paragraph_format.widow_control=True
        s.paragraph_format.alignment=WD_ALIGN_PARAGRAPH.JUSTIFY
    for name, size in [('Title', 21), ('Subtitle', 11), ('Heading 1', 14), ('Heading 2', 12)]:
        s=ensure_style(doc, name); font(s, east='Heiti SC', size=size, bold=name!='Subtitle')
        s.font.color.rgb=RGBColor.from_string('18384A' if name!='Subtitle' else '63717A')
        s.paragraph_format.space_before=Pt(13 if name.startswith('Heading') else 6)
        s.paragraph_format.space_after=Pt(7)
        s.paragraph_format.keep_with_next=True
        s.paragraph_format.line_spacing=1.12
        if name in ('Title', 'Subtitle'): s.paragraph_format.alignment=WD_ALIGN_PARAGRAPH.CENTER
    for name in ('Source Code', 'Verbatim Char'):
        s=ensure_style(doc, name, WD_STYLE_TYPE.CHARACTER if name=='Verbatim Char' else
WD_STYLE_TYPE.PARAGRAPH)
        font(s, western='Courier New', size=8.5)
        if name=='Source Code':
            s.paragraph_format.line_spacing=1.05; s.paragraph_format.space_after=Pt(6)
            s.paragraph_format.left_indent=Cm(.15); s.paragraph_format.right_indent=Cm(.15)
    for name in ('Hyperlink', 'Footnote Text'):
        if name in doc.styles: font(doc.styles[name], size=10)
    table=ensure_style(doc, 'Table', WD_STYLE_TYPE.TABLE)
    font(table, size=9.5)
    ppr=OxmlElement('w:pPr')

```



```

spacing=OxmElement('w:spacing');spacing.set(qn('w:after'),'25');spacing.set(qn('w:line'),'250');spacing
.set(qn('w:lineRule'),'auto');ppr.append(spacing)
table.element.insert(list(table.element).index(table.element.find(qn('w:rPr'))),ppr)
tblpr=OxmElement('w:tblPr')
borders=OxmElement('w:tblBorders')
for name,val,sz,color in [('top','single','10','264B61'),('left','nil','0','FFFFFF'),
    ('bottom','single','10','264B61'),('right','nil','0','FFFFFF'),
    ('insideH','single','3','CBD7DD'),('insideV','nil','0','FFFFFF')]:
b=OxmElement('w:'+name);b.set(qn('w:val'),val);b.set(qn('w:sz'),sz);b.set(qn('w:color'),color);borders.
append(b)
tblpr.append(borders)
margins=OxmElement('w:tblCellMar')
for name,val in [('top','65'),('left','70'),('bottom','65'),('right','70')]:
    n=OxmElement('w:'+name);n.set(qn('w:w'),val);n.set(qn('w:type'),'dxa');margins.append(n)
tblpr.append(margins);table.element.append(tblpr)
trpr=OxmElement('w:trPr');trpr.append(OxmElement('w:cantSplit'));table.element.append(trpr)
first=OxmElement('w:tblStylePr');first.set(qn('w:type'),'firstRow')
rpr=OxmElement('w:rPr');rpr.append(OxmElement('w:b'));first.append(rpr)

tcpr=OxmElement('w:tcPr');shd=OxmElement('w:shd');shd.set(qn('w:fill'),'EAF0F4');shd.set(qn('w:val'),'
clear');tcpr.append(shd);first.append(tcpr)
table.element.append(first)
header=section.header.paragraphs[0]
header.alignment=WD_ALIGN_PARAGRAPH.RIGHT
run=header.add_run('第三问 / 模型与操作说明');run.font.name='Heiti
SC';run.font.size=Pt(9);run.font.color.rgb=RGBColor.from_string('687C88')
footer=section.footer.paragraphs[0];footer.alignment=WD_ALIGN_PARAGRAPH.CENTER
r=footer.add_run('- ');r.font.size=Pt(9)
field=OxmElement('w:fldSimple');field.set(qn('w:instr'),'PAGE');footer._p.append(field)
r=footer.add_run(' -');r.font.size=Pt(9)
settings=doc.settings.element
zoom=settings.find(qn('w:zoom'))
if zoom is None:zoom=OxmElement('w:zoom');settings.insert(0,zoom)
zoom.set(qn('w:percent'),'100')
math=settings.find(qn('m:mathPr'))
if math is None:math=OxmElement('m:mathPr');settings.append(math)
mf=math.find(qn('m:mathFont'))
if mf is None:mf=OxmElement('m:mathFont');math.append(mf)
mf.set(qn('m:val'),'STIX Two Math')
doc.add_paragraph('参考模板占位: 最终转换仅继承样式, 不包含此行。')
doc.save(path)

def main():
    OUT.mkdir(parents=True,exist_ok=True)
    src=ROOT/'notes/问题三_模型与复现说明.md'
    text=src.read_text(encoding='utf-8')
    text=text.split('\n',1)[1].lstrip()
    text=re.sub(r'^### ', '## ',text,flags=re.M)
    # Main headings were ##; avoid shifting the just-created subheadings by
    # matching their original numbered section pattern only.
    text=re.sub(r'^## (\d+\.[^\n]+)$',r'# \1',text,flags=re.M)
    text=text.replace(r'\[', '$$').replace(r'\]', '$$').replace(r'\(', '$').replace(r'\)', '$')
    text=text.replace('I_{D,t}=[D\ 00{:}10+10t\text{分钟},\ D\ 00{:}10+10(t+1)\text{分钟}].',
        'I_{D,t}=[\tau_D+t\Delta t,\tau_D+(t+1)\Delta t],\quad \Delta t=10\mathrm{min}.')
    text=text.replace('因此0、6、12、18时版本', '式中$\tau_D$表示D日00:10。因此0、6、12、18时版本')
    # Long displays are split without changing any coefficients or equations.
    old=r'\boxed{J=\sum_t\left[p_{tg_t+1.5p_{ta_t}-
p_{tb_t+0.5p_{tb_t+5p_{te_t}\right]}+\n+r'=\sum_t\left[p_{tg_t+1.5p_{ta_t}-0.5p_{tb_t+5p_{te_t}\right]}.}
new=r'J=\sum_t[p_{tg_t+1.5p_{ta_t}-
p_{tb_t+0.5p_{tb_t+5p_{te_t}}]+\n+\$n\$n\$n+r'J=\sum_t[p_{tg_t+1.5p_{ta_t}-0.5p_{tb_t+5p_{te_t}}].'
    text=text.replace(old,new)
    text=text.replace(r'\le c_{t,s}\le\frac{5000}{6}u_{t,s},\quad+\n+r'\le d_{t,s}\le\frac{5000}{
6}(1-u_{t,s}),\quad u_{t,s}\in\{0,1\}',

```

```

r'\0\le c_{t,s}\le\frac{5000}{6}u_{t,s}.'+'n$$\n\n$$\n'+r'\0\le d_{t,s}\le\frac{5000}{6}(1-
u_{t,s}),\quad u_{t,s}\in\{0,1\}.'')
text=text.replace(r'\$\text{历史窗口结束时刻}\le\text{当前决策时刻}$','“历史窗口结束时刻不晚于当前决策时刻”')
# Explicit example label prevents the command from being read as a winner.
text=text.replace('局部因果执行对照的命令如下，目录H0_6_12_18可换为实际要检查的主策略：',
'以下H0_6_12_18为操作示例，最终选择结果已另册提供；使用时将目录替换为实际要检查的主策略：')
text=text.replace('q3_sensitivity.py`预设的有界设计','q3_sensitivity.py`预设的有界设计')
# Pandoc honors proportional pipe-table widths when the separator exceeds
# its default text width; allocate room to meanings and long descriptions.
text=text.replace('| 代码 | 启用时刻 | 日内正式调整块 |\n|_|_|_|_|',
'| 代码 | 启用时刻 | 日内正式调整块 |\n|_|_|_|_|')
text=text.replace('| 符号 | 含义 | 单位 |\n|_|_|_|_|',
'| 符号 | 含义 | 单位 |\n|_|_|_|_|')
text=text.replace('| 参数 | 主设置 | 性质 |\n|_|_|_|_|',
'| 参数 | 主设置 | 性质 |\n|_|_|_|_|')
text=text.replace('| 试验 | 日期范围与初态 | 变体 | 回答的问题 |\n|_|_|_|_|',
'| 试验 | 日期范围与初态 | 变体 | 回答的问题 |\n|_|_|_|_|')
text=text.replace('# 14. 资料来源',
'''{=openxml}<n<w:p><w:r><w:br w:type="page"/></w:r></w:p>\n'''\n\n# 14. 资料来源')
# Narrower operational code for A4. Each continuation is runnable shell.
lines=text.splitlines();code=False;wrapped=[]
for line in lines:
    if line.startswith('``'):code=not code;wrapped.append(line);continue
    if code and line.startswith('python ') and len(line)>86:
        tokens=line.split(' ');parts=[];cur=''
        for token in tokens:
            if len(cur)+len(token)+1>76 and cur:
                parts.append(cur+' \\\');cur=' '+token
            else:cur+=(' ' if cur else '')+token
        if cur:parts.append(cur)
        wrapped.extend(parts)
    else:wrapped.append(line)
meta='---\ntitle: "第三问：模型与操作说明"\nsubtitle: "方法章节 · 交易时序 · 可复现验证 | 费用及选型结果另
册"\n---\n\n'
rendered=meta+'\n'.join(wrapped)+'\n'
(OUT/'方法说明_排版源.md').write_text(rendered,encoding='utf-8')
reference(OUT/'中文A4参考模板.docx')
manifest=
{'original_markdown':str(src),'original_sha256':hashlib.sha256(src.read_bytes()).hexdigest(),
'presentation_markdown':str(OUT/'方法说明_排版源.md'),
'changes':['reference styles, heading levels and page geometry','equivalent display line
breaks',
'short equivalent interval notation with its definition','shell continuation
wrapping',
'explicit H0_6_12_18 example label','proportional table widths and non-splitting
table rows'],
'math_method_changed':False,'document_scope':'Q3 methods chapter, not a complete contest paper',
'fonts':{'body-east_asian':'Songti SC','headings-east_asian':'Heiti SC','latin':'Times New
Roman','math':'STIX Two Math'}}
(OUT/'排版来源.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8')
print(str(OUT/'方法说明_排版源.md'))

if __name__=='__main__':main()

```

40. question3/planning/code/q3_audit.py

源文件 SHA-256: d6261aee3b64f787d8f14b554de5a2d95ea648765bc5667ee4457b03764c5991 ; 代码行数: 495。

```
"""Independent Q3 ledger, physical and information-availability checks.
```

```
This module reads CSV/JSON artifacts; it never invokes the optimizer or edits
the source ledger. Flow columns are kwh and input/actual power columns kW.
Use ``audit_run(strategy_directory, input_data)`` after a complete run.
"""
```

```
from __future__ import annotations
```

```
import argparse
import csv
import datetime as dt
import hashlib
import json
from pathlib import Path
import re
```

```
import numpy as np
```

```
FLOW_TOL = 1e-5
```

```
FEE_TOL = 1e-4
```

```
START = {0: 0, 6: 35, 12: 71, 18: 107}
```

```
HOURS = (0, 6, 12, 18)
```

```
REQUIRED = (
```

```
    'strategy', 'window_date', 'interval_start', 'interval_end',
    'forecast_issue_time', 'price_yuan_per_kwh',
    'load_forecast_kw', 'pv_forecast_kw', 'load_actual_kw', 'pv_actual_kw',
    'actual_estimated', 'g_kwh', 'q_kwh', 'increase_kwh', 'decrease_kwh',
    'charge_kwh', 'discharge_kwh', 'emergency_kwh', 'unused_paid_kwh',
    'curtailed_pv_kwh', 'soc_start_kwh', 'soc_end_kwh', 'plan_cost_yuan',
    'increase_cost_yuan', 'refund_yuan', 'penalty_yuan',
    'adjustment_net_cost_yuan', 'emergency_cost_yuan', 'cash_cost_yuan',
)
```

```
TEXT = {'strategy', 'window_date', 'interval_start', 'interval_end',
        'forecast_issue_time', 'actual_estimated'}
```

```
NUMERIC = tuple(x for x in REQUIRED if x not in TEXT)
```

```
def _date(value):
```

```
    if isinstance(value, dt.datetime):
        return value.date()
    if isinstance(value, dt.date):
        return value
    return dt.date.fromisoformat(str(value)[:10])
```

```
def _time(value):
```

```
    if isinstance(value, dt.datetime):
        return value.replace(tzinfo=None)
    return dt.datetime.fromisoformat(str(value).replace('Z', '+00:00')).replace(tzinfo=None)
```

```
def _bool(value):
```

```
    if isinstance(value, (bool, np.bool_)):
        return bool(value)
    if str(value).lower().strip() in {'true', '1', 'yes'}:
        return True
    if str(value).lower().strip() in {'false', '0', 'no'}:
        return False
    raise ValueError(f'Invalid Boolean flag: {value!r}')
```

```
def _json(path):
```

```
    return json.loads(Path(path).read_text(encoding='utf-8'))
```

```

def _sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

class _Checks:
    def __init__(self):
        self.items = []
        self.skipped = []

    def record(self, name, passed, **details):
        self.items.append({'name': name, 'passed': bool(passed), **details})

    def residual(self, name, values, tol=FLOW_TOL):
        a = np.asarray(values, dtype=float)
        maximum = float(np.max(np.abs(a))) if a.size else 0.0
        self.record(name, np.isfinite(a).all() and maximum ≤ tol,
                    max_abs=maximum, tolerance=tol,
                    failed_rows=np.flatnonzero(~np.isfinite(a) | (np.abs(a) > tol))[:8].tolist())

    def mask(self, name, mask):
        a = np.asarray(mask, dtype=bool)
        self.record(name, a.all(), failed_count=int((~a).sum()),
                    failed_rows=np.flatnonzero(~a)[:8].tolist())

def _hours(config, names):
    for key in names:
        if key in config:
            value = config[key]
            if isinstance(value, str):
                value = re.findall(r'\d+', value)
            try:
                hours = sorted({int(v) for v in value})
            except (TypeError, ValueError):
                return None
            return hours
    return None

def _config(run, event_obj):
    result = {}
    for name in ('config.json', 'summary.json', 'run_config.json'):
        path = run / name
        if path.exists():
            obj = _json(path)
            if isinstance(obj, dict):
                result.update(obj.get('config', obj))
    if isinstance(event_obj, dict):
        for key in ('config', 'parameters', 'strategy_config'):
            if isinstance(event_obj.get(key), dict):
                result.update(event_obj[key])
        for key in ('trade_hours', 'forecast_hours', 'enabled_hours', 'enabled_forecast_hours'):
            if key in event_obj:
                result[key] = event_obj[key]
    return result

def _events(obj):
    if isinstance(obj, list):
        return obj
    if isinstance(obj, dict):
        for key in ('events', 'event_log', 'records', 'decisions'):
            if isinstance(obj.get(key), list):
                return obj[key]
    return []

```

```

def _risk_records(event):
    """Locate availability metadata without depending on one nesting label."""
    out = []
    if not isinstance(event, dict):
        return out
    if 'history_indices' in event or 'history_latest_end' in event:
        out.append(event)
    for value in event.values():
        if isinstance(value, dict):
            out.extend(_risk_records(value))
    return out

def _load_input(input_data):
    if input_data is None:
        default = Path(__file__).resolve().parents[1] / 'data' / 'q3_inputs.npz'
        if not default.exists():
            return None
        input_data = default
    if isinstance(input_data, (str, Path)):
        with np.load(input_data, allow_pickle=False) as bundle:
            result = {k: bundle[k].copy() for k in bundle.files}
        metadata_path = Path(input_data).parent/'input_audit.json'
        if metadata_path.exists():
            metadata = _json(metadata_path)
            first = metadata.get('actual', {}).get('first_unexecuted_interval')
            if first is not None:
                result['first_unexecuted_interval'] = first
        return result
    return input_data

def _audit_events(check, events, config, inputs, window_dates):
    trades = _hours(config, ('trade_hours', 'enabled_hours', 'allowed_trade_hours'))
    forecasts = _hours(config, ('forecast_hours', 'enabled_forecast_hours',
                                'allowed_forecast_hours', 'enabled_hours'))
    # A runner may store the entire enabled set only once, as trade_hours.
    if forecasts is None and trades is not None and not config.get('old_forecast_only', False):
        forecasts = trades
    check.record('trade_configuration_present', trades is not None, hours=trades)
    check.record('forecast_configuration_present', forecasts is not None, hours=forecasts)
    if trades is not None:
        check.record('trade_configuration_valid', 0 in trades and set(trades) ≤ set(HOURS))
    if forecasts is not None:
        check.record('forecast_configuration_valid', 0 in forecasts and set(forecasts) ≤ set(HOURS))
    check.record('event_log_nonempty', bool(events), events=len(events))
    history_count = 0
    commit_count = 0
    covered = {}
    for j, event in enumerate(events):
        if not isinstance(event, dict):
            check.record(f'event_{j}_object', False)
            continue
        now_value = event.get('decision_at', event.get('decision_time', event.get('time')))
        risks = _risk_records(event)
        if now_value is None and risks:
            now_value = risks[0].get('decision_at')
        if now_value is None:
            check.record(f'event_{j}_decision_timestamp_present', False)
            continue
        now = _time(now_value)
        for risk in risks:
            history_count += 1
            issue = risk.get('issued_at', risk.get('forecast_issue_time'))

```

```

hour = risk.get('forecast_hour')
if issue is not None:
    issue_time = _time(issue)
    check.record(f'event_{j}_forecast_released', issue_time ≤ now)
    if hour is None:
        hour = issue_time.hour
if forecasts is not None and hour is not None:
    check.record(f'event_{j}_forecast_enabled', int(hour) in forecasts,
        forecast_hour=int(hour))
    check.record(f'event_{j}_forecast_hour_available',
        int(hour) ≤ now.hour and int(hour) in HOURS)
check.record(f'event_{j}_no_extra_margin', not risk.get('margin_added', False))
indices = [int(x) for x in risk.get('history_indices', [])]
draws = [int(x) for x in risk.get('draw_indices', [])]
check.record(f'event_{j}_draw_from_available_history',
    set(draws) ≤ set(indices) and len(draws) == len(set(draws)))
latest = risk.get('history_latest_end')
if latest is not None:
    check.record(f'event_{j}_latest_history_released', _time(latest) ≤ now,
        latest=str(latest), decision_at=now.isoformat())
if inputs is not None and indices:
    ends = inputs.get('window_end')
    if ends is None:
        check.record(f'event_{j}_input_window_ends_present', False)
    else:
        valid = all(0 ≤ i < len(ends) for i in indices)
        check.record(f'event_{j}_history_indices_valid', valid)
        if valid:
            check.record(f'event_{j}_all_history_released',
                all(_time(ends[i]) ≤ now for i in indices))
            if latest is not None:
                check.record(f'event_{j}_history_latest_consistent',
                    max(_time(ends[i]) for i in indices) == _time(latest))
        elif indices:
            check.skipped.append(f'event_{j}: all historical indices need input_data; latest
timestamp only checked')

mode = str(event.get('mode', event.get('kind', event.get('type', ''))).lower()
begin = event.get('commit_start', event.get('block_start'))
end = event.get('commit_end', event.get('block_end'))
if isinstance(event.get('commit_block'), (list, tuple)) and len(event['commit_block']) == 2:
    begin, end = event['commit_block']
if 'update' in mode or 'adjust' in mode or begin is not None:
    # Midnight commits all NEW g; only intraday q needs block validation.
    if now.hour == 0:
        continue
    commit_count += 1
    if trades is None:
        continue
    allowed = now.hour in trades and now.hour in START and now.minute == 0
    check.record(f'event_{j}_trade_enabled', allowed)
    if begin is None or end is None:
        check.record(f'event_{j}_commit_block_present', False)
        continue
    begin, end = int(begin), int(end)
    if allowed:
        expected_begin = START[now.hour]
        later = [START[h] for h in trades if h > now.hour and h in START]
        expected_end = min(later) if later else 144
        check.record(f'event_{j}_commit_block_correct',
            (begin, end) == (expected_begin, expected_end),
            actual=[begin, end], expected=[expected_begin, expected_end])
    day = str(event.get('window_date', now.date()))[:10]
    occupied = covered.setdefault(day, set())
    slots = set(range(begin, end))
    check.record(f'event_{j}_each_interval_adjusted_once', not (occupied & slots))

```

```

        occupied.update(slots)
    check.record('history_metadata_present', history_count > 0, records=history_count)
    if trades is not None:
        expected = len(window_dates) * (len(trades) - 1)
        check.record('all_scheduled_update_events_present', commit_count == expected,
                     actual=commit_count, expected=expected)
    return trades, forecasts

def _audit_startup(check, run, config, inputs, main_start, main_soc):
    path = run/'startup.csv'
    expected = int(config.get('start_day', 0)) == 0
    check.record('startup_presence', path.exists() == expected,
                 expected=expected, present=path.exists())
    if not path.exists():
        return 0.0
    with path.open(encoding='utf-8-sig', newline='') as stream:
        rows = list(csv.DictReader(stream))
    check.record('one_startup_interval', len(rows) == 1)
    if len(rows) != 1:
        return 0.0
    row = rows[0]
    values = {key: float(row[key]) for key in NUMERIC}
    t0, t1 = _time(row['interval_start']), _time(row['interval_end'])
    check.record('startup_immediately_precedes_main',
                 t1 == main_start and t1-t0 == dt.timedelta(minutes=10))
    p=values['price_yuan_per_kwh']; g=values['g_kwh']; q=values['q_kwh']
    e=values['emergency_kwh']; L=values['load_actual_kw']/6; G=values['pv_actual_kw']/6
    c=values['charge_kwh']; d=values['discharge_kwh']
    v=values['unused_paid_kwh']; r=values['curtailed_pv_kwh']
    check.residual('startup_idle_battery', [c,d])
    check.residual('startup_preserves_experiment_initial_soc',
                  [values['soc_start_kwh']-main_soc, values['soc_end_kwh']-main_soc])
    check.residual('startup_energy_balance', [q-v+G-r+d+e-L-c])
    check.residual('startup_contract_unchanged', [q-g, values['increase_kwh'], values['decrease_kwh']])
    check.residual('startup_emergency_deficit', [e-max(L-q-G,0)])
    check.residual('startup_cash', [values['cash_cost_yuan']-(p*g+5*p*e)], FEE_TOL)
    if inputs is not None:
        load0=float(np.asarray(inputs['load_f'])[0,0]);
        pv0=float(np.asarray(inputs['pv_unused_first_mean_kw'])[0])
        check.residual('startup_common_hold_forecast', [values['load_forecast_kw']-load0,
                                                         values['pv_forecast_kw']-pv0, g-max((load0-pv0)/6,0)])
        check.residual('startup_price', [p-float(np.asarray(inputs['price'])[-1])], FEE_TOL)
        # Full adapter dictionary contains observed midnight boundary metadata;
        # the compact NPZ does not, so the main trace source checks are separate.
        actual = inputs.get('first_unexecuted_interval')
        if actual is not None:
            check.residual('startup_actual_nodes_source',
                          [values['load_actual_kw']-actual['load_mean_kw'],
                           values['pv_actual_kw']-actual['pv_mean_kw']])
    return values['cash_cost_yuan']

def audit_run(run_dir, input_data=None):
    """Audit one strategy directory containing trace.csv and event_log.json.

    Returns a serializable dictionary; does not mutate input artifacts. Missing
    evidence is not silently marked as passing. Optional input_data is the
    adapter dictionary or its q3_inputs.npz archive.
    """
    run = Path(run_dir)
    trace_path, events_path = run / 'trace.csv', run / 'event_log.json'
    check = _Checks()
    for p in (trace_path, events_path):
        if not p.exists():
            return {'passed': False, 'run_directory': str(run),

```



```

        'errors': [{'Required artifact missing: {p.name}}']}
with trace_path.open(encoding='utf-8-sig', newline='') as stream:
    reader = csv.DictReader(stream)
    fields = reader.fieldnames or []
    missing = sorted(set(REQUIRED) - set(fields))
    if missing:
        return {'passed': False, 'run_directory': str(run),
                'errors': ['Missing trace columns: ' + ', '.join(missing)]}
    rows = list(reader)
if not rows:
    return {'passed': False, 'run_directory': str(run), 'errors': ['Empty trace']}
try:
    a = {key: np.asarray([float(row[key]) for row in rows]) for key in NUMERIC}
    starts = [_time(r['interval_start']) for r in rows]
    ends = [_time(r['interval_end']) for r in rows]
    days = [_date(r['window_date']) for r in rows]
    issues = [_time(r['forecast_issue_time']) for r in rows]
    estimated = np.asarray([bool(r['actual_estimated']) for r in rows])
except (KeyError, TypeError, ValueError) as exc:
    return {'passed': False, 'run_directory': str(run), 'errors': [str(exc)]}
event_obj = _json(events_path)
config = _config(run, event_obj)
inputs = _load_input(input_data)
window_dates = sorted(set(days))
check.record('single_strategy', len({r['strategy'] for r in rows}) == 1)
for key, val in a.items():
    check.mask('finite_' + key, np.isfinite(val))
nonnegative = ('price_yuan_per_kwh', 'load_forecast_kw', 'pv_forecast_kw',
               'load_actual_kw', 'pv_actual_kw', 'g_kwh', 'q_kwh', 'increase_kwh', 'decrease_kwh',
               'charge_kwh', 'discharge_kwh', 'emergency_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh')
for key in nonnegative:
    check.mask('nonnegative_' + key, a[key] ≥ -FLOW_TOL)
check.mask('positive_price', a['price_yuan_per_kwh'] > 0)
p = a['price_yuan_per_kwh']; g = a['g_kwh']; q = a['q_kwh']
inc = a['increase_kwh']; dec = a['decrease_kwh']
c = a['charge_kwh']; d = a['discharge_kwh']; e = a['emergency_kwh']
v = a['unused_paid_kwh']; r = a['curtailed_pv_kwh']
ss = a['soc_start_kwh']; se = a['soc_end_kwh']
load = a['load_actual_kw'] / 6.; pv = a['pv_actual_kw'] / 6.
check.residual('energy_balance_kwh', q-v+pv-r+d+e-load-c)
check.residual('battery_recurrence_kwh', se-ss-.9*c+d/.9)
check.residual('continuous_soc_kwh', ss[1:]-se[:-1])
check.mask('soc_start_bounds', (ss ≥ 1200-FLOW_TOL) & (ss ≤ 10800+FLOW_TOL))
check.mask('soc_end_bounds', (se ≥ 1200-FLOW_TOL) & (se ≤ 10800+FLOW_TOL))
check.mask('charge_power_limit', c ≤ 5000/6 + FLOW_TOL)
check.mask('discharge_power_limit', d ≤ 5000/6 + FLOW_TOL)
check.mask('charge_discharge_exclusion', ~((c > FLOW_TOL) & (d > FLOW_TOL)))
check.mask('emergency_does_not_charge', ~((c > FLOW_TOL) & (e > FLOW_TOL)))
check.mask('emergency_load_bound', e ≤ load + FLOW_TOL)
check.mask('curtailment_pv_bound', r ≤ pv + FLOW_TOL)
check.mask('unused_paid_bound', v ≤ q + FLOW_TOL)
surplus = np.maximum(q+pv-load, 0)
deficit = np.maximum(load-q-pv, 0)
check.mask('charging_only_from_ordinary_surplus', c ≤ surplus + FLOW_TOL)
check.mask('discharging_only_to_ordinary_deficit', d ≤ deficit + FLOW_TOL)
check.residual('emergency_deficit_after_discharge', e-deficit+d)
check.residual('unused_surplus_after_charging', v+r-surplus+c)
check.mask('cancel_original_bound', dec ≤ g + FLOW_TOL)
check.residual('adjusted_contract_identity', q-g-inc+dec)
check.residual('increase_direction', inc-np.maximum(q-g, 0))
check.residual('decrease_direction', dec-np.maximum(g-q, 0))
# refund_yuan is a positive refund AMOUNT; subtract it in the cash ledger.
expected_fees = {'plan_cost_yuan': p*g, 'increase_cost_yuan': 1.5*p*inc,
                 'refund_yuan': p*dec, 'penalty_yuan': .5*p*dec,
                 'adjustment_net_cost_yuan': 1.5*p*inc-.5*p*dec,
                 'emergency_cost_yuan': 5*p*e,

```



```

        'cash_cost_yuan':p*g+1.5*p*inc-.5*p*dec+5*p*e}
    for key, expected in expected_fees.items():
        check.residual('fee_' + key, a[key]-expected, FEE_TOL)
    check.residual('fee_components_sum', a['cash_cost_yuan']-
        (a['plan_cost_yuan']+a['increase_cost_yuan']-a['refund_yuan']+
        a['penalty_yuan']+a['emergency_cost_yuan']), FEE_TOL)
    check.mask('interval_duration_10min', [b-a == dt.timedelta(minutes=10) for a,b in zip(starts,ends)])
    check.mask('interval_sequence_continuous', [b == c for b,c in zip(ends,starts[1:])])
    check.mask('forecast_released_for_execution', [f ≤ t for f,t in zip(issues,starts)])
    slot = []
    for day in window_dates:
        ix = np.flatnonzero(np.asarray([x == day for x in days]))
        origin = dt.datetime.combine(day, dt.time(0,10))
        check.record(f'window_{day}_144_slots', len(ix) == 144)
        check.record(f'window_{day}_correct_targets', len(ix) == 144 and
            all(starts[i] == origin + dt.timedelta(minutes=10*j) for j,i in enumerate(ix)))
        check.residual(f'window_{day}_inventory_identity',
            [se[ix[-1]]-ss[ix[0]]-.9*c[ix].sum()+d[ix].sum()/0.9])
    for day,time in zip(days,starts):
        slot.append(int((time-dt.datetime.combine(day,dt.time(0,10))).total_seconds()/600))
    trades, forecasts = _audit_events(check, _events(event_obj), config, inputs, window_dates)
    if forecasts is not None and 0 in forecasts and set(forecasts) ≤ set(HOURS):
        check.mask('execution_uses_enabled_forecast', [f.hour in forecasts for f in issues])
        expected_issues = [dt.datetime.combine(day, dt.time(max(h for h in forecasts if START[h] ≤ k))]
            for day,k in zip(days,slot) if 0 ≤ k < 144]
        if len(expected_issues) == len(issues):
            check.mask('execution_uses_latest_enabled_vintage', [a == b for a,b in
zip(issues,expected_issues)])
        if trades is not None and set(trades) ≤ set(HOURS):
            first_update = min((START[h] for h in trades if h), default=144)
            before = np.asarray(slot) < first_update
            check.residual('no_adjustment_before_first_enabled_update', (q-g)[before])
    if inputs is not None:
        index = {_date(value): i for i,value in enumerate(inputs['dates'])}
        if all(day in index for day in days) and all(0 ≤ k < 144 for k in slot):
            ids = np.asarray([index[day] for day in days]); slots = np.asarray(slot)
            check.residual('input_actual_load_kw', a['load_actual_kw']-np.asarray(inputs['load_a'])
[ids,slots])
            check.residual('input_actual_pv_kw', a['pv_actual_kw']-np.asarray(inputs['pv_a'])
[ids,slots])
            check.residual('input_load_forecast_kw', a['load_forecast_kw']-np.asarray(inputs['load_f'])
[ids,slots])
            check.residual('input_prices', p-np.asarray(inputs['price'])[slots], FEE_TOL)
            check.mask('input_estimated_flags', estimated == np.asarray(inputs['actual_estimated'])
[ids,slots])
            pvkey = 'pv_linear_f' if config.get('pv_source') == 'linear' else 'pv_f'
            if pvkey in inputs and all(f.hour in HOURS for f in issues):
                vintage = np.asarray([HOURS.index(f.hour) for f in issues])
                check.residual('input_matching_pv_vintage_kw', a['pv_forecast_kw']-
                    np.asarray(inputs[pvkey])[ids,vintage,slots])
            else:
                check.record('trace_targets_present_in_input', False)
        else:
            check.skipped.append('Input identity and full history-index availability: input_data
unavailable')
        sums = {key: float(a[key].sum()) for key in (
            'g_kwh', 'q_kwh', 'increase_kwh', 'decrease_kwh', 'charge_kwh', 'discharge_kwh',
            'emergency_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh', *expected_fees.keys())}
        startup_cash = _audit_startup(check, run, config, inputs, starts[0], float(ss[0]))
        summary_path = run/'summary.json'
        if summary_path.exists():
            summary = _json(summary_path)
            for key, expected in {'cash_cost_yuan':sums['cash_cost_yuan'],
                'soc_start_kwh':float(ss[0]), 'soc_end_kwh':float(se[-1]),
                'original_plan_cost_yuan':sums['plan_cost_yuan'],
                'net_adjustment_cost_yuan':sums['adjustment_net_cost_yuan'],

```

```

        'emergency_cost_yuan':sums['emergency_cost_yuan'],
        'emergency_kwh':sums['emergency_kwh'], 'startup_cash_cost_yuan':startup_cash}.items():
    if key in summary:
        check.residual('summary_'+key, [float(summary[key])-expected], FEE_TOL)
    check.record('summary_row_window_counts', summary.get('intervals') == len(rows)
        and summary.get('windows') == len(window_dates))
    last_day = window_dates[-1]
    natural_cash = float(a['cash_cost_yuan'][np.asarray([x.date()≤last_day for x in
starts])]).sum()+startup_cash
    if 'natural_period_cash_cost_yuan' in summary:
        check.residual('summary_natural_period_cash',
            [float(summary['natural_period_cash_cost_yuan'])-natural_cash], FEE_TOL)
    return {'passed': all(item['passed'] for item in check.items) and not check.skipped,
        'run_directory': str(run.resolve()), 'strategy': rows[0]['strategy'],
        'rows': len(rows), 'windows': len(window_dates),
        'first_interval': starts[0].isoformat(), 'last_interval_end': ends[-1].isoformat(),
        'initial_soc_kwh':float(ss[0]), 'final_soc_kwh':float(se[-1]),
        'estimated_intervals':int(estimated.sum()), 'sums':sums,
        'checks':check.items, 'skipped':check.skipped,
        'failed_checks':[item for item in check.items if not item['passed']],
        'trace_sha256':sha(trace_path), 'events_sha256':_sha(events_path),
        'startup_cash_cost_yuan':startup_cash,
        'refund_convention':'refund_yuan is a positive amount subtracted from cash; penalty_yuan is
positive',
        'scope':'existing artifact audit only; supplied forecast generation causality not certified',
        'evidence_limit':'event blocks and final g/q are checked; immutable signed quantities
additionally rely on runner chronology tests because event logs do not archive each signed vector'}

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('run_dir', type=Path)
    parser.add_argument('--inputs', type=Path)
    parser.add_argument('--out', type=Path)
    args = parser.parse_args()
    result = audit_run(args.run_dir, args.inputs)
    output = args.out or args.run_dir/'independent_audit.json'
    output.write_text(json.dumps(result, ensure_ascii=False, indent=2),encoding='utf-8')
    print(json.dumps({'passed':result['passed'], 'out':str(output),
        'failed_checks':result.get('failed_checks',result.get('errors'))},
        ensure_ascii=False))
    raise SystemExit(0 if result['passed'] else 1)

if __name__ == '__main__':
    main()

```

41. question3/planning/code/q3_audit_batch.py

源文件 SHA-256: 4234daab665e0a44a27a967ba67fd8aacba7b1ba3dc3f83f1c2f17d9287a1392 ; 代码行数: 111。

```

"""Audit newly completed Q3 primary/ablation artifacts and frozen provenance.

Only completed summary.json files are read. --watch checks once per minute and
never changes a numerical model, input, contract, or execution trace.
"""
import argparse
import hashlib
import json
from pathlib import Path
import time

from q3_audit import audit_run

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def read(path):
    return json.loads(Path(path).read_text(encoding='utf-8'))

def write(path, obj):
    path=Path(path);path.parent.mkdir(parents=True,exist_ok=True)
    path.write_text(json.dumps(obj,ensure_ascii=False,indent=2),encoding='utf-8')

def verify_sources(project):
    project=Path(project);data=project/'data';provenance=project/'provenance'
    manifest=read(data/'source_manifest.json')
    checks=[]
    declared=read(data/'input_audit.json')['source_sha256']
    for source in manifest:
        filename=source['filename'];actual=sha(data/filename)
        checks.append({'file':filename,'actual_sha256':actual,
            'copy_manifest_sha256':source['sha256'],'input_audit_sha256':declared.get(filename),
            'passed':actual==source['sha256']==declared.get(filename)})
    core_manifest=read(provenance/'numerical_core_manifest.json')['files_sha256']
    snapshot=[]
    for filename,expected in core_manifest.items():
        frozen=sha(provenance/'numerical_core_used'/filename)
        current=sha(project/'code'/filename)
        snapshot.append({'file':filename,'manifest_sha256':expected,'snapshot_sha256':frozen,
            'current_sha256':current,'snapshot_matches_manifest':frozen==expected,
            'current_matches_snapshot':current==frozen})
    return {'passed':all(c['passed'] for c in checks) and all(c['snapshot_matches_manifest'] for c in
snapshot),
        'input_files':checks,'numerical_core':snapshot,
        'current_code_note': 'Historical runs are validated against frozen numerical_core_used; a
documented later cache/validation guard change need not alter their results.'}

def historical_signature(project,config):
    """Reproduce the original batch fingerprint using its exact frozen code."""
    project=Path(project)
    digest=hashlib.sha256(json.dumps(config,sort_keys=True).encode())
    for path in sorted((project/'provenance'/'numerical_core_used').glob('q3_*.py')):
        digest.update(path.name.encode());digest.update(path.read_bytes())
    digest.update((project/'data'/'source_manifest.json').read_bytes())
    return digest.hexdigest()

def audit_completed(project,seen=None):
    project=Path(project);root=project/'results';seen={} if seen is None else seen

```

```

sources=verify_sources(project)
write(project/'provenance/'/'batch_source_validation.json',sources)
paths=list((root/'main').glob('*/*summary.json'))
paths += [root/'ablation'/name/'summary.json' for name in
('old_forecast_all_trades','all_trades_greedy')]
for path in sorted(paths):
    if not path.exists():continue
    summary=read(path)
    if not summary.get('complete'):continue
    key=str(path.parent.relative_to(root));fingerprint=sha(path)
    if key in seen and seen[key].get('summary_sha256')==fingerprint:continue
    report=audit_run(path.parent,project/'data'/'q3_inputs.npz')
    expected=historical_signature(project,summary['config'])
    provenance={'passed':sources['passed'] and summary['run_signature']==expected,
        'recorded_run_signature':summary['run_signature'],'reconstructed_run_signature':expected,
        'frozen_code_path':'provenance/numerical_core_used','inputs_validated':sources['passed']}
    report['historical_provenance']=provenance
    report['passed']=report['passed'] and provenance['passed']
    write(path.parent/'independent_audit.json',report)
    seen[key]={ 'run':key,'passed':report['passed'],'rows':report.get('rows'),
        'windows':report.get('windows'),'checks':len(report.get('checks',[])),
        'trace_sha256':report.get('trace_sha256'),'summary_sha256':fingerprint,
        'cash_cost_yuan':report.get('sums',{}).get('cash_cost_yuan'),
        'historical_signature_valid':provenance['passed'],
        'failed_checks':report.get('failed_checks',report.get('errors'))}
    print(json.dumps(seen[key],ensure_ascii=False),flush=True)
main=[v for k,v in seen.items() if k.startswith('main/')]
ablation=[v for k,v in seen.items() if k.startswith('ablation/')]
aggregate={'passed':sources['passed'] and bool(seen) and all(x['passed'] for x in seen.values()),
    'complete_main':len(main)==8 and all(x['windows']==334 for x in main),
    'complete_ablation':len(ablation)==2 and all(x['windows']==334 for x in ablation),
    'main_rows':sum(x['rows'] for x in main),'ablation_rows':sum(x['rows'] for x in ablation),
    'main':main,'ablation':ablation,'sources':sources,
    'scope':'Independent physics/cash/information checks and exact historical input/core hash
verification; forecast generation itself not certified'}
write(root/'audit_main_validation.json',aggregate)
write(project/'notes'/'audit_main_validation.json',aggregate)
return aggregate,seen

if __name__=='__main__':
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--project',type=Path,default=Path(__file__).resolve().parents[1])
    parser.add_argument('--watch',action='store_true')
    args=parser.parse_args();seen={}
    while True:
        result,seen=audit_completed(args.project,seen)
        if not args.watch or result['complete_main'] and result['complete_ablation']:
            break
        time.sleep(60)
    print(json.dumps({k:result[k] for k in
('passed','complete_main','complete_ablation','main_rows','ablation_rows')},ensure_ascii=False))

```

42. question3/planning/code/q3_boundaries.py

源文件 SHA-256: e1d06edef203031a1b96621e5c2123a214a9fc107890fc66f9f4b20fea28caf3 ; 代码行数: 156。

```

"""Transparent start/end boundary sensitivity, preserving every interior order.

The startup is outside the user-confirmed 00:10 contract experiment. Its
missing load forecast is estimated only from the already issued first forecast
row, and the battery is held idle. The final actual right endpoint is missing;
only the last executed slot is re-evaluated under the recorded alternatives.
Neither operation is a new full-year optimization or a new observation.
"""
from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path

import numpy as np

from q3_io import load_inputs
from q3_optimizer import solve_dispatch, cash_components, ETA_C, ETA_D, RATE, LOW, HIGH


def _write_csv(path, rows):
    with open(path, "w", encoding="utf-8-sig", newline="") as f:
        w=csv.DictWriter(f, fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)


def startup_cases(data):
    """Common one-slot extension; forecast first, realized cash afterwards."""
    f=np.asarray(data["load_f"][0,:6],float)
    x=np.arange(1,7,dtype=float)
    slope=float((x-x.mean()) @ (f-f.mean())) / ((x-x.mean()) @ (x-x.mean()))
    candidates={"hold":float(f[0]),"linear2":float(max(2*f[0]-f[1],0)),
               "ols6":float(max(f.mean()-slope*x.mean(),0))}
    pv_forecast=float(data["pv_unused_first_mean_kw"][0])
    p=float(data["price"][-1])
    actual=data["first_unexecuted_interval"]
    load,pv=float(actual["load_mean_kw"]/6),float(actual["pv_mean_kw"]/6)
    rows=[]
    for method,load_forecast in candidates.items():
        g=max((load_forecast-pv_forecast)/6,0.)
        emergency=max(load-g-pv,0.)
        waste=max(g+pv-load,0.)
        unused=min(g,waste);curtail=max(waste-unused,0.)
        cash=p*g+5*p*emergency

    rows.append(dict(method=method,interval_start=actual["interval_start"],interval_end=actual["interval_end"],
                    load_forecast_estimated_kw=load_forecast,pv_forecast_supplied_kw=pv_forecast,
                    load_actual_mean_kw=load*6,pv_actual_mean_kw=pv*6,price_yuan_per_kwh=p,
                    g_kwh=g,q_kwh=g,charge_kwh=0.,discharge_kwh=0.,emergency_kwh=emergency,
                    unused_paid_kwh=unused,curtailed_pv_kwh=curtail,
                    original_cost_yuan=p*g,emergency_cost_yuan=5*p*emergency,cash_cost_yuan=cash,
                    change_from_hold_yuan=0.,battery_state_change_kwh=0.,common_to_all_strategies=True))

    for row in rows:
        row["change_from_hold_yuan"]=row["cash_cost_yuan"]-rows[0]["cash_cost_yuan"]
    return rows


def final_slot(g,q,state,p,load_mean_kw,pv_mean_kw,kappa):
    """Re-evaluate one frozen-contract slot, with the same soft terminal value."""
    r=solve_dispatch([[load_mean_kw/6]], [[pv_mean_kw/6]], [p], state,
                    mode="control",base_g=[g],booked_q=[q],locked_mask=[True],
                    terminal_weight=kappa,time_limit=10,mip_gap=1e-4)
    # Match the main runner's weakly dominant maximum-charge tie rule. Above

```

```

# the soft reference, MILP can otherwise return a smaller free charge with
# the same objective and create an artificial endpoint-state difference.
surplus=max(q+(pv_mean_kw-load_mean_kw)/6,0.)
if surplus > 1e-9:
    c=min(surplus,RATE,max((HIGH-state)/ETA_C,0.))
    r["c"][0,0]=c;r["d"][0,0]=r["e"][0,0]=0.
    r["s"][0,-1]=state+ETA_C*c
    r["terminal_cost"]=kappa*max(6000-r["s"][0,-1],0.)
c,d,e=(float(r[key][0,0]) for key in ("c","d","e"))
ending=float(r["s"][0,-1]);load,pv=load_mean_kw/6,pv_mean_kw/6
waste=max(q+pv+d+e-load-c,0.);v=min(q,waste);curtail=max(waste-v,0.)
physical=max(abs(q+pv+d+e-v-curtail-load-c),abs(ending-state-ETA_C*c+d/ETA_D),
              c-RATE,d-RATE,LOW-ending,ending-HIGH,min(c,d),min(c,e),-min(c,d,e,v,curtail),0.)
if physical>1e-5:
    raise RuntimeError("Canonical last-slot execution violates physical constraints.")
bill=cash_components([p],[g],[q],r["e"][0])
return {"charge_kwh":float(r["c"][0,0]),"discharge_kwh":float(r["d"][0,0]),
        "emergency_kwh":float(r["e"][0,0]),"soc_end_kwh":float(r["s"][0,-1]),
        "cash_cost_yuan":bill["total_cash_cost"],
        "terminal_penalty_yuan":r["terminal_cost"],
        "solver_status":r["status"],"solver_gap":r["gap"],
        "physical_max_violation_kwh":physical,
        "free_surplus_tie_rule":"Maximum feasible charge, matching the main execution rule"}

def run_boundary_study(data_dir,main_dir,outdir):
    data=load_inputs(data_dir)
    root,out=Path(main_dir),Path(outdir);out.mkdir(parents=True,exist_ok=True)
    paths=[root] if (root/"trace.csv").exists() else sorted(p.parent for p in
root.glob("*/summary.json") if (p.parent/"trace.csv").exists())
    if not paths:
        raise ValueError("No completed main strategy traces were found.")
    startup=startup_cases(data)
    endpoint=data["metadata"]["actual"]["last_right_endpoint_candidates"]
    expected_end=data["window_end"][-1].isoformat()
    p=float(data["price"][-1]);kappa=float(np.min(data["price"])/ETA_C)
    terminal=[]
    for folder in paths:
        summary=json.loads((folder/"summary.json").read_text())
        if not summary.get("complete") or summary["windows"] != len(data["dates"]):
            raise ValueError(f"Full-horizon boundary study requires all {len(data['dates'])} windows:
{folder}")
        with open(folder/"trace.csv",encoding="utf-8-sig",newline="") as f:
            trace=list(csv.DictReader(f))
            last=trace[-1]
            if len(trace)!=144*len(data["dates"]) or last["interval_end"]!=expected_end:
                raise ValueError("A strategy does not end at the missing final endpoint.")
            if str(last["actual_estimated"]).lower() not in ("true","1"):
                raise ValueError("Final trace does not label its actual endpoint as estimated.")
            if "terminal_weight" in summary["config"] and abs(summary["config"]["terminal_weight"]-
kappa)>1e-12:
                raise ValueError("Main terminal weight differs from the common minimum-price/.9 basis.")
            g,q=float(last["g_kwh"]),float(last["q_kwh"])

s=float(last["soc_start_kwh"]);cash0=float(last["cash_cost_yuan"]);end0=float(last["soc_end_kwh"])
if abs(float(last["price_yuan_per_kwh"])-p)>1e-9:
    raise ValueError("The final tariff differs from the common data basis.")
for method in ("hold","linear2","ols6"):
    load=endpoint["load"][method]["mean_kw"];pv=endpoint["pv"][method]["mean_kw"]
    r=final_slot(g,q,s,p,load,pv,kappa)
    delta=r["cash_cost_yuan"]-cash0;state_delta=r["soc_end_kwh"]-end0
    if method=="hold" and (abs(delta)>1e-5 or abs(state_delta)>1e-5):
        raise RuntimeError("Hold endpoint does not reproduce the main last-slot cash and
state.")
    terminal.append(dict(strategy=summary["strategy"],method=method,
                        interval_start=last["interval_start"],interval_end=last["interval_end"],

```



```

        load_actual_mean_estimated_kw=load,pv_actual_mean_estimated_kw=pv,
        g_fixed_kwh=g,q_fixed_kwh=q,soc_start_fixed_kwh=s,
        **r,terminal_state_delta_from_main_kwh=state_delta,
        last_slot_cash_delta_from_main_yuan=delta,
        main_contract_cash_total_yuan=summary["cash_cost_yuan"],
        revised_contract_cash_total_yuan=summary["cash_cost_yuan"]+delta,
        inventory_adjusted_total_delta_yuan=delta-kappa*state_delta,
        same_cash_basis=True))
_write_csv(out/"startup_sensitivity.csv",startup)
_write_csv(out/"final_endpoint_sensitivity.csv",terminal)
result={"startup":startup,"final_endpoint":terminal,"terminal_weight":kappa,
        "startup_scope":"An optional idle-battery extension of the omitted first 00:00-00:10, separate
from 334 contract-window totals",
        "startup_forecast_methods":{"hold":"first issued interval mean",
        "linear2":"max(2*f[0]-f[1],0)",
        "ols6":"OLS through the first six issued means at x=1..6, evaluated at x=0 and clipped
nonnegative"},
        "startup_causality":"Only the already-issued first forecast row is used; actual first-interval
means enter evaluation, not the order",
        "startup_state":"Battery idle; no change to the confirmed first 00:10 initial state or any later
state",
        "final_endpoint_scope":"Freeze last original g, formally signed q and state; solve only the
final slot",
        "same_cash_basis":True,
        "cash_basis":"Original fee + 1.5p increase - 0.5p cancellation + 5p emergency; terminal penalty
excluded",
        "terminal_delta":"Separately report terminal state and cash-kappa*inventory comparisons",
        "limitations":"Endpoint alternatives are estimates; these calculations do not rerun or optimize
the interior policy",
        "checks_passed":all(r["physical_max_violation_kwh"]≤1e-5 for r in terminal)}
(out/"summary.json").write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding="utf-8")
return result

if __name__=="__main__":
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data",required=True);parser.add_argument("--main",required=True)
    parser.add_argument("--out",required=True);args=parser.parse_args()
    result=run_boundary_study(args.data,args.main,args.out)

    print(json.dumps({"checks_passed":result["checks_passed"],"strategy_cases":len(result["final_endpoint"])
    },ensure_ascii=False))

```

43. question3/planning/code/q3_causal_replay.py

源文件 SHA-256: d821286f9034b074e12a3e4c5816ffed9faf911be7a2e0e45dfc1afd35bb7924 ; 代码行数: 257。

```

"""Execution-timing sensitivity under the same timestamp-revealed contracts.

Compare the main ten-minute mean-feedback approximation with one-minute causal
left-node feedback. Reoptimize battery execution only, never original orders or
formal adjustments. The simulator linearly interpolates observed ten-minute
nodes into an artificial one-minute environment; the controller only receives
a node when its timestamp is reached. This is a discretization sensitivity,
not higher-frequency measurement or an exact physical validation.
"""

from __future__ import annotations

import argparse
import csv
import datetime as dt
import hashlib
import json
from pathlib import Path

import numpy as np

from q3_io import load_inputs
from q3_optimizer import solve_dispatch, LOW, HIGH, RATE, ETA_C, ETA_D
from q3_risk import HOURS, START, latest_hour, future_mask, current_block


def _read_csv(path):
    with open(path, encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f))


def _write_csv(path, rows):
    with open(path, "w", encoding="utf-8-sig", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader(); w.writerows(rows)


def run_replay(data_dir, main_run_dir, outdir, start_day=24, days=7):
    data = load_inputs(data_dir)
    source, out = Path(main_run_dir), Path(outdir)
    out.mkdir(parents=True, exist_ok=True)
    log = json.loads((source/"event_log.json").read_text())
    cfg = log["config"]
    trade = tuple(cfg["trade_hours"])
    forecasts = tuple(cfg["forecast_hours"])
    if cfg.get("pv_source", "supplied") != "supplied":
        data["pv_f"] = data["pv_linear_f"].copy()
    stop = start_day+days
    if start_day < 0 or days < 1 or stop > len(data["dates"]):
        raise ValueError("Requested local replay window is outside input dates.")
    selected = {str(d): [] for d in data["dates"][start_day:stop]}
    for row in _read_csv(source/"trace.csv"):
        if row["window_date"] in selected:
            selected[row["window_date"]].append(row)
    events = {(r["window_date"], r["kind"], r["decision_at"]): r for r in log["events"]}
    original, final_q = {}, {}
    for day in range(start_day, stop):
        date = str(data["dates"][day])
        rows = sorted(selected[date], key=lambda r:r["interval_start"])
        if len(rows) != 144:
            raise ValueError(f"Main trace does not contain all 144 slots for {date}.")
        for t, row in enumerate(rows):
            target = data["window_start"][day]+dt.timedelta(minutes=10*t)
            if row["interval_start"] != target.isoformat():
                raise ValueError("Main trace timestamps do not match the contract window.")

```



```

        if abs(float(row["price_yuan_per_kwh"])-data["price"][t]) > 1e-9:
            raise ValueError("Main trace and input tariff differ.")
        for key in ("load", "pv"):
            if abs(float(row[f"{key}_actual_kw"])-data[f"{key}_a"][day,t]) > 1e-7:
                raise ValueError("Main trace and input observed interval means differ.")
            if abs(float(row["load_forecast_kw"])-data["load_f"][day,t]) > 1e-7:
                raise ValueError("Main trace and input load forecasts differ.")
    selected[date] = rows
    original[day] = np.array([float(r["g_kwh"]) for r in rows])
    final_q[day] = np.array([float(r["q_kwh"]) for r in rows])
    midnight = dt.datetime.combine(data["dates"][day],dt.time()).isoformat()
    if (date,"dayahead",midnight) not in events:
        raise ValueError("A day-ahead contract has no timestamped release event.")
    adjustable = np.zeros(144, bool)
    for h in trade:
        if h:
            a,b = current_block(trade,h)
            key=(date,"adjust",dt.datetime.combine(data["dates"][day],dt.time(h)).isoformat())
            if key not in events or (events[key]["commit_start"],events[key]["commit_end"]) !=
(a,b):
                raise ValueError("A formal adjustment lacks its matching commit event.")
            adjustable[a:b] = True
        if np.max(np.abs((final_q[day]-original[day])[~adjustable]), initial=0) > 1e-5:
            raise ValueError("An order was changed outside the enabled adjustment blocks.")
    initial = float(selected[str(data["dates"][start_day]][0]["soc_start_kwh"])
    p = data["price"]
    kappa = cfg.get("terminal_weight", float(np.min(p)/ETA_C))
    solver_options = dict(terminal_weight=kappa, time_limit=cfg.get("time_limit",10),
                           mip_gap=cfg.get("mip_gap",1e-4))
    summaries, trajectories = {}, {}
    for variant in ("10min_mean", "1min_causal"):
        state = initial
        rows, minute_rows, reveals = [], [], []
        stats = dict(solves=0,solve_seconds=0.,max_violation_kwh=0.,unproven=0)

    def command(L,G,prices,base,booked,future,now_l,now_pv):
        surplus=max(booked[0]+now_pv-now_l,0.)
        deficit=max(now_l-booked[0]-now_pv,0.)
        if surplus > 1e-9:
            return min(surplus,RATE,max((HIGH-state)/ETA_C,0.)),0.,0.,True
        if cfg.get("controller","mpc") == "greedy":
            d=min(deficit,RATE,max((state-LOW)*ETA_D,0.))
            return 0.,d,max(deficit-d,0.),False
        L,G=L.copy(),G.copy();L[0]=now_l;G[0]=now_pv
        z=solve_dispatch(L[None,:],G[None,:],prices,state,mode="control",
                        base_g=base,booked_q=booked,future_adjust_mask=future,**solver_options)
        stats["solves"]+=1;stats["solve_seconds"]+=z["solve_seconds"]
        stats["unproven"]+=int(not z["optimality_proven"])
        return float(z["c"][0,0]),float(z["d"][0,0]),float(z["e"][0,0]),bool(z["u"][0,0])

    def check_step(gq,L,G,c,d,e,v,r,sbefore,safter,rate):
        error=max(abs(gq+G-r-v+d+e-L-c),
                  abs(safter-sbefore-ETA_C*c+d/ETA_D),
                  LOW-safter,safter-HIGH,c-rate,d-rate,-min(c,d,e,v,r),
                  min(c,d),min(c,e),v-gq,r-G,e-L,0.)
        stats["max_violation_kwh"]=max(stats["max_violation_kwh"],error)
        if error > 1e-5:
            raise RuntimeError(f"{variant} physical violation: {error}")

    def execute(day,t,L,G,prices,base,booked,future,fh):
        nonlocal state
        before=state
        begin=data["window_start"][day]+dt.timedelta(minutes=10*t)
        if variant == "10min_mean":
            load=float(data["load_a"][day,t]/6);pv=float(data["pv_a"][day,t]/6)
            c,d,e,direction=command(L,G,prices,base,booked,future,load,pv)

```

```

waste=max(booked[0]+pv+d+e-load-c,0.)
v=min(booked[0],waste);r=max(waste-v,0.)
state+=ETA_C*c-d/ETA_D
check_step(booked[0],load,pv,c,d,e,v,r,before,state,RATE)
else:
    # The controller receives only the current left node here.
    now_l=float(data["load_nodes"][day,t]/6)
    now_g=float(data["pv_nodes"][day,t]/6)
    cp,dp,ep,direction=command(L,G,prices,base,booked,future,now_l,now_g)
    remaining_d=dp
    c=d=e=v=r=load=pv=0.
    for minute in range(10):
        # Artificial environment: expose only the node at NOW.
        # A right endpoint is used to construct the environment,
        # never passed to command() before it becomes observable.
        observed_load=float((1-minute/10)*data["load_nodes"][day,t]
            +(minute/10)*data["load_nodes"][day,t+1])
        observed_pv=float((1-minute/10)*data["pv_nodes"][day,t]
            +(minute/10)*data["pv_nodes"][day,t+1])
        lm,gm=observed_load/60,observed_pv/60
        qm=float(booked[0]/10)
        A=max(qm+gm-lm,0.);D=max(lm-qm-gm,0.)
        cm=min(A,RATE/10,max((HIGH-state)/ETA_C,0.)) if direction else 0.
        dm=0. if direction else min(D,RATE/10,max((state-LOW)*ETA_D,0.),remaining_d)
        em=max(D-dm,0.)
        wm=max(A-cm,0.);vm=min(qm,wm);rm=max(wm-vm,0.)
        sb=state;state+=ETA_C*cm-dm/ETA_D
        remaining_d=max(remaining_d-dm,0.)
        check_step(qm,lm,gm,cm,dm,em,vm,rm,sb,state,RATE/10)
        minute_rows.append(dict(interval_start=
(begin+dt.timedelta(minutes=minute)).isoformat(),
            interval_end=(begin+dt.timedelta(minutes=minute+1)).isoformat(),
            load_observed_left_kw=observed_load,pv_observed_left_kw=observed_pv,
            q_kwh=qm,charge_kwh=cm,discharge_kwh=dm,emergency_kwh=em,
            unused_paid_kwh=vm,curtailed_pv_kwh=rm,soc_start_kwh=sb,soc_end_kwh=state,
            command_direction="charge" if direction else "discharge_or_idle",
            remaining_block_discharge_budget_kwh=remaining_d))
        c+=cm;d+=dm;e+=em;v+=vm;r+=rm;load+=lm;pv+=gm
    ordinary=float(p[t]*base[0]);increase=max(booked[0]-base[0],0.);decrease=max(base[0]-
booked[0],0.)
    adjustment=float(p[t]*(1.5*increase-.5*decrease));emergency=float(5*p[t]*e)
    rows.append(dict(variant=variant>window_date=str(data["dates"][day]),
        interval_start=begin.isoformat(),interval_end=
(begin+dt.timedelta(minutes=10)).isoformat(),
        forecast_hour=fh,g_kwh=float(base[0]),q_kwh=float(booked[0]),
        load_energy_kwh=load,pv_energy_kwh=pv,charge_kwh=c,discharge_kwh=d,emergency_kwh=e,
        unused_paid_kwh=v,curtailed_pv_kwh=r,soc_start_kwh=before,soc_end_kwh=state,
        ordinary_cost_yuan=ordinary,adjustment_cost_yuan=adjustment,
        emergency_cost_yuan=emergency,cash_cost_yuan=ordinary+adjustment+emergency))

for day in range(start_day,stop):
    # New g becomes visible at this midnight, before the old tail is
    # observed. Future formal q remains inaccessible until its gate.
    if day > start_day:
        oldg,oldq=g.copy(),q.copy()
    g=original[day].copy();q=g.copy()
    reveals.append(dict(at=dt.datetime.combine(data["dates"][day],dt.time()).isoformat(),
        kind="dayahead",window_date=str(data["dates"][day]),start=0,end=144))
    if day > start_day:
        oldfh=latest_hour(forecasts,143)
        lf=np.r_[data["load_f"][day-1,143],data["load_f"][day]]/6
        pf=np.r_[data["pv_f"][day-1,HOURS.index(oldfh),143],data["pv_f"][day,0]]/6
        execute(day-1,143,lf,pf,np.r_[p[-1],p],np.r_[oldg[-1],g],np.r_[oldq[-1],q],
            np.r_[False,future_mask(trade,0)],oldfh)
    for t in range(143):
        h=next((h for h in trade if h and START[h]==t),None)

```

```

        if h is not None:
            a,b=current_block(trade,h)
            q[a:b]=final_q[day][a:b] # Only current committed block.
            reveals.append(dict(at=dt.datetime.combine(data["dates"]
[day],dt.time(h)).isoformat(),
                                kind="adjust",window_date=str(data["dates"]
[day]),start=a,end=b))
            fh=latest_hour(forecasts,t)
            execute(day,t,data["load_f"][day,t:]/6,data["pv_f"][day,HOURS.index(fh),t:]/6,
                    p[t:],g[t:],q[t:],future_mask(trade,t),fh)
            if not np.allclose(q,final_q[day],rtol=0,atol=1e-5):
                raise RuntimeError("Timestamp-revealed order differs from the fixed main order.")
            day=stop-1;fh=latest_hour(forecasts,143)
            execute(day,143,data["load_f"][day,143:]/6,data["pv_f"][day,HOURS.index(fh),143:]/6,
                    p[-1:],g[-1:],q[-1:],np.zeros(1,bool),fh)
            if len(rows)≠144*days:
                raise RuntimeError("Wrong local replay length.")
            totals={key:float(sum(row[key] for row in rows)) for key in
                    ("cash_cost_yuan","ordinary_cost_yuan","adjustment_cost_yuan","emergency_cost_yuan",
                    "emergency_kwh","load_energy_kwh","pv_energy_kwh","charge_kwh","discharge_kwh")}
            totals.update(soc_start_kwh=initial,soc_end_kwh=state,
                        inventory_adjusted_cost_yuan=totals["cash_cost_yuan"]-kappa*(state-initial),
                        intervals=len(rows),minute_intervals=len(minute_rows),solver=stats)
            summaries[variant]=totals;trajectories[variant]=rows
            _write_csv(out/f"trace_{variant}.csv",rows)
            if minute_rows:
                _write_csv(out/"minute_trace.csv",minute_rows)
            (out/f"contract_reveals_{variant}.json").write_text(json.dumps(reveals,indent=2))
            base,fine=summaries["10min_mean"],summaries["1min_causal"]
            for key in ("ordinary_cost_yuan","adjustment_cost_yuan"):
                if abs(base[key]-fine[key])>1e-6:
                    raise RuntimeError("The two controllers did not use identical signed contracts.")
            # Continuous linear-node left-Riemann discrepancy telescopes across windows.
            expected_bias={key:float((data[f"{key}_nodes"][start_day,0]-data[f"{key}_nodes"][stop-1,-1])/120)
                            for key in ("load","pv")}
            observed_bias={key:fine[f"{key}_energy_kwh"]-base[f"{key}_energy_kwh"] for key in ("load","pv")}
            if any(abs(expected_bias[k]-observed_bias[k])>1e-5 for k in expected_bias):
                raise RuntimeError("Minute discretization energy difference disagrees with its analytical
identity.")
            result=dict(source_run=str(source.resolve()),source_strategy=cfg["strategy"],
                        source_hashes={name:hashlib.sha256((source/name).read_bytes()).hexdigest()
                                        for name in ("trace.csv","event_log.json","summary.json")},
                        start_day=start_day,days=days,start=str(data["window_start"]
[start_day]),end=str(data["window_end"][stop-1]),
                        local_initial_state="Main trace state at selected first 00:10, identical for both controllers",
                        end_rule="Both controllers stop at the selected final 00:10; neither reads the next unselected
contract",
                        contracts="Same fixed g/q, revealed only at original midnight and enabled intraday commit
times",
                        minute_method="Artificial linear-node environment; left power held for one minute, frozen 10-
minute direction and discharge budget",
                        limitations=["Not high-frequency measured data or exact physical validation",
                                    "One-minute left-endpoint power hold differs from ten-minute endpoint-average
energy",
                                    "Within-block direction cannot reverse; physical stopping, free disposal and
emergency balancing remain allowed",
                                    "Local fixed-contract sensitivity does not reselect the best annual contract
policy"],
                        summary=summaries,one_minute_minus_mean={k:fine[k]-base[k] for k in
                            ("cash_cost_yuan","emergency_kwh","soc_end_kwh","inventory_adjusted_cost_yuan")},
                        discretization_energy_bias_kwh=observed_bias,analytical_energy_bias_kwh=expected_bias,
                        checks_passed=True)
            (out/"summary.json").write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding="utf-8")
            return result

```

```
if __name__=="__main__":
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data",required=True);parser.add_argument("--main-run",required=True)
    parser.add_argument("--out",required=True);parser.add_argument("--start-day",type=int,default=24)
    parser.add_argument("--days",type=int,default=7)
    args=parser.parse_args()
    result=run_replay(args.data,args.main_run,args.out,args.start_day,args.days)
    print(json.dumps({"checks_passed":result["checks_passed"],
                     "one_minute_minus_mean":result["one_minute_minus_mean"]},ensure_ascii=False))
```

44. question3/planning/code/q3_conclusions.py

源文件 SHA-256: e7e6bfe530e255ec28f63de0b99c9a1ed68fb1d9bb021dabd655f6786fc3e7da ; 代码行数: 396。

```

"""Build Chinese conclusions from completed Q3 artifacts, never assumed wins.

The report distinguishes development-selected policies, retrospective rankings,
local algorithm comparisons, and fixed-contract execution sensitivities.
Missing experiments remain explicitly pending; this file does not optimize.
"""
from __future__ import annotations

import argparse
import csv
from collections import defaultdict
import json
from pathlib import Path

def read_json(path, default=None):
    path=Path(path)
    return json.loads(path.read_text(encoding='utf-8')) if path.exists() else default

def money(x):
    return '待完成' if x is None else f'{{float(x):,.2f}}'

def number(x, places=3):
    return '待完成' if x is None else f'{{float(x):,.{places}f}}'

def schedule(row):
    hours=row.get('config', {}).get('trade_hours')
    return '、'.join(f'{{int(h):02d}}:00' for h in hours) if hours else row['strategy']

def savings(candidate, base):
    delta=float(base)-float(candidate)
    return {'saving_yuan':delta, 'saving_percent':None if abs(base)<1e-12 else 100*delta/float(base)}

def comparison_sentence(candidate_label, base_label, candidate, base):
    result=savings(candidate, base)
    delta=result['saving_yuan']; pct=result['saving_percent']
    if abs(delta)<1e-5:
        return f'{{candidate_label}}与{{base_label}}现金费用在0.00001元容差内相同。'
    word='减少' if delta>0 else '增加'
    tail='' if pct is None else f' ({{abs(pct):.6f}}%) '
    return f'{{candidate_label}}相对{{base_label}}, 现金费用{{word}}{{money(abs(delta))}}元{{tail}}。'

def completed_rows(folder):
    folder=Path(folder)
    obj=read_json(folder/'comparison.json')
    if isinstance(obj, list):
        return [x for x in obj if x.get('complete')]
    rows=[]
    for path in sorted(folder.glob('*summary.json')):
        row=read_json(path)
        if row.get('complete'):
            rows.append(row)
    return rows

def daily_rows(folder):
    path=Path(folder)/'daily.csv'
    if not path.exists():

```

```

        return []
    with path.open(encoding='utf-8-sig',newline='') as stream:
        return list(csv.DictReader(stream))

def table(headers,rows):
    return ['| '+'| '.join(headers)+'| ','| '+'| '.join('---' for _ in headers)+'| ',
            *['| '+'| '.join(str(x) for x in row)+'| ' for row in rows],']

def _first_json(paths):
    for path in paths:
        if Path(path).exists():
            return read_json(path),str(Path(path))
    return None,None

def _alg_table(rows):
    labels={'stochastic_mpc':'场景MILP+MPC','deterministic_mpc':'点预测MILP+MPC',
            'stochastic_greedy':'场景MILP+贪心','scenes_4':'4场景','scenes_8':'8场景',
            'scenes_16':'16场景','initial_1200':'初态1200','initial_10800':'初态10800',
            'linear_reconstruction':'直接线性复原PV'}
    return table(['方案','合同数','现金费用（元）','库存修正指标（元）','紧急电量（kWh）',
                '初态→末态（kWh）','总求解秒数'],
                [[labels.get(r['strategy'],r['strategy']).replace('seed_','种
子'),r['windows'],money(r['cash_cost_yuan']),money(r['inventory_adjusted_cost_yuan']),
                number(r['emergency_kwh']),f"{number(r['soc_start_kwh'])}→{number(r['soc_end_kwh'])}]",
                number(r.get('solver',{}).get('seconds'))] for r in rows])

def build_conclusions(results_root,out_md):
    root=Path(results_root);out=Path(out_md)
    main=root/'main'
    rows=completed_rows(main)
    by_name={r['strategy']:r for r in rows}
    selection=read_json(main/'selection.json',{})
    selected=selection.get('selected_strategy')
    hindsight=selection.get('retrospective_best')
    complete=len(rows)==8 and all(r.get('windows')==334 for r in rows)
    development={r['strategy']:r for r in selection.get('development',[])}
    tests={r['strategy']:r for r in selection.get('test',[])}
    analysis={'complete_main':complete,'completed_main_strategies':len(rows),
            'selected_strategy':selected,'retrospective_best':hindsight,
            'main':rows,'selection':selection,'pending':[],
            'cash_basis':'334 D00:10-D+1 00:10 contract windows; startup is separate; terminal soft penalty
excluded',
            'refund_convention':'positive refund amount is subtracted; 50% cancellation penalty is added',
            'execution_assumption':'current ten-minute actual mean as fast-feedback approximation',
            'scope_limitations':['supplied forecast generator causality not independently certified',
            'rolling two-stage scenario recourse is not a strict nonanticipative multistage optimum',
            'local sensitivity results do not establish global or annual dominance']}
    lines=['# 第三问计算结论与模型评价','']
    if not complete:
        lines += [f'**阶段性文件：主计算目前完成{len(rows)}/8个组合，不能据此宣布完整年度最优组合。**','']
        analysis['pending'].append('全部8组334合同主计算及开发选择记录')
    elif selected in by_name:
        lines += [f'按预先规定的开发期选择规则，选定组合为 **{schedule(by_name[selected])}**.','
            '下面分别报告该组合的后续测试结果和八个组合的全年回溯比较；两者不混为一谈。','']
        lines += ['主实验统一使用用户提供的区间光伏预测与第二问负载预测，比较0点必选、6/12/18点可选的8个组合。'
            '每个组合从共同6000 kWh初态独立连续运行，之后不每天重置。合同从本日00:10到次日00:10，功率只乘1/6转为电
量一次。',
            '', '## 一、八个组合的实际费用','']
        lines += table(['组合','合同数','合同现金总费（元）','初态（kWh）','末态（kWh）','紧急电（kWh）','开发期指标
（元）','测试期现金（元）'],
            [[schedule(r),r['windows'],money(r['cash_cost_yuan']),number(r['soc_start_kwh']),number(r['soc_end_kwh'])

```



```

),
    number(r['emergency_kwh']),money(development.get(r['strategy'],{}).get('score')),
    money(tests.get(r['strategy'],{}).get('cash_cost_yuan'))]
    for r in sorted(rows,key=lambda r:(len(r['config']['trade_hours']),r['config']
['trade_hours'])))
    lines += ['现金费用=原计划费+1.5倍增购费-取消退款+50%违约金+5倍紧急费。开发指标另用统一库存价值修正，'
            '用于避免靠消耗更多库存获得不公平优势；它不是另一个电费账单。表中合同总费不包含共同启动段，也不包含库存软
惩罚。','']
    if selected in by_name:
        s=by_name[selected]
        lines += [f"选定组合的原计划费为{money(s['original_plan_cost_yuan'])}元，调整净费为
{money(s['net_adjustment_cost_yuan'])}元，"
            f"紧急费为{money(s['emergency_cost_yuan'])}元。共同启动段另计
{money(s.get('startup_cash_cost_yuan'))}元；"
            f"按自然日2月1日00:00至次年1月1日00:00汇总为{money(s.get('natural_period_cash_cost_yuan'))}
元。",']
    if 'H0' in by_name:
        b=by_name['H0']
        analysis['selected_vs_H0']=savings(s['cash_cost_yuan'],b['cash_cost_yuan'])
        lines += [comparison_sentence('选定组合','只启用0点',s['cash_cost_yuan'],b['cash_cost_yuan'])+
            f" 两者末态分别为{number(s['soc_end_kwh'])}和{number(b['soc_end_kwh'])} kWh。",']
    if selected in tests and 'H0' in tests:
        st,bt=tests[selected],tests['H0']
        analysis['selected_test_vs_H0']=savings(st['cash_cost_yuan'],bt['cash_cost_yuan'])
        lines += [f"冻结后的测试期分别含{st['windows']}和{bt['windows']}个完整合同。"+
            comparison_sentence('选定组合','只启用0
点',st['cash_cost_yuan'],bt['cash_cost_yuan']),']
    lines += ['## 二、开发选择与全年回溯排名','',
        '主设置在比较前预设：最多8场景、最近28个已兑现历史合同、至少14条历史才启用场景、固定随机种子20260912。'
        '场景误差按发行版本配对，不额外叠加分位数裕量。开发窗口为2月17-23日，2月25日零点前冻结组合；'
        '2月24日合同末段尚未在25日零点兑现，因此不进入选择。后续测试从2月25日开始。',']
    if selected in by_name and hindsight in by_name:
        relation='相同' if selected==hindsight else '不同'
        lines += [f"开发期选定{schedule(by_name[selected])}，全年回溯库存修正指标最低为
{schedule(by_name[hindsight])}，两者{relation}。"
            '后者使用了全年结果，只能称回溯最佳，不能替换开发选择后继续冒充完全独立测试。',']
        cash_best=min(rows,key=lambda r:r['cash_cost_yuan'])
        analysis['retrospective_cash_best']=cash_best['strategy']
        lines += [f"若只按未修正的现金总费排序，最低组合是{schedule(cash_best)}，费用
{money(cash_best['cash_cost_yuan'])}元。",']
    else:
        lines += ['开发选择或完整回溯记录尚未齐备，本节暂不宣布胜出组合。',']

monthly={}
for r in rows:
    sums=defaultdict(float)
    for row in daily_rows(main/r['strategy']):
        sums[row['window_date'][:7]]+=float(row['cash_cost_yuan'])
    monthly[r['strategy']]=dict(sums)
analysis['monthly_contract_cash']=monthly
if complete and monthly and all(monthly.values()):
    months=sorted(set.intersection(*(set(x) for x in monthly.values()))
    winners={month:min(rows,key=lambda r:monthly[r['strategy']][month])['strategy'] for month in
months}
analysis['monthly_winner']=winners
lines += ['## 三、分月稳定性','']
lines += table(['合同月份','当月现金最低组合','最低费用（元）','开发选定费用（元）','选定相对H0节省
（元）'],
    [[month,schedule(by_name[winners[month]]),money(monthly[winners[month]][month]),
    money(monthly.get(selected,{}).get(month)),
    money(monthly['H0'][month]-monthly[selected][month]) if selected in monthly and 'H0' in
monthly else '待完成']
    for month in months])
if selected in monthly and 'H0' in monthly:
    diff=[monthly['H0'][m]-monthly[selected][m] for m in months]
    lower=sum(d>1e-5 for d in diff);higher=sum(d< -1e-5 for d in diff);tie=len(diff)-lower-

```

```

higher
    analysis['monthly_selected_vs_H0_counts']={'lower':lower,'higher':higher,'tie':tie}
    lines += [f'选定组合相对H0, 在{lower}个月费用较低、{higher}个月较高、{tie}个月相同。'
              '这里按合同起始日期所属月汇总, 不把单个展示日当作全年选型依据。','']

    lines += ['## 四、新预报本身的价值','']
    old=read_json(root/'ablation'/'old_forecast_all_trades'/'summary.json')
    rich=by_name.get('H0_6_12_18')
    if old and old.get('complete') and rich and old['windows']==rich['windows']:
        analysis['new_forecast_ablation']={'old':old,'new':rich,
        **savings(rich['cash_cost_yuan'],old['cash_cost_yuan'])}
        lines += table(['PV信息','调单时刻','现金费用(元)','紧急电量(kWh)','期末库存(kWh)'],
        [['保持0点版本','0、6、12、
18',money(old['cash_cost_yuan']),number(old['emergency_kwh']),number(old['soc_end_kwh'])],
        ['使用当时新版本','0、6、12、
18',money(rich['cash_cost_yuan']),number(rich['emergency_kwh']),number(rich['soc_end_kwh'])]])
        lines += [comparison_sentence('使用新版光伏','相同调单权限但只用0点
版',rich['cash_cost_yuan'],old['cash_cost_yuan'])+
        '这是同权限的信息消融, 分别随各自真实库存重新规划, 并非强制整年使用同一份g。','']
    else:
        analysis['pending'].append('同权限旧预报全年对照')
        lines += ['相同调单权限、仅保留0点PV版本的全年对照尚未完成, 暂不量化信息带来的费用收益。','']
    forecast=read_json(root.parent/'notes'/'forecast_comparison.json')
    if isinstance(forecast,list):
        analysis['forecast_accuracy_comparison']=forecast
        lines += table(['更新时刻','同目标区间数/日','旧版MAE(kW)','新版MAE(kW)','旧版RMSE(kW)','新版RMSE
(kW)'],
        [[f"{r['hour']:02d}:00",r['intervals'],number(r['old_mae_kw']),number(r['new_mae_kw']),
        number(r['old_rmse_kw']),number(r['new_rmse_kw'])] for r in forecast])
        lines += ['预测误差比较按同一时刻以后的同一批目标区间进行。MAE/RMSE改善不自动等于费用改善, 仍需上面的运行费用
验证。'
        '18点后的剩余光伏整体较少, 其绝对误差不能直接与中午版本横向判断信息价值。','']

    algorithms=completed_rows(root/'sensitivity'/'algorithms')
    alg={r['strategy']:r for r in algorithms}
    analysis['algorithm_sensitivity']=alg
    lines += ['## 五、场景模型和MPC是否都值得保留','']
    target_names=('stochastic_mpc','deterministic_mpc','stochastic_greedy')
    present=[alg[n] for n in target_names if n in alg]
    if present:
        lines += _alg_table(present)
        lines += ['这组试验固定丰富组合0/6/12/18, 自2月25日开始各自连续回放60个合同, 共用局部初态和终点规则。'
        '它没有强制三组原计划g相同: 不同控制动作会改变后续真实状态, 继而影响下一次订电。'
        '因此它是完整算法策略的短窗比较, 不能把差额全称为纯MPC在同g下的因果收益, 也不能把60窗费用冒称全年费
用。','']
    else:
        analysis['pending'].append('60窗算法对照')
        lines += ['预定60窗算法对照尚未产生完整结果。','']
    if all(n in alg for n in target_names):
        st,det,gr=(alg[n] for n in target_names)
        analysis['stochastic_vs_deterministic']=savings(st['cash_cost_yuan'],det['cash_cost_yuan'])
        analysis['mpc_vs_greedy']=savings(st['cash_cost_yuan'],gr['cash_cost_yuan'])
        lines += [comparison_sentence('场景MILP+MPC','点预测
MILP+MPC',st['cash_cost_yuan'],det['cash_cost_yuan']),
        '',comparison_sentence('场景MILP+MPC','场景MILP+贪心反
馈',st['cash_cost_yuan'],gr['cash_cost_yuan']),'']
        if st['cash_cost_yuan']≥gr['cash_cost_yuan']-1e-5:
            lines += ['本段数据没有证明MPC比贪心更省电费。若差额很小, 应把两者视为接近, 并考虑贪心的简洁与运行时间;'
            '保留MPC主要是为了远期库存与调单机会的可解释规划能力, 不能用此前推荐代替当前证据。','']
        else:
            lines += ['MPC在本段完整策略比较中费用更低; 是否全年仍成立, 需要更长或跨季对照, 不能从局部试验直接推
广。','']
    counterfactuals=read_json(root.parent/'notes'/'control_difference_counterfactuals.json')
    if isinstance(counterfactuals,list) and counterfactuals:
        analysis['controller_difference_counterfactual_count']=len(counterfactuals)
        lines += [f'另对短窗中{len(counterfactuals)}个“段初状态相同但首步放电开始分歧”的时点进行了严格重解与强制贪

```



```

心反事实。'
        '该审查区分预测目标上的库存取舍和实际兑现后的现金差异；不能把预测内更优直接解释成实际必然更省。'
        '详细数字和原轨迹校验见《控制器差异审查》。','']
    annual_greedy=read_json(root/'ablation'/'all_trades_greedy'/'summary.json')
    if annual_greedy and annual_greedy.get('complete') and rich and
annual_greedy['windows']==rich['windows']==334:
        comparison=savings(rich['cash_cost_yuan'],annual_greedy['cash_cost_yuan'])
        comparison.update(mpc=rich,greedy=annual_greedy,
            emergency_difference_mpc_minus_greedy_kwh=rich['emergency_kwh']-
annual_greedy['emergency_kwh'],
            final_inventory_difference_mpc_minus_greedy_kwh=rich['soc_end_kwh']-
annual_greedy['soc_end_kwh'])
        analysis['annual_mpc_vs_greedy']=comparison
        lines += ['**全部时刻启用下的全年控制器补充对照：**','']
        lines += table(['控制器','合同数','现金费(元)','库存修正指标(元)','紧急电(kWh)','初态→末态
(kWh)','总求解秒数'],
            [[name,r['windows'],money(r['cash_cost_yuan']),money(r['inventory_adjusted_cost_yuan']),
            number(r['emergency_kwh']),f"{number(r['soc_start_kwh'])}→{number(r['soc_end_kwh'])}",
            number(r.get('solver',{}).get('seconds'))]
            for name,r in [( '场景MILP+MPC',rich),( '场景MILP+贪心',annual_greedy)]]])
        lines += [comparison_sentence('全年MPC策略','全年贪心策
略',rich['cash_cost_yuan'],annual_greedy['cash_cost_yuan']),
            '',f"MPC减贪心的紧急电量差为
{number(comparison['emergency_difference_mpc_minus_greedy_kwh'])} kWh, "
            f"期末库存差为{number(comparison['final_inventory_difference_mpc_minus_greedy_kwh'])}
kWh."
        '两组都从6000开始，但各自按真实状态重新签订后续订单，因此全年比较仍是完整闭环策略差异，不是强制同g的
纯电动控制实验。','']
        percent=comparison['saving_percent']
        if percent is not None and abs(percent)<.1:
            lines += ['两者现金相对差不到千分之一，现有证据不足以宣称某控制器有明显经济优势。'
                '应同时考虑计算时间、库存和控制解释需要，不把微小差额夸大模型压倒性优势，也没有进行统计显著性检
验。','']
        else:
            lines += ['上述数值描述本次全年回放的实际差异，尚不能称为对其他年份、预测器或误差分布的普遍优势。','']
            lines += ['全年贪心是控制器追加验证，不重写此前开发期已经冻结的预报组合选择；如果据这次全年对照更换最终控制器，
应明确属于追加回溯选型。','']
        else:
            analysis['pending'].append('全启用组合的334窗贪心/MPC补充对照')
            lines += ['全启用组合的334窗贪心/MPC补充对照尚未齐备，暂不把60窗近乎持平的结论扩大为全年结论。','']
            if 'linear_reconstruction' in alg and 'stochastic_mpc' in alg:
                alt=alg['linear_reconstruction'];base=alg['stochastic_mpc']
                analysis['pv_reconstruction_sensitivity']=savings(alt['cash_cost_yuan'],base['cash_cost_yuan'])
                lines += ['用户交付文件与同发行版本直接线性复原不完全相同，因此另做PV输入来源敏感性。'+
                    comparison_sentence('直接线性复原PV','用户交付
PV',alt['cash_cost_yuan'],base['cash_cost_yuan'])+
                    '该对照保持其他条件一致，不擅自替换主输入，也不能仅凭费用差异认定哪份预测生成过程有未来信息泄露。','']

    dev_rows=completed_rows(root/'sensitivity'/'development');dev={r['strategy']:r for r in dev_rows}
    analysis['development_sensitivity']=dev
    lines += ['## 六、场景数、随机种子与初态','']
    scenes=[dev[n] for n in ('scenes_4','scenes_8','scenes_16') if n in dev]
    if scenes:
        lines += _alg_table(scenes)
        lines += ['场景数试验只用2月17-23日七个开发合同，三组初态与末端规则一致。表中4/8/16为请求上限，'
            '有效历史不足时使用可用数量；不重复同一轨迹伪造更多独立信息。时间为签单、MPC和精修合计，受并发负载影
响，不是隔离硬件基准。','']
        if 'scenes_8' in dev:
            for n in ('scenes_4','scenes_16'):
                if n in dev:
                    lines += [comparison_sentence(n.replace('scenes_', '')+'场景','8场景',dev[n]
['cash_cost_yuan'],dev['scenes_8']['cash_cost_yuan'])+
                        f" 总求解时间分别为{number(dev[n]['solver']['seconds'])}和
{number(dev['scenes_8']['solver']['seconds'])}秒。",'']
        else:
            analysis['pending'].append('场景数开发敏感性');lines+=['场景数敏感性尚未完成。','']

```

```

seeds=[r for r in dev_rows if r['strategy'].startswith('seed_')]
if 'scenes_8' in dev and seeds:
    sample=[dev['scenes_8'],*seeds]
    lines += _alg_table(sample)
    cash=[r['cash_cost_yuan'] for r in sample]
    span=max(cash)-min(cash)
    analysis['seed_cash_range_yuan']=span
    lines += [f'同为8场景的{len(sample)}个种子，七窗现金费用跨度为{money(span)}元。'
              '该结果用于检验抽样稳定性，没有按事后费用挑种子；它不证明全年结果对所有随机种子不敏感。','']
initial=[alg[n] for n in ('initial_1200','stochastic_mpc','initial_10800') if n in alg]
if len(initial)==3:
    lines += _alg_table(initial)
    analysis['initial_soc_sensitivity']=initial
    lines += ['三种初态各自独立运行相同60窗，期末规则相同；不能直接把全年主轨迹截前60窗替代局部6000基线。'
              '比较初态时应同时看现金与库存修正指标。这只检查同一丰富组合的初态影响，没有证明所有初态下八组合冠军不'
              '变。','']
else:
    analysis['pending'].append('三初态60窗对照');lines+=['三种初态的完整匹配对照尚未齐备。','']

boundaries,bpath=_first_json([root/'boundaries'/'summary.json',root/'sensitivity'/'boundaries'/'summary.json'])
lines += ['## 七、首尾边界与执行近似','']
if boundaries:
    analysis['boundaries']=boundaries
    starts=boundaries.get('startup',[]);final=boundaries.get('final_endpoint',[])
    if starts:
        lines += table(['启动段负载补法','预测负载 (kW)','启动段费用 (元)','相对持平变化 (元)'],

[[r['method'],number(r['load_forecast_estimated_kw']),money(r['cash_cost_yuan']),money(r['change_from_hold_yuan'])] for r in starts])
        lines += ['启动段电池静置，因而这些补法不改变已确认的首合同00:10初态，也不改变其后的任何订单；差额是八组相同的共同边界费用。','']
    if final:
        groups=defaultdict(list)
        for row in final:groups[row['strategy']].append(row)
        lines += table(['组合','末端三补法现金跨度 (元)','末态跨度 (kWh)'],
                        [[name,money(max(r['revised_contract_cash_total_yuan'] for r in rr))-min(r['revised_contract_cash_total_yuan'] for r in rr)),
                         number(max(r['soc_end_kwh'] for r in rr)-min(r['soc_end_kwh'] for r in rr))] for
name,rr in sorted(groups.items())])
        ranks={m:r['strategy'] for r in sorted((r for r in final if r['method']==m),key=lambda r:r['revised_contract_cash_total_yuan'])}
        for m in ('hold','linear2','ols6')
        stable=all(rank==ranks['hold'] for rank in ranks.values())
        analysis['endpoint_cash_ranks']=ranks;analysis['endpoint_cash_ranks_unchanged']=stable
        if complete and len(groups)==8:
            lines += ['三种末端补法下，八组合现金排名保持相同。' if stable else '末端补法改变了至少部分现金排名，应对接近的组合保留排序不确定性。'],'']
            lines += ['最后缺失端点仍是估计。敏感性固定原g、正式q和末段初态，只重新执行末段；它不是另一轮全年优化。','']
        else:
            analysis['pending'].append('首尾边界敏感性');lines+=['首尾持平、线性和OLS边界敏感性尚待完整主轨迹后生成。','']
causal,cpath=_first_json([root/'causal'/'summary.json',root/'causal_selected'/'summary.json',
root/'sensitivity'/'causal'/'summary.json',root/'causal_pilot'/'summary.json'])
if causal:
    analysis['causal_execution']=causal;analysis['causal_source']=cpath
    cs=causal.get('summary',{})
    if '10min_mean' in cs and '1min_causal' in cs:
        oldc,newc=cs['10min_mean'],cs['1min_causal']
        lines += table(['执行方法','区间数','现金费用 (元)','紧急电量 (kWh)','期末库存 (kWh)'],

[[name,r['intervals'],money(r['cash_cost_yuan']),number(r['emergency_kwh']),number(r['soc_end_kwh'])]
for name,r in [(('10分钟均值快反馈',oldc),('1分钟因果仿真',newc))])
        lines += [f'此对照来自{causal.get('source_strategy')}固定合同局部{causal.get('days')}窗，"

```

```

        '只在原来的正式签单时刻揭示相应订单，两种执行方法共用同一组g/q及相同局部末端规则。'+
        comparison_sentence('1分钟因果仿真','10分钟均值快反
    馈',newc['cash_cost_yuan'],oldc['cash_cost_yuan']),
        '', '分钟曲线是按原始端点线性构造的仿真，采用一分钟左端功率保持，存在离散化能量差异。'
        '因此差额同时反映执行规则与离散近似，不是高频实测验证，更不能据这几个窗口宣布全年组合排名已经被因
    果分钟回放证明。','')
    if newc['cash_cost_yuan']>oldc['cash_cost_yuan']+1e-5:
        lines += ['该对照提示10分钟均值反馈费用可能偏乐观，报告主费用时必须保留执行近似说明。','']
    if 'causal_pilot' in str(cpath):analysis['pending'].append('选定组合的预定较长因果分钟对照（目前只有
    pilot）')
    else:
        analysis['pending'].append('因果分钟执行敏感性');lines+=['因果分钟执行敏感性尚未完成，当前主费用仍属于10分
    钟均值快反馈仿真口径。','']

    timing=[]
    for dirname,label in [(('causal_H0','只启用0点'),('causal_old_forecast','全时刻调单但只读0点版'),
    ('causal_selected','开发选定组合')):
        item=read_json(root/dirname/'summary.json',{})
        if item:
            ss=item['summary'];a=ss['10min_mean'];b=ss['1min_causal']
            timing.append([label,money(a['cash_cost_yuan']),money(b['cash_cost_yuan']),
                number(a['soc_start_kwh']),number(b['soc_end_kwh'])])

    if len(timing)==3:
        analysis['timing_comparison']=timing
        lines += ['**同一七窗的执行时序补充比较: **','']
        lines += table(['策略','10分钟快反反馈费（元）','1分钟因果仿真费（元）','局部初态（kWh）','分钟末态
    (kWh)'],timing)
        lines += ['各策略固定自己的正式合同，局部初态取各自自主轨迹该日起点，未把未来签单提前透露给控制器。'
            '该表是从各自历史状态开始的条件敏感性，初态并不完全相同；它不是三条全年分钟级闭环重新优化，'
            '也不把七窗差额外推成全年费用。','']

    lines += ['## 八、求解状态与结论边界','']
    if rows:
        stats=[]
        for r in rows:
            ss=r.get('solver',{})
            events=read_json(main/r['strategy']/'event_log.json',{}).get('events',[])
            unproven=sum(e.get('status')!='optimal_within_requested_gap' for e in events)
            stats.append([schedule(r),ss.get('solves','待完成'),ss.get('unproven','待完成'),

len(events),unproven,number(float(ss.get('max_gap',0))*100,6),number(ss.get('seconds'))])
        lines += table(['组合','总求解次数','未证请求精度次数','签单事件数','其中未证次数','最大相对Gap（%）','总
    求解秒数'],stats)
        lines += ['总求解统计包含日前、日内签单和电池控制，未证次数不是“未证最优的天数”。每次接受的解经过整数模式精修和
    物理核验：'

            '相对Gap使用原MILP全局下界，不能把固定模式LP的最优当原MILP全局最优。','']
    lines += ['这是一套在给定的输入、合同窗口、退款解释、实验初态和执行近似下的可复现策略。'
        '随机规划中的未来补救含两阶段预见性近似，用户预测生成的训练因果性也尚待队伍代码佐证。'
        '四个指定日期应从连续账本按自然日提取，不单独重置电池求解；原模板、费用与结果图必须使用同一版轨迹。','']
    if analysis['pending']:
        lines += ['本文件仍待补充的结果：',',',*[f'- {item}。' for item in analysis['pending']],',']
    out.parent.mkdir(parents=True,exist_ok=True)
    out.write_text('\n'.join(lines),encoding='utf-8')
    result_path=out.parent/'analysis_summary.json'
    result_path.write_text(json.dumps(analysis,ensure_ascii=False,indent=2),encoding='utf-8')
    return analysis

if __name__=='__main__':
    ap=argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--results',required=True,type=Path)
    ap.add_argument('--out',required=True,type=Path)
    args=ap.parse_args()
    result=build_conclusions(args.results,args.out)

```

```
print(json.dumps({'complete_main':result['complete_main'],'selected':result['selected_strategy'],  
                  'pending':result['pending'],'output':str(args.out)},ensure_ascii=False))
```

45. question3/planning/code/q3_figures.py

源文件 SHA-256: 1d592fd9c28261d3c7105f3daf3a54994b80131278891111072643c49e29a8ba ; 代码行数: 95。

```

"""Reproducible plots from frozen ledgers; never reruns or changes dispatch."""
import os
os.environ.setdefault('MPLCONFIGDIR', '/tmp/cumcm_q3_matplotlib')
import argparse, json
from pathlib import Path
import numpy as np, pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from q3_io import load_inputs
from q3_risk import HOURS, START

def build_figures(data_dir, results_dir, out_dir):
    out=Path(out_dir);out.mkdir(parents=True,exist_ok=True);root=Path(results_dir)
    plt.rcParams.update({'font.sans-serif':['Arial Unicode MS','Heiti TC','DejaVu Sans'],
        'axes.unicode_minus':False,'font.size':10,'axes.spines.top':False,'axes.spines.right':False,
        'figure.dpi':130,'savefig.dpi':180})
    meta=[]
    def save(fig,name,caption):
        fig.tight_layout();fig.savefig(out/(name+'.png'),bbox_inches='tight')
        fig.savefig(out/(name+'.pdf'),bbox_inches='tight');plt.close(fig)
        meta.append(dict(file=name+'.png',caption=caption))
    data=load_inputs(data_dir)
    fig,ax=plt.subplots(figsize=(8,3.8));positions=np.arange(3);old=[];new=[]
    for k,h in enumerate(HOURS[1:],1):
        start=START[h];actual=data['pv_a'][:,start:]
        old.append(np.abs(data['pv_f'][:,0,start:]-actual).mean())
        new.append(np.abs(data['pv_f'][:,k,start:]-actual).mean())
    ax.bar(positions-.17,old,.34,label='仍使用0点预报',color='#829bb1')
    ax.bar(positions+.17,new,.34,label='使用该时刻新预报',color='#168a87')
    ax.set(xticks=positions,xticklabels=['6点至合同末','12点至合同末','18点至合同末'],ylabel='同一目标范围光伏
    MAE (kW)')
    ax.legend();save(fig,'01_新旧预报同范围误差','每组柱子比较完全相同的目标时段；三组覆盖范围不同，不能直接按柱高比
    较发布时间优劣。')
    selection=json.loads((root/'main/selection.json').read_text());chosen=selection['selected_strategy']
    summaries=json.loads((root/'main/comparison.json').read_text())

    s=pd.DataFrame(summaries).sort_values('cash_cost_yuan');labels=s.strategy.str.replace('H',' ',regex=False)
    ).str.replace('_', '+', regex=False)
    fig,ax=plt.subplots(figsize=(9,4));colors=['#168a87' if x==chosen else '#9caeab' for x in
    s.strategy]
    bars=ax.barh(labels,s.cash_cost_yuan/1e4,color=colors);ax.invert_yaxis();ax.set_xlabel('334个合同窗口
    实际现金费用（万元）')
    for b,v in
    zip(bars,s.cash_cost_yuan):ax.text(b.get_width()+1,b.get_y()+b.get_height()/2,f'{v/1e4:.2f}',va='center'
    ,fontSize=9)
    ax.set_xlim(0,s.cash_cost_yuan.max()/1e4*1.12)
    save(fig,'02_八组合完整费用','绿色为开发期选定组合；全年最低仅作回溯排名。334窗从2月1日00:10至次年1月1日
    00:10。')
    daily={x['strategy']:pd.read_csv(root/'main'/x['strategy']/'daily.csv',parse_dates=['window_date'])
    for x in summaries}
    fig,ax=plt.subplots(figsize=(9,3.8))
    for name,col in [('H0','#7f91a5'),(chosen,'#168a87')]:
        ds=daily[name].set_index('window_date').cash_cost_yuan.resample('MS').sum()/1e4
        ax.plot(ds.index.month,ds.values,'o-',label=name.replace('H',' ').replace('_', '+'),color=col)
    ax.set(xlabel='合同开始月份',ylabel='合同窗口费用（万元）',xticks=range(2,13));ax.legend()
    save(fig,'03_逐月费用','同一输入、初始SOC、物理条件；每条策略的SOC独立连续。月份按合同开始日归属。')
    fig,ax=plt.subplots(figsize=(9,3.8));order=s.strategy.tolist();x=np.arange(len(order))
    ordinary=[next(v for v in summaries if v['strategy']==n)['original_plan_cost_yuan']+next(v for v in
    summaries if v['strategy']==n)['net_adjustment_cost_yuan'] for n in order]
    emergency=[next(v for v in summaries if v['strategy']==n)['emergency_cost_yuan'] for n in order]
    ax.bar(x,np.array(ordinary)/1e4,label='普通电结算（含退款、违约及增购）',color='#8fa6b4')
    ax.bar(x,np.array(emergency)/1e4,bottom=np.array(ordinary)/1e4,label='紧急购电',color='#dc8a50')
    ax.set(xticks=x,xticklabels=[n.replace('H',' ').replace('_', '+') for n in order],ylabel='实际现金费用

```

```

(万元)');ax.legend(fontsize=8)
save(fig, '04_实际电费组成', '未来暂定调单及终端库存软惩罚不计入现金账单。')
trace=pd.read_csv(root/'main'/chosen/'trace.csv', parse_dates=['interval_start', 'interval_end'])
dates=['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
fig, axes=plt.subplots(2, 2, figsize=(11, 6), sharex=True, sharey=True)
for ax, date in zip(axes.flat, dates):
    d=trace[trace.interval_start.dt.strftime('%Y-%m-%d')==date];hours=d.interval_start.dt.hour+d.interval_start.dt.minute/60

ax.plot(np.r_[hours.values, 24], np.r_[d.soc_start_kwh.values, d.soc_end_kwh.iloc[-1]], color='#168a87')
ax.axhline(1200, c='#dc8a50', ls='--', lw=.8);ax.axhline(10800, c='#dc8a50', ls='--', lw=.8)
ax.set(title=date, xlabel='自然日时间 (h)', ylabel='储电量 (kWh)', xticks=[0, 6, 12, 18, 24], ylim=
(0, 12000))
save(fig, '05_四日真实储电轨迹', '自然日00:00首段接前一合同, 连续至24:00; 不存在每日重置6000。')
fig, axes=plt.subplots(2, 2, figsize=(11, 6), sharex=True)
for ax, date in zip(axes.flat, dates):
    d=trace[trace.interval_start.dt.strftime('%Y-%m-%d')==date];hours=d.interval_start.dt.hour+d.interval_start.dt.minute/60
    ax.plot(hours, d.g_kwh*6, label='0点原计划', lw=1, color='#a4afb8')
    ax.plot(hours, d.q_kwh*6, label='正式普通购电', lw=1, color='#168a87')
    ax.fill_between(hours, d.emergency_kwh*6, color='#dc8a50', alpha=.7, label='紧急电')
    ax.set(title=date, xlabel='自然日时间 (h)', ylabel='等效平均功率 (kW)', xticks=[0, 6, 12, 18, 24])
    axes.flat[0].legend(fontsize=8);save(fig, '06_四日购电与应急', '电量乘6仅为画等效平均功率; 账本仍保存kWh, 不重
复换算费用。')
    path=root/'sensitivity/algorithms/comparison.json'
    if path.exists():
        ss={r['strategy']:r for r in json.loads(path.read_text())}
        keys=['deterministic_mpc', 'stochastic_greedy', 'stochastic_mpc'];names=['点预测+MPC', '场景+贪
心', '场景+MPC']
        fig, ax=plt.subplots(figsize=(8, 3.8));xx=np.arange(3)
        ax.bar(xx-.18, [ss[k]['cash_cost_yuan']/1e4 for k in keys], .36, label='真实现金费', color='#8fa6b4')
        ax.bar(xx+.18, [ss[k]['inventory_adjusted_cost_yuan']/1e4 for k in keys], .36, label='附库存调整比较
值', color='#168a87')
        ax.set(xticks=xx, xticklabels=names, ylabel='60合同窗口 (万元)');ax.legend()
        save(fig, '07_场景层与控制层对照', '预设全时刻更新、2月25日起60窗、共同局部初态6000; 只作60窗算法检验, 不冒称
全年。')
        path=root/'sensitivity/development/comparison.json'
        if path.exists():
            ss={r['strategy']:r for r in json.loads(path.read_text())};keys=
['scenes_4', 'scenes_8', 'scenes_16']
            fig, axs=plt.subplots(1, 2, figsize=(9, 3.5));xx=[4, 8, 16]
            axs[0].plot(xx, [ss[k]['inventory_adjusted_cost_yuan']/1e4 for k in keys], 'o-', c='#168a87')
            axs[0].set(xlabel='请求场景数', ylabel='开发7窗库存调整费用 (万元)', xticks=xx)
            axs[1].plot(xx, [ss[k]['elapsed_seconds'] for k in keys], 'o-', c='#dc8a50')
            axs[1].set(xlabel='请求场景数', ylabel='7窗计算耗时 (秒)', xticks=xx)
            save(fig, '08_场景数敏感性', '16条不足时按可用历史数计算; 8为预设主参数, 本图不用于回头按测试结果选参。耗时受同
机并行负载影响。')
            (out/'figure_manifest.json').write_text(json.dumps(meta, ensure_ascii=False, indent=2))
            return meta

if __name__=='__main__':
    a=argparse.ArgumentParser();a.add_argument('--data', required=True);a.add_argument('--
results', required=True);a.add_argument('--out', required=True);args=a.parse_args()
    build_figures(args.data, args.results, args.out)

```

46. question3/planning/code/q3_guard_review.py

源文件 SHA-256: afd18909b79fb22e9e6d8638e71e16053f1fa9956de4bfb9ec4b9cfd4b39a03e ; 代码行数: 148。


```

"""Independent evidence for IO/cache guard changes, without annual re-solving."""
import ast
import copy
import csv
import hashlib
import json
from pathlib import Path
import shutil
import tempfile

import numpy as np
from openpyxl import load_workbook

from q3_audit import NUMERIC, audit_run
from q3_runner import run_strategy, signature

ROOT=Path(__file__).resolve().parents[1]

def sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()
def read(path):return json.loads(Path(path).read_text(encoding='utf-8'))
def rows(path):
    with Path(path).open(encoding='utf-8-sig',newline='') as stream:return list(csv.DictReader(stream))

def review(project=ROOT):
    project=Path(project);data=project/'data';prov=project/'provenance';code=project/'code'
    before=project/'results/guard_regression/before/guard_rich'
    after=project/'results/guard_regression/after/guard_rich'
    bs,ns=read(before/'summary.json'),read(after/'summary.json')
    bt,nt=rows(before/'trace.csv'),rows(after/'trace.csv')
    if bs['config']!=ns['config'] or len(bt)!=432 or len(nt)!=432:
        raise RuntimeError('Regression requires identical settings and 3 complete windows')
    columns={key:float(np.max(np.abs(np.asarray([float(r[key]) for r in bt])-
                                                np.asarray([float(r[key]) for r in nt])))) for key in NUMERIC}
    text_keys=set(bt[0])-set(NUMERIC)
    texts=all(all(a[k]==b[k] for k in text_keys) for a,b in zip(bt,nt))
    equivalent=texts and max(columns.values())<=1e-7
    hashes={name:{'before':sha(prov/'numerical_core_used'/name),'after':sha(code/name)}
            for name in ('q3_io.py','q3_optimizer.py','q3_risk.py','q3_runner.py')}
    def functions(path):
        return {n.name:n for n in ast.parse(Path(path).read_text()).body if isinstance(n,
        (ast.FunctionDef,ast.AsyncFunctionDef))}
    old=functions(prov/'numerical_core_used'/q3_runner.py);new=functions(code/'q3_runner.py')
    changed=[name for name in old if name not in new or
    ast.dump(old[name],include_attributes=False)!=ast.dump(new[name],include_attributes=False)]
    additions=sorted(set(new)-set(old))
    stripped=copy.deepcopy(new[run_strategy])
    stripped.body=[node for node in stripped.body if not(isinstance(node,ast.Expr) and
    isinstance(node.value,ast.Call)
    and isinstance(node.value.func,ast.Name) and node.value.func.id=='validate_run_arguments'))]
    run_unchanged=ast.dump(stripped,include_attributes=False)==ast.dump(old[run_strategy],include_attri
    butes=False)
    other_cores_same=all(hashes[name]['before']==hashes[name]['after'] for name in hashes if
    name!='q3_runner.py')
    ba,na=audit_run(before),audit_run(after)
    for directory,result in ((before,ba),(after,na)):
        (directory/'independent_audit.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding=
        'utf-8')
        source_names=('attachment1.xlsx','attachment2.xlsx','attachment3.xlsx',
        'load_forecasts.xlsx','pv_forecasts.xlsx','result3_template.xlsx','source_manifest.json')

```

```

original_hashes={name:sha(data/name) for name in source_names}
target_before=sha(after/'summary.json')
with tempfile.TemporaryDirectory(prefix='q3-cache-proof-') as temporary:
    copied=Path(temporary)
    for name in source_names:shutil.copy2(data/name,copied/name)
    sig_before=signature(copied,ns['config'])
    cached=run_strategy(copied,after.parent,(0,6,12,18),start_day=16,days=3,name='guard_rich')
    original_cache_reused=cached['run_signature']==ns['run_signature']==sig_before
    book=load_workbook(copied/'pv_forecasts.xlsx')
    sheet=book['00时预报'];previous=float(sheet['B2'].value);sheet['B2']=previous+123.45
    book.save(copied/'pv_forecasts.xlsx');book.close()
    manifest_unchanged=sha(copied/'source_manifest.json')==original_hashes['source_manifest.json']
    sig_after=signature(copied,ns['config'])
    rejected=False;message=None
    try:
        run_strategy(copied,after.parent,(0,6,12,18),start_day=16,days=3,name='guard_rich')
    except RuntimeError as exc:
        message=str(exc);rejected='Incompatible existing run' in message
    cache={'identical_copied_xlsx_reused_existing_result':original_cache_reused,
        'actual_modified_workbook':'temporary pv_forecasts.xlsx, 00时预报!B2',
        'old_cell_value':previous,'new_cell_value':previous+123.45,
        'manifest_bytes_unchanged':manifest_unchanged,
        'fingerprint_changed':sig_before!=sig_after,
        'cached_result_reuse_rejected':rejected,'rejection_message':message,
        'original_result_summary_unchanged':sha(after/'summary.json')==target_before,
        'private_mutation_fixture_removed':True}
    inputs_unchanged=all(sha(data/name)==digest for name,digest in original_hashes.items())
    manifest_snapshot=sha(data/'source_manifest.json')==sha(prov/'source_manifest.json')
    result={'passed':equivalent and other_cores_same and run_unchanged and ba['passed'] and na['passed']
        and all(cache[k] for k in
('identical_copied_xlsx_reused_existing_result','manifest_bytes_unchanged',
    'fingerprint_changed','cached_result_reuse_rejected','original_result_summary_unchanged'))
        and inputs_unchanged and manifest_snapshot,
        'change_scope':'Only input/cache guards; no optimization equations, price rules, scenarios or
execution equations changed',
        'core_hashes':hashes,'changed_existing_runner_functions':changed,

'added_runner_functions':additions,'run_strategy_same_after_removing_validation_call':run_unchanged,
    'other_three_numerical_modules_byte_identical':other_cores_same,
    'regression':{'strategy':'all 0/6/12/18 enabled; 8 scenarios; MPC','start_day':16,'windows':3,
        'intervals':432,'start':bt[0]['interval_start'],'end':bt[-1]['interval_end'],
        'config':bs['config'],'same_text_fields':texts,'max_numeric_difference_by_column':columns,
        'max_numeric_difference':max(columns.values()),'comparison_tolerance':1e-7,
        'before_cash_yuan':bs['cash_cost_yuan'],'after_cash_yuan':ns['cash_cost_yuan'],
        'before_final_soc_kwh':bs['soc_end_kwh'],'after_final_soc_kwh':ns['soc_end_kwh'],
        'before_signature':bs['run_signature'],'after_signature':ns['run_signature'],
        'before_trace_sha256':sha(before/'trace.csv'),'after_trace_sha256':sha(after/'trace.csv'),
        'both_independent_ledger_audits_passed':ba['passed'] and na['passed']},
        'cache_rejection_test':cache,'original_input_files_unchanged':inputs_unchanged,
        'original_source_manifest_exact_snapshot_matches':manifest_snapshot,
        'full_primary_calculation_rerun':False,
        'why_no_annual_resolve':'The valid-input numerical path is structurally unchanged, the
optimizer/risk/adaptor bytes match, and the 3-window regression is equivalent; only cache key and
rejection of invalid arguments changed.',
        'reproduction_note':'Use the original numerical_core_used snapshot for historical code replay,
or the guarded current entrypoint into a NEW output directory. Time-limited MILP can vary with runtime
load.']}

(prov/'io_guard_change.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-
8')

text=f'''# 代码版本与复现说明

```

本次全年主结果使用的原始数值核心已经完整保存于`provenance/numerical\_core\_used`，对应四个文件的SHA256写在`numerical\_core\_manifest.json`；参与原运行签名的`source\_manifest.json`也保存了精确字节副本。

全年主计算和两个消融均已结束后，只对当前入口增加了两类保护：实际六个XLSX文件的字节加入缓存指纹；非法时刻、配置和初态在计算

前被拒绝。原来的缓存仅包含来源清单，单独替换预测XLSX而不改清单可能错误复用旧结果，这次已经修复。小时标签必须为真正整数，浮点和布尔值不再混入。

数学与数值过程是否改变

`q3\_io.py`、`q3\_risk.py`、`q3\_optimizer.py`与原快照逐字节相同。`q3\_runner.py`只改了缓存签名、增加参数校验函数，并在入口调用它。移除这一校验调用后，运行主函数的抽象语法树与原版相同；没有改变MILP方程、风险场景、调单费用、电池动作或时间循环。

采用0/6/12/18全部启用、8场景、相同初态，从2月17日00:10起分别用原快照和新入口独立运行3个合同、432段。两版现金费均为 ****{bs['cash_cost_yuan']:.9f}元****，末态均为 ****{bs['soc_end_kwh']:.9f} kWh****；全部数值列的最大差为 ****{max(columns.values()):.12g}****，文本时间与版本字段也相同。两份轨迹均通过独立物理与账本检查。

另外复制真实PV工作簿到临时目录，修改其中一个预测单元格，同时保持来源清单字节不变。原样副本能够正确使用缓存；修改后的副本触发新指纹，程序明确拒绝复用旧结果。原始六个XLSX及既有结果没有被修改，临时错误输入已经清理。

既有全年结果的适用状态

****这次只做了上述3窗对照与缓存反例，没有重新计算334窗主结果。合法输入下数值路径未变，因而缓存保护修复不要求对未受影响的全年数学结果重算。主结果仍应标注其原始核心版本，不冒称由修复后的入口重新跑出。**

8条主轨迹共384768段、2条消融共96192段的逐段审计已完成；原始输入哈希、冻结核心哈希及历史运行签名均可追溯。详见`results/audit\_main\_validation.json`。具体补丁、逐列回归差异和缓存拒绝证据见`provenance/io\_guard\_change.json`。

怎样复现

核对这次保存结果的原始代码时，使用`provenance/numerical\_core\_used`内的四个原文件。它们保留当时行为，只用于历史来源与数值复核；旧缓存不足也一并保留，不建议作为日常替换预测后的入口。

日常复算使用当前`code/run\_all.py`。默认输出目录已经改为`results/reproduced`；也可以显式选择一个新目录：

```
```bash
python code/run_all.py --out results/reproduced_01
```
```

当前代码的版本与缓存指纹有意发生变化。不要把既有`results/main`当成新入口可以原样继承的缓存；出现“不兼容的既有结果”时，应选新目录，而不是修改原结果以绕过检查。

固定随机种子控制的是历史场景抽样，不保证含求解时限的MILP在所有机器上逐位一致。复现时还要记录求解器版本、时间预算、可行性与Gap；并发负载可能影响截止时已经找到的可行解。上述3窗对照是本机同环境下的等价验证，不承诺所有年度输出在所有硬件上完全一致。

```
'''
    (project/'notes/代码版本与复现.md').write_text(text,encoding='utf-8')
    print(json.dumps({'passed':result['passed'],'max_difference':max(columns.values()),

'cash':bs['cash_cost_yuan'],'cache_rejected':cache['cached_result_reuse_rejected']},ensure_ascii=False))
    return result

if __name__=='__main__':
    raise SystemExit(0 if review()['passed'] else 1)
```

47. question3/planning/code/q3_io.py

源文件 SHA-256: 3bd8d306c4314029ac76632274eeb10c38f622f577d521a240e49b9dce50b696 ; 代码行数: 397。

```

"""Q3 strict data adapter: interval means, forecast vintages and observed nodes.

User workbooks are read-only. The returned power arrays are kW; optimization
must multiply them by 1/6 exactly once. The contract window is D 00:10 to
D+1 00:10. No actual value is implicitly substituted for a supplied forecast.
"""

from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import json
import re
import shutil

import numpy as np
from openpyxl import load_workbook

ROOT = Path(__file__).resolve().parents[1]
ISSUE_HOURS = (0, 6, 12, 18)
ISSUE_START = {0: 0, 6: 35, 12: 71, 18: 107}
WECHAT = Path('/Users/xzy/Library/Containers/com.tencent.xinWeChat/Data/Documents/xwechat_files/wxid_euf4op8bmams22_1b0d/temp/RWTemp/2026-09/a3bee7770c01c7d37f45899f94a1dcc5')
ATTACHMENTS = Path('/Users/xzy/Downloads/CUMCM2026Problems/C题/附件')
SOURCES = {
    'pv_forecasts.xlsx': WECHAT / '问题三_光伏区间预测.xlsx',
    'load_forecasts.xlsx': WECHAT / '问题二_标准化区间预测.xlsx',
    'attachment1.xlsx': ATTACHMENTS / '附件1.xlsx',
    'attachment2.xlsx': ATTACHMENTS / '附件2.xlsx',
    'attachment3.xlsx': ATTACHMENTS / '附件3.xlsx',
    'result3_template.xlsx': ATTACHMENTS / '附件5/result3.xlsx',
}

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def as_date(value):
    if isinstance(value, dt.datetime):
        return value.date()
    if isinstance(value, dt.date):
        return value
    return dt.date(*map(int, str(value).split('-')))

def clock_minute(value):
    if isinstance(value, dt.time):
        return value.hour * 60 + value.minute
    s = str(value).strip().replace(':', ':')
    tomorrow = ('次日' in s) or ('+1' in s)
    s = s.replace('次日', '').replace('+1', '')
    m = re.fullmatch(r'(\d{1,2}):(\d{1,2})', s)
    if not m:
        raise ValueError(f'Invalid clock: {value!r}')
    h, minute = map(int, m.groups())
    if h > 24 or minute > 59 or (h == 24 and minute):
        raise ValueError(f'Invalid clock: {value!r}')
    return h * 60 + minute + (1440 if tomorrow and h < 24 else 0)

def interval_header(value):

```

```

pieces = re.split(r'[-~\-\-]', str(value).strip())
if len(pieces) != 2:
    raise ValueError(f'Explicit interval required: {value!r}')
a, b = map(clock_minute, pieces)
if a == 1430 and b == 0:
    b = 1440
if b - a != 10:
    raise ValueError(f'Interval is not 10 minutes: {value!r}')
return a, b


def validate_dates(values, label, *, expected=None):
    dates = [as_date(v) for v in values]
    if dates != sorted(set(dates)):
        raise ValueError(f'{label}: dates must be unique and ascending')
    if any(b - a != dt.timedelta(days=1) for a, b in zip(dates, dates[1:])):
        raise ValueError(f'{label}: missing interior date')
    if expected is not None and dates != expected:
        raise ValueError(f'{label}: forecast dates differ')
    return dates


def numeric_array(rows, shape, label):
    try:
        a = np.asarray(rows, dtype=float)
    except (ValueError, TypeError) as exc:
        raise ValueError(f'{label}: nonnumeric values') from exc
    if a.shape != shape or not np.isfinite(a).all() or (a < 0).any():
        raise ValueError(f'{label}: missing, negative, or invalid data; shape={a.shape}')
    return a


def read_load(path):
    w = load_workbook(path, read_only=True, data_only=True)
    try:
        rows = list(w['XGBoost_负载'].values)
    finally:
        w.close()
    if [interval_header(x) for x in rows[0][1:]] != [(t,t+10) for t in range(10,1441,10)]:
        raise ValueError('Load must cover 144 means from D 00:10 to D+1 00:10')
    dates = validate_dates([r[0] for r in rows[1:]], 'load')
    a = numeric_array([r[1:] for r in rows[1:]], (len(dates),144), 'load')
    return dates, a


def read_pv(path, dates):
    """Keep all supplied numerical values; crop only 00h's unused first mean.

    All four supplied sheets contain an isolated header typo at 23:50. It is
    explicitly repaired in this adapter only after verifying every other
    target, date and width. The source workbooks are never edited.
    """
    w = load_workbook(path, read_only=True, data_only=True)
    pv = np.full((len(dates),4,144), np.nan)
    valid = np.zeros((4,144), bool)
    corrections = []
    shapes = []
    first = None
    try:
        for k,h in enumerate(ISSUE_HOURS):
            sheet = f'{h:02d}时预报'
            rows = list(w[sheet].values)
            headers = list(rows[0][1:])
            expected = [(t,t+10) for t in range(h*60,1441,10)]
            if len(headers) != len(expected):
                raise ValueError(f'{sheet}: wrong forecast width')

```

```

        for j,(header,target) in enumerate(zip(headers,expected)):
            if header == '23:50-次日00:10' and target == (1430,1440):
                headers[j] = '23:50-次日00:00'
                corrections.append({'sheet':sheet,'excel_column':j+2,
                                    'original':header,'normalized':headers[j],
                                    'reason':'isolated 20-minute label overlaps the next explicit 10-minute
interval; ordered 10-minute grid and supplied load format identify intended target',
                                    'numerical_values_changed':False})
            if [interval_header(v) for v in headers] != expected:
                raise ValueError(f'{sheet}: target intervals differ from expected grid')
            validate_dates([r[0] for r in rows[1:]],sheet,expected=dates)
            a = numeric_array([r[1:] for r in rows[1:]], (len(dates),len(expected)), sheet)
            if h == 0:
                first = a[:,0].copy()
                a = a[:,1:]
                start = ISSUE_START[h]
                pv[:,k,start:] = a
                valid[k,start:] = True
                shapes.append({'sheet':sheet,'rows':len(rows),'columns':len(headers)+1,
                                'source_intervals':len(expected),'used_intervals':144-start,
                                'issued_hour':h,'target_first_minute':max(10,h*60),'target_last_end_minute':1450,
                                'power_min_kw':float(a.min()),'power_max_kw':float(a.max())})
            finally:
                w.close()
        return pv,valid,first,{'sheets':shapes,'header_corrections':corrections,
                                '00h_first_interval_cropped':True,'input_value_kind':'interval_mean_kw',
                                'averaging_of_supplied_values_performed':False}

def read_actual_source(path):
    w = load_workbook(path, read_only=True, data_only=True)
    result = {}
    dates = None
    try:
        for sheet,key in [('小区负载','load'),('光伏发电实际功率','pv')]:
            rows = list(w[sheet].values)
            if [clock_minute(v) for v in rows[0][1:]] != list(range(10,1441,10)):
                raise ValueError(f'{sheet}: must contain 144 instantaneous nodes')
            ds = validate_dates([r[0] for r in rows[1:]],sheet,expected=dates)
            dates = ds
            vals = numeric_array([r[1:] for r in rows[1:]],(len(ds),144),sheet)
            result[key] = {day: vals[i] for i,day in enumerate(ds)}
    finally:
        w.close()
    return result,dates

def boundary_candidates(y):
    x = np.arange(6,dtype=float); z = y[-6:]
    slope = float(np.dot(x-x.mean(),z-z.mean()) / np.dot(x-x.mean(),x-x.mean()))
    return {'hold':float(y[-1]),'linear2':float(max(0,2*y[-1]-y[-2])),
            'ols6':float(max(0,z.mean()+slope*(6-x.mean())))}

def attach_actuals(data,path):
    raw,source_dates = read_actual_source(path)
    n = len(data['dates'])
    estimated = np.zeros((n,144),bool)
    boundaries = {}
    for key in ('load','pv'):
        nodes = np.empty((n,145))
        for i,day in enumerate(data['dates']):
            if day not in raw[key]:
                raise ValueError('Forecast date missing from observed nodes')
            y = raw[key][day]
            nodes[i,:144] = y

```

```

        tomorrow = day + dt.timedelta(days=1)
        if tomorrow in raw[key]:
            nodes[i,144] = raw[key][tomorrow][0]
        elif day == source_dates[-1]:
            candidates = boundary_candidates(y)
            nodes[i,144] = candidates['hold']
            estimated[i,143] = True
            boundaries[key] = {method:{'right_endpoint_kw':v, 'mean_kw':float((y[-1]+v)/2)}}
                for method,v in candidates.items()
        else:
            raise ValueError('Missing interior endpoint cannot be extrapolated')
        data[f'{key}_nodes'] = nodes
        data[f'{key}_a'] = (nodes[:, :-1]+nodes[:, 1:])/2
    data['actual_estimated'] = estimated
    day0 = data['dates'][0]; prev = day0-dt.timedelta(days=1)
    if prev not in raw['load']:
        raise ValueError('Need observed previous midnight boundary for audit')
    data['first_unexecuted_interval'] = {
        'interval_start':f'{day0}T00:00:00', 'interval_end':f'{day0}T00:10:00',
        'status':'not part of the user-confirmed experiment; no automatic forecast/order/state
invented',
        **{f'{key}_{label}_kw':float(v) for key in ('load', 'pv')
            for label,v in [(('left_node', raw[key][prev][-1]), ('right_node', raw[key][day0][0]),
                            ('mean', (raw[key][prev][-1]+raw[key][day0][0])/2))]}
    data['metadata']['actual'] = {'source_kind':'instantaneous_kw',
        'mean_kind':'adjacent endpoint arithmetic mean (linear between-node approximation)',
        'cross_midnight':'last node of row D paired with first node of row D+1',
        'available_at':'interval_end for complete mean; node observation available at its timestamp',
        'estimated_interval_count':int(estimated.sum()),
        'last_right_endpoint_candidates':boundaries,
        'first_unexecuted_interval':data['first_unexecuted_interval']}
    return raw

def read_price(path):
    w=load_workbook(path, read_only=True, data_only=True)
    try:
        rows=list(w.active.values)
    finally:
        w.close()
    if len(rows)≠145 or tuple(rows[0][:2])≠('时间', '电价'):
        raise ValueError('Attachment1 dimensions/headers invalid')
    if [clock_minute(r[0]) for r in rows[1:]]≠list(range(10,1441,10)):
        raise ValueError('Attachment1 must contain instantaneous 00:10..24:00')
    p=numeric_array([r[1] for r in rows[1:]], (144,), 'price')
    if (p≤0).any():
        raise ValueError('Prices must be positive')
    natural_nodes=np.r_[p[-1], p]
    natural=(natural_nodes[:-1]+natural_nodes[1:])/2
    return np.r_[natural[1:], natural[0]], {'source_kind':'instantaneous_price',
        '00:00_boundary':p(00:00)=p(24:00), periodic daily tariff',
        'settlement':'interval mean price approximation',
        'first_contract_price':float(natural[1]), 'last_contract_price':float(natural[0])}

def read_hourly_source(path):
    w=load_workbook(path, read_only=True, data_only=True)
    try:
        rows=list(w.active.values)
    finally:
        w.close()
    if len(rows)≠1461 or len(rows[0])≠26 or list(rows[0][2:])≠[f'预报{i}小时' for i in range(1,25)]:
        raise ValueError('Attachment3 hourly source dimensions/headers invalid')
    raw={}; date=None
    for r in rows[1:]:
        if r[0]: date=as_date(r[0])

```

```

h=clock_minute(r[1])//60
if h not in ISSUE_HOURS or (date,h) in raw:
    raise ValueError('Invalid or duplicate hourly forecast issuance')
raw[date,h]=numeric_array(r[2:],(24,),'hourly forecast')
return raw

def audit_linear_reconstruction(data,path,actual_source):
    raw=read_hourly_source(path)
    linear=np.full_like(data['pv_f'],np.nan)
    rows=[]
    for k,h in enumerate(ISSUE_HOURS):
        start=ISSUE_START[h]
        for i,day in enumerate(data['dates']):
            # Only the observation AT release time, not a later actual node.
            if h==0:
                observed=actual_source['pv'][day-dt.timedelta(days=1)][-1]
            else:
                observed=actual_source['pv'][day][h*6-1]
            hourly_nodes=np.r_[observed,raw[day,h]]
            minutes=np.arange(max(10,h*60),1451,10)
            fine=np.interp(minutes,np.arange(h,h+25)*60,hourly_nodes)
            linear[i,k,start:]=(fine[:-1]+fine[1:])/2
            supplied=data['pv_f'][:,k,start:]; ref=linear[:,k,start:]
            error=supplied-ref
            rows.append({'issue_hour':h,'compared_values':int(error.size),
                'mean_absolute_difference_kw':float(np.abs(error).mean()),
                'max_absolute_difference_kw':float(np.abs(error).max()),
                'match_fraction_at_1e-7_kw':float(np.mean(np.abs(error)≤1e-7))})
    data['pv_linear_f']=linear
    data['metadata']['pv_generation_audit']={
        'finding':'supplied values are not identical to direct piecewise-linear interpolation of the
same issue hourly nodes followed by endpoint averaging',
        'comparison':rows,
        'main_input':'supplied interval means, unchanged',
        'alternative_input':'pv_linear_f reconstructed only from same-issue hourly forecast and release-
time observation',
        'linear_00h_final_boundary':'hold forecast node at next 00:00 to next 00:10, without reading a
future forecast',
        'causality_status':'supplied generator unavailable; causality not independently certified;
discrepancy alone is not evidence of future-data leakage',
        'load_generator_status':'supplied midnight forecasts; training cutoff/provenance not
independently certified'}

def inspect_template(path):
    w=load_workbook(path,read_only=True,data_only=True)
    try:
        shapes={s.title:{'rows':s.max_row,'columns':s.max_column} for s in w}
        for name in ('计划购电量','调整购电量'):
            s=w[name]; r=next(s.values)
            # The final source label 0:00-0:10+1 is a known template shorthand
            # for NEXT-day 00:00-00:10, not a 24h10m interval.
            for i,h in enumerate(r[1:145]):
                if i==143:
                    if h≠'0:00-0:10+1': raise ValueError('Unexpected final template header')
                    elif interval_header(h)≠(10*(i+1),10*(i+2)):
                        raise ValueError('Template contract intervals do not match')
            finally:
                w.close()
    return {'sheets':shapes,'purchase_sheets':'144 contract slots D 00:10 to D+1 00:10',
        'storage_sheet':'natural-day blocks and true 00:00/24:00 states; example rows include ellipsis
and must be expanded for all dates',
        'emergency_sheet':'example rows include ellipsis; export all actual episodes, split at natural-
day midnight',
        'final_purchase_header_alias':'0:00-0:10+1 means next-day 00:00-00:10',

```

```

    'first_natural_day_gap': 'first 00:00-00:10 is outside this experiment; must be explicitly marked
rather than filled with invented dispatch'}

def copy_sources(destination=ROOT/'data'):
    destination=Path(destination);destination.mkdir(parents=True,exist_ok=True)
    copied=[]
    for name,path in SOURCES.items():
        target=destination/name
        source_hash=sha(path)
        if target.exists() and sha(target)!=source_hash:
            raise ValueError(f'Refuse to overwrite a differing copied input: {target}')
        if not target.exists(): shutil.copy2(path,target)
        if sha(target)!=source_hash: raise AssertionError('Copy integrity failed')

    copied.append({'filename':name, 'original_path':str(path), 'sha256':source_hash, 'bytes':path.stat().st_size})

    (destination/'source_manifest.json').write_text(json.dumps(copied,ensure_ascii=False,indent=2),encoding='utf-8')
    return copied

def load_inputs(data_dir=ROOT/'data'):
    """Return datetime-based metadata plus numerical arrays (all power in kW).

    pv_f[day, vintage_index, slot] uses vintage indices 0/1/2/3 for hours
    0/6/12/18. Values before that vintage is released are deliberately NaN;
    downstream code must select only the released vintage's valid tail.
    """
    data_dir=Path(data_dir)
    dates,load=read_load(data_dir/'load_forecasts.xlsx')
    pv,valid,pv_first,pm=read_pv(data_dir/'pv_forecasts.xlsx',dates)
    price,price_meta=read_price(data_dir/'attachment1.xlsx')
    starts=[dt.datetime.combine(d,dt.time(0,10)) for d in dates]
    data={'dates':dates, 'window_start':starts,
        'window_end':[a+dt.timedelta(days=1) for a in starts],
        'issued_at':[a-dt.timedelta(minutes=10) for a in starts],
        'issue_hours':ISSUE_HOURS, 'issue_start':ISSUE_START.copy(),
        'load_f':load, 'pv_f':pv, 'pv_valid':valid,
        'pv_unused_first_mean_kw':pv_first, 'price':price,
        'metadata':{'window':'D 00:10 to D+1 00:10, 144 intervals',
            'time_zone':'Asia/Shanghai', 'value_kind':'interval_mean_kw',
            'energy_conversion':'multiply kW by 1/6 h exactly once; never re-average supplied means',
            'initial_soc_kwh':6000, 'initial_soc_basis':'user-prescribed common experimental state at
first 00:10; not derived February state',
            'forecast':pm, 'price':price_meta,
            'source_sha256':{'name':sha(data_dir/name) for name in SOURCES},
            'disabled_forecast_vintages':'must not be read by policy or residual generator for a
disabled issuance',
            'history_availability':'require all actual interval endpoints used by a history track ≤
current decision timestamp'}}
    actual_source=attach_actuals(data,data_dir/'attachment2.xlsx')
    audit_linear_reconstruction(data,data_dir/'attachment3.xlsx',actual_source)
    data['metadata']['template']=inspect_template(data_dir/'result3_template.xlsx')
    if dates[0]!=dt.date(2025,2,1) or dates[-1]!=dt.date(2025,12,31) or len(dates)!=334:
        raise ValueError('Current experiment expects 2025-02-01..2025-12-31, 334 windows')
    # Compatibility conveniences; these are views, never additional inputs.
    data['pv_f_by_hour']={h:pv[:,k,:] for k,h in enumerate(ISSUE_HOURS)}
    data['pv0_f']=pv[:,0,:]
    return data

def save_bundle(data,out=ROOT/'data'):
    out=Path(out);out.mkdir(exist_ok=True,parents=True)
    arrays={key:value for key,value in data.items() if isinstance(value,np.ndarray)}
```



```

arrays.update(dates=np.array([str(d) for d in data['dates']]),
              window_start=np.array([x.isoformat() for x in data['window_start']]),
              window_end=np.array([x.isoformat() for x in data['window_end']]),
              issued_at=np.array([x.isoformat() for x in data['issued_at']]),
              issue_hours=np.asarray(ISSUE_HOURS), issue_start_indices=np.asarray([ISSUE_START[h] for h in
ISSUE_HOURS]))
np.savez_compressed(out/'q3_inputs.npz', **arrays)

(out/'input_audit.json').write_text(json.dumps(data['metadata'], ensure_ascii=False, indent=2), encoding='u
tf-8')
return out/'q3_inputs.npz'

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--data', type=Path, default=ROOT/'data')
    parser.add_argument('--copy-originals', action='store_true')
    args = parser.parse_args()
    if args.copy_originals: copy_sources(args.data)
    bundle = load_inputs(args.data)
    artifact = save_bundle(bundle, args.data)
    print(json.dumps({'bundle': str(artifact), 'windows': len(bundle['dates']),
                      'load_shape': list(bundle['load_f'].shape), 'pv_shape': list(bundle['pv_f'].shape),
                      'estimated_actual_intervals': int(bundle['actual_estimated'].sum()),
                      'header_corrections': len(bundle['metadata']['forecast']
['header_corrections'])}, ensure_ascii=False))

```

48. question3/planning/code/q3_optimizer.py

源文件 SHA-256: 846926ed39be5a60f5be06a52ca5a1c27f9047da8129035fb1e2e290aaf4c8b5 ; 代码行数: 330。

```

"""Question 3 contract and battery MILP; all flows supplied in kWh.

Day-ahead orders and current adjustments are shared across scenarios. Future
adjustments and battery recourse foresee an entire scenario: this is explicitly
a rolling two-stage approximation, not a nonanticipative multistage policy.
Only ``signed_mask`` entries can be contracted by the caller. The optimizer
never records a transaction, and forecast recourse costs are not actual bills.
"""
from __future__ import annotations

from time import perf_counter
import os
import warnings

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, linprog, milp
from scipy.sparse import coo_matrix, vstack

LOW = 1200.0
HIGH = 10800.0
RATE = 5000.0 / 6.0
ETA_C = ETA_D = float(os.environ.get("MCM_BATTERY_ETA", "0.9"))
PHYSICAL_TOL = 1e-5
OBJECTIVE_TOL = 1e-4
SOLVER_TOL = 1e-9

def _vector(value, name, n=None):
    a = np.asarray(value, dtype=float).copy()
    if (a.ndim != 1 or not len(a) or (n is not None and len(a) != n)
        or not np.isfinite(a).all() or np.any(a < 0)):
        raise ValueError(f"{name} must be a finite nonnegative vector of length {n}.")
    return a

def _mask(value, name, n):
    if value is None:
        return np.zeros(n, dtype=bool)
    a = np.asarray(value)
    if a.shape != (n,) or not np.isin(a, [False, True]).all():
        raise ValueError(f"{name} must be a Boolean vector of length {n}.")
    return a.astype(bool)

def cash_components(p, base_g, delivered_q, emergency, *, adjustment_mask=None):
    """Single-settlement actual bill, separate from any soft inventory penalty.

    Supply only formally contracted ``delivered_q``; never pass future scenario
    proposals. Original charges cover all supplied g. ``adjustment_mask`` can
    restrict this particular adjustment event; otherwise all slots are billed.
    Each slot may be adjusted only once relative to base_g under this policy.
    A cancelled kWh refunds its original price and incurs a 50% penalty.
    """
    p = _vector(p, "p")
    n = len(p)
    g = _vector(base_g, "base_g", n)
    q = _vector(delivered_q, "delivered_q", n)
    e = _vector(emergency, "emergency", n)
    mask = np.ones(n, bool) if adjustment_mask is None else _mask(adjustment_mask, "adjustment_mask", n)
    increase = np.maximum(q-g, 0) * mask
    decrease = np.maximum(g-q, 0) * mask
    original = float(p @ g)
    increase_cost = float(1.5 * p @ increase)
    refund = float(p @ decrease)

```

```

penalty = .5 * refund
emergency_cost = float(5 * p @ e)
net_adjustment = increase_cost-refund+penalty
return {"original_cost": original, "increase_cost": increase_cost,
        "refund": refund, "cancellation_penalty": penalty,
        "net_adjustment_cost": net_adjustment,
        "emergency_cost": emergency_cost,
        "total_cash_cost": original+net_adjustment+emergency_cost}

def _polish(c, matrix, row_lo, row_hi, lower, upper, integer, incumbent, limit):
    """Fix exact rounded MILP modes then solve the unchanged continuous LP."""
    binary = integer.astype(bool)
    rounded = np rint(incumbent[binary])
    error = float(np.max(np.abs(incumbent[binary]-rounded)))
    if error > PHYSICAL_TOL or np.any(rounded < 0) or np.any(rounded > 1):
        raise RuntimeError(f"Unacceptable incumbent integrality error: {error}")
    lo, hi = lower.copy(), upper.copy()
    lo[binary] = hi[binary] = rounded
    equal = np.isfinite(row_lo) & np.isfinite(row_hi) & (row_lo == row_hi)
    finite_hi = np.isfinite(row_hi) & ~equal
    finite_lo = np.isfinite(row_lo) & ~equal
    start = perf_counter()
    result = linprog(c, A_ub=vstack((matrix[finite_hi], -matrix[finite_lo]), format="csc"),
                    b_ub=np.r_[row_hi[finite_hi], -row_lo[finite_lo]],
                    A_eq=matrix[equal], b_eq=row_lo[equal],
                    bounds=np.column_stack((lo, hi)), method="highs-ds",
                    options={"time_limit": float(limit), "disp": False,
                             "primal_feasibility_tolerance": SOLVER_TOL,
                             "dual_feasibility_tolerance": SOLVER_TOL})
    if result.status != 0 or result.x is None or not np.isfinite(result.x).all():
        raise RuntimeError(f"Fixed-mode LP polish failed: {result.message}")
    return result, error, perf_counter()-start

def solve_dispatch(load_scenarios, pv_scenarios, p, s0, *, mode="dayahead",
                  base_g=None, booked_q=None, locked_mask=None, current_mask=None,
                  future_adjust_mask=None, probabilities=None, terminal_weight=None,
                  s_ref=6000.0, time_limit=10.0, mip_gap=1e-4):
    """Solve a day-ahead, adjustment, or control problem with identical physics.

    Inputs load_scenarios/pv_scenarios have shape (M,n), in kWh; p is yuan/kWh.
    ``locked_mask``, ``current_mask`` and ``future_adjust_mask`` are disjoint.
    In day-ahead mode, unlocked g is shared and newly signed; locked slots use
    booked_q (e.g. the previous contract's midnight bridge) without recharging
    their sunk original fee. Only future_adjust_mask permits scenario q != g.
    In adjust mode, base_g is the immutable original contract; current_mask q
    is shared and formally adjustable now, future q is scenario-dependent, and
    all other q is booked_q. In control mode current_mask must be empty, so no
    contract can be signed. Per-slot additions cost 1.5p, cancellations net -0.5p.

    Every scenario has exact charge/discharge exclusivity; emergency energy
    cannot charge the battery. Discarding energy while discharging is excluded,
    matching the Question 2 ordinary-surplus/deficit execution convention.
    Terminal shortfall is a soft algorithmic penalty, not electricity expense.
    MILP and the mandatory fixed-mode LP each receive time_limit seconds.
    """
    start = perf_counter()
    price = _vector(p, "p")
    n = len(price)
    load, pv = np.asarray(load_scenarios, float).copy(), np.asarray(pv_scenarios, float).copy()
    if (load.ndim != 2 or load.shape[0] == 0 or load.shape[1] != n or pv.shape != load.shape
        or not np.isfinite(load).all() or not np.isfinite(pv).all()
        or np.any(load < 0) or np.any(pv < 0)):
        raise ValueError("load_scenarios/pv_scenarios must have identical finite nonnegative (M,n)
        shapes.")

```

```

m = load.shape[0]
if mode not in {"dayahead", "adjust", "control"}:
    raise ValueError("mode must be dayahead, adjust, or control.")
locked = _mask(locked_mask, "locked_mask", n)
current = _mask(current_mask, "current_mask", n)
future = _mask(future_adjust_mask, "future_adjust_mask", n)
if np.any((locked.astype(int)+current+future) > 1):
    raise ValueError("locked/current/future masks must be disjoint.")
if mode != "adjust" and current.any():
    raise ValueError("Only adjust mode may have a nonempty current_mask.")
probability = np.full(m, 1/m) if probabilities is None else _vector(probabilities, "probabilities",
m)
if abs(float(probability.sum())-1) > 1e-8:
    raise ValueError("probabilities must sum to one.")
probability /= probability.sum()
if not np.isfinite(s0) or not LOW-PHYSICAL_TOL ≤ s0 ≤ HIGH+PHYSICAL_TOL:
    raise ValueError("s0 must lie in the battery operating range.")
s0 = float(np.clip(s0, LOW, HIGH))
kappa = float(np.min(price)/ETA_C if terminal_weight is None else terminal_weight)
if (not np.isfinite(kappa) or kappa < 0 or not np.isfinite(s_ref) or s_ref < 0
    or not np.isfinite(time_limit) or time_limit ≤ 0
    or not np.isfinite(mip_gap) or not 0 ≤ mip_gap < 1):
    raise ValueError("Invalid terminal reference/weight, time_limit, or mip_gap.")
if mode != "dayahead" and base_g is None:
    raise ValueError("adjust/control mode requires immutable base_g.")
g_input = np.zeros(n) if base_g is None else _vector(base_g, "base_g", n)
if booked_q is None:
    if mode == "dayahead" and locked.any():
        raise ValueError("dayahead locked slots require booked_q.")
    booked = g_input.copy()
else:
    booked = _vector(booked_q, "booked_q", n)
adjust = future | current
sufficient = load.max(axis=0)+RATE
cap = np.maximum.reduce([sufficient, g_input, booked])
# Shared original g plus shared currently executable q; then each scene
# stores q,a,b,c,d,e,v,r,S,u,xi. Fixed/unused blocks keep interface explicit.
width = 10*n+1
size = 2*n+m*width
cost, lower, upper = np.zeros(size), np.zeros(size), np.full(size, np.inf)
integer = np.zeros(size, np.uint8)
upper[:2*n] = np.tile(cap, 2)
if mode == "dayahead":
    cost[:n] = price * ~locked
    lower[:n][locked] = upper[:n][locked] = booked[locked]
else:
    lower[:n] = upper[:n] = g_input
    lower[n:2*n][~current] = upper[n:2*n][~current] = booked[~current]
row_i, col_i, coeff, row_lo, row_hi = [], [], [], [], []

def row(entries, lo=-np.inf, hi=np.inf):
    idx = len(row_lo)
    for col, value in entries:
        if value:
            row_i.append(idx); col_i.append(col); coeff.append(value)
            row_lo.append(lo); row_hi.append(hi)

if mode == "dayahead":
    for t in range(n):
        row([(n+t, 1), (t, -1)], 0, 0)
for j in range(m):
    block = 2*n+j*width
    upper[block:block+3*n] = np.tile(cap, 3)
    upper[block+3*n:block+5*n] = RATE
    upper[block+5*n:block+6*n] = load[j]
    upper[block+6*n:block+7*n] = cap

```

```

upper[block+7*n:block+8*n] = pv[j]
lower[block+8*n:block+9*n] = LOW
upper[block+8*n:block+9*n] = HIGH
upper[block+9*n:block+10*n] = 1
integer[block+9*n:block+10*n] = 1
xi = block+10*n
upper[xi] = max(s_ref-LOW, 0)
cost[block+n:block+2*n] = probability[j]*1.5*price
cost[block+2*n:block+3*n] = -probability[j]*.5*price
cost[block+5*n:block+6*n] = probability[j]*5*price
cost[xi] = probability[j]*kappa
for t in range(n):
    q, a, b, c, d, e, v, r, s, u = (block+k*n+t for k in range(10))
    if adjust[t]:
        row([(q, 1), (t, -1), (a, -1), (b, 1)], 0, 0)
        row([(b, 1), (t, -1)], hi=0)
    else:
        upper[a] = upper[b] = 0
    if not future[t]:
        row([(q, 1), (n+t, -1)], 0, 0)
    L, G = float(load[j, t]), float(pv[j, t])
    row([(q, 1), (c, -1), (d, 1), (e, 1), (v, -1), (r, -1)], L-G, L-G)
    state = [(s, 1), (c, -ETA_C), (d, 1/ETA_D)]
    if t:
        state.append((s-1, -1))
    row(state, s0 if t == 0 else 0, s0 if t == 0 else 0)
    row([(c, 1), (u, -RATE)], hi=0)
    row([(d, 1), (u, RATE)], hi=RATE)
    row([(e, 1), (u, L)], hi=L)
    row([(v, 1), (r, 1), (u, -(cap[t]+G))], hi=0)
    row([(v, 1), (q, -1)], hi=0)
    row([(block+9*n-1, -1), (xi, -1)], hi=-s_ref)
matrix = coo_matrix((coeff, (row_i, col_i)), shape=(len(row_lo), size)).tocsc()
row_lo, row_hi = np.asarray(row_lo), np.asarray(row_hi)
mip_start = perf_counter()
with warnings.catch_warnings():
    warnings.filterwarnings("ignore", category=RuntimeWarning,
        message=r"Unrecognized options detected: \\{'mip_feasibility_tolerance'\\}. These will be
passed to HiGHS verbatim\\.")
    result = milp(cost, integrality=integer, bounds=Bounds(lower, upper),
        constraints=LinearConstraint(matrix, row_lo, row_hi),
        options={"time_limit": float(time_limit), "mip_rel_gap": float(mip_gap),
            "mip_feasibility_tolerance": SOLVER_TOL, "disp": False})
mip_seconds = perf_counter()-mip_start
if result.status not in (0, 1) or result.x is None or not np.isfinite(result.x).all():
    raise RuntimeError(f"No accepted MILP incumbent: {result.status}; {result.message}")
polished, old_integer_error, polish_seconds = _polish(
    cost, matrix, row_lo, row_hi, lower, upper, integer, result.x, time_limit)
x = polished.x
g = np.maximum(x[:n], 0)
fields = {key: np.empty((m, n)) for key in ["q_scenarios", "a", "b", "c", "d", "e", "v", "r"]}
raw_states, raw_modes, raw_xi = np.empty((m,n)), np.empty((m,n)), np.empty(m)
for j in range(m):
    offset = 2*n+j*width
    for k, key in enumerate(fields):
        fields[key][j] = np.maximum(x[offset+k*n:offset+(k+1)*n], 0)
    raw_states[j] = x[offset+8*n:offset+9*n]
    raw_modes[j] = x[offset+9*n:offset+10*n]
    raw_xi[j] = x[offset+10*n]
q_s, c, d, e, v, r = (fields[key] for key in ["q_scenarios", "c", "d", "e", "v", "r"])
# Zero-price LP degeneracy may represent a=q-g+b with both positive.
# Positive-part normalization is equivalent and does not alter q or cost.
fields["a"] = np.maximum(q_s-g, 0)*adjust
fields["b"] = np.maximum(g-q_s, 0)*adjust
modes = np rint(raw_modes).astype(int)
states = np.column_stack([np.full(m, s0), s0+np.cumsum(ETA_C*c-d/ETA_D, axis=1)])

```

```

shared_q = g.copy() if mode == "dayahead" else booked.copy()
shared_q[current] = q_s[0,current]
signed = (~locked).copy() if mode == "dayahead" else current.copy()
original_cost = float(price[~locked] @ g[~locked]) if mode == "dayahead" else 0.0
scene_adjustment = (1.5*fields["a"]-.5*fields["b"]) @ price
scene_emergency = 5*e @ price
shortfall = np.maximum(s_ref-states[:,-1], 0)
scene_terminal = kappa*shortfall
value = original_cost+float(probability @ (scene_adjustment+scene_emergency+scene_terminal))
surplus, deficit = np.maximum(q_s+pv-load, 0), np.maximum(load-q_s-pv, 0)
def maxabs(a):
    return float(np.max(np.abs(a))) if np.size(a) else 0.
def positive(a):
    return float(max(np.max(a), 0)) if np.size(a) else 0.
row_value = matrix @ x
checks = {
    "constraint_matrix_violation_kwh": max(positive(row_lo-row_value), positive(row_value-row_hi)),
    "variable_bound_violation_kwh": max(positive(lower-x), positive(x-upper)),
    "energy_balance_kwh": maxabs(q_s+pv-v-r+d+e-load-c),
    "soc_solver_agreement_kwh": maxabs(states[:,1:]-raw_states),
    "soc_lower_kwh": positive(LOW-states), "soc_upper_kwh": positive(states-HIGH),
    "charge_rate_kwh": positive(c-RATE), "discharge_rate_kwh": positive(d-RATE),
    "charge_mode_kwh": positive(c-RATE*modes),
    "discharge_mode_kwh": positive(d-RATE*(1-modes)),
    "emergency_charge_kwh": maxabs(np.minimum(c,e)),
    "simultaneous_charge_discharge_kwh": maxabs(np.minimum(c,d)),
    "charge_exceeds_surplus_kwh": positive(c-surplus),
    "discharge_exceeds_deficit_kwh": positive(d-deficit),
    "emergency_deficit_equation_kwh": maxabs(e-deficit+d),
    "unused_surplus_equation_kwh": maxabs(v+r-surplus+c),
    "unused_contract_bound_kwh": positive(v-q_s),
    "pv_curtailment_bound_kwh": positive(r-pv),
    "adjustment_relation_kwh": maxabs((q_s-g-fields["a"]+fields["b"])[:,adjust]),
    "shared_current_q_kwh": maxabs(q_s[:,current]-shared_q[current]),
    "fixed_q_kwh": maxabs(q_s[:,~future & ~current]-shared_q[~future & ~current]),
    "terminal_shortfall_kwh": positive(s_ref-states[:,-1]-raw_xi),
}
checks["max_violation_kwh"] = max(checks.values())
checks["integrality_error"] = maxabs(raw_modes-modes)
checks["original_integrality_error"] = old_integer_error
checks["objective_difference_yuan"] = abs(value-float(polished.fun))
checks["passed"] = bool(checks["max_violation_kwh"] ≤ PHYSICAL_TOL
    and checks["integrality_error"] ≤ PHYSICAL_TOL
    and checks["objective_difference_yuan"] ≤ OBJECTIVE_TOL)
if not checks["passed"]:
    raise RuntimeError(f"Physical/accounting verification failed: {checks}")
dual = getattr(result, "mip_dual_bound", None)
dual = None if dual is None or not np.isfinite(dual) else float(dual)
consistent = dual is not None and dual ≤ value+OBJECTIVE_TOL
absolute_gap = max(value-dual, 0) if consistent else None
gap = absolute_gap/max(abs(value), 1e-10) if consistent else None
proven = bool(result.status == 0 and gap is not None and gap ≤ mip_gap+1e-10)
return {"g": g, "q": shared_q, **fields, "s": states, "u": modes,
        "signed_mask": signed, "signed_q": shared_q[signed].copy(),
        "current_mask": current, "future_adjust_mask": future, "locked_mask": locked,
        "objective": value, "original_cost_in_objective": original_cost,
        "new_plan_cost": original_cost, "scenario_adjustment_cost": scene_adjustment,
        "scenario_emergency_cost": scene_emergency, "scenario_terminal_cost": scene_terminal,
        "expected_adjustment_cost": float(probability @ scene_adjustment),
        "expected_emergency_cost": float(probability @ scene_emergency),
        "terminal_cost": float(probability @ scene_terminal),
        "probabilities": probability, "terminal_weight": kappa, "s_ref": s_ref,
        "status": "optimal_within_requested_gap" if proven else "feasible_unproven",
        "status_code": int(result.status), "optimality_proven": proven,
        "gap": gap, "mip_gap": gap, "dual_bound": dual, "absolute_gap_yuan": absolute_gap,
        "checks": checks, "success": True, "solve_seconds": perf_counter()-start,

```

```
        "mip_seconds": mip_seconds, "polish_seconds": polish_seconds,  
        "requested_mip_gap": mip_gap, "time_limit_per_stage": time_limit,  
        "gap_basis": "Polished primal versus original global MILP dual bound, not the fixed-mode LP  
bound.",  
        "recourse_information": "Future adjustment and storage recourse foresee each scenario;  
rolling two-stage approximation.",  
        "cash_warning": "objective includes unexecuted future proposals and a soft inventory  
penalty; it is not an actual bill."}
```

49. question3/planning/code/q3_reports.py

源文件 SHA-256: 52cbfd3c5b3f2008053a99c3078a2a3a958b7827c4c8ffb6d8ec7c2e481f1794 ; 代码行数: 451。


```

"""Build audited Q3 workbooks and the four required paper tables from ledgers.

This module never optimizes dispatch or modifies source ledgers/templates.
Call build_reports(results_dir, data_dir, out_dir) after the main replay.
"""
from __future__ import annotations

import argparse
import csv
from datetime import datetime, timedelta
import hashlib
import json
import math
from pathlib import Path
import shutil

from openpyxl import Workbook, load_workbook
from openpyxl.chart import BarChart, Reference
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
from openpyxl.utils import get_column_letter

DATES=('2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21')
SLOTS=('10:00', '12:00', '14:00', '16:00', '18:00', '20:00')
EPS=1e-5
NUMERIC=('price_yuan_per_kwh', 'load_forecast_kw', 'pv_forecast_kw', 'load_actual_kw',
        'pv_actual_kw', 'g_kwh', 'q_kwh', 'increase_kwh', 'decrease_kwh', 'charge_kwh',
        'discharge_kwh', 'emergency_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh',
        'soc_start_kwh', 'soc_end_kwh', 'plan_cost_yuan', 'increase_cost_yuan', 'refund_yuan',
        'penalty_yuan', 'adjustment_net_cost_yuan', 'emergency_cost_yuan', 'cash_cost_yuan')
SUM_FIELDS=('g_kwh', 'q_kwh', 'increase_kwh', 'decrease_kwh', 'charge_kwh', 'discharge_kwh',
            'emergency_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh', 'plan_cost_yuan',
            'increase_cost_yuan', 'refund_yuan', 'penalty_yuan', 'adjustment_net_cost_yuan',
            'emergency_cost_yuan', 'cash_cost_yuan')
LABELS={'strategy': '策略', 'date': '日期', 'g_kwh': '零点原计划量(kWh)',
        'q_kwh': '调整后普通购电量(kWh)', 'increase_kwh': '增购量(kWh)',
        'decrease_kwh': '取消量(kWh)', 'charge_kwh': '充电量(kWh)',
        'discharge_kwh': '放电量(kWh)', 'emergency_kwh': '紧急购电量(kWh)',
        'unused_paid_kwh': '已付未用电量(kWh)', 'curtailed_pv_kwh': '弃光量(kWh)',
        'plan_cost_yuan': '原计划费(元)', 'increase_cost_yuan': '增购费(元)',
        'refund_yuan': '取消退原费(元, 正数)', 'penalty_yuan': '取消违约金(元)',
        'adjustment_net_cost_yuan': '调整净费用(元)', 'emergency_cost_yuan': '紧急费(元)',
        'cash_cost_yuan': '总现金费(元)', 'ordinary_cost_yuan': '普通购电结算费(元)',
        'soc_start_kwh': '起点储电量(kWh)', 'soc_end_kwh': '终点储电量(kWh)',
        'startup_cost_yuan': '首日初始化段费(元)', 'cash_including_startup_yuan': '含首段总费(元)',
        'inventory_adjusted_cost_yuan': '库存修正评价(元, 非电费)',
        'natural_period_cash_cost_yuan': '2月1日至12月31日自然日期间费(元)',
        'intervals': '区间数', 'selected': '事先规则选定', 'retrospective_best': '全期回溯最低(库存修正)'}

def sha(path): return hashlib.sha256(Path(path).read_bytes()).hexdigest()
def total(rows, key): return math.fsum(r[key] for r in rows)
def as_bool(v): return str(v).strip().lower() in ('true', '1', 'yes')
def near(a, b, label, tol=EPS):
    error=abs(a-b)
    if error>tol: raise AssertionError(f'{label}: {a} vs {b}, error={error}')
    return error

def read_trace(path, *, strategy=None):
    with Path(path).open(encoding='utf-8-sig', newline='') as f: raw=list(csv.DictReader(f))
    if not raw: raise ValueError(f'Empty ledger: {path}')
    rows=[]
    for r in raw:
        if strategy is not None and r.get('strategy') and r['strategy']!=strategy:

```

```

        # A shared startup file can contain all strategy rows; select exact.
        continue
    v=dict(r)
    for k in NUMERIC:
        if k not in r or r[k] in (',None'):raise ValueError(f'{path}: missing {k}')
        v[k]=float(r[k])
        if not math.isfinite(v[k]):raise ValueError(f'{path}: nonfinite {k}')
    v['actual_estimated']=as_bool(r.get('actual_estimated',False))
    for key in ('interval_start','interval_end'):
        v[key]=datetime.fromisoformat(r[key]).isoformat()
    if strategy is not None:v['strategy']=strategy
    rows.append(v)
rows.sort(key=lambda r:r['interval_start'])
return rows

def audit_trace(rows,*,label,require_contiguous=True):
    """Verify ledger algebra. Finer execution causality remains runner-tested."""
    max_balance=max_soc=max_cash=max_components=0.
    starts=set();prev=None
    for r in rows:
        a=datetime.fromisoformat(r['interval_start']);b=datetime.fromisoformat(r['interval_end'])
        if b-a!=timedelta(minutes=10) or a.minute%10:raise AssertionError('Bad interval')
        if a in starts:raise AssertionError('Duplicate execution interval')
        starts.add(a)
        if prev is not None and require_contiguous:
            if prev['interval_end']!=r['interval_start']:raise AssertionError('Noncontiguous ledger')
            near(prev['soc_end_kwh'],r['soc_start_kwh'],'cross-interval SOC')
        p=r['price_yuan_per_kwh'];g=r['g_kwh'];q=r['q_kwh'];up=r['increase_kwh'];down=r['decrease_kwh']
        c=r['charge_kwh'];d=r['discharge_kwh'];e=r['emergency_kwh']
        v=r['unused_paid_kwh'];curtail=r['curtailed_pv_kwh']
        load=r['load_actual_kw']/6;pv=r['pv_actual_kw']/6
        if min(p,g,q,up,down,c,d,e,v,curtail)<-EPS:raise AssertionError('Negative physical value')
        if min(up,down)>EPS:raise AssertionError('Simultaneous increase/cancellation')
        if max(c,d)>5000/6+EPS:raise AssertionError('Battery power limit')
        if v>q+EPS or curtail>pv+EPS:raise AssertionError('Waste exceeds available source')
        for s in ('soc_start_kwh','soc_end_kwh'):
            if not 1200-EPS<=r[s]<=10800+EPS:raise AssertionError('SOC bound')
        max_balance=max(max_balance,near(q-v+pv-curtail+d+e,load+c,'AC energy balance'))
        max_soc=max(max_soc,near(r['soc_end_kwh'],r['soc_start_kwh']+.9*c-d/.9,'SOC equation'))
        values=((q,g+up-down,'q=g+increase-decrease'),
            (r['plan_cost_yuan'],p*g,'original cost'),
            (r['increase_cost_yuan'],1.5*p*up,'increase cost'),
            (r['refund_yuan'],p*down,'refund positive magnitude'),
            (r['penalty_yuan'],.5*p*down,'penalty'),
            (r['adjustment_net_cost_yuan'],1.5*p*up-.5*p*down,'net adjustment'),
            (r['emergency_cost_yuan'],5*p*e,'emergency cost'))
        for x,y,name in values:max_components=max(max_components,near(x,y,name))
        max_cash=max(max_cash,near(r['cash_cost_yuan'],p*g+1.5*p*up-.5*p*down+5*p*e,'cash'))
        prev=r
    return {'label':label,'intervals':len(rows),'first_interval':rows[0]['interval_start'] if rows else
None,
        'last_interval_end':rows[-1]['interval_end'] if rows else None,
        'max_balance_error_kwh':max_balance,'max_soc_error_kwh':max_soc,
        'max_component_error':max_components,'max_cash_error_yuan':max_cash,
        'actual_estimated_intervals':sum(r['actual_estimated'] for r in rows),
        'scope':'aggregate physical/transaction ledger; within-interval mode/causality checked
separately in runner tests'}

def summarize(rows,strategy,date=None):
    out={k:total(rows,k) for k in SUM_FIELDS}
    out.update(strategy=strategy,date=date,intervals=len(rows),soc_start_kwh=rows[0]['soc_start_kwh'],
        soc_end_kwh=rows[-1]
['soc_end_kwh'],ordinary_cost_yuan=out['plan_cost_yuan']+out['adjustment_net_cost_yuan'])
    near(out['cash_cost_yuan'],out['ordinary_cost_yuan']+out['emergency_cost_yuan'],'summary cash')

```

```

    return out

def day_groups(rows, *, full_only=True):
    groups={}
    for r in rows: groups.setdefault(r['interval_start'][:10], []).append(r)
    out={}
    for date, rs in sorted(groups.items()):
        start=datetime.fromisoformat(date)
        full=len(rs)==144 and
all(datetime.fromisoformat(r['interval_start'])==start+timedelta(minutes=10*i) for i, r in enumerate(rs))
        if full: out[date]=rs
        elif not full_only: out[date]=rs
    return out

def episodes(rows):
    """Merge positive e intervals; never join across a calendar midnight."""
    out=[]; active=None
    for r in rows:
        same=active is not None and active['end']==r['interval_start'] and
active['date']==r['interval_start'][:10]
        if r['emergency_kwh']>EPS:
            if not same:
                if active is not None: out.append(active)
                active={'date':r['interval_start']
[:10], 'start':r['interval_start'], 'end':r['interval_end'],
                'parts':[r['emergency_kwh']]
            else:
                active['end']=r['interval_end']; active['parts'].append(r['emergency_kwh'])
            elif active is not None:
                out.append(active); active=None
        if active is not None: out.append(active)
    for e in out:
        e['kwh']=math.fsum(e.pop('parts'))
        end='24:00' if e['end'][:10]!=e['date'] else e['end'][11:16]
        e['label']=e['start'][11:16]+'-'+end
        near(math.fsum(e['kwh']) for e in out)+math.fsum(r['emergency_kwh'] for r in rows if
r['emergency_kwh'] ≤ EPS),
        total(rows, 'emergency_kwh'), 'episode sum')
    return out

def style(ws, *, width=20, header_rows=(1,)):
    ws.sheet_view.showGridLines=False
    ws.freeze_panes='B2'
    ws.sheet_properties.pageSetUpPr.fitToPage=True
    ws.page_setup.orientation='landscape'; ws.page_setup.paperSize=ws.PAPERSIZE_A4
    ws.page_setup.fitToWidth=1; ws.page_setup.fitToHeight=0
    for col in range(1, ws.max_column+1): ws.column_dimensions[get_column_letter(col)].width=width
    for row in ws:
        ws.row_dimensions[row[0].row].height=27
        for c in row:
            c.font=Font(name='微软雅黑', size=10, color='25354A')
            c.alignment=Alignment(horizontal='center', vertical='center', wrap_text=True)
            if isinstance(c.value, (int, float)): c.number_format='#,##0.0000'
            if c.value is not None: c.border=Border(bottom=Side(style='hair', color='D5DFE7'))
            if c.row in header_rows:
                c.fill=PatternFill('solid', fgColor='213C56'); c.font=Font(name='微软雅黑', size=10, bold=True, color='FFFFFF')
            elif c.row%2==0: c.fill=PatternFill('solid', fgColor='F1F6F8')
        ws.print_area=ws.dimensions

def table_sheet(book, name, rows, fields):
    ws=book.create_sheet(name); ws.append([LABELS.get(k, k) for k in fields])

```

```

for r in rows:ws.append([r.get(k) for k in fields])
style(ws)
ws.auto_filter.ref=ws.dimensions
for i,k in enumerate(fields,1):
    if k.endswith('_yuan'):
        for j in range(2,ws.max_row+1):ws.cell(j,i).number_format='#,##0.00'
return ws

def add_notes(book,selected,selection,*,startup_present):
    ws=book.create_sheet('口径说明',0)
    ws.append(['事项','说明'])
    notes=[('选择策略',selected),('选择规则',json.dumps(selection,ensure_ascii=False)),
    ('回溯最低含义','retrospective_best按主程序库修正费用选择，不一定是纯现金最低；现金和库存修正评价分别显
示。'),
    ('合同窗口','本日00:10至次日00:10，144段；原计划g与调整后最终量q分别保存。'),
    ('取消结算','取消部分退原费用再收50%违约金。退费列按正数展示；净调整费=增购费-退费+违约金。'),
    ('原计划购电表','填原g，全天量Σg，全天费Σp·g。'),
    ('调整购电表','填调整后的普通购电总量q，不填差额；全天量Σq，全天购电费填总现金费Σp·g+调整净费用+Σ5p·e，紧急
费仅计一次。普通结算费另列现金明细。'),
    ('总现金费','原计划费+增购费-取消退原费+取消违约金+紧急费。未成交展望、控制目标和库存软惩罚不计入账单。'),
    ('论文表1','指定10分钟量使用最终普通q；日总普通量Σq；日购电费包含紧急费。另列原计划、净调整与紧急费用以免混
淆。'),
    ('自然日储能','按实际interval_start选00:00-24:00；首段来自前一合同，不按window_date整行汇总。'),
    ('首日初始化段','使用startup.csv单列的00:00-00:10共同初始化假设，并入自然日报表；不修改首合同00:10的
6000kWh初态。' if startup_present else '未提供startup.csv；首个自然日缺00:00-00:10，相关自然日表不伪造完整
值。'),
    ('末尾跨年段','12月31日合同包含次年1月1日00:00-00:10；该段在合同费中只记一次，单列跨年运行，不并入12月31日自
然日储能。'),
    ('单位','功率kW，账本电量kWh，费用元。导出程序不再乘1/6处理已经是kWh的购电/充放电量。'),
    ('紧急事件','合并连续e>1e-5kWh的区间，跨自然日拆分；原紧急表仅列合同日期范围内的自然日，次年1月1日尾段只保留在
跨年明细；低于阈值量保留在精确汇总中。'),
    ('预测来源','使用用户交付的已插值并端点平均后的光伏；不再平均。生成过程的信息集未独立完整认证，主结果以该输入为
条件。'),
    ('范围','更换预测器后需重建场景误差并重评。回溯最低与依据事先规则选定的策略分别标记。')]
    for row in notes:ws.append(row)
    style(ws);ws.column_dimensions['A'].width=22;ws.column_dimensions['B'].width=110
    for i in range(2,ws.max_row+1):ws.row_dimensions[i].height=45
    return ws

def save_verified(book,path):
    expected={(ws.title,c.coordinate):c.value for ws in book for row in ws for c in row if c.value is
not None}
    book.save(path)
    wb=load_workbook(path,data_only=False,read_only=True)
    errors=0;checked=0
    try:
        for ws in wb:
            for row in ws:
                for c in row:
                    if c.data_type=='e':raise AssertionError(f'Excel error {ws.title}!{c.coordinate}')
                    if c.value is None:continue
                    want=expected[(ws.title,c.coordinate)]
                    if isinstance(want,(int,float)) and not isinstance(want,bool):
                        if abs(float(c.value)-want)>max(1e-8,abs(want)*1e-12):raise
AssertionError('Workbook numeric drift')
                    elif isinstance(want,datetime):
                        if c.value!=want:raise AssertionError('Workbook datetime drift')
                    elif c.value!=want:raise AssertionError(f'Workbook text drift: {want!r} →
{c.value!r}')
                checked+=1
    finally:wb.close()
    return {'path':str(path),'cells_checked':checked,'excel_errors':errors,'sha256':sha(path)}

```

```

def export_result3(template, path, rows, startup, selected, selection):
    book=load_workbook(template)
    contracts={}
    for r in rows:contracts.setdefault(r['window_date'], []).append(r)
    for name in ('计划购电量', '调整购电量', '充放电量', '紧急购电量'):
        ws=book[name]
        for merged in list(ws.merged_cells.ranges):
            if merged.min_row ≥ 2:ws.unmerge_cells(str(merged))
            if ws.max_row ≥ 2:ws.delete_rows(2, ws.max_row-1)
    daily_financial=[]
    for day, rs in sorted(contracts.items()):
        if len(rs) ≠ 144:raise AssertionError('Incomplete contract in result3')
        zero=datetime.fromisoformat(day)
        if rs[0]['interval_start'] ≠ (zero+timedelta(minutes=10)).isoformat():raise
AssertionError('Contract start')
        s=summarize(rs, selected, day);daily_financial.append(s)
        book['计划购电量'].append([zero]+[r['g_kwh'] for r in rs]+[s['g_kwh'], s['plan_cost_yuan']]])
        book['调整购电量'].append([zero]+[r['q_kwh'] for r in rs]+[s['q_kwh'], s['cash_cost_yuan']]])
    natural=day_groups(startup+rows)
    for day in contracts:
        ws=book['充放电量']
        if day not in natural:
            for k in range(6):ws.append([datetime.fromisoformat(day) if k==0 else None,
                f'{4*k}:00-{4*(k+1)}:00', '缺初始段, 未编造' if k==0 else None, None,
                '0:00' if k==0 else ('24:00' if k==1 else None), None])
            continue
        rs=natural[day]
        for k in range(6):
            block=rs[k*24:(k+1)*24]
            ws.append([datetime.fromisoformat(day) if k==0 else None, f'{4*k}:00-{4*(k+1)}:00',
                total(block, 'charge_kwh'), total(block, 'discharge_kwh'),
                '0:00' if k==0 else ('24:00' if k==1 else None),
                rs[0]['soc_start_kwh'] if k==0 else (rs[-1]['soc_end_kwh'] if k==1 else None)])
    all_episodes=episodes(startup+rows)
    e_by_day={}
    for e in all_episodes:e_by_day.setdefault(e['date'], []).append(e)
    # The original sheet covers only the requested calendar dates. The final
    # next-year interval stays in the contract cash and separate tail ledger.
    for day in sorted(contracts):
        es=e_by_day.get(day, [])
        if es:
            for j, e in enumerate(es):book['紧急购电量'].append([datetime.fromisoformat(day) if j==0 else
None, e['label'], e['kwh']]])
            else:book['紧急购电量'].append([datetime.fromisoformat(day), '无紧急购电', 0.])
    for name in ('计划购电量', '调整购电量'):
        ws=book[name];style(ws, width=14)
        ws.column_dimensions['A'].width=16;ws.freeze_panes='B2'
        for row in range(2, ws.max_row+1):
            ws.cell(row, 1).number_format='yyyy-mm-dd';ws.cell(row, 147).number_format='#, ##0.00'
    for name in ('充放电量', '紧急购电量'):
        ws=book[name];style(ws, width=22)
        for row in range(2, ws.max_row+1):
            if isinstance(ws.cell(row, 1).value, datetime):ws.cell(row, 1).number_format='yyyy-mm-dd'
    add_notes(book, selected, selection, startup_present=bool(startup))
    table_sheet(book, '逐合同现金明细', daily_financial, ('date',)+SUM_FIELDS+
('ordinary_cost_yuan', 'soc_start_kwh', 'soc_end_kwh'))
    nat=[summarize(rs, selected, day) for day, rs in natural.items()]
    table_sheet(book, '逐自然日现金明细', nat, ('date',)+SUM_FIELDS+
('ordinary_cost_yuan', 'soc_start_kwh', 'soc_end_kwh'))
    if startup:
        table_sheet(book, '首日初始化假设', startup, ('interval_start', 'interval_end')+SUM_FIELDS+
('soc_start_kwh', 'soc_end_kwh'))
        tail=[r for r in rows if r['interval_start'][:10] > max(contracts)]
        if tail:table_sheet(book, '跨年合同末段', tail, ('interval_start', 'interval_end')+SUM_FIELDS+
('soc_start_kwh', 'soc_end_kwh'))
    return save_verified(book, path), natural

```

```

def paper_tables(path,md_path,natural,selected):
    book=Workbook();book.remove(book.active)
    summaries={}
    s1=book.create_sheet('表1_指定购电');s2=book.create_sheet('表2_充放电')
    heads1=[];heads2=[];md=[ '# 第三问指定四日论文表格',' ',f'方案: `{selected}`。日期按00:00-24:00自然日, 电量
    kWh, 费用元.',
        '表1购电量是调整后的普通总量q, 全天购电费包括紧急费; 原计划费与调整净费用另列。数值按全精度汇总后显示.',' ' ]
    for date in DATES:
        if date not in natural:raise AssertionError(f'Missing required full calendar day {date}')
        rs=natural[date];s=summarize(rs,selected,date);summaries[date]=s
        for ws,heads,title in ((s1,heads1,'表1 购电量及费用'),(s2,heads2,'表2 实际充放电')):
            ws.append([f'{date}| {selected}| {title}']);heads.extend([ws.max_row,ws.max_row+1])
            ws.merge_cells(start_row=ws.max_row,start_column=1,end_row=ws.max_row,end_column=6)
            s1.append(['时间段','购电量q','时间段','购电量q','时间段','购电量q'])
            md.extend([f'## {date}',' ', '表1: 购电量及费用',' ', ' | 时间段 | 购电量 | 时间段 | 购电量 | 时间段 | 购电
            量 |','|---:|---:|---:|---:|---:|---:|'])
            for ts in (SLOTS[:3],SLOTS[3:]):
                row=[]
                for t in ts:
                    ix=int(t[:2])*6;row.extend([f'{t}-{t[:3]}10',rs[ix]['q_kwh']])
                s1.append(row);md.append(' | '+' | '.join(f'{x:.4f}' if isinstance(x,(int,float)) else x for
                x in row)+' |')
                s1.append(['全天普通购电量',s['q_kwh'],'全天购电费(含紧急)',s['cash_cost_yuan'],'全天紧急
                量',s['emergency_kwh']])
                s1.append(['原计划费',s['plan_cost_yuan'],'调整净费用',s['adjustment_net_cost_yuan'],'紧急
                费',s['emergency_cost_yuan']]);s1.append([])
                md.extend([' ',f"全天普通购电量 **{s['q_kwh']:.4f} kWh**; 全天购电费 (含紧急) **
                {s['cash_cost_yuan']:.2f}元**。",
                    f"原计划费{s['plan_cost_yuan']:.2f}元, 调整净费用{s['adjustment_net_cost_yuan']:.2f}元, 紧急费
                {s['emergency_cost_yuan']:.2f}元; 紧急电量{s['emergency_kwh']:.4f} kWh。",',',
                    '表2: 实际充放电',' ', ' | 时间段 | 充电量 | 放电量 | 时间段 | 充电量 | 放电量 |','|---:|---:|---:|---:|---:|---:|'])
                s2.append(['时间段','充电量','放电量','时间段','充电量','放电量'])
                for i in (0,2,4):
                    row=[]
                    for j in (i,i+1):
                        block=rs[j*24:(j+1)*24]
                        row.extend([f'{4*j:02d}:00-{4*
                (j+1):02d}:00',total(block,'charge_kwh'),total(block,'discharge_kwh')])
                    s2.append(row);md.append(' | '+' | '.join(f'{x:.4f}' if isinstance(x,(int,float)) else x for
                    x in row)+' |')
                    s2.append(['00:00储电量',s['soc_start_kwh'],None,'24:00储电
                    量',s['soc_end_kwh'],None]);s2.append([])
                    md.extend([' ',f"00:00储电量: **{s['soc_start_kwh']:.4f} kWh**; 24:00储电量: **
                    {s['soc_end_kwh']:.4f} kWh**。",','])
                    style(s1,header_rows=heads1);style(s2,header_rows=heads2)
                s3=book.create_sheet('表3_连续紧急购电')
                s3.append([f'{selected}| 表3 指定四日紧急购电量(kWh)']);s3.merge_cells('A1:H1')
                s3.append([v for d in DATES for v in (d,None)])
                for col in (1,3,5,7):s3.merge_cells(start_row=2,start_column=col,end_row=2,end_column=col+1)
                s3.append(['时间段','购电量']*4)
                es={date:episodes(natural[date]) for date in DATES}
                md.extend(['## 表3: 连续紧急购电',' ', ' | '+' | '.join(d+'时段 | 电量' for d in DATES)+' |','|---:|---:|---:|---:|---:|---:|'])
                for i in range(max(1,max(map(len,es.values())))):
                    row=[]
                    for date in DATES:
                        ee=es[date]
                        row.extend([ee[i]['label'],ee[i]['kwh']] if i<len(ee) else ([ '无紧急购电',0.] if not ee and
                i==0 else [None,None]))
                    s3.append(row);md.append(' | '+' | '.join(' ' if x is None else (f'{x:.4f}' if isinstance(x,
                (int,float)) else x) for x in row)+' |')
                    style(s3,header_rows=(1,2,3));s3.page_setup.fitToHeight=1
                    table_sheet(book,'四日现金汇总',List(summaries.values()),('date',)+SUM_FIELDS+

```



```

('ordinary_cost_yuan', 'soc_start_kwh', 'soc_end_kwh'))
    md.extend(['', '连续事件合并阈值为1e-5 kWh。跨午夜事件拆开：低于阈值量仍保留在每日精确总量中。',
        '四个展示日均由真实时间戳重组：00:00-00:10取前一合同，其余143段取当日合同，不重签、不重复计费。'])
    md_path.write_text('\n'.join(md)+'\n', encoding='utf-8')
    return save_verified(book, path), summaries

def build_reports(results_dir, data_dir, out_dir):
    results_dir=Path(results_dir);data_dir=Path(data_dir);out_dir=Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    selection=json.loads((results_dir/'selection.json').read_text(encoding='utf-8'))
    selected=selection['selected_strategy'];retrospective=selection.get('retrospective_best')
    if
isinstance(retrospective, dict):retrospective=retrospective.get('strategy',retrospective.get('name'))
    paths=sorted(results_dir.glob('H*/trace.csv'))
    if not paths:raise ValueError('No H*/trace.csv files')
    by_strategy={};startup_by_strategy={};audits=[];summary=[];source_hashes={};monthly=[]
    for path in paths:
        strategy=path.parent.name
        rows=read_trace(path, strategy=strategy)
        if not rows:raise ValueError(f'No rows for {strategy}')
        by_strategy[strategy]=rows
        source_hashes[str(path)]=sha(path)
        audits.append(audit_trace(rows, label=strategy))
        sp=path.parent/'startup.csv'
        if not sp.exists():sp=results_dir/'startup.csv'
        startup=read_trace(sp, strategy=strategy) if sp.exists() else []
        startup_by_strategy[strategy]=startup
        if startup:
            source_hashes[str(sp)]=sha(sp)
            audits.append(audit_trace(startup+rows, label=strategy+' plus common startup'))
        s=summarize(rows, strategy)
        summary_path=path.parent/'summary.json'
        source_summary=json.loads(summary_path.read_text(encoding='utf-8')) if summary_path.exists()
    else {}
        kappa=source_summary.get('config', {}).get('terminal_weight', min(r['price_yuan_per_kwh'] for r in
rows)/.9)
        s['inventory_adjusted_cost_yuan']=s['cash_cost_yuan']-kappa*(s['soc_end_kwh']-
s['soc_start_kwh'])
        if summary_path.exists():
            source_hashes[str(summary_path)]=sha(summary_path)
            near(s['cash_cost_yuan'], source_summary['cash_cost_yuan'], 'main summary cash', tol=1e-4)
            near(s['inventory_adjusted_cost_yuan'], source_summary['inventory_adjusted_cost_yuan'], 'main
summary inventory score', tol=1e-4)
        s.update(startup_cost_yuan=total(startup, 'cash_cost_yuan') if startup else 0.,
            selected=strategy==selected, retrospective_best=strategy==retrospective)
        s['cash_including_startup_yuan']=s['cash_cost_yuan']+s['startup_cost_yuan']
        last_day=max(r['window_date'] for r in rows)
        s['natural_period_cash_cost_yuan']=math.fsum(r['cash_cost_yuan'] for r in rows if
r['interval_start'][:10]≤last_day)+s['startup_cost_yuan']
        summary.append(s)
        groups={}
        for r in rows:groups.setdefault(r['window_date'][:7], []).append(r)
        monthly.extend(summarize(rs, strategy, month) for month, rs in groups.items())
    if selected not in by_strategy:raise ValueError('Selected strategy ledger unavailable')
    summary.sort(key=lambda s:s['cash_cost_yuan'])
    book=Workbook();book.remove(book.active)
    add_notes(book, selected, selection, startup_present=bool(startup_by_strategy[selected]))
    fields=
('strategy', 'selected', 'retrospective_best', 'cash_cost_yuan', 'inventory_adjusted_cost_yuan', 'plan_cost_y
uan',
    'increase_cost_yuan', 'refund_yuan', 'penalty_yuan', 'adjustment_net_cost_yuan',

'emergency_cost_yuan', 'q_kwh', 'emergency_kwh', 'soc_end_kwh', 'startup_cost_yuan', 'cash_including_startup_
yuan', 'natural_period_cash_cost_yuan')
    ws=table_sheet(book, '八策略费用比较', summary, fields)

```



```

chart=BarChart();chart.title='各组合合同窗口实际现金费';chart.y_axis.title='元';chart.x_axis.title='策略'
chart.add_data(Reference(ws,min_col=4,min_row=1,max_row=len(summary)+1),titles_from_data=True)

chart.set_categories(Reference(ws,min_col=1,min_row=2,max_row=len(summary)+1));chart.height=9;chart.width=20
ws.add_chart(chart,f'A{len(summary)+4}')
table_sheet(book,'分月合同费用',monthly,('strategy','date','cash_cost_yuan','plan_cost_yuan',
    'adjustment_net_cost_yuan','emergency_cost_yuan','emergency_kwh','soc_end_kwh'))
comp_verify=save_verified(book,out_dir/'问题三_策略比较.xlsx')
template=data_dir/'result3_template.xlsx'
source_hashes[str(template)]=sha(template)
template_verify,natural=export_result3(template,out_dir/'result3.xlsx',by_strategy[selected],
    startup_by_strategy[selected],selected,selection)
paper_verify,chosen_days=paper_tables(out_dir/'问题三_指定四日填表.xlsx',out_dir/'问题三_指定四日论文表格.md',natural,selected)
shutil.copy2(results_dir/selected/'trace.csv',out_dir/f'{selected}_完整逐段账本.csv')
for source,digest in source_hashes.items():
    if sha(source)!=digest:raise AssertionError('Report export changed an input')
report=
{'selected_strategy':selected,'retrospective_best':retrospective,'strategy_count':len(summary),
    'strategy_summaries':summary,'trace_audits':audits,
    'four_day_summary':chosen_days,'workbook_checks':[comp_verify,template_verify,paper_verify],
    'source_sha256':source_hashes,'source_files_unchanged':True,
    'cash_coverage':'main summary is 334 shifted contracts; common first natural-day startup is
separately listed and included only in the explicit including-startup column',
    'natural_days_exported':len(natural),'first_day_complete':min(r['window_date'] for r in
by_strategy[selected]) in natural}
(out_dir/'报表与账本核验.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
with (out_dir/'策略费用比较.csv').open('w',newline='',encoding='utf-8-sig') as f:
    writer=csv.DictWriter(f,fieldnames=fields);writer.writeheader();writer.writerows({k:r.get(k) for
k in fields} for r in summary)
(out_dir/'阅读说明.md').write_text(f'# 第三问结果报表\n\n选定策略: **{selected}**; 全期回溯最低: **
{retrospective}**. \n\n'
    '原result3模板的计划表保存原g及原计划费Σpg, 调整表保存最终普通量q; 调整表全天购电费填最终总现金费Σpg+调整净费用+Σ5pe。'
    '紧急费只计一次, 普通结算费与紧急费仍在现金明细中分列。论文四日表1全天购电费同样包含紧急费。 \n\n'
    '充放电与紧急事件按自然日时间戳汇总。共同初始化首段单独来自startup.csv; 跨年末段单独列出, 账单不重复。'
    '所有电量保持原始kwh精度, 显示四位小数; 费用显示两位小数, 显示数相加可能有舍入差。 \n\n'
    '报表程序只读取执行账本, 不再次求解、不修改计划或电池状态。报表与账本核验.json保存逐段物理/费用检查及工作簿重开
核验结果。 \n',encoding='utf-8')
return report

if __name__=='__main__':
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--results',type=Path,required=True)
    parser.add_argument('--data',type=Path,required=True)
    parser.add_argument('--out',type=Path,required=True)
    args=parser.parse_args()
    report=build_reports(args.results,args.data,args.out)

print(json.dumps({'selected_strategy':report['selected_strategy'],'strategy_count':report['strategy_count'],
    'natural_days_exported':report['natural_days_exported'],'output':str(args.out)},ensure_ascii=False))

```

50. question3/planning/code/q3_risk.py

源文件 SHA-256: b5f5de1c5dc0e0007b5a4e5f603245f106545eb92c5c910e8b1a176d09403da7 ; 代码行数: 67.

```

"""Causal, paired full-remaining-window residual scenarios for Q3.

No risk margin is added. A scenario samples one historical date for BOTH load
and the matching PV forecast vintage. All its target intervals must have ended
before the current decision. Excluded forecast vintages are never read.
"""

import datetime as dt
import numpy as np

START = {0: 0, 6: 35, 12: 71, 18: 107}
HOURS = (0, 6, 12, 18)

def available_history(data, day, decision_hour, window=28):
    decision = dt.datetime.combine(data['dates'][day], dt.time(decision_hour))
    return [j for j in range(day) if data['window_end'][j] ≤ decision[-window:]]

def scenario_inputs(data, day, forecast_hour, decision_hour, begin, config):
    if forecast_hour > decision_hour or forecast_hour not in HOURS:
        raise ValueError('Forecast vintage is not available at decision time')
    if begin < START[forecast_hour]:
        raise ValueError('Forecast vintage does not cover the requested beginning')
    source = config.get('pv_source', 'supplied')
    pv = data['pv_f'] if source == 'supplied' else data['pv_linear_f']
    k = HOURS.index(forecast_hour)
    lf = data['load_f'][day, begin:].copy()
    pf = pv[day, k, begin:].copy()
    if not np.isfinite(lf).all() or not np.isfinite(pf).all():
        raise ValueError('Missing issued forecast')
    history = available_history(data, day, decision_hour, config.get('history_days', 28))
    meta = {'forecast_hour': forecast_hour, 'decision_hour': decision_hour,
            'issued_at': str(dt.datetime.combine(data['dates'][day], dt.time(forecast_hour))),
            'decision_at': str(dt.datetime.combine(data['dates'][day], dt.time(decision_hour))),
            'history_indices': history, 'history_days': len(history),
            'history_latest_end': None if not history else data['window_end'][history[-1]].isoformat(),
            'pv_source': source, 'margin_added': False}
    count = config.get('scenarios', 8)
    if count == 1 or len(history) < config.get('min_history', 14):
        meta.update(scenarios=1, fallback=count ≠ 1, draw_indices=[])
        return lf[None,:]/6, pf[None,:]/6, meta
    lr = data['load_a'][history, begin:] - data['load_f'][history, begin:]
    pr = data['pv_a'][history, begin:] - pv[history, k, begin:]
    rng = np.random.default_rng(config.get('seed', 20260912) + 1009*day + 59*forecast_hour)
    m = min(count, len(history))
    draw = rng.choice(len(history), m, replace=False)
    loads = np.maximum(lf + lr[draw], 0)
    pvs = np.maximum(pf + pr[draw], 0)
    meta.update(scenarios=m, fallback=False, draw_indices=np.asarray(history)[draw].tolist(),
                sampling='paired complete residual paths, without replacement, uniform weights')
    return loads/6, pvs/6, meta

def latest_hour(enabled_hours, target_index):
    return max(h for h in enabled_hours if START[h] ≤ target_index)

def future_mask(trade_hours, target_index, *, include_current=False):
    later = [START[h] for h in trade_hours if h and
              (START[h] ≥ target_index if include_current else START[h] > target_index)]
    return np.arange(target_index, 144) ≥ min(later) if later else np.zeros(144-target_index, bool)

def current_block(trade_hours, hour):

```

```
start = START[hour]
later = [START[h] for h in trade_hours if h > hour]
return start, min(later) if later else 144
```

51. question3/planning/code/q3_runner.py

源文件 SHA-256: 40ea917fa11b0949bf63ae200b5283208be62df900de1e7b6477bd286749e2cf ; 代码行数: 265。

```

"""Continuous Q3 replay. Only release events sign orders; MPC only executes.

The main 10-minute execution uses the current interval mean as a fast-feedback
approximation. Historical means enter scenario fitting only after interval end.
No daily SOC reset. At midnight new orders are signed before the old tail is
observed, using a locked one-interval bridge. Future scene orders are tentative.
"""

from __future__ import annotations
import argparse, csv, datetime as dt, hashlib, itertools, json, os, time
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor, as_completed
import numpy as np
from q3_io import load_inputs
from q3_risk import HOURS, START, scenario_inputs, latest_hour, future_mask, current_block
from q3_optimizer import solve_dispatch, LOW, HIGH, RATE, ETA_C, ETA_D

DEFAULT = dict(scenarios=8, history_days=28, min_history=14, seed=20260912,
               controller='mpc', pv_source='supplied', initial_soc=6000.,
               time_limit=10., mip_gap=1e-4)
SCHEDULES = [(0,)+s for n in range(4) for s in itertools.combinations((6,12,18),n)]

def dump(path, value):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(json.dumps(value,ensure_ascii=False,indent=2,default=str))

def write_csv(path, rows):
    if not rows: return
    with open(path,'w',newline='',encoding='utf-8-sig') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)

def signature(data_dir, config):
    h=hashlib.sha256(json.dumps(config,sort_keys=True).encode())
    for p in sorted(Path(__file__).parent.glob('q3_*.py')):
        if p.name in ('q3_runner.py','q3_optimizer.py','q3_io.py','q3_risk.py'):
            h.update(p.name.encode());h.update(p.read_bytes())
    h.update((Path(data_dir)/'source_manifest.json').read_bytes())
    # The source manifest is historical provenance, not a substitute for
    # hashing the files currently being supplied. A user can replace a forecast
    # workbook without editing that manifest; such replacement must invalidate
    # every cached result.
    for name in ('attachment1.xlsx','attachment2.xlsx','attachment3.xlsx',
                'load_forecasts.xlsx','pv_forecasts.xlsx','result3_template.xlsx'):
        h.update(name.encode());h.update((Path(data_dir)/name).read_bytes())
    return h.hexdigest()

def validate_run_arguments(trade_hours, forecast_hours, config, start_day, days):
    for label, hours in [('trade_hours', trade_hours), ('forecast_hours', forecast_hours)]:
        if not hours or any(type(h) is not int for h in hours) or
        tuple(sorted(set(hours))) != tuple(hours) or hours[0] != 0 or not set(hours) <= set(HOURS):
            raise ValueError(f'{label} must be a sorted unique subset of (0,6,12,18), including 0')
    if type(start_day) is not int or not 0 <= start_day < 334:
        raise ValueError('start_day must be an integer in 0..333')
    if days is not None and (type(days) is not int or days <= 0):
        raise ValueError('days must be a positive integer or None')
    for key in ('scenarios', 'history_days', 'min_history'):
        if type(config[key]) is not int or config[key] < 1: raise ValueError(f'{key} must be positive integer')
    if config['history_days'] < config['min_history']: raise ValueError('history_days must cover min_history')
    if config['controller'] not in ('mpc', 'greedy'): raise ValueError('Unknown controller')
    if config['pv_source'] not in ('supplied', 'linear'): raise ValueError('Unknown PV source')
    if not np.isfinite(config['initial_soc']) or not LOW <= config['initial_soc'] <= HIGH:
        raise ValueError('Initial SOC outside physical bounds')

```

```

def make_record(data, day, t, name, g, q, c, d, e, state, forecast_hour, *, startup=False):
    if startup:
        z=data['first_unexecuted_interval']; L=z['load_mean_kw'];G=z['pv_mean_kw']
        begin=dt.datetime.combine(data['dates'][0],dt.time())
        lf=float(data['load_f'][0,0]);pf=float(data['pv_unused_first_mean_kw'][0]);p=float(data['price']
[-1])
    else:
        L=float(data['load_a'][day,t]);G=float(data['pv_a'][day,t])
        begin=data['window_start'][day]+dt.timedelta(minutes=10*t)
        lf=float(data['load_f'][day,t]);pf=float(data['pv_f']
[day,HOURS.index(forecast_hour),t]);p=float(data['price'][t])
        waste=max(q+G/6+d+e-L/6-c,0.)
        # Bookkeeping allocation: use PV first, then paid contract; neither refund.
        v=min(q,waste);r=max(waste-v,0.)
        a=max(q-g,0.);b=max(g-q,0.);end_state=state+ETA_C*c-d/ETA_D
    return dict(strategy=name,window_date=str(data['dates'][day]),interval_start=begin.isoformat(),
        interval_end=(begin+dt.timedelta(minutes=10)).isoformat(),
        forecast_issue_time=dt.datetime.combine(data['dates'][day],dt.time(forecast_hour)).isoformat(),
        price_yuan_per_kwh=p,load_forecast_kw=lf,pv_forecast_kw=pf,load_actual_kw=L,pv_actual_kw=G,
        actual_estimated=bool(False if startup else data['actual_estimated'][day,t]),
        g_kwh=float(g),q_kwh=float(q),increase_kwh=a,decrease_kwh=b,
        charge_kwh=float(c),discharge_kwh=float(d),emergency_kwh=float(e),
        unused_paid_kwh=v,curtailed_pv_kwh=r,soc_start_kwh=float(state),soc_end_kwh=float(end_state),
        plan_cost_yuan=p*g,increase_cost_yuan=1.5*p*a,refund_yuan=p*b,penalty_yuan=.5*p*b,
        adjustment_net_cost_yuan=1.5*p*a-.5*p*b,emergency_cost_yuan=5*p*e,
        cash_cost_yuan=p*g+1.5*p*a-.5*p*b+5*p*e)

def run_strategy(data_dir, out_dir, trade_hours=(0,6,12,18), *, forecast_hours=None,
        config=None, start_day=0, days=None, name=None, force=False):
    config={**DEFAULT,**(config or {})}; trade_hours=tuple(trade_hours)
    forecast_hours=tuple(trade_hours if forecast_hours is None else forecast_hours)
    validate_run_arguments(trade_hours,forecast_hours,config,start_day,days)
    name=name or 'H'+'_''.join(map(str,trade_hours))
    config.update(trade_hours=list(trade_hours),forecast_hours=list(forecast_hours),
        start_day=start_day,days=days,strategy=name)
    if 0 not in trade_hours or 0 not in forecast_hours or not set(forecast_hours)≤set(HOURS):
        raise ValueError('Bad schedule')
    target=Path(out_dir)/name;target.mkdir(parents=True,exist_ok=True)
    sig=signature(data_dir,config)
    if (target/'summary.json').exists() and not force:
        old=json.loads((target/'summary.json').read_text())
        if old.get('run_signature')==sig and old.get('complete'): return old
        raise RuntimeError(f'Incompatible existing run: {target}. Use new output or --force.')
    data=load_inputs(data_dir)
    if config['pv_source']≠'supplied':
        data['pv_f']=data['pv_linear_f'].copy()
    stop=min(len(data['dates']),start_day+(days or len(data['dates'])))
    p=data['price']; kappa=config.get('terminal_weight',float(np.min(p)/ETA_C))
    opts=dict(terminal_weight=kappa,time_limit=config['time_limit'],mip_gap=config['mip_gap'])
    state=float(config['initial_soc']);events=[];rows=[];daily=[];g=q=None
    stats=dict(solves=0,seconds=0,unproven=0,max_gap=0,max_physical_error=0,surplus_direct_steps=0)
    beginning=time.perf_counter()

    def solve(L,G,prices,s,**kwargs):
        sol=solve_dispatch(L,G,prices,s,**opts,**kwargs)
        stats['solves']+=1;stats['seconds']+=sol['solve_seconds']
        stats['unproven']+=int(not sol['optimality_proven'])
        stats['max_gap']=max(stats['max_gap'],sol['gap'] or 0.)
        stats['max_physical_error']=max(stats['max_physical_error'],sol['checks']['max_violation_kwh'])
        return sol

    def log_event(day,h,kind,sol,meta,first,last,bridge=False):
        events.append(dict(window_date=str(data['dates'][day]),day_index=day,kind=kind,
            decision_at=dt.datetime.combine(data['dates'][day],dt.time(h)).isoformat(),
            forecast_hour=meta['forecast_hour'],history_indices=meta['history_indices'],
            draw_indices=meta['draw_indices'],history_latest_end=meta['history_latest_end'],

```

```

scenarios=meta['scenarios'],commit_start=first,commit_end=last,locked_bridge=bridge,
status=sol['status'],gap=sol['gap'],solve_seconds=sol['solve_seconds'],
objective=sol['objective'],objective_is_cash=False,
max_violation_kwh=sol['checks']['max_violation_kwh']))

def control(L,G,prices,base,booked,future,actual_l,actual_pv):
    nonlocal state
    surplus=max(booked[0]+actual_pv-actual_l,0.)
    deficit=max(actual_l-booked[0]-actual_pv,0.)
    if surplus>1e-9:
        stats['surplus_direct_steps']+=1
        c=min(surplus,RATE,max((HIGH-state)/ETA_C,0.));d=e=0.
    elif config['controller']=='greedy':
        c=0.;d=min(deficit,RATE,max((state-LOW)*ETA_D,0.));e=max(deficit-d,0.)
    else:
        L=L.copy();G=G.copy();L[0]=actual_l;G[0]=actual_pv
        sol=solve(L[None,:],G[None,:],prices,state,mode='control',base_g=base,
            booked_q=booked,future_adjust_mask=future)
        c=float(sol['c'][0,0]);d=float(sol['d'][0,0]);e=float(sol['e'][0,0])
    return c,d,e

def execute(day,t,L,G,prices,base,booked,future,fh):
    nonlocal state
    c,d,e=control(L,G,prices,base,booked,future,data['load_a'][day,t]/6,data['pv_a'][day,t]/6)
    row=make_record(data,day,t,name,float(base[0]),float(booked[0]),c,d,e,state,fh)
    rows.append(row);state=row['soc_end_kwh']
    if not LOW-1e-5≤state≤HIGH+1e-5: raise RuntimeError('Executed SOC violation')

def finish_day(day):
    part=rows[-144:]
    if len(part)≠144 or len(set(r['window_date'] for r in part))≠1: raise RuntimeError('Incomplete
window')
    cash=sum(r['cash_cost_yuan'] for r in part)
    daily.append(dict(strategy=name>window_date=day,cash_cost_yuan=cash,
        soc_start_kwh=part[0]['soc_start_kwh'],soc_end_kwh=part[-1]['soc_end_kwh'],
        inventory_adjusted_cost_yuan=cash-kappa*(part[-1]['soc_end_kwh']-part[0]['soc_start_kwh']),
        emergency_kwh=sum(r['emergency_kwh'] for r in part)))

for day in range(start_day,stop):
    # 00:00: old last interval is not known; sign new 144 intervals now.
    L,G,meta=scenario_inputs(data,day,0,0,0,config)
    fm=future_mask(trade_hours,0); locked=np.zeros(144,bool)
    bridge=day>start_day
    if bridge:
        oldg,oldq=g.copy(),q.copy();oldfh=latest_hour(forecast_hours,143)
        oldL=float(data['load_f'][day-1,143]/6)
        # Only already issued forecasts; do not read the unexecuted actual.
        oldG=float(data['pv_f'][day-1,HOURS.index(oldfh),143]/6)
        L=np.column_stack((np.full(len(L),oldL),L));G=np.column_stack((np.full(len(G),oldG),G))
        locked=np.r_[True,locked];fm=np.r_[False,fm];prices=np.r_[p[-1],p]
        booked=np.r_[oldq[-1],np.zeros(144)]
    else: prices=p;booked=np.zeros(144)

sol=solve(L,G,prices,state,mode='dayahead',locked_mask=locked,booked_q=booked,future_adjust_mask=fm)
g=sol['g'][-144:].copy();q=g.copy()
log_event(day,0,'dayahead',sol,meta,0,144,bridge)
if bridge:
    base=np.r_[oldg[-1],g];booked=np.r_[oldq[-1],q]
    lf=np.r_[oldL,data['load_f'][day]/6];pf=np.r_[oldG,data['pv_f'][day,0]/6]
    execute(day-1,143,lf,pf,prices,base,booked,fm,oldfh)
    finish_day(day-1)
for t in range(143):
    h=next((h for h in trade_hours if h and START[h]==t),None)
    fh=latest_hour(forecast_hours,t)
    if h is not None:
        first,last=current_block(trade_hours,h)

```



```

L,G,meta=scenario_inputs(data,day,fh,h,t,config)
current=np.arange(t,144)<last
sol=solve(L,G,p[t:],state,mode='adjust',base_g=g[t:],booked_q=q[t:],
          current_mask=current,future_mask=future_mask(trade_hours,t))
if not np.array_equal(sol['signed_mask'],current): raise RuntimeError('Wrong signed
block')

q[t:last]=sol['q'][:last-t]
log_event(day,h,'adjust',sol,meta,first,last)
lf=data['load_f'][day,t:]/6;pf=data['pv_f'][day,HOURS.index(fh),t:]/6
execute(day,t,lf,pf,p[t:],g[t:],q[t:],future_mask(trade_hours,t),fh)
if (day-start_day+1)%10==0 or day==stop-1:
    dump(target/'progress.json',dict(day=str(data['dates'][day]),days_started=day-start_day+1,
        total_days=stop-start_day,elapsed_seconds=time.perf_counter()-beginning,solver=stats))
    print(f'{name}: {day-start_day+1}/{stop-start_day}, {time.perf_counter()-
beginning:.1f}s',flush=True)
    # No next-day contract/forecast exists at the end of the selected experiment.
    day=stop-1;fh=latest_hour(forecast_hours,143)
    execute(day,143,data['load_f'][day,143:]/6,data['pv_f'][day,HOURS.index(fh),143:]/6,
        p[143:],g[143:],q[143:],np.zeros(1,bool),fh)
    finish_day(day)
    write_csv(target/'trace.csv',rows);write_csv(target/'daily.csv',daily)
    dump(target/'event_log.json',dict(config=config,events=events))
    startup=[]
    if start_day==0:
        z=data['first_unexecuted_interval'];G=float(data['pv_unused_first_mean_kw'][:0]/6)
        g0=max(float(data['load_f'][:0,0]/6)-G,0.)
        e0=max(z['load_mean_kw']/6-g0-z['pv_mean_kw']/6,0.)
        startup=[make_record(data,0,0,name,g0,g0,0.,0.,e0,config['initial_soc'],0,startup=True)]
        write_csv(target/'startup.csv',startup)
    total=sum(r['cash_cost_yuan'] for r in rows)
    summary=dict(strategy=name,config=config,complete=True,run_signature=sig,intervals=len(rows),
        windows=stop-
start_day,cash_cost_yuan=total,soc_start_kwh=config['initial_soc'],soc_end_kwh=state,
        inventory_adjusted_cost_yuan=total-kappa*(state-config['initial_soc']),
        emergency_kwh=sum(r['emergency_kwh'] for r in rows),
        original_plan_cost_yuan=sum(r['plan_cost_yuan'] for r in rows),
        net_adjustment_cost_yuan=sum(r['adjustment_net_cost_yuan'] for r in rows),
        emergency_cost_yuan=sum(r['emergency_cost_yuan'] for r in rows),
        startup_cash_cost_yuan=sum(r['cash_cost_yuan'] for r in startup),
        natural_period_cash_cost_yuan=sum(r['cash_cost_yuan'] for r in rows if r['interval_start'][:10]
≤str(data['dates'][stop-1]))+sum(r['cash_cost_yuan'] for r in startup),
        execution='current 10-min actual mean fast-feedback approximation; future means unavailable',
        elapsed_seconds=time.perf_counter()-beginning,solver=stats)
    dump(target/'summary.json',summary)
    return summary

def select_and_compare(out_dir):
    import pandas as pd
    root=Path(out_dir);table=[];dev=[];test=[]
    for path in sorted(root.glob('H*/summary.json')):
        s=json.loads(path.read_text());table.append(s)
        d=pd.read_csv(path.parent/'daily.csv')
        v=d[d.window_date.between('2025-02-17','2025-02-23')]
        w=d[d.window_date≥'2025-02-25']

    dev.append(dict(strategy=s['strategy'],windows=len(v),score=float(v.inventory_adjusted_cost_yuan.sum()))
)

    test.append(dict(strategy=s['strategy'],windows=len(w),cash_cost_yuan=float(w.cash_cost_yuan.sum()),
        score=float(w.inventory_adjusted_cost_yuan.sum()))
    if not table:return
    selected=min(dev,key=lambda x:x['score'])['strategy']
    hindsight=min(table,key=lambda x:x['inventory_adjusted_cost_yuan'])['strategy']
    dump(root/'selection.json',dict(selected_strategy=selected,retrospective_best=hindsight,
        freeze_time='2025-02-25T00:00:00',development_windows='2025-02-17 through 2025-02-23',
        score='cash minus minimum-price inventory change; not an electricity bill',

```



```

        parameters='8 scenes,28 history days,min14,seed20260912 preset before comparison',
        selection_is_historical_simulation=True,development=dev,test=test))
dump(root/'comparison.json',table)

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--
data',default=str(Path(__file__).resolve().parent.parent/'data'))
    ap.add_argument('--out',required=True);ap.add_argument('--days',type=int);ap.add_argument('--
workers',type=int,default=4)
    ap.add_argument('--schedule',default='all');ap.add_argument('--scenarios',type=int,default=8)
    ap.add_argument('--force',action='store_true');args=ap.parse_args()
    schedules=SCHEDULES if args.schedule=='all' else [tuple(map(int,args.schedule.split(',')))]
    with ProcessPoolExecutor(max_workers=min(args.workers,len(schedules))) as pool:
        jobs=[pool.submit(run_strategy,args.data,args.out,s,config=
{'scenarios':args.scenarios},days=args.days,force=args.force) for s in schedules]
        for job in as_completed(jobs):
            r=job.result();print(r['strategy'],r['cash_cost_yuan'],flush=True)
    select_and_compare(args.out)

if __name__=='__main__':main()

```

52. question3/planning/code/q3_sensitivity.py

源文件 SHA-256: d22d719477dddca6e3c4b2630ea4c5e2492410f9a266202be31f3dd2b54ead27 ; 代码行数: 40。

```

"""Predeclared bounded robustness study; never tune on the test-period outcome."""
import argparse, json
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor, as_completed
from q3_runner import run_strategy, dump

def jobs_for(stage):
    if stage=='development':
        return [(f'scenes_{m}',16,7,dict(scenarios=m)) for m in (4,8,16)]+[
            (f'seed_{seed}',16,7,dict(seed=seed)) for seed in (20260913,20260914)]
    if stage=='algorithms':
        return [(f'stochastic_mpc',24,60,{}),(f'deterministic_mpc',24,60,dict(scenarios=1)),
            (f'stochastic_greedy',24,60,dict(controller='greedy')),
            (f'initial_1200',24,60,dict(initial_soc=1200)),
            (f'initial_10800',24,60,dict(initial_soc=10800)),
            (f'linear_reconstruction',24,60,dict(pv_source='linear'))]
    raise ValueError(stage)

def run_study(data,out,stage,workers=2):
    target=Path(out)/stage;target.mkdir(parents=True,exist_ok=True)
    spec=jobs_for(stage)
    dump(target/'design.json',dict(schedule=[0,6,12,18],jobs=spec,
        purpose='preset rich schedule; sensitivity only, not parameter selection',
        local_initial_states=True,not_full_year=True,
        terminal_rule='same final one-step horizon in every matched run'))
    results=[]
    with ProcessPoolExecutor(max_workers=workers) as pool:
        futures=[pool.submit(run_strategy,data,target,(0,6,12,18),config=cfg,
            start_day=start,days=days,name=name) for name,start,days,cfg in spec]
        for future in as_completed(futures):
            r=future.result();results.append(r)
            print(stage,r['strategy'],r['cash_cost_yuan'],flush=True)
    dump(target/'comparison.json',results)
    return results

if __name__=='__main__':
    ap=argparse.ArgumentParser();ap.add_argument('--
data',default=str(Path(__file__).resolve().parents[1]/'data'))
    ap.add_argument('--out',required=True);ap.add_argument('--stage',choices=
['development','algorithms'],required=True)
    ap.add_argument('--workers',type=int,default=2);a=ap.parse_args()
    run_study(a.data,a.out,a.stage,a.workers)

```

53. question3/planning/code/run_all.py

源文件 SHA-256: e5f88633f3bddff63baab78c8180bfd6e146b134d60b5d9496d167a4fb3294a6 ; 代码行数: 46。

```

"""Complete reproducible Q3 study. All outputs stay under the chosen directory."""
import argparse, json, subprocess, sys
from pathlib import Path
from q3_runner import run_strategy
from q3_sensitivity import run_study
from q3_audit import audit_run
from q3_reports import build_reports
from q3_boundaries import run_boundary_study
from q3_causal_replay import run_replay
from q3_figures import build_figures

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--data', default=str(Path(__file__).resolve().parents[1]/'data'))
    parser.add_argument('--out', default=str(Path(__file__).resolve().parents[1]/'results/reproduced'))
    parser.add_argument('--workers', type=int, default=4)
    args=parser.parse_args(); root=Path(args.out); code=Path(__file__).resolve().parent
    if args.workers<1: parser.error('--workers must be positive')
    subprocess.run([sys.executable, str(code/'q3_io.py'), '--data', args.data], check=True)
    subprocess.run([sys.executable, '-m', 'unittest', 'discover', '-s', str(code), '-p', 'test*.py'], check=True)
    subprocess.run([sys.executable, str(code/'q3_runner.py'), '--data', args.data, '--out', str(root/'main'),
                    '--workers', str(args.workers)], check=True)
    for stage in ('development', 'algorithms'):
        run_study(args.data, root/'sensitivity', stage, min(2, args.workers))
        run_strategy(args.data, root/'ablation', (0, 6, 12, 18), forecast_hours=(0, ), name='old_forecast_all_trades')
        run_strategy(args.data, root/'ablation', (0, 6, 12, 18), config={'controller': 'greedy'}, name='all_trades_greedy')
        audits={}
        for folder in (root/'main', root/'ablation', root/'sensitivity/development', root/'sensitivity/algorithms'):
            for p in sorted(folder.glob('*/*summary.json')):
                result=audit_run(p.parent, Path(args.data)/'q3_inputs.npz')
                audits[str(p.parent.relative_to(root))]=result
                if not result['passed']:
                    raise RuntimeError(f'Independent audit failed: {p.parent}')
    {result.get("failed_checks")}
    (root/'audit_all.json').write_text(json.dumps(audits, ensure_ascii=False, indent=2, default=str))
    selected=json.loads((root/'main/selection.json').read_text())['selected_strategy']
    run_replay(args.data, root/'main'/selected, root/'causal_selected', 24, 7)
    run_replay(args.data, root/'main'/'H0', root/'causal_H0', 24, 7)
    run_replay(args.data, root/'ablation'/'old_forecast_all_trades', root/'causal_old_forecast', 24, 7)
    run_boundary_study(args.data, root/'main', root/'boundaries')
    build_reports(root/'main', args.data, root/'reports')
    build_figures(args.data, root, root/'figures')
    from q3_conclusions import build_conclusions
    build_conclusions(root, root/'reports/问题三_结果与评价.md')
    print('Complete:', root)

if __name__=='__main__': main()

```

54. question3/planning/code/test_optimizer.py

源文件 SHA-256: b267ff4fb2c267c995f1a3c1d8629278cdb0a63239bfc7afb033fc36a63a0a3f ; 代码行数: 119。

```

"""Physical, contractual and settlement tests for the Q3 optimizer."""
import unittest
import numpy as np

from q3_optimizer import (solve_dispatch, cash_components, LOW, HIGH, RATE,
                          ETA_C, ETA_D)

class DispatchTests(unittest.TestCase):
    def solve(self, load, pv=None, p=None, s0=LOW, **kwargs):
        load = np.atleast_2d(np.asarray(load, float))
        pv = np.zeros_like(load) if pv is None else np.atleast_2d(pv)
        p = np.ones(load.shape[1]) if p is None else p
        return solve_dispatch(load, pv, p, s0, terminal_weight=0, **kwargs)

    def test_cancellation_refund_100_to_80_cost_90(self):
        r = self.solve([80], mode="adjust", base_g=[100], current_mask=[True])
        self.assertAlmostEqual(r["q"][0], 80, places=6)
        self.assertAlmostEqual(r["objective"], -10, places=6)
        cash = cash_components([1], [100], r["q"], r["e"][0])
        self.assertAlmostEqual(cash["total_cash_cost"], 90, places=6)
        self.assertAlmostEqual(cash["refund"], 20, places=6)
        self.assertAlmostEqual(cash["cancellation_penalty"], 10, places=6)

    def test_increase_100_to_120_cost_130(self):
        r = self.solve([120], mode="adjust", base_g=[100], current_mask=[True])
        self.assertAlmostEqual(r["q"][0], 120, places=6)
        self.assertAlmostEqual(r["objective"], 30, places=6)
        self.assertAlmostEqual(cash_components([1], [100], r["q"], r["e"][0])["total_cash_cost"], 130,
        places=6)

    def test_fixed_contract_does_not_receive_automatic_refund(self):
        r = self.solve([80], mode="control", base_g=[100], booked_q=[100], locked_mask=[True])
        self.assertEqual(r["q"][0], 100)
        self.assertEqual(r["signed_mask"].sum(), 0)
        self.assertEqual(r["a"].sum()+r["b"].sum(), 0)
        self.assertEqual(r["original_cost_in_objective"], 0)
        self.assertEqual(r["expected_adjustment_cost"], 0)
        self.assertAlmostEqual(cash_components([1], [100], r["q"], r["e"][0])["total_cash_cost"], 100)

    def test_current_q_is_shared_and_future_q_can_differ(self):
        r = self.solve([[0, 0], [100, 100]], mode="adjust", base_g=[0, 0],
                        current_mask=[True, False], future_adjust_mask=[False, True])
        self.assertAlmostEqual(r["q_scenarios"][0,0], r["q_scenarios"][1,0])
        self.assertGreater(r["q_scenarios"][1,1]-r["q_scenarios"][0,1], 1)
        np.testing.assert_array_equal(r["signed_mask"], [True, False])
        self.assertEqual(r["q"][1], 0) # Future proposal is not signed.

    def test_future_control_can_plan_adjustment_but_signs_nothing(self):
        r = self.solve([10, 100], mode="control", base_g=[10, 0],
                        locked_mask=[True, False], future_adjust_mask=[False, True])
        self.assertEqual(r["signed_mask"].sum(), 0)
        np.testing.assert_array_equal(r["q"], [10, 0])
        np.testing.assert_allclose(r["q_scenarios"], [[10, 100]], atol=1e-6)
        self.assertAlmostEqual(r["objective"], 150, places=5)

    def test_locked_adjusted_bridge_is_honored_without_second_charge(self):
        r = self.solve([120, 80], mode="dayahead", booked_q=[120, 0],
                        base_g=[100, 0], locked_mask=[True, False])
        np.testing.assert_allclose(r["g"], [120, 80], atol=1e-6)
        np.testing.assert_array_equal(r["signed_mask"], [False, True])
        self.assertAlmostEqual(r["new_plan_cost"], 80, places=6)
        self.assertAlmostEqual(r["objective"], 80, places=6)

```

```

def test_dayahead_no_future_adjustments_is_q2_structure(self):
    r = self.solve([[10, 100], [20, 120]], p=[.5, 1.5])
    np.testing.assert_allclose(r["q_scenarios"], np.broadcast_to(r["g"], (2,2)), atol=1e-7)
    self.assertEqual(r["expected_adjustment_cost"], 0)
    self.assertAlmostEqual(r["objective"], np.array([.5,1.5]) @ r["g"]+r["expected_emergency_cost"],
places=6)

def test_dayahead_future_recourse_and_original_fee(self):
    r = self.solve([[0], [100]], future_adjust_mask=[True], probabilities=[.9, .1])
    self.assertAlmostEqual(r["g"][0], 0, places=6)
    np.testing.assert_allclose(r["q_scenarios"], [[0],[100]], atol=1e-7)
    self.assertAlmostEqual(r["objective"], 15, places=6)

def test_storage_rate_and_efficiency(self):
    r = self.solve([0, 1000], pv=[[2000, 0]], mode="control", base_g=[0,0])
    self.assertAlmostEqual(r["c"][0,0], RATE, places=5)
    self.assertAlmostEqual(r["d"][0,1], RATE*ETA_C*ETA_D, places=5)
    self.assertAlmostEqual(r["e"][0,1], 1000-RATE*ETA_C*ETA_D, places=5)
    self.assertTrue(r["checks"]["passed"])
    self.assertAlmostEqual(r["s"][0,-1], LOW, places=5)

def test_emergency_cannot_charge_and_upper_state_holds(self):
    r = self.solve([1000,0], pv=[[0,3000]], s0=HIGH, mode="control", base_g=[0,0])
    self.assertLessEqual(r["s"].max(), HIGH+1e-6)
    self.assertLessEqual(np.max(np.minimum(r["c"],r["e"])), 1e-7)
    self.assertLessEqual(np.max(np.minimum(r["c"],r["d"])), 1e-7)
    self.assertTrue(r["checks"]["passed"])

def test_original_large_contract_above_sufficient_bound(self):
    r = self.solve([0], mode="control", base_g=[5000], booked_q=[5000], locked_mask=[True])
    self.assertEqual(r["q"][0], 5000)
    self.assertTrue(r["checks"]["passed"])

def test_soft_terminal_penalty_is_not_cash(self):
    r = solve_dispatch([[0]], [[0]], [1], LOW, mode="control", base_g=[0],
terminal_weight=2, s_ref=6000)
    self.assertAlmostEqual(r["terminal_cost"], 9600)
    self.assertAlmostEqual(r["objective"], 9600)
    self.assertEqual(cash_components([1], [0], [0], [0])["total_cash_cost"], 0)

def test_input_rejections(self):
    for kwargs in [{"mode":"bad"}, {"mode":"adjust"},
{"current_mask":[True]}, {"locked_mask":[True]},
{"future_adjust_mask":[True],"locked_mask":[True]},
{"probabilities": [.9]}, {"mip_gap":-1}]:
        with self.subTest(kwargs=kwargs), self.assertRaises(ValueError):
            self.solve([1], **kwargs)

def test_zero_price_degeneracy_is_verified(self):
    r = self.solve([[10],[20]], p=[0], mode="adjust", base_g=[100], current_mask=[True])
    self.assertEqual(r["objective"], 0)
    self.assertEqual(np.minimum(r["a"],r["b"]).sum(), 0)
    self.assertTrue(r["checks"]["passed"])

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

55. question3/planning/code/test_q3_audit.py

源文件 SHA-256: 48cfbeea46667ea369adbe6037e0f6f34da24e8f42c9d2eec344d28319fdf354 ; 代码行数: 116。

```

"""Independent negative controls for the exported-ledger audit (no optimizer)."""
import csv
import datetime as dt
import json
from pathlib import Path
import tempfile
import unittest

import numpy as np

from q3_audit import REQUIRED, audit_run

class LedgerAuditTests(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.run = Path(self.tmp.name)
        self.day = dt.date(2025,2,2)
        self.origin = dt.datetime(2025,2,2,0,10)
        self.rows = []
        for i in range(144):
            row = {k:0.0 for k in REQUIRED}
            row.update(strategy='H0_6', window_date=str(self.day),
                       interval_start=(self.origin+dt.timedelta(minutes=10*i)).isoformat(),
                       interval_end=(self.origin+dt.timedelta(minutes=10*(i+1))).isoformat(),
                       forecast_issue_time=dt.datetime(2025,2,2,0 if i<35 else 6).isoformat(),
                       price_yuan_per_kwh=1.,load_forecast_kw=600.,load_actual_kw=600.,
                       actual_estimated=False,g_kwh=100.,q_kwh=100.,soc_start_kwh=6000.,
                       soc_end_kwh=6000.,plan_cost_yuan=100.,cash_cost_yuan=100.)
            self.rows.append(row)
        self.events = {'config':{'trade_hours':[0,6], 'forecast_hours':[0,6], 'start_day':1},
                       'events':[]}
        for h,kind,begin,end in ((0,'dayahead',0,144),(6,'adjust',35,144)):
            self.events['events'].append(dict(kind=kind,window_date=str(self.day),
                                              decision_at=dt.datetime(2025,2,2,h).isoformat(),forecast_hour=h,
                                              history_indices=[],draw_indices=[],history_latest_end=None,
                                              commit_start=begin,commit_end=end,scenarios=1))
        self.inputs = {'dates':[self.day], 'window_end':[self.origin+dt.timedelta(days=1)],
                       'load_f':np.full((1,144),600.), 'load_a':np.full((1,144),600.),
                       'pv_f':np.zeros((1,4,144)), 'pv_a':np.zeros((1,144)),
                       'price':np.ones(144), 'actual_estimated':np.zeros((1,144),bool)}

    def tearDown(self):
        self.tmp.cleanup()

    def audit(self):
        with (self.run/'trace.csv').open('w',newline='',encoding='utf-8') as f:
            writer=csv.DictWriter(f,fieldnames=REQUIRED)
            writer.writeheader();writer.writerows(self.rows)
        (self.run/'event_log.json').write_text(json.dumps(self.events))
        return audit_run(self.run,self.inputs)

    def assertFails(self,result,name):
        self.assertFalse(result['passed'])
        self.assertTrue(any(c['name']==name for c in result['failed_checks']),result['failed_checks'])

    def test_balanced_ledger_passes(self):
        result=self.audit()
        self.assertTrue(result['passed'],result['failed_checks'])
        self.assertEqual(result['sums']['cash_cost_yuan'],14400.)

    def cancel_example(self):
        self.rows[35].update(q_kwh=80.,decrease_kwh=20.,load_actual_kw=480.,
                             refund_yuan=20.,penalty_yuan=10.,adjustment_net_cost_yuan=-10.,cash_cost_yuan=90.)

```

```

        self.inputs['load_a'][0,35]=480.

def test_refund_then_half_penalty_is_ninety(self):
    self.cancel_example()
    result=self.audit()
    self.assertTrue(result['passed'],result['failed_checks'])
    self.assertEqual(result['sums']['cash_cost_yuan'],14390.)

def test_wrong_refund_sign_rejected(self):
    self.cancel_example()
    self.rows[35]['refund_yuan']=-20.
    self.assertFails(self.audit(), 'fee_refund_yuan')

def test_energy_conversion_error_rejected(self):
    self.rows[0]['load_actual_kwh']=100.
    self.assertFails(self.audit(), 'energy_balance_kwh')

def test_soc_reset_rejected(self):
    self.rows[72]['soc_start_kwh']=5000.
    self.assertFails(self.audit(), 'continuous_soc_kwh')

def test_emergency_charging_rejected(self):
    self.rows[50].update(charge_kwh=1., emergency_kwh=1., soc_end_kwh=6000.9,
                        emergency_cost_yuan=5., cash_cost_yuan=105.)
    self.assertFails(self.audit(), 'emergency_does_not_charge')

def test_missing_contract_interval_rejected(self):
    self.rows.pop()
    self.assertFails(self.audit(), 'window_2025-02-02_144_slots')

def test_disabled_future_forecast_rejected(self):
    self.rows[35]['forecast_issue_time']='2025-02-02T12:00:00'
    self.assertFails(self.audit(), 'execution_uses_enabled_forecast')

def test_unrealized_history_rejected(self):
    self.events['events'][1].update(history_indices=[0], draw_indices=[0],
                                     history_latest_end='2025-02-03T00:10:00')
    self.assertFails(self.audit(), 'event_1_all_history_released')

def test_wrong_six_hour_commit_block_rejected(self):
    self.events['events'][1]['commit_end']=71
    self.assertFails(self.audit(), 'event_1_commit_block_correct')

def test_unscheduled_early_adjustment_rejected(self):
    self.rows[10].update(q_kwh=90., decrease_kwh=10., emergency_kwh=10.,
                        refund_yuan=10., penalty_yuan=5., adjustment_net_cost_yuan=-5.,
                        emergency_cost_yuan=50., cash_cost_yuan=145.)
    self.assertFails(self.audit(), 'no_adjustment_before_first_enabled_update')

if __name__=='__main__':
    unittest.main()

```

56. question3/planning/code/test_q3_cache.py

源文件 SHA-256: 8a1d6016bcd8d3bd460fb7d74b6285323243d818730e2252a5405e42d8f9a7d2 ; 代码行数: 28.


```

import tempfile, unittest
from pathlib import Path
from q3_runner import signature, validate_run_arguments, DEFAULT, SCHEDULES

class CacheGuardTests(unittest.TestCase):
    def test_actual_forecast_change_invalidates_cache_with_same_manifest(self):
        with tempfile.TemporaryDirectory() as tmp:
            root=Path(tmp)
            names=['source_manifest.json', 'attachment1.xlsx', 'attachment2.xlsx', 'attachment3.xlsx',
                  'load_forecasts.xlsx', 'pv_forecasts.xlsx', 'result3_template.xlsx']
            for name in names: (root/name).write_bytes(b'unchanged fixture')
            old=signature(root, DEFAULT)
            (root/'pv_forecasts.xlsx').write_bytes(b'replacement forecasts')
            self.assertNotEqual(old, signature(root, DEFAULT))
            self.assertEqual((root/'source_manifest.json').read_bytes(), b'unchanged fixture')

    def test_eight_schedules_remain_accepted(self):
        for schedule in SCHEDULES: validate_run_arguments(schedule, schedule, DEFAULT, 0, None)
        validate_run_arguments((0, 6, 12, 18), (0, ), DEFAULT, 24, 60)

    def test_invalid_configuration_rejected_before_computation(self):
        for hours in [(0, 7), (0, 6, 6), (6, 0), (6, ), (0.0, 6.0), (False, 6)]:
            with self.assertRaises(ValueError): validate_run_arguments(hours, (0, ), DEFAULT, 0, 3)
        for cfg in [{'pv_source': 'typo'}, {'controller': 'typo'}, {'scenarios': 0}, {'initial_soc': 0}]:
            with self.assertRaises(ValueError): validate_run_arguments((0, ), (0, ), {**DEFAULT, **cfg}, 0, 3)
            with self.assertRaises(ValueError): validate_run_arguments((0, ), (0, ), DEFAULT, 0, 0)

if __name__=='__main__': unittest.main()

```

57. question3/planning/code/test_q3_chronology.py

源文件 SHA-256: c116f9d715dc487322fbb4b1256fa78bfd9f6725b43f4fa99d2237adcb2546a4 ; 代码行数: 49。

```

"""Counterfactual information-barrier tests on real first-day inputs."""
import tempfile, unittest
from pathlib import Path
from unittest.mock import patch
import numpy as np
import pandas as pd
from q3_io import load_inputs
from q3_risk import available_history
from q3_runner import run_strategy

ROOT=Path(__file__).resolve().parents[1]

class ChronologyTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls): cls.data=load_inputs(ROOT/'data')

    def replay(self,data,schedule=(0,6),days=1):
        with tempfile.TemporaryDirectory() as tmp,patch('q3_runner.load_inputs',return_value=data):
            r=run_strategy(ROOT/'data',tmp,schedule,days=days,config={'scenarios':1})
            return pd.read_csv(Path(tmp)/r['strategy']/r['trace.csv'])

    def test_disabled_versions_cannot_change_plan_or_execution(self):
        a=self.replay(self.data)
        d={**self.data,'pv_f':self.data['pv_f'].copy()}
        d['pv_f'][:,2:,:]=np.nan_to_num(d['pv_f'][:,2:,:])*10+123456
        b=self.replay(d)
        np.testing.assert_allclose(a.select_dtypes('number'),b.select_dtypes('number'),atol=1e-8)

    def test_unobserved_afternoon_cannot_change_morning(self):
        a=self.replay(self.data)
        d={**self.data,'load_a':self.data['load_a'].copy()}
        d['load_a'][0,100:]*=1.5
        b=self.replay(d)
        cols=['g_kwh','q_kwh','charge_kwh','discharge_kwh','soc_end_kwh']
        np.testing.assert_allclose(a.loc[:99,cols],b.loc[:99,cols],atol=1e-8)

    def test_midnight_old_tail_not_seen_when_signing_new_day(self):
        a=self.replay(self.data,days=2)
        d={**self.data,'load_a':self.data['load_a'].copy()}
        d['load_a'][0,143]*=1.4
        b=self.replay(d,days=2)
        np.testing.assert_allclose(a.loc[144:,'g_kwh'],b.loc[144:,'g_kwh'],atol=1e-8)

    def test_history_release_boundary(self):
        self.assertNotIn(13,available_history(self.data,14,0))
        self.assertIn(13,available_history(self.data,14,6))
        self.assertEqual(len(available_history(self.data,40,0)),28)

if __name__=='__main__':unittest.main()

```

58. question3/planning/code/test_q3_conclusions.py

源文件 SHA-256: fb23ad5ba0bbb766cf50f4d1f6938d07d98d5246f4a1abeba5fcad5fcadcaaf5 ; 代码行数: 37.

```

"""Guard report direction and prevent absent experiments becoming conclusions."""
import json
from pathlib import Path
import tempfile
import unittest

from q3_conclusions import build_conclusions, comparison_sentence, savings

class ConclusionTests(unittest.TestCase):
    def test_cheaper_candidate_reports_saving(self):
        self.assertEqual(savings(90,100),{'saving_yuan':10.,'saving_percent':10.})
        self.assertIn('减少10.00元',comparison_sentence('甲','乙',90,100))

    def test_costlier_mpc_is_not_called_better(self):
        text=comparison_sentence('MPC','贪心',2006593.253,2006577.520)
        self.assertIn('增加15.73元',text)
        self.assertNotIn('减少',text)

    def test_equal_and_zero_reference_have_no_false_percentage(self):
        self.assertIn('相同',comparison_sentence('甲','乙',0,0))
        self.assertIsNone(savings(1,0)['saving_percent'])

    def test_missing_artifacts_are_explicitly_pending(self):
        with tempfile.TemporaryDirectory() as directory:
            root=Path(directory)/'results';root.mkdir()
            out=Path(directory)/'report.md'
            result=build_conclusions(root,out)
            self.assertFalse(result['complete_main'])
            self.assertIsNone(result['selected_strategy'])
            self.assertGreater(len(result['pending']),0)
            self.assertIn('不能据此宣布',out.read_text())
            self.assertEqual(json.loads(out.parent/'analysis_summary.json').read_text()['main'],[])

if __name__=='__main__':
    unittest.main()

```

59. question3/planning/code/test_q3_io.py

源文件 SHA-256: 4f9bdaee7ae05fa3b162c958c62bafd9ac3b0691e154d9e8f3ff56cc4c0e07d0 ; 代码行数: 99。

```

"""Meaningful checks of supplied Q3 data boundaries and vintage causality."""
import copy
import datetime as dt
from pathlib import Path
import tempfile
import unittest

import numpy as np
from openpyxl import load_workbook

from q3_io import (ROOT, ISSUE_HOURS, ISSUE_START, load_inputs, read_pv,
                   read_actual_source, audit_linear_reconstruction, interval_header,
                   clock_minute, save_bundle, sha)

class InputTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.data=load_inputs()

    def test_confirmed_contract_span(self):
        d=self.data
        self.assertEqual(d['window_start'][0],dt.datetime(2025,2,1,0,10))
        self.assertEqual(d['window_end'][-1],dt.datetime(2026,1,1,0,10))
        self.assertEqual(len(d['dates'])*144,48096)

    def test_vintage_release_mask(self):
        for k,h in enumerate(ISSUE_HOURS):
            s=ISSUE_START[h]
            self.assertTrue(np.isnan(self.data['pv_f'][:,k,s]).all())
            self.assertTrue(np.isfinite(self.data['pv_f'][:,k,s]).all())
            self.assertEqual(self.data['pv_valid'][k].sum(),144-s)

    def test_six_am_column_no_time_shift(self):
        self.assertAlmostEqual(self.data['pv_f'][0,1,35],18.16377916666667)
        self.assertEqual(self.data['window_start']
[0]+dt.timedelta(minutes=35*10),dt.datetime(2025,2,1,6))

    def test_midnight_prefix_kept_separate(self):
        self.assertEqual(self.data['pv_unused_first_mean_kw'].shape,(334,))
        self.assertEqual(self.data['load_f'].shape,(334,144))
        self.assertEqual(self.data['first_unexecuted_interval']['load_mean_kw'],2537.9711)

    def test_observed_cross_midnight_identity(self):
        for key in ('load','pv'):
            n=self.data[f'{key}_nodes']
            np.testing.assert_array_equal(n[:-1,-1],n[1:,0])
            np.testing.assert_array_equal(self.data[f'{key}_a'],(n[:, :-1]+n[:, 1:])/2)

    def test_only_final_actual_estimated(self):
        mask=self.data['actual_estimated']
        self.assertEqual(mask.sum(),1)
        self.assertTrue(mask[-1,-1])

    def test_price_endpoints_averaged_and_rotated_once(self):
        w=load_workbook(ROOT/'data/attachment1.xlsx',read_only=True,data_only=True)
        rows=list(w.active.values);w.close()
        node=np.array([r[1] for r in rows[1:]])
        self.assertAlmostEqual(self.data['price'][0],(node[0]+node[1])/2)
        self.assertAlmostEqual(self.data['price'][-1],(node[-1]+node[0])/2)

    def test_supplied_values_not_reinterpolated(self):
        w=load_workbook(ROOT/'data/pv_forecasts.xlsx',read_only=True,data_only=True)
        for k,h in enumerate(ISSUE_HOURS):

```

```

        a=np.array([r[1:] for r in list(w[f'{h:02d}时报'].values())[1:]))
        if h==0:a=a[:,1:]
        np.testing.assert_array_equal(a,self.data['pv_f'][:,k,ISSUE_START[h]:])
    w.close()

    def test_bad_other_header_rejected(self):
        with tempfile.TemporaryDirectory() as tmp:
            p=Path(tmp)/'bad.xlsx'
            w=load_workbook(ROOT/'data/pv_forecasts.xlsx')
            w['06时报']['B1']='06:00-06:20'
            w.save(p);w.close()
            with self.assertRaises(ValueError):read_pv(p,self.data['dates'])

    def test_linearly_reconstructed_06_does_not_read_0610_actual(self):
        source,_=read_actual_source(ROOT/'data/attachment2.xlsx')
        source['pv'][self.data['dates'][0]][36]+=100000 # 06:10, unavailable at 06:00
        clone={'dates':self.data['dates'],'pv_f':self.data['pv_f'],'metadata':{}}
        audit_linear_reconstruction(clone,ROOT/'data/attachment3.xlsx',source)
        np.testing.assert_array_equal(clone['pv_linear_f'][:,1,:],self.data['pv_linear_f'][:,1,:])

    def test_npz_roundtrip_no_pickle(self):
        with tempfile.TemporaryDirectory() as tmp:
            path=save_bundle(self.data,tmp)
            with np.load(path,allow_pickle=False) as a:
                np.testing.assert_array_equal(a['load_f'],self.data['load_f'])
                np.testing.assert_array_equal(a['pv_f'],self.data['pv_f'])
                self.assertEqual(str(a['dates'][0]),'2025-02-01')

    def test_typo_repairs_are_four_explicit_metadata_entries(self):
        edits=self.data['metadata']['forecast']['header_corrections']
        self.assertEqual(len(edits),4)
        self.assertTrue(all(not e['numerical_values_changed'] for e in edits))
        self.assertTrue(all(e['normalized']=='23:50-次日00:00' for e in edits))

if __name__=='__main__':unittest.main()

```

60. question3/planning/code/test_q3_reports.py

源文件 SHA-256: 3f1b7b816f7f806d35bac3b5f3c332f046bbc6028e5fb6c45e6d776ba8c4e264 ; 代码行数: 140。

```

"""Accounting and natural-calendar-day regression tests for Q3 reporting."""
import copy
from datetime import datetime, timedelta
from pathlib import Path
import tempfile
import unittest

from openpyxl import load_workbook
from q3_reports import (NUMERIC, DATES, audit_trace, day_groups, episodes, export_result3,
                        paper_tables, summarize)

def record(a, *, g=100., q=80., emergency=0., soc=6000.):
    up=max(q-g, 0.); down=max(g-q, 0.)
    r={k: 0. for k in NUMERIC}
    r.update(strategy='H0', window_date=(a-timedelta(minutes=10)).date().isoformat(),
            interval_start=a.isoformat(), interval_end=(a+timedelta(minutes=10)).isoformat(),
            forecast_issue_time=a.replace(hour=0, minute=0).isoformat(), price_yuan_per_kwh=1.,
            load_forecast_kw=6*(q+emergency), load_actual_kw=6*(q+emergency), actual_estimated=False,
            g_kwh=g, q_kwh=q, increase_kwh=up, decrease_kwh=down, emergency_kwh=emergency,
            soc_start_kwh=soc, soc_end_kwh=soc, plan_cost_yuan=g, increase_cost_yuan=1.5*up,
            refund_yuan=down, penalty_yuan=.5*down, adjustment_net_cost_yuan=1.5*up-.5*down,
            emergency_cost_yuan=5*emergency, cash_cost_yuan=g+1.5*up-.5*down+5*emergency)
    return r

class ReportTests(unittest.TestCase):
    def test_cancel_twenty_costs_ninety(self):
        r=record(datetime(2025, 2, 1))
        audit_trace([r], label='cancel')
        self.assertEqual(r['cash_cost_yuan'], 90.)

    def test_increase_twenty_costs_one_thirty(self):
        r=record(datetime(2025, 2, 1), q=120.)
        audit_trace([r], label='increase')
        self.assertEqual(r['cash_cost_yuan'], 130.)

    def test_wrong_double_charged_emergency_fails(self):
        r=record(datetime(2025, 2, 1), emergency=20.)
        r['cash_cost_yuan']+=100.
        with self.assertRaises(AssertionError): audit_trace([r], label='bad')

    def test_natural_day_first_slot_belongs_previous_contract(self):
        zero=datetime(2025, 3, 20)
        rows=[record(zero+timedelta(minutes=10*i)) for i in range(144)]
        group=day_groups(rows)['2025-03-20']
        self.assertEqual(group[0]['window_date'], '2025-03-19')
        self.assertEqual(group[1]['window_date'], '2025-03-20')
        self.assertEqual(summarize(group, 'H0')['q_kwh'], 144*80.)

    def test_emergency_episode_split_midnight(self):
        zero=datetime(2025, 3, 20, 23, 50)
        rows=[record(zero+timedelta(minutes=10*i), emergency=2.) for i in range(3)]
        event=episodes(rows)
        self.assertEqual(len(event), 2)
        self.assertEqual(event[0]['label'], '23:50-24:00')
        self.assertEqual(event[1]['label'], '00:00-00:20')
        self.assertEqual(event[1]['kwh'], 4.)

    def test_numerical_dust_does_not_create_event(self):
        rows=[record(datetime(2025, 3, 20), emergency=1e-8)]
        self.assertEqual(episodes(rows), [])
        self.assertEqual(summarize(rows, 'H0')['emergency_kwh'], 1e-8)

```

```

def test_template_keeps_original_g_and_adjusted_q(self):
    zero=datetime(2025,2,1)
    rows=[record(zero+timedelta(minutes=10*(i+1))) for i in range(144)]
    template=Path(__file__).resolve().parents[1]/'data/result3_template.xlsx'
    with tempfile.TemporaryDirectory() as tmp:
        p=Path(tmp)/'result3.xlsx'
        export_result3(template,p,rows,[record(zero)], 'H0',{'selected_strategy':'H0'})
        w=load_workbook(p,data_only=True)
        self.assertEqual(w['计划购电量']['B2'].value,100.)
        self.assertEqual(w['调整购电量']['B2'].value,80.)
        self.assertEqual(w['调整购电量'].cell(2,147).value,90.*144)
        self.assertEqual(w['充放电量'].max_row,7)
        self.assertEqual(w['充放电量']['F2'].value,6000.)
        w.close()

def test_template_final_fee_includes_emergency_exactly_once(self):
    zero=datetime(2025,2,1)
    rows=[record(zero+timedelta(minutes=10*(i+1))) for i in range(144)]
    # Include cancellation, increase and nonzero emergency in one ledger.
    rows[0]=record(zero+timedelta(minutes=10),emergency=20.)
    rows[1]=record(zero+timedelta(minutes=20),q=120.,emergency=3.)
    audit_trace(rows,label='export cash')
    template=Path(__file__).resolve().parents[1]/'data/result3_template.xlsx'
    with tempfile.TemporaryDirectory() as tmp:
        p=Path(tmp)/'result3.xlsx'
        export_result3(template,p,rows,[record(zero,emergency=7.)], 'H0',{'selected_strategy':'H0'})
        w=load_workbook(p,data_only=True)
        ordinary=144*90.+40.;emergency=5.*23.
        self.assertEqual(w['计划购电量'].cell(2,147).value,144*100.)
        self.assertEqual(w['调整购电量'].cell(2,147).value,ordinary+emergency)
        detail=w['逐合同现金明细']
        headers={c.value:c.column for c in detail[1]}
        self.assertEqual(detail.cell(2,headers['普通购电结算费(元)']).value,ordinary)
        self.assertEqual(detail.cell(2,headers['紧急费(元)']).value,emergency)
        self.assertEqual(detail.cell(2,headers['总现金费(元)']).value,ordinary+emergency)
        # The separate startup emergency belongs to the natural-day view,
        # never to this shifted contract's final fee.
        self.assertEqual(w['调整购电量'].cell(2,147).value,ordinary+emergency+35.)
        w.close()

def test_final_next_year_emergency_only_in_tail_details(self):
    zero=datetime(2025,12,31)
    rows=[record(zero+timedelta(minutes=10*(i+1))) for i in range(144)]
    rows[-2]=record(zero+timedelta(hours=23,minutes=50),emergency=2.)
    rows[-1]=record(datetime(2026,1,1),emergency=3.)
    audit_trace(rows,label='year boundary')
    template=Path(__file__).resolve().parents[1]/'data/result3_template.xlsx'
    with tempfile.TemporaryDirectory() as tmp:
        p=Path(tmp)/'result3.xlsx'
        export_result3(template,p,rows,[record(zero)], 'H0',{'selected_strategy':'H0'})
        w=load_workbook(p,data_only=True)
        emergency=w['紧急购电量']
        self.assertEqual(emergency.max_row,2)
        self.assertEqual(emergency['A2'].value,zero)
        self.assertEqual(emergency['B2'].value,'23:50-24:00')
        self.assertEqual(emergency['C2'].value,2.)
        tail=w['跨年合同末段'];headers={c.value:c.column for c in tail[1]}
        self.assertEqual(tail.max_row,2)
        self.assertEqual(tail.cell(2,headers['紧急购电量(kWh)']).value,3.)
        self.assertEqual(tail.cell(2,headers['紧急费(元)']).value,15.)
        # Contract cash includes both sides of midnight, with no duplication.
        self.assertEqual(w['调整购电量'].cell(2,147).value,144*90.+25.)
        w.close()

def test_all_four_paper_dates_written(self):
    natural={date:[record(datetime.fromisoformat(date)+timedelta(minutes=10*i)) for i in range(144)]

```



```
for date in DATES}
    with tempfile.TemporaryDirectory() as tmp:
        tmp=Path(tmp)
        check,summaries=paper_tables(tmp/'paper.xlsx',tmp/'paper.md',natural,'H0')
        self.assertEqual(set(summaries),set(DATES))
        self.assertEqual(check['excel_errors'],0)
        txt=(tmp/'paper.md').read_text()
        self.assertTrue(all(date in txt for date in DATES))

if __name__=='__main__':unittest.main()
```

61. question3/prediction/build_pv_interval_forecasts.py

源文件 SHA-256: 8ce868fa41184a250398d9feb713599eca3781460c2c9453817e3324d69d124d ; 代码行数: 547。

"""生成问题三 0/6/12/18 时发布的十分钟区间光伏预测。

候选方案:

A: 当前实测边界 + 最新小时预报的分段线性插值;

B: A + 过去 28 个完整日的小时内历史中位形状残差 (整点修正为 0);

C: 根据发布前最近 3 小时“旧预报-实测”残差, 对最新小时节点作指数衰减修正。

最终方案每天、每个发布时间只使用此前 28 天已经实现的候选误差滚动选优;

历史不足 7 天时回退到 A。正式输出从 2025-02-01 至 2025-12-31。

参考工作簿只读取第一行表头, 绝不读取其数据区。

从项目根目录运行:

```
python 附件/question3/build_pv_interval_forecasts.py
"""
```

```
from __future__ import annotations

import argparse
import hashlib
import json
import math
import platform
from collections import defaultdict, deque
from dataclasses import dataclass
from datetime import time
from pathlib import Path

import numpy as np
import pandas as pd
from openpyxl import Workbook, load_workbook
from openpyxl.styles import Alignment, Font, PatternFill

PROJECT_ROOT = Path(__file__).resolve().parents[2]
DEFAULT_ACTUAL = PROJECT_ROOT / "附件" / "附件2.xlsx"
DEFAULT_FORECAST = PROJECT_ROOT / "附件" / "附件3.xlsx"
DEFAULT_REFERENCE = PROJECT_ROOT / "演示数据用于参考格式.xlsx"
DEFAULT_OUTPUT = Path(__file__).resolve().parent / "问题三_光伏区间预测.xlsx"
DEFAULT_RESULTS_DIR = Path(__file__).resolve().parent / "forecast_results"
OUTPUT_START = pd.Timestamp("2025-02-01")
OUTPUT_END = pd.Timestamp("2025-12-31")
CALCULATION_START = pd.Timestamp("2025-01-08")
RELEASE_HOURS = (0, 6, 12, 18)
METHODS = ("A_线性插值", "B_历史形状", "C_残差同化")
SHAPE_DAYS = 28
SELECTION_DAYS = 28
MIN_SELECTION_DAYS = 7
RECENT_HOURS = 3
BIAS_DECAY_HOURS = 3.0
EXTRA_ENDPOINT_HISTORY_DAYS = 7

@dataclass(frozen=True)
class Inputs:
    actual: dict[pd.Timestamp, float]
    forecasts: dict[pd.Timestamp, np.ndarray]
    reference_headers: list[object]

def sha256(path: Path) → str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for block in iter(lambda: stream.read(1024 * 1024), b""):
```

```

        digest.update(block)
    return digest.hexdigest()

def parse_header_minute(value: object) → int:
    if isinstance(value, time):
        return value.hour * 60 + value.minute
    if str(value).strip() in {"0:00+1", "00:00+1", "24:00"}:
        return 1440
    raise ValueError(f"无法识别附件 2 时间表头: {value!r}")

def read_actual(path: Path) → dict[pd.Timestamp, float]:
    workbook = load_workbook(path, read_only=True, data_only=True)
    worksheet = workbook.worksheets[1]
    shape = (worksheet.max_row, worksheet.max_column)
    workbook.close()
    if shape ≠ (366, 145):
        raise ValueError(f"附件 2 光伏表应为 366×145, 实际为 {shape}")

    frame = pd.read_excel(path, sheet_name=1, header=None)
    minutes = [parse_header_minute(value) for value in frame.iloc[0, 1:]]
    if minutes ≠ list(range(10, 1441, 10)):
        raise ValueError("附件 2 光伏端点不是从 00:10 到次日 00:00 的连续十分钟序列")

    actual: dict[pd.Timestamp, float] = {}
    for row_index in range(1, len(frame)):
        date = pd.Timestamp(frame.iloc[row_index, 0]).normalize()
        values = pd.to_numeric(frame.iloc[row_index, 1:], errors="raise").to_numpy(float)
        if values.shape ≠ (144,) or not np.isfinite(values).all() or np.any(values < 0):
            raise ValueError(f"附件 2 第 {row_index + 1} 行光伏数据异常")
        for minute, value in zip(minutes, values, strict=True):
            actual[date + pd.Timedelta(minutes=minute)] = float(value)
    return actual

def read_forecasts(path: Path) → dict[pd.Timestamp, np.ndarray]:
    workbook = load_workbook(path, read_only=True, data_only=True)
    worksheet = workbook.worksheets[0]
    shape = (worksheet.max_row, worksheet.max_column)
    workbook.close()
    if shape ≠ (1461, 26):
        raise ValueError(f"附件 3 应为 1461×26, 实际为 {shape}")

    frame = pd.read_excel(path, sheet_name=0, header=None)
    dates = frame.iloc[1:, 0].replace("", np.nan).ffill()
    releases = frame.iloc[1:, 1].astype(str).str.strip()
    values = frame.iloc[1:, 2:].apply(pd.to_numeric, errors="raise").to_numpy(float)
    if values.shape ≠ (1460, 24) or not np.isfinite(values).all() or np.any(values < 0):
        raise ValueError("附件 3 预测矩阵异常")

    forecasts: dict[pd.Timestamp, np.ndarray] = {}
    for index, (date_value, release_value) in enumerate(zip(dates, releases, strict=True)):
        date = pd.Timestamp(str(date_value)).normalize()
        release_hour = int(release_value.split(":", maxsplit=1)[0])
        if release_hour not in RELEASE_HOURS:
            raise ValueError(f"未知发布时间: {release_value}")
        issue = date + pd.Timedelta(hours=release_hour)
        forecasts[issue] = values[index].copy()
    if len(forecasts) ≠ 1460:
        raise ValueError(f"附件 3 发布时间数量应为 1460, 实际为 {len(forecasts)}")
    return forecasts

def read_reference_headers_only(path: Path) → list[object]:
    """只读取参考文件第一张工作表第一行, 禁止访问第二行及以后。"""

```

```

workbook = load_workbook(path, read_only=True, data_only=False)
worksheet = workbook.worksheets[0]
headers = list(next(worksheet.iter_rows(min_row=1, max_row=1, values_only=True)))
workbook.close()
if len(headers) != 146 or headers[0] != "日期\\区间":
    raise ValueError("参考表第一行应为日期列加 145 个区间表头")
if headers[-1] != "次日00:00-次日00:10":
    raise ValueError("参考表最后一列表头不是次日00:00-次日00:10")
return headers

def load_inputs(actual_path: Path, forecast_path: Path, reference_path: Path) → Inputs:
    return Inputs(
        actual=read_actual(actual_path),
        forecasts=read_forecasts(forecast_path),
        reference_headers=read_reference_headers_only(reference_path),
    )

def actual_value(actual: dict[pd.Timestamp, float], timestamp: pd.Timestamp) → float:
    value = actual.get(timestamp)
    if value is None:
        raise KeyError(f"缺少真实光伏端点: {timestamp}")
    return float(value)

def linear_endpoints(
    issue: pd.Timestamp,
    forecasts: dict[pd.Timestamp, np.ndarray],
    actual: dict[pd.Timestamp, float],
) → np.ndarray:
    """方案 A: 生成 issue 至当天 24:00 的十分钟瞬时端点。"""
    release_hour = issue.hour
    horizon_hours = 24 - release_hour
    hourly_knots = np.concatenate(
        ([actual_value(actual, issue)], forecasts[issue][:horizon_hours])
    )
    endpoint_count = horizon_hours * 6
    endpoints = np.empty(endpoint_count + 1, dtype=float)
    endpoints[0] = hourly_knots[0]
    for index in range(1, endpoint_count + 1):
        right_hour = math.ceil(index / 6)
        offset = index - 6 * (right_hour - 1)
        ratio = offset / 6.0
        endpoints[index] = (
            (1.0 - ratio) * hourly_knots[right_hour - 1]
            + ratio * hourly_knots[right_hour]
        )
    return np.maximum(endpoints, 0.0)

def historical_shape_endpoints(
    issue: pd.Timestamp,
    base_endpoints: np.ndarray,
    actual: dict[pd.Timestamp, float],
) → np.ndarray:
    """方案 B: 用此前完整日的小时内残差修正 A, 整点不变。"""
    corrected = base_endpoints.copy()
    history_dates = [
        issue.normalize() - pd.Timedelta(days=lag)
        for lag in range(1, SHAPE_DAYS + 1)
    ]
    for index in range(1, len(corrected)):
        if index % 6 == 0:
            continue
        target = issue + pd.Timedelta(minutes=10 * index)

```

```

hour_start = target.floor("h")
fraction = target.minute / 60.0
residuals = []
for history_date in history_dates:
    historical_start = history_date + pd.Timedelta(hours=hour_start.hour)
    historical_target = historical_start + pd.Timedelta(minutes=target.minute)
    historical_end = historical_start + pd.Timedelta(hours=1)
    if (
        historical_start not in actual
        or historical_target not in actual
        or historical_end not in actual
    ):
        continue
    straight = (
        (1.0 - fraction) * actual[historical_start]
        + fraction * actual[historical_end]
    )
    residuals.append(actual[historical_target] - straight)
if residuals:
    corrected[index] += float(np.median(residuals))
return np.maximum(corrected, 0.0)

def previous_forecast_endpoint(
    previous_issue: pd.Timestamp,
    target: pd.Timestamp,
    forecasts: dict[pd.Timestamp, np.ndarray],
    actual: dict[pd.Timestamp, float],
) → float:
    minutes = int((target - previous_issue).total_seconds() // 60)
    if minutes < 0 or minutes > 24 * 60 or minutes % 10 != 0:
        raise ValueError("旧预报目标时刻超出可插值范围")
    if minutes == 0:
        return actual_value(actual, previous_issue)
    right_hour = math.ceil(minutes / 60)
    offset = minutes - 60 * (right_hour - 1)
    ratio = offset / 60.0
    left = (
        actual_value(actual, previous_issue)
        if right_hour == 1
        else float(forecasts[previous_issue][right_hour - 2])
    )
    right = float(forecasts[previous_issue][right_hour - 1])
    return (1.0 - ratio) * left + ratio * right

def recent_bias(
    issue: pd.Timestamp,
    forecasts: dict[pd.Timestamp, np.ndarray],
    actual: dict[pd.Timestamp, float],
) → float:
    """根据发布前最近 3 小时旧预报的日间端点残差估计当前偏差。"""
    old_issue = issue - pd.Timedelta(hours=6)
    residuals = []
    for step in range(RECENT_HOURS * 6 + 1):
        target = issue - pd.Timedelta(minutes=10 * (RECENT_HOURS * 6 - step))
        truth = actual_value(actual, target)
        prediction = previous_forecast_endpoint(old_issue, target, forecasts, actual)
        if truth > 1.0 or prediction > 1.0:
            residuals.append(truth - prediction)
    return float(np.median(residuals)) if len(residuals) ≥ 6 else 0.0

def assimilated_endpoints(
    issue: pd.Timestamp,
    base_endpoints: np.ndarray,

```

```

    forecasts: dict[pd.Timestamp, np.ndarray],
    actual: dict[pd.Timestamp, float],
) → np.ndarray:
    """方案 C: 将近期残差以指数衰减形式作用于未来端点。"""
    bias = recent_bias(issue, forecasts, actual)
    corrected = base_endpoints.copy()
    for index in range(1, len(corrected)):
        # 附件 3 已明确给出夜间零功率; 不能把白天残差外推成虚假夜间发电。
        if base_endpoints[index] ≤ 1.0:
            continue
        lead_hours = index / 6.0
        corrected[index] += bias * math.exp(-lead_hours / BIAS_DECAY_HOURS)
    return np.maximum(corrected, 0.0)

def endpoints_to_intervals(endpoints: np.ndarray) → np.ndarray:
    intervals = (endpoints[:-1] + endpoints[1:]) / 2.0
    if not np.isfinite(intervals).all() or np.any(intervals < 0):
        raise ValueError("端点转换后的区间平均功率异常")
    return intervals

def candidate_intervals(issue: pd.Timestamp, inputs: Inputs) → dict[str, np.ndarray]:
    base = linear_endpoints(issue, inputs.forecasts, inputs.actual)
    shaped = historical_shape_endpoints(issue, base, inputs.actual)
    assimilated = assimilated_endpoints(issue, base, inputs.forecasts, inputs.actual)
    return {
        "A_线性插值": endpoints_to_intervals(base),
        "B_历史形状": endpoints_to_intervals(shaped),
        "C_残差同化": endpoints_to_intervals(assimilated),
    }

def true_intervals(issue: pd.Timestamp, actual: dict[pd.Timestamp, float]) → np.ndarray:
    endpoint_count = (24 - issue.hour) * 6
    endpoints = np.array(
        [actual_value(actual, issue + pd.Timedelta(minutes=10 * index)) for index in
range(endpoint_count + 1)],
        dtype=float,
    )
    return endpoints_to_intervals(endpoints)

def extra_next_day_interval(
    date: pd.Timestamp,
    left_endpoint: float,
    actual: dict[pd.Timestamp, float],
) → float:
    """用发布日前 7 个完整日的 00:10 真实端点均值补次日 00:10。"""
    history = [
        actual_value(actual, date - pd.Timedelta(days=lag) + pd.Timedelta(minutes=10))
        for lag in range(1, EXTRA_ENDPOINT_HISTORY_DAYS + 1)
    ]
    next_day_0010 = float(np.mean(history))
    return max(0.0, (left_endpoint + next_day_0010) / 2.0)

def rolling_build(inputs: Inputs) → tuple[dict[int, list[np.ndarray]], pd.DataFrame, pd.DataFrame]:
    """无泄漏地滚动选择 A/B/C, 并返回四个发布时间的正式输出。"""
    error_history: dict[int, dict[str, deque[float]]] = {
        release: {method: deque(maxlen=SELECTION_DAYS) for method in METHODS}
        for release in RELEASE_HOURS
    }
    outputs: dict[int, list[np.ndarray]] = defaultdict(list)
    metric_records: list[dict[str, object]] = []
    choice_records: list[dict[str, object]] = []

```

```

for date in pd.date_range(CALCULATION_START, OUTPUT_END, freq="D"):
    for release_hour in RELEASE_HOURS:
        issue = date + pd.Timedelta(hours=release_hour)
        candidates = candidate_intervals(issue, inputs)
        truth = true_intervals(issue, inputs.actual)
        history = error_history[release_hour]
        history_size = len(history[METHODS[0]])
        history_means_before = {
            method: float(np.mean(history[method])) if history_size else np.nan
            for method in METHODS
        }
        if history_size < MIN_SELECTION_DAYS:
            selected = METHODS[0]
        else:
            selected = min(METHODS, key=lambda name: history_means_before[name])

        daily_errors = {
            method: float(np.mean(np.abs(values - truth)))
            for method, values in candidates.items()
        }
        choice_records.append(
            {
                "日期": date,
                "发布时间": release_hour,
                "选择方法": selected,
                "选择依据历史天数": history_size,
                "选择前历史MAE_A_kw": history_means_before[METHODS[0]],
                "选择前历史MAE_B_kw": history_means_before[METHODS[1]],
                "选择前历史MAE_C_kw": history_means_before[METHODS[2]],
            }
        )
        for method, error in daily_errors.items():
            history[method].append(error)
            metric_records.append(
                {
                    "日期": date,
                    "发布时间": release_hour,
                    "方法": method,
                    "当日有效区间MAE_kw": error,
                }
            )

    if OUTPUT_START ≤ date ≤ OUTPUT_END:
        chosen = candidates[selected]
        final_hour_endpoint = float(inputs.forecasts[issue][24 - release_hour - 1])
        extra = extra_next_day_interval(date, final_hour_endpoint, inputs.actual)
        output_values = np.concatenate((chosen, [extra]))
        expected = 145 - 6 * release_hour
        if len(output_values) ≠ expected:
            raise AssertionError(
                f"{date.date()} {release_hour}:00 应输出 {expected} 个区间, 实际 {len(output_values)}"
            )
        outputs[release_hour].append(output_values)

    return outputs, pd.DataFrame(metric_records), pd.DataFrame(choice_records)

def write_forecast_workbook(
    output_path: Path,
    headers: list[object],
    outputs: dict[int, list[np.ndarray]],
) → None:
    workbook = Workbook()
    workbook.remove(workbook.active)

```



```

dates = list(pd.date_range(OUTPUT_START, OUTPUT_END, freq="D"))
fill = PatternFill("solid", fgColor="D9EAF7")

for release_hour in RELEASE_HOURS:
    worksheet = workbook.create_sheet(f"{release_hour:02d}时预报")
    start_interval = release_hour * 6
    sheet_headers = [headers[0], *headers[1 + start_interval :]]
    worksheet.append(sheet_headers)
    for cell in worksheet[1]:
        cell.font = Font(bold=True)
        cell.fill = fill
        cell.alignment = Alignment(horizontal="center", vertical="center")
    if len(outputs[release_hour]) != len(dates):
        raise AssertionError(f"{release_hour}:00 输出日期数异常")
    for date, values in zip(dates, outputs[release_hour], strict=True):
        worksheet.append([date.to_pydatetime(), *[float(value) for value in values]])
    worksheet.freeze_panes = "B2"
    worksheet.column_dimensions["A"].width = 13
    for cell in worksheet["A"][1:]:
        cell.number_format = "yyyy-mm-dd"

output_path.parent.mkdir(parents=True, exist_ok=True)
workbook.save(output_path)

def write_supporting_results(
    results_dir: Path,
    metrics: pd.DataFrame,
    choices: pd.DataFrame,
    inputs: Inputs,
    actual_path: Path,
    forecast_path: Path,
    reference_path: Path,
    output_path: Path,
) → None:
    results_dir.mkdir(parents=True, exist_ok=True)
    formal_metrics = metrics[metrics["日期"] ≥ OUTPUT_START].copy()
    summary = (
        formal_metrics.groupby(["发布时间", "方法", sort=True])["当日有效区间MAE_kW"]
        .agg(["count", "mean", "median"])
        .reset_index()
        .rename(columns={"count": "天数", "mean": "日MAE均值_kW", "median": "日MAE中位数_kW"})
    )
    formal_choices = choices[choices["日期"] ≥ OUTPUT_START].copy()
    choice_summary = (
        formal_choices.groupby(["发布时间", "选择方法", sort=True])
        .size()
        .reset_index(name="选择天数")
    )
    summary.to_csv(results_dir / "ABC方案误差汇总.csv", index=False, encoding="utf-8-sig")
    formal_choices.to_csv(results_dir / "每日滚动方法选择.csv", index=False, encoding="utf-8-sig")
    choice_summary.to_csv(results_dir / "方法选择次数.csv", index=False, encoding="utf-8-sig")

manifest = {
    "purpose": "问题三光伏小时预报还原为十分钟区间预测",
    "output_period": [str(OUTPUT_START.date()), str(OUTPUT_END.date())],
    "methods": list(METHODS),
    "selection": f"同发布时间此前{SELECTION_DAYS}天滚动日MAE最小；不足{MIN_SELECTION_DAYS}天用A",
    "historical_shape_days": SHAPE_DAYS,
    "recent_residual_hours": RECENT_HOURS,
    "bias_decay_hours": BIAS_DECAY_HOURS,
    "extra_0010_rule": "发布日前7个完整日的00:10真实端点平均，再与24:00预测端点取平均",
    "reference_access": "只读取演示数据用于参考格式.xlsx第一张表第一行表头",
    "inputs": {
        str(actual_path.relative_to(PROJECT_ROOT)): sha256(actual_path),
        str(forecast_path.relative_to(PROJECT_ROOT)): sha256(forecast_path),
    }
}

```

```

    },
    "reference_header_source": str(reference_path.relative_to(PROJECT_ROOT)),
    "program": {
        str(Path(__file__).resolve().relative_to(PROJECT_ROOT)): sha256(Path(__file__).resolve())
    },
    "output": {
        str(output_path.relative_to(PROJECT_ROOT)): sha256(output_path),
        "sheet_shapes": {
            f"{release:02d}时预报": [335, 146 - 6 * release]
            for release in RELEASE_HOURS
        },
    },
    "runtime": {
        "python": platform.python_version(),
        "numpy": np.__version__,
        "pandas": pd.__version__,
    },
    "reproduce_from_project_root": "python 附件/question3/build_pv_interval_forecasts.py",
}
(results_dir / "复现清单.json").write_text(
    json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8"
)

def validate_output(path: Path, headers: list[object]) → None:
    workbook = load_workbook(path, read_only=True, data_only=True)
    expected_sheets = [f"{release:02d}时预报" for release in RELEASE_HOURS]
    if workbook.sheetnames != expected_sheets:
        raise AssertionError(f"工作表名称异常: {workbook.sheetnames}")
    for release_hour, sheet_name in zip(RELEASE_HOURS, expected_sheets, strict=True):
        worksheet = workbook[sheet_name]
        expected_columns = 146 - 6 * release_hour
        if (worksheet.max_row, worksheet.max_column) != (335, expected_columns):
            raise AssertionError(
                f"{sheet_name} 应为 335×{expected_columns}, 实际为 "
                f"{worksheet.max_row}×{worksheet.max_column}"
            )
        actual_headers = [cell.value for cell in worksheet[1]]
        expected_headers = [headers[0], *headers[1 + 6 * release_hour :]]
        if actual_headers != expected_headers:
            raise AssertionError(f"{sheet_name} 表头未按参考文件截取")
        rows = list(worksheet.iter_rows(min_row=2, values_only=True))
        dates = [pd.Timestamp(row[0]).normalize() for row in rows]
        if dates[0] != OUTPUT_START or dates[-1] != OUTPUT_END or len(set(dates)) != 334:
            raise AssertionError(f"{sheet_name} 日期范围或唯一性异常")
        values = np.asarray([row[1:] for row in rows], dtype=float)
        if not np.isfinite(values).all() or np.any(values < 0):
            raise AssertionError(f"{sheet_name} 存在缺失、无穷或负功率")
    workbook.close()

def parse_args() → argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--actual", type=Path, default=DEFAULT_ACTUAL)
    parser.add_argument("--forecast", type=Path, default=DEFAULT_FORECAST)
    parser.add_argument("--reference", type=Path, default=DEFAULT_REFERENCE)
    parser.add_argument("--output", type=Path, default=DEFAULT_OUTPUT)
    parser.add_argument("--results-dir", type=Path, default=DEFAULT_RESULTS_DIR)
    return parser.parse_args()

def main() → None:
    args = parse_args()
    actual_path = args.actual.resolve()
    forecast_path = args.forecast.resolve()
    reference_path = args.reference.resolve()

```

```
output_path = args.output.resolve()
results_dir = args.results_dir.resolve()
inputs = load_inputs(actual_path, forecast_path, reference_path)
outputs, metrics, choices = rolling_build(inputs)
write_forecast_workbook(output_path, inputs.reference_headers, outputs)
validate_output(output_path, inputs.reference_headers)
write_supporting_results(
    results_dir,
    metrics,
    choices,
    inputs,
    actual_path,
    forecast_path,
    reference_path,
    output_path,
)
print(f"已生成: {output_path}")
print("工作表: 00时预报、06时预报、12时预报、18时预报")
print("数据行: 每张表 334 天; 仅含日期和光伏区间预测")

if __name__ == "__main__":
    main()
```

62. question3/prediction/compare_forecast_versions.py

源文件 SHA-256: b7dbd945d5024479cef7484781eb201dbf5fd4dfd60a4744e22013932bd120e6 ; 代码行数: 486。

"""比较“上一发布时刻延伸预报”和“当前最新预报”的准确性。

比较遵循严格的配对原则：

1. 两版预报必须针对同一个目标时刻；
2. 只评价当前发布时刻至当天 24:00、且旧预报仍能覆盖的区间；
3. 小时端点评价使用附件 2 的同一整点真实功率；
4. 10 分钟区间评价时，两种方案都用当前发布时刻的真实功率作共同边界，再分别对旧、新小时节点作线性插值和梯形平均。

从项目根目录运行：

```
python 附件/question3/compare_forecast_versions.py
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import hashlib
```

```
import json
```

```
import math
```

```
import platform
```

```
from dataclasses import dataclass
```

```
from datetime import time
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import pandas as pd
```

```
from openpyxl import load_workbook
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[2]
```

```
DEFAULT_ACTUAL = PROJECT_ROOT / "附件" / "附件2.xlsx"
```

```
DEFAULT_FORECAST = PROJECT_ROOT / "附件" / "附件3.xlsx"
```

```
DEFAULT_OUTPUT_DIR = Path(__file__).resolve().parent / "results"
```

```
START_DATE = pd.Timestamp("2025-02-01")
```

```
END_DATE = pd.Timestamp("2025-12-31")
```

```
RELEASE_HOURS = (0, 6, 12, 18)
```

```
@dataclass(frozen=True)
```

```
class InputData:
```

```
    actual: dict[pd.Timestamp, float]
```

```
    forecasts: dict[pd.Timestamp, np.ndarray]
```

```
def file_sha256(path: Path) → str:
```

```
    digest = hashlib.sha256()
```

```
    with path.open("rb") as stream:
```

```
        for block in iter(lambda: stream.read(1024 * 1024), b""):
```

```
            digest.update(block)
```

```
    return digest.hexdigest()
```

```
def validate_workbook_shape(path: Path, sheet_index: int, rows: int, columns: int) → None:
```

```
    workbook = load_workbook(path, read_only=True, data_only=True)
```

```
    worksheet = workbook.worksheets[sheet_index]
```

```
    actual_shape = (worksheet.max_row, worksheet.max_column)
```

```
    workbook.close()
```

```
    if actual_shape != (rows, columns):
```

```
        raise ValueError(
```

```
            f"{path.name} 第 {sheet_index + 1} 张表尺寸应为 {(rows, columns)}, "
```

```
            f"实际为 {actual_shape}"
```

```
)
```

```

def parse_header_minute(value: object) → int:
    if isinstance(value, time):
        return value.hour * 60 + value.minute
    text = str(value).strip()
    if text in {"0:00+1", "00:00+1", "24:00"}:
        return 1440
    raise ValueError(f"无法识别附件 2 时间表头: {value!r}")

def read_actual_pv(path: Path) → dict[pd.Timestamp, float]:
    """读取附件 2 第二张表, 返回所有 10 分钟真实端点。"""
    validate_workbook_shape(path, sheet_index=1, rows=366, columns=145)
    frame = pd.read_excel(path, sheet_name=1, header=None)
    minutes = [parse_header_minute(value) for value in frame.iloc[0, 1:]]
    if minutes != list(range(10, 1441, 10)):
        raise ValueError("附件 2 光伏表头不是 00:10 至次日 00:00 的连续 10 分钟端点")

    actual: dict[pd.Timestamp, float] = {}
    for row_index in range(1, len(frame)):
        date = pd.Timestamp(frame.iloc[row_index, 0]).normalize()
        values = pd.to_numeric(frame.iloc[row_index, 1:], errors="raise").to_numpy(float)
        if values.shape != (144,) or not np.isfinite(values).all() or np.any(values < 0):
            raise ValueError(f"附件 2 第 {row_index + 1} 行光伏数据异常")
        for minute, value in zip(minutes, values, strict=True):
            actual[date + pd.Timedelta(minutes=minute)] = float(value)

    expected_first = pd.Timestamp("2025-01-01 00:10")
    expected_last = pd.Timestamp("2026-01-01 00:00")
    if expected_first not in actual or expected_last not in actual:
        raise ValueError("附件 2 实际数据首末时间不符合预期")
    return actual

def read_forecasts(path: Path) → dict[pd.Timestamp, np.ndarray]:
    """读取附件 3, 键为完整发布时间, 值为未来 1-24 小时端点预测。"""
    validate_workbook_shape(path, sheet_index=0, rows=1461, columns=26)
    frame = pd.read_excel(path, sheet_name=0, header=None)
    dates = frame.iloc[1:, 0].replace("", np.nan).ffill()
    release_text = frame.iloc[1:, 1].astype(str).strip()
    values = frame.iloc[1:, 2:].apply(pd.to_numeric, errors="raise").to_numpy(float)
    if values.shape != (1460, 24) or not np.isfinite(values).all() or np.any(values < 0):
        raise ValueError("附件 3 预测矩阵尺寸或数值异常")

    forecasts: dict[pd.Timestamp, np.ndarray] = {}
    for index, (date_value, release_value) in enumerate(zip(dates, release_text, strict=True)):
        date = pd.Timestamp(str(date_value)).normalize()
        hour = int(release_value.split(":", maxsplit=1)[0])
        if hour not in RELEASE_HOURS:
            raise ValueError(f"附件 3 出现非预期发布时间: {release_value}")
        issue = date + pd.Timedelta(hours=hour)
        if issue in forecasts:
            raise ValueError(f"附件 3 发布时间重复: {issue}")
        forecasts[issue] = values[index].copy()

    if len(forecasts) != 365 * 4:
        raise ValueError(f"附件 3 应有 1460 个发布时间, 实际为 {len(forecasts)}")
    return forecasts

def load_inputs(actual_path: Path, forecast_path: Path) → InputData:
    return InputData(
        actual=read_actual_pv(actual_path),
        forecasts=read_forecasts(forecast_path),
    )

```

```

def overlap_hours(release_hour: int) → int:
    """当前日剩余范围与上一版 24 小时预报覆盖范围的交集小时数。"""
    return min(24 - release_hour, 18)

def previous_issue(issue: pd.Timestamp) → pd.Timestamp:
    return issue - pd.Timedelta(hours=6)

def collect_hourly_pairs(data: InputData) → pd.DataFrame:
    records: list[dict[str, object]] = []
    for date in pd.date_range(START_DATE, END_DATE, freq="D"):
        for release_hour in RELEASE_HOURS:
            issue = date + pd.Timedelta(hours=release_hour)
            old_issue = previous_issue(issue)
            if issue not in data.forecasts or old_issue not in data.forecasts:
                raise KeyError(f"缺少发布时间 {issue} 或 {old_issue} 的预测")
            new_values = data.forecasts[issue]
            old_values = data.forecasts[old_issue]
            for horizon in range(1, overlap_hours(release_hour) + 1):
                target = issue + pd.Timedelta(hours=horizon)
                old_lead = horizon + 6
                truth = data.actual.get(target)
                if truth is None:
                    raise KeyError(f"缺少目标时刻 {target} 的实际光伏功率")
                old_prediction = float(old_values[old_lead - 1])
                new_prediction = float(new_values[horizon - 1])
                records.append(
                    {
                        "日期": date,
                        "当前发布时间": issue,
                        "上一发布时间": old_issue,
                        "发布小时": release_hour,
                        "当前预测提前小时": horizon,
                        "旧预测提前小时": old_lead,
                        "目标时刻": target,
                        "真实功率_kW": truth,
                        "旧版延伸预测_kW": old_prediction,
                        "新版最新预测_kW": new_prediction,
                        "旧版绝对误差_kW": abs(old_prediction - truth),
                        "新版绝对误差_kW": abs(new_prediction - truth),
                    }
                )
    result = pd.DataFrame.from_records(records)
    result["绝对误差改善_kW"] = result["旧版绝对误差_kW"] - result["新版绝对误差_kW"]
    return result

def interpolate_endpoint(knots: np.ndarray, ten_minute_index: int) → float:
    """从逐小时节点线性插值得到发布后的第 ten_minute_index 个端点。"""
    hour_right = math.ceil(ten_minute_index / 6)
    offset = ten_minute_index - 6 * (hour_right - 1)
    ratio = offset / 6.0
    return float((1.0 - ratio) * knots[hour_right - 1] + ratio * knots[hour_right])

def collect_interval_pairs(data: InputData) → pd.DataFrame:
    """比较共同实测边界下，两版预测重构得到的 10 分钟区间平均功率。"""
    records: list[dict[str, object]] = []
    for date in pd.date_range(START_DATE, END_DATE, freq="D"):
        for release_hour in RELEASE_HOURS:
            issue = date + pd.Timedelta(hours=release_hour)
            old_issue = previous_issue(issue)
            hours = overlap_hours(release_hour)
            boundary = data.actual.get(issue)
            if boundary is None:

```

```

        raise KeyError(f"缺少发布边界 {issue} 的实际光伏功率")

    new_hourly = data.forecasts[issue][:hours]
    old_hourly = data.forecasts[old_issue][6 : 6 + hours]
    new_knots = np.concatenate([[boundary], new_hourly])
    old_knots = np.concatenate([[boundary], old_hourly])

    previous_new_endpoint = float(boundary)
    previous_old_endpoint = float(boundary)
    previous_actual_endpoint = float(boundary)
    for ten_minute_index in range(1, hours * 6 + 1):
        target = issue + pd.Timedelta(minutes=10 * ten_minute_index)
        actual_endpoint = data.actual.get(target)
        if actual_endpoint is None:
            raise KeyError(f"缺少目标时刻 {target} 的实际光伏功率")
        new_endpoint = interpolate_endpoint(new_knots, ten_minute_index)
        old_endpoint = interpolate_endpoint(old_knots, ten_minute_index)
        actual_interval = (previous_actual_endpoint + actual_endpoint) / 2.0
        new_interval = (previous_new_endpoint + new_endpoint) / 2.0
        old_interval = (previous_old_endpoint + old_endpoint) / 2.0
        records.append(
            {
                "日期": date,
                "当前发布时间": issue,
                "上一发布时间": old_issue,
                "发布小时": release_hour,
                "发布后区间序号": ten_minute_index,
                "当前预测提前小时": math.ceil(ten_minute_index / 6),
                "区间开始": target - pd.Timedelta(minutes=10),
                "区间结束": target,
                "真实区间平均功率_kW": actual_interval,
                "旧版延伸区间预测_kW": old_interval,
                "新版最新区间预测_kW": new_interval,
                "旧版绝对误差_kW": abs(old_interval - actual_interval),
                "新版绝对误差_kW": abs(new_interval - actual_interval),
            }
        )
        previous_new_endpoint = new_endpoint
        previous_old_endpoint = old_endpoint
        previous_actual_endpoint = float(actual_endpoint)

    result = pd.DataFrame.from_records(records)
    result["绝对误差改善_kW"] = result["旧版绝对误差_kW"] - result["新版绝对误差_kW"]
    return result

def metrics(group: pd.DataFrame, truth_column: str, old_column: str, new_column: str) → pd.Series:
    truth = group[truth_column].to_numpy(float)
    old = group[old_column].to_numpy(float)
    new = group[new_column].to_numpy(float)
    old_error = old - truth
    new_error = new - truth
    old_mae = float(np.mean(np.abs(old_error)))
    new_mae = float(np.mean(np.abs(new_error)))
    denominator = float(np.sum(np.abs(truth)))
    old_error_sum = float(np.sum(np.abs(old_error)))
    new_error_sum = float(np.sum(np.abs(new_error)))
    if old_mae == 0.0:
        if new_mae != 0.0:
            raise ValueError("旧版 MAE 为 0 而新版非 0, MAE 相对改善率无有限定义")
        mae_improvement = 0.0
    else:
        mae_improvement = 100.0 * (old_mae - new_mae) / old_mae
    if denominator == 0.0:
        if old_error_sum != 0.0 or new_error_sum != 0.0:
            raise ValueError("真实功率全为 0 但预测误差非 0, WAPE 无有限定义")

```



```

        old_wape = 0.0
        new_wape = 0.0
    else:
        old_wape = 100.0 * old_error_sum / denominator
        new_wape = 100.0 * new_error_sum / denominator
    return pd.Series(
        {
            "样本数": len(group),
            "旧版MAE_kw": old_mae,
            "新版MAE_kw": new_mae,
            "MAE改善率_percent": mae_improvement,
            "旧版RMSE_kw": float(np.sqrt(np.mean(old_error**2))),
            "新版RMSE_kw": float(np.sqrt(np.mean(new_error**2))),
            "旧版WAPE_percent": old_wape,
            "新版WAPE_percent": new_wape,
            "旧版Bias_kw": float(np.mean(old_error)),
            "新版Bias_kw": float(np.mean(new_error)),
            "新版绝对误差更小比例_percent": 100.0 * float(np.mean(np.abs(new_error) < np.abs(old_error))),
            "两版绝对误差相同比例_percent": 100.0 * float(np.mean(np.isclose(np.abs(new_error),
np.abs(old_error)))))
        }
    )

def summarize(
    frame: pd.DataFrame,
    truth_column: str,
    old_column: str,
    new_column: str,
) → tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    overall = metrics(frame, truth_column, old_column, new_column).to_frame().T
    by_release = (
        frame.groupby("发布小时", sort=True)
        .apply(metrics, truth_column, old_column, new_column, include_groups=False)
        .reset_index()
    )
    by_horizon = (
        frame.groupby(["发布小时", "当前预测提前小时"], sort=True)
        .apply(metrics, truth_column, old_column, new_column, include_groups=False)
        .reset_index()
    )
    return overall, by_release, by_horizon

def monthly_summary(
    frame: pd.DataFrame,
    truth_column: str,
    old_column: str,
    new_column: str,
) → pd.DataFrame:
    local = frame.copy()
    local["月份"] = pd.to_datetime(local["日期"]).dt.month
    return (
        local.groupby(["月份", "发布小时"], sort=True)
        .apply(metrics, truth_column, old_column, new_column, include_groups=False)
        .reset_index()
    )

def conclusion_table(interval_by_release: pd.DataFrame) → pd.DataFrame:
    rows = []
    for _, row in interval_by_release.iterrows():
        improvement = float(row["MAE改善率_percent"])
        absolute_difference = float(row["旧版MAE_kw"] - row["新版MAE_kw"])
        if improvement ≥ 0.5:
            comparison = "新版明显更优"

```

```

        recommendation = "只用当前最新预报"
    elif abs(improvement) < 0.5 or abs(absolute_difference) < 1.0:
        comparison = "两版精度近似等价"
        recommendation = "为保持规则统一和流程简洁，只用当前最新预报"
    else:
        comparison = "旧版明显更优"
        recommendation = "暂不舍弃旧版，进一步研究组合或分时选择"
    rows.append(
        {
            "发布时间": f"{int(row['发布小时']):02d}:00",
            "旧版区间MAE_kW": row["旧版MAE_kW"],
            "新版区间MAE_kW": row["新版MAE_kW"],
            "新版MAE改善率_percent": improvement,
            "精度判断": comparison,
            "建议": recommendation,
        }
    )
)
return pd.DataFrame(rows)

def write_conclusion_markdown(path: Path, decisions: pd.DataFrame) → None:
    lines = [
        "# 延伸预报与最新预报对比结论",
        "",
        "比较采用同一目标时刻配对，并且只评价当前发布时刻至当天 24:00、旧版仍可覆盖的共同区间。",
        "10 分钟区间重构时，两版均以当前发布时刻的真实光伏功率为共同边界。",
        "",
        "| 发布时间 | 旧版区间 MAE/kW | 新版区间 MAE/kW | 新版改善率 | 判断 |",
        "|:---:|:---:|:---:|:---:|",
    ]
    for _, row in decisions.iterrows():
        lines.append(
            f"| {row['发布时间']} | {row['旧版区间MAE_kW']:.4f} | "
            f"{row['新版区间MAE_kW']:.4f} | {row['新版MAE改善率_percent']:.2f}% | "
            f"{row['精度判断']} | "
        )
    lines.extend(
        [
            "",
            "## 建议",
            "",
            "当天 0:00 制定计划时，不将前一天 18:00 延伸预报混入中央预测，直接采用当天 0:00 最新预报。",
            "同理，6:00、12:00 和 18:00 调整时均采用当前最新发布版本覆盖尚未执行区间。",
            "旧版预报只保留用于预测修订分析、缺失备用和不确定性场景，不参与主方案插值。",
            "",
            "18:00 后两版区间 MAE 的差异远小于 1 kW，主要因为该时段基本进入夜间，",
            "不能把这种数值级微差解释为旧版具有实际优势。",
            "纯夜间分组若真实值、旧预测和新预测均为 0，则其 WAPE 与 MAE 改善率按约定记为 0，",
            "表示两版在该全零分组中均无误差。",
        ]
    )
)
path.write_text("\n".join(lines) + "\n", encoding="utf-8")

def write_outputs(
    output_dir: Path,
    actual_path: Path,
    forecast_path: Path,
    hourly: pd.DataFrame,
    intervals: pd.DataFrame,
) → None:
    output_dir.mkdir(parents=True, exist_ok=True)
    hourly_overall, hourly_by_release, hourly_by_horizon = summarize(
        hourly,
        "真实功率_kW",
        "旧版延伸预测_kW",
    )

```

```

        "新版最新预测_kw",
    )
    interval_overall, interval_by_release, interval_by_horizon = summarize(
        intervals,
        "真实区间平均功率_kw",
        "旧版延伸区间预测_kw",
        "新版最新区间预测_kw",
    )
    hourly_monthly = monthly_summary(
        hourly,
        "真实功率_kw",
        "旧版延伸预测_kw",
        "新版最新预测_kw",
    )
    interval_monthly = monthly_summary(
        intervals,
        "真实区间平均功率_kw",
        "旧版延伸区间预测_kw",
        "新版最新区间预测_kw",
    )
    decisions = conclusion_table(interval_by_release)

    hourly.to_csv(output_dir / "逐整点配对明细.csv", index=False, encoding="utf-8-sig")
    intervals.to_csv(output_dir / "逐10分钟区间配对明细.csv", index=False, encoding="utf-8-sig")
    interval_by_release.to_csv(output_dir / "核心对比_按发布时间.csv", index=False, encoding="utf-8-sig")
    write_conclusion_markdown(output_dir / "结论说明.md", decisions)

    workbook_path = output_dir / "延伸预报与最新预报准确率对比.xlsx"
    with pd.ExcelWriter(workbook_path, engine="openpyxl") as writer:
        decisions.to_excel(writer, sheet_name="结论", index=False)
        hourly_overall.to_excel(writer, sheet_name="整点_总体", index=False)
        hourly_by_release.to_excel(writer, sheet_name="整点_按发布时间", index=False)
        hourly_by_horizon.to_excel(writer, sheet_name="整点_按时距", index=False)
        hourly_monthly.to_excel(writer, sheet_name="整点_逐月", index=False)
        interval_overall.to_excel(writer, sheet_name="区间_总体", index=False)
        interval_by_release.to_excel(writer, sheet_name="区间_按发布时间", index=False)
        interval_by_horizon.to_excel(writer, sheet_name="区间_按时距", index=False)
        interval_monthly.to_excel(writer, sheet_name="区间_逐月", index=False)

    manifest = {
        "purpose": "同目标时刻配对比较上一发布时刻延伸预报与当前最新预报",
        "evaluation_period": [str(START_DATE.date()), str(END_DATE.date())],
        "release_hours": list(RELEASE_HOURS),
        "comparison_scope": "当前发布时刻至当天24:00, 且旧版预报仍可覆盖的共同范围",
        "interval_reconstruction": "当前实测边界锚定、小时节点线性插值、相邻端点梯形平均",
        "inputs": {
            str(actual_path.relative_to(PROJECT_ROOT)): file_sha256(actual_path),
            str(forecast_path.relative_to(PROJECT_ROOT)): file_sha256(forecast_path),
        },
        "program": {
            str(Path(__file__).resolve().relative_to(PROJECT_ROOT)):
file_sha256(Path(__file__).resolve()),
        },
        "outputs": {
            "hourly_pairs": len(hourly),
            "ten_minute_interval_pairs": len(intervals),
            "workbook": workbook_path.name,
        },
        "runtime": {
            "python": platform.python_version(),
            "numpy": np.__version__,
            "pandas": pd.__version__,
        },
        "reproduce_from_project_root": "python 附件/question3/compare_forecast_versions.py",
    }
    (output_dir / "复现清单.json").write_text(

```

```
        json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8"
    )

def parse_args() → argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--actual", type=Path, default=DEFAULT_ACTUAL)
    parser.add_argument("--forecast", type=Path, default=DEFAULT_FORECAST)
    parser.add_argument("--output-dir", type=Path, default=DEFAULT_OUTPUT_DIR)
    return parser.parse_args()

def main() → None:
    args = parse_args()
    actual_path = args.actual.resolve()
    forecast_path = args.forecast.resolve()
    output_dir = args.output_dir.resolve()
    data = load_inputs(actual_path, forecast_path)
    hourly = collect_hourly_pairs(data)
    intervals = collect_interval_pairs(data)
    write_outputs(output_dir, actual_path, forecast_path, hourly, intervals)
    print(f"逐整点配对样本数: {len(hourly)}")
    print(f"逐10分钟区间配对样本数: {len(intervals)}")
    print(f"结果目录: {output_dir}")

if __name__ == "__main__":
    main()
```

63. question4/code/audit_price_inputs.py

源文件 SHA-256: c2883b05db3048a1f1f9b199a25566cdbf0fed1d79443ce60b060a8c92b00762 ; 代码行数: 72。

```

"""Independent value, source, publication and boundary checks on price inputs."""
from pathlib import Path
import hashlib
import json
import numpy as np

from price_inputs import load_price_data, read_attachment4, ROOT

def sha(path): return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def audit_prices(data_dir=ROOT/'data'):
    data_dir=Path(data_dir); p=load_price_data(data_dir/'price_inputs.npz')
    raw=read_attachment4(data_dir/'attachment4.xlsx'); flat=raw.ravel()
    with np.load(data_dir/'xgboost_midnight_predictions.npz') as z: zero=z['predictions'].copy()
    np.testing.assert_array_equal(zero, p.nodes[:, 0])
    np.testing.assert_array_equal((zero[:, :-1]+zero[:, 1:])/2, p.predicted[:, 0])

    expected=np.empty((334, 145)); expected[:, :144]=raw[31:]; expected[:, -1, -1]=raw[32:, 0]; expected[-1, -1]=np.na
    n
    np.testing.assert_array_equal(expected, p.raw_actual_nodes)
    held=expected.copy(); held[-1, -1]=flat[-1]
    np.testing.assert_array_equal(held, p.actual_nodes)
    np.testing.assert_array_equal((held[:, :-1]+held[:, 1:])/2, p.actual)
    expected_est=np.zeros((334, 144), bool); expected_est[-1, -1]=True
    np.testing.assert_array_equal(expected_est, p.actual_estimated)
    actual0=raw[30:364, 143]
    np.testing.assert_array_equal(actual0, p.known_midnight)
    np.testing.assert_array_equal((actual0+zero[:, 0])/2, p.forecast_bridge)
    np.testing.assert_array_equal((actual0+raw[31:, 0])/2, p.actual_bridge)
    checks=0
    for v, hour, slot in zip(range(4), p.vintages, p.gate_slots):
        assert np.isnan(p.nodes[:, v, :slot]).all()
        assert np.isnan(p.predicted[:, v, :slot]).all()
        assert np.isfinite(p.predicted[:, v, slot]).all()
        if v: np.testing.assert_array_equal(p.nodes[:, v, slot], raw[31:, slot])
        np.testing.assert_array_equal(p.predicted[:, v, slot:],
        (p.nodes[:, v, slot:-1]+p.nodes[:, v, slot+1:])/2)
        checks+=334*(144-slot)
    fits=json.loads((data_dir/'intraday_price_fit_diagnostics.json').read_text())
    fitcount=0
    cfg=json.loads((data_dir/'xgboost_midnight_summary.json').read_text())['config']
    for hour in (6, 12, 18):
        ds=fits[str(hour)]; assert len(ds)==334
        for day, d in zip(range(31, 365), ds):
            cutoff=day*144+hour*6
            assert d['day_index']==day and d['issue_hour']==hour
            assert d['same_issue_hour']==hour and d['config']==cfg
            assert d['max_label_index']<cutoff
            assert d['last_training_feature_source_index']==(day-1)*144+hour*6-1
            assert d['current_feature_source_last_index']==cutoff-1
            assert not d['clipped'] and not d['hyperparameters_retuned']
            fitcount+=1
        assert fitcount==1002
    for name, digest in p.audit['provenance']['sources'].items(): assert sha(data_dir/name)==digest
    for name, digest in p.audit['provenance']['core'].items(): assert
    sha(Path(__file__).parent/name)==digest
    all_price_nodes=p.nodes[np.isfinite(p.nodes)]
    assert np.all(all_price_nodes>0) and np.all(p.actual>0)
    result={'passed': True, 'price_input_sha256': sha(data_dir/'price_inputs.npz'),

    'midnight_inherited_nodes_verified': int(zero.size), 'available_versioned_intervals_verified': checks,
    'actual_intervals_verified': int(p.actual.size), 'intraday_fit_cutoffs_verified': fitcount,

```

```
'known_release_node_verified':True, 'unavailable_past_prices_remain_nan':True,
'estimated_final_interval_count':1, 'startup_actual':p.startup_actual,
'minimum_predicted_interval_price':float(np.nanmin(p.predicted)),
'minimum_actual_interval_price':float(p.actual.min()),
'minimum_known_or_forecast_node':float(all_price_nodes.min()),
'max_zero_node_inheritance_error':float(np.max(np.abs(zero-p.nodes[:,0]))),
'all_source_and_numerical_core_hashes_match':True,
'note':'This audit verifies generated values and stated fit cutoffs. The executable future-
perturbation tests independently validate code causality.}'

(data_dir/'price_inputs_independent_validation.json').write_text(json.dumps(result,ensure_ascii=False,in
dent=2))
    return result

if __name__=='__main__':print(json.dumps(audit_prices(),ensure_ascii=False,indent=2))
```

64. question4/code/build_document.py

源文件 SHA-256: 55646a6151db32c7b5579e538becbed5edf9e5e56b1add4dcf5228bad3ae0918 ; 代码行数: 175。

text+='本说明是第四问的专项计算报告，配套完整源代码、两份原结果模板和四个指定自然日表。它不是整份竞赛论文的替代稿。'\n\n'


```

template=build/'A4参考样式.docx';d=Document()
for section in d.sections:
    section.page_width=Cm(21);section.page_height=Cm(29.7)
    section.top_margin=section.bottom_margin=Cm(2)
    section.left_margin=section.right_margin=Cm(2.2)
    section.header_distance=section.footer_distance=Cm(.8)
    section.header.paragraphs[0].text='第四问 | 波动电价下的购电与储能调度'
    set_font(d.styles['Header'],'Heiti SC',8)
    f=section.footer.paragraphs[0];f.alignment=1
    field=OxmlElement('w:fldSimple');field.set(qn('w:instr'),'PAGE');f._p.append(field)
for name in ('Normal','Body Text','First Paragraph','Compact'):
    if name not in d.styles:d.styles.add_style(name,WD_STYLE_TYPE.PARAGRAPH)
    set_font(d.styles[name],'Songti SC',10.5)
    d.styles[name].paragraph_format.line_spacing=1.3
    d.styles[name].paragraph_format.space_after=Pt(5)
if 'Table' not in d.styles:d.styles.add_style('Table',WD_STYLE_TYPE.TABLE)
set_font(d.styles['Table'],'Songti SC',8.5)
set_font(d.styles['Title'],'Heiti SC',22)
for n,size in ((1,16),(2,13),(3,11)):
    set_font(d.styles[f'Heading {n}'],'Heiti SC',size)
    d.styles[f'Heading {n}'].paragraph_format.keep_with_next=True
d.save(template)
intermediate=build/'converted.docx';docx=result/'reports/第四问_模型与结果说明.docx'
command=[executable('pandoc',pandoc),str(source),'-f','markdown+tex_math_dollars+raw_attribute',
'--reference-doc',str(template),'-o',str(intermediate)]
conversion=subprocess.run(command,capture_output=True,text=True,check=True)
if conversion.stderr.strip():raise RuntimeError('Review Pandoc warnings before release:
'+conversion.stderr)
d=Document(intermediate)
# Pandoc's default zoom can omit the required OOXML percentage attribute.
zoom=d.settings.element.find(qn('w:zoom'))
if zoom is not None and zoom.get(qn('w:percent')) is None:
    zoom.set(qn('w:percent'),'100')
from docx.enum.table import WD_TABLE_ALIGNMENT
for table in d.tables:
    # Explicit cell widths prevent Pandoc tables collapsing in LibreOffice.
    table.autofit=False;table.alignment=WD_TABLE_ALIGNMENT.CENTER
    count=len(table.columns)
    header=[cell.text for cell in table.rows[0].cells]
    weights=[1.0]*count
    if count==3 and header[0].startswith('对比'):weights=[2.,1.,1.3]
    elif count==3 and header[0].startswith('对照'):weights=[2.4,1.,1.]
    elif count==5 and header[1]=='变体':weights=[.5,1.7,1.1,1.4,1.1]
    elif count==5 and header[0]=='策略':weights=[1.5,1.3,1.,1.3,.9]
    elif count==5 and header[0]=='分支':weights=[1.1,1.2,1.4,.9,1.]
    total=9411
    widths=[round(total*w/sum(weights)) for w in weights]
    widths[-1]=total-sum(widths[:-1])
    prop=table._tbl.tblPr
    width=prop.find(qn('w:tblW'));width.set(qn('w:type'),'dxa');width.set(qn('w:w'),str(total))
    borders=OxmlElement('w:tblBorders')
    for side in ('top','left','bottom','right','insideH','insideV'):
        edge=OxmlElement('w:'+side);edge.set(qn('w:val'),'single' if side in ('top','bottom') else
'nil')
        edge.set(qn('w:sz'),'8');edge.set(qn('w:color'),'415E75');borders.append(edge)
    prop.insert_element_before(borders,'w:shd','w:tblLayout','w:tblCellMar','w:tblLook')
    for column,w in zip(table.columns,widths):column.width=Cm(w/567.)
    for i,row in enumerate(table.rows):
        prop=row._tr.get_or_add_trPr();prop.append(OxmlElement('w:cantSplit'))
        if i==0 and prop.find(qn('w:tblHeader')) is None:prop.append(OxmlElement('w:tblHeader'))
        for cell,w in zip(row.cells,widths):
            cell.width=Cm(w/567.)
            if i==0:
                cb=OxmlElement('w:tcBorders');edge=OxmlElement('w:bottom')
                edge.set(qn('w:val'),'single');edge.set(qn('w:sz'),'4');edge.set(qn('w:color'),'415E75')

```

```

        cb.append(edge);cell._tc.get_or_add_tcPr().append(cb)
    for para in cell.paragraphs:
        para.paragraph_format.space_after=Pt(3);para.paragraph_format.space_before=Pt(3)
        para.paragraph_format.line_spacing=1.1
        if len(table.rows)≤10 and i<len(table.rows)-1:
            para.paragraph_format.keep_with_next=True
        for run in para.runs:
            run.font.size=Pt(8.5)
            if i==0:run.bold=True
paragraphs=d.paragraphs
for j,para in enumerate(paragraphs):
    if para.style.name=='Caption':para.paragraph_format.keep_with_next=False
    if para._p.findall('.//'+qn('w:drawing')):
        para.paragraph_format.keep_with_next=True
    if (j+1<len(paragraphs) and para._p.findall('.//'+qn('m:oMath'))
        and paragraphs[j+1]._p.findall('.//'+qn('m:oMath'))):
        para.paragraph_format.keep_with_next=True
d.save(docx)
env=os.environ.copy()
# Local font discovery for headless LibreOffice; no fonts are downloaded.
if Path('/System/Library/Fonts').exists():
    fontconfig=build/'fonts.conf';cache=build/'fontcache';cache.mkdir(exist_ok=True)
    fontconfig.write_text('<?xml version="1.0"?><!DOCTYPE fontconfig SYSTEM
"urn:fontconfig:fonts.dtd"><fontconfig>
    <dir>/System/Library/Fonts</dir><dir>/System/Library/Fonts/Supplemental</dir>
<dir>/Library/Fonts</dir>
    f'<cachedir>{cache}</cachedir><alias><family>Cambria Math</family><prefer><family>STIX Two
Math</family></prefer></alias></fontconfig>')
    env['FONTCONFIG_FILE']=str(fontconfig)
    with tempfile.TemporaryDirectory(prefix='q4_doc_') as profile:
        render=subprocess.run([executable('soffice','soffice'),'--
env:UserInstallation='+Path(profile).as_uri(),
        '--headless','--convert-to','pdf','--outdir',str(docx.parent),str(docx)],
        capture_output=True,text=True,env=env,timeout=60,check=True)
    pdf=docx.with_suffix('.pdf')
    if not pdf.exists():raise RuntimeError('PDF renderer did not create the expected file')
    with zipfile.ZipFile(docx) as z:
        root=ET.fromstring(z.read('word/document.xml'))
        ns=
{'m':'http://schemas.openxmlformats.org/officeDocument/2006/math','w':'http://schemas.openxmlformats.org
/wordprocessingml/2006/main'}
        counts={'native_formulas':len(root.findall('.//m:oMath',ns)),
            'tables':len(root.findall('.//w:tbl',ns)),
            'figures':len([x for x in z.namelist() if x.startswith('word/media/')])}
        if counts['native_formulas']<6 or counts['figures']≠3:raise RuntimeError('Document object count
mismatch')
        manifest={'complete':True,'scope':'Q4 stage report, not full contest manuscript',
            'sources':{(str(x.relative_to(ROOT)) if x.is_relative_to(ROOT) else str(x)):sha(x) for x in
sources},
            'source_markdown_sha256':sha(source),'template_sha256':sha(template),
            'pandoc_output_sha256':sha(intermediate),'docx_sha256':sha(docx),'pdf_sha256':sha(pdf),
            'counts':counts,'presentation_changes':['A4 Chinese styles','explicit table/grid/cell
widths','three-line tables','8.5pt repeated-header non-splitting table rows','valid paragraph-level page
field','required zoom percentage'],

'numerical_outputs_modified':False,'pandoc_stderr':conversion.stderr,'render_stdout':render.stdout,
    'visual_review_required':True}
    (docx.parent/'文稿生成记
录.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8')
    return manifest

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--results',type=Path,default=ROOT/'results/final')

```

```
p.add_argument('--pandoc');p.add_argument('--soffice');a=p.parse_args()  
print(json.dumps(build_document(a.results,a.pandoc,a.soffice),ensure_ascii=False,indent=2))
```

65. question4/code/package_delivery.py

源文件 SHA-256: 3d778471ced89927ed4a172c0ba33bd8df4072786fcc1432627be50ab67f89da ; 代码行数: 98。

```

"""Assemble Q4 deliverables and verify ZIP contents without changing inputs."""
from pathlib import Path
import argparse, hashlib, json, shutil, zipfile

ROOT=Path(__file__).resolve().parents[1]

def sha(path):
    h=hashlib.sha256()
    with Path(path).open('rb') as stream:
        for block in iter(lambda:stream.read(1024*1024),b''):h.update(block)
    return h.hexdigest()

def copy_tree(source,target):
    def ignore(directory,names):
        return [name for name in names if name in ('__pycache__', 'fontcache', 'rendered_pages')
                or name.endswith((''.pyc', '.pending.npz'))]
    shutil.copytree(source,target,ignore=ignore)

def assemble(destination,results_dir=ROOT/'results/final'):
    destination=Path(destination).resolve();results=Path(results_dir).resolve()
    if destination==ROOT or destination in ROOT.parents:raise ValueError('Unsafe delivery destination')
    if destination.exists():raise FileExistsError('Destination already exists; use a new folder, do not
    overwrite unrelated work')
    checks=[results/'run_all_manifest.json',results/'independent_audit.json',
            results/'causal_replay/independent_audit.json',results/'reports/报表与账本核验.json',
            results/'reports/文稿验证.json']
    for path in checks:
        result=json.loads(path.read_text())
        if not result.get('complete',result.get('passed',False)):
            raise RuntimeError(f'Incomplete required validation: {path}')
        if path.name=='文稿验证.json' and not result.get('visual_reviewed'):
            raise RuntimeError('Document pages must be visually reviewed before packaging')
    destination.mkdir(parents=True)
    for folder in ('code', 'data', 'notes'):
        copy_tree(ROOT/folder,destination/folder)
    (destination/'results').mkdir()
    copy_tree(results,destination/'results/final')
    profile=ROOT/'results/data/profile.json'
    if profile.exists():shutil.copy2(profile,destination/'results/data/profile.json')
    for filename in ('README.md','requirements.txt','requirements-documents.txt','第四问_模型与操作方
    法.md'):
        shutil.copy2(ROOT/filename,destination/filename)
    for filename in ('第四问_结果与结论.md','第四问_模型与结果说明.docx','第四问_模型与结果说明.pdf',
        'result4-2.xlsx','result4-3.xlsx'):
        shutil.copy2(results/'reports'/filename,destination/filename)
    copy_tree(results/'figures',destination/'figures')
    provenance=destination/'provenance'
    if (ROOT/'provenance').exists():copy_tree(ROOT/'provenance',provenance)
    else:provenance.mkdir()
    from q4_runner import CORE
    core=provenance/'numerical_core_used';core.mkdir(exist_ok=True)
    for filename in CORE:shutil.copy2(ROOT/'code'/filename,core/filename)
    support=destination/'support';support.mkdir()
    (support/'阅读与验收说明.md').write_text(''# 支撑材料说明

```

完整数值源码、实际输入、逐段轨迹和签名在**code/data/results**；根目录两个**result4**文件是**reports**中原模板的相同副本。根**figures**是正式9图的方便阅读副本。全部实际数值以**results/final**为准。

方法、结论和Word/PDF为第四问专项说明，未冒称整篇竞赛论文或官方AI报告完成。**notes**中保留独立审查、通用扫描兼容性、数据口径、求解精度、分钟执行和运行日志。

交付包不含平台相关**vendor**与**Python**虚拟环境；**requirements**锁定实际库版本。数值复现已在同版本环境核验，不冒称新环境安装验收。前阶段电价比较的完整代码仍保留在另行交付《第四问_电价预测》，本包直接使用它的冻结零点输入，并包括本次日内**XGBoost**更新代码。

本包未推送**GitHub**。原用户文件和前问冻结结果未改。

```
'''encoding='utf-8')
# Development pilots are evidence of engineering, not formal fee results.
for path in (destination/'notes').glob('pilot*.txt'):path.unlink()
return destination

def seal(destination):
    destination=Path(destination).resolve()
    files=[p for p in destination.rglob('*') if p.is_file() and p.name!='交付文件清单.json']
    manifest={'file_count':len(files),'files':{str(p.relative_to(destination)):
{'sha256':sha(p),'bytes':p.stat().st_size}
        for p in sorted(files)},'scope':'Q4 dispatch with frozen XGBoost price inputs'}
    (destination/'交付文件清单.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8')
    archive=destination.with_suffix('.zip')
    if archive.exists():raise FileExistsError('ZIP already exists; explicitly choose another delivery name')
    with zipfile.ZipFile(archive,'w',compression=zipfile.ZIP_DEFLATED,compresslevel=6) as z:
        for p in sorted(destination.rglob('*')):
            if p.is_file():z.write(p,str(Path(destination.name)/p.relative_to(destination)))
    with zipfile.ZipFile(archive) as z:
        bad=z.testzip()
        if bad is not None:raise RuntimeError('ZIP CRC failed: '+bad)
        for name,info in manifest['files'].items():
            contents=z.read(str(Path(destination.name)/name))
            if hashlib.sha256(contents).hexdigest()!=info['sha256']:raise RuntimeError('ZIP file SHA mismatch')
        checked={'passed':True,'files_plus_manifest':len(files)+1,'zip_bytes':archive.stat().st_size,
'zip_sha256':sha(archive),'zip':str(archive),'directory':str(destination),'crc_and_per_file_sha256_verified':True}
        (destination.parent/(destination.name+'_交付校验.json')).write_text(json.dumps(checked,ensure_ascii=False,indent=2),encoding='utf-8')
        return checked

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--dest',type=Path,required=True)
    p.add_argument('--results',type=Path,default=ROOT/'results/final');p.add_argument('--seal-only',action='store_true')
    a=p.parse_args()
    if not a.seal_only:assemble(a.dest,a.results)
    print(json.dumps(seal(a.dest),ensure_ascii=False,indent=2))
```

66. question4/code/physical_io.py

源文件 SHA-256: 3bd8d306c4314029ac76632274eeb10c38f622f577d521a240e49b9dce50b696 ; 代码行数: 397。

```

"""Q3 strict data adapter: interval means, forecast vintages and observed nodes.

User workbooks are read-only. The returned power arrays are kW; optimization
must multiply them by 1/6 exactly once. The contract window is D 00:10 to
D+1 00:10. No actual value is implicitly substituted for a supplied forecast.
"""

from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import json
import re
import shutil

import numpy as np
from openpyxl import load_workbook

ROOT = Path(__file__).resolve().parents[1]
ISSUE_HOURS = (0, 6, 12, 18)
ISSUE_START = {0: 0, 6: 35, 12: 71, 18: 107}
WECHAT =
Path('/Users/xzy/Library/Containers/com.tencent.xinWeChat/Data/Documents/xwechat_files/wxid_euf4op8bmams
22_1b0d/temp/RWTemp/2026-09/a3bee7770c01c7d37f45899f94a1dcc5')
ATTACHMENTS = Path('/Users/xzy/Downloads/CUMCM2026Problems/C题/附件')
SOURCES = {
    'pv_forecasts.xlsx': WECHAT / '问题三_光伏区间预测.xlsx',
    'load_forecasts.xlsx': WECHAT / '问题二_标准化区间预测.xlsx',
    'attachment1.xlsx': ATTACHMENTS / '附件1.xlsx',
    'attachment2.xlsx': ATTACHMENTS / '附件2.xlsx',
    'attachment3.xlsx': ATTACHMENTS / '附件3.xlsx',
    'result3_template.xlsx': ATTACHMENTS / '附件5/result3.xlsx',
}

def sha(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def as_date(value):
    if isinstance(value, dt.datetime):
        return value.date()
    if isinstance(value, dt.date):
        return value
    return dt.date(*map(int, str(value).split('-')))

def clock_minute(value):
    if isinstance(value, dt.time):
        return value.hour * 60 + value.minute
    s = str(value).strip().replace(':', ':')
    tomorrow = ('次日' in s) or ('+1' in s)
    s = s.replace('次日', '').replace('+1', '')
    m = re.fullmatch(r'(\d{1,2}):(\d{1,2})', s)
    if not m:
        raise ValueError(f'Invalid clock: {value!r}')
    h, minute = map(int, m.groups())
    if h > 24 or minute > 59 or (h == 24 and minute):
        raise ValueError(f'Invalid clock: {value!r}')
    return h * 60 + minute + (1440 if tomorrow and h < 24 else 0)

def interval_header(value):

```

```

pieces = re.split(r'[-~\-\-]', str(value).strip())
if len(pieces) != 2:
    raise ValueError(f'Explicit interval required: {value!r}')
a, b = map(clock_minute, pieces)
if a == 1430 and b == 0:
    b = 1440
if b - a != 10:
    raise ValueError(f'Interval is not 10 minutes: {value!r}')
return a, b


def validate_dates(values, label, *, expected=None):
    dates = [as_date(v) for v in values]
    if dates != sorted(set(dates)):
        raise ValueError(f'{label}: dates must be unique and ascending')
    if any(b - a != dt.timedelta(days=1) for a, b in zip(dates, dates[1:])):
        raise ValueError(f'{label}: missing interior date')
    if expected is not None and dates != expected:
        raise ValueError(f'{label}: forecast dates differ')
    return dates


def numeric_array(rows, shape, label):
    try:
        a = np.asarray(rows, dtype=float)
    except (ValueError, TypeError) as exc:
        raise ValueError(f'{label}: nonnumeric values') from exc
    if a.shape != shape or not np.isfinite(a).all() or (a < 0).any():
        raise ValueError(f'{label}: missing, negative, or invalid data; shape={a.shape}')
    return a


def read_load(path):
    w = load_workbook(path, read_only=True, data_only=True)
    try:
        rows = list(w['XGBoost_负载'].values)
    finally:
        w.close()
    if [interval_header(x) for x in rows[0][1:]] != [(t,t+10) for t in range(10,1441,10)]:
        raise ValueError('Load must cover 144 means from D 00:10 to D+1 00:10')
    dates = validate_dates([r[0] for r in rows[1:]], 'load')
    a = numeric_array([r[1:] for r in rows[1:]], (len(dates),144), 'load')
    return dates, a


def read_pv(path, dates):
    """Keep all supplied numerical values; crop only 00h's unused first mean.

    All four supplied sheets contain an isolated header typo at 23:50. It is
    explicitly repaired in this adapter only after verifying every other
    target, date and width. The source workbooks are never edited.
    """
    w = load_workbook(path, read_only=True, data_only=True)
    pv = np.full((len(dates),4,144), np.nan)
    valid = np.zeros((4,144), bool)
    corrections = []
    shapes = []
    first = None
    try:
        for k,h in enumerate(ISSUE_HOURS):
            sheet = f'{h:02d}时预报'
            rows = list(w[sheet].values)
            headers = list(rows[0][1:])
            expected = [(t,t+10) for t in range(h*60,1441,10)]
            if len(headers) != len(expected):
                raise ValueError(f'{sheet}: wrong forecast width')

```



```

        for j,(header,target) in enumerate(zip(headers,expected)):
            if header == '23:50-次日00:10' and target == (1430,1440):
                headers[j] = '23:50-次日00:00'
                corrections.append({'sheet':sheet,'excel_column':j+2,
                                   'original':header,'normalized':headers[j],
                                   'reason':'isolated 20-minute label overlaps the next explicit 10-minute
interval; ordered 10-minute grid and supplied load format identify intended target',
                                   'numerical_values_changed':False})
            if [interval_header(v) for v in headers] != expected:
                raise ValueError(f'{sheet}: target intervals differ from expected grid')
            validate_dates([r[0] for r in rows[1:]],sheet,expected=dates)
            a = numeric_array([r[1:] for r in rows[1:]], (len(dates),len(expected)), sheet)
            if h == 0:
                first = a[:,0].copy()
                a = a[:,1:]
                start = ISSUE_START[h]
                pv[:,k,start:] = a
                valid[k,start:] = True
                shapes.append({'sheet':sheet,'rows':len(rows),'columns':len(headers)+1,
                               'source_intervals':len(expected),'used_intervals':144-start,
                               'issued_hour':h,'target_first_minute':max(10,h*60),'target_last_end_minute':1450,
                               'power_min_kw':float(a.min()),'power_max_kw':float(a.max())})
            finally:
                w.close()
        return pv,valid,first,{'sheets':shapes,'header_corrections':corrections,
                                '00h_first_interval_cropped':True,'input_value_kind':'interval_mean_kw',
                                'averaging_of_supplied_values_performed':False}

def read_actual_source(path):
    w = load_workbook(path, read_only=True, data_only=True)
    result = {}
    dates = None
    try:
        for sheet,key in [('小区负载','load'),('光伏发电实际功率','pv')]:
            rows = list(w[sheet].values)
            if [clock_minute(v) for v in rows[0][1:]] != list(range(10,1441,10)):
                raise ValueError(f'{sheet}: must contain 144 instantaneous nodes')
            ds = validate_dates([r[0] for r in rows[1:]],sheet,expected=dates)
            dates = ds
            vals = numeric_array([r[1:] for r in rows[1:]],(len(ds),144),sheet)
            result[key] = {day: vals[i] for i,day in enumerate(ds)}
    finally:
        w.close()
    return result,dates

def boundary_candidates(y):
    x = np.arange(6,dtype=float); z = y[-6:]
    slope = float(np.dot(x-x.mean(),z-z.mean()) / np.dot(x-x.mean(),x-x.mean()))
    return {'hold':float(y[-1]),'linear2':float(max(0,2*y[-1]-y[-2])),
            'ols6':float(max(0,z.mean()+slope*(6-x.mean())))}

def attach_actuals(data,path):
    raw,source_dates = read_actual_source(path)
    n = len(data['dates'])
    estimated = np.zeros((n,144),bool)
    boundaries = {}
    for key in ('load','pv'):
        nodes = np.empty((n,145))
        for i,day in enumerate(data['dates']):
            if day not in raw[key]:
                raise ValueError('Forecast date missing from observed nodes')
            y = raw[key][day]
            nodes[i,:144] = y

```

```

        tomorrow = day + dt.timedelta(days=1)
        if tomorrow in raw[key]:
            nodes[i,144] = raw[key][tomorrow][0]
        elif day == source_dates[-1]:
            candidates = boundary_candidates(y)
            nodes[i,144] = candidates['hold']
            estimated[i,143] = True
            boundaries[key] = {method:{'right_endpoint_kw':v, 'mean_kw':float((y[-1]+v)/2)}}
                for method,v in candidates.items()
        else:
            raise ValueError('Missing interior endpoint cannot be extrapolated')
        data[f'{key}_nodes'] = nodes
        data[f'{key}_a'] = (nodes[:, :-1]+nodes[:, 1:])/2
    data['actual_estimated'] = estimated
    day0 = data['dates'][0]; prev = day0-dt.timedelta(days=1)
    if prev not in raw['load']:
        raise ValueError('Need observed previous midnight boundary for audit')
    data['first_unexecuted_interval'] = {
        'interval_start':f'{day0}T00:00:00', 'interval_end':f'{day0}T00:10:00',
        'status':'not part of the user-confirmed experiment; no automatic forecast/order/state
invented',
        **{f'{key}_{label}_kw':float(v) for key in ('load', 'pv')
            for label,v in [(('left_node', raw[key][prev][-1]), ('right_node', raw[key][day0][0]),
                            ('mean', (raw[key][prev][-1]+raw[key][day0][0])/2))]}
    data['metadata']['actual'] = {'source_kind':'instantaneous_kw',
        'mean_kind':'adjacent endpoint arithmetic mean (linear between-node approximation)',
        'cross_midnight':'last node of row D paired with first node of row D+1',
        'available_at':'interval_end for complete mean; node observation available at its timestamp',
        'estimated_interval_count':int(estimated.sum()),
        'last_right_endpoint_candidates':boundaries,
        'first_unexecuted_interval':data['first_unexecuted_interval']}
    return raw

def read_price(path):
    w=load_workbook(path, read_only=True, data_only=True)
    try:
        rows=list(w.active.values)
    finally:
        w.close()
    if len(rows)≠145 or tuple(rows[0][:2])≠('时间', '电价'):
        raise ValueError('Attachment1 dimensions/headers invalid')
    if [clock_minute(r[0]) for r in rows[1:]]≠list(range(10,1441,10)):
        raise ValueError('Attachment1 must contain instantaneous 00:10..24:00')
    p=numeric_array([r[1] for r in rows[1:]], (144,), 'price')
    if (p≤0).any():
        raise ValueError('Prices must be positive')
    natural_nodes=np.r_[p[-1], p]
    natural=(natural_nodes[:-1]+natural_nodes[1:])/2
    return np.r_[natural[1:], natural[0]], {'source_kind':'instantaneous_price',
        '00:00_boundary':p(00:00)=p(24:00), periodic daily tariff',
        'settlement':'interval mean price approximation',
        'first_contract_price':float(natural[1]), 'last_contract_price':float(natural[0])}

def read_hourly_source(path):
    w=load_workbook(path, read_only=True, data_only=True)
    try:
        rows=list(w.active.values)
    finally:
        w.close()
    if len(rows)≠1461 or len(rows[0])≠26 or list(rows[0][2:])≠[f'预报{i}小时' for i in range(1,25)]:
        raise ValueError('Attachment3 hourly source dimensions/headers invalid')
    raw={}; date=None
    for r in rows[1:]:
        if r[0]: date=as_date(r[0])

```

```

h=clock_minute(r[1])//60
if h not in ISSUE_HOURS or (date,h) in raw:
    raise ValueError('Invalid or duplicate hourly forecast issuance')
raw[date,h]=numeric_array(r[2:],(24,),'hourly forecast')
return raw

def audit_linear_reconstruction(data,path,actual_source):
    raw=read_hourly_source(path)
    linear=np.full_like(data['pv_f'],np.nan)
    rows=[]
    for k,h in enumerate(ISSUE_HOURS):
        start=ISSUE_START[h]
        for i,day in enumerate(data['dates']):
            # Only the observation AT release time, not a later actual node.
            if h==0:
                observed=actual_source['pv'][day-dt.timedelta(days=1)][-1]
            else:
                observed=actual_source['pv'][day][h*6-1]
            hourly_nodes=np.r_[observed,raw[day,h]]
            minutes=np.arange(max(10,h*60),1451,10)
            fine=np.interp(minutes,np.arange(h,h+25)*60,hourly_nodes)
            linear[i,k,start:]=(fine[:-1]+fine[1:])/2
            supplied=data['pv_f'][:,k,start:]; ref=linear[:,k,start:]
            error=supplied-ref
            rows.append({'issue_hour':h,'compared_values':int(error.size),
                'mean_absolute_difference_kw':float(np.abs(error).mean()),
                'max_absolute_difference_kw':float(np.abs(error).max()),
                'match_fraction_at_1e-7_kw':float(np.mean(np.abs(error)≤1e-7))})
    data['pv_linear_f']=linear
    data['metadata']['pv_generation_audit']={
        'finding':'supplied values are not identical to direct piecewise-linear interpolation of the
same issue hourly nodes followed by endpoint averaging',
        'comparison':rows,
        'main_input':'supplied interval means, unchanged',
        'alternative_input':'pv_linear_f reconstructed only from same-issue hourly forecast and release-
time observation',
        'linear_00h_final_boundary':'hold forecast node at next 00:00 to next 00:10, without reading a
future forecast',
        'causality_status':'supplied generator unavailable; causality not independently certified;
discrepancy alone is not evidence of future-data leakage',
        'load_generator_status':'supplied midnight forecasts; training cutoff/provenance not
independently certified'}

def inspect_template(path):
    w=load_workbook(path,read_only=True,data_only=True)
    try:
        shapes={s.title:{'rows':s.max_row,'columns':s.max_column} for s in w}
        for name in ('计划购电量','调整购电量'):
            s=w[name]; r=next(s.values)
            # The final source label 0:00-0:10+1 is a known template shorthand
            # for NEXT-day 00:00-00:10, not a 24h10m interval.
            for i,h in enumerate(r[1:145]):
                if i==143:
                    if h≠'0:00-0:10+1': raise ValueError('Unexpected final template header')
                elif interval_header(h)≠(10*(i+1),10*(i+2)):
                    raise ValueError('Template contract intervals do not match')
    finally:
        w.close()
    return {'sheets':shapes,'purchase_sheets':'144 contract slots D 00:10 to D+1 00:10',
        'storage_sheet':'natural-day blocks and true 00:00/24:00 states; example rows include ellipsis
and must be expanded for all dates',
        'emergency_sheet':'example rows include ellipsis; export all actual episodes, split at natural-
day midnight',
        'final_purchase_header_alias':'0:00-0:10+1 means next-day 00:00-00:10',

```

```

    'first_natural_day_gap': 'first 00:00-00:10 is outside this experiment; must be explicitly marked
rather than filled with invented dispatch'}

def copy_sources(destination=ROOT/'data'):
    destination=Path(destination);destination.mkdir(parents=True,exist_ok=True)
    copied=[]
    for name,path in SOURCES.items():
        target=destination/name
        source_hash=sha(path)
        if target.exists() and sha(target)!=source_hash:
            raise ValueError(f'Refuse to overwrite a differing copied input: {target}')
        if not target.exists(): shutil.copy2(path,target)
        if sha(target)!=source_hash: raise AssertionError('Copy integrity failed')

    copied.append({'filename':name, 'original_path':str(path), 'sha256':source_hash, 'bytes':path.stat().st_size})

    (destination/'source_manifest.json').write_text(json.dumps(copied,ensure_ascii=False,indent=2),encoding='utf-8')
    return copied

def load_inputs(data_dir=ROOT/'data'):
    """Return datetime-based metadata plus numerical arrays (all power in kW).

    pv_f[day, vintage_index, slot] uses vintage indices 0/1/2/3 for hours
    0/6/12/18. Values before that vintage is released are deliberately NaN;
    downstream code must select only the released vintage's valid tail.
    """
    data_dir=Path(data_dir)
    dates,load=read_load(data_dir/'load_forecasts.xlsx')
    pv,valid,pv_first,pm=read_pv(data_dir/'pv_forecasts.xlsx',dates)
    price,price_meta=read_price(data_dir/'attachment1.xlsx')
    starts=[dt.datetime.combine(d,dt.time(0,10)) for d in dates]
    data={'dates':dates, 'window_start':starts,
        'window_end':[a+dt.timedelta(days=1) for a in starts],
        'issued_at':[a-dt.timedelta(minutes=10) for a in starts],
        'issue_hours':ISSUE_HOURS, 'issue_start':ISSUE_START.copy(),
        'load_f':load, 'pv_f':pv, 'pv_valid':valid,
        'pv_unused_first_mean_kw':pv_first, 'price':price,
        'metadata':{'window':'D 00:10 to D+1 00:10, 144 intervals',
            'time_zone':'Asia/Shanghai', 'value_kind':'interval_mean_kw',
            'energy_conversion':'multiply kW by 1/6 h exactly once; never re-average supplied means',
            'initial_soc_kwh':6000, 'initial_soc_basis':'user-prescribed common experimental state at
first 00:10; not derived February state',
            'forecast':pm, 'price':price_meta,
            'source_sha256':{'name':sha(data_dir/name) for name in SOURCES},
            'disabled_forecast_vintages':'must not be read by policy or residual generator for a
disabled issuance',
            'history_availability':'require all actual interval endpoints used by a history track ≤
current decision timestamp'}}
    actual_source=attach_actuals(data,data_dir/'attachment2.xlsx')
    audit_linear_reconstruction(data,data_dir/'attachment3.xlsx',actual_source)
    data['metadata']['template']=inspect_template(data_dir/'result3_template.xlsx')
    if dates[0]!=dt.date(2025,2,1) or dates[-1]!=dt.date(2025,12,31) or len(dates)!=334:
        raise ValueError('Current experiment expects 2025-02-01..2025-12-31, 334 windows')
    # Compatibility conveniences; these are views, never additional inputs.
    data['pv_f_by_hour']={h:pv[:,k,:] for k,h in enumerate(ISSUE_HOURS)}
    data['pv0_f']=pv[:,0,:]
    return data

def save_bundle(data,out=ROOT/'data'):
    out=Path(out);out.mkdir(exist_ok=True,parents=True)
    arrays={key:value for key,value in data.items() if isinstance(value,np.ndarray)}
```

```

arrays.update(dates=np.array([str(d) for d in data['dates']]),
              window_start=np.array([x.isoformat() for x in data['window_start']]),
              window_end=np.array([x.isoformat() for x in data['window_end']]),
              issued_at=np.array([x.isoformat() for x in data['issued_at']]),
              issue_hours=np.asarray(ISSUE_HOURS), issue_start_indices=np.asarray([ISSUE_START[h] for h in
ISSUE_HOURS]))
np.savez_compressed(out/'q3_inputs.npz', **arrays)

(out/'input_audit.json').write_text(json.dumps(data['metadata'], ensure_ascii=False, indent=2), encoding='u
tf-8')
return out/'q3_inputs.npz'

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--data', type=Path, default=ROOT/'data')
    parser.add_argument('--copy-originals', action='store_true')
    args = parser.parse_args()
    if args.copy_originals: copy_sources(args.data)
    bundle = load_inputs(args.data)
    artifact = save_bundle(bundle, args.data)
    print(json.dumps({'bundle': str(artifact), 'windows': len(bundle['dates']),
                      'load_shape': list(bundle['load_f'].shape), 'pv_shape': list(bundle['pv_f'].shape),
                      'estimated_actual_intervals': int(bundle['actual_estimated'].sum()),
                      'header_corrections': len(bundle['metadata']['forecast']
['header_corrections'])}, ensure_ascii=False))

```

67. question4/code/plot_q4_relative_schedule.py

源文件 SHA-256: fccf2917cabfba4fe885da59de5440acc1ab2b7f319d0c7c777e88f70d2a9ebf ; 代码行数: 82。

```

"""绘制第四问图17：八种组合相对主策略的库存评价差。"""
from pathlib import Path
import json

import matplotlib.pyplot as plt
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
CSV = ROOT / "results" / "final" / "analysis" / "主实验比较.csv"
OUT = Path(r"F:\work\mcm2026\CUMCM2026Problems\C题\附件\多时间尺度与动态电价下微网购电及储能协同调控模型\figures")

def main() → None:
    df = pd.read_csv(CSV)
    # 图17比较问题4-3的八种光伏预报/调单组合。
    df = df[(df["branch"] == "4-3") & (df["price_mode"] == "point")].copy()
    df["label"] = df["pv_hours"].map(lambda x: "/" .join(f"{int(h):02d}" for h in x.split(",)))
    selected = df.loc[df["selected_by_development"], "strategy"].iloc[0]
    selection = json.loads((ROOT / "results" / "final" / "selection.json").read_text(encoding="utf-8"))
    development = {row["strategy"]: row["score"] for row in selection["development"]
                    if row["branch"] == "4-3" and row["price_mode"] == "point"}

    panels = [
        ("开发期7窗", "开发期7窗"),
        ("留出期310窗", "留出期310窗"),
    ]
    # 主策略作为共同零点：两面板均以每窗口库存评价表示，避免把合同窗总费
    # 与自然日费用混用。
    base = df.loc[df["strategy"] == selected, "test_inventory_score_yuan"].iloc[0] / 310
    dev_base = development[selected] / 7
    values = {
        "开发期7窗": (df["strategy"].map(development) / 7 - dev_base),
        "留出期310窗": (df["test_inventory_score_yuan"] / 310 - base),
    }

    plt.rcParams.update({
        "font.family": "sans-serif",
        "font.sans-serif": ["Microsoft YaHei", "SimHei", "DejaVu Sans"],
        "axes.unicode_minus": False,
        "font.size": 10,
    })
    fig, axes = plt.subplots(1, 2, figsize=(12.0, 5.2), sharey=True)
    color_main = "#2F7F73"
    color_other = "#9AAEBA"

    for ax, (title, xlabel) in zip(axes, panels):
        plot_df = df[["label", "strategy"]].copy()
        plot_df["delta"] = values[title].to_numpy()
        plot_df = plot_df.sort_values("delta", ascending=True)
        colors = [color_main if s == selected else color_other for s in plot_df["strategy"]]
        bars = ax.barh(plot_df["label"], plot_df["delta"], color=colors, edgecolor="white",
        linewidth=0.7)
        ax.axvline(0, color="#333333", linewidth=1.0)
        selected_pos = plot_df.index[plot_df["strategy"] == selected][0]
        selected_y = list(plot_df.index).index(selected_pos)
        ax.scatter([0], [selected_y], marker="D", s=30, color=color_main, zorder=4)
        ax.grid(axis="x", color="#D9E0E3", linewidth=0.8, alpha=0.8)
        ax.set_axisbelow(True)
        ax.set_title(title, pad=10, fontsize=12, fontweight="bold")
        ax.set_xlabel("相对主策略多出的库存评价（元/窗）")
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)

```

```

ax.spines["left"].set_color("#555555")
ax.spines["bottom"].set_color("#555555")
max_delta = max(plot_df["delta"].max(), 1)
ax.set_xlim(-max_delta * 0.10, max_delta * 1.18)
for bar, val in zip(bars, plot_df["delta"]):
    x = val + max_delta * 0.025
    ax.text(x, bar.get_y() + bar.get_height() / 2, f"{val:+.0f}", va="center", ha="left",
           fontsize=9)

axes[0].set_ylabel("光伏预报/调单时刻组合")
fig.suptitle("八种策略相对主策略的费用差", fontsize=14, fontweight="bold", y=1.02)
fig.text(0.5, -0.02, "绿色: 开发期冻结的主策略（相对值为0）", ha="center", fontsize=9)
fig.tight_layout()
OUT.mkdir(parents=True, exist_ok=True)
fig.savefig(OUT / "q4_eight_schedules.png", dpi=350, bbox_inches="tight")
fig.savefig(OUT / "q4_eight_schedules.pdf", bbox_inches="tight")
plt.close(fig)

if __name__ == "__main__":
    main()

```

68. question4/code/price_inputs.py

源文件 SHA-256: 98451339aa4f2022449d5e45618d3859bab37cee3461c89e91d1ae2611daffe3d ; 代码行数: 240。


```

"""Versioned Q4 price inputs: inherited midnight and causal XGBoost updates.

All price arrays use CNY/kWh. No multiplication by 1/6 occurs in this module.
The 144 contract intervals start D 00:10 and end D+1 00:10.
"""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor, as_completed
import argparse
import hashlib
import importlib.metadata
import json
import numpy as np
import pandas as pd
import openpyxl

from xgb_intraday import forecast_intraday

ROOT=Path(__file__).resolve().parents[1]
VINTAGES=(0,6,12,18)
GATE_SLOTS=(0,35,71,107)

def _sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def read_attachment4(path: Path) → np.ndarray:
    book=openpyxl.load_workbook(path,read_only=True,data_only=False)
    try:
        sheet=book.active
        if len(book.sheetnames)≠1 or (sheet.max_row,sheet.max_column)≠(366,145):
            raise ValueError('Attachment4 must contain 365 dates by 144 instantaneous nodes')
        rows=list(sheet.iter_rows(values_only=True))
        dates=pd.DatetimeIndex([r[0] for r in rows[1:]])
        if not dates.equals(pd.date_range('2025-01-01',periods=365)):
            raise ValueError('Price dates are not the expected complete 2025 daily sequence')
        # This source is the hash-verified original attachment, but still check
        # that its node headers run 00:10 ... 24:00 rather than interval means.
        for j,label in enumerate(rows[0][1:]):
            minutes=(label.hour*60+label.minute) if hasattr(label,'hour') else None
            if isinstance(label,str):
                text=label.replace(' ','').replace('次日','').replace('+1','').strip()
                h,m=map(int,text.split(':')[2]);minutes=h*60+m
                if j==143 and minutes==0:minutes=1440
            if hasattr(label,'total_seconds'):minutes=label.total_seconds()/60
            if minutes≠(j+1)*10:raise ValueError(f'Unexpected instantaneous node header at column
{j+2}')
        values=np.asarray([r[1:] for r in rows[1:]],dtype=float)
        if values.shape≠(365,144) or not np.isfinite(values).all():raise ValueError('Invalid actual
price values')
        return values
    finally:book.close()

def extrapolate_last_node(history: np.ndarray, method: str) → float:
    """Only the final missing Jan1 00:10 node is extrapolated, and flagged."""
    y=np.asarray(history,dtype=float).reshape(-1)
    if len(y)<6 or not np.isfinite(y[-6:]).all():raise ValueError('Need six finite historical nodes')
    if method=='hold':return float(y[-1])
    if method=='linear2':return float(2*y[-1]-y[-2])
    if method=='ols6':return float(np.polyval(np.polyfit(np.arange(6),y[-6:],1),6))
    raise ValueError('Boundary method must be hold, linear2 or ols6')

```

```

@dataclass(frozen=True)
class PriceData:
    dates: pd.DatetimeIndex
    predicted: np.ndarray
    nodes: np.ndarray
    actual: np.ndarray
    actual_nodes: np.ndarray
    raw_actual_nodes: np.ndarray
    actual_estimated: np.ndarray
    forecast_bridge: np.ndarray
    actual_bridge: np.ndarray
    known_midnight: np.ndarray
    audit: dict
    vintages: tuple=VINTAGES
    gate_slots: tuple=GATE_SLOTS

    @property
    def point_nodes(self):return self.nodes

    @property
    def startup_actual(self):return float(self.actual_bridge[0])

    @property
    def boundary_alternatives(self):return self.audit['final_boundary_alternatives']

    def day_index(self, day):
        if isinstance(day,(int,np.integer)) and not isinstance(day,bool):index=int(day)
        else:index=int(self.dates.get_indexer([pd.Timestamp(day)]))[0]
        if not 0≤index<len(self.dates):raise IndexError('Price date outside experiment')
        return index

    def for_day(self, day, version_hour=0, remaining=False):
        i=self.day_index(day)
        if version_hour not in self.vintages:raise ValueError('Price version must be 0,6,12 or18')
        v=self.vintages.index(version_hour)
        return self.predicted[i,v,self.gate_slots[v]:] if remaining else self.predicted[i,v]

    def actual_with_boundary(self, method='hold'):
        out=self.actual.copy()
        out[-1,-1]=self.boundary_alternatives[method]['last_interval_price']
        return out

def load_price_data(path: Path | str = ROOT/'data/price_inputs.npz') → PriceData:
    with np.load(path,allow_pickle=False) as z:
        keys=['predicted','nodes','actual','actual_nodes','raw_actual_nodes','actual_estimated',
            'forecast_bridge','actual_bridge','known_midnight']
        arrays={k:z[k].copy() for k in keys}
        dates=pd.DatetimeIndex(z['dates'].astype(str))
        audit=json.loads(str(z['audit_json']).item())
    if arrays['predicted'].shape≠(334,4,144) or arrays['nodes'].shape≠(334,4,145):
        raise ValueError('Versioned price arrays have unexpected dimensions')
    for x in arrays.values():x.setflags(write=False)
    return PriceData(dates=dates,**arrays,audit=audit)

def _forecast_vintage(hour,values,config):
    predictions=[];diagnostics=[]
    for i,day in enumerate(range(31,365)):
        cutoff=day*144+hour*6
        p,d=forecast_intraday(values[:cutoff],day,hour,config)
        predictions.append(p);diagnostics.append(d)
        if (i+1)%40==0:print(f'price {hour:02d}:00: {i+1}/334 forecasts',flush=True)
    return hour,np.asarray(predictions),diagnostics

```

```

def assemble_arrays(raw: np.ndarray, midnight: np.ndarray, updates: dict) → dict:
    """Pure time mapping: 145 inherited nodes become 144 intervals."""
    raw=np.asarray(raw,float)
    if raw.shape≠(365,144) or midnight.shape≠(334,145):raise ValueError('Wrong source shapes')
    flat=raw.ravel();nodes=np.full((334,4,145),np.nan)
    nodes[:,0]=midnight
    raw_actual_nodes=np.full((334,145),np.nan)
    midnight_actual=raw[np.arange(31,365)-1,143]
    for i,day in enumerate(range(31,365)):
        first=day*144
        end=min(first+145,len(flat))
        raw_actual_nodes[i,:end-first]=flat[first:end]
        for v,hour in enumerate(VINTAGES[1:],1):
            slot=GATE_SLOTS[v]
            nodes[i,v,slot]=flat[first+slot] # known release-time left node
            nodes[i,v,slot+1]=updates[hour][i]
    actual_nodes=raw_actual_nodes.copy()
    actual_nodes[-1,-1]=extrapolate_last_node(flat,'hold')
    predicted=(nodes[:,:,:-1]+nodes[:,:,:1])/2
    actual=(actual_nodes[:,:,:-1]+actual_nodes[:,:,:1])/2
    estimated=~(np.isfinite(raw_actual_nodes[:,:,:-1])&np.isfinite(raw_actual_nodes[:,:,:1]))
    bridge=(midnight_actual+midnight[:,0])/2
    actual_bridge=(midnight_actual+raw[31:,0])/2
    if not np.array_equal(nodes[:,0],midnight):raise AssertionError('Midnight node inheritance changed')
    for v,slot in enumerate(GATE_SLOTS):
        if not np.isnan(predicted[:,v,:slot]).all() or not np.isfinite(predicted[:,v,:slot]).all():
            raise AssertionError('Wrong pre/post-vintage availability')
    assert estimated.sum()==1 and estimated[-1,-1]
    return dict(predicted=predicted,nodes=nodes,actual=actual,actual_nodes=actual_nodes,
                raw_actual_nodes=raw_actual_nodes,actual_estimated=estimated,forecast_bridge=bridge,
                actual_bridge=actual_bridge,known_midnight=midnight_actual)

def build_price_inputs(data_dir: Path | str = ROOT/'data', workers: int=3, force=False) → PriceData:
    data_dir=Path(data_dir);data_dir.mkdir(parents=True,exist_ok=True)
    sources={name:data_dir/name for name in
('attachment4.xlsx','xgboost_midnight_predictions.npz','xgboost_midnight_summary.json')}
    summary=json.loads(sources['xgboost_midnight_summary.json'].read_text())
    if summary['family']≠'XGBoost' or not summary.get('complete'):raise ValueError('Completed XGBoost
midnight run required')
    config=summary['config']
    hashes={name:_sha(path) for name,path in sources.items()}
    if hashes['attachment4.xlsx']≠summary['source_sha256']:raise ValueError('Actual input differs from
source used for frozen midnight forecasts')
    core_hashes={name:_sha(Path(__file__).parent/name) for name in
('price_inputs.py','xgb_intraday.py')}
    spec=
{'sources':hashes,'core':core_hashes,'config':config,'xgboost_version':importlib.metadata.version('xgboo
st')}
    signature=hashlib.sha256(json.dumps(spec,sort_keys=True).encode()).hexdigest()
    output=data_dir/'price_inputs.npz'
    if output.exists() and not force:
        old=load_price_data(output)
        if old.audit.get('signature')≠signature:raise ValueError('Cached price inputs use different
sources/code; use a new data directory or explicit force rebuild')
        return old
    raw=read_attachment4(sources['attachment4.xlsx']);flat=raw.ravel()
    with np.load(sources['xgboost_midnight_predictions.npz'],allow_pickle=False) as z:
        midnight=z['predictions'].copy();origins=z['origins'].copy();old_truth=z['actual'].copy()
        if not np.array_equal(origins,np.arange(31,365)) or midnight.shape≠(334,145) or not
np.isfinite(midnight).all():
            raise ValueError('Midnight predictions must have all 334 requested dates and 145 finite nodes')
        if not np.array_equal(old_truth[:,:,:144],raw[31,:]):raise ValueError('Frozen midnight truth and
original attachment disagree')

```

```

updates={};fit_diagnostics={}
with ProcessPoolExecutor(max_workers=min(3,workers)) as pool:
    tasks=[pool.submit(_forecast_vintage,hour,flat,config) for hour in (6,12,18)]
    for task in as_completed(tasks):
        hour,pred,diag=task.result();updates[hour]=pred;fit_diagnostics[str(hour)]=diag
arrays=assemble_arrays(raw,midnight,updates)
boundary={method:{'last_node_price':extrapolate_last_node(flat,method),
                  'last_interval_price':(flat[-1]+extrapolate_last_node(flat,method))/2}
          for method in ('hold','linear2','ols6')}
comparison=[]
for v, hour in enumerate(VINTAGES[1:],1):
    slot=GATE_SLOTS[v]
    truth=arrays['raw_actual_nodes'][:,slot+1:]
    new=arrays['nodes'][:,v,slot+1:];old=midnight[:,slot+1:]
    good=np.isfinite(truth)
    comparison.append({'issue_hour':hour,'scored_future_nodes':int(good.sum()),
                      'target_note':'same target right nodes after issue; known issue node excluded; absent final
actual excluded',
                      'updated_MAE':float(np.abs(new[good]-truth[good]).mean()),
                      'midnight_MAE_same_targets':float(np.abs(old[good]-truth[good]).mean()),
                      'updated_bias':float((new[good]-truth[good]).mean())})
finite_nodes=arrays['nodes'][np.isfinite(arrays['nodes'])]
audit={'signature':signature,'provenance':spec,'complete':True,
       'quantity':'interval price CNY/kWh formed from instantaneous node endpoint average',
       'price_times_1_over_6':False,'forecast_origins':'2025-02-01 through 2025-12-31 at 0/6/12/18',
       'contract_window':'D00:10 through D+1 00:10, 144 ten-minute intervals',
       'vintages':list(VINTAGES),'gate_slots':list(GATE_SLOTS),'array_shape':[334,4,144],
       'midnight_exact_inheritance':True,'midnight_retuned_or_refit':False,
       'intraday_method':'same-hour direct multi-horizon XGBoost; frozen midnight hyperparameters; daily
refit using only observed labels',
       'new_intraday_hyperparameters_selected_on_test':False,
       'minimum_price_node_all_versions_including_known_left':float(finite_nodes.min()),
       'nonpositive_price_nodes':int((finite_nodes<=0).sum()),'negative_prices_clipped':False,
       'estimated_actual_intervals':int(arrays['actual_estimated'].sum()),
       'missing_actual_right_node':'2026-01-01 00:10; default hold; still flagged estimated',
       'final_boundary_alternatives':boundary,
       'midnight_bridge':'mean of currently known D00:00 actual and inherited prediction D00:10',
       'startup_actual_interval_price':float(arrays['actual_bridge'][0]),
       'intraday_fits':1002,'fit_seconds':sum(d['fit_seconds'] for ds in fit_diagnostics.values() for d
in ds),
       'forecast_diagnostic_comparisons':comparison,
       'diagnostic_comparison_used_to_retune':False,
       'scope_limit':'Intraday same-hour forecasters are an explicit no-retuning extension; midnight
validation did not validate these new issue-hour designs.'}
dates=pd.date_range('2025-02-01',periods=334).strftime('%Y-%m-%d').to_numpy(dtype='U10')
temp=data_dir/'price_inputs.pending.npz'
np.savez_compressed(temp,dates=dates,**arrays,audit_json=json.dumps(audit,ensure_ascii=False))
temp.replace(output)
(data_dir/'price_inputs_audit.json').write_text(json.dumps(audit,ensure_ascii=False,indent=2))

(data_dir/'intraday_price_fit_diagnostics.json').write_text(json.dumps(fit_diagnostics,ensure_ascii=False,indent=2))
return load_price_data(output)

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--data-dir',type=Path,default=ROOT/'data');p.add_argument('--workers',type=int,default=3);p.add_argument('--force',action='store_true')
    a=p.parse_args()
    if a.workers<1:p.error('workers must be positive')
    data=build_price_inputs(a.data_dir,a.workers,a.force)
    print(json.dumps(data.audit,ensure_ascii=False,indent=2))

```

69. question4/code/q4_analysis.py

源文件 SHA-256: 17ab83396b8545dfa79f4b5d02e12521b93da82f2584652c31f6ca4afc8074f5 ; 代码行数: 393。

"""Frozen-choice Q4 comparisons, dependent-data intervals and tail sensitivity.

No original contract is changed. The endpoint experiment only re-executes the last step from its stored initial state and then re-settles that step. Actual endpoint prices are never inputs to the control optimization.

API: build_analysis(data_dir, results_dir, outdir=None).

CLI: python code/q4_analysis.py --data data --results results/final

"""

from __future__ import annotations

import argparse

import hashlib

import itertools

import json

from pathlib import Path

import numpy as np

import pandas as pd

from q4_io import load_inputs

from q4_optimizer import solve_fixed_control

from q4_audit import audit_selection, independent_signature

ROOT=Path(__file__).resolve().parents[1]

METHODS=('hold', 'linear2', 'ols6')

TEST_START='2025-02-25'

TEST_END='2025-12-31'

SEED=20260912

def ensure(condition, message):

if not condition: raise ValueError(message)

def near(a, b, name, tol=1e-5):

error=float(np.max(np.abs(np.asarray(a, float)-np.asarray(b, float))))

ensure(np.isfinite(error) and error≤tol, f'{name}: difference {error}')

return error

def sha(path): return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def json_write(path, value):

Path(path).write_text(json.dumps(value, ensure_ascii=False, indent=2, allow_nan=False), encoding='utf-8')

def moving_block_mean_interval(differences, block_length, replicates=2000, seed=SEED):

"""Percentile CI for the mean of paired daily differences.

Overlapping, non-circular blocks preserve dependence within each sampled

block. The final drawn block is truncated to keep exactly N observations.

This is a confidence interval for mean difference, not a prediction interval.

"""

values=np.asarray(differences, float)

ensure(values.ndim==1 and len(values)>1 and np.isfinite(values).all(), 'Need finite one-dimensional paired differences')

ensure(isinstance(block_length, int) and 1≤block_length≤len(values), 'Invalid block length')

ensure(isinstance(replicates, int) and replicates≥2, 'Invalid bootstrap replicate count')

n=len(values); nblocks=(n+block_length-1)//block_length

rng=np.random.default_rng(seed)

starts=rng.integers(0, n-block_length+1, size=(replicates, nblocks))

```

indices=(starts[:, :, None]+np.arange(block_length)[None, None, :]).reshape(replicates, -1)[ :, :n]
means=values[indices].mean(axis=1)
lo, hi=np.quantile(means, [.025, .975])
return {'observed_days':n, 'mean_daily_difference_yuan':float(values.mean()),
        'total_difference_yuan':float(values.sum()), 'block_days':block_length,
        'bootstrap_replicates':replicates, 'seed':seed, 'confidence_level':.95,
        'mean_difference_ci_lower_yuan':float(lo), 'mean_difference_ci_upper_yuan':float(hi),
        'ci_excludes_zero':bool(lo>0 or hi<0), 'method':'overlapping non-circular moving-block percentile
bootstrap',
        'simultaneous_coverage_claimed':False, 'multiplicity_adjustment':False,
        'interpretation':'CI for mean paired daily cost difference; not a prediction interval or proof
of optimality'}

def paired_daily_difference(left, right, column='cash_cost_yuan'):
    """Require exact dates instead of silently dropping unmatched days."""
    expected=pd.date_range(TEST_START, TEST_END).strftime('%Y-%m-%d').tolist()
    a=left[left['window_date'].between(TEST_START, TEST_END)].copy()
    b=right[right['window_date'].between(TEST_START, TEST_END)].copy()
    ensure(a['window_date'].tolist()==expected and b['window_date'].tolist()==expected,
           'Paired test comparison requires all 310 dates in identical chronological order')
    return a[column].to_numpy(float)-b[column].to_numpy(float), expected

def execute_tail(base_row, load_kw, pv_kw, *, terminal_weight, controller='mpc'):
    """Re-execute one final step; the price argument is the stored forecast."""
    q=float(base_row['q_kwh']); state=float(base_row['soc_start_kwh'])
    load=float(load_kw)/6; pv=float(pv_kw)/6
    price=float(base_row['price_forecast_yuan_per_kwh'])
    ensure(all(np.isfinite(v) for v in (q, state, load, pv, price, terminal_weight)), 'Nonfinite tail input')
    ensure(q≥0 and load≥0 and pv≥0 and price>0 and 1200-1e-5≤state≤10800+1e-5, 'Invalid tail
physical input')
    surplus=max(q+pv-load, 0); deficit=max(load-q-pv, 0)
    if surplus>1e-9:
        c=min(surplus, 5000/6, max((10800-state)/.9, 0.)); d=e=0.
        method='same surplus direct rule'
    elif controller=='greedy':
        c=0.; d=min(deficit, 5000/6, max((state-1200)*.9, 0.)); e=max(deficit-d, 0.)
        method='same greedy rule'
    else:
        ensure(controller=='mpc', 'Unknown tail controller')
        # Only normalize floating-point boundary noise, within the same 1e-5
        # kWh physical tolerance; stored SOC remains the recursion origin.
        sol=solve_fixed_control([load], [pv], [price],
[q], float(np.clip(state, 1200, 10800)), terminal_weight=terminal_weight)
        c=float(sol['c'][0]); d=float(sol['d'][0]); e=float(sol['e'][0]); method='same one-step fixed-
contract LP'
        end=state+.9*c-d/.9
        waste=max(q+pv+d+e-load-c, 0.); v=min(q, waste); r=max(waste-v, 0.)
        near(q-v+pv-r+d+e, load+c, 'tail energy conservation')
        ensure(1200-1e-5≤end≤10800+1e-5, 'tail SOC violation')
        ensure(max(c, d)≤5000/6+1e-5 and min(c, d)≤1e-5 and min(c, e)≤1e-5, 'tail physical exclusivity
violation')
        return {'charge_kwh':c, 'discharge_kwh':d, 'emergency_kwh':e, 'soc_end_kwh':end,

'unused_paid_kwh':v, 'curtailed_pv_kwh':r, 'control_price_yuan_per_kwh':price, 'control_method':method}

def settle_tail(base_row, execution, actual_price):
    """Delivery-price accounting only; cannot change execution."""
    p=float(actual_price)
    ensure(np.isfinite(p) and p>0, 'Tail price must remain in the declared positive-price domain')
    g=float(base_row['g_kwh']); q=float(base_row['q_kwh'])
    return p*(g+1.5*max(q-g, 0)-.5*max(g-q, 0))+5*execution['emergency_kwh'])

```



```

def endpoint_variants(strategy, summary, base_row, power_candidates, price_candidates):
    """3 power endpoint methods × 3 price methods; no full-run reoptimization."""
    rows=[]
    nu=float(summary['common_inventory_value'])
    for power_method in METHODS:
        L=float(power_candidates['load'][power_method]['mean_kw'])
        G=float(power_candidates['pv'][power_method]['mean_kw'])
        execution=execute_tail(base_row,L,G,terminal_weight=summary['terminal_weight'],
                               controller=summary['config']['controller'])
        if power_method=='hold':
            for field in ('charge_kwh','discharge_kwh','emergency_kwh','soc_end_kwh'):
                near(execution[field],float(base_row[field]),'unchanged hold-tail '+field)
        for price_method in METHODS:
            price=float(price_candidates[price_method]['last_interval_price'])
            tail_cash=settle_tail(base_row,execution,price)
            total=float(summary['cash_cost_yuan'])-float(base_row['cash_cost_yuan'])+tail_cash
            inventory=total-nu*(execution['soc_end_kwh']-summary['soc_start_kwh'])
            row=
{'strategy':strategy,'power_endpoint_method':power_method,'price_endpoint_method':price_method,
 'interval_start':base_row['interval_start'],'interval_end':base_row['interval_end'],
 'load_actual_mean_kw':L,'pv_actual_mean_kw':G,'actual_price_yuan_per_kwh':price,
 'g_kwh':float(base_row['g_kwh']),'q_kwh':float(base_row['q_kwh']),
 'soc_start_kwh':float(base_row['soc_start_kwh']),**execution,
 'tail_cash_cost_yuan':tail_cash,'full_cash_cost_yuan':total,
 'full_cash_change_yuan':total-summary['cash_cost_yuan'],
 'final_soc_change_kwh':execution['soc_end_kwh']-summary['soc_end_kwh'],
 'full_inventory_adjusted_cost_yuan':inventory,
 'inventory_score_change_yuan':inventory-summary['inventory_adjusted_cost_yuan'],
 'estimated_power_right_endpoint':True,'estimated_price_right_endpoint':True,
 'all_earlier_actions_unchanged':True,'last_g_q_and_initial_soc_fixed':True,
 'actual_price_used_for_control':False}
            if power_method==price_method=='hold':near(row['full_cash_change_yuan'],0.,'hold/hold
reproduces cash',1e-4)
            rows.append(row)
    return rows

def completed_stage(results_dir, stage, data_dir):
    directory=Path(results_dir)/stage;design_path=directory/'design.json'
    ensure(design_path.exists(),f'{stage} design missing; wait for declared experiment')
    jobs=json.loads(design_path.read_text())['jobs']
    ensure(jobs and all(job.get('name') for job in jobs),f'{stage}: missing named jobs')
    names=[job['name'] for job in jobs];ensure(len(names)==len(set(names)),f'{stage}: duplicate jobs')
    runs={}
    for name in names:
        path=directory/name;summary_path=path/'summary.json'
        ensure(summary_path.exists(),f'Incomplete {stage}: {name}')
        summary=json.loads(summary_path.read_text());ensure(summary.get('complete'),f'Incomplete
{stage}: {name}')
        ensure(summary['run_signature']==independent_signature(data_dir,summary['config']),f'{name}:
changed input/code signature')
        daily=pd.read_csv(path/'daily.csv')
        ensure(len(daily)==summary['windows'] and not daily['window_date'].duplicated().any(),f'{name}:
incomplete daily series')
        near(float(daily['cash_cost_yuan'].sum()),summary['cash_cost_yuan'],f'{name}: daily cash
sum',1e-4)
        near(float(daily['inventory_adjusted_cost_yuan'].sum()),summary['inventory_adjusted_cost_yuan'],f'{name}:
inventory telescoping',1e-4)
        events=json.loads((path/'event_log.json').read_text())['events']
        counts=[len(event['history_indices']) for event in events]
        ensure(counts and all(count==event['history_days'] for count,event in
zip(counts,events)),f'{name}: missing/inconsistent actual history counts')
        ensure(max(counts)≤summary['config']['history_days'],f'{name}: history exceeds cap')
        runs[name]={ 'path':path, 'summary':summary, 'daily':daily,
                    'actual_history_min':min(counts), 'actual_history_max':max(counts)}

```

```

return runs

def main_summary(runs,selection):
    rows=[]
    selected={selection['selected_q42'],selection['selected_q43']}
    chosen=selection['branch_selections']['4-3'];mode=chosen['price_mode']
    other='joint' if mode=='point' else 'point'
    schedules=[(0,)+part for n in range(4) for part in itertools.combinations((6,12,18),n)]
    expected_names={'Q42_point_H0','Q42_joint_H0','Q42_matched_Q43_PV',
        'Q43_'+other+'_H'+'_'.join(map(str,chosen['trade_hours']))}
    expected_names.update('Q43_'+mode+'_H'+'_'.join(map(str,hours)) for hours in schedules)
    ensure(set(runs)==expected_names,'Main experiment must include all twelve declared runs and all
eight Q43 schedules')
    for name,run in runs.items():
        s=run['summary'];daily=run['daily'];cfg=s['config']
        ensure(s['windows']==334 and cfg['start_day']==0,f'{name}: main run must be complete 334-window
replay')
        expected=pd.date_range('2025-02-01',periods=334).strftime('%Y-%m-%d').tolist()
        ensure(daily['window_date'].tolist()==expected,f'{name}: main dates')
        test=daily[daily['window_date'].between(TEST_START,TEST_END)]
        ensure(len(test)==310,f'{name}: test dates')
        rows.append({'strategy':name,'branch':s['branch'],'selected_by_development':name in selected,
            'price_mode':cfg['price_mode'],'trade_hours':','.join(map(str,cfg['trade_hours']))},

'pv_hours':','.join(map(str,cfg['forecast_hours'])), 'price_hours':','.join(map(str,cfg['price_hours'])),
'pv_source':cfg['pv_source'], 'full_windows':334, 'test_windows':310,
'full_cash_yuan':s['cash_cost_yuan'], 'test_cash_yuan':float(test['cash_cost_yuan'].sum()),
'development_and_warmup_cash_yuan':float(daily.iloc[:24]['cash_cost_yuan'].sum()),
'full_inventory_score_yuan':s['inventory_adjusted_cost_yuan'],
'test_inventory_score_yuan':float(test['inventory_adjusted_cost_yuan'].sum()),
'test_initial_soc_kwh':float(test.iloc[0]['soc_start_kwh']),
'initial_soc_kwh':s['soc_start_kwh'], 'final_soc_kwh':s['soc_end_kwh'],
'emergency_kwh':s['emergency_kwh'], 'emergency_cost_yuan':s['emergency_cost_yuan'],

'original_plan_cost_yuan':s['original_plan_cost_yuan'], 'net_adjustment_cost_yuan':s['net_adjustment_cost
_yuan'],

'startup_cash_yuan':s['startup_cash_cost_yuan'], 'natural_period_cash_yuan':s['natural_period_cash_cost_y
uan'],
        'common_inventory_value':s['common_inventory_value'], 'solver_seconds':s['solver']
['seconds'],
        'solver_unproven_calls':s['solver']['unproven'], 'solver_max_gap':s['solver']['max_gap'])}
    return rows

def compare_pairs(runs,selection):
    schedule='_'.join(map(str,selection['branch_selections']['4-3']['trade_hours']))
    mode=selection['branch_selections']['4-3']['price_mode']
    pairs=[('4-2: 联合价格场景减点价格', 'Q42_joint_H0', 'Q42_point_H0'),
        ('4-3: 联合价格场景减点价格', f'Q43_joint_H{schedule}', f'Q43_point_H{schedule}'),
        ('4-3: 开发期选中组合减仅零点', selection['selected_q43'], f'Q43_{mode}_H0'),
        ('同光伏信息: 4-3减固定普通计划', selection['selected_q43'], 'Q42_matched_Q43_PV'),
        ('主策略总对照: 4-3减4-2 (光伏信息也不同)', selection['selected_q43'], selection['selected_q42'])]
    differences=[];intervals=[];comparisons=[]
    for label,left,right in pairs:
        ensure(left in runs and right in runs,f'Missing paired main comparison: {left}, {right}')
        a=runs[left];b=runs[right]
        full_cash=a['summary']['cash_cost_yuan']-b['summary']['cash_cost_yuan']
        test_cash,dates=paired_daily_difference(a['daily'],b['daily'])
        comparison={'comparison':label,'left_strategy':left,'right_strategy':right,
            'sign':'left minus right; negative means left costs
less', 'full_cash_difference_yuan':full_cash,

'test_cash_difference_yuan':float(test_cash.sum()), 'mean_test_daily_cash_difference_yuan':float(test_cas
h.mean()),

```

```

        'full_cash_change_percent':100*full_cash/b['summary']['cash_cost_yuan'],
        'reoptimized_original_contracts':True, 'test_initial_states_may_differ':True,
        'not_proof_of_global_optimality':True}
    comparisons.append(comparison)
    for metric in ('cash_cost_yuan', 'inventory_adjusted_cost_yuan'):
        diff, dates=paired_daily_difference(a['daily'], b['daily'], metric)
        for date, value in zip(dates, diff):
            differences.append({'comparison':label, 'left_strategy':left, 'right_strategy':right,
                               'metric':metric, 'window_date':date, 'paired_difference_yuan':float(value)})
        for block in (3, 7, 14):
            intervals.append({'comparison':label, 'left_strategy':left, 'right_strategy':right, 'metric':metric,
                             **moving_block_mean_interval(diff, block, replicates=2000, seed=SEED)})
    return comparisons, differences, intervals

def sensitivity_summary(runs):
    rows=[]
    for name, run in runs.items():
        s=run['summary']; cfg=s['config']; prefix='Q'+s['branch'].replace('-', '')
        reference=prefix+'_reference'
        if cfg['price_mode'] in ('mean', 'shuffled') and prefix+'_joint_reference' in runs:
            reference=prefix+'_joint_reference'
        ensure(reference in runs, f'Missing independent 14-day sensitivity reference: {reference}')
        ref=runs[reference]['summary']
        ensure(s['windows']==ref['windows']==14 and cfg['start_day']==ref['config']['start_day']==24,
              'Sensitivity comparisons must have the same independent 14-day horizon')
        rows.append({'strategy':name, 'reference_strategy':reference, 'branch':s['branch'],
                    'start_date':'2025-02-25', 'last_contract_date':'2025-03-10', 'windows':14,
                    'cash_yuan':s['cash_cost_yuan'], 'cash_change_yuan':s['cash_cost_yuan']-
ref['cash_cost_yuan'],
                    'inventory_score_yuan':s['inventory_adjusted_cost_yuan'],
                    'inventory_score_change_yuan':s['inventory_adjusted_cost_yuan']-
ref['inventory_adjusted_cost_yuan'],
                    'final_soc_kwh':s['soc_end_kwh'], 'initial_soc_kwh':s['soc_start_kwh'],

                    'emergency_kwh':s['emergency_kwh'], 'price_mode':cfg['price_mode'], 'controller':cfg['controller'],
                    'history_days':cfg['history_days'], 'scenarios':cfg['scenarios'], 'seed':cfg['seed'],

                    'actual_history_paths_min':run['actual_history_min'], 'actual_history_paths_max':run['actual_history_max']
                    ],
                    'history_cap_filled_at_all_events':run['actual_history_min']==cfg['history_days'],

                    'terminal_multiplier':cfg['terminal_multiplier'], 'price_hours':', '.join(map(str, cfg['price_hours'])),
                    'scope':'14 independent continuous windows; not a 334-window sensitivity result'})
    return rows

def history_check_summary(runs):
    rows=[]
    for branch in ('4-2', '4-3'):
        prefix='Q'+branch.replace('-', ''); reference=prefix+'_aug_history28'
        ensure(reference in runs, 'Missing mature-history 28-cap reference')
        ref=runs[reference]['summary']
        for cap in (14, 28, 56):
            name=prefix+f'_aug_history{cap}'
            ensure(name in runs, 'Missing declared mature-history candidate')
            run=runs[name]; s=run['summary']; cfg=s['config']
            ensure(s['windows']==14 and cfg['start_day']==181 and cfg['history_days']==cap,
                  'Mature-history check must use Aug1..14 and declared cap')
            ensure(s['soc_start_kwh']==6000, 'Mature-history checks require common independent 6000
initial state')
            ensure(run['actual_history_min']==run['actual_history_max']==cap, 'Mature-history experiment
did not actually fill its cap')
            rows.append({'strategy':name, 'branch':branch, 'reference_strategy':reference,
                        'start_date':'2025-08-01', 'last_contract_date':'2025-08-14', 'windows':14,

```

```

        'history_cap_days':cap, 'actual_history_paths_min':run['actual_history_min'],
        'actual_history_paths_max':run['actual_history_max'], 'all_events_fill_history_cap':True,
        'cash_yuan':s['cash_cost_yuan'], 'cash_change_from_28_yuan':s['cash_cost_yuan']-
ref['cash_cost_yuan'],
        'inventory_score_yuan':s['inventory_adjusted_cost_yuan'],
        'inventory_score_change_from_28_yuan':s['inventory_adjusted_cost_yuan']-
ref['inventory_adjusted_cost_yuan'],
        'initial_soc_kwh':s['soc_start_kwh'], 'final_soc_kwh':s['soc_end_kwh'],
        'used_for_reselection':False, 'purpose':'Supplement the early sensitivity where a 56-
history cap was not actually filled'})
    ensure(len(runs)==6, 'Mature-history stage must have exactly six declared runs')
    return rows

def render_markdown(result):
    selected={result['selection']['selected_q42'],result['selection']['selected_q43']}
    lines=['# 第四问规划结果与比较分析','',
        '主策略按开发期固定选择；下列测试比较不会更改这一选择。334 个合同窗口覆盖 2 月 1 日 00:10 至次年 1 月 1 日
00:10, 310 个测试窗口从 2 月 25 日开始。库存校正评价指标, 不是实际电费。','',
        '| 策略 | 开发期选中 | 334窗现金/元 | 310测试窗现金/元 | 310测试窗库存校正/元 | 期末SOC/kWh |',
        '|-----:|-----:|-----:|-----:|']
    for row in result['main_summary']:
        lines.append(f"| {row['strategy']} | {'是' if row['strategy'] in selected else ''} |
{row['full_cash_yuan']:, .2f} | {row['test_cash_yuan']:, .2f} | {row['test_inventory_score_yuan']:, .2f} |
{row['final_soc_kwh']:, .3f} |")
        lines += ['', '所有成对差值按“左策略减右策略”计算，负值表示左策略费用较低。各完整策略会重新制定原单，测试起点 SOC
也可能因前期连续运行而不同，因此不把这些结果写成严格同合同、同状态的局部因果收益。','',
        '| 对比 | 334窗现金差/元 | 310测试窗现金差/元 |', '|-----:|-----:|']
    for row in result['comparisons']:
        lines.append(f"| {row['comparison']} | {row['full_cash_difference_yuan']:, .2f} |
{row['test_cash_difference_yuan']:, .2f} |")
        lines += ['', '对配对的日费用差采用 3、7、14 日移动块重采样，每种 2000 次，固定种子 20260912。下表列 7 日块的现
金差结果，完整三种块长及库存校正结果在 CSV/JSON 中。置信区间描述平均日费用差的抽样不确定性，不是未来某一天费用的预测区
间，也不证明全局最优。','',
        '| 对比 | 平均日现金差/元 | 95%置信区间/元 |', '|-----:|-----:|']
    for row in result['bootstrap_intervals']:
        if row['metric']=='cash_cost_yuan' and row['block_days']==7:
            lines.append(f"| {row['comparison']} | {row['mean_daily_difference_yuan']:, .3f} |
[{row['mean_difference_ci_lower_yuan']:, .3f}, {row['mean_difference_ci_upper_yuan']:, .3f}] |")
            lines += ['', '移动块使用连续等长片段且不跨样本首尾拼接，概念见 [arch 官方文档]
(https://arch.readthedocs.io/en/latest/bootstrap/generated/arch.bootstrap.MovingBlockBootstrap.html)。本
项目用 NumPy 直接生成相同类型的重采样索引，不依赖 arch 包。跨块相关性会被切断；季节变化、单年样本、跨日 SOC 依赖及仅 7
个开发窗口的选择不确定性，都会限制上述区间的解释。更换块长用于显示这种敏感性，不从三种块长挑最有利的一种。这些是各项比较自
身的区间，未作多重比较校正，不声称所有区间同时具有95%覆盖率。','',
        '4-2 与 4-3 主输入的光伏信息不同；“主策略总对照”同时包含预测信息和合同机制变化。“同光伏信息”对照可以减少这种混
淆，但双方仍各自优化原单和连续运行，不能声称仅改变一个已签合同的许可。','',
        '末段边界试验固定全部历史操作、最后 g/q 及入段 SOC，功率右端点按持平、两点线性、六点回归三种方法分别重新执行最
后一步，再组合三种电价右端点估计进行结算。实际均价只进账本；原控制价格保持不变。九种组合均属于含估计的敏感性，不被标为实
测。','',
        '| 策略 | 九组合现金最大减小/元 | 相对基准最大绝对现金变化/元 | 期末SOC范围/kWh |',
        '|-----:|-----:|-----:|']
    for row in result['endpoint_summary']:
        lines.append(f"| {row['strategy']} | {row['cash_span_yuan']:, .6f} |
{row['max_absolute_cash_change_yuan']:, .6f} | [{row['final_soc_min_kwh']:, .6f},
{row['final_soc_max_kwh']:, .6f}] |")
        lines += ['', '其余参数敏感性为相同起点日期、相同独立 14 窗时域的试验，不能解释为 334 窗全年敏感性。不同初态使用库
存校正辅助比较；库存单价在所有策略中固定，不随终端控制权重改变。','',
        '历史窗参数指“最多使用多少个已完成残差窗口”。2月25日至3月10日的早期试验尚没有56条完整历史，所以 history56 只
检验上限56，不能称为实际使用56窗的效果。参数表附有每个正式决策事件的实际历史条数最小/最大值。为核实填满历史后的行为，另外
从8月1日6000 kWh共同实验初态运行到8月14日，比较14/28/56上限；这6条是针对早期历史不足的补充验证，不改变原冻结选
择。','',
        '| 分支 | 历史上限 | 实际历史条数范围 | 14窗现金/元 | 相对28上限现金差/元 | 库存校正差/元 |',
        '|-----:|-----:|-----:|-----:|']
    for row in result['history_check_summary']:
        lines.append(f"| {row['branch']} | {row['history_cap_days']} |
{row['actual_history_paths_min']}-{row['actual_history_paths_max']} | {row['cash_yuan']:, .2f} |

```

```

{row['cash_change_from_28_yuan'],.2f} | {row['inventory_score_change_from_28_yuan'],.2f} |")
    lines += [' ',
        '本报告未重新训练负载/光伏，未利用测试期费用修改 XGBoost 参数，也没有把随机情景优化目标当成兑现账单。正式预测生
成器的历史特征可用性，以及当前十分钟功率均值快反馈近似，仍须结合独立数据审计和严格因果执行对照解释。']
    return '\n'.join(lines)+'\n'

def build_analysis(data_dir=ROOT/'data',results_dir=ROOT/'results/final',outdir=None):
    data_dir=Path(data_dir).resolve();results_dir=Path(results_dir).resolve()
    outdir=Path(outdir).resolve() if outdir else results_dir/'analysis'
    selection=json.loads((results_dir/'selection.json').read_text())
    selection_check=audit_selection(results_dir)
    ensure(selection_check['present'] and selection_check['passed'],'Frozen development selection audit
failed')
    main=completed_stage(results_dir,'main',data_dir)
    sensitivity=completed_stage(results_dir,'sensitivity',data_dir)
    history_check=completed_stage(results_dir,'history_check',data_dir)
    rows=main_summary(main,selection)
    comparisons,differences,intervals=compare_pairs(main,selection)
    data=load_inputs(data_dir)
    endpoint=[]
    for name,run in main.items():
        trace=pd.read_csv(run['path']/'trace.csv')
        row=trace.iloc[-1].to_dict()
        ensure(row['interval_start']=='2026-01-01T00:00:00' and row['interval_end']=='2026-01-
01T00:10:00','Endpoint sensitivity requires final missing boundary')
        endpoint.extend(endpoint_variants(name,run['summary'],row,
            data['metadata']['actual']
['last_right_endpoint_candidates'],data['price_bundle'].boundary_alternatives))
        endpoint_summary=[]
        for name in main:
            part=[r for r in endpoint if r['strategy']==name]
            endpoint_summary.append({'strategy':name,'combinations':len(part),
                'cash_span_yuan':max(r['full_cash_cost_yuan'] for r in part)-min(r['full_cash_cost_yuan']
for r in part),
                'max_absolute_cash_change_yuan':max(abs(r['full_cash_change_yuan']) for r in part),
                'final_soc_min_kwh':min(r['soc_end_kwh'] for r in
part),'final_soc_max_kwh':max(r['soc_end_kwh'] for r in part)})
            result={'complete':True,'selection':selection,'selection_audit':selection_check,
                'main_summary':rows,'comparisons':comparisons,'bootstrap_intervals':intervals,
                'endpoint_combinations':endpoint,'endpoint_summary':endpoint_summary,
                'sensitivity_summary':sensitivity_summary(sensitivity),
                'history_check_summary':history_check_summary(history_check),
                'interpretation':{'period':'334 shifted contract windows, not 365 days; last interval includes
estimated endpoint',
                    'test':'310 windows Feb25..Dec31; no test-driven reselection',
                    'bootstrap':'paired daily test differences; overlapping non-circular blocks3/7/14,2000
replicates; confidence intervals for mean differences',
                    'boundary':'same existing last g/q and initial SOC; only last action re-executed with old
forecast price, then actual estimated price settled',
                    'scope':'conditional empirical comparisons, not globally optimal policy proof'},
                'provenance':{'selection_sha256':sha(results_dir/'selection.json'),
                    'analysis_code_sha256':sha(Path(__file__)),
                    'main_signatures':{'name:run['summary']['run_signature'] for name,run in main.items()},
                    'sensitivity_signatures':{'name:run['summary']['run_signature'] for name,run in
sensitivity.items()},
                    'history_check_signatures':{'name:run['summary']['run_signature'] for name,run in
history_check.items()}}}
            outdir.mkdir(parents=True,exist_ok=True)
            for filename,records in [(主实验比较.csv',rows),('成对费用比较.csv',comparisons),('配对逐日差
值.csv',differences),
                ('移动块置信区间.csv',intervals),('末端九组合敏感性.csv',endpoint),('末端敏感性汇
总.csv',endpoint_summary),
                ('参数敏感性比较.csv',result['sensitivity_summary']),('八月完整历史窗验
证.csv',result['history_check_summary'])]:
                pd.DataFrame(records).to_csv(outdir/filename,index=False,encoding='utf-8-sig')

```



```
json_write(outdir/'analysis.json',result)
(outdir/'结果与比较分析.md').write_text(render_markdown(result),encoding='utf-8')
return result

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,default=R00T/'data')
    p.add_argument('--results',type=Path,default=R00T/'results/final');p.add_argument('--out',type=Path)
    a=p.parse_args();r=build_analysis(a.data,a.results,a.out)
    print(json.dumps({'complete':r['complete'],'main_runs':len(r['main_summary']),
    'sensitivity_runs':len(r['sensitivity_summary']),'endpoint_cases':len(r['endpoint_combinations'])},ensure_ascii=False))
```

70. question4/code/q4_audit.py

源文件 SHA-256: 0a897fb7af74a13f033f99b12b1d2df6a916d53138d3d03ca33bc105af2d7612 ; 代码行数: 391。

```

"""Read-only, independent trace/event/summary audit for completed Q4 runs.

Usage: python code/q4_audit.py --data data --results results/final
API: audit_all(data_dir, results_dir) → JSON-serializable dict.
This module does not call an optimizer or use its cash/physical check helpers.
"""

from __future__ import annotations

import argparse
import datetime as dt
import hashlib
import importlib.metadata
import json
from pathlib import Path

import numpy as np
import pandas as pd
from openpyxl import load_workbook

ROOT=Path(__file__).resolve().parents[1]
CORE=(
    'physical_io.py', 'q4_io.py', 'q4_optimizer.py', 'q4_risk.py', 'q4_runner.py', 'price_inputs.py', 'xgb_intrad
ay.py')
START={0:0,6:35,12:71,18:107}
HOURS=(0,6,12,18)
TOL=1e-5

def require(condition,message):
    if not condition:raise AssertionError(message)

def close(actual,expected,name,atol=TOL):
    a,b=np.asarray(actual,float),np.asarray(expected,float)
    require(a.shape==b.shape,f'{name}: shape {a.shape} ≠ {b.shape}')
    require(np.isfinite(a).all() and np.isfinite(b).all(),f'{name}: nonfinite')
    err=float(np.max(np.abs(a-b))) if a.size else 0.
    require(err≤atol,f'{name}: maximum difference {err} > {atol}')
    return err

def display(path):
    p=Path(path).resolve()
    try:return str(p.relative_to(ROOT))
    except ValueError:return str(p)

def file_sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()

def independent_signature(data_dir,config):
    h=hashlib.sha256(json.dumps(config,sort_keys=True).encode())
    for name in CORE:
        p=ROOT/'code'/name
        if p.exists():h.update(name.encode());h.update(p.read_bytes())
    for p in sorted(Path(data_dir).glob('*')):
        if p.is_file() and p.suffix in ('.xlsx','.npz'):
            h.update(p.name.encode());h.update(p.read_bytes())
    for name in ('numpy','scipy','pandas','openpyxl'):
        h.update((name+importlib.metadata.version(name)).encode())
    return h.hexdigest()

def sheet_values(path,sheet=None):

```



```

wb=load_workbook(path,read_only=True,data_only=True)
try:return list((wb[sheet] if sheet else wb.active).values)
finally:wb.close()

def source_data(data_dir):
    """Read original values and rederive means without the production adapter."""
    data_dir=Path(data_dir)
    dates=pd.date_range('2025-02-01',periods=334)
    out={'dates':dates}
    for key,sheet in (('load','小区负载'),('pv','光伏发电实际功率'),('price',None)):
        rows=sheet_values(data_dir/('attachment4.xlsx' if key=='price' else 'attachment2.xlsx'),sheet)
        all_dates=pd.DatetimeIndex([r[0] for r in rows[1:]])
        require(all_dates.equals(pd.date_range('2025-01-01',periods=365)),f'{key}: original dates')
        raw=np.asarray([r[1:] for r in rows[1:]],float)
        require(raw.shape==(365,144) and np.isfinite(raw).all(),f'{key}: original nodes')
        nodes=np.column_stack((raw[31:],np.r_[raw[32:,0],raw[-1,-1]]))
        out[key+'_nodes']=nodes
        out[key+'_actual']=(nodes[:,:-1]+nodes[:,1:])/2
        out[key+'_startup']=(raw[30,143]+raw[31,0])/2
        if key=='pv':out['pv_raw']=raw
        if key=='price':out['price_raw']=raw
    with np.load(data_dir/'price_inputs.npz',allow_pickle=False) as z:
        out['price_forecast']=z['predicted'].copy();out['price_nodes_f']=z['nodes'].copy()
        close(z['actual'],out['price_actual'],'price bundle actual means',1e-12)
        close(z['actual_nodes'],out['price_nodes'],'price bundle actual nodes',1e-12)
        out['price_audit']=json.loads(str(z['audit_json'].item()))
    p=out['price_nodes_f']
    for v,h in enumerate(HOURS):
        close(out['price_forecast'][:,v,START[h:]],
              ((p[:,:-1]+p[:,1:])/2)[:,v,START[h:]],'forecast endpoint average',1e-12)
        require(np.isnan(out['price_forecast'][:,v,START[h:]]).all(),'unreleased prices available')
    with np.load(data_dir/'xgboost_midnight_predictions.npz',allow_pickle=False) as z:
        close(p[:,0],z['predictions'],'unchanged XGBoost midnight nodes',0)
        close(z['actual'][:,0:144],out['price_raw'][31:], 'original XGBoost source truth',0)
        require(out['price_audit'].get('complete') is True,'incomplete price bundle')
    for name,digest in out['price_audit']['provenance']['sources'].items():
        require(file_sha(data_dir/name)==digest,f'price bundle provenance changed: {name}')
    raw=out['price_raw'].ravel()
    times=pd.date_range('2025-01-01 00:10',periods=len(raw),freq='10min')
    mask=(times[:-1].month==1)&(times[:-1].hour<6)&(times[1:]<pd.Timestamp('2025-02-01'))
    out['nu']=float(np.median((raw[:-1]+raw[1:])[mask])/2)/.9)
    out['nu_intervals']=int(mask.sum())
    out['bridge_price']=(out['price_raw'][30:364,143]+p[:,0,0])/2
    for key,sheet in (('load_f','XGBoost_负载'),('q2_pv','自适应加权_光伏')):
        rows=sheet_values(data_dir/'load_forecasts.xlsx',sheet)
        require(pd.DatetimeIndex([r[0] for r in rows[1:]]).equals(dates),f'{key}: dates')
        out[key]=np.asarray([r[1:] for r in rows[1:]],float)
        require(out[key].shape==(334,144),f'{key}: mean grid')
    pv=np.full((334,4,144),np.nan)
    for v,h in enumerate(HOURS):
        rows=sheet_values(data_dir/'pv_forecasts.xlsx',f'{h:02d}时预报')
        require(pd.DatetimeIndex([r[0] for r in rows[1:]]).equals(dates),'PV dates')
        values=np.asarray([r[1:] for r in rows[1:]],float)
        if h==0:out['pv_startup_forecast']=values[:,0].copy();values=values[:,1:]
        pv[:,v,START[h:]]=values
    out['pv_f']=pv
    rows=sheet_values(data_dir/'attachment3.xlsx');hourly={};date=None
    for row in rows[1:]:
        if row[0]:date=pd.Timestamp(row[0]).date()
        hour=row[1].hour if hasattr(row[1],'hour') else int(str(row[1]).split(':')[0])
        hourly[date,hour]=np.asarray(row[2:],float)
    linear=np.full_like(pv,np.nan)
    for i,date in enumerate(dates):
        for v,h in enumerate(HOURS):
            observed=out['pv_raw'][i+30,143] if h==0 else out['pv_raw'][i+31,h*6-1]

```

```

        knots=np.r_[observed,hourly[date.date(),h]]
        t=np.arange(max(10,h*60),1451,10)
        nodes=np.interp(t,np.arange(h,h+25)*60,knots)
        linear[i,v,START[h,:]]=(nodes[:-1]+nodes[1:])/2
    out['pv_linear']=linear
    out['source_sha256']={p.name:file_sha(p) for p in data_dir.iterdir() if p.is_file()}
    return out

def as_bool(series):
    mapped=series.astype(str).str.lower().map({'true':True,'false':False,'1':True,'0':False})
    require(mapped.notna().all(), 'invalid Boolean provenance flag')
    return np.asarray(mapped,bool)

def audit_run(run_dir,data_dir,source=None,verify_signature=True):
    run_dir=Path(run_dir).resolve();data_dir=Path(data_dir).resolve()
    src=source if source is not None else source_data(data_dir)
    summary=json.loads((run_dir/'summary.json').read_text())
    trace=pd.read_csv(run_dir/'trace.csv')
    daily=pd.read_csv(run_dir/'daily.csv')
    eventdoc=json.loads((run_dir/'event_log.json').read_text());events=eventdoc['events']
    cfg=summary['config'];start=cfg['start_day'];windows=summary['windows'];stop=start+windows
    require(summary.get('complete') is True, 'incomplete run')
    require(eventdoc['config']==cfg, 'event configuration disagrees')
    if verify_signature:
        require(summary['run_signature']==independent_signature(data_dir,cfg), 'source/code/run signature mismatch')
        require(0≤start<stop≤334, 'invalid run date range')
        require(windows==min(334,start+(cfg['days'] or 334))-start, 'wrong configured run length')
        require(len(trace)==144*windows==summary['intervals'], 'incomplete/duplicate trace')
        require(trace['strategy'].eq(summary['strategy']).all(), 'strategy names disagree')
        require(trace['branch'].eq(summary['branch']).all(), 'branch names disagree')
        begins=pd.date_range(src['dates'][start]+pd.Timedelta(minutes=10),periods=len(trace),freq='10min')
        require(pd.DatetimeIndex(trace['interval_start']).equals(begins), 'wrong interval start grid')
        require(pd.DatetimeIndex(trace['interval_end']).equals(begins+pd.Timedelta(minutes=10)), 'wrong end grid')
        days=np.repeat(np.arange(start,stop),144);slots=np.tile(np.arange(144),windows)
        require(trace['window_date'].tolist()==[str(src['dates'][i].date()) for i in days], 'contract assignment mismatch')
        for key,col in (('load','load_actual_kw'),('pv','pv_actual_kw'),('price','price_yuan_per_kwh')):
            close(trace[col],src[key+'_actual'][days,slots],f'actual {key} endpoint means')
            expected_est=(days==333)&(slots==143)
            require(np.array_equal(as_bool(trace['actual_estimated']),expected_est), 'power estimated boundary flag')
            require(np.array_equal(as_bool(trace['price_estimated']),expected_est), 'price estimated boundary flag')
            close(trace['load_forecast_kw'],src['load_f'][days,slots], 'supplied load means unchanged')
            pv_hours=cfg['forecast_hours'];price_hours=cfg['price_hours'];trade_hours=cfg['trade_hours']
            expected_fh=np.array([max(h for h in pv_hours if START[h]≤t) for t in slots])
            expected_ph=np.array([max(h for h in price_hours if START[h]≤t) for t in slots])
            bridge=(slots==143)&(days<stop-1)
            expected_ph[bridge]=0
            expected_price_issue=[src['dates'][d+int(b)]+pd.Timedelta(hours=int(h)) for d,b,h in
zip(days,bridge,expected_ph)]
            expected_pv_issue=[src['dates'][d]+pd.Timedelta(hours=int(h)) for d,h in zip(days,expected_fh)]

    require(pd.DatetimeIndex(trace['price_forecast_issue_time']).equals(pd.DatetimeIndex(expected_price_issue)), 'wrong price version timestamp')

    require(pd.DatetimeIndex(trace['forecast_issue_time']).equals(pd.DatetimeIndex(expected_pv_issue)), 'wrong PV version timestamp')
    require((pd.DatetimeIndex(expected_price_issue)≤begins).all(), 'unreleased price version used')
    close(trace['price_forecast_hour'],expected_ph, 'price clock permission',0)
    expected_pf=np.empty(len(trace));expected_pv=np.empty(len(trace))
    q2pv=cfg['branch']=='4-2' and cfg['pv_source']!='q3_all'

```

```

pv_source=src['pv_linear'] if cfg['pv_source']=='linear' else src['pv_f']
for k,(d,t,ph,fh,b) in enumerate(zip(days,slots,expected_ph,expected_fh,bridge)):
    expected_pf[k]=src['bridge_price'][d+1] if b else (src['price_nodes'][d,t]+src['price_nodes_f']
[d,HOURLS.index(ph),t+1])/2
    expected_pv[k]=src['q2_pv'][d,t] if q2pv else pv_source[d,HOURLS.index(fh),t]
    close(trace['price_forecast_yuan_per_kwh'],expected_pf,'known-left/predicted-right control
prices',1e-10)
    close(trace['pv_forecast_kw'],expected_pv,'permitted PV mean')
    g,q,c,d,e,v,r,S0,S1=[trace[col].to_numpy(float) for col in
('g_kwh','q_kwh','charge_kwh','discharge_kwh','emergency_kwh','unused_paid_kwh','curtailed_pv_kwh','soc_
start_kwh','soc_end_kwh')]
    L=trace['load_actual_kw'].to_numpy(float)/6;G=trace['pv_actual_kw'].to_numpy(float)/6
    p=trace['price_yuan_per_kwh'].to_numpy(float)
    require(np.isfinite(np.r_[g,q,c,d,e,v,r,S0,S1,p])).all(),'nonfinite physical values')
    require(min(np.min(x) for x in (g,q,c,d,e,v,r))>=-TOL,'negative physical flow')
    physical={}
    physical['balance']=close(q-v+G-r+d+e,L+c,'energy conservation')
    physical['soc_recursion']=close(S1,S0+.9*c-d/.9,'SOC recursion')
    physical['continuity']=close(S0[1:],S1[:-1],'continuous actual SOC')
    require(min(S0.min(),S1.min())>=1200-TOL and max(S0.max(),S1.max())<=10800+TOL,'SOC bounds')
    require(max(c.max(),d.max())<=5000/6+TOL,'power/rate limit')
    require(np.max(np.minimum(c,d))<=TOL,'simultaneous charge and discharge')
    require(np.max(np.minimum(c,e))<=TOL,'emergency charging')
    require(np.max(v-q)<=TOL and np.max(r-G)<=TOL,'discard bounds')
    A=np.maximum(q+G-L,0);D=np.maximum(L-q-G,0)
    require(np.max(c-A)<=TOL and np.max(d-D)<=TOL,'ordinary surplus/deficit control convention')
    close(e,D-d,'emergency deficit closure');close(v+r,A-c,'unused surplus closure')
    a=np.maximum(q-g,0);b=np.maximum(g-q,0)
    close(trace['increase_kwh'],a,'signed increase');close(trace['decrease_kwh'],b,'signed decrease')
    costs={'plan_cost_yuan':p*g,'increase_cost_yuan':1.5*p*a,'refund_yuan':p*b,
        'penalty_yuan':.5*p*b,'adjustment_net_cost_yuan':1.5*p*a-.5*p*b,
        'emergency_cost_yuan':5*p*e,'cash_cost_yuan':p*g+1.5*p*a-.5*p*b+5*p*e}
    for col,value in costs.items():close(trace[col],value,col,1e-6)
    close(summary['soc_start_kwh'],S0[0],'summary initial');close(S0[0],cfg['initial_soc'],'configured
initial')
    close(summary['soc_end_kwh'],S1[-1],'summary final')
    close(summary['common_inventory_value'],src['nu'],'January-frozen inventory value',1e-12)
    close(summary['terminal_weight'],src['nu']*cfg['terminal_multiplier'],'terminal weight',1e-12)
    for key,expected in {
'cash_cost_yuan':costs['cash_cost_yuan'].sum(),'original_plan_cost_yuan':costs['plan_cost_yuan'].sum(),
'net_adjustment_cost_yuan':costs['adjustment_net_cost_yuan'].sum(),'emergency_cost_yuan':costs['emergenc
y_cost_yuan'].sum(),
'emergency_kwh':e.sum(),'inventory_adjusted_cost_yuan':costs['cash_cost_yuan'].sum()-src['nu']*(
S1[-1]-S0[0])}.items():
        close(summary[key],expected,'summary '+key,1e-4)
        require(len(daily)==windows,'daily count')
        close(daily['cash_cost_yuan'],costs['cash_cost_yuan'].reshape(windows,144).sum(axis=1),'daily
cash',1e-5)
        close(daily['soc_start_kwh'],S0[::144],'daily
initial');close(daily['soc_end_kwh'],S1[143::144],'daily terminal')
        close(daily['emergency_kwh'],e.reshape(windows,144).sum(axis=1),'daily emergency')

close(daily['inventory_adjusted_cost_yuan'],costs['cash_cost_yuan'].reshape(windows,144).sum(axis=1)-
src['nu']*(S1[143::144]-S0[::144]),'daily stock correction',1e-5)
# Reconstruct actual signed q from immutable original g and event blocks.
require(len(events)==windows*len(trade_hours),'unexpected/missing transaction events')
expected_events=[(d,h) for d in range(start,stop) for h in trade_hours]
require([(ev['day_index'],pd.Timestamp(ev['decision_at']).hour) for ev in events]==expected_events,
'duplicate/missing/out-of-order formal transaction')
signed=np.full((windows,144),np.nan);original=np.full_like(signed,np.nan)
unproven=[]
for index,ev in enumerate(events):
    day=ev['day_index'];local=day-start

```

```

require(0≤local<windows,'event outside run')
decision=pd.Timestamp(ev['decision_at']);h=decision.hour
require(decision==src['dates'][day]+pd.Timedelta(hours=h),'event date/time')
require(h in trade_hours,'price/PV update improperly grants trade')
require(ev['objective_is_cash'] is False,'optimization objective mislabeled cash')
first,last=ev['commit_start'],ev['commit_end'];qty=np.asarray(ev['signed_q'],float)
require(qty.shape==(last-first,) and np.isfinite(qty).all(), 'bad signed quantity block')
if h==0:
    require(ev['kind']=='dayahead' and first==0 and last==144,'bad original order')
    require(np.isnan(original[local]).all(), 'duplicate original signing')
    original[local]=qty;signed[local]=qty
    require(ev['locked_bridge']==(local>0),'midnight bridge lock flag')
else:
    later=[START[j] for j in trade_hours if j>h]
    require(ev['kind']=='adjust' and first==START[h] and last==(min(later) if later else
144),'adjustment outside authorized block')
    require(np.isfinite(original[local]).all(), 'adjustment before original order')
    signed[local,first:last]=qty
    expected_pvh=max(j for j in pv_hours if j≤h)
    expected_priceh=max(j for j in price_hours if j≤h)
    require(ev['forecast_hour']==expected_pvh and ev['price_hour']==expected_priceh,'event
information permissions')
    require(ev['decision_hour']==h,'event history cutoff hour')
    history=[j for j in range(day) if src['dates'][j]+pd.Timedelta(days=1,minutes=10)≤decision][-
cfg['history_days']:]
    require(ev['history_indices']==history,'future/incorrect residual history')
    require(ev['history_days']==len(history),'history length')
    latest=None if not history else (src['dates']
[history[-1]]+pd.Timedelta(days=1,minutes=10)).isoformat()
    require(ev['history_latest_end']==latest,'history availability metadata')
    require(ev['price_mode']==cfg['price_mode'] and ev['margin_added'] is False,'wrong risk
variant')
    if cfg['scenarios']==1 or len(history)<cfg['min_history']:
        require(ev['scenarios']==1 and ev['draw_indices']==[], 'wrong point fallback')
    else:
draw=np.random.default_rng(cfg['seed']+1009*day+59*h).choice(history,min(cfg['scenarios'],len(history)),
replace=False).tolist()
    require(ev['draw_indices']==draw and ev['scenarios']==len(draw),'unpaired/reseeded scenario
sampling')
    price_version=HOURS.index(expected_priceh)
    point=src['price_forecast'][day,price_version,first:]
    past_pred=src['price_forecast'][draw,price_version,first:]
    past_actual=src['price_actual'][draw,first:]
    require((past_pred>0).all() and (past_actual>0).all(), 'invalid positive-price residual
domain')
    scen=point[None,:]*(past_actual/past_pred)
    if cfg['price_mode']=='point':scen=np.tile(point,(len(draw),1))
    elif cfg['price_mode']=='mean':scen=np.tile(scen.mean(axis=0),(len(draw),1))
    # Shuffling rows changes the pairing, but not global min/max.
    close(ev['scenario_price_min'],scen.min(), 'scenario minimum from same historical prices',1e-
10)
    close(ev['scenario_price_max'],scen.max(), 'scenario maximum from same historical prices',1e-
10)
    require(ev['max_violation_kwh']≤TOL,'solver event infeasibility')
    if
ev['status']≠'optimal_within_requested_gap':unproven.append({'event':index,'gap':ev['gap'],'decision_at
':ev['decision_at']})
    close(g,original.ravel(), 'immutable original plan from signing events')
    close(q,signed.ravel(), 'only formal adjustments change delivered contract')
    startup_cash=0.
    if start==0:
        sr=pd.read_csv(run_dir/'startup.csv');require(len(sr)==1,'missing startup convention')
        row=sr.iloc[0]
        require(row['interval_start']=='2025-02-01T00:00:00','startup interval')
        close(row['price_yuan_per_kwh'],src['price_startup'],'startup actual price',1e-12)

```

```

close(row['load_actual_kw'],src['load_startup'],'startup actual load')
close(row['pv_actual_kw'],src['pv_startup'],'startup actual PV')
pv_startup_forecast=src['q2_pv'][0,0] if q2pv else src['pv_startup_forecast'][0]
expected_g=max((src['load_f'][0,0]-pv_startup_forecast)/6,0.)
expected_e=max((src['load_startup']-src['pv_startup'])/6-expected_g,0.)
close(row['g_kwh'],expected_g,'startup causal original quantity')
close(row['q_kwh'],expected_g,'startup no adjustment')
close(row['emergency_kwh'],expected_e,'startup emergency')
# Startup policy is battery idle; its supply is recorded separately.
close(row['charge_kwh'],0.,'startup idle charge');close(row['discharge_kwh'],0.,'startup idle
discharge')
close(row['soc_start_kwh'],cfg['initial_soc'],'startup
SOC');close(row['soc_end_kwh'],cfg['initial_soc'],'startup SOC unchanged')
startup_cash=float(row['price_yuan_per_kwh']*(row['g_kwh']+5*row['emergency_kwh']))
close(row['cash_cost_yuan'],startup_cash,'startup cash')
close(summary['startup_cash_cost_yuan'],startup_cash,'startup total')
natural=costs['cash_cost_yuan'][begins.normalize()≤src['dates'][stop-1]].sum()+startup_cash
close(summary['natural_period_cash_cost_yuan'],natural,'natural-period total',1e-4)
return
{'passed':True,'path':display(run_dir),'strategy':summary['strategy'],'branch':summary['branch'],
'windows':windows,'intervals':len(trace),'cash_cost_yuan':float(costs['cash_cost_yuan'].sum()),
'physical_maximum_error_kwh':max(physical.values()),'january_inventory_value':src['nu'],
'january_night_intervals':src['nu_intervals'],'unproven_logged_events':unproven,
'configured_history_cap':cfg['history_days'],
'actual_history_paths_min':min(len(ev['history_indices']) for ev in events),
'actual_history_paths_max':max(len(ev['history_indices']) for ev in events),
'scope':'Full trace/event/cash and current source audit; optimizer control calls not all logged,
so only event-level Gap is enumerated.'}

def audit_selection(results_dir):
    """Recompute frozen choices using development windows, never test winners."""
    root=Path(results_dir)
    path=root/'selection.json'
    if not path.exists():return {'present':False,'passed':True,'reason':'selection not yet frozen or
auditing a stage folder'}
    saved=json.loads(path.read_text())
    require(saved['freeze_time']=='2025-02-25T00:00:00','wrong selection freeze time')
    require(saved['model_or_schedule_reselection_from_test'] is False,'test-based reselection claimed')
    require(len(saved['development'])==18,'incomplete declared development candidates')
    recomputed=[]
    for record in saved['development']:
        run=root/'calibration'/record['strategy']
        summary=json.loads((run/'summary.json').read_text())
        daily=pd.read_csv(run/'daily.csv')
        dev=daily[daily['window_date'].between('2025-02-17','2025-02-23')]
        require(dev['window_date'].tolist()==pd.date_range('2025-02-17',periods=7).strftime('%Y-%m-
%d').tolist(),
            'development must use seven completed windows before freeze')
        score=float(dev['inventory_adjusted_cost_yuan'].sum())
        close(record['score'],score,'development inventory-corrected score',1e-5)
        close(record['cash'],float(dev['cash_cost_yuan'].sum()),'development cash',1e-5)
        require(record['run_signature']==summary['run_signature'],'selection source signature mismatch')
        require(record['branch']==summary['branch'] and record['price_mode']==summary['config']
['price_mode']
            and record['trade_hours']==summary['config']['trade_hours'],'selection candidate
configuration mismatch')
        recomputed.append(**record,'score':score)
    winners={}
    for branch,key in (('4-2','selected_q42'),('4-3','selected_q43')):
        best=min((r for r in recomputed if r['branch']==branch),key=lambda r:(r['score'],r['strategy']))
        require(saved[key]==best['strategy'],'selected strategy is not development minimum')
        require(saved['branch_selections'][branch]==next(r for r in saved['development'] if
r['strategy']==best['strategy']),
            'branch selection differs from development record')
        winners[branch]=best['strategy']

```



```

return {'present':True,'passed':True,'development_candidates':18,'development_windows':7,
        'selection':winners,'uses_test_cash_for_choice':False}

def audit_all(data_dir=ROOT/'data',results_dir=ROOT/'results'):
    data_dir=Path(data_dir).resolve();results_dir=Path(results_dir).resolve()
    source=source_data(data_dir)
    reports=[];errors=[];pending=[];designs=[]
    for design_path in sorted(results_dir.rglob('design.json')):
        design=json.loads(design_path.read_text())
        if 'jobs' not in design:continue
        names=[job.get('name') for job in design['jobs']]
        require(all(names) and len(names)==len(set(names)),f'{design_path}: invalid/duplicate planned
jobs')
        designs.append({'path':display(design_path),'expected_runs':len(names)})
        for name in names:
            summary_path=design_path.parent/name/'summary.json'
            if not summary_path.exists() or not json.loads(summary_path.read_text()).get('complete'):
                pending.append(display(summary_path.parent))
    for path in sorted(results_dir.rglob('summary.json')):
        doc=json.loads(path.read_text())
        if not (doc.get('complete') and doc.get('branch') in ('4-2','4-3') and 'run_signature' in
doc):continue
        try:reports.append(audit_run(path.parent,data_dir,source))
        except Exception as
exc:errors.append({'path':display(path.parent),'error':f'{type(exc).__name__}: {exc}'})
        try:selection=audit_selection(results_dir)
        except Exception as exc:
            selection=
{'present':True,'passed':False};errors.append({'path':display(results_dir/'selection.json'),'error':f'{t
ype(exc).__name__}: {exc}'})
        return {'passed':bool(reports) and not errors and not
pending,'audited_runs':len(reports),'errors':errors,
                'pending_runs':pending,'declared_designs':designs,'selection_audit':selection,
                'audited_intervals':sum(r['intervals'] for r in reports),'runs':reports,
                'source_sha256':source['source_sha256'],'source_observation_kind':'original instantaneous nodes
averaged exactly once',
                'covered_run_format':'q4_runner strategy traces/events; causal micro-step replay uses its
separate q4_causal_audit',
                'settlement':'actual delivery endpoint-mean price; original+1.5*increase-
0.5*decrease+5*emergency',
                'boundary':'last missing actual right node held and flagged; no future node fabricated as
observed'}

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,default=ROOT/'data')
    p.add_argument('--results',type=Path,default=ROOT/'results');p.add_argument('--output',type=Path)
    a=p.parse_args();result=audit_all(a.data,a.results)
    output=a.output or
a.results/'independent_audit.json';output.parent.mkdir(parents=True,exist_ok=True)
    output.write_text(json.dumps(result,ensure_ascii=False,indent=2))
    print(json.dumps({'passed':result['passed'],'audited_runs':result['audited_runs'],
                    'pending_runs':result['pending_runs'],'errors':result['errors']},ensure_ascii=False))
    raise SystemExit(0 if result['passed'] else 1)

```

71. question4/code/q4_causal_audit.py

源文件 SHA-256: 8e935cdacdfd2a5f1921864e2ad4975d3e3643d9924a9833e26e545ae18ef429 ; 代码行数: 204。

```

"""Read-only independent ledger audit of fixed-contract micro-step experiments."""
import argparse
import datetime as dt
import hashlib
import json
from pathlib import Path

import numpy as np
import pandas as pd

from q4_io import load_inputs

ROOT=Path(__file__).resolve().parents[1]

def audit_replay(folder, data):
    folder=Path(folder);report=json.loads((folder/'summary.json').read_text())
    # Prefer this package's frozen main ledger; an old absolute origin may
    # still exist after copying, and must not mask tampering in the copy.
    source=folder.parent/'main'/report['source_strategy']
    if not source.exists():source=Path(report['source_run'])
    for name,digest in report['source_hashes'].items():
        if hashlib.sha256((source/name).read_bytes()).hexdigest()!=digest:raise ValueError('Source
digest mismatch')
    for name,digest in report['replay_code_sha256'].items():
        if hashlib.sha256((ROOT/'code'/name).read_bytes()).hexdigest()!=digest:raise ValueError('Replay
code digest mismatch')
    if report['input_source_sha256']!=data.get('metadata',{}).get('source_sha256',{}):
        raise ValueError('Replay input source digest mismatch')
    config=json.loads((source/'event_log.json').read_text())['config']
    starts={0:0,6:35,12:71,18:107};hours=(0,6,12,18)
    price_hours=config.get('price_hours',[0,6,12,18]);pv_hours=config['forecast_hours']
    nu=float(data['inventory_value'])
    np.testing.assert_allclose(report['common_inventory_value'],nu,atol=1e-12,rtol=0)

np.testing.assert_allclose(report['terminal_weight'],nu*config.get('terminal_multiplier',1.),atol=1e-
12,rtol=0)
    frozen=pd.read_csv(source/'trace.csv').set_index('interval_start')
    a=report['start_day'];b=a+report['days'];date_map={str(d):i for i,d in enumerate(data['dates'])}
    expected_index=pd.date_range(data['window_start'][a],periods=144*report['days'],freq='10min')
    expected_reveals=[]
    for day in range(a,b):
        date=data['dates'][day]
        expected_reveals.append(dict(at=dt.datetime.combine(date,dt.time()).isoformat(),kind='dayahead',
                                window_date=str(date),start=0,end=144))
        for hour in config['trade_hours']:
            if not hour:continue
            later=[starts[h] for h in config['trade_hours'] if h>hour]
            expected_reveals.append(dict(at=dt.datetime.combine(date,dt.time(hour)).isoformat(),kind='adjust',
                                window_date=str(date),start=starts[hour],end=min(later) if
later else 144))
        checks=[];recomputed={}
        for label in report['summary']:
            trace=pd.read_csv(folder/f'trace_{label}.csv')
            if json.loads((folder/f'contract_reveals_{label}.json').read_text())!=expected_reveals:
                raise ValueError('Contract reveal times or blocks disagree with the enabled source policy')
            numeric=
['g_kwh','q_kwh','load_energy_kwh','pv_energy_kwh','charge_kwh','discharge_kwh','emergency_kwh',

'unused_paid_kwh','curtailed_pv_kwh','soc_start_kwh','soc_end_kwh','control_price_forecast',

'settlement_price','ordinary_cost_yuan','adjustment_cost_yuan','emergency_cost_yuan','cash_cost_yuan','b
attery_loss_kwh']

```



```

        if not np.isfinite(trace[numeric].to_numpy(dtype=float)).all(): raise ValueError('Nonfinite
replay ledger value')

np.testing.assert_array_equal(pd.to_datetime(trace.interval_start).to_numpy(), expected_index.to_numpy())
np.testing.assert_array_equal(pd.to_datetime(trace.interval_end).to_numpy(),
(expected_index+pd.Timedelta(minutes=10)).to_numpy())
original=frozen.loc[trace.interval_start]
np.testing.assert_allclose(trace.g_kwh, original.g_kwh, atol=1e-8, rtol=0)
np.testing.assert_allclose(trace.q_kwh, original.q_kwh, atol=1e-8, rtol=0)
p=data['price_a'][a:b].ravel()
np.testing.assert_allclose(trace.settlement_price, p, atol=1e-10, rtol=0)
for j, row in trace.iterrows():
    day=a+j//144; t=j%144
    issued=date_map[row.price_forecast_issue_date]
    expected_issue=day+1 if t==143 and day<b-1 else day
    expected_hour=0 if expected_issue==day+1 else max(h for h in price_hours if starts[h]≤t)
    if issued≠expected_issue or row.price_forecast_hour≠expected_hour:
        raise ValueError('Price is not the latest enabled vintage under the declared local end
rule')

    if row.forecast_hour≠max(h for h in pv_hours if starts[h]≤t):
        raise ValueError('PV is not the latest enabled forecast vintage')
    if issued==day+1:
        if t≠143 or row.price_forecast_hour≠0: raise ValueError('Unavailable next-date price
vintage')

        expected=data['bridge_price_forecast'][issued]
    else:
        if issued≠day or row.price_forecast_hour not in (0,6,12,18):
            raise ValueError('Unavailable price issue date or hour')
        if t<{0:0,6:35,12:71,18:107}[row.price_forecast_hour]:
            raise ValueError('Unavailable future price vintage')
        version=(0,6,12,18).index(row.price_forecast_hour)
        expected=(data['price_nodes_a'][day,t]+data['price_nodes_f'][day,version,t+1])/2
        if not np.isfinite(expected) or expected≤0: raise ValueError('Invalid expected causal
price')

        if abs(row.control_price_forecast-expected)>1e-9: raise ValueError('Control used an
unavailable price endpoint/version')
        ordinary=p*trace.g_kwh
        adjustment=p*(1.5*np.maximum(trace.q_kwh-trace.g_kwh,0)-.5*np.maximum(trace.g_kwh-
trace.q_kwh,0))
        emergency=p*5*trace.emergency_kwh
        cash=ordinary+adjustment+emergency
        for column, values in [('ordinary_cost_yuan', ordinary), ('adjustment_cost_yuan', adjustment),
('emergency_cost_yuan', emergency)]:
            np.testing.assert_allclose(trace[column], values, atol=1e-7, rtol=0)
            np.testing.assert_allclose(trace.cash_cost_yuan, cash, atol=1e-7, rtol=0)
            np.testing.assert_allclose(trace.soc_start_kwh.iloc[1:], trace.soc_end_kwh.iloc[:-1], atol=1e-
8, rtol=0)

            if abs(trace.soc_start_kwh.iloc[0]-6000.)>1e-8: raise ValueError('Different local initial SOC')
            loss=.1*trace.charge_kwh+(1/.9-1)*trace.discharge_kwh
            np.testing.assert_allclose(trace.battery_loss_kwh, loss, atol=1e-8, rtol=0)
            flow=trace.q_kwh+trace.pv_energy_kwh-trace.curtailed_pv_kwh-
trace.unused_paid_kwh+trace.discharge_kwh+trace.emergency_kwh-trace.load_energy_kwh-trace.charge_kwh
            storage=trace.soc_end_kwh-trace.soc_start_kwh-.9*trace.charge_kwh+trace.discharge_kwh/.9
            np.testing.assert_allclose(flow, 0, atol=1e-7, rtol=0)
            np.testing.assert_allclose(storage, 0, atol=1e-7, rtol=0)
            errors=np.concatenate((1200-trace.soc_end_kwh, trace.soc_end_kwh-10800,
            trace.charge_kwh-5000/6, trace.discharge_kwh-5000/6,
            -trace.g_kwh, -trace.q_kwh, -trace.charge_kwh, -trace.discharge_kwh, -trace.emergency_kwh,
            -trace.unused_paid_kwh, -trace.curtailed_pv_kwh,
            np.minimum(trace.charge_kwh, trace.discharge_kwh), trace.unused_paid_kwh-trace.q_kwh,
            trace.curtailed_pv_kwh-trace.pv_energy_kwh, trace.emergency_kwh-trace.load_energy_kwh))
            if errors.max()>1e-6: raise ValueError('Ten-minute physical limits fail')
            summary={key: float(trace[key].sum()) for key in
('cash_cost_yuan', 'ordinary_cost_yuan', 'adjustment_cost_yuan',

'emergency_cost_yuan', 'emergency_kwh', 'load_energy_kwh', 'pv_energy_kwh', 'charge_kwh', 'discharge_kwh',

```

```

        'battery_loss_kwh', 'q_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh'})

summary.update(soc_start_kwh=float(trace.soc_start_kwh.iloc[0]), soc_end_kwh=float(trace.soc_end_kwh.iloc[-1]),
               inventory_adjusted_cost_yuan=float(cash.sum()-nu*(trace.soc_end_kwh.iloc[-1]-trace.soc_start_kwh.iloc[0])),
               intervals=len(trace), minute_intervals=0)
summary['energy_balance_residual_kwh']=float(flow.sum())
summary['battery_balance_residual_kwh']=float(storage.sum())
ledger_error=float(np.max(np.abs(cash-trace.cash_cost_yuan)))
check=dict(variant=label, tenminute_rows=len(trace), maximum_cash_error_yuan=ledger_error)
if label=='10min_mean':
    np.testing.assert_allclose(trace.load_energy_kwh, data['load_a'][a:b].ravel()/6, atol=1e-8, rtol=0)
    np.testing.assert_allclose(trace.pv_energy_kwh, data['pv_a'][a:b].ravel()/6, atol=1e-8, rtol=0)
    if np.minimum(trace.charge_kwh, trace.emergency_kwh).max()>1e-7:raise ValueError('Mean model emergency energy charged')
else:
    micro=pd.read_csv(folder/f'micro_trace_{label}.csv.gz')
    minutes=float(label.removesuffix('min_causal'));count=int(round(10/minutes))
    if len(micro)!=len(trace)*count:raise ValueError('Wrong number of micro intervals')
    begin=pd.to_datetime(micro.interval_start);end=pd.to_datetime(micro.interval_end)
    np.testing.assert_allclose((end-begin).dt.total_seconds(), minutes*60, atol=1e-9, rtol=0)
    np.testing.assert_array_equal(begin.iloc[1:].to_numpy(), end.iloc[:-1].to_numpy())
    c=micro.charge_kwh; d=micro.discharge_kwh; e=micro.emergency_kwh
    L=micro.load_observed_left_kw*minutes/60; G=micro.pv_observed_left_kw*minutes/60
    if not np.isfinite(micro.select_dtypes(include='number').to_numpy()).all():raise
ValueError('Nonfinite micro ledger value')
    flow=micro.q_kwh+G-micro.curtailed_pv_kwh-micro.unused_paid_kwh+d+e-L-c
    storage=micro.soc_end_kwh-micro.soc_start_kwh-.9*c+d/.9
    errors=np.concatenate((np.abs(flow), np.abs(storage), 1200-micro.soc_end_kwh,
                           micro.soc_end_kwh-10800, c-5000*minutes/60, d-5000*minutes/60,
                           -c, -d, -e, -micro.unused_paid_kwh, -micro.curtailed_pv_kwh,
                           np.minimum(c, d), np.minimum(c, e), micro.unused_paid_kwh-micro.q_kwh,
                           micro.curtailed_pv_kwh-G, e-L))
    physical=float(max(errors.max(), 0))
    if physical>1e-6:raise ValueError(f'Micro physical constraints fail: {physical}')
    np.testing.assert_allclose(micro.soc_start_kwh.iloc[1:], micro.soc_end_kwh.iloc[:-1], atol=1e-8, rtol=0)
    np.testing.assert_allclose(micro.soc_start_kwh.iloc[:count], trace.soc_start_kwh, atol=1e-8, rtol=0)
    np.testing.assert_allclose(micro.soc_end_kwh.iloc[count-1::count], trace.soc_end_kwh, atol=1e-8, rtol=0)

np.testing.assert_allclose(micro.q_kwh, np.repeat(trace.q_kwh.to_numpy()/count, count), atol=1e-8, rtol=0)
group=np.arange(len(micro))//count
for column in
('q_kwh', 'charge_kwh', 'discharge_kwh', 'emergency_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh'):
    values=micro[column].groupby(group).sum()
    np.testing.assert_allclose(values, trace[column], atol=1e-7, rtol=0)
    np.testing.assert_allclose(L.groupby(group).sum(), trace.load_energy_kwh, atol=1e-7, rtol=0)
    np.testing.assert_allclose(G.groupby(group).sum(), trace.pv_energy_kwh, atol=1e-7, rtol=0)
    direction=micro.command_direction.to_numpy().reshape(-1, count)
    np.testing.assert_array_equal(direction, np.repeat(direction[:, 0, None], count, axis=1))
    if not np.isin(direction, ['charge', 'discharge_or_idle']).all():raise ValueError('Unknown control direction')
    if np.where(micro.command_direction=='charge', d, c).max()>1e-7:raise ValueError('Micro action contradicts frozen direction')

budget=np.cumsum(d.to_numpy().reshape(-1, count), axis=1)+micro.remaining_block_discharge_budget_kwh.to_numpy().reshape(-1, count)
np.testing.assert_allclose(budget, np.repeat(budget[:, 0, None], count, axis=1), atol=1e-7, rtol=0)
if budget.max()>5000/6+1e-7 or micro.remaining_block_discharge_budget_kwh.min()<-1e-7:raise
ValueError('Discharge budget is invalid')
if np.max(np.minimum(trace.charge_kwh, trace.discharge_kwh))>1e-7:
    raise ValueError('Ten-minute direction reversed')

```

```

        # Every finer substep is settled at its enclosing ten-minute rate.
        np.testing.assert_allclose(micro.tenminute_settlement_price, np.repeat(p, count), atol=1e-
10, rtol=0)

        # Independently verify linear artificial signal construction and
        # left-hold integration; this check is separate from optimization.
        frac=np.tile(np.arange(count)/count, len(trace))
        for key in ('load', 'pv'):
            nodes=data[f'{key}_nodes'][a:b]
            signal=np.repeat(nodes[:, :-1].ravel(), count)*(1-
frac)+np.repeat(nodes[:, 1:].ravel(), count)*frac
            np.testing.assert_allclose(micro[f'{key}_observed_left_kw'], signal, atol=1e-7, rtol=0)
            price_nodes=data['price_nodes_a'][a:b]
            price_signal=np.repeat(price_nodes[:, :-1].ravel(), count)*(1-
frac)+np.repeat(price_nodes[:, 1:].ravel(), count)*frac
            np.testing.assert_allclose(micro.price_observed_left, price_signal, atol=1e-9, rtol=0)
            summary['minute_intervals']=len(micro)
            check.update(micro_rows=len(micro), maximum_physical_error_kwh=physical)
        for key, value in summary.items():
            np.testing.assert_allclose(report['summary'][label][key], value, atol=1e-
5, rtol=0, err_msg=f'Summary {label}/{key} differs from ledger')
            recomputed[label]=summary
            checks.append(check)
        for label, summary in recomputed.items():
            if label=='10min_mean':continue
            minutes=float(label.removesuffix('min_causal'))
            for key, value in report['fine_minus_mean'][label].items():
                np.testing.assert_allclose(value, summary[key]-recomputed['10min_mean'][key], atol=1e-
5, rtol=0)
            for key in ('load', 'pv'):
                actual=summary[f'{key}_energy_kwh']-recomputed['10min_mean'][f'{key}_energy_kwh']
                theoretical=minutes*(data[f'{key}_nodes'][a, 0]-data[f'{key}_nodes'][b-1, -1])/120
                np.testing.assert_allclose(actual, theoretical, atol=1e-5, rtol=0)
            for kind in ('observed_bias_kwh', 'analytical_bias_kwh'):
                np.testing.assert_allclose(report['signal_energy_integration_checks'][label][kind]
[key], actual, atol=1e-5, rtol=0)
            result=dict(strategy=report['source_strategy'], checks=checks, passed=True)
            (folder/'independent_audit.json').write_text(json.dumps(result, indent=2, ensure_ascii=False))
        return result

def audit_study(result_root, data_dir=ROOT/'data'):

    data=load_inputs(data_dir); folders=sorted((Path(result_root)/'causal_replay').glob('Q*/summary.json'))
    if len(folders)!=4:raise ValueError('Expected four completed replay policies')
    results=[audit_replay(path.parent, data) for path in folders]
    result=dict(passed=True, policies=results)

    (Path(result_root)/'causal_replay/independent_audit.json').write_text(json.dumps(result, indent=2, ensure_
ascii=False))
    return result

if __name__=='__main__':
    p=argparse.ArgumentParser(); p.add_argument('--results', type=Path, default=ROOT/'results/final')
    p.add_argument('--data', type=Path, default=ROOT/'data'); args=p.parse_args()
    print(json.dumps(audit_study(args.results, args.data), ensure_ascii=False))

```

72. question4/code/q4_causal_replay.py

源文件 SHA-256: e7ab421909b7eda686cfe112c730bdd4af14be4fbfd60c1172c84e39e1cd1da0 ; 代码行数: 330。

```

"""Execution-timing sensitivity under the same timestamp-revealed contracts.

Compare the main ten-minute mean-feedback approximation with micro-step causal
left-node feedback. Reoptimize battery execution only, never original orders or
formal adjustments. The simulator linearly interpolates observed ten-minute
nodes into an artificial finer-step environment; the controller only receives
a node when its timestamp is reached. This is a discretization sensitivity,
not higher-frequency measurement or an exact physical validation.
"""
from __future__ import annotations

import argparse
import csv
import datetime as dt
import hashlib
import json
import gzip
from pathlib import Path

import numpy as np

from q4_io import load_inputs, selected_pv
from q4_optimizer import solve_dispatch, solve_fixed_control, LOW, HIGH, RATE, ETA_C, ETA_D
from q4_risk import HOURS, START, latest_hour, future_mask, current_block

def _read_csv(path):
    with open(path, encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f))

def _write_csv(path, rows):
    if not rows: return
    with open(path, "w", encoding="utf-8-sig", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader(); w.writerows(rows)

def causal_price_vector(data, day, t, price_hour):
    """Current actual left + forecast right; future right actual is inaccessible."""
    if price_hour not in HOURS or not 0 ≤ t < 144 or START[price_hour] > t:
        raise ValueError('Requested price forecast is not available at this control time')
    v = HOURS.index(price_hour)
    prices = data['price_f'][day, v, t:].copy()
    prices[0] = (data['price_nodes_a'][day, t] + data['price_nodes_f'][day, v, t+1]) / 2
    if not np.isfinite(prices).all() or (prices ≤ 0).any():
        raise ValueError('Invalid causal price vector; no silent clipping or future-version fill')
    return prices

def run_replay(data_dir, main_run_dir, outdir, start_day=24, days=7,
               initial_soc=6000., fine_steps=(1.,), *, data_override=None):
    data = load_inputs(data_dir) if data_override is None else data_override
    source, out = Path(main_run_dir), Path(outdir)
    out.mkdir(parents=True, exist_ok=True)
    log = json.loads((source/"event_log.json").read_text())
    cfg = log["config"]
    trade = tuple(cfg["trade_hours"])
    forecasts = tuple(cfg["forecast_hours"])
    price_hours = tuple(cfg.get('price_hours', [0, 6, 12, 18]))
    data['policy_pv'] = selected_pv(data, cfg['branch'], cfg.get('pv_source', 'supplied'))
    if not LOW ≤ initial_soc ≤ HIGH: raise ValueError('Local common SOC is outside physical limits')
    fine_steps = tuple(float(x) for x in fine_steps)
    if not fine_steps or len(set(fine_steps)) ≠ len(fine_steps) or any(x ≤ 0 or x > 10 or abs(10/x -

```

```

round(10/x)>1e-8 for x in fine_steps):
    raise ValueError('Fine steps must be unique positive divisors of ten minutes')
stop = start_day+days
if start_day < 0 or days < 1 or stop > len(data["dates"]):
    raise ValueError("Requested local replay window is outside input dates.")
selected = {str(d): [] for d in data["dates"][start_day:stop]}
for row in _read_csv(source/"trace.csv"):
    if row["window_date"] in selected:
        selected[row["window_date"]].append(row)
events = {(r["window_date"], r["kind"], r["decision_at"]): r for r in log["events"]}
original, final_q = {}, {}
for day in range(start_day, stop):
    date = str(data["dates"][day])
    rows = sorted(selected[date], key=lambda r:r["interval_start"])
    if len(rows) != 144:
        raise ValueError(f"Main trace does not contain all 144 slots for {date}.")
    for t, row in enumerate(rows):
        target = data["window_start"][day]+dt.timedelta(minutes=10*t)
        if row["interval_start"] != target.isoformat():
            raise ValueError("Main trace timestamps do not match the contract window.")
        if abs(float(row["price_yuan_per_kwh"])-data["price_a"][day,t]) > 1e-9:
            raise ValueError("Main trace and input tariff differ.")
        for key in ("load", "pv"):
            if abs(float(row[f"{key}_actual_kw"])-data[f"{key}_a"][day,t]) > 1e-7:
                raise ValueError("Main trace and input observed interval means differ.")
            if abs(float(row["load_forecast_kw"])-data["load_f"][day,t]) > 1e-7:
                raise ValueError("Main trace and input load forecasts differ.")
    selected[date] = rows
    original[day] = np.array([float(r["g_kwh"]) for r in rows])
    final_q[day] = np.array([float(r["q_kwh"]) for r in rows])
    midnight = dt.datetime.combine(data["dates"][day],dt.time()).isoformat()
    if (date,"dayahead",midnight) not in events:
        raise ValueError("A day-ahead contract has no timestamped release event.")
    adjustable = np.zeros(144, bool)
    for h in trade:
        if h:
            a,b = current_block(trade,h)
            key=(date,"adjust",dt.datetime.combine(data["dates"][day],dt.time(h)).isoformat())
            if key not in events or (events[key]["commit_start"],events[key]["commit_end"]) !=
(a,b):
                raise ValueError("A formal adjustment lacks its matching commit event.")
            adjustable[a:b] = True
    if np.max(np.abs((final_q[day]-original[day])[~adjustable]), initial=0) > 1e-5:
        raise ValueError("An order was changed outside the enabled adjustment blocks.")
main_initial = float(selected[str(data["dates"][start_day])][0]["soc_start_kwh"])
initial = float(initial_soc)
nu=float(data['inventory_value'])
kappa = nu*float(cfg.get('terminal_multiplier',1.))
solver_options = dict(terminal_weight=kappa, time_limit=cfg.get("time_limit",10),
                        mip_gap=cfg.get("mip_gap",1e-4))
summaries, trajectories = {}, {}
variants=[('10min_mean',None)]+[(f'{step:g}min_causal',step) for step in fine_steps]
for variant,micro_step in variants:
    state = initial
    rows, minute_rows, reveals = [], [], []
    stats = dict(solves=0,solve_seconds=0.,max_violation_kwh=0.,unproven=0,
                 lp_solves=0,milp_solves=0,max_tenminute_direction_overlap_kwh=0.,
                 max_charge_power_kw=0.,max_discharge_power_kw=0.,
                 min_soc_kwh=initial,max_soc_kwh=initial)

    def command(L,G,prices,base,booked,future,now_l,now_pv):
        surplus=max(booked[0]+now_pv-now_l,0.)
        deficit=max(now_l-booked[0]-now_pv,0.)
        if surplus > 1e-9:
            return min(surplus,RATE,max((HIGH-state)/ETA_C,0.)),0.,0.,True
        if cfg.get("controller","mpc") == "greedy":

```



```

        d=min(deficit,RATE,max((state-LOW)*ETA_D,0.))
        return 0.,d,max(deficit-d,0.),False
    L,G=L.copy(),G.copy();L[0]=now_l;G[0]=now_pv
    if not future.any():
        z=solve_fixed_control(L,G,prices,booked,state,terminal_weight=kappa,
                               time_limit=config.get('time_limit',10))
        stats['solves']+=1;stats['lp_solves']+=1;stats['solve_seconds']+=z['solve_seconds']
        return float(z['c'][0]),float(z['d'][0]),float(z['e'][0]),False
    z=solve_dispatch(L[None,:],G[None,:],prices,state,mode="control",
                    base_g=base,booked_q=booked,future_adjust_mask=future,**solver_options)
    stats["solves"]+=1;stats["solve_seconds"]+=z["solve_seconds"]
    stats['milp_solves']+=1
    stats["unproven"]+=int(not z["optimality_proven"])
    return float(z["c"][0,0]),float(z["d"][0,0]),float(z["e"][0,0]),False

def check_step(gq,L,G,c,d,e,v,r,sbefore,safter,rate):
    error=max(abs(gq+G-r-v+d+e-L-c),
              abs(safter-sbefore-ETA_C*c+d/ETA_D),
              LOW-safter,safter-HIGH,c-rate,d-rate,-min(c,d,e,v,r),
              min(c,d),min(c,e),v-gq,r-G,e-L,0.)
    stats["max_violation_kwh"]=max(stats["max_violation_kwh"],error)
    stats['max_charge_power_kw']=max(stats['max_charge_power_kw'],c/rate*5000.)
    stats['max_discharge_power_kw']=max(stats['max_discharge_power_kw'],d/rate*5000.)
    stats['min_soc_kwh']=min(stats['min_soc_kwh'],safter)
    stats['max_soc_kwh']=max(stats['max_soc_kwh'],safter)
    if error > 1e-5:
        raise RuntimeError(f"{variant} physical violation: {error}")

def execute(day,t,L,G,prices,base,booked,future,fh,ph,price_issue_day=None):
    nonlocal state
    before=state
    begin=data["window_start"][day]+dt.timedelta(minutes=10*t)
    if variant == "10min_mean":
        load=float(data["load_a"][day,t]/6);pv=float(data["pv_a"][day,t]/6)
        c,d,e,direction=command(L,G,prices,base,booked,future,load,pv)
        waste=max(booked[0]+pv+d+e-load-c,0.)
        v=min(booked[0],waste);r=max(waste-v,0.)
        state+=ETA_C*c-d/ETA_D
        check_step(booked[0],load,pv,c,d,e,v,r,before,state,RATE)
    else:
        # The controller receives only the current left node here.
        now_l=float(data["load_nodes"][day,t]/6)
        now_g=float(data["pv_nodes"][day,t]/6)
        cp,dp,ep,direction=command(L,G,prices,base,booked,future,now_l,now_g)
        remaining_d=dp
        c=d=e=v=r=load=pv=0.
        micro_count=int(round(10/micro_step))
        for j in range(micro_count):
            # Artificial environment: expose only the node at NOW.
            # A right endpoint is used to construct the environment,
            # never passed to command() before it becomes observable.
            fraction=j/micro_count
            observed_load=float((1-fraction)*data["load_nodes"][day,t]
                                +fraction*data["load_nodes"][day,t+1])
            observed_pv=float((1-fraction)*data["pv_nodes"][day,t]
                               +fraction*data["pv_nodes"][day,t+1])
            # Price is only a now-observed signal here; the committed
            # ten-minute direction does not optimize on future prices.
            observed_price=float((1-fraction)*data['price_nodes_a'][day,t]
                                 +fraction*data['price_nodes_a'][day,t+1])
            lm,gm=observed_load*micro_step/60,observed_pv*micro_step/60
            qm=float(booked[0]/micro_count)
            A=max(qm+gm-lm,0.);D=max(lm-qm-gm,0.)
            cm=min(A,RATE/micro_count,max((HIGH-state)/ETA_C,0.)) if direction else 0.
            dm=0. if direction else min(D,RATE/micro_count,max((state-
LOW)*ETA_D,0.),remaining_d)

```

```

em=max(D-dm,0.)
wm=max(A-cm,0.);vm=min(qm,wm);rm=max(wm-vm,0.)
sb=state;state+=ETA_C*cm-dm/ETA_D
remaining_d=max(remaining_d-dm,0.)
check_step(qm,lm,gm,cm,dm,em,vm,rm,sb,state,RATE/micro_count)
minute_rows.append(dict(interval_start=
(begin+dt.timedelta(minutes=j*micro_step)).isoformat(),
interval_end=(begin+dt.timedelta(minutes=(j+1)*micro_step)).isoformat(),
load_observed_left_kw=observed_load,pv_observed_left_kw=observed_pv,

price_observed_left=observed_price,tenminute_settlement_price=float(data['price_a'][day,t]),
q_kwh=qm,charge_kwh=cm,discharge_kwh=dm,emergency_kwh=em,
unused_paid_kwh=vm,curtailed_pv_kwh=rm,soc_start_kwh=sb,soc_end_kwh=state,
command_direction="charge" if direction else "discharge_or_idle",
remaining_block_discharge_budget_kwh=remaining_d))
c+=cm;d+=dm;e+=em;v+=vm;r+=rm;load+=lm;pv+=gm
if min(c,d)>1e-7 or c>RATE+1e-6 or d>dp+1e-6:
    raise RuntimeError('Within-block direction, power or discharge budget violated')

stats['max_tenminute_direction_overlap_kwh']=max(stats['max_tenminute_direction_overlap_kwh'],min(c,d))
# Contract settlement is the same realized ten-minute mean tariff
# for ALL variants, never minute-price weighting or forecast price.
settlement=float(data['price_a'][day,t])
ordinary=float(settlement*base[0]);increase=max(booked[0]-base[0],0.);decrease=max(base[0]-
booked[0],0.)
adjustment=float(settlement*(1.5*increase-.5*decrease));emergency=float(5*settlement*e)
rows.append(dict(variant=variant>window_date=str(data["dates"][day]),
interval_start=begin.isoformat(),interval_end=
(begin+dt.timedelta(minutes=10)).isoformat(),
forecast_hour=fh,price_forecast_hour=ph,
price_forecast_issue_date=str(data['dates'][day] if price_issue_day is None else
price_issue_day),
control_price_forecast=float(prices[0]),settlement_price=settlement,
g_kwh=float(base[0]),q_kwh=float(booked[0]),
load_energy_kwh=load,pv_energy_kwh=pv,charge_kwh=c,discharge_kwh=d,emergency_kwh=e,
unused_paid_kwh=v,curtailed_pv_kwh=r,soc_start_kwh=before,soc_end_kwh=state,
ordinary_cost_yuan=ordinary,adjustment_cost_yuan=adjustment,
emergency_cost_yuan=emergency,cash_cost_yuan=ordinary+adjustment+emergency))
rows[-1]['battery_loss_kwh']=(1-ETA_C)*c+(1/ETA_D-1)*d

for day in range(start_day,stop):
    # New g becomes visible at this midnight, before the old tail is
    # observed. Future formal q remains inaccessible until its gate.
    if day > start_day:
        oldg,oldq=g.copy(),q.copy()
        g=original[day].copy();q=g.copy()
        reveals.append(dict(at=dt.datetime.combine(data["dates"][day],dt.time()).isoformat(),
kind="dayahead",window_date=str(data["dates"][day]),start=0,end=144))
    if day > start_day:
        oldfh=latest_hour(forecasts,143)
        lf=np.r_[data["load_f"][day-1,143],data["load_f"][day]]/6
        pf=np.r_[data["policy_pv"][day-1,HOURS.index(oldfh),143],data["policy_pv"][day,0]]/6
        pp=np.r_[data["bridge_price_forecast"][day],data["price_f"][day,0]]
        execute(day-1,143,lf,pf,pp,np.r_[oldg[-1],g],np.r_[oldq[-1],q],
np.r_[False,future_mask(trade,0)],oldfh,0,price_issue_day=data['dates'][day])
    for t in range(143):
        h=next((h for h in trade if h and START[h]==t),None)
        if h is not None:
            a,b=current_block(trade,h)
            q[a:b]=final_q[day][a:b] # Only current committed block.
            reveals.append(dict(at=dt.datetime.combine(data["dates"]
[day],dt.time(h)).isoformat(),
kind="adjust",window_date=str(data["dates"]
[day]),start=a,end=b))
            fh=latest_hour(forecasts,t)
            ph=latest_hour(price_hours,t)

```



```

        execute(day,t,data["load_f"][day,t:]/6,data["policy_pv"][day,HOURS.index(fh),t:]/6,
                causal_price_vector(data,day,t,ph),g[t:],q[t:],future_mask(trade,t),fh,ph)
    if not np.allclose(q,final_q[day],rtol=0,atol=1e-5):
        raise RuntimeError("Timestamp-revealed order differs from the fixed main order.")
    day=stop-1;fh=latest_hour(forecasts,143);ph=latest_hour(price_hours,143)
    execute(day,143,data["load_f"][day,143:]/6,data["policy_pv"][day,HOURS.index(fh),143:]/6,
            causal_price_vector(data,day,143,ph),g[-1:],q[-1:],np.zeros(1,bool),fh,ph)
    if len(rows)≠144*days:
        raise RuntimeError("Wrong local replay length.")
    totals={key:float(sum(row[key] for row in rows)) for key in
            ("cash_cost_yuan","ordinary_cost_yuan","adjustment_cost_yuan","emergency_cost_yuan",
"emergency_kwh","load_energy_kwh","pv_energy_kwh","charge_kwh","discharge_kwh","battery_loss_kwh")}
    totals.update({key:float(sum(row[key] for row in rows))
                    for key in ('q_kwh','unused_paid_kwh','curtailed_pv_kwh')})
    totals['energy_balance_residual_kwh']=(totals['q_kwh']+totals['pv_energy_kwh']+
    totals['discharge_kwh']+totals['emergency_kwh']-totals['load_energy_kwh']-
    totals['charge_kwh']-totals['unused_paid_kwh']-totals['curtailed_pv_kwh'])
    totals['battery_balance_residual_kwh']=(state-initial-totals['charge_kwh']+
    totals['discharge_kwh']+totals['battery_loss_kwh'])
    if
max(abs(totals['energy_balance_residual_kwh']),abs(totals['battery_balance_residual_kwh']))>1e-5:
    raise RuntimeError('Aggregate energy or battery balance failed')
    totals.update(soc_start_kwh=initial,soc_end_kwh=state,
                  inventory_adjusted_cost_yuan=totals["cash_cost_yuan"]-nu*(state-initial),
                  intervals=len(rows),minute_intervals=len(minute_rows),solver=stats)
    summaries[variant]=totals;trajectories[variant]=rows
    _write_csv(out/f"trace_{variant}.csv",rows)
    if minute_rows:
        with gzip.open(out/f"micro_trace_{variant}.csv.gz",'wt',encoding='utf-8-sig',newline='') as
stream:
writer=csv.DictWriter(stream,fieldnames=list(minute_rows[0]));writer.writeheader();writer.writerows(minu
te_rows)
        (out/f"contract_reveals_{variant}.json").write_text(json.dumps(reveals,indent=2))
        base=summaries['10min_mean'];differences={};energy_checks={}
        for step in fine_steps:
            label=f'{step:g}min-causal';fine=summaries[label]
            for key in ("ordinary_cost_yuan","adjustment_cost_yuan"):
                if abs(base[key]-fine[key])>1e-6:
                    raise RuntimeError("The controllers did not use identical signed contracts.")
            # Continuous linear-node left-Riemann discrepancy telescopes across windows.
            expected_bias={key:float(step*(data[f"{key}_nodes"][start_day,0]-data[f"{key}_nodes"][stop-
1,-1])/120)
                            for key in ("load","pv")}
            observed_bias={key:fine[f"{key}_energy_kwh"]-base[f"{key}_energy_kwh"] for key in ("load","pv")}
            if any(abs(expected_bias[k]-observed_bias[k])>1e-5 for k in expected_bias):
                raise RuntimeError("Minute discretization energy difference disagrees with its analytical
identity.")
            differences[label]={k:fine[k]-base[k] for k in
('cash_cost_yuan','emergency_kwh','soc_end_kwh','inventory_adjusted_cost_yuan','battery_loss_kwh')}
            energy_checks[label]=dict(observed_bias_kwh=observed_bias,analytical_bias_kwh=expected_bias)
            result=dict(source_run=str(source.resolve()),source_strategy=cfg["strategy"],
                        source_hashes={name:hashlib.sha256((source/name).read_bytes()).hexdigest()
                                        for name in ("trace.csv","event_log.json","summary.json")},
                        replay_code_sha256=
{name:hashlib.sha256((Path(__file__).resolve().parent/name).read_bytes()).hexdigest()
                                for name in ('q4_causal_replay.py','q4_optimizer.py','q4_risk.py')},
                        input_source_sha256=data.get('metadata',{}).get('source_sha256',{}),
                        start_day=start_day,days=days,start=str(data["window_start"]
[start_day]),end=str(data["window_end"][stop-1]),
                        local_initial_state="Common configured SOC at first selected 00:10 for every branch/variant;
frozen orders were NOT reoptimized for this state",
                        local_initial_soc=initial,main_trace_original_local_initial_soc=main_initial,
                        end_rule="Both controllers stop at the selected final 00:10; neither reads the next unselected
contract",

```

```

        contracts="Same fixed g/q, revealed only at original midnight and enabled intraday commit
times",
        minute_method="Artificial linear-node environment; left power held for each micro-step, frozen
ten-minute direction and discharge budget",
        fine_step_minutes=list(fine_steps),settlement='All variants use the same actual ten-minute mean
price; controller price sees only current left plus predicted right',
        common_inventory_value=nu,terminal_weight=kappa,
        limitations=["Not high-frequency measured data or exact physical validation",
                    "Micro-step left-endpoint power hold differs from exact linear-signal integration;
analytical total energy discrepancy and refinement are reported",
                    "Within-block direction cannot reverse; physical stopping, free disposal and
emergency balancing remain allowed",
                    "Local fixed-contract sensitivity does not reselect the best annual contract
policy"],
        summary=summaries,fine_minus_mean=differences,signal_energy_integration_checks=energy_checks,
        checks_passed=True)
    (out/"summary.json").write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding="utf-8")
    return result

if __name__=="__main__":
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data",required=True);parser.add_argument("--main-run",required=True)
    parser.add_argument("--out",required=True);parser.add_argument("--start-day",type=int,default=24)
    parser.add_argument("--days",type=int,default=7)
    parser.add_argument("--initial-soc",type=float,default=6000.)
    parser.add_argument('--fine-steps',default='1',help='Comma-separated step minutes, e.g. 1,0.1')
    args=parser.parse_args()

    result=run_replay(args.data,args.main_run,args.out,args.start_day,args.days,args.initial_soc,tuple(map(f
loat,args.fine_steps.split(','))))
    print(json.dumps({"checks_passed":result["checks_passed"],
                    "fine_minus_mean":result["fine_minus_mean"]},ensure_ascii=False))

```

73. question4/code/q4_causal_study.py

源文件 SHA-256: e5350fdf3059957f48c9e5be70f7f15ab7971874e5b1e2434efc1ec7397c9e7e ; 代码行数: 126.

```

"""Run and summarize the preregistered short fixed-contract execution checks."""
import argparse
import csv
import hashlib
import json
from pathlib import Path

from q4_causal_replay import run_replay
from q4_io import load_inputs

ROOT=Path(__file__).resolve().parents[1]

def planned_jobs(result_root):
    selected=json.loads((Path(result_root)/'selection.json').read_text())
    winners=[selected['selected_q42'],selected['selected_q43']]
    names=[]
    for name in winners:
        names.extend([name,name.replace('_point_', '_joint_') if '_point_' in name else
name.replace('_joint_', '_point_')])
    return [(name,(1.,.1) if name in winners else (1.,)) for name in names]

def summarize(result_root):
    result_root=Path(result_root);folder=result_root/'causal_replay'
    rows=[];checks=[]
    for name,steps in planned_jobs(result_root):
        path=folder/name/'summary.json'
        if not path.exists():raise FileNotFoundError(f'Incomplete causal study: {path}')
        r=json.loads(path.read_text())
        if not r['checks_passed'] or r['local_initial_soc']!=6000. or r['days']!=7 or
r['start_day']!=24:
            raise ValueError('Inconsistent local replay design')
        for source,digest in r['source_hashes'].items():
            if hashlib.sha256((result_root/'main'/name/source).read_bytes()).hexdigest()!=digest:
                raise ValueError('Frozen source changed after replay')
        for source,digest in r['replay_code_sha256'].items():
            if hashlib.sha256((ROOT/'code'/source).read_bytes()).hexdigest()!=digest:
                raise ValueError('Replay numerical implementation changed')
        for variant,z in r['summary'].items():
            rows.append(dict(strategy=name,variant=variant,days=r['days'],initial_soc_kwh=r['local_initial_soc'],
cash_cost_yuan=z['cash_cost_yuan'],inventory_adjusted_cost_yuan=z['inventory_adjusted_cost_yuan'],
soc_end_kwh=z['soc_end_kwh'],emergency_kwh=z['emergency_kwh'],battery_loss_kwh=z['battery_loss_kwh'],
cash_difference_vs_mean=z['cash_cost_yuan']-r['summary']['10min_mean']
['cash_cost_yuan'],
max_physical_error_kwh=z['solver']['max_violation_kwh'],
max_charge_power_kw=z['solver']['max_charge_power_kw'],
max_discharge_power_kw=z['solver']['max_discharge_power_kw'],
min_soc_kwh=z['solver']['min_soc_kwh'],max_soc_kwh=z['solver']['max_soc_kwh']))
            checks.append(dict(strategy=name,summary_sha256=hashlib.sha256(path.read_bytes()).hexdigest(),

fine_step_minutes=r['fine_step_minutes'],signal_energy_integration_checks=r['signal_energy_integration_c
hecks']))
    with (folder/'comparison.csv').open('w',newline='',encoding='utf-8-sig') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
    report=dict(scope='2025-02-25 00:10 to 2025-03-04 00:10, seven windows, identical configured6000
start; policy-specific frozen orders',
        rows=rows,checks=checks,checks_passed=True,selection_revised=False)
    (folder/'comparison.json').write_text(json.dumps(report,ensure_ascii=False,indent=2))
    lines=['# 分钟因果执行敏感性','',
        '这是固定正式合同条件下的执行近似检验，不是重新训练预测器或重新选择整年交易政策。四条策略均在 2025 年 2 月 25

```

```

日 00:10 以 6000 kWh 开始, 连续执行到 3 月 4 日 00:10; 各自原合同来自完整主回放, 没有为这个局部起点重新优化。',
'',
'策略名中的 point 仅指价格采用点预测, 负载与光伏仍按主模型处理场景; joint 表示价格也加入同日配对场景。这里的电
池执行统一使用当时点预测进行滚动优化。',
'',
'每条策略本地重放一次十分钟实测均值反馈, 另做一分钟因果反馈; 冻结开发期选中的两条策略再细化到 0.1 分钟。两种执行
共用该策略的原 g 和正式 q, 只在原签单时刻揭示。跨午夜先揭示下一日原 g, 然后执行旧合同最后十分钟。未来调单只能作为优化预
案, 不能提前读最终正式 q。',
'',
'一分钟和 0.1 分钟仅改变人工环境积分与底层约束执行步长, MPC 都仍然每十分钟重算一次; 不是每分钟或每六秒重新优化
电池计划。因此, 二者费用差不能解释为提高 MPC 重算频率的收益或损失。',
'',
'每十分钟下指令时, 细步控制只收到此刻的负载、光伏、真实电价左端节点; 本段优化价格为真实左端与当时版本预测右端的平
均。未来实际右端价格只供事后结算和人工环境生成, 不进入优化目标。未来负载/光伏沿用当时发布的区间预测。',
'',
'人工环境在原十分钟实际节点之间线性插值。每个微步只读取当前节点并持平执行该微步: 每十分钟内充放电方向不反转, 放电
不超过该十分钟原指令预算; 微步可按 SOC、功率与负载缺口提前停止。紧急电只补当时缺口, 不能给电池充电。该实验没有高频实测, 因
此不能称精确物理验证。',
'',
'十分钟合同电量 q 按恒定供电功率均匀分配到各微步, 这是把原十分钟合同转为细步执行的补充假设。一个十分钟内可能先出
现充电、后出现紧急购电, 但二者在同一微步不能同时发生; 该十分钟的充电/放电方向仍不反转。',
'',
'所有方案的现金账单均按同一个实际十分钟端点均价结算, 即  $p \cdot g + 1.5p \cdot \text{增加量} - 0.5p \cdot \text{取销量} + 5p \cdot \text{紧急电量}$ 。没有
把微步价格与微步功率相乘引入另一种结算规则。末态共同库存价值保持一月确定的数值, 库存调整费只用于比较, 不是实际付款。',
'',
'策略|执行|现金费(元)|较本地十分钟增加(元)|库存调整费(元)|末 SOC(kWh)|紧急电(kWh)|电池损耗
(kWh)|',
'|---|---|---|---|---|---|---|',
for r in rows:
    lines.append(f'|{r["strategy"]} | {r["variant"]} | {r["cash_cost_yuan"]:.2f} |
{r["cash_difference_vs_mean"]:+.2f} | {r["inventory_adjusted_cost_yuan"]:.2f} | {r["soc_end_kwh"]:.3f} |
{r["emergency_kwh"]:.3f} | {r["battery_loss_kwh"]:.3f} |')
    lines+=['', '细化步长还改变了左端持平的积分近似。连续节点环境下, 步长 6 分钟导致的总电量差为  $6 \times (\text{首个瞬时功率} - \text{最后瞬}
时功率) / 120 \text{ kWh}$ , 相对于准确端点平均积分计算。程序对负载和光伏分别核验此恒等式: 从 1 分钟细化到 0.1 分钟, 该信号积分差缩
小十倍。费用变化还包含未知段内变化与冻结方向/放电预算的影响, 不等同于单纯积分误差。', '']
    for name, _ in planned_jobs(result_root):
        rr={r['variant']:r for r in rows if r['strategy']==name}
        if '0.1min-causal' in rr:
            delta=rr['0.1min-causal']['cash_cost_yuan']-rr['1min-causal']['cash_cost_yuan']
            inventory_delta=rr['0.1min-causal']['inventory_adjusted_cost_yuan']-rr['1min-causal']
['inventory_adjusted_cost_yuan']
            lines.append(f'- {name}: 0.1 分钟相对 1 分钟现金变化 {delta:+.3f} 元, 库存调整费变化
{inventory_delta:+.3f} 元。这是本轮细化稳定性结果, 不外推为全年比例。")
        lines+=['', '四条策略之间合同不同, 共同 6000 起点只能使本次局部执行条件一致, 不能把固定合同比较解释成重新优化后的完
整政策优劣。跨策略只比较相同步长, 不能用一条策略的 0.1 分钟费用与另一条的一分钟费用判优; 两种细步也不足以证明已经收敛到连
续时间精确结果。分钟结果不用于改写已冻结的开发期选择。',
'', '复现: 在工程根目录执行 `python code/q4-causal-study.py --results results/final`。输入主轨迹需已完
整生成; 输出包括十分钟轨迹、压缩微步轨迹、逐时签单揭示记录、物理核验与汇总。无需重新训练电价模型。']
        (ROOT/'notes/分钟因果执行敏感性.md').write_text('\n'.join(lines)+'\n', encoding='utf-8')
    return report

def run_study(result_root, data_dir=ROOT/'data', only=None):
    result_root=Path(result_root); jobs=planned_jobs(result_root)
    if only is not None: jobs=[(name, steps) for name, steps in jobs if name in only]
    data=load_inputs(data_dir)
    for name, steps in jobs:
        source=result_root/'main'/name; target=result_root/'causal-replay'/name
        if not (source/'summary.json').exists(): raise FileNotFoundError(f'Main run must finish first:
{source}')
        if (target/'summary.json').exists():
            r=json.loads((target/'summary.json').read_text())
            if r['fine_step_minutes']!=list(steps): raise ValueError('Existing replay has different fine
steps')
            if not r['checks_passed'] or r['start_day']!=24 or r['days']!=7 or
r['local_initial_soc']!=6000.:

```

```

        raise ValueError('Existing replay has a different local design or failed checks')
    if r['input_source_sha256'] != data.get('metadata', {}).get('source_sha256', {}):
        raise ValueError('Replay source inputs changed; use a new output directory')
    for filename, digest in r['source_hashes'].items():
        if hashlib.sha256((source/filename).read_bytes()).hexdigest() != digest: raise
ValueError('Replay source changed')
    for filename, digest in r['replay_code_sha256'].items():
        if hashlib.sha256((ROOT/'code'/filename).read_bytes()).hexdigest() != digest: raise
ValueError('Replay code changed')
    print(f'Reused matching replay: {name}', flush=True)
    continue
    print(f'Replaying {name}: {steps} minute(s)', flush=True)
    run_replay(data_dir, source, target, 24, 7, 6000., steps, data_override=data)
    print(f'Completed: {name}', flush=True)
    # Partial jobs may run concurrently; only a full sequential call or the
    # explicit summarize command writes the shared comparison artifacts.
    if only is None and all((result_root/'causal_replay'/name/'summary.json').exists() for name, _ in
planned_jobs(result_root)):
        return summarize(result_root)
    return {'partial': True, 'completed_requested': [name for name, _ in jobs]}

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--results', type=Path, default=ROOT/'results/final')
    parser.add_argument('--data', type=Path, default=ROOT/'data')
    parser.add_argument('--only', help='Optional comma-separated names as their main runs finish')
    parser.add_argument('--summarize-only', action='store_true')
    args = parser.parse_args()
    report = summarize(args.results) if args.summarize_only else run_study(args.results, args.data, None if
args.only is None else args.only.split(','))
    print(json.dumps({key: value for key, value in report.items() if key in
('checks_passed', 'partial', 'completed_requested')}, ensure_ascii=False))

```

74. question4/code/q4_conclusions.py

源文件 SHA-256: b1691a9f4e4b25f8e02e8049e38590d93c9f6a88bd5057f83fe9097c288264fb ; 代码行数: 110.


```

"""Build a compact narrative from completed, independently checked results."""
from pathlib import Path
import argparse, json

ROOT=Path(__file__).resolve().parents[1]

def read(path):return json.loads(Path(path).read_text(encoding='utf-8'))
def money(x):return f'{x:,.2f}'
def movement(x):return ('增加' if x ≥ 0 else '减少')+money(abs(x))+ '元'

def build_conclusions(results_dir=ROOT/'results/final', output=None):
    result=Path(results_dir).resolve()
    a=read(result/'analysis/analysis.json');r=read(result/'reports/报表与账本核验.json')
    audit=read(result/'independent_audit.json');ca=read(result/'causal_replay/comparison.json')
    causal_audit=read(result/'causal_replay/independent_audit.json')
    solver_check=read(result/'solver_check/summary.json')
    if not
all([a['complete'],r['complete'],audit['passed'],ca['checks_passed'],causal_audit['passed'],solver_check
['complete']]):
    raise RuntimeError('Narrative requires complete, audited real results')
    sel=a['selection'];names=[sel['selected_q42'],sel['selected_q43']]
    runs={x['strategy']:x for x in a['main_summary']};chosen=[runs[n] for n in names]
    mode=lambda s: '电价点预测+负载/PV场景' if s=='point' else '负载/PV/价格联合场景'
    lines=['# 第四问: 计算结果与结论', '',
        f"按照预先确定的开发期选择规则, 4-2采用**{mode(chosen[0]['price_mode'])}**; 4-3采用**{mode(chosen[1]
['price_mode'])}**, 在**{chosen[1]['trade_hours']}**启用PV预报并正式调单。两分支电池均保留MPC, 价格均在0、6、
12、18点更新。XGBoost价格模型是用户指定的冻结预测器。", '',
        '这不是先看全年谁最便宜再选谁: 只用2月17-23日七个已兑现开发窗口, 在模拟2月25日00:00冻结。之后310窗检验不改变冻
结选择。开发窗数量有限, 不能据此声称该选择对所有季节稳定最优。', '',
        '## 1. 完整费用和原模板', '',
        '|分支|334合同窗现金费/元|2月1日至12月31日自然日期间费/元|期末SOC/kWh|紧急电量/kWh|',
        '|———:——:——:——:|',
        for label,z in zip(('4-2 固定普通计划','4-3 允许指定时刻调单'),chosen):
            lines.append(f"|{label}|{money(z['full_cash_yuan'])}|{money(z['natural_period_cash_yuan'])}|
{z['final_soc_kwh']:,.3f}|{z['emergency_kwh']:,.3f}|")
            diff=chosen[1]['full_cash_yuan']-chosen[0]['full_cash_yuan']
            lines += ['|','f"4-3相对4-2的334合同窗总费{movement(diff)} ({100*abs(diff)/chosen[0]
['full_cash_yuan']:,.4f}%)。双方PV预测也不同, 因此这不是单独的调单净收益。相同PV版本下的控制比较见下文。", '',
                '两个原模板result4-2.xlsx、result4-3.xlsx和四指定日表均在reports目录。购电sheet按D00:10至D+1 00:10合同
窗, 储能/紧急sheet按自然日。首日00:00-00:10启动段与跨年尾段另列并完成对账; 上面两种范围的总费不能混用。合同总费含热身/
开发窗, 最后一段含估计右端点, 不是365日全年实测。', '',
                '主现金组成如下: 退款已从原费扣回, 违约金另加, 库存软罚不计真实电费。', '',
                '|分支|原计划费/元|净调整费/元|紧急费/元|', '|———:——:——:|']
        for label,z in zip(('4-2','4-3'),chosen):
            lines.append(f"|{label}|{money(z['original_plan_cost_yuan'])}|
{money(z['net_adjustment_cost_yuan'])}|{money(z['emergency_cost_yuan'])}|")
            lines += ['|','## 2. 点价格、联合价格和调单的对比', '',
                '这里点价格模式仍包含负载/PV场景。联合方案只额外加入同日配对的价格比例误差, 未额外叠加裕量或CVaR。全部策略重新签
原单, 各自连续运行, 因此测试初SOC可能不同; 另报共同库存价值校正, 避免忽略库存差异。', '',
                '|对照 (左减右, 负数左方便宜) |334窗现金差/元|310测试窗现金差/元|',
                '|———:——:|']
        for z in a['comparisons']:
            lines.append(f"|{z['comparison']}|{z['full_cash_difference_yuan']:+, .2f}|
{z['test_cash_difference_yuan']:+, .2f}|")
        for z in a['comparisons'][:2]:
            branch='4-2' if z['comparison'].startswith('4-2') else '4-3'
            lines.append(f"\n{branch}联合价格情景在310测试窗使现金费{movement(z['test_cash_difference_yuan'])}。
这是留出期结果, 不回写7开发窗的选择; 若与开发判断不同, 应解释为选择不稳定并扩大开发覆盖, 不能悄悄改名为事前最优。")
            lines += ['|','7日移动块重采样对平均日现金差给出下列95%区间; 3/14日块结果也保存在analysis中, 均为2000次。区间不是
价格预测区间, 单年季节性、跨年库存和开发选择误差限制其解释。', '',
                '|对比|平均日差/元|95%区间/元|', '|———:——:|']
        for z in a['bootstrap_intervals']:

```

```

        if z['block_days']==7 and z['metric']=='cash_cost_yuan':
            lines.append(f"|{z['comparison']}|{z['mean_daily_difference_yuan']:+,.3f}|
[{z['mean_difference_ci_lower_yuan']:+,.3f}, {z['mean_difference_ci_upper_yuan']:+,.3f}]|")
        lines += ['|', '## 3. 是否需要其他时刻PV预报和调单', '|',
f"价格模式固定为{chosen[1]['price_mode']}, 重新计算全部8个组合。价格更新权限共同保持0/6/12/18; 以下是策略整体比较, PV版本与调单机会随组合一起变化。星号表示开发期选中, 测试排名不改变它。", '|',
'|启用PV/调单时刻|334合同窗费/元|310测试窗费/元|期末SOC/kWh|', '|', '———:———:———:|']
        schedule_rows=sorted([z for z in a['main_summary'] if z['branch']=='4-3' and
z['price_mode']==chosen[1]['price_mode']],key=lambda x:(len(x['trade_hours']),x['trade_hours']))
        for z in schedule_rows:
            star='★' if z['strategy']==names[1] else ''
            lines.append(f"|{z['trade_hours']}|{star}|{money(z['full_cash_yuan'])}|
{money(z['test_cash_yuan'])}|{z['final_soc_kwh']:+,.3f}|")
            best=min(schedule_rows,key=lambda x:x['test_inventory_score_yuan'])
            lines += ['|', f"按测试库存校正回顾, 8组合中最低的是{best['trade_hours']}点, {'与开发期选择一致' if
best['strategy']==names[1] else '与开发期选择不同; 仅作事后诊断, 主交付仍保留原冻结选择'}。每个目标段只在最近启用时
刻正式调整一次, 是本次策略约束, 不是题面穷尽的所有可能政策。", '|',
'## 4. 参数与MPC敏感性', '|',
'|以下都是2月25日起共同6000初态的独立14窗试验, 改变初态的两组除外; 局部终止规则相同。不是把主334窗截取片段当基
准, 也不把14窗结论外推到全年。κ变化时评价的v仍固定为一月估计0.5163888889。', '|',
'|分支|变体|现金费/元|相对所参考的库存校正差/元|参考|', '|', '———:———:———:|']
        for z in a['sensitivity_summary']:
            lines.append(f"|{z['branch']}|{z['strategy'].split('_',1)[1]}|{money(z['cash_yuan'])}|
{z['inventory_score_change_yuan']:+,.2f}|{z['reference_strategy'].split('_',1)[1]}|")
            sensitivity={z['strategy']:z for z in a['sensitivity_summary']}
            for prefix, label in [('Q42', '4-2'), ('Q43', '4-3')]:
                greedy=sensitivity[prefix+'_greedy'];midnight=sensitivity[prefix+'_price_midnight_only']
                lines.append(f"\n{label}: 贪心相对MPC现金费{movement(greedy['cash_change_yuan'])}; 仅零点价格相对
0/6/12/18价格更新现金费{movement(midnight['cash_change_yuan'])}。原单随状态变化重优化, 这是完整局部策略对比, 不能
称同g消融; 分钟回放才锁定正式合同。")
            lines += ['|', '原二月底history56是上限, 实际只有23-36 (4-2) 或23-37 (4-3) 条完整历史。为检验真正的56窗, 追加8月
1-14的同6000初态检查: ', '|',
'|分支|历史上限/实际条数|14窗现金费/元|相对28窗的库存校正差/元|',
'|', '———:———:———:|']
        for z in a['history_check_summary']:
            lines.append(f"|{z['branch']}|{z['history_cap_days']}|{z['actual_history_paths_min']}|
{money(z['cash_yuan'])}|{z['inventory_score_change_from_28_yuan']:+,.2f}|")
            lines += ['|', '八月结果不用于重新选择原模型, 且与二月底试验不同季节, 不把两组现金水平直接作优劣比较。']
            lines += ['|', '## 5. 更严格的分钟执行与末端估计', '|',
'|主回放当前10分钟实际均值反馈是等效近似。四策略用原正式合同、共同6000初态独立重放7窗, 并与只读当前节点的分钟反馈
比较; 当时尚未正式签定的未来q不提前揭示。环境在十分钟实际节点间人工线性插值, 没有高频实测, 故不是精确物理验证。', '|',
'|策略|执行|7窗现金费/元|相对本地10分钟均值差/元|末SOC/kWh|', '|', '———:———:———:|']
        for z in ca['rows']:
            lines.append(f"|{z['strategy']}|{z['variant']}|{money(z['cash_cost_yuan'])}|
{z['cash_difference_vs_mean']:+,.2f}|{z['soc_end_kwh']:+,.3f}|")
            lines += ['|', '两条冻结选中分支还有0.1分钟细化, 信号左端积分差按解析式缩小10倍; 费用还受段内方向/放电预算限制影
响, 不能把全部差当积分误差。上表只支持7窗执行近似评估, 不用它重新选择年度合同模型。', '|',
'|最后跨年尾段固定已签g/q和入段SOC, 三种功率端点估计重新执行一步, 再组合三种实际价估计结算; 控制仍用原预测价。选
中方案的九组合范围: ', '|',
'|策略|总现金跨度/元|末SOC范围/kWh|', '|', '———:———:———:|']
        for z in a['endpoint_summary']:
            if z['strategy'] in names:
                lines.append(f"|{z['strategy']}|{z['cash_span_yuan']:+.6f}|{z['final_soc_min_kwh']:+.6f},
{z['final_soc_max_kwh']:+.6f}|")
            lines += ['|', '这里的跨度是最大费用减最小费用, 并非相对默认持平方案的单边变化; 两条选中方案相对默认值的最大绝对变化
都小于5.804元。', '|',
'## 6. 四个指定自然日', '|',
'|详细表1、表2、表3已分别写入四日Excel和论文表格Markdown; 不只给日总费。此处汇总便于核对, 所有日期均144个完整自
然日区间, 00:00首段取前一合同, SOC不重置。', '|',
'|日期|4-2现金费/元|4-3现金费/元|4-2紧急电/kWh|4-3紧急电/kWh|', '|', '———:———:———:|']
        for date, x in r['four_day_summary']['Q4-2'].items():
            y=r['four_day_summary']['Q4-3'][date]
            lines.append(f"|{date}|{money(x['cash_cost_yuan'])}|{money(y['cash_cost_yuan'])}|
{x['emergency_kwh']:+.4f}|{y['emergency_kwh']:+.4f}|")
            maxgap=max(z['solver_max_gap'] for z in a['main_summary'])
            unproven=sum(z['solver_unproven_calls'] for z in a['main_summary'])

```



```

lines += ['','## 7. 数值核验与结论适用范围','',
    f"独立审计通过{audit['audited_runs']}条正式校准/主/敏感性轨迹，共{audit['audited_intervals']:,}段：原始节点均值、订单时序、跨场景共享、SOC、能量、退费和费用均重算。主12策略合计{unproven}次求解调用未被原程序标记为已证明达到请求Gap，最大保存相对Gap为{100*maxgap:.6f}%；所有被执行计划都通过可行性核验。这个计数包括净目标接近零时的浮点比例放大，不能将最大百分比解释成全年费用误差或全部次数都是限时难解。详细状态保留，不将固定模式LP精修称为MILP全局最优。",'',
    f"独立抽查{solver_check['unique_frozen_states']}个固定状态、{solver_check['recorded_solver_calls']}次重解，其中{solver_check['forty_five_second_calls']}次延长至45秒并收紧Gap：最大绝对目标界差仅{solver_check['max_absolute_objective_bound_gap_yuan']:.3g}元，却可显示{100*solver_check['max_relative_gap']:.4f}%的相对值。原因是控制净目标近零，分母被程序限定为 $1\times 10^{-10}$ 。首动作相对主轨迹最大差{solver_check['max_action_difference_from_main_kwh']:.3g}kWh。这支持数值比例放大的解释，但不冒称定位并重解了所有未记录的控制调用。正式签单的Gap与剩余控制净目标的Gap应分开读，详见solver_check。",'',
    '结论是当前队伍预测、XGBoost价格输入、统一初态和供电期均价结算约定下的经验比较。有限开发窗、两阶段未来回补近似、10分钟均值反馈和单年数据，均限制推广。更换预测后必须重建历史误差并重新评估；队伍预测训练信息集仍需其代码记录佐证。原始文件和前三问冻结结果没有修改，本轮没有推送GitHub。','',
    '复现命令、依赖和分步流程见README；模型公式与外部原始方法出处见《第四问_模型与操作方法》。']
text='\n'.join(lines)+'\n'
destination=Path(output) if output else result/'reports/第四问_结果与结论.md'
destination.parent.mkdir(parents=True,exist_ok=True);destination.write_text(text,encoding='utf-8')
return {'output':str(destination),'selected':names,'complete':True}

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--results',type=Path,default=ROOT/'results/final')
    args=p.parse_args();print(json.dumps(build_conclusions(args.results),ensure_ascii=False))

```

75. question4/code/q4_figures.py

源文件 SHA-256: 19a1f5cc4452d340e3974fd67f67a71ec43d0e0f0698b1f4d1dc4691de1c5c14 ; 代码行数: 316。

```

"""Read-only scientific figures and summary tables for Q4 dispatch experiments.

Selection is read from the calibration freeze; held-out rankings and parameter
sensitivities never replace the frozen displayed policies. No numerical-core imports.
"""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
import argparse
import hashlib
import json
import os

import numpy as np
import pandas as pd

ROOT=Path(__file__).resolve().parents[1]
DATES=('2025-03-20','2025-06-21','2025-09-23','2025-12-21')
BRANCH_LABEL={'4-2':'Q4-2','4-3':'Q4-3'}
SENS_LABEL={'reference':'冻结方案参考','history14':'残差历史14天','history56':'残差历史56天','scenes4':'4个情景','scenes16':'16个情景',
            'terminal_half':'软末态权重×0.5','terminal_double':'软末态权重×2','seed13':'随机种子13','seed14':'随机种子14',
            'price_midnight_only':'仅零点电价预报','greedy':'电池贪心执行','initial1200':'初态1200kWh','initial10800':'初态10800kWh',
            'joint_mean':'联合价格均值','joint_shuffled':'打乱价格配对','joint_reference':'联合情景参考'}

def read_json(path):return json.loads(Path(path).read_text(encoding='utf-8'))
def digest(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()
def write_json(path,value):Path(path).write_text(json.dumps(value,ensure_ascii=False,indent=2,default=str),encoding='utf-8')
def close(a,b,label,tol=1e-4):
    if not np.isclose(a,b,rtol=1e-10,atol=tol):raise ValueError(f'{label}: {a} ≠ {b}')

@dataclass
class Run:
    name:str
    path:Path
    summary:dict
    daily:pd.DataFrame

    @property
    def branch(self):return self.summary['config']['branch']
    @property
    def mode(self):return self.summary['config']['price_mode']
    @property
    def schedule(self):return tuple(self.summary['config']['trade_hours'])

@dataclass
class Study:
    root:Path
    selection:dict
    main:dict[str,Run]
    sensitivity:dict[str,Run]
    source_hashes:dict[str,str]

    def selected(self,branch):return self.main[self.selection['selected_q42' if branch=='4-2' else 'selected_q43']]

```

```

def load_run(path, expected_days=None):
    path=Path(path);summary=read_json(path/'summary.json')
    if not summary.get('complete'):raise ValueError(f'Incomplete result: {path}')
    daily=pd.read_csv(path/'daily.csv')
    required=
('window_date', 'cash_cost_yuan', 'inventory_adjusted_cost_yuan', 'soc_start_kwh', 'soc_end_kwh', 'emergency_
kwh')
    if any(k not in daily for k in required):raise ValueError(f'Missing daily ledger fields: {path}')
    dates=pd.to_datetime(daily.window_date)
    if dates.duplicated().any() or not dates.is_monotonic_increasing:raise ValueError('Daily dates must
be sorted and unique')
    if len(daily)!=summary['windows'] or (expected_days is not None and len(daily)!=expected_days):raise
ValueError('Incomplete daily coverage')
    if len(dates)>1 and not (dates.diff().dropna()==pd.Timedelta(days=1)).all():raise ValueError('Daily
gaps in continuous run')
    close(daily.cash_cost_yuan.sum(),summary['cash_cost_yuan'],'summary cash')
    close(daily.inventory_adjusted_cost_yuan.sum(),summary['inventory_adjusted_cost_yuan'],'summary
inventory evaluation')
    if len(daily)>1 and not np.allclose(daily.soc_end_kwh.to_numpy()[:-1],daily.soc_start_kwh.to_numpy()
[1:],atol=1e-5,rtol=0):raise ValueError('Daily SOC is not continuous')
    return Run(path.name,path,summary,daily)

def _load_design_runs(directory, expected_days=None):
    directory=Path(directory);design=read_json(directory/'design.json');runs={}
    for job in design['jobs']:
        name=job['name'];runs[name]=load_run(directory/name,expected_days)
    if len(runs)!=len(design['jobs']):raise ValueError('Duplicate design names')
    return runs

def load_study(results_dir):
    root=Path(results_dir);selection=read_json(root/'selection.json')
    if selection.get('model_or_schedule_reselection_from_test') is not False:raise ValueError('Need
explicit pre-test selection freeze')
    main=_load_design_runs(root/'main',334);sensitivity=_load_design_runs(root/'sensitivity',14)
    for branch,key in [('4-2','selected_q42'),('4-3','selected_q43')]:
        if selection[key] not in main:raise ValueError('Frozen selected main run is missing')
        choice=selection['branch_selections'][branch];run=main[selection[key]]
        if run.branch!=branch or run.mode!=choice['price_mode'] or
run.schedule!=tuple(choice['trade_hours']):raise ValueError('Frozen choice/config mismatch')
        if run.daily.window_date.iloc[0]!='2025-02-01' or run.daily.window_date.iloc[-1]!='2025-12-
31':raise ValueError('Expected February-December contracts')
        paths=[root/'selection.json',root/'main/design.json',root/'sensitivity/design.json']
        for run in
list(main.values())+list(sensitivity.values()):paths.extend([run.path/'summary.json',run.path/'daily.csv
'])
    return Study(root,selection,main,sensitivity,{str(path):digest(path) for path in paths})

def fee_components(study):
    out=[]
    for branch in ('4-2','4-3'):
        run=study.selected(branch);trace=pd.read_csv(run.path/'trace.csv');study.source_hashes[str(run.path/'tra
ce.csv')]=digest(run.path/'trace.csv')
        fields=
('plan_cost_yuan', 'increase_cost_yuan', 'refund_yuan', 'penalty_yuan', 'emergency_cost_yuan')
        row={k:float(trace[k].sum()) for k in fields}

        row.update(branch=branch,strategy=run.name,cash_cost_yuan=run.summary['cash_cost_yuan'],soc_start_kwh=ru
n.summary['soc_start_kwh'],soc_end_kwh=run.summary['soc_end_kwh'])
        close(row['plan_cost_yuan']+row['increase_cost_yuan']-
row['refund_yuan']+row['penalty_yuan']+row['emergency_cost_yuan'],row['cash_cost_yuan'],'fee
decomposition')

```

```

        out.append(row)
    return pd.DataFrame(out)

def monthly_main(study):
    records=[]
    for branch in ('4-2', '4-3'):
        run=study.selected(branch);frame=run.daily.copy();frame['month']=frame.window_date.str[:7]
        for month,rows in frame.groupby('month',sort=True):

records.append(dict(branch=branch,strategy=run.name,month=month,windows=len(rows),cash_cost_yuan=float(r
ows.cash_cost_yuan.sum()),inventory_adjusted_cost_yuan=float(rows.inventory_adjusted_cost_yuan.sum()),em
ergency_kwh=float(rows.emergency_kwh.sum()))
    return pd.DataFrame(records)

def schedule_comparison(study):
    choice=study.selection['branch_selections']['4-
3'];mode=choice['price_mode'];selected=study.selection['selected_q43'];records=[]
    development=[r for r in study.selection['development'] if r['branch']=='4-3' and
r['price_mode']==mode]
    if len(development)!=8:raise ValueError('Eight Q4-3 schedules required within frozen price mode')
    for dev in development:
        run=study.main[dev['strategy']];test=run.daily[run.daily.window_date.between('2025-02-25','2025-
12-31')]
        if len(test)!=310:raise ValueError('Held-out schedule comparison requires 310 windows')
        records.append(dict(strategy=run.name,schedule='/'.join(f'{h:02d}' for h in
run.schedule),price_mode=mode,selected_before_test=run.name==selected,

development_windows=7,development_inventory_score_yuan=float(dev['score']),development_cash_yuan=float(d
ev['cash']),

development_daily_score_yuan=float(dev['score']/7,test_windows=310,test_cash_yuan=float(test.cash_cost_
yuan.sum()),

test_inventory_score_yuan=float(test.inventory_adjusted_cost_yuan.sum()),test_daily_score_yuan=float(tes
t.inventory_adjusted_cost_yuan.mean()))
    frame=pd.DataFrame(records)
    frame['development_rank']=frame.development_inventory_score_yuan.rank(method='min').astype(int)
    frame['test_rank_retrospective']=frame.test_inventory_score_yuan.rank(method='min').astype(int)
    return frame.sort_values(['development_rank','strategy']).reset_index(drop=True)

def paired_price_modes(study):
    records=[];summaries=[]
    for branch in ('4-2', '4-3'):
        schedule=study.selected(branch).schedule
        candidates={r.mode:r for r in study.main.values() if r.branch==branch and r.schedule==schedule
and r.mode in ('point','joint') and r.summary['config'].get('pv_source','supplied')==supplied}
        if set(candidates)!={'point','joint'}:raise ValueError(f'Missing matched point/joint experiment:
{branch}, {schedule}')
        point,joint=candidates['point'],candidates['joint']
        if not point.daily.window_date.equals(joint.daily.window_date):raise ValueError('Paired runs use
different dates')
        if point.summary['soc_start_kwh']!=joint.summary['soc_start_kwh']:raise ValueError('Paired runs
start with different stock')
        a=point.daily;b=joint.daily;delta=b.cash_cost_yuan.to_numpy()-
a.cash_cost_yuan.to_numpy();delta_inv=b.inventory_adjusted_cost_yuan.to_numpy()-
a.inventory_adjusted_cost_yuan.to_numpy()
        for i,date in enumerate(a.window_date):

records.append(dict(branch=branch>window_date=date,point_strategy=point.name,joint_strategy=joint.name,

point_cash_yuan=float(a.cash_cost_yuan.iloc[i]),joint_cash_yuan=float(b.cash_cost_yuan.iloc[i]),joint_mi
nus_point_cash_yuan=float(delta[i]),

```

```

cumulative_cash_difference_yuan=float(delta[:i+1].sum()),joint_minus_point_inventory_score_yuan=float(delta_inv[i]),

cumulative_inventory_difference_yuan=float(delta_inv[:i+1].sum()),joint_minus_point_emergency_kwh=float(b.emergency_kwh.iloc[i]-a.emergency_kwh.iloc[i]))

summaries.append(dict(branch=branch,point_strategy=point.name,joint_strategy=joint.name,windows=len(delta),joint_minus_point_cash_yuan=float(delta.sum()),

joint_minus_point_inventory_score_yuan=float(delta_inv.sum()),joint_cheaper_windows=int((delta<=1e-5).sum()),point_cheaper_windows=int((delta>1e-5).sum()),

joint_cash_difference_percent=100*float(delta.sum())/float(a.cash_cost_yuan.sum()),point_end_soc=point.summary['soc_end_kwh'],joint_end_soc=joint.summary['soc_end_kwh']))

    return pd.DataFrame(records),pd.DataFrame(summaries)

def sensitivity_comparison(study):
    records=[]
    for branch in ('4-2','4-3'):
        prefix='Q'+branch.replace('-','')+ '_';reference=study.sensitivity[prefix+'reference']
        joint_reference=study.sensitivity.get(prefix+'joint_reference',reference)
        for name,run in study.sensitivity.items():
            if run.branch!=branch:continue
            label=name.removeprefix(prefix);base=reference

s=run.summary;r=base.summary;cash=float(s['cash_cost_yuan']);score=float(s['inventory_adjusted_cost_yuan'])

            if not run.daily.window_date.equals(base.daily.window_date):raise ValueError('Sensitivity dates are not matched')

row=dict(branch=branch,experiment=name,label=label,label_zh=SENS_LABEL.get(label,label),reference=base.name,windows=len(run.daily),

first_date=run.daily.window_date.iloc[0],last_date=run.daily.window_date.iloc[-1],cash_cost_yuan=cash,inventory_score_yuan=score,
        cash_delta_yuan=cash-r['cash_cost_yuan'],inventory_delta_yuan=score-r['inventory_adjusted_cost_yuan'],
        inventory_delta_percent=100*(score-r['inventory_adjusted_cost_yuan'])/r['inventory_adjusted_cost_yuan'],

initial_soc=s['soc_start_kwh'],final_soc=s['soc_end_kwh'],emergency_kwh=s['emergency_kwh'])
            if label in ('joint_mean','joint_shuffled'):

row.update(joint_layer_reference=joint_reference.name,joint_layer_inventory_delta_yuan=score-joint_reference.summary['inventory_adjusted_cost_yuan'])
            records.append(row)

    return pd.DataFrame(records)

def selected_day_profiles(study):
    profiles=[]
    for branch in ('4-2','4-3'):
        run=study.selected(branch);trace=pd.read_csv(run.path/'trace.csv')
        study.source_hashes[str(run.path/'trace.csv')]=digest(run.path/'trace.csv')
        trace['date']=trace.interval_start.str[:10]
        for date in DATES:
            rows=trace[trace.date==date].copy()
            if len(rows)!=144:raise ValueError('Specified-day plot requires full natural day')
            expected=pd.date_range(date,periods=144,freq='10min')
            if not
np.array_equal(pd.to_datetime(rows.interval_start).to_numpy(),expected.to_numpy()):raise
ValueError('Specified day contains shifted slots')
            rows['branch']=branch;profiles.append(rows)

    return pd.concat(profiles,ignore_index=True)

```

```

def build_tables(study):
    paired,paired_summary=paired_price_modes(study)
    return {'主策略费用组成':fee_components(study),'主策略逐月费用':monthly_main(study),'八组合开发与留出比较':schedule_comparison(study),
            '电价点值与联合场景逐窗配对':paired,'电价点值与联合场景比较':paired_summary,'参数敏感性比较':sensitivity_comparison(study),'指定四日逐段图形数据':selected_day_profiles(study)}

def _plot_setup(out_dir):
    os.environ.setdefault('MPLCONFIGDIR',str(Path(out_dir)/'.mplconfig'))
    import matplotlib
    matplotlib.use('Agg')
    from matplotlib import pyplot as plt,font_manager
    candidate=Path('/System/Library/Fonts/Supplemental/Songti.ttc')
    if
candidate.exists():font_manager.fontManager.addfont(str(candidate));font=font_manager.FontProperties(fname=str(candidate)).get_name()
    else:
        fonts=[f.name for f in font_manager.fontManager.ttflist if any(x in f.name for x in ('Songti','SimSun','Noto Serif CJK'))]

        if not fonts:raise RuntimeError('Install Songti, SimSun, or Noto Serif CJK for legible Chinese plots')
        font=fonts[0]

plt.rcParams.update({'font.family':font,'font.size':11,'axes.unicode_minus':False,'axes.spines.top':False,'axes.spines.right':False,
'axes.linewidth':.7,'pdf.fonttype':42,'savefig.dpi':240,'xtick.direction':'out','ytick.direction':'out',
'axes.labelpad':8})
    return plt,font

def create_figures(study,tables,out_dir):
    out_dir=Path(out_dir);out_dir.mkdir(parents=True,exist_ok=True);plt,font=_plot_setup(out_dir)
    import matplotlib.dates as mdates
    manifest=[];colors={'4-2':'#4D718C','4-3':'#467E76'}
    def save(fig,name,caption,family):

fig.savefig(out_dir/(name+'.png'),bbox_inches='tight',facecolor='white');fig.savefig(out_dir/(name+'.pdf'),bbox_inches='tight',facecolor='white');plt.close(fig)

    manifest.append(dict(stem=name,family=family,caption=caption,font=font,png_dpi=240,in_figure_title=False))
    # 1. Signed refund is shown below zero instead of silently omitted.
    fees=tables['主策略费用组成'];fig,ax=plt.subplots(figsize=(10.8,4.5),layout='constrained')
    fields=[('plan_cost_yuan','原计划费','#829AA8'),('increase_cost_yuan','增购费','#5E8D81'),
('penalty_yuan','违约金','#B5A492'),('emergency_cost_yuan','紧急费','#BE8E61')]
    pos=np.zeros(2)
    for field,label,color in fields:

values=fees[field].to_numpy()/10000;ax.barh([0,1],values,left=pos,label=label,color=color,height=.48);pos+=values
        refund=-fees.refund_yuan.to_numpy()/10000;ax.barh([0,1],refund,label='取消退费',color='#77818A',hatch='///',height=.48)
        ax.set_yticks([0,1],['Q4-2主策略','Q4-3主策略']);ax.invert_yaxis();ax.axvline(0,color='#444444',linewidth=.7);ax.set_xlabel('334合同窗口费用组成(万元)')
        for i,row in fees.iterrows():ax.text(pos[i]+max(pos)*.025,i,f"净现金{row.cash_cost_yuan/10000:.2f}",va='center',fontsize=10)
        ax.set_xlim(min(refund)*1.15,max(pos)*1.30);ax.legend(ncol=3,frameon=False,loc='upper center',bbox_to_anchor=(.5,1.28));ax.grid(axis='x',alpha=.16)
        save(fig,'01_selected_fee_components','两主策略的334个合同窗口费用;取消退费为负项,库存评价和启动段另列。两分支输入信息不同,跨分支差额不能全部归因于调单权限。',1)
    # 2. Monthly cash, not the optimization objective.

```



```

month=tables['主策略逐月费用'];fig,ax=plt.subplots(figsize=(10.8,4.6),layout='constrained')
for branch in ('4-2','4-3'):
    rows=month[month.branch==branch];ax.plot([int(v[-2:]) for v in
rows.month],rows.cash_cost_yuan/10000,color=colors[branch],marker='o',markersize=4,label=BRANCH_LABEL[branch],linewidth=1.8)
    ax.set_xticks(range(2,13));ax.set_xlabel('合同发行月份');ax.set_ylabel('实际合同费用(万元)');ax.legend(frameon=False);ax.grid(axis='y',alpha=.2)
    save(fig,'02_monthly_selected_cost','各月按合同发行日分组,比较两项事先选定主策略的实际现金费,不是自然日费用,也不据单月最低重选策略。',2)
# 3. Same schedule order in development and held-out panels.
schedules=tables['八组合开发与留出比较'];fig,axes=plt.subplots(1,2,figsize=(12,5.6),sharey=True,layout='constrained')
labels=[f'{row.schedule} {'*' if row.selected_before_test else ''}' for row in
schedules.itertuples()]
for ax,column,xlabel in [(axes[0],'development_daily_score_yuan','开发7窗日均库存评价(万元/窗)'),
(axes[1],'test_daily_score_yuan','留出310窗日均库存评价(万元/窗)')]:
    vals=schedules[column].to_numpy()/10000;ax.barh(np.arange(8),vals,color=['#467E76' if x else
'#9AAEB9' for x in schedules.selected_before_test],height=.55)
    ax.set_yticks(range(8),labels);ax.invert_yaxis() if not ax.yaxis_inverted() else
None;ax.set_xlabel(xlabel);ax.grid(axis='x',alpha=.18)
    ax.set_xlim(0,max(vals)*1.19)
    ranks=schedules.development_rank if column.startswith('development') else
schedules.test_rank_retrospective
    for i,(val,rank) in enumerate(zip(vals,ranks)):ax.text(val+max(vals)*.016,i,f'{val:.3f} #{rank}',va='center',fontsize=9)
    axes[0].set_ylabel('启用调单时刻(*为事先选定)');axes[1].tick_params(labelleft=True)
    save(fig,'03_eight_schedule_development_holdout','同一个冻结价格模式下的8种Q4-3调单组合。按开发期顺序排列;右图排名只是留出期回顾,绿色及星号始终表示开发期冻结的主策略。两期长度不同,画每窗平均的共同库存评价,非真实电费。',3)
# 4. Four independent 3x1 panels; no unreadable twelve-panel page.
profiles=tables['指定四日逐段图形数据']
for date in DATES:
    fig,axes=plt.subplots(3,1,figsize=(10.8,8.8),sharex=True,layout='constrained');h=np.arange(144)/6
    selected_rows={branch:profiles[(profiles.branch==branch)&(profiles.date==date)] for branch in
('4-2','4-3')}
    for branch,rows in selected_rows.items():
        soc=np.r_[rows.soc_start_kwh.iloc[0],rows.soc_end_kwh.to_numpy()]
axes[0].plot(np.arange(145)/6,soc,color=colors[branch],label=BRANCH_LABEL[branch],linewidth=1.7)
    shift=.035 if branch=='4-2' else .11
axes[1].bar(h+shift,rows.emergency_kwh,width=.065,color=colors[branch],label=BRANCH_LABEL[branch])
    for bound in (1200,10800):axes[0].axhline(bound,color='#A5AAD',linestyle=':',linewidth=.9)
    axes[0].set_ylim(850,11150);axes[0].set_ylabel('储电量(kWh)');axes[0].legend(frameon=False,ncol=2)
    axes[1].set_ylabel('每10分钟紧急电量(kWh)')
    peak_emergency=float(profiles[profiles.interval_start.str[:10]==date].emergency_kwh.max())
    axes[1].set_ylim(0,max(1.,peak_emergency*1.12))
    if peak_emergency<1e-7:axes[1].text(.5,.5,'两项主策略当日均无紧急购电',transform=axes[1].transAxes,ha='center',va='center',color='#58636B')
    left=selected_rows['4-2'];right=selected_rows['4-3']
    if not np.allclose(left.price_yuan_per_kwh,right.price_yuan_per_kwh,atol=1e-12,rtol=0):raise
ValueError('Selected branches do not share actual prices')
    axes[2].plot(h,left.price_yuan_per_kwh,color='#41474B',linewidth=1.4,label='实际结算区间价')
    same=np.allclose(left.price_forecast_yuan_per_kwh,right.price_forecast_yuan_per_kwh,atol=1e-12,rtol=0)
    axes[2].plot(h,left.price_forecast_yuan_per_kwh,color=colors['4-2'],linestyle='--',linewidth=1.3,label='决策时采用的预测价' if same else 'Q4-2决策预测价')
    if not same:axes[2].plot(h,right.price_forecast_yuan_per_kwh,color=colors['4-3'],linestyle='-',linewidth=1.3,label='Q4-3决策预测价')
    axes[2].set_ylabel('区间电价(元/kWh)');axes[2].legend(frameon=False,ncol=3,fontsize=10)
    axes[-1].set_xticks(range(0,25,4),[f'{v:02d}:00' for v in range(0,25,4)]);axes[-1].set_xlabel('自然日执行区间起点 / 储电状态时刻');axes[-1].set_xlim(0,24)
    for ax in axes:ax.grid(axis='y',alpha=.17)
    save(fig,'04_'+date.replace('-','')+ '_state_emergency_price',f'{date}自然日两主策略的实际SOC、紧急电量及执行时采用的预测/结算区间价格。00:00首段取前一合同;SOC虚线为1200和10800kWh边界,价格为区间端点均值,不是原始瞬时

```



```

表。',4)
# 5. Independent continuously run policies, not forced identical g or state.
paired=tables['电价点值与联合场景逐窗配对'];fig,axes=plt.subplots(1,2,figsize=
(12,4.6),layout='constrained')
for ax,branch in zip(axes,('4-2','4-3')):
    rows=paired[paired.branch==branch];dates=pd.to_datetime(rows.window_date)

ax.plot(dates,rows.cumulative_cash_difference_yuan/10000,color=colors[branch],linewidth=1.8,label='实际现
金差')
    ax.plot(dates,rows.cumulative_inventory_difference_yuan/10000,color=colors[branch],linestyle='--
',linewidth=1.3,label='共同库存评价差')
    ax.axhline(0,color='#666666',linewidth=.8);ax.set_ylabel(f'{BRANCH_LABEL[branch]} 联合 - 点价累计差
(万元)');ax.set_xlabel('合同发行日期')

ax.set_xlim(dates.iloc[0],dates.iloc[-1]);ax.xaxis.set_major_locator(mdates.MonthLocator(bymonth=
[2,4,6,8,10,12]));ax.xaxis.set_major_formatter(mdates.DateFormatter('%m
月'));ax.legend(frameon=False);ax.grid(axis='y',alpha=.17)
    save(fig,'05_point_joint_paired_difference','同一分支、相同调单时刻和负载/光伏信息,比较电价点值与同日联合价格
情景;两边均保留负载/光伏情景。负值为联合价格方案累计费用较低。各自从共同初态连续运行,不强制相同购电或逐日SOC。',5)
# 6. Common inventory valuation keeps initial-state comparisons interpretable.
sens=tables['参数敏感性比较'];fig,axes=plt.subplots(1,2,figsize=(12.6,7.8),layout='constrained')
for ax,branch in zip(axes,('4-2','4-3')):
    rows=sens[(sens.branch==branch)&
(sens.label!='reference')].reset_index(drop=True);positions=np.arange(len(rows))

values=rows.inventory_delta_percent.to_numpy();ax.barh(positions,values,color=colors[branch],height=.55,
alpha=.85)

ax.set_yticks(positions,rows.label_zh);ax.invert_yaxis();ax.axvline(0,color='#4C5357',linewidth=.7)
    ax.set_xlabel(f'{BRANCH_LABEL[branch]} 相对冻结参考的库存评价变化 (%)');ax.grid(axis='x',alpha=.17)
    save(fig,'06_parameter_sensitivity','两分支分别在同一14个发行窗口中检验参数及执行方式;每组独立从明确初态运行。
以共同库存评价衡量初态差,负值为评价费用下降。均值/打乱联合价格等实验不是事后调参依据,条件联合参考差另列CSV。',6)

result=dict(family=6,figure_files=len(manifest),figures=manifest,selected_q42=study.selection['
selected_q42'],selected_q43=study.selection['selected_q43'],selection_changed=False)
    write_json(out_dir/'figure_manifest.json',result)
    (out_dir/'图注.md').write_text('\n\n'.join(f'图{row['family']}' {row['stem']} : {row['caption']}") for
row in manifest)+'\n',encoding='utf-8')
    return result

def build_figures(results_dir=ROOT/'results/final',out_dir=None,tables_dir=None):
    results_dir=Path(results_dir);out_dir=Path(out_dir) if out_dir else
results_dir/'figures';tables_dir=Path(tables_dir) if tables_dir else results_dir/'reports/图形数据'

study=load_study(results_dir);tables=build_tables(study);tables_dir.mkdir(parents=True,exist_ok=True)
    for name,frame in tables.items():frame.to_csv(tables_dir/(name+'.csv'),index=False,encoding='utf-8-
sig')
    manifest=create_figures(study,tables,out_dir)
    for path,old in study.source_hashes.items():
        if digest(path)!=old:raise RuntimeError('Figure generation changed an input')

summary=dict(complete=True,selected_q42=study.selection['selected_q42'],selected_q43=study.selection['se
lected_q43'],

model_or_schedule_reselected=False,main_strategy_count=len(study.main),sensitivity_count=len(study.sensi
tivity),figure_manifest=manifest,
    point_joint_comparison=tables['电价点值与联合场景比
较'].to_dict(orient='records'),source_sha256=study.source_hashes,source_files_unchanged=True)
    write_json(tables_dir/'图形数据核验.json',summary)
    return summary

if __name__=='__main__':
    parser=argparse.ArgumentParser(description=__doc__);parser.add_argument('--
results',type=Path,default=ROOT/'results/final');parser.add_argument('--

```

```
out', type=Path); parser.add_argument('--tables', type=Path)

args=parser.parse_args(); value=build_figures(args.results, args.out, args.tables); print(json.dumps({'complete': True, 'figure_families': 6, 'figure_files': value['figure_manifest']
['figure_files'], 'selected_q42': value['selected_q42'], 'selected_q43': value['selected_q43']} , ensure_ascii=False))
```

76. question4/code/q4_four_day_review.py

源文件 SHA-256: acab1e3d58e6232bf276c1c407c020e5d8bd2c8e19fad8376ed280c28cf07d70 ; 代码行数: 91。

```

"""Independent selected-day/template cell review, without report-builder helpers."""
import argparse
import json
from pathlib import Path
import pandas as pd
from openpyxl import load_workbook

ROOT=Path(__file__).resolve().parents[1]
DATES=('2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21')

def sheets(path):
    book=load_workbook(path,read_only=True,data_only=True)
    try:return {s.title:list(s.values) for s in book}
    finally:book.close()

def review_four_days(results_dir=ROOT/'results/final'):
    root=Path(results_dir).resolve();out=root/'reports'
    selected=json.loads((root/'selection.json').read_text());checks=[];cells=0;worst=0.
    def near(a,b,label):
        nonlocal cells,worst
        error=abs(float(a)-float(b));cells+=1;worst=max(worst,error)
        if not error<1e-5:raise AssertionError((label,a,b,error))
    def cell(s,row,col):return s[row-1][col-1]
    for branch,key in [('Q4-2','selected_q42'),('Q4-3','selected_q43')]:
        name=selected[key];trace=pd.read_csv(root/'main'/name/'trace.csv')
        trace['start']=pd.to_datetime(trace.interval_start)
        paper=sheets(out/f'第四问_{branch}_指定四日填表.xlsx')
        template=sheets(out/(f'result4-2.xlsx' if branch=='Q4-2' else 'result4-3.xlsx'))
        for k,date in enumerate(DATES):
            day=trace[trace.start.dt.strftime('%Y-%m-%d')==date].copy()
            if len(day)!=144:raise AssertionError('Required natural date incomplete')
            if day.start.iloc[0]!=pd.Timestamp(date) or
            day.start.iloc[-1]!=pd.Timestamp(date)+pd.Timedelta(hours=23,minutes=50):raise AssertionError('Wrong
            natural-day boundaries')
            if day.window_date.iloc[0]!=str((pd.Timestamp(date)-pd.Timedelta(days=1)).date()):raise
            AssertionError('First interval must use prior contract')
            s=paper['表1_指定购电'];base=1+8*k;quantity='g_kwh' if branch=='Q4-2' else 'q_kwh'
            for pos,hour in enumerate((10,12,14,16,18,20)):
                near(cell(s,base+2+pos//3,2+2*
                (pos%3)),day.loc[day.start.dt.hour.eq(hour)&day.start.dt.minute.eq(0),quantity].iloc[0],f'{branch}
                {date} hour{hour}'))
                for row,col,value,label in [(base+4,2,day[quantity].sum(),'daily ordinary'),
                (base+4,4,day.cash_cost_yuan.sum(),'daily cash'),
                (base+4,6,day.emergency_kwh.sum(),'daily emergency'),
                (base+5,2,day[quantity].sum()+day.emergency_kwh.sum(),'total purchased'),
                (base+5,4,day.plan_cost_yuan.sum(),'plan cash'),
                (base+5,6,day.emergency_cost_yuan.sum(),'emergency cash')]:
                    near(cell(s,row,col),value,label)
            for col,field in [(2,'increase_cost_yuan'),(4,'refund_yuan'),
            (6,'penalty_yuan')]:near(cell(s,base+6,col),day[field].sum(),field)
            s=paper['表2_充放储电'];base=1+7*k
            for block in range(6):
                rows=day[day.start.dt.hour.between(block*4,block*4+3)]
                rr=base+2+block//2;cc=2+3*(block%2)
                near(cell(s,rr,cc),rows.charge_kwh.sum(),'block charge')
                near(cell(s,rr,cc+1),rows.discharge_kwh.sum(),'block discharge')
                near(cell(s,base+5,2),day.soc_start_kwh.iloc[0],'natural midnight SOC')
                near(cell(s,base+5,5),day.soc_end_kwh.iloc[-1],'natural 24 SOC')
                events=[];current=None
            for _,row in day.iterrows():
                if row.emergency_kwh>1e-5:
                    if current is None:current=

```

```

[row.interval_start,row.interval_end,float(row.emergency_kwh)]
        else:current[1]=row.interval_end;current[2]+=row.emergency_kwh
        elif current is not None:events.append(current);current=None
    if current is not None:events.append(current)
    s=paper['表3_连续紧急购电'];col=2*k+1
    if not events:
        if cell(s,4,col)≠'无紧急购电':raise AssertionError('Missing no-emergency declaration')
        near(cell(s,4,col+1),0,'no emergency')
    else:
        for j,(a,b,e) in enumerate(events):
            label=a[11:16]+'-'+('24:00' if b[:10]≠date else b[11:16])
            if cell(s,4+j,col)≠label:raise AssertionError((date,label,cell(s,4+j,col)))
            near(cell(s,4+j,col+1),e,'emergency episode')
        contract=trace[trace.window_date==date]
        rr=(pd.Timestamp(date)-pd.Timestamp('2025-02-01')).days+2;s=template['计划购电量']
        for t,v in enumerate(contract.g_kwh):near(cell(s,rr,t+2),v,'original template g')
        near(cell(s,rr,146),contract.g_kwh.sum(),'original template total')
        near(cell(s,rr,147),(contract.cash_cost_yuan if branch=='Q4-2' else
contract.plan_cost_yuan).sum(),'original template cash')
        if branch=='Q4-3':
            s=template['调整购电量']
            for t,v in enumerate(contract.q_kwh):near(cell(s,rr,t+2),v,'adjusted template q')
            near(cell(s,rr,146),contract.q_kwh.sum(),'adjusted template total')
            near(cell(s,rr,147),contract.cash_cost_yuan.sum(),'adjusted total cash once')
        checks.append({'branch':branch,'strategy':name,'natural_date':date,'intervals':144,

'cash_yuan':float(day.cash_cost_yuan.sum()),'emergency_kwh':float(day.emergency_kwh.sum()),
        'episodes':len(events),'first_interval_from_previous_contract':True})
    result={'passed':True,'checked_numeric_cells':cells,'maximum_difference':worst,'days':checks,
        'scope':'Independent from report builder: six specified slots, all six storage blocks, natural
midnight states, merged emergency episodes, and 144 g/q cells in original templates on all four required
dates; full-template audit remains separate.'}
    (out/'独立四日与关键数核
验.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
    return result

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--
results',type=Path,default=ROOT/'results/final');a=p.parse_args()
    r=review_four_days(a.results)
    print(json.dumps({k:r[k] for k in
('passed','checked_numeric_cells','maximum_difference')},ensure_ascii=False))

```

77. question4/code/q4_history_check.py

源文件 SHA-256: ffd1c21800cbf409b814828f26ee30a1efccbbdeea529ae25002c1adb18a647 ; 代码行数: 58。

```

"""Additional history-cap verification where all 56 old paths are available.

The original Feb25--Mar10 sensitivity cannot fill a 56-window residual cap.
This supplementary Aug1--14 experiment checks that limitation without changing
the frozen branch choices, price model or any original sensitivity run.
"""
from __future__ import annotations

import argparse
import datetime as dt
import json
from pathlib import Path

from q4_audit import audit_selection
from q4_study import run_jobs

ROOT=Path(__file__).resolve().parents[1]
START_DAY=(dt.date(2025,8,1)-dt.date(2025,2,1)).days

def history_jobs(selection):
    jobs=[]
    for branch in ('4-2','4-3'):
        chosen=selection['branch_selections'][branch]
        for cap in (14,28,56):
            jobs.append({'branch':branch,'trade_hours':tuple(chosen['trade_hours']),
                'start_day':START_DAY,'days':14,
                'config':{'price_mode':chosen['price_mode'],'history_days':cap,'initial_soc':6000.},
                'name':'Q'+branch.replace('-','')+f'_aug_history{cap}'})
    return jobs

def run_history_check(data_dir=ROOT/'data',results_dir=ROOT/'results/final',workers=2):
    data_dir=Path(data_dir).resolve();results_dir=Path(results_dir).resolve()
    if not isinstance(workers,int) or workers<1:raise ValueError('workers must be positive')
    selected_path=results_dir/'selection.json';before=selected_path.read_bytes()
    selection=json.loads(before)
    audit=audit_selection(results_dir)
    if not audit.get('present') or not audit.get('passed'):raise RuntimeError('Frozen development
selection not verified')
    out=results_dir/'history_check';out.mkdir(parents=True,exist_ok=True)
    manifest={'reason':'The original Feb25--Mar10 14-window sensitivity cannot fill a residual-history
cap of 56; add a mature-history verification.',
        'period':'2025-08-01..2025-08-14, fourteen shifted contract windows',
        'start_day':START_DAY,'days':14,'history_caps':[14,28,56],'initial_soc_kwh':6000.,
        'same_independent_initial_state_and_end_rule':True,
        'posthoc_scope_correction':True,'used_for_reselection':False,
        'frozen_branch_choices':{b:selection['branch_selections'][b]['strategy'] for b in ('4-2','4-
3')},
        'interpretation':'Caps denote maximum completed residual paths; event logs certify actual
realized history counts. This supplements, not replaces, original sensitivities.'}
    (out/'scope.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8')
    results=run_jobs(data_dir,out,history_jobs(selection),workers)
    if selected_path.read_bytes()≠before:raise RuntimeError('Frozen selection changed during
supplementary experiment')
    return {'complete':True,'runs':results,'scope':manifest,'selection_unchanged':True}

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,default=ROOT/'data')
    p.add_argument('--results',type=Path,default=ROOT/'results/final');p.add_argument('--
workers',type=int,default=2)
    a=p.parse_args();r=run_history_check(a.data,a.results,a.workers)

```

```
print(json.dumps({'complete':r['complete'],'runs':len(r['runs']),'selection_unchanged':r['selection_unchanged']}),ensure_ascii=False))
```

78. question4/code/q4_io.py

源文件 SHA-256: 62c9abeb244430e26f9610d99bfc139dc7ea87bba1c91441d27a2977791045ff ; 代码行数: 64。

```

"""Combine unchanged Q2/Q3 physical inputs with frozen XGBoost price vintages."""
from pathlib import Path
import json
import numpy as np
from openpyxl import load_workbook
import pandas as pd
from physical_io import load_inputs as load_physical, interval_header, numeric_array, sha
from price_inputs import load_price_data
from q4_optimizer import ETA_C

ROOT=Path(__file__).resolve().parents[1]

def load_inputs(data_dir=ROOT/'data'):
    data_dir=Path(data_dir)
    data=load_physical(data_dir)
    book=load_workbook(data_dir/'load_forecasts.xlsx',read_only=True,data_only=True)
    try:
        rows=list(book['自适应加权_光伏'].values)
    finally:
        book.close()
    if [interval_header(x) for x in rows[0][1:]] != [(m,m+10) for m in range(10,1441,10)]:
        raise ValueError('Q2 PV must contain exactly 144 interval means')
    data['q2_pv_f']=numeric_array([r[1:] for r in rows[1:]],(334,144),'Q2 PV forecast')
    prices=load_price_data(data_dir/'price_inputs.npz')
    if [str(x)[:10] for x in prices.dates] != [str(x) for x in data['dates']]:
        raise ValueError('Price dates disagree with power forecast dates')
    data['price_f']=prices.predicted
    data['price_nodes_f']=prices.nodes
    data['price_a']=prices.actual
    data['price_nodes_a']=prices.actual_nodes
    data['price_estimated']=prices.actual_estimated
    data['price_bundle']=prices
    book=load_workbook(data_dir/'attachment4.xlsx',read_only=True,data_only=True)
    try:
        raw=np.asarray([row[1:] for row in list(book.active.values)[1:]],float).reshape(-1)
    finally:
        book.close()
    stamps=pd.date_range('2025-01-01 00:10',periods=len(raw),freq='10min')
    night=(stamps[:-1].month==1)&(stamps[:-1].hour<6)&(stamps[1:]<pd.Timestamp('2025-02-01'))
    january_night=(raw[:-1]+raw[1:])[night]/2
    data['inventory_value']=float(np.median(january_night)/ETA_C)
    first=31*144
    data['startup_price_actual']=float((raw[first-1]+raw[first])/2)
    data['midnight_observed_price']=raw[np.arange(31,365)*144-1].copy()
    data['bridge_price_forecast']=(data['midnight_observed_price']+data['price_nodes_f'][:,0,0])/2
    # This file belongs to price forecasting provenance and never determines
    # a dispatch hyperparameter from test outcomes.
    price_audit=json.loads((data_dir/'price_inputs_audit.json').read_text())
    data['metadata']['price']=price_audit
    data['metadata']['source_sha256'].update({p.name:sha(p) for p in data_dir.glob('*') if p.is_file()})
    data['metadata']['q4_settlement']='Quantity contracted in advance; price not locked. All fees use
delivery interval actual endpoint-mean price, with cancellation refund and 50% penalty.'
    data['metadata']['common_inventory_value']={'value':data['inventory_value'],
        'basis':'median of completely observed January intervals starting 00:00..05:50, divided by 0.9',
        'included_intervals':int(night.sum()),'unknown_Jan1_midnight_not_filled':True,
        'valuation_fixed_across_strategies_and_terminal_weight_sensitivity':True}
    del data['price'] # Prevent accidental reuse of the Q3 fixed tariff.
    return data

def selected_pv(data,branch,pv_source='supplied'):
    if branch=='4-2' and pv_source!='q3_all':

```



```
return np.repeat(data['q2_pv_f'][:,None,:],4,axis=1)
return data['pv_f'] if pv_source!='linear' else data['pv_linear_f']
```

79. question4/code/q4_optimizer.py

源文件 SHA-256: 789aa3ef6aad62462122dc9bfa97ae3798a11b94b2ee7fcc564fae8386a57dad ; 代码行数: 378。

```

"""Question 4 contract and battery MILP; all flows supplied in kWh.

Day-ahead orders and current adjustments are shared across scenarios. Future
adjustments and battery recourse foresee an entire scenario: this is explicitly
a rolling two-stage approximation, not a nonanticipative multistage policy.
Only ``signed_mask`` entries can be contracted by the caller. The optimizer
never records a transaction, and forecast recourse costs are not actual bills.
"""
from __future__ import annotations

from time import perf_counter
import os
import warnings

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, linprog, milp
from scipy.sparse import coo_matrix, vstack

LOW = 1200.0
HIGH = 10800.0
RATE = 5000.0 / 6.0
ETA_C = ETA_D = float(os.environ.get("MCM_BATTERY_ETA", "0.9"))
PHYSICAL_TOL = 1e-5
OBJECTIVE_TOL = 1e-4
SOLVER_TOL = 1e-9

def solve_fixed_control(load, pv, price, booked, s0, *, terminal_weight, s_ref=6000., time_limit=10.):
    """Exact LP for fixed contracts under the ordinary surplus/deficit rule.

    With  $A = \max(q + PV - \text{load}, 0)$ ,  $D = \max(\text{load} - q - PV, 0)$ ,  $c \leq A$  and  $d \leq D$ 
    imply charge/discharge exclusivity without binary variables. Emergency
    energy is  $D - d$  and can never charge the battery. No transaction is signed.
    """
    start = perf_counter()
    L = _vector(load, 'load'); n = len(L)
    G = _vector(pv, 'pv', n); p = _vector(price, 'price', n); q = _vector(booked, 'booked', n)
    if (p ≤ 0).any() or not LOW ≤ s0 ≤ HIGH or terminal_weight < 0:
        raise ValueError('Fixed-control inputs outside admissible domain')
    A = np.maximum(q + G - L, 0); D = np.maximum(L - q - G, 0)
    cmax = np.minimum(A, RATE); dmax = np.minimum(D, RATE)
    cost = np.r_[np.zeros(n), -5 * p, np.zeros(n), terminal_weight]
    ri = []; ci = []; values = []
    for t in range(n):
        for col, v in ((t, -ETA_C), (n + t, 1 / ETA_D), (2 * n + t, 1.)):
            ri.append(t); ci.append(col); values.append(v)
        if t:
            ri.append(t); ci.append(2 * n + t - 1); values.append(-1.)
    eq = coo_matrix((values, (ri, ci)), shape=(n, 3 * n + 1)).tocsc()
    b = np.r_[s0, np.zeros(n - 1)]
    terminal = coo_matrix((( [-1., -1.], ([0, 0], [3 * n - 1, 3 * n]))), shape=(1, 3 * n + 1)).tocsc()
    bounds = np.column_stack((np.r_[np.zeros(2 * n), np.full(n, LOW), 0.],
                                np.r_[cmax, dmax, np.full(n, HIGH), max(s_ref - LOW, 0)]))
    res = linprog(cost, A_eq=eq, b_eq=b, A_ub=terminal, b_ub=[-s_ref], bounds=bounds, method='highs-ds',
                  options={'time_limit': float(time_limit), 'primal_feasibility_tolerance': SOLVER_TOL,
                           'dual_feasibility_tolerance': SOLVER_TOL})
    if not res.success or res.x is None:
        raise RuntimeError(f'Fixed-contract control LP failed: {res.message}')
    c = np.maximum(res.x[:n], 0); d = np.maximum(res.x[n:2 * n], 0); e = np.maximum(D - d, 0)
    s = np.r_[s0, s0 + np.cumsum(ETA_C * c - d / ETA_D)]
    error = max(float(np.max(np.abs(eq @ res.x - b))), float(max(LOW - s.min(), s.max() - HIGH, 0.)),
                float(np.max(np.minimum(c, d))), float(np.max(np.minimum(c, e))))
    if error > PHYSICAL_TOL:
        raise RuntimeError(f'LP physical violation {error}')

```

```

return {'c':c,'d':d,'e':e,'s':s,'solve_seconds':perf_counter()-start,'max_violation_kwh':error,
        'objective':float(5*p@D+res.fun),'optimality_proven':True,'method':'fixed_contract_LP'}

def _vector(value, name, n=None):
    a = np.asarray(value, dtype=float).copy()
    if (a.ndim != 1 or not len(a) or (n is not None and len(a) != n)
        or not np.isfinite(a).all() or np.any(a < 0)):
        raise ValueError(f"{name} must be a finite nonnegative vector of length {n}.")
    return a

def _mask(value, name, n):
    if value is None:
        return np.zeros(n, dtype=bool)
    a = np.asarray(value)
    if a.shape != (n,) or not np.isin(a, [False, True]).all():
        raise ValueError(f"{name} must be a Boolean vector of length {n}.")
    return a.astype(bool)

def cash_components(p, base_g, delivered_q, emergency, *, adjustment_mask=None):
    """Single-settlement actual bill, separate from any soft inventory penalty.

    Supply only formally contracted ``delivered_q``; never pass future scenario
    proposals. Original charges cover all supplied g. ``adjustment_mask`` can
    restrict this particular adjustment event; otherwise all slots are billed.
    Each slot may be adjusted only once relative to base_g under this policy.
    A cancelled kWh refunds its original price and incurs a 50% penalty.
    """
    p = _vector(p, "p")
    n = len(p)
    g = _vector(base_g, "base_g", n)
    q = _vector(delivered_q, "delivered_q", n)
    e = _vector(emergency, "emergency", n)
    mask = np.ones(n, bool) if adjustment_mask is None else _mask(adjustment_mask, "adjustment_mask", n)
    increase = np.maximum(q-g, 0) * mask
    decrease = np.maximum(g-q, 0) * mask
    original = float(p @ g)
    increase_cost = float(1.5 * p @ increase)
    refund = float(p @ decrease)
    penalty = .5 * refund
    emergency_cost = float(5 * p @ e)
    net_adjustment = increase_cost-refund+penalty
    return {"original_cost": original, "increase_cost": increase_cost,
            "refund": refund, "cancellation_penalty": penalty,
            "net_adjustment_cost": net_adjustment,
            "emergency_cost": emergency_cost,
            "total_cash_cost": original+net_adjustment+emergency_cost}

def _polish(c, matrix, row_lo, row_hi, lower, upper, integer, incumbent, limit):
    """Fix exact rounded MILP modes then solve the unchanged continuous LP."""
    binary = integer.astype(bool)
    rounded = np rint(incumbent[binary])
    error = float(np.max(np.abs(incumbent[binary]-rounded)))
    if error > PHYSICAL_TOL or np.any(rounded < 0) or np.any(rounded > 1):
        raise RuntimeError(f"Unacceptable incumbent integrality error: {error}")
    lo, hi = lower.copy(), upper.copy()
    lo[binary] = hi[binary] = rounded
    equal = np.isfinite(row_lo) & np.isfinite(row_hi) & (row_lo == row_hi)
    finite_hi = np.isfinite(row_hi) & ~equal
    finite_lo = np.isfinite(row_lo) & ~equal
    start = perf_counter()
    result = linprog(c, A_ub=vstack((matrix[finite_hi], -matrix[finite_lo]), format="csc"),
                    b_ub=np.r_[row_hi[finite_hi], -row_lo[finite_lo]],

```

```

        A_eq=matrix[equal], b_eq=row_lo[equal],
        bounds=np.column_stack((lo, hi)), method="highs-ds",
        options={"time_limit": float(limit), "disp": False,
                  "primal_feasibility_tolerance": SOLVER_TOL,
                  "dual_feasibility_tolerance": SOLVER_TOL})
    if result.status != 0 or result.x is None or not np.isfinite(result.x).all():
        raise RuntimeError(f"Fixed-mode LP polish failed: {result.message}")
    return result, error, perf_counter()-start

def solve_dispatch(load_scenarios, pv_scenarios, p, s0, *, mode="dayahead",
                  base_g=None, booked_q=None, locked_mask=None, current_mask=None,
                  future_adjust_mask=None, probabilities=None, terminal_weight=None,
                  s_ref=6000.0, time_limit=10.0, mip_gap=1e-4):
    """Solve a day-ahead, adjustment, or control problem with identical physics.

    Inputs load_scenarios/pv_scenarios have shape (M,n), in kWh.
    p is a positive vector (n,) or scenario array (M,n), in yuan/kWh.
    ``locked_mask``, ``current_mask`` and ``future_adjust_mask`` are disjoint.
    In day-ahead mode, unlocked g is shared and newly signed; locked slots use
    booked_q (e.g. the previous contract's midnight bridge) without recharging
    their sunk original fee. Only future_adjust_mask permits scenario q ≠ g.
    In adjust mode, base_g is the immutable original contract; current_mask q
    is shared and formally adjustable now, future q is scenario-dependent, and
    all other q is booked_q. In control mode current_mask must be empty, so no
    contract can be signed. Per-slot additions cost 1.5p, cancellations net -0.5p.

    Every scenario has exact charge/discharge exclusivity; emergency energy
    cannot charge the battery. Discarding energy while discharging is excluded,
    matching the Question 2 ordinary-surplus/deficit execution convention.
    Terminal shortfall is a soft algorithmic penalty, not electricity expense.
    MILP and the mandatory fixed-mode LP each receive time_limit seconds.
    """
    start = perf_counter()
    load, pv = np.asarray(load_scenarios, float).copy(), np.asarray(pv_scenarios, float).copy()
    if (load.ndim != 2 or load.shape[0] == 0 or load.shape[1] == 0 or pv.shape != load.shape
        or not np.isfinite(load).all() or not np.isfinite(pv).all()
        or np.any(load < 0) or np.any(pv < 0)):
        raise ValueError("load_scenarios/pv_scenarios must have identical finite nonnegative (M,n)
shapes.")
    m = load.shape[0]
    n = load.shape[1]
    price = np.asarray(p, float).copy()
    if price.shape == (n,):
        price = np.broadcast_to(price, (m,n)).copy()
    if price.shape != (m,n) or not np.isfinite(price).all() or (price ≤ 0).any():
        raise ValueError('p must have shape (n,) or (M,n) and strictly positive finite prices')
    if mode not in {"dayahead", "adjust", "control"}:
        raise ValueError("mode must be dayahead, adjust, or control.")
    locked = _mask(locked_mask, "locked_mask", n)
    current = _mask(current_mask, "current_mask", n)
    future = _mask(future_adjust_mask, "future_adjust_mask", n)
    if np.any((locked.astype(int)+current+future) > 1):
        raise ValueError("locked/current/future masks must be disjoint.")
    if mode != "adjust" and current.any():
        raise ValueError("Only adjust mode may have a nonempty current_mask.")
    probability = np.full(m, 1/m) if probabilities is None else _vector(probabilities, "probabilities",
m)
    if abs(float(probability.sum())-1) > 1e-8:
        raise ValueError("probabilities must sum to one.")
    probability /= probability.sum()
    expected_price = probability @ price
    if not np.isfinite(s0) or not LOW-PHYSICAL-TOL ≤ s0 ≤ HIGH+PHYSICAL-TOL:
        raise ValueError("s0 must lie in the battery operating range.")
    s0 = float(np.clip(s0, LOW, HIGH))
    kappa = float(np.min(price)/ETA_C if terminal_weight is None else terminal_weight)

```

```

if (not np.isfinite(kappa) or kappa < 0 or not np.isfinite(s_ref) or s_ref < 0
    or not np.isfinite(time_limit) or time_limit ≤ 0
    or not np.isfinite(mip_gap) or not 0 ≤ mip_gap < 1):
    raise ValueError("Invalid terminal reference/weight, time_limit, or mip_gap.")
if mode ≠ "dayahead" and base_g is None:
    raise ValueError("adjust/control mode requires immutable base_g.")
g_input = np.zeros(n) if base_g is None else _vector(base_g, "base_g", n)
if booked_q is None:
    if mode == "dayahead" and locked.any():
        raise ValueError("dayahead locked slots require booked_q.")
    booked = g_input.copy()
else:
    booked = _vector(booked_q, "booked_q", n)
adjust = future | current
sufficient = load.max(axis=0)+RATE
cap = np.maximum.reduce([sufficient, g_input, booked])
# Shared original g plus shared currently executable q; then each scene
# stores q,a,b,c,d,e,v,r,S,u,xi. Fixed/unused blocks keep interface explicit.
width = 10*n+1
size = 2*n+m*width
cost, lower, upper = np.zeros(size), np.zeros(size), np.full(size, np.inf)
integer = np.zeros(size, np.uint8)
upper[:2*n] = np.tile(cap, 2)
if mode == "dayahead":
    cost[:n] = expected_price * ~locked
    lower[:n][locked] = upper[:n][locked] = booked[locked]
else:
    lower[:n] = upper[:n] = g_input
    lower[n:2*n][~current] = upper[n:2*n][~current] = booked[~current]
row_i, col_i, coeff, row_lo, row_hi = [], [], [], [], []

def row(entries, lo=-np.inf, hi=np.inf):
    idx = len(row_lo)
    for col, value in entries:
        if value:
            row_i.append(idx); col_i.append(col); coeff.append(value)
    row_lo.append(lo); row_hi.append(hi)

if mode == "dayahead":
    for t in range(n):
        row([(n+t, 1), (t, -1)], 0, 0)
for j in range(m):
    block = 2*n+j*width
    upper[block:block+3*n] = np.tile(cap, 3)
    upper[block+3*n:block+5*n] = RATE
    upper[block+5*n:block+6*n] = load[j]
    upper[block+6*n:block+7*n] = cap
    upper[block+7*n:block+8*n] = pv[j]
    lower[block+8*n:block+9*n] = LOW
    upper[block+8*n:block+9*n] = HIGH
    upper[block+9*n:block+10*n] = 1
    integer[block+9*n:block+10*n] = 1
    xi = block+10*n
    upper[xi] = max(s_ref-LOW, 0)
    cost[block+n:block+2*n] = probability[j]*1.5*price[j]
    cost[block+2*n:block+3*n] = -probability[j]*.5*price[j]
    cost[block+5*n:block+6*n] = probability[j]*5*price[j]
    cost[xi] = probability[j]*kappa
    for t in range(n):
        q, a, b, c, d, e, v, r, s, u = (block+k*n+t for k in range(10))
        if adjust[t]:
            row([(q, 1), (t, -1), (a, -1), (b, 1)], 0, 0)
            row([(b, 1), (t, -1)], hi=0)
        else:
            upper[a] = upper[b] = 0
        if not future[t]:

```

```

        row[(q, 1), (n+t, -1)], 0, 0)
    L, G = float(load[j, t]), float(pv[j, t])
    row[(q, 1), (c, -1), (d, 1), (e, 1), (v, -1), (r, -1)], L-G, L-G)
    state = [(s, 1), (c, -ETA_C), (d, 1/ETA_D)]
    if t:
        state.append((s-1, -1))
    row(state, s0 if t == 0 else 0, s0 if t == 0 else 0)
    row([(c, 1), (u, -RATE)], hi=0)
    row([(d, 1), (u, RATE)], hi=RATE)
    row([(e, 1), (u, L)], hi=L)
    row([(v, 1), (r, 1), (u, -(cap[t]+G))], hi=0)
    row([(v, 1), (q, -1)], hi=0)
    row([(block+9*n-1, -1), (xi, -1)], hi=-s_ref)
    matrix = coo_matrix((coeff, (row_i, col_i)), shape=(len(row_lo), size)).tocsc()
    row_lo, row_hi = np.asarray(row_lo), np.asarray(row_hi)
    mip_start = perf_counter()
    with warnings.catch_warnings():
        warnings.filterwarnings("ignore", category=RuntimeWarning,
                                message=r"Unrecognized options detected: \{'mip_feasibility_tolerance'\}\. These will be
passed to HiGHS verbatim\.")
        result = milp(cost, integrality=integer, bounds=Bounds(lower, upper),
                      constraints=LinearConstraint(matrix, row_lo, row_hi),
                      options={"time_limit": float(time_limit), "mip_rel_gap": float(mip_gap),
                              "mip_feasibility_tolerance": SOLVER_TOL, "disp": False})
    mip_seconds = perf_counter()-mip_start
    if result.status not in (0, 1) or result.x is None or not np.isfinite(result.x).all():
        raise RuntimeError(f"No accepted MILP incumbent: {result.status}; {result.message}")
    polished, old_integer_error, polish_seconds = _polish(
        cost, matrix, row_lo, row_hi, lower, upper, integer, result.x, time_limit)
    x = polished.x
    g = np.maximum(x[:n], 0)
    fields = {key: np.empty((m, n)) for key in ["q_scenarios", "a", "b", "c", "d", "e", "v", "r"]}
    raw_states, raw_modes, raw_xi = np.empty((m,n)), np.empty((m,n)), np.empty(m)
    for j in range(m):
        offset = 2*n+j*width
        for k, key in enumerate(fields):
            fields[key][j] = np.maximum(x[offset+k*n:offset+(k+1)*n], 0)
        raw_states[j] = x[offset+8*n:offset+9*n]
        raw_modes[j] = x[offset+9*n:offset+10*n]
        raw_xi[j] = x[offset+10*n]
    q_s, c, d, e, v, r = (fields[key] for key in ["q_scenarios", "c", "d", "e", "v", "r"])
    # Zero-price LP degeneracy may represent a=q-g+b with both positive.
    # Positive-part normalization is equivalent and does not alter q or cost.
    fields["a"] = np.maximum(q_s-g, 0)*adjust
    fields["b"] = np.maximum(g-q_s, 0)*adjust
    modes = np rint(raw_modes).astype(int)
    states = np.column_stack([np.full(m, s0), s0+np.cumsum(ETA_C*c-d/ETA_D, axis=1)])
    shared_q = g.copy() if mode == "dayahead" else booked.copy()
    shared_q[current] = q_s[0,current]
    signed = (~locked).copy() if mode == "dayahead" else current.copy()
    original_cost = float(expected_price[~locked] @ g[~locked]) if mode == "dayahead" else 0.0
    scene_adjustment = np.sum((1.5*fields["a"]-.5*fields["b"])*price, axis=1)
    scene_emergency = np.sum(5*e*price, axis=1)
    shortfall = np.maximum(s_ref-states[:,-1], 0)
    scene_terminal = kappa*shortfall
    value = original_cost+float(probability @ (scene_adjustment+scene_emergency+scene_terminal))
    surplus, deficit = np.maximum(q_s+pv-load, 0), np.maximum(load-q_s-pv, 0)
    def maxabs(a):
        return float(np.max(np.abs(a))) if np.size(a) else 0.
    def positive(a):
        return float(max(np.max(a), 0)) if np.size(a) else 0.
    row_value = matrix @ x
    checks = {
        "constraint_matrix_violation_kwh": max(positive(row_lo-row_value), positive(row_value-row_hi)),
        "variable_bound_violation_kwh": max(positive(lower-x), positive(x-upper)),
        "energy_balance_kwh": maxabs(q_s+pv-v-r+d+e-load-c),

```

```

"soc_solver_agreement_kwh": maxabs(states[:,1:]-raw_states),
"soc_lower_kwh": positive(LOW-states), "soc_upper_kwh": positive(states-HIGH),
"charge_rate_kwh": positive(c-RATE), "discharge_rate_kwh": positive(d-RATE),
"charge_mode_kwh": positive(c-RATE*modes),
"discharge_mode_kwh": positive(d-RATE*(1-modes)),
"emergency_charge_kwh": maxabs(np.minimum(c,e)),
"simultaneous_charge_discharge_kwh": maxabs(np.minimum(c,d)),
"charge_exceeds_surplus_kwh": positive(c-surplus),
"discharge_exceeds_deficit_kwh": positive(d-deficit),
"emergency_deficit_equation_kwh": maxabs(e-deficit+d),
"unused_surplus_equation_kwh": maxabs(v+r-surplus+c),
"unused_contract_bound_kwh": positive(v-q_s),
"pv_curtailment_bound_kwh": positive(r-pv),
"adjustment_relation_kwh": maxabs((q_s-g-fields["a"]+fields["b"])[:,adjust]),
"shared_current_q_kwh": maxabs(q_s[:,current]-shared_q[current]),
"fixed_q_kwh": maxabs(q_s[:,~future & ~current]-shared_q[~future & ~current]),
"terminal_shortfall_kwh": positive(s_ref-states[:,-1]-raw_xi),
}
checks["max_violation_kwh"] = max(checks.values())
checks["integrality_error"] = maxabs(raw_modes-modes)
checks["original_integrality_error"] = old_integer_error
checks["objective_difference_yuan"] = abs(value-float(polished.fun))
checks["passed"] = bool(checks["max_violation_kwh"] ≤ PHYSICAL_TOL
    and checks["integrality_error"] ≤ PHYSICAL_TOL
    and checks["objective_difference_yuan"] ≤ OBJECTIVE_TOL)
if not checks["passed"]:
    raise RuntimeError(f"Physical/accounting verification failed: {checks}")
dual = getattr(result, "mip_dual_bound", None)
dual = None if dual is None or not np.isfinite(dual) else float(dual)
consistent = dual is not None and dual ≤ value+OBJECTIVE_TOL
absolute_gap = max(value-dual, 0) if consistent else None
gap = absolute_gap/max(abs(value), 1e-10) if consistent else None
proven = bool(result.status == 0 and gap is not None and gap ≤ mip_gap+1e-10)
return {"g": g, "q": shared_q, **fields, "s": states, "u": modes,
    "signed_mask": signed, "signed_q": shared_q[signed].copy(),
    "current_mask": current, "future_adjust_mask": future, "locked_mask": locked,
    "objective": value, "original_cost_in_objective": original_cost,
    "new_plan_cost": original_cost, "scenario_adjustment_cost": scene_adjustment,
    "scenario_emergency_cost": scene_emergency, "scenario_terminal_cost": scene_terminal,
    "expected_adjustment_cost": float(probability @ scene_adjustment),
    "expected_emergency_cost": float(probability @ scene_emergency),
    "terminal_cost": float(probability @ scene_terminal),
    "probabilities": probability, "scenario_prices": price,
    "expected_price": expected_price, "terminal_weight": kappa, "s_ref": s_ref,
    "status": "optimal_within_requested_gap" if proven else "feasible_unproven",
    "status_code": int(result.status), "optimality_proven": proven,
    "gap": gap, "mip_gap": gap, "dual_bound": dual, "absolute_gap_yuan": absolute_gap,
    "checks": checks, "success": True, "solve_seconds": perf_counter()-start,
    "mip_seconds": mip_seconds, "polish_seconds": polish_seconds,
    "requested_mip_gap": mip_gap, "time_limit_per_stage": time_limit,
    "gap_basis": "Polished primal versus original global MILP dual bound, not the fixed-mode LP
bound.",
    "recourse_information": "Future adjustment and storage recourse foresee each scenario;
rolling two-stage approximation.",
    "cash_warning": "objective includes unexecuted future proposals and a soft inventory
penalty; it is not an actual bill."}

```

80. question4/code/q4_price_reports.py

源文件 SHA-256: 265a952c3a02413618547357b7602bbd475b993311f0f50b5482779636e2f0f3 ; 代码行数: 155。


```

"""Portable, read-only export of frozen XGBoost prices and actual-node status."""
import argparse
import hashlib
import json
from pathlib import Path

import numpy as np
import openpyxl
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter

from price_inputs import load_price_data

ROOT=Path(__file__).resolve().parents[1]

def clock_label(minutes):
    prefix='次日' if minutes ≥ 1440 else ''
    return f'{prefix}{minutes%1440//60:02d}:{minutes%60:02d}'

def _specifications(price):
    dates=[x.strftime('%Y-%m-%d') for x in price.dates]
    specs={}
    for v, hour in enumerate(price.vintages):
        slot=price.gate_slots[v]
        first=0 if hour==0 else slot+1
        known=price.known_midnight if hour==0 else price.nodes[:,v,slot]
        node_data=np.column_stack((known,price.nodes[:,v,first:]))
        headers=[f'发行日期',f'已知{hour:02d}:00瞬时值（非预测）']+ [clock_label(10*(j+1)) for j in
range(first,145)]
        specs[f'{hour:02d}时_瞬时预测']=(headers,node_data,dates,None)
        labels=[f'{clock_label(10*(j+1))}-{clock_label(10*(j+2))}' for j in range(slot,144)]
        specs[f'{hour:02d}时_区间均价']=( [f'发行日期']+labels,price.predicted[:,v,slot:],dates,None)
    node_status=[f'实测']*len(dates);node_status[-1]='最后次日00:10缺实测; 留空'
    specs[f'实际瞬时节点_缺失留空']=( [f'合同窗口日期']+ [clock_label(10*(j+1)) for j in
range(145)],price.raw_actual_nodes,dates,node_status)
    raw_means=(price.raw_actual_nodes[:,-1]+price.raw_actual_nodes[:,1:])/2
    interval_status=[f'实测端点均价']*len(dates);interval_status[-1]='最后区间右端缺实测; 均价留空'
    labels=[f'{clock_label(10*(j+1))}-{clock_label(10*(j+2))}' for j in range(144)]
    specs[f'实际区间均价_缺失留空']=( [f'合同窗口日期']+labels,raw_means,dates,interval_status)
    specs[f'零点桥接预测']=( [f'发行日期',f'已知00:00（非预测）',f'预测00:10',f'00:00-00:10预测均价'],
np.column_stack((price.known_midnight,price.nodes[:,0,0],price.forecast_bridge)),dates,None)
    return specs

def _style_sheet(sheet, known=False):
    fill=PatternFill('solid',fgColor='17365D');pale=PatternFill('solid',fgColor='EDF3F8')
    for cell in sheet[1]:
        cell.fill=fill;cell.font=Font(name='Calibri',color='FFFFFF',bold=True,size=11)
        cell.alignment=Alignment(horizontal='center',vertical='center',wrap_text=True)
    sheet.row_dimensions[1].height=44
    sheet.freeze_panes='C2' if known else 'B2'
    sheet.auto_filter.ref=sheet.dimensions
    sheet.column_dimensions['A'].width=16
    for column in
range(2,sheet.max_column+1):sheet.column_dimensions[get_column_letter(column)].width=18
    if known:sheet.column_dimensions['B'].width=25
    for row in sheet.iter_rows(min_row=2):
        for cell in row:
            cell.font=Font(name='Calibri',size=11)
            if isinstance(cell.value,(int,float)):cell.number_format='0.000000'
            if cell.row%2==0:cell.fill=pale
            if cell.value is None:cell.fill=PatternFill('solid',fgColor='FFF2CC')

```

```

sheet.sheet_view.zoomScale=85

def verify_price_report(path, data_dir):
    path=Path(path);data_dir=Path(data_dir);price=load_price_data(data_dir/'price_inputs.npz')
    with np.load(data_dir/'xgboost_midnight_predictions.npz',allow_pickle=False) as old:
        np.testing.assert_array_equal(price.nodes[:,0],old['predictions'])
    book=openpyxl.load_workbook(path,read_only=True,data_only=False)
    counts={};errors={}
    try:
        specs=_specifications(price)
        if set(book.sheetnames)!=set(specs)|{'口径与来源','末端估计_不是实测'}:raise ValueError('Unexpected
price workbook sheets')
        for name,(headers,values,dates,status) in specs.items():
            sheet=book[name];rows=list(sheet.iter_rows(values_only=True))
            expected_headers=headers+(['尾节点或尾段状态'] if status is not None else [])
            if list(rows[0])!=expected_headers or len(rows)!=335:raise ValueError(f'Wrong header or row
count: {name}')
            if [row[0] for row in rows[1:]]!=dates:raise ValueError('Price workbook dates changed')
            n=values.shape[1]
            observed=np.asarray([[np.nan if cell is None else float(cell) for cell in row[1:n+1]] for
row in rows[1:]])
            if not np.array_equal(np.isnan(observed),np.isnan(values)):raise ValueError('Missing actual
value was silently filled')
            np.testing.assert_allclose(observed,values,atol=2e-14,rtol=0,equal_nan=True)
            errors[name]=float(np.nanmax(np.abs(observed-values)))
            if status is not None and [row[-1] for row in rows[1:]]!=status:raise ValueError('Missing
actual flags changed')

        counts[name]=dict(data_rows=334,numeric_columns=n,verified_numeric_cells=int(np.isfinite(values).sum()),
missing_cells=int(np.isnan(values).sum()))
        for row in rows:
            for value in row:
                if isinstance(value,str) and (value.startswith('=') or value in
{'#VALUE!','#DIV/0!','#REF!','#NAME?','#NULL!','#NUM!','#N/A'}):
                    raise ValueError('Unexpected formula or Excel error')
                boundary=list(book['末端估计_不是实测'].values)
                if boundary[0]!='方法','缺失节点估计 (元/kWh)','最后区间估计均价 (元/kWh)','状态':raise
ValueError('Wrong boundary estimate header')
                for row,method in zip(boundary[1:],('hold','linear2','ols6')):
                    expected=price.boundary_alternatives[method]
                    if row[0]!=method or row[3]!='估计, 不是实测':raise ValueError('Boundary status changed')
                    np.testing.assert_allclose(row[1:3],
[expected['last_node_price'],expected['last_interval_price']],atol=2e-14,rtol=0)
            finally:book.close()
        return dict(passed=True,workbook_sha256=hashlib.sha256(path.read_bytes()).hexdigest(),sheets=counts,
maximum_excel_roundtrip_difference=max(errors.values()),per_sheet_roundtrip_difference=errors,
original_midnight_predictions_inherited_exactly_before_serialization=True,
missing_actual_node_left_blank=True,missing_actual_interval_left_blank=True,
source_sha256={name:hashlib.sha256((data_dir/name).read_bytes()).hexdigest()
for name in ('price_inputs.npz','xgboost_midnight_predictions.npz')},
formulas_written=False,prices_multiplied_by_one_sixth=False)

def build_price_report(data_dir,results_dir):
    data_dir=Path(data_dir);output=Path(results_dir)/'reports';output.mkdir(parents=True,exist_ok=True)
    price=load_price_data(data_dir/'price_inputs.npz')
    # Prove exact reuse before the decimal serialization used by Excel.
    with np.load(data_dir/'xgboost_midnight_predictions.npz',allow_pickle=False) as old:
        np.testing.assert_array_equal(price.nodes[:,0],old['predictions'])
    workbook=openpyxl.Workbook();readme=workbook.active;readme.title='口径与来源'
    info=[('项目','说明'),('用途','第四问调度的冻结 XGBoost 分版本电价输入;本工作簿只导出已有数组,不训练、不选
参。'),
        ('单位','全部价格单位元/kWh: 价格不乘 1/6。只有功率转电量时才乘小时数。'),
        ('合同窗口','发行日期 D 的 00:10 至 D+1 00:10, 共 144 个十分钟区间。'),
        ('0 点瞬时预测','00:10 至次日00:10, 共145个未来节点。B列是发行时已知的00:00节点, 不是预测。原生成的零点

```

```

XGBoost 预测逐值继承。'),
    ('6/12/18 点瞬时预测','分别从06:10/12:10/18:10至次日00:10,共109/73/37个未来节点。B列为相应发行时刻已经观测的左端节点,不计作预测。'),
    ('对应区间均价','0点144段,6/12/18点109/73/37段。节点端点取平均。日内版本首段使用发行时已知左端与预测右端;后续段两端都是当时预测。'),
    ('十分钟实时控制','工作簿保留每版发行时的原始预测。后续MPC首段应再以当前已观测左端替换预测左端,右端仍使用该版预测;不能用实际右端参与当时决策。'),
    ('零点桥接','单独记录已知00:00与预测00:10形成的00:00-00:10桥接预测均价,不把它塞进本合同144段重复计费。'),
    ('用于评估的实测曲线','单独放在“实际瞬时节点”“实际区间均价”两表中,不混入未来预测列;预测表仅另列发行时已知的观测节点。最后2026-01-01 00:10缺实测,节点及相关区间均价均留空并标记。'),
    ('末端估计','默认持平、2点线性、6点OLS估计单列“末端估计_不是实测”。不将这些数字写进实测表。主调度使用持平估计总结并标记estimated。'),
    ('日内模型','使用同一组冻结 XGBoost 超参数,按相同发行小时的历史样本每天重拟合;仅用发行时已兑现标签,不因本次测试误差重选参数。'),
    ('重叠目标','不同发行版本可能预测相同目标;按发行日期及版本分别保留,不能将重叠节点当独立额外实测样本。'),
    ('显示与保存','显示6位小数,单元格保存完整数值;逐值重开验证允许不超过2e-14的Excel十进制往返误差,不主动舍入原数组。'),
    ('原价预测文件
SHA256',hashlib.sha256((data_dir/'xgboost_midnight_predictions.npz').read_bytes()).hexdigest()),
    ('分版本价格数组 SHA256',hashlib.sha256((data_dir/'price_inputs.npz').read_bytes()).hexdigest()))
    for row in info:readme.append(row)
    _style_sheet(readme);readme.column_dimensions['A'].width=28;readme.column_dimensions['B'].width=115
    for row in readme.iter_rows(min_row=2):
        readme.row_dimensions[row[0].row].height=44
        for cell in row:cell.alignment=Alignment(vertical='center',wrap_text=True)
    for name,(headers,values,dates,status) in _specifications(price).items():
        sheet=workbook.create_sheet(name);sheet.append(headers+(['尾节点或尾段状态'] if status is not None
else []))
        for i,date in enumerate(dates):
            sheet.append([date]+[float(x) if np.isfinite(x) else None for x in values[i]]+[status[i]]
if status is not None else []))
        _style_sheet(sheet,known=('瞬时预测' in name or name=='零点桥接预测'))
        if status is not None:sheet.column_dimensions[get_column_letter(sheet.max_column)].width=44
    sheet=workbook.create_sheet('末端估计_不是实测')
    sheet.append(['方法','缺失节点估计(元/kWh)','最后区间估计均价(元/kWh)','状态'])
    for method in ('hold','linear2','ols6'):
        record=price.boundary_alternatives[method]
        sheet.append([method,record['last_node_price'],record['last_interval_price'],'估计,不是实测'])
    _style_sheet(sheet)
    for column in ('B','C','D'):sheet.column_dimensions[column].width=32
    path=output/'第四问_XGBoost分版本电价.xlsx';workbook.save(path)
    verification=verify_price_report(path,data_dir)
    (output/'第四问_XGBoost分版本电价_核验.json').write_text(json.dumps(verification,ensure_ascii=False,indent=2))
    return verification

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,default=R00T/'data')
    p.add_argument('--results',type=Path,default=R00T/'results/final');args=p.parse_args()
    print(json.dumps(build_price_report(args.data,args.results),ensure_ascii=False))

```

81. question4/code/q4_reports.py

源文件 SHA-256: 4d9fcd6a593c7dcfd28499e3603be2f5e3b5a81e1ea9d1c3604d3f72a9510c0c ; 代码行数: 467。

```

"""Build audited Q4-2/Q4-3 workbooks and both sets of required paper tables.

This module never optimizes dispatch or modifies source ledgers/templates.
Call build_reports(data_dir, results_dir, selected_q42, selected_q43, out_dir) after replay.
"""
from __future__ import annotations

import argparse
import csv
from datetime import datetime, timedelta
import hashlib
import json
import math
from pathlib import Path
import shutil

from openpyxl import Workbook, load_workbook
from openpyxl.chart import BarChart, Reference
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side
from openpyxl.utils import get_column_letter

DATES=('2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21')
SLOTS=('10:00', '12:00', '14:00', '16:00', '18:00', '20:00')
EPS=1e-5
NUMERIC=('price_yuan_per_kwh', 'load_forecast_kw', 'pv_forecast_kw', 'load_actual_kw',
         'pv_actual_kw', 'g_kwh', 'q_kwh', 'increase_kwh', 'decrease_kwh', 'charge_kwh',
         'discharge_kwh', 'emergency_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh',
         'soc_start_kwh', 'soc_end_kwh', 'plan_cost_yuan', 'increase_cost_yuan', 'refund_yuan',
         'penalty_yuan', 'adjustment_net_cost_yuan', 'emergency_cost_yuan', 'cash_cost_yuan')
SUM_FIELDS=('g_kwh', 'q_kwh', 'increase_kwh', 'decrease_kwh', 'charge_kwh', 'discharge_kwh',
            'emergency_kwh', 'unused_paid_kwh', 'curtailed_pv_kwh', 'plan_cost_yuan',
            'increase_cost_yuan', 'refund_yuan', 'penalty_yuan', 'adjustment_net_cost_yuan',
            'emergency_cost_yuan', 'cash_cost_yuan')
LABELS={'strategy': '策略', 'date': '日期', 'g_kwh': '零点原计划量(kWh)',
        'q_kwh': '调整后普通购电量(kWh)', 'increase_kwh': '增购量(kWh)',
        'decrease_kwh': '取销量(kWh)', 'charge_kwh': '充电量(kWh)',
        'discharge_kwh': '放电量(kWh)', 'emergency_kwh': '紧急购电量(kWh)',
        'unused_paid_kwh': '已付未用电量(kWh)', 'curtailed_pv_kwh': '弃光量(kWh)',
        'plan_cost_yuan': '原计划费(元)', 'increase_cost_yuan': '增购费(元)',
        'refund_yuan': '取消退费(元, 正数)', 'penalty_yuan': '取消违约费(元)',
        'adjustment_net_cost_yuan': '调整净费用(元)', 'emergency_cost_yuan': '紧急费(元)',
        'cash_cost_yuan': '总现金费(元)', 'ordinary_cost_yuan': '普通购电结算费(元)',
        'soc_start_kwh': '起点储电量(kWh)', 'soc_end_kwh': '终点储电量(kWh)',
        'startup_cost_yuan': '首日初始化段费(元)', 'cash_including_startup_yuan': '含首段总费(元)',
        'inventory_adjusted_cost_yuan': '库存修正评价(元, 非电费)',
        'natural_period_cash_cost_yuan': '2月1日至12月31日自然日期间费(元)',
        'intervals': '区间数', 'selected': '主展示策略', 'retrospective_best': '同分支回溯现金最低'}

LABELS.update({'solver_calls': '优化求解次数', 'solver_unproven': '未证明达到请求Gap次
数', 'solver_max_gap_percent': '最大相对Gap(%)', 'solver_seconds': '累计求解时间(秒)', 'branch': '问题分
支', 'price_mode': '价格不确定性方案', 'trade_hours': '合同调整时刻', 'forecast_hours': '光伏预报启用时
刻', 'price_hours': '电价预报启用时刻', 'pv_source': '光伏数据来源模式', 'main_selection_candidate': '是否属于原主候选
集合', 'common_inventory_value': '共同库存评价价(元/kWh, 非电费)', 'tail_cost_yuan': '跨年末段费
(元)', 'price_yuan_per_kwh': '实际区间电价(元/kWh)', 'price_forecast_yuan_per_kwh': '优化预测区间电价
(元/kWh)', 'actual_estimated': '功率实际量含估计', 'price_estimated': '实际价格含边界估计', 'interval_start': '执行区
间起点', 'interval_end': '执行区间终点'})

def sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()
def total(rows, key):return math.fsum(r[key] for r in rows)
def as_bool(v):return str(v).strip().lower() in ('true', '1', 'yes')
def near(a, b, label, tol=EPS):
    error=abs(a-b)

```

```

if error>tol:raise AssertionError(f'{label}: {a} vs {b}, error={error}')
return error

def read_trace(path,*,strategy=None):
    with Path(path).open(encoding='utf-8-sig',newline='') as f:raw=list(csv.DictReader(f))
    if not raw:raise ValueError(f'Empty ledger: {path}')
    rows=[]
    for r in raw:
        if strategy is not None and r.get('strategy') and r['strategy']!=strategy:
            # A shared startup file can contain all strategy rows; select exact.
            continue
        v=dict(r)
        for k in NUMERIC:
            if k not in r or r[k] in ('',None):raise ValueError(f'{path}: missing {k}')
            v[k]=float(r[k])
            if not math.isfinite(v[k]):raise ValueError(f'{path}: nonfinite {k}')
        v['actual_estimated']=as_bool(r.get('actual_estimated',False))
        v['price_estimated']=as_bool(r.get('price_estimated',False))
        if r.get('price_forecast_yuan_per_kwh') not in ('',None):
            v['price_forecast_yuan_per_kwh']=float(r['price_forecast_yuan_per_kwh'])
            if not math.isfinite(v['price_forecast_yuan_per_kwh']):raise ValueError('Nonfinite forecast
price')
        for key in ('interval_start','interval_end'):
            v[key]=datetime.fromisoformat(r[key]).isoformat()
        if strategy is not None:v['strategy']=strategy
        rows.append(v)
    rows.sort(key=lambda r:r['interval_start'])
    return rows

def audit_trace(rows,*,label,require_contiguous=True):
    """Verify ledger algebra. Finer execution causality remains runner-tested."""
    max_balance=max_soc=max_cash=max_components=0.
    starts=set();prev=None
    for r in rows:
        a=datetime.fromisoformat(r['interval_start']);b=datetime.fromisoformat(r['interval_end'])
        if b-a!=timedelta(minutes=10) or a.minute%10:raise AssertionError('Bad interval')
        if a in starts:raise AssertionError('Duplicate execution interval')
        starts.add(a)
        if prev is not None and require_contiguous:
            if prev['interval_end']!=r['interval_start']:raise AssertionError('Noncontiguous ledger')
            near(prev['soc_end_kwh'],r['soc_start_kwh'],'cross-interval SOC')
        p=r['price_yuan_per_kwh'];g=r['g_kwh'];q=r['q_kwh'];up=r['increase_kwh'];down=r['decrease_kwh']
        c=r['charge_kwh'];d=r['discharge_kwh'];e=r['emergency_kwh']
        v=r['unused_paid_kwh'];curtail=r['curtailed_pv_kwh']
        load=r['load_actual_kw']/6;pv=r['pv_actual_kw']/6
        if min(p,g,q,up,down,c,d,e,v,curtail)<-EPS:raise AssertionError('Negative physical value')
        if min(up,down)>EPS:raise AssertionError('Simultaneous increase/cancellation')
        if max(c,d)>5000/6+EPS:raise AssertionError('Battery power limit')
        if min(c,d)>EPS:raise AssertionError('Simultaneous charge and discharge')
        if e>EPS and c>EPS:raise AssertionError('Emergency purchase cannot charge battery')
        if v>q+EPS or curtail>pv+EPS:raise AssertionError('Waste exceeds available source')
        for s in ('soc_start_kwh','soc_end_kwh'):
            if not 1200-EPS<=r[s]<=10800+EPS:raise AssertionError('SOC bound')
        max_balance=max(max_balance,near(q-v+pv-curtail+d+e,load+c,'AC energy balance'))
        max_soc=max(max_soc,near(r['soc_end_kwh'],r['soc_start_kwh']+.9*c-d/.9,'SOC equation'))
        values=((q,g+up-down,'q=g+increase-decrease'),
            (r['plan_cost_yuan'],p*g,'original cost'),
            (r['increase_cost_yuan'],1.5*p*up,'increase cost'),
            (r['refund_yuan'],p*down,'refund positive magnitude'),
            (r['penalty_yuan'],.5*p*down,'penalty'),
            (r['adjustment_net_cost_yuan'],1.5*p*up-.5*p*down,'net adjustment'),
            (r['emergency_cost_yuan'],5*p*e,'emergency cost'))
        for x,y,name in values:max_components=max(max_components,near(x,y,name))
        max_cash=max(max_cash,near(r['cash_cost_yuan'],p*g+1.5*p*up-.5*p*down+5*p*e,'cash'))

```



```

    prev=r
    return {'label':label,'intervals':len(rows),'first_interval':rows[0]['interval_start'] if rows else
None,
        'last_interval_end':rows[-1]['interval_end'] if rows else None,
        'max_balance_error_kwh':max_balance,'max_soc_error_kwh':max_soc,
        'max_component_error':max_components,'max_cash_error_yuan':max_cash,
        'actual_estimated_intervals':sum(r['actual_estimated'] for r in rows),
        'price_estimated_intervals':sum(r.get('price_estimated',False) for r in rows),
        'scope':'aggregate physical/transaction ledger; within-interval mode/causality checked
separately in runner tests'}

def summarize(rows,strategy,date=None):
    out={k:total(rows,k) for k in SUM_FIELDS}
    out.update(strategy=strategy,date=date,intervals=len(rows),soc_start_kwh=rows[0]['soc_start_kwh'],
        soc_end_kwh=rows[-1]
['soc_end_kwh'],ordinary_cost_yuan=out['plan_cost_yuan']+out['adjustment_net_cost_yuan'])
    near(out['cash_cost_yuan'],out['ordinary_cost_yuan']+out['emergency_cost_yuan'],'summary cash')
    return out

def day_groups(rows,*,full_only=True):
    groups={}
    for r in rows:groups.setdefault(r['interval_start'][:10],[]).append(r)
    out={}
    for date,rs in sorted(groups.items()):
        start=datetime.fromisoformat(date)
        full=len(rs)==144 and
all(datetime.fromisoformat(r['interval_start'])==start+timedelta(minutes=10*i) for i,r in enumerate(rs))
        if full:out[date]=rs
        elif not full_only:out[date]=rs
    return out

def episodes(rows):
    """Merge positive e intervals; never join across a calendar midnight."""
    out=[];active=None
    for r in rows:
        same=active is not None and active['end']==r['interval_start'] and
active['date']==r['interval_start'][:10]
        if r['emergency_kwh']>EPS:
            if not same:
                if active is not None:out.append(active)
                active={'date':r['interval_start']
[:10],'start':r['interval_start'],'end':r['interval_end'],
                    'parts':[r['emergency_kwh']]
                }
            else:
                active['end']=r['interval_end'];active['parts'].append(r['emergency_kwh'])
            elif active is not None:
                out.append(active);active=None
        if active is not None:out.append(active)
    for e in out:
        e['kwh']=math.fsum(e.pop('parts'))
        end='24:00' if e['end'][:10]!=e['date'] else e['end'][11:16]
        e['label']=e['start'][11:16]+'-'+end
        near(math.fsum(e['kwh'] for e in out)+math.fsum(r['emergency_kwh'] for r in rows if
r['emergency_kwh'] ≤ EPS),
            total(rows,'emergency_kwh'),'episode sum')
    return out

def style(ws,*,width=20,header_rows=(1,)):
    ws.sheet_view.showGridLines=False
    ws.freeze_panes='B2'
    ws.sheet_properties.pageSetUpPr.fitToPage=True
    ws.page_setup.orientation='landscape';ws.page_setup.paperSize=ws.PAPERSIZE_A4

```

```

ws.page_setup.fitToWidth=1;ws.page_setup.fitToHeight=0
for col in range(1,ws.max_column+1):ws.column_dimensions[get_column_letter(col)].width=width
for row in ws:
    ws.row_dimensions[row[0].row].height=27
    for c in row:
        c.font=Font(name='微软雅黑',size=10,color='25354A')
        c.alignment=Alignment(horizontal='center',vertical='center',wrap_text=True)
        if isinstance(c.value,(int,float)):c.number_format='#,##0.0000'
        if c.value is not None:c.border=Border(bottom=Side(style='hair',color='D5DFE7'))
        if c.row in header_rows:
            c.fill=PatternFill('solid',fgColor='213C56');c.font=Font(name='微软雅黑',size=10,bold=True,color='FFFFFF')
            elif c.row%2==0:c.fill=PatternFill('solid',fgColor='F1F6F8')
ws.print_area=ws.dimensions

def table_sheet(book,name,rows,fields):
    ws=book.create_sheet(name);ws.append([LABELS.get(k,k) for k in fields])
    for r in rows:ws.append([r.get(k) for k in fields])
    style(ws)
    ws.auto_filter.ref=ws.dimensions
    for i,k in enumerate(fields,1):
        if k.endswith('_yuan'):
            for j in range(2,ws.max_row+1):ws.cell(j,i).number_format='#,##0.00'
    return ws

def add_notes(book,selected,variant,selection,*,startup_present):
    ws=book.create_sheet('口径说明',0);ws.append(['事项','说明'])
    notes=[('问题分支',variant),('主展示策略',selected),('选择记录',json.dumps(selection,ensure_ascii=False,default=str)),
        ('合同窗口','行日期D的D00:10至D+1 00:10,共144个10分钟区间;不能当成00:00-24:00自然日。'),
        ('波动电价','price_yuan_per_kwh为供电区间实际价格端点均值;本轮所有实际费用项采用同一交付段实际均价。预测价格只进入优化,不冒充实际结算价。'),
        ('价格近似','节点均价是区间均价近似;它与任意连续价格、功率曲线的精确积分并非自动相同。'),
        ('原计划购电表','Q4-2: 填g,全天量Σg,全天费=实际原计划费+紧急费。Q4-3: 填原g,全天量Σg,全天费仅为原计划结算费。'),
        ('调整购电表','仅Q4-3: 填最终普通总量q,不填差额;全天量Σq,全天费=原计划费+增购费-退费+违约金+紧急费。'),
        ('费用公式','Q4-2: pg+5pe。Q4-3: pg+1.5pa-pb+0.5pb+5pe。取消退费按正数列示、汇总时减去;紧急费计一次。'),
        ('付款与用电','普通g/q按合同支付;已付未用v不自动退款。正式取消b才按退原费用并收50%违约金结算。紧急电仅补负载,不充电。'),
        ('自然日储能','按interval_start汇总00:00-24:00;首段来自前一合同,00:00/24:00储电量取实际执行状态。'),
        ('首日启动段','单独读取本轮startup.csv;电池静置保持统一6000kWh实验初态,首段g使用本轮预测、紧急量和费用按实际运行结算。' if startup_present else '未提供首段,不补造自然日结果。'),
        ('初态限制','6000kWh是本轮明确共同实验初态;不是已模拟1月份运行所得到的2月1日实际SOC。本轮首段和费用不沿用旧第三问数字。'),
        ('跨年末段','12月31日合同含次年1月1日00:00-00:10;该段计合同费,单列跨年明细,不混入12月31日自然日储能和紧急表。'),
        ('总体范围','主总计是334个移位合同;自然日期间费=主合同费+首段费-跨年尾段费;库存修正评价不是实际电费。'),
        ('论文四日表1','Q4-2指定普通量用g;Q4-3用q。全天现金费均含紧急费,另给普通量+紧急量以及费用拆分。'),
        ('紧急事件','连续e>1e-5kWh段合并,跨自然日拆开;低于阈值的量仍在精确现金与每日电量汇总中。无事件日期明确填0。'),
        ('估计标记','actual_estimated标记功率边界估计,price_estimated标记价格端点估计;这些段不冒充全部为实测。'),
        ('单位','功率kW,电量kWh,价格元/kWh,费用元;报表不对账本电量再次乘1/6。'),
        ('表格精度','按未舍入数值求和;显示电量四位小数、费用两位小数,显示数逐项相加可能有舍入差。'),
        ('信息与最优性','本表只读执行轨迹;预测生成、求解Gap、滚动信息可用性以主runner审计为准。不由报表汇总声称全局最优。')]
    for row in notes:ws.append(row)
    style(ws);ws.column_dimensions['A'].width=24;ws.column_dimensions['B'].width=115
    for i in range(2,ws.max_row+1):ws.row_dimensions[i].height=46

def save_verified(book,path):
    expected={ws.title,c.coordinate}:c.value for ws in book for row in ws for c in row if c.value is not None}

```



```

book.save(path);wb=load_workbook(path,data_only=False,read_only=True);checked=0
try:
    for ws in wb:
        for row in ws:
            for c in row:
                if c.data_type=='e':raise AssertionError(f'Excel error {ws.title}!{c.coordinate}')
                if c.value is None:continue
                want=expected[(ws.title,c.coordinate)]
                if isinstance(want,(int,float)) and not isinstance(want,bool):
                    if abs(float(c.value)-want)>max(1e-8,abs(want)*1e-12):raise
AssertionError('Workbook numeric drift')
                elif c.value!=want:raise AssertionError(f'Workbook text/date drift: {want!r} →
{c.value!r}')
                checked+=1
    finally:wb.close()
    return {'path':str(path),'cells_checked':checked,'excel_errors':0,'sha256':sha(path)}

def _contract_groups(rows):
    groups={}
    for row in rows:groups.setdefault(row['window_date'],[]).append(row)
    for day,group in groups.items():
        zero=datetime.fromisoformat(day)
        if len(group)!=144:raise AssertionError(f'Incomplete contract: {day}')
        for i,row in enumerate(group):
            if datetime.fromisoformat(row['interval_start'])!=zero+timedelta(minutes=10*(i+1)):
                raise AssertionError(f'Shifted/missing contract slot: {day}, {i}')
    return dict(sorted(groups.items()))

def _check_template(book,variant):
    required=['计划购电量','充放电量','紧急购电量']+(['调整购电量'] if variant=='Q4-3' else [])
    if any(name not in book for name in required):raise ValueError('Missing original template sheet')
    for name in ['计划购电量']+(['调整购电量'] if variant=='Q4-3' else []):
        ws=book[name]
        if ws.max_column!=147:raise ValueError('Original purchase template must have 147 columns')
        headers=[c.value for c in ws[1]]
        if headers[1]!='0:10-0:20' or headers[143]!='23:50-0:00+1' or headers[144]!='0:00-0:10+1':
            raise ValueError('Original purchase interval headers differ from audited contract')
        if headers[145:]!=['全天购电量','全天购电费']:raise ValueError('Missing template totals')
    return required

def export_result(template,path,rows,startup,selected,variant,selection):
    if variant not in ('Q4-2','Q4-3'):raise ValueError('Unknown question variant')
    book=load_workbook(template);required=_check_template(book,variant);contracts=_contract_groups(rows)
    headers={name:[c.value for c in book[name][1]] for name in required}
    for name in required:
        ws=book[name]
        for merged in list(ws.merged_cells.ranges):
            if merged.min_row>=2:ws.unmerge_cells(str(merged))
        if ws.max_row>=2:ws.delete_rows(2,ws.max_row-1)
    daily_financial=[]
    for day,rs in contracts.items():
        zero=datetime.fromisoformat(day);summary=summarize(rs,selected,day);daily_financial.append(summary)
        fee=summary['cash_cost_yuan'] if variant=='Q4-2' else summary['plan_cost_yuan']
        book['计划购电量'].append([zero]+[r['g_kwh'] for r in rs]+[summary['g_kwh'],fee])
        if variant=='Q4-3':
            book['调整购电量'].append([zero]+[r['q_kwh'] for r in rs]+
[summary['q_kwh'],summary['cash_cost_yuan']])
        natural=day_groups(startup+rows)
        if any(day not in natural for day in contracts):raise AssertionError('Every exported natural day
requires all 144 observed/executed slots including startup')
        for day in contracts:
            rs=natural[day];ws=book['充放电量']

```

```

        for k in range(6):
            block=rs[k*24:(k+1)*24]
            ws.append([datetime.fromisoformat(day) if k==0 else None,f'{4*k}:00-{4*(k+1)}:00',
                        total(block,'charge_kwh'),total(block,'discharge_kwh'),
                        '0:00' if k==0 else ('24:00' if k==1 else None),
                        rs[0]['soc_start_kwh'] if k==0 else (rs[-1]['soc_end_kwh'] if k==1 else None)])
    events_by_day={}
    for event in episodes(startup+rows):events_by_day.setdefault(event['date'],[]).append(event)
    for day in contracts:
        events=events_by_day.get(day,[])
        if events:
            for j,event in enumerate(events):book['紧急购电量'].append([datetime.fromisoformat(day) if
j==0 else None,event['label'],event['kwh']])
            else:book['紧急购电量'].append([datetime.fromisoformat(day),'无紧急购电',0.])
        for name in required:
            if [c.value for c in book[name][1]]!=headers[name]:raise AssertionError('Original header
modified')
            style(book[name],width=14 if name in ('计划购电量','调整购电量') else 22)
            book[name].column_dimensions['A'].width=16
            for r in range(2,book[name].max_row+1):
                if isinstance(book[name].cell(r,1).value,datetime):book[name].cell(r,1).number_format='yyyy-
mm-dd'
                if name in ('计划购电量','调整购电量'):book[name].cell(r,147).number_format='#,##0.00'
            add_notes(book,selected,variant,selection,startup_present=bool(startup))
            table_sheet(book,'逐合同现金明细',daily_financial,('date',)+SUM_FIELDS+
('ordinary_cost_yuan','soc_start_kwh','soc_end_kwh'))
            natural_summaries=[summarize(natural[day],selected,day) for day in contracts]
            table_sheet(book,'逐自然日现金明细',natural_summaries,('date',)+SUM_FIELDS+
('ordinary_cost_yuan','soc_start_kwh','soc_end_kwh'))
            table_sheet(book,'首日启动段',startup,
('interval_start','interval_end','price_forecast_yuan_per_kwh','price_yuan_per_kwh')+SUM_FIELDS+
('soc_start_kwh','soc_end_kwh','actual_estimated','price_estimated'))
            tail=[r for r in rows if r['interval_start'][:10]>max(contracts)]
            table_sheet(book,'跨年合同末段',tail,
('interval_start','interval_end','price_forecast_yuan_per_kwh','price_yuan_per_kwh')+SUM_FIELDS+
('soc_start_kwh','soc_end_kwh','actual_estimated','price_estimated'))
            checked=save_verified(book,path)

checked.update(contract_days=len(contracts),natural_days=len(natural_summaries),storage_blocks=6*len(con
tracts),emergency_events=sum(len(events_by_day.get(day,[])) for day in contracts))
    return checked,natural

def paper_tables(path,md_path,natural,selected,variant):
    book=Workbook();book.remove(book.active);summaries={};s1=book.create_sheet('表1_指定购
电');s2=book.create_sheet('表2_充放储电')
    quantity='g_kwh' if variant=='Q4-2' else 'q_kwh';symbol='g' if variant=='Q4-2' else 'q'
    heads1=[];heads2=[];md=[f'# 第四问 {variant} 指定四日论文表格','',f'方案: `{selected}`。日期按00:00-24:00
自然日,电量kwh,费用元。',
        f'表1普通购电量使用{symbol}: 全天费用含紧急费。首段取前一合同,按真实时间戳重组,不每日重置。','']
    for date in DATES:
        if date not in natural:raise AssertionError(f'Missing required natural day {date}')
        rs=natural[date];summary=summarize(rs,selected,date);summaries[date]=summary
        for ws,heads,title in [(s1,heads1,'表1 购电量及费用'),(s2,heads2,'表2 实际充放电量')]:
            ws.append([f'{date}|{variant}|{selected}|
{title}']);heads.extend([ws.max_row,ws.max_row+1]);ws.merge_cells(start_row=ws.max_row,start_column=1,en
d_row=ws.max_row,end_column=6)
            s1.append([f'时间段',f'普通购电量{symbol}',f'时间段',f'普通购电量{symbol}',f'时间段',f'普通购电量
{symbol}'])
            md.extend([f'## {date}','',f'表1: 普通购电量与费用','',f' | 时间段 | 购电量 | 时间段 | 购电量 | 时间段 |
购电量 |','|---|---:|---|---:|---|---:|'])
            for times in (SLOTS[:3],SLOTS[3:]):
                row=[]
                for t in times:
                    ix=int(t[:2])*6
                    if rs[ix]['interval_start'][:11:16]!=t:raise AssertionError('Specified 10-minute table

```

```

index shifted')
        row.extend([f'{t}-{t[:3]}10',rs[ix][quantity]])
        s1.append(row);md.append(' | '+' | '.join(f'{x:.4f}' if isinstance(x,(int,float)) else x for
x in row)+' | ')
        s1.append(['全天普通购电量',summary[quantity],'全天购电费(含紧急)',summary['cash_cost_yuan'],'全天紧急
量',summary['emergency_kwh']])
        s1.append(['全天总购入量',summary[quantity]+summary['emergency_kwh'],'原计划
费',summary['plan_cost_yuan'],'紧急费',summary['emergency_cost_yuan']])
        s1.append(['增购费',summary['increase_cost_yuan'],'退原费用(正数)',summary['refund_yuan'],'违约
费',summary['penalty_yuan']]);s1.append([])
        md.extend([' ','全天普通购电量 **{summary[quantity]:.4f} kWh**; 全天现金费(含紧急) **
{summary['cash_cost_yuan']:.2f}元**; 紧急电量{summary['emergency_kwh']:.4f} kWh。",'',
        '表2: 实际充放电量',' ', ' | 时间段 | 充电量 | 放电量 | 时间段 | 充电量 | 放电量 | ','|---|---|---|---|---|---|
+---+---+---+---+---+---|'])
        s2.append(['时间段','充电量','放电量','时间段','充电量','放电量'])
        for i in (0,2,4):
            row=[]
            for j in (i,i+1):
                block=rs[j*24:(j+1)*24];row.extend([f'{4*j:02d}:00-{4*
(j+1):02d}:00',total(block,'charge_kwh'),total(block,'discharge_kwh')])
                s2.append(row);md.append(' | '+' | '.join(f'{x:.4f}' if isinstance(x,(int,float)) else x for
x in row)+' | ')
                s2.append(['00:00储电量',summary['soc_start_kwh'],None,'24:00储电
量',summary['soc_end_kwh'],None]);s2.append([])
                md.extend([' ','00:00储电量: **{summary['soc_start_kwh']:.4f} kWh**; 24:00储电量: **
{summary['soc_end_kwh']:.4f} kWh**。",''])
                style(s1,header_rows=heads1);style(s2,header_rows=heads2)
                s3=book.create_sheet('表3_连续紧急购电');s3.append([f'{variant} | {selected} | 指定四日紧急购电
(kWh)']);s3.merge_cells('A1:H1')
                s3.append([v for d in DATES for v in (d,None)])
                for col in (1,3,5,7):s3.merge_cells(start_row=2,start_column=col,end_row=2,end_column=col+1)
                s3.append(['时间段','购电量']*4);by_date={date:episodes(natural[date]) for date in DATES}
                md.extend(['## 表3: 连续紧急购电',' ', ' | '+' | '.join(d+'时段 | 电量' for d in DATES)+' | ','|---|---|---|---|---|---|
+---+---+---+---+---+---|'])
                for i in range(max(1,max(map(len,by_date.values())))):
                    row=[]
                    for date in DATES:
                        es=by_date[date];row.extend([es[i]['label'],es[i]['kwh']] if i<len(es) else ([ '无紧急购电',0.]
if not es and i==0 else [None,None]))
                    s3.append(row);md.append(' | '+' | '.join(' if x is None else (f'{x:.4f}' if isinstance(x,
(int,float)) else x) for x in row)+' | ')
                    style(s3,header_rows=(1,2,3));table_sheet(book,'四日现金汇总',list(summaries.values()),
('date',)+SUM_FIELDS+('ordinary_cost_yuan','soc_start_kwh','soc_end_kwh'))
                    md.extend([' ','连续事件阈值为1e-5kWh, 跨午夜拆开; 低于阈值电量仍保留于完整账本的精确电量和费用。'])
                    Path(md_path).write_text('\n'.join(md)+'\n',encoding='utf-8');return
save_verified(book,path),summaries

def _write_csv(path,rows,fields):
    with Path(path).open('w',newline='',encoding='utf-8-sig') as stream:

writer=csv.DictWriter(stream,fieldnames=fields);writer.writeheader();writer.writerows({key:r.get(key)
for key in fields} for r in rows)

def build_reports(data_dir,results_dir,selected_q42,selected_q43,out_dir=None):
    """Export both selected branches and compare every completed main strategy.

    ``results_dir`` normally contains main/<strategy>/trace.csv and selection.json.
    Explicit selected names are mandatory and never replaced by retrospective cost.
    """
    data_dir=Path(data_dir);results_dir=Path(results_dir);out_dir=Path(out_dir) if out_dir else
results_dir/'reports';out_dir.mkdir(parents=True,exist_ok=True)
    main=results_dir/'main' if (results_dir/'main').exists() else results_dir

selection_path=results_dir/'selection.json';selection=json.loads(selection_path.read_text(encoding='utf-

```

```

8')) if selection_path.exists() else
{'selected_q42':selected_q42,'selected_q43':selected_q43,'source':'explicit build_reports arguments'}
    for key,value in [('selected_q42',selected_q42),('selected_q43',selected_q43)]:
        if key in selection and isinstance(selection[key],str) and selection[key]!=value:raise
AssertionError('Explicit selection conflicts with frozen selection file')
    paths=sorted(main.glob('*/*trace.csv'))
    if not paths:raise ValueError('No completed main strategy traces')
    selected_rows={};selected_startup={};audits=[];summaries=[];monthly=[];hashes={}
    expected_dates=[(datetime(2025,2,1)+timedelta(days=i)).date().isoformat() for i in range(334)]
    for path in paths:
        name=path.parent.name;rows=read_trace(path,strategy=name);hashes[str(path)]=sha(path)
        contracts=_contract_groups(rows)
        if list(contracts)!=expected_dates:raise AssertionError(f'{name}: expected all 334 February-
December contracts')
        sp=path.parent/'startup.csv'
        if not sp.exists():raise FileNotFoundError(f'Missing first natural-day segment for {name}')
        startup=read_trace(sp,strategy=name);hashes[str(sp)]=sha(sp)
        if len(startup)!=1 or startup[0]['interval_start']!='2025-02-01T00:00:00' or startup[0]
['interval_end']!='2025-02-01T00:00:00':
            raise AssertionError('Expected one explicit 00:00-00:10 startup segment')
        audits.append(audit_trace(startup+rows,label=name+' including startup'))
        if name.startswith('Q42'):
            for r in rows:
                if abs(r['q_kwh']-r['g_kwh'])>EPS or max(r['increase_kwh'],r['decrease_kwh'])>EPS:raise
AssertionError('Q4-2 must retain fixed ordinary contract')
            source_path=path.parent/'summary.json';source=json.loads(source_path.read_text(encoding='utf-
8'));hashes[str(source_path)]=sha(source_path)
            if source.get('complete') is not True:raise ValueError('Incomplete main strategy cannot be
reported as final')
            s=summarize(rows,name);near(s['cash_cost_yuan'],source['cash_cost_yuan'],'runner main
cash',tol=1e-4)
            if 'common_inventory_value' not in source:raise ValueError('Need ex-ante common_inventory_value;
never infer it from future actual prices')

inventory_value=float(source['common_inventory_value']);s['common_inventory_value']=inventory_value
    s['inventory_adjusted_cost_yuan']=s['cash_cost_yuan']-inventory_value*(s['soc_end_kwh']-
s['soc_start_kwh'])
    near(s['inventory_adjusted_cost_yuan'],source['inventory_adjusted_cost_yuan'],'runner inventory
evaluation',tol=1e-4)
    natural=day_groups(startup+rows)
    if not all(day in natural for day in expected_dates):raise AssertionError('Natural-day coverage
incomplete')
    startup_cash=total(startup,'cash_cost_yuan');tail=[r for r in rows if r['interval_start']
[:10]>'2025-12-31']
    s.update(branch='Q4-2' if name.startswith('Q42') else 'Q4-3',selected=name in
(selected_q42,selected_q43),

startup_cost_yuan=startup_cash,tail_cost_yuan=total(tail,'cash_cost_yuan'),cash_including_startup_yuan=s
['cash_cost_yuan']+startup_cash,
        natural_period_cash_cost_yuan=sum(total(natural[day],'cash_cost_yuan') for day in
expected_dates),
        actual_estimated_intervals=sum(r['actual_estimated'] for r in
rows),price_estimated_intervals=sum(r.get('price_estimated',False) for r in rows),
        solver_calls=source.get('solver',{}).get('solves'),solver_unproven=source.get('solver',
{}).get('unproven'),solver_max_gap_percent=100*source.get('solver',
{}).get('max_gap',0),solver_seconds=source.get('solver',{}).get('seconds'),
        price_mode=source.get('config',{}).get('price_mode'),pv_source=source.get('config',
{}).get('pv_source','supplied'),main_selection_candidate=source.get('config',
{}).get('pv_source','supplied')==supplied,trade_hours=json.dumps(source.get('config',
{}).get('trade_hours')),
        forecast_hours=json.dumps(source.get('config',
{}).get('forecast_hours')),price_hours=json.dumps(source.get('config',{}).get('price_hours')))
    near(s['natural_period_cash_cost_yuan'],s['cash_cost_yuan']+startup_cash-
s['tail_cost_yuan'],'natural/contract fee identity',tol=1e-4)
    summaries.append(s)
    monthgroups={}

```

```

    for r in rows: monthgroups.setdefault(r['window_date'][:7], []).append(r)
    monthly.extend(summarize(rs, name, month) for month, rs in monthgroups.items())
    if name in (selected_q42, selected_q43): selected_rows[name] = rows; selected_startup[name] = startup
    if set(selected_rows) != {selected_q42, selected_q43}: raise ValueError('Selected Q4-2 or Q4-3 trace
unavailable')
    for s in summaries:
        branch = [r for r in summaries if r['branch'] == s['branch'] and r['main_selection_candidate']]
        s['retrospective_best'] = s['main_selection_candidate'] and s['strategy'] == min(branch, key=lambda
r: r['cash_cost_yuan'])['strategy']
        summaries.sort(key=lambda s: (s['branch'], s['cash_cost_yuan']))
        book = Workbook(); book.remove(book.active)
        fields =
('strategy', 'branch', 'selected', 'retrospective_best', 'cash_cost_yuan', 'inventory_adjusted_cost_yuan', 'pl
an_cost_yuan', 'increase_cost_yuan', 'refund_yuan', 'penalty_yuan', 'adjustment_net_cost_yuan', 'emergency_co
st_yuan', 'q_kwh', 'emergency_kwh', 'soc_start_kwh', 'soc_end_kwh', 'startup_cost_yuan', 'tail_cost_yuan', 'nat
ural_period_cash_cost_yuan', 'common_inventory_value', 'solver_calls', 'solver_unproven', 'solver_max_gap_pe
rcent', 'solver_seconds', 'price_mode', 'pv_source', 'main_selection_candidate', 'trade_hours', 'forecast_hour
s', 'price_hours')
        table_sheet(book, '全策略费用比较', summaries, fields); table_sheet(book, '分月合同费用', monthly,
('strategy', 'date', 'cash_cost_yuan', 'plan_cost_yuan', 'adjustment_net_cost_yuan', 'emergency_cost_yuan', 'e
mergency_kwh', 'soc_end_kwh'))
        note = book.create_sheet('总体口径'); note.append(['事项', '解释']); note.append(['主推荐', f'Q4-2=
{selected_q42}; Q4-3={selected_q43}']); note.append(['费用范围', '主费用是334个00:10至次日00:10合同; 自然日费另加
启动、减跨年尾段。']); note.append(['比较限制', '共同初态、物理规则和结算口径; Q4-2与Q4-3的光伏信息不同, 另设同光伏对
照。事后最低指已完整回放的原输入主候选中现金最低, 同PV消融不参与; 不代表全部16组合同完成全年比较; 库存评价非实际
费。']); style(note)
        checks = [save_verified(book, out_dir/'第四问_总体比较.xlsx')]; four_days = {}; natural_counts = {}
        for variant, name, filename in [(('Q4-2', selected_q42, 'result4-2.xlsx'), ('Q4-3', selected_q43, 'result4-
3.xlsx'))]:
            template = data_dir/f'{filename[:-5]}_template.xlsx'
            if not template.exists(): template = data_dir/filename
            hashes[str(template)] = sha(template)

checked, natural = export_result(template, out_dir/filename, selected_rows[name], selected_startup[name], name,
variant, selection); checks.append(checked); natural_counts[variant] = checked['natural_days']
    paper, days = paper_tables(out_dir/f'第四问_{variant}_指定四日填表.xlsx', out_dir/f'第四问_{variant}_指定
四日论文表格.md', natural, name, variant); checks.append(paper); four_days[variant] = days
    source = main/name/'trace.csv'; shutil.copy2(source, out_dir/f'{name}_完整逐段账本.csv')
    _write_csv(out_dir/f'{variant}_指定四日逐段.csv', [r for day in DATES for r in
natural[day]], list(natural[DATES[0]][0]))
    for path, digest in hashes.items():
        if sha(path) != digest: raise AssertionError('Report export modified an input')
    _write_csv(out_dir/'第四问_策略费用比较.csv', summaries, fields)
    report =
{'complete': True, 'selected_q42': selected_q42, 'selected_q43': selected_q43, 'strategy_count': len(summaries)
, 'strategy_summaries': summaries, 'trace_audits': audits, 'four_day_summary': four_days, 'workbook_checks': che
cks, 'source_sha256': hashes, 'source_files_unchanged': True, 'natural_days_exported': natural_counts, 'settleme
nt_rule': 'all realized components use the delivery interval actual endpoint-average price; forecasts
are planning-only', 'cash_coverage': '334 shifted contracts; startup and next-year tail separately
reconciled to 334 natural days'}
    (out_dir/'报表与账本核
验.json').write_text(json.dumps(report, ensure_ascii=False, indent=2, default=str), encoding='utf-8')
    (out_dir/'阅读说明.md').write_text(f'# 第四问调度报表\n\nQ4-2主展示: **{selected_q42}**; Q4-3主展示: **
{selected_q43}**. \n\n'
        'result4-2的计划表全天费包括普通计划费与紧急费。result4-3原计划表全天费仅原计划费, 调整表全天费为最终现金总费
并含紧急费一次; 调整144列填最终普通量q. \n\n'
        '全部实际费用使用目标交付区间实际价格端点均值, 预测价格只影响决策。取消部分退原费后另收50%违约金。 \n\n'
        '储能、紧急事件和指定四日均按自然日汇总; 00:00首段取前一合同或本轮启动段, 年末次日首段单列。原模板保留表头, 示例
行扩展为全部334日期。 \n\n'
        '总体比较中334合同费、自然日期间费和库存评价分别展示, 不能混称同一实际账单。精确数据未舍入, 电量显示四位小数、费
用两位小数。 \n', encoding='utf-8')
    return report

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__); parser.add_argument('--

```



```
data', type=Path, required=True); parser.add_argument('--
results', type=Path, required=True); parser.add_argument('--selected-
q42', required=True); parser.add_argument('--selected-q43', required=True); parser.add_argument('--
out', type=Path)

args=parser.parse_args(); report=build_reports(args.data, args.results, args.selected_q42, args.selected_q43
, args.out)

print(json.dumps({'complete':report['complete'], 'strategy_count':report['strategy_count'], 'selected_q42'
:report['selected_q42'], 'selected_q43':report['selected_q43']}, ensure_ascii=False))
```

82. question4/code/q4_risk.py

源文件 SHA-256: e5514eda9e75511282c1ed919769f35a406522197bef94b5da440c2dd7cf248e ; 代码行数: 71。

```

"""Joint whole-path residual scenarios with explicitly separate release clocks."""
import datetime as dt
import numpy as np

START={0:0,6:35,12:71,18:107}
HOURS=(0,6,12,18)

def available_history(data,day,decision_hour,window=28):
    cutoff=dt.datetime.combine(data['dates'][day],dt.time(decision_hour))
    return [j for j in range(day) if data['window_end'][j]≤cutoff[-window:]]

def latest_hour(enabled_hours,target_index):
    return max(h for h in enabled_hours if START[h]≤target_index)

def future_mask(trade_hours,target_index):
    later=[START[h] for h in trade_hours if h and START[h]>target_index]
    return np.arange(target_index,144)≥min(later) if later else np.zeros(144-target_index,bool)

def current_block(trade_hours,hour):
    later=[START[h] for h in trade_hours if h>hour]
    return START[hour],min(later) if later else 144

def scenario_inputs(data,day,pv_hour,price_hour,decision_hour,begin,config):
    if pv_hour>decision_hour or price_hour>decision_hour or begin<max(START[pv_hour],START[price_hour]):
        raise ValueError('Requested forecast was not yet released or does not cover interval')
    lf=data['load_f'][day,begin:].copy()
    pv=data['policy_pv']
    pf=pv[day,HOURS.index(pv_hour),begin:].copy()
    p=data['price_f'][day,HOURS.index(price_hour),begin:].copy()
    if not np.isfinite(np.r_[lf,pf,p]).all() or (p≤0).any():
        raise ValueError('Invalid available point forecast')
    history=available_history(data,day,decision_hour,config['history_days'])
    meta=dict(forecast_hour=pv_hour,price_hour=price_hour,decision_hour=decision_hour,
              history_indices=history,history_days=len(history),
              history_latest_end=None if not history else data['window_end'][history[-1]].isoformat(),
              price_mode=config['price_mode'],margin_added=False)
    count=config['scenarios']
    if count==1 or len(history)<config['min_history']:
        meta.update(scenarios=1,fallback=count≠1,draw_indices=[])
        return lf[None,:]/6,pf[None,:]/6,p[None,:],meta
    rng=np.random.default_rng(config['seed']+1009*day+59*decision_hour)
    draw=rng.choice(history,min(count,len(history)),replace=False)
    loads=np.maximum(lf+data['load_a'][draw,begin:]-data['load_f'][draw,begin:],0)
    pvs=np.maximum(pf+data['pv_a'][draw,begin:]-pv[draw,HOURS.index(pv_hour),begin:],0)
    predicted_history=data['price_f'][draw,HOURS.index(price_hour),begin:]
    actual_history=data['price_a'][draw,begin:]
    if (predicted_history≤0).any() or (actual_history≤0).any():
        raise ValueError('Multiplicative price scenarios require positive prices')
    log_error=np.log(actual_history/predicted_history)
    if config['price_mode']=='point':
        prices=np.repeat(p[None,:],len(draw),axis=0)
    elif config['price_mode'] in ('joint','shuffled','mean'):
        prices=p*np.exp(log_error)
        if config['price_mode']=='shuffled':
            prices=prices[np.roll(np.arange(len(draw)),1)]
        elif config['price_mode']=='mean':
            prices=np.repeat(prices.mean(axis=0,keepdims=True),len(draw),axis=0)
    else:
        raise ValueError('Unknown price scenario mode')

```



```
if not np.isfinite(prices).all() or (prices ≤ 0).any():  
    raise ValueError('Invalid positive price scenarios; no silent clipping')  
meta.update(scenarios=len(draw), fallback=False, draw_indices=draw.tolist(),  
            sampling='same-date paired full paths; uniform without replacement',  
            price_residual='log(actual/forecast); positive multiplicative reconstruction',  
            scenario_price_min=float(prices.min()), scenario_price_max=float(prices.max()))  
return loads/6, pvs/6, prices, meta
```

83. question4/code/q4_runner.py

源文件 SHA-256: e0d82f871f54c7cd23772379dec88adef188dfaa6abe954e4b07c837ca3e4afb ; 代码行数: 262。

```

"""Q4 rolling policy: causal price information, delivery-price cash ledger.

Current load/PV interval means are fast-feedback execution approximations.
Only actual prices at or before the action timestamp may affect a decision.
All interval actual prices are used for settlement AFTER the action is chosen.
"""

from pathlib import Path
from concurrent.futures import ProcessPoolExecutor, as_completed
import argparse, csv, datetime as dt, hashlib, importlib.metadata, itertools, json, time
import numpy as np
from q4_io import load_inputs, selected_pv
from q4_risk import HOURS, START, scenario_inputs, latest_hour, future_mask, current_block
from q4_optimizer import solve_dispatch, solve_fixed_control, LOW, HIGH, RATE, ETA_C, ETA_D

ROOT=Path(__file__).resolve().parents[1]
DEFAULT=dict(scenarios=8, history_days=28, min_history=14, seed=20260912, controller='mpc',
             pv_source='supplied', initial_soc=6000., time_limit=10., mip_gap=1e-4, price_mode='joint',
             price_hours=[0, 6, 12, 18], terminal_multiplier=1.)
SCHEDULES=[(0,)+s for n in range(4) for s in itertools.combinations((6, 12, 18), n)]
CORE=
('physical_io.py', 'q4_io.py', 'q4_optimizer.py', 'q4_risk.py', 'q4_runner.py', 'price_inputs.py', 'xgb_intrad
ay.py')

def dump(path, value):
    Path(path).parent.mkdir(exist_ok=True, parents=True)
    Path(path).write_text(json.dumps(value, indent=2, ensure_ascii=False, default=str), encoding='utf-8')

def write_csv(path, rows):
    if not rows: return
    with open(path, 'w', newline='', encoding='utf-8-sig') as f:
        w=csv.DictWriter(f, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)

def signature(data_dir, config):
    h=hashlib.sha256(json.dumps(config, sort_keys=True).encode())
    for name in CORE:
        path=ROOT/'code'/name
        if path.exists(): h.update(name.encode()); h.update(path.read_bytes())
    for path in sorted(Path(data_dir).glob('*')):
        if path.is_file() and path.suffix in ('.xlsx', '.npz'):
            h.update(path.name.encode()); h.update(path.read_bytes())
    for name in ('numpy', 'scipy', 'pandas', 'openpyxl'):
        h.update((name+importlib.metadata.version(name)).encode())
    return h.hexdigest()

def validate_arguments(branch, trade_hours, forecast_hours, config, start_day, days):
    if branch not in ('4-2', '4-3'): raise ValueError('Unknown question branch')
    for hours in (trade_hours, forecast_hours, config['price_hours']):
        if not hours or tuple(sorted(set(hours))) != tuple(hours) or hours[0] != 0 or not set(hours)
        ≤ set(HOURS):
            raise ValueError('Release schedule must be a sorted subset of 0/6/12/18 containing 0')
    if branch == '4-2' and tuple(trade_hours) != (0,): raise ValueError('Q4-2 cannot adjust ordinary orders')
    if not isinstance(start_day, int) or not 0 ≤ start_day < 334 or (days is not None and (not
    isinstance(days, int) or days < 1)):
        raise ValueError('Invalid start or length')
    for key in ('scenarios', 'history_days', 'min_history'):
        if not isinstance(config[key], int) or config[key] < 1: raise ValueError('Invalid history/scenario
        count')
    if config['history_days'] < config['min_history']: raise ValueError('History shorter than minimum')
    if config['price_mode'] not in ('point', 'joint', 'mean', 'shuffled'): raise ValueError('Unknown price
    mode')

```

```

        if config['controller'] not in ('mpc', 'greedy'): raise ValueError('Unknown controller')
        if not np.isfinite(config['initial_soc']) or not LOW ≤ config['initial_soc'] ≤ HIGH: raise
ValueError('Invalid initial SOC')
        if not np.isfinite(config['terminal_multiplier']) or config['terminal_multiplier'] < 0: raise
ValueError('Invalid terminal weight')

def
make_record(data, day, t, name, branch, g, q, c, d, e, state, pv_hour, price_hour, pforecast, *, startup=False, price_is
sue_day=None):
    if startup:
        z=data['first_unexecuted_interval']; L=z['load_mean_kw']; G=z['pv_mean_kw']
        begin=dt.datetime.combine(data['dates'][0], dt.time())
        lf=float(data['load_f'][0,0]); pf=float(data['startup_pv_forecast'])
        p=float(data['startup_price_actual']); estimated=False
    else:
        L=float(data['load_a'][day, t]); G=float(data['pv_a'][day, t])
        begin=data['window_start'][day]+dt.timedelta(minutes=10*t)
        lf=float(data['load_f'][day, t]); pf=float(data['policy_pv'][day, HOURS.index(pv_hour), t])
        p=float(data['price_a'][day, t]); estimated=bool(data['actual_estimated'][day, t])
    waste=max(q+G/6+d+e-L/6-c, 0.)
    v=min(q, waste); r=max(waste-v, 0.)
    a=max(q-g, 0.); b=max(g-q, 0.)
    end_state=state+ETA_C*c-d/ETA_D
    # A bridge uses the NEW day's price release while preserving the old PV
    # version and old ordinary contract. Record the actual price release date.
    price_day=data['dates'][day] if price_issue_day is None else price_issue_day
    return dict(strategy=name, branch=branch, window_date=str(data['dates'][day]),
        interval_start=begin.isoformat(), interval_end=(begin+dt.timedelta(minutes=10)).isoformat(),
        forecast_issue_time=dt.datetime.combine(data['dates'][day], dt.time(pv_hour)).isoformat(),
        price_forecast_issue_time=dt.datetime.combine(price_day, dt.time(price_hour)).isoformat(),

price_forecast_hour=price_hour, price_forecast_yuan_per_kwh=float(pforecast), price_yuan_per_kwh=p,
    load_forecast_kw=lf, pv_forecast_kw=pf, load_actual_kw=L, pv_actual_kw=G,
    actual_estimated=estimated, price_estimated=bool(False if startup else data['price_estimated']
[day, t]),
    g_kwh=float(g), q_kwh=float(q), increase_kwh=float(a), decrease_kwh=float(b),
    charge_kwh=float(c), discharge_kwh=float(d), emergency_kwh=float(e),

unused_paid_kwh=float(v), curtailed_pv_kwh=float(r), soc_start_kwh=float(state), soc_end_kwh=float(end_stat
e),

    plan_cost_yuan=p*g, increase_cost_yuan=1.5*p*a, refund_yuan=p*b, penalty_yuan=.5*p*b,
    adjustment_net_cost_yuan=1.5*p*a-.5*p*b, emergency_cost_yuan=5*p*e,
    cash_cost_yuan=p*g+1.5*p*a-.5*p*b+5*p*e)

def run_strategy(data_dir, out_dir, trade_hours=(0, 6, 12, 18), *, branch='4-3', forecast_hours=None,
    config=None, start_day=0, days=None, name=None):
    config={**DEFAULT, **(config or {})}; trade_hours=tuple(trade_hours)
    if forecast_hours is None:
        forecast_hours=(0,) if branch=='4-2' and config['pv_source'] != 'q3_all' else trade_hours
    forecast_hours=tuple(forecast_hours)
    validate_arguments(branch, trade_hours, forecast_hours, config, start_day, days)
    name=name or 'Q'+branch.replace('-', '')+'_'+config['price_mode']+'_H'+'.join(map(str, trade_hours))
    config.update(branch=branch, trade_hours=list(trade_hours), forecast_hours=list(forecast_hours),
        start_day=start_day, days=days, strategy=name)
    target=Path(out_dir)/name; target.mkdir(exist_ok=True, parents=True)
    sig=signature(data_dir, config)
    if (target/'summary.json').exists():
        old=json.loads((target/'summary.json').read_text())
        if old.get('complete') and old.get('run_signature')==sig: return old
        raise RuntimeError(f'Incompatible cached run {target}; choose new output directory')
    data=load_inputs(data_dir)
    data['policy_pv']=selected_pv(data, branch, config['pv_source'])
    data['startup_pv_forecast']=float(data['q2_pv_f'][0,0] if branch=='4-2' and
config['pv_source'] != 'q3_all'

```

```

        else data['pv_unused_first_mean_kw'][0])
    nu=data['inventory_value'];kappa=nu*config['terminal_multiplier']
    opts=dict(terminal_weight=kappa,time_limit=config['time_limit'],mip_gap=config['mip_gap'])
    stop=min(334,start_day+(days or 334));state=float(config['initial_soc'])
    rows=[];daily=[];events=[];g=q=None
    stats=dict(solves=0,milp_solves=0,lp_solves=0,seconds=0.,unproven=0,max_gap=0.,
        max_physical_error=0.,surplus_direct_steps=0)
    beginning=time.perf_counter()

    def solve(L,G,p,s,**kwargs):
        sol=solve_dispatch(L,G,p,s,**opts,**kwargs)
        stats['solves']+=1;stats['milp_solves']+=1;stats['seconds']+=sol['solve_seconds']
        stats['unproven']+=int(not sol['optimality_proven'])
        stats['max_gap']=max(stats['max_gap'],sol['gap'] or 0.)
        stats['max_physical_error']=max(stats['max_physical_error'],sol['checks']['max_violation_kwh'])
        return sol

    def log_event(day,h,kind,sol,meta,first,last,bridge=False):
        events.append(dict(window_date=str(data['dates'][day]),day_index=day,kind=kind,
            decision_at=dt.datetime.combine(data['dates'][day],dt.time(h)).isoformat(),
            **meta,commit_start=first,commit_end=last,locked_bridge=bridge,
            signed_q=sol['signed_q'].tolist(),status=sol['status'],gap=sol['gap'],
            solve_seconds=sol['solve_seconds'],objective=sol['objective'],objective_is_cash=False,
            max_violation_kwh=sol['checks']['max_violation_kwh']))

    def control(L,G,p,base,booked,future,actual_l,actual_pv):
        surplus=max(booked[0]+actual_pv-actual_l,0.)
        deficit=max(actual_l-booked[0]-actual_pv,0.)
        if surplus>1e-9:
            stats['surplus_direct_steps']+=1
            return min(surplus,RATE,max((HIGH-state)/ETA_C,0.)),0.,0.
        if config['controller']=='greedy':
            d=min(deficit,RATE,max((state-LOW)*ETA_D,0.))
            return 0.,d,max(deficit-d,0.)
        L=L.copy();G=G.copy();L[0]=actual_l;G[0]=actual_pv
        if not future.any():

    sol=solve_fixed_control(L,G,p,booked,state,terminal_weight=kappa,time_limit=config['time_limit'])
    stats['solves']+=1;stats['lp_solves']+=1;stats['seconds']+=sol['solve_seconds']
    stats['max_physical_error']=max(stats['max_physical_error'],sol['max_violation_kwh'])
    return float(sol['c'][0]),float(sol['d'][0]),float(sol['e'][0])

    sol=solve(L[None,:],G[None,:],p,state,mode='control',base_g=base,booked_q=booked,future_adjust_mask=future)

    return float(sol['c'][0,0]),float(sol['d'][0,0]),float(sol['e'][0,0])

    def execute(day,t,L,G,p,base,booked,future,fh,ph,price_issue_day=None):
        nonlocal state
        c,d,e=control(L,G,p,base,booked,future,data['load_a'][day,t]/6,data['pv_a'][day,t]/6)

    r=make_record(data,day,t,name,branch,float(base[0]),float(booked[0]),c,d,e,state,fh,ph,float(p[0]),price_issue_day=price_issue_day)
    rows.append(r);state=r['soc_end_kwh']
    if not LOW-1e-5<=state<=HIGH+1e-5:raise RuntimeError('Executed SOC violation')

    def finish_day(day):
        part=rows[-144:]
        if len(part)!=144 or any(r['window_date']!=str(data['dates'][day]) for r in part):
            raise RuntimeError('Incomplete contract window')
        cash=sum(r['cash_cost_yuan'] for r in part)
        daily.append(dict(strategy=name>window_date=str(data['dates'][day]),cash_cost_yuan=cash,
            soc_start_kwh=part[0]['soc_start_kwh'],soc_end_kwh=part[-1]['soc_end_kwh'],
            inventory_adjusted_cost_yuan=cash-nu*(part[-1]['soc_end_kwh']-part[0]['soc_start_kwh']),
            emergency_kwh=sum(r['emergency_kwh'] for r in part)))

    def point_prices(day,t,ph):

```

```

p=data['price_f'][day,HOURS.index(ph),t:].copy()
# Only the left endpoint is available at interval start. Never average
# it with the unknown right ACTUAL endpoint inside a control objective.
p[0]=(data['price_nodes_a'][day,t]+data['price_nodes_f'][day,HOURS.index(ph),t+1])/2
return p

for day in range(start_day,stop):
    L,G,P,meta=scenario_inputs(data,day,0,0,0,config)
    fm=future_mask(trade_hours,0);locked=np.zeros(144,bool)
    bridge=day>start_day
    if bridge:
        oldg,oldq=g.copy(),q.copy();oldfh=latest_hour(forecast_hours,143)
        oldL=float(data['load_f'][day-1,143]/6)
        oldG=float(data['policy_pv'][day-1,HOURS.index(oldfh),143]/6)
        bp=float(data['bridge_price_forecast'][day])
        L=np.column_stack((np.full(len(L),oldL),L));G=np.column_stack((np.full(len(G),oldG),G))
        P=np.column_stack((np.full(len(P),bp),P))
        locked=np.r_[True,locked];fm=np.r_[False,fm];booked=np.r_[oldq[-1],np.zeros(144)]
    else:booked=np.zeros(144)
    sol=solve(L,G,P,state,mode='dayahead',locked_mask=locked,booked_q=booked,future_adjust_mask=fm)
    g=sol['g'][-144:].copy();q=g.copy()
    log_event(day,0,'dayahead',sol,meta,0,144,bridge)
    if bridge:
        base=np.r_[oldg[-1],g];booked=np.r_[oldq[-1],q]
        lf=np.r_[oldL,data['load_f'][day]/6];pf=np.r_[oldG,data['policy_pv'][day,0]/6]
        pp=np.r_[bp,data['price_f'][day,0]]
        execute(day-1,143,lf,pf,pp,base,booked,fm,oldfh,0,price_issue_day=data['dates'][day])
        finish_day(day-1)
    for t in range(143):
        h=next((h for h in trade_hours if h and START[h]==t),None)
        fh=latest_hour(forecast_hours,t);ph=latest_hour(config['price_hours'],t)
        if h is not None:
            first,last=current_block(trade_hours,h)
            L,G,P,meta=scenario_inputs(data,day,fh,ph,h,t,config)
            current=np.arange(t,144)<last
            sol=solve(L,G,P,state,mode='adjust',base_g=g[t:],booked_q=q[t:],current_mask=current,
                future_adjust_mask=future_mask(trade_hours,t))
            if not np.array_equal(sol['signed_mask'],current):raise RuntimeError('Wrong signed
block')

            q[t:last]=sol['q'][:last-t]
            log_event(day,h,'adjust',sol,meta,first,last)
            execute(day,t,data['load_f'][day,t]/6,data['policy_pv'][day,HOURS.index(fh),t]/6,
                point_prices(day,t,ph),g[t:],q[t:],future_mask(trade_hours,t),fh,ph)
        if (day-start_day+1)%10==0 or day==stop-1:
            dump(target/'progress.json',dict(day=str(data['dates'][day]),days_started=day-start_day+1,
                total_days=stop-start_day,elapsed_seconds=time.perf_counter()-beginning,solver=stats))
            print(f'{name}: {day-start_day+1}/{stop-start_day}, {time.perf_counter()-
beginning:.1f}s',flush=True)
        day=stop-1;fh=latest_hour(forecast_hours,143);ph=latest_hour(config['price_hours'],143)
        execute(day,143,data['load_f'][day,143]/6,data['policy_pv'][day,HOURS.index(fh),143]/6,
            point_prices(day,143,ph),g[143:],q[143:],np.zeros(1,bool),fh,ph)
        finish_day(day)
    write_csv(target/'trace.csv',rows);write_csv(target/'daily.csv',daily)
    dump(target/'event_log.json',dict(config=config,events=events))
    startup=[]
    if start_day==0:
        z=data['first_unexecuted_interval'];G=data['startup_pv_forecast']/6
        g0=max(float(data['load_f'][0,0]/6)-G,0.)
        e0=max(z['load_mean_kw']/6-g0-z['pv_mean_kw']/6,0.)
        startup=[make_record(data,0,0,name,branch,g0,g0,0.,0.,e0,config['initial_soc'],0,0,
            data['bridge_price_forecast'][0],startup=True)]
        write_csv(target/'startup.csv',startup)
    total=sum(r['cash_cost_yuan'] for r in rows)
    summary=dict(strategy=name,branch=branch,config=config,complete=True,run_signature=sig,
        intervals=len(rows),windows=stop-start_day,cash_cost_yuan=total,

```

```

soc_start_kwh=config['initial_soc'],soc_end_kwh=state,common_inventory_value=nu,terminal_weight=kappa,
    inventory_adjusted_cost_yuan=total-nu*(state-config['initial_soc']),
    emergency_kwh=sum(r['emergency_kwh'] for r in rows),
    original_plan_cost_yuan=sum(r['plan_cost_yuan'] for r in rows),
    net_adjustment_cost_yuan=sum(r['adjustment_net_cost_yuan'] for r in rows),
    emergency_cost_yuan=sum(r['emergency_cost_yuan'] for r in rows),
    startup_cash_cost_yuan=sum(r['cash_cost_yuan'] for r in startup),
    natural_period_cash_cost_yuan=sum(r['cash_cost_yuan'] for r in rows if r['interval_start'][:10]
≤str(data['dates'][stop-1]))+sum(r['cash_cost_yuan'] for r in startup),
    execution='Current 10-minute actual load/PV mean fast-feedback approximation; actual right-end
prices never used for decisions.',
    elapsed_seconds=time.perf_counter()-beginning,solver=stats)
dump(target/'summary.json',summary)
return summary

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--data',type=Path,default=ROOT/'data')
    p.add_argument('--out',type=Path,required=True);p.add_argument('--branch',choices=('4-2','4-
3'),default='4-3')
    p.add_argument('--price-mode',default='joint');p.add_argument('--schedule',default='0,6,12,18')
    p.add_argument('--days',type=int);p.add_argument('--start-day',type=int,default=0)
    a=p.parse_args()
    r=run_strategy(a.data,a.out,tuple(map(int,a.schedule.split(','))),branch=a.branch,
        config={'price_mode':a.price_mode},days=a.days,start_day=a.start_day)
    print(json.dumps(r,ensure_ascii=False,default=str))

```

84. question4/code/q4_solver_check.py

源文件 SHA-256: c1cb98bfd9080b3ecc286e22c6387a343e8edc2a152a5284fd0d1e96e41cd37e ; 代码行数: 210。

```

"""Read-only frozen-state solver sensitivity; never replaces selected/main runs.

The main ledger records executed actions, not every MPC dual bound. This module
reconstructs selected ordinary intraday MPC problems from the immutable source
forecasts and the original trajectory state. Future contract changes are hidden
until their real gate; only formal already-signed changes are revealed.
"""

from pathlib import Path
import argparse, hashlib, json
import numpy as np
import pandas as pd
from q4_io import load_inputs, selected_pv
from q4_risk import HOURS, future_mask, latest_hour
from q4_optimizer import solve_dispatch, LOW

ROOT = Path(__file__).resolve().parents[1]

def available_contract(original, final, future):
    """Future final orders must not leak back into an earlier MPC call."""
    original=np.asarray(original,float);final=np.asarray(final,float)
    future=np.asarray(future,bool)
    if original.shape!=final.shape or original.shape!=future.shape:
        raise ValueError('Contract arrays must align')
    return np.where(future,original,final)

def reconstruct_control(data, trace, config, day, t):
    """Return original normal intraday MPC inputs, excluding midnight bridges."""
    if not 0 ≤ t < 143:
        raise ValueError('Only ordinary intraday steps 0..142; bridges need a different reconstruction')
    date=str(data['dates'][day]);part=trace.loc[trace.window_date==date].reset_index(drop=True)
    if len(part)!=144:
        raise ValueError('A complete immutable contract trace is required')
    expected=data['window_start'][day]+pd.Timedelta(minutes=10*t)
    row=part.iloc[t]
    if pd.Timestamp(row.interval_start)!=pd.Timestamp(expected):
        raise ValueError('Trace timestamp does not match contract slot')
    fh=latest_hour(config['forecast_hours'],t);ph=latest_hour(config['price_hours'],t)
    future=future_mask(config['trade_hours'],t)
    if not future.any():raise ValueError('This call is an LP/direct action, not the questioned MILP control')
    g=part.g_kwh.to_numpy()[t:]
    q=available_contract(g,part.q_kwh.to_numpy()[t:],future)
    L=data['load_f'][day,t:].copy()/6
    G=data['policy_pv'][day,HOURS.index(fh),t:].copy()/6
    L[0]=data['load_a'][day,t]/6;G[0]=data['pv_a'][day,t]/6
    p=data['price_f'][day,HOURS.index(ph),t:].copy()
    p[0]=(data['price_nodes_a'][day,t]+data['price_nodes_f'][day,HOURS.index(ph),t+1])/2
    if L[0]-q[0]-G[0] < -1e-9:raise ValueError('Surplus is executed directly, not an MILP control')
    if not np.isclose(q[0],row.q_kwh,rtol=0,atol=1e-7):raise ValueError('Current contract is not the executed order')
    if not np.isclose(p[0],row.price_forecast_yuan_per_kwh,rtol=0,atol=1e-10):raise
ValueError('Information price differs from main run')
    if not np.isclose(data['policy_pv'][day,HOURS.index(fh),t],row.pv_forecast_kw,rtol=0,atol=1e-6):
        raise ValueError('PV forecast version differs from main run')
    return dict(load=L[None,:],pv=G[None,:],price=p,s0=float(row.soc_start_kwh),
        base_g=g,booked_q=q,future_adjust_mask=future,day=day,t=t,date=date,
        interval_start=row.interval_start,forecast_hour=fh,price_hour=ph,
        original_action={k:float(row[k]) for k in ['charge_kwh','discharge_kwh','emergency_kwh']})

def candidates(trace,config):
    """Specified-day stratified diagnostic sample, selected without solver output."""

```



```

selected=[]
for date in ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']:
    part=trace.loc[trace.window_date==date].reset_index(drop=True)
    if len(part)!=144:raise ValueError('Missing specified day')
    deficit=part.load_actual_kw/6-part.q_kwh-part.pv_actual_kw/6
    eligible=[i for i in range(143) if deficit.iloc[i]>1e-6
              and part.soc_start_kwh.iloc[i]>LOW+.1 and future_mask(config['trade_hours'],i).any()]
    used=set()
    for target in (30,55,90):
        remaining=[i for i in eligible if i not in used]
        if remaining:
            t=min(remaining,key=lambda i:(abs(i-target),i));used.add(t);selected.append((date,t))
    return selected

def summarize(sol, inputs, limit):
    c=float(sol['c'][0,0]);d=float(sol['d'][0,0]);e=float(sol['e'][0,0])
    return dict(window_date=inputs['date'],slot=inputs['t'],interval_start=inputs['interval_start'],

time_limit_seconds=limit,status=sol['status'],solver_status_code=sol['status_code'],requested_mip_gap=sol['requested_mip_gap'],objective_yuan=sol['objective'],

dual_bound_yuan=sol['dual_bound'],absolute_gap_yuan=sol['absolute_gap_yuan'],relative_gap=sol['gap'],
    original_cost_excluded_yuan=float(inputs['price']@inputs['base_g']),

expected_adjustment_yuan=sol['expected_adjustment_cost'],expected_emergency_yuan=sol['expected_emergency_cost'],

terminal_yuan=sol['terminal_cost'],solve_seconds=sol['solve_seconds'],charge_kwh=c,discharge_kwh=d,emergency_kwh=e,
    charge_minus_main_kwh=c-inputs['original_action']['charge_kwh'],
    discharge_minus_main_kwh=d-inputs['original_action']['discharge_kwh'],
    emergency_minus_main_kwh=e-inputs['original_action']['emergency_kwh'],
    max_physical_violation_kwh=sol['checks']['max_violation_kwh'],
    signed_order_count=int(np.count_nonzero(sol['signed_mask'])))

def
run_check(data_dir=ROOT/'data',results_dir=ROOT/'results/final',strategy='Q43_point_H0_18',out_dir=None,
refine_count=4):
    out=Path(out_dir or ROOT/'results/solver_check')/strategy;out.mkdir(parents=True,exist_ok=True)
    source=Path(results_dir)/'main'/strategy
    summary=json.loads((source/'summary.json').read_text());config=summary['config']
    trace=pd.read_csv(source/'trace.csv')
    data=load_inputs(data_dir);data['policy_pv']=selected_pv(data,config['branch'],config['pv_source'])
    bydate={str(x):i for i,x in enumerate(data['dates'])}
    chosen=candidates(trace,config);rows=[];problems=[]
    meta={'strategy':strategy,'main_run_signature':summary['run_signature'],
        'source_trace_sha256':hashlib.sha256((source/'trace.csv').read_bytes()).hexdigest(),
        'sample_rule':'For each four specified day, nearest eligible deficit/SOC step to slots30,55,90;
no solver-output screening.',
        'main_state_and_contracts_frozen':True,'main_selection_unchanged':True,
        'baseline_time_limit':10,'refined_time_limit':45,'refined_mip_gap':1e-6,
        'replay_limitations':'Only sampled same-state controls; not a counterfactual full trajectory,
not a global policy cost bound. The original maximum-gap control timestamp was not logged.',
        'relative_gap_definition':'max(polished primal - original MILP dual bound,0)/abs(polished
primal); the objective excludes original sunk plan fees and includes future cancellation refunds and
inventory penalty.'}
    def solve_one(inp,limit,gap):
        return solve_dispatch(inp['load'],inp['pv'],inp['price'],inp['s0'],mode='control',
            base_g=inp['base_g'],booked_q=inp['booked_q'],future_adjust_mask=inp['future_adjust_mask'],

terminal_weight=data['inventory_value']*config['terminal_multiplier'],time_limit=limit,mip_gap=gap)
    def save():
        (out/'checks.json').write_text(json.dumps(**meta,'complete':False,'checks':rows,indent=2,ensure_ascii=

```

```

False))
    pd.DataFrame(rows).to_csv(out/'checks.csv',index=False,encoding='utf-8-sig')
    for date,t in chosen:
        inp=reconstruct_control(data,trace,config,bydate[date],t);problems.append(inp)
        rows.append(summarize(solve_one(inp,10,config['mip_gap']),inp,10));save()
        print(json.dumps(rows[-1],ensure_ascii=False),flush=True)
    # Refine largest certified absolute gaps, with relative gap and main-action
    # discrepancy as tie breakers. This deliberately targets the numerical issue.
    ordered=sorted(range(len(rows)),key=lambda i:(rows[i]['absolute_gap_yuan'] or 0,rows[i]
['relative_gap'] or 0,
        abs(rows[i]['discharge_minus_main_kwh'])),reverse=True)[:refine_count]
    for i in ordered:
        inp=problems[i];r=summarize(solve_one(inp,45,1e-6),inp,45);base=rows[i]
        r.update(objective_minus_10sec_yuan=r['objective_yuan']-base['objective_yuan'],
            discharge_minus_10sec_kwh=r['discharge_kwh']-base['discharge_kwh'],
            charge_minus_10sec_kwh=r['charge_kwh']-base['charge_kwh'],
            emergency_minus_10sec_kwh=r['emergency_kwh']-base['emergency_kwh'])
        rows.append(r);save();print(json.dumps(r,ensure_ascii=False),flush=True)
    meta['complete']=True;meta['checks']=rows

meta['main_trace_unchanged']=hashlib.sha256((source/'trace.csv').read_bytes()).hexdigest()==meta['source
_trace_sha256']
    if not meta['main_trace_unchanged']:raise RuntimeError('Main trace changed during diagnostic')
    (out/'checks.json').write_text(json.dumps(meta,indent=2,ensure_ascii=False))
    return meta

def study_signature(data_dir,results_dir):
    """Cache depends on the frozen source state, data and reconstruction/core code."""
    from q4_runner import CORE
    import scipy
    root=Path(results_dir)/'main'/'Q43_point_H0_18'
    h=hashlib.sha256((np.__version__+'|'+scipy.__version__).encode())
    for path in [root/'summary.json',root/'trace.csv']:
        h.update(path.name.encode());h.update(path.read_bytes())
    for path in sorted(Path(data_dir).glob('*')):
        if path.is_file():h.update(path.name.encode());h.update(path.read_bytes())
    for name in (*CORE,'q4_solver_check.py','q4_solver_zero_screen.py'):
        path=ROOT/'code'/name;h.update(name.encode());h.update(path.read_bytes())
    return h.hexdigest()

def finalize_solver_study(data_dir,results_dir,out_dir=None):
    """Aggregate completed diagnostic records; refresh formal-order inventory."""
    results_dir=Path(results_dir);out=Path(out_dir or results_dir/'solver_check')
    sample=out/'Q43_point_H0_18'

a=json.loads((sample/'checks.json').read_text());b=json.loads((sample/'zero_objective_screen.json').read
_text())
    if not a['complete'] or not b['complete']:raise ValueError('Incomplete solver diagnostics')
    rows=a['checks']+b['sampled_controls']+b['refinements']
    source=results_dir/'main'/'Q43_point_H0_18'
    if hashlib.sha256((source/'trace.csv').read_bytes()).hexdigest()!=a['source_trace_sha256']:
        raise ValueError('Diagnostic source trajectory has changed')
    formal=[]
    for path in sorted((results_dir/'main').glob('*/*summary.json')):
        summary=json.loads(path.read_text())
        if not summary.get('complete'):continue
        events=json.loads((path.parent/'event_log.json').read_text())['events']
        largest=max(events,key=lambda e:e['gap'] or 0)
        formal.append(dict(strategy=path.parent.name,all_solver_max_relative_gap=summary['solver']
['max_gap'],
            formal_events=len(events),formal_event_max_relative_gap=largest['gap'],
            formal_max_gap_time=largest['decision_at'],formal_max_gap_kind=largest['kind'],
            formal_max_gap_objective_yuan=largest['objective'],
            formal_max_gap_absolute_yuan_from_logged_gap=(largest['gap'] or
0)*max(abs(largest['objective']),1e-10),

```

```

        formal_max_absolute_gap_yuan=max((e['gap'] or 0)*max(abs(e['objective']),1e-10) for e in
events)))
        action_error=max(abs(r[k]) for r in rows for k in
('charge_minus_main_kwh','discharge_minus_main_kwh','emergency_minus_main_kwh'))
        result=dict(complete=True,study_signature=study_signature(data_dir,results_dir),
            strategy='Q43_point_H0_18',main_run_signature=a['main_run_signature'],
            recorded_solver_calls=len(rows),unique_frozen_states=len({(r['window_date'],r['slot']) for r in
rows})),
            ten_second_calls=sum(r['time_limit_seconds']==10 for r in
rows),forty_five_second_calls=sum(r['time_limit_seconds']==45 for r in rows),

max_action_difference_from_main_kwh=action_error,all_sampled_actions_same_as_main=action_error<1e-7,
max_absolute_objective_bound_gap_yuan=max(r['absolute_gap_yuan'] or 0 for r in rows),
max_relative_gap=max(r['relative_gap'] or 0 for r in rows),
max_physical_violation_kwh=max(r['max_physical_violation_kwh'] for r in rows),
near_zero_high_relative_gap_cases=[r for r in b['refinements']],
max_45sec_vs_10sec_action_difference_kwh=max(abs(r['discharge_minus_10sec_kwh']) for r in
a['checks']+b['refinements'] if 'discharge_minus_10sec_kwh' in r),
main_results_changed=False,formal_order_diagnostics=formal,
limitation=f'{len(rows)} reported calls at {len({(r["window_date"],r["slot"]) for r in rows})}
frozen states (development reruns not counted). Neither every main control nor the original maximum-gap
step was logged/replayed. This demonstrates the denominator artifact, not a full-year policy-optimality
proof. Formal event absolute gaps are recovered from the exact logged relative-gap formula; they are not
actual bill bounds.')
        (out/'summary.json').write_text(json.dumps(result,indent=2,ensure_ascii=False))
        pd.DataFrame(rows).to_csv(out/'全部同状态重解.csv',index=False,encoding='utf-8-sig')
        pd.DataFrame(formal).to_csv(out/'正式签单求解精度.csv',index=False,encoding='utf-8-sig')
        return result

def run_solver_check(data_dir=ROOT/'data',results_dir=ROOT/'results/final'):
    """Public entry point for run_all; source/code hashes guard the diagnostic cache."""
    out=Path(results_dir)/'solver_check';cached=out/'summary.json'
    if cached.exists():
        prior=json.loads(cached.read_text())
        if prior.get('complete') and
prior.get('study_signature')==study_signature(data_dir,results_dir):
            return finalize_solver_study(data_dir,results_dir,out)
            raise RuntimeError('Stale solver diagnostic cache; choose a new result directory')
    from q4_solver_zero_screen import run_zero_screen
    run_check(data_dir,results_dir,out_dir=out)
    run_zero_screen(data_dir,results_dir,out_dir=out)
    return finalize_solver_study(data_dir,results_dir,out)

if __name__=='__main__':
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--data',type=Path,default=ROOT/'data')
    parser.add_argument('--results',type=Path,default=ROOT/'results/final')
    args=parser.parse_args()
    print(json.dumps(run_solver_check(args.data,args.results),indent=2,ensure_ascii=False))

```

85. question4/code/q4_solver_zero_screen.py

源文件 SHA-256: bcf86f38011beb4de40d43505c379c9fc1aea6fc99ec7ebd42fdc8692c6e443b ; 代码行数: 57.

```

"""Bounded supplemental search for near-zero control objectives; read-only inputs."""
from pathlib import Path
import json
import numpy as np
import pandas as pd
from q4_solver_check import ROOT, reconstruct_control, summarize
from q4_io import load_inputs, selected_pv
from q4_risk import future_mask
from q4_optimizer import solve_dispatch

def run_zero_screen(data_dir, results_dir, strategy="Q43_point_H0_18", out_dir=None):
    out=Path(out_dir or ROOT/"results/solver_check")/strategy; out.mkdir(parents=True, exist_ok=True)
    source=Path(results_dir)/"main"/strategy
    config=json.loads((source/'summary.json').read_text())
['config']; trace=pd.read_csv(source/'trace.csv')
    data=load_inputs(data_dir); data['policy_pv']=selected_pv(data, config['branch'], config['pv_source'])
    rows=[]; pairs=[]
    # Inspect two early weeks plus each later month's first contract window; select
    # up to six evenly spaced eligible deficit steps per day, max144 calls in all.
    chosen=list(range(14))+[i for i, date in enumerate(data['dates']) if date.day==1 and date.month>2]
    for day in chosen:
        part=trace.loc[trace.window_date==str(data['dates'][day])].reset_index(drop=True)
        D=part.load_actual_kw/6-part.q_kwh-part.pv_actual_kw/6
        eligible=[t for t in range(107) if D.iloc[t]>1e-6 and future_mask(config['trade_hours'], t).any()]
        for pos in np.unique(np.linspace(0, len(eligible)-1, min(6, len(eligible))).astype(int)):
            pairs.append((day, eligible[pos]))
    for day, t in pairs:
        inp=reconstruct_control(data, trace, config, day, t)

sol=solve_dispatch(inp['load'], inp['pv'], inp['price'], inp['s0'], mode='control', base_g=inp['base_g'],

booked_q=inp['booked_q'], future_adjust_mask=inp['future_adjust_mask'], terminal_weight=data['inventory_va
lue'], time_limit=10, mip_gap=1e-4)
    row=summarize(sol, inp, 10); rows.append(row)
    result={'selection_rule': 'First14days and firstday of eachlatermonth; at most6 evenly spaced
positive-deficit ordinary MILP steps/day, up to144 calls. Stop after3 relative gaps over1%. Read-only
frozenstate diagnostic.',
        'complete': False, 'sampled_controls': rows}
    (out/'zero_objective_screen.json').write_text(json.dumps(result, indent=2, ensure_ascii=False))
    if row['relative_gap'] is not None and row['relative_gap']>.01:
        print(json.dumps(row, ensure_ascii=False), flush=True)
    if sum((r['relative_gap'] or 0)>.01 for r in rows)≥3: break
    refinements=[]
    for baseline in [r for r in rows if (r['relative_gap'] or 0)>.01]:
        day=next(i for i, date in enumerate(data['dates']) if str(date)==baseline['window_date'])
        inp=reconstruct_control(data, trace, config, day, baseline['slot'])

sol=solve_dispatch(inp['load'], inp['pv'], inp['price'], inp['s0'], mode='control', base_g=inp['base_g'],

booked_q=inp['booked_q'], future_adjust_mask=inp['future_adjust_mask'], terminal_weight=data['inventory_va
lue'], time_limit=45, mip_gap=1e-6)
    row=summarize(sol, inp, 45)
    row.update(objective_minus_10sec_yuan=row['objective_yuan']-baseline['objective_yuan'],
        discharge_minus_10sec_kwh=row['discharge_kwh']-baseline['discharge_kwh'],
        emergency_minus_10sec_kwh=row['emergency_kwh']-baseline['emergency_kwh'])
    refinements.append(row)
    result['complete']=True; result['calls']=len(rows); result['planned_maximum_calls']=len(pairs)
    result['refinements']=refinements
    result['limitations']='Original maximum-gap step was not logged. Similar large relative gaps were
reproduced and shown to have tiny absolute gaps; this bounded audit does not classify every unproven
step in the334-window run.'
    (out/'zero_objective_screen.json').write_text(json.dumps(result, indent=2, ensure_ascii=False))

```

```
print(json.dumps({'calls':len(rows),'planned_maximum_calls':len(pairs),'largest_gaps':sorted(rows,key=la
mbda r:r['relative_gap'] or 0,reverse=True)[:5]},ensure_ascii=False),flush=True)
    return result

if __name__=="__main__":
    run_zero_screen(ROOT/"data",ROOT/"results/final")
```

86. question4/code/q4_study.py

源文件 SHA-256: c6327fdc9df03cfff7545eff89bf640f4a1deb15ced86d213d62a6d2a3a9b4f4 ; 代码行数: 109。

```

"""Predeclared calibration, frozen-choice main comparisons and sensitivities."""
from pathlib import Path
from concurrent.futures import ProcessPoolExecutor, as_completed
import argparse, json
import pandas as pd
from q4_runner import ROOT, SCHEDULES, run_strategy, dump, signature

def strategy_name(branch, mode, schedule):
    return 'Q'+branch.replace('-', '')+'_'+mode+'_H'+ '_' .join(map(str, schedule))

def jobs_for_calibration():
    jobs=[]
    for branch, schedules in [('4-2', [(0,)]), ('4-3', SCHEDULES)]:
        for mode in ('point', 'joint'):
            for schedule in schedules:
                jobs.append(dict(branch=branch, trade_hours=schedule, config={'price_mode':mode},
                                name=strategy_name(branch, mode, schedule), days=24))
    return jobs

def run_jobs(data_dir, out_dir, jobs, workers):
    out_dir=Path(out_dir);out_dir.mkdir(parents=True, exist_ok=True)
    dump(out_dir/'design.json', {'jobs':jobs, 'workers':workers})
    results=[]
    with ProcessPoolExecutor(max_workers=min(workers, len(jobs))) as pool:
        futures={pool.submit(run_strategy, data_dir, out_dir, **job):job for job in jobs}
        for future in as_completed(futures):
            result=future.result();results.append(result)
            print('Finished', result['strategy'], f"cash={result['cash_cost_yuan']:.4f}", flush=True)
    dump(out_dir/'comparison.json', results)
    return results

def freeze(results_dir, data_dir=ROOT/'data'):
    results_dir=Path(results_dir);records=[]
    for job in jobs_for_calibration():
        path=results_dir/'calibration'/job['name']
        summary=json.loads((path/'summary.json').read_text())
        if not summary['complete']:raise RuntimeError('Incomplete calibration')
        if summary['run_signature']!=signature(data_dir, summary['config']):
            raise RuntimeError('Calibration input/core/config signature changed; rerun in a new output
directory')
        daily=pd.read_csv(path/'daily.csv')
        development=daily[daily.window_date.between('2025-02-17', '2025-02-23')]
        if len(development)!=7:raise ValueError('Calibration must include seven completely realized
windows')
        records.append(dict(strategy=job['name'], branch=job['branch'], price_mode=job['config']
['price_mode'],

trade_hours=list(job['trade_hours']), score=float(development.inventory_adjusted_cost_yuan.sum()),
cash=float(development.cash_cost_yuan.sum()), run_signature=summary['run_signature']))
        selected={branch:min([r for r in records if r['branch']==branch], key=lambda r:
(r['score'], r['strategy'])))
            for branch in ('4-2', '4-3')}
        payload=dict(freeze_time='2025-02-25T00:00:00', development_windows='2025-02-17..2025-02-23',
excluded_last_unrealized_window='2025-02-24 ends at 2025-02-25 00:10',
test_windows='2025-02-25..2025-12-31', selected_q42=selected['4-2']['strategy'],
selected_q43=selected['4-3']['strategy'], branch_selections=selected, development=records,
price_forecaster='XGBoost explicitly selected by user; midnight predictions reused unchanged',
score='sum(cash - fixed January inventory value * state change)',
model_or_schedule_reselection_from_test=False)
        dump(results_dir/'selection.json', payload)

```



```

    return payload

def main_jobs(selection):
    mode=selection['branch_selections']['4-3']['price_mode']
    chosen_schedule=tuple(selection['branch_selections']['4-3']['trade_hours'])
    other='point' if mode=='joint' else 'joint'
    jobs=[dict(branch='4-2',trade_hours=(0,),config={'price_mode':m},name=strategy_name('4-2',m,(0,)))
          for m in ('point','joint'))
    jobs += [dict(branch='4-3',trade_hours=s,config={'price_mode':mode},name=strategy_name('4-
3',mode,s)) for s in SCHEDULES]
    jobs += [dict(branch='4-3',trade_hours=chosen_schedule,config=
{'price_mode':other,name=strategy_name('4-3',other,chosen_schedule)})]
    jobs += [dict(branch='4-2',trade_hours=(0,),forecast_hours=chosen_schedule,
        config={'price_mode':mode,'pv_source':'q3_all'},name='Q42_matched_Q43_PV')]
    return jobs

def sensitivity_jobs(selection):
    variants=[('reference',{'history14',{'history_days':14}},('history56',{'history_days':56}),
        ('scenes4',{'scenarios':4}),('scenes16',{'scenarios':16}),
        ('terminal_half',{'terminal_multiplier':.5}),('terminal_double',{'terminal_multiplier':2.}),
        ('seed13',{'seed':20260913}),('seed14',{'seed':20260914}),
        ('price_midnight_only',{'price_hours':[0]}),('greedy',{'controller':'greedy'}),
        ('initial1200',{'initial_soc':1200.}),('initial10800',{'initial_soc':10800.}),
        ('joint_mean',{'price_mode':'mean'}),('joint_shuffled',{'price_mode':'shuffled'})]
    jobs=[]
    for branch in ('4-2','4-3'):
        chosen=selection['branch_selections'][branch]
        for label,change in variants:
            jobs.append(dict(branch=branch,trade_hours=tuple(chosen['trade_hours']),start_day=24,days=14,
                config={'price_mode':chosen['price_mode'],**change},name='Q'+branch.replace('-','')+ '_' +label))
                if chosen['price_mode']=='point':

            jobs.append(dict(branch=branch,trade_hours=tuple(chosen['trade_hours']),start_day=24,days=14,
                config={'price_mode':'joint'},name='Q'+branch.replace('-','')+ '_joint_reference'))
    return jobs

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--stage',choices=
('calibration','freeze','main','sensitivity'),required=True)
    p.add_argument('--data',type=Path,default=ROOT/'data');p.add_argument('--
out',type=Path,default=ROOT/'results')
    p.add_argument('--workers',type=int,default=6);a=p.parse_args()
    if a.workers<1:p.error('workers must be positive')
    if a.stage=='calibration':run_jobs(a.data,a.out/'calibration',jobs_for_calibration(),a.workers)
    elif a.stage=='freeze':print(json.dumps(freeze(a.out,a.data),ensure_ascii=False,indent=2))
    else:
        selection=json.loads((a.out/'selection.json').read_text())
        verified=freeze(a.out,a.data)
        if (selection['selected_q42'],selection['selected_q43'])≠
(verified['selected_q42'],verified['selected_q43']):
            raise RuntimeError('Saved selection differs from calibration')
        jobs=main_jobs(selection) if a.stage=='main' else sensitivity_jobs(selection)
        run_jobs(a.data,a.out/('main' if a.stage=='main' else 'sensitivity'),jobs,a.workers)

```

87. question4/code/run_all.py

源文件 SHA-256: 2df6be24ba65b17d932329b0a59473ed50bc0dbe2032b922284ea4d7259d9fd8 ; 代码行数: 89.


```

"""Reproduce Q4 in the declared chronological order, with checked caches."""
from pathlib import Path
import argparse, datetime as dt, hashlib, json, subprocess, sys, time
from price_inputs import build_price_inputs
from audit_price_inputs import audit_prices
from q4_io import load_inputs
from q4_runner import ROOT, dump
from q4_study import run_jobs, jobs_for_calibration, freeze, main_jobs, sensitivity_jobs
from q4_audit import audit_all
from q4_causal_study import run_study as run_causal_study
from q4_causal_audit import audit_study as audit_causal_study
from q4_reports import build_reports
from q4_figures import build_figures
from q4_history_check import run_history_check
from q4_solver_check import run_solver_check
from q4_price_reports import build_price_report
from q4_four_day_review import review_four_days

def causal_jobs(selection):
    schedule=selection['branch_selections']['4-3']['trade_hours']
    suffix='_H'+'_'.join(map(str,schedule))
    selected={selection['selected_q42'],selection['selected_q43']}
    return [(name,(1.,1) if name in selected else (1.,)) for name in
            ['Q42_point_H0','Q42_joint_H0','Q43_point'+suffix,'Q43_joint'+suffix]]

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--data',type=Path,default=ROOT/'data')
    parser.add_argument('--out',type=Path,default=ROOT/'results/reproduced')
    parser.add_argument('--workers',type=int,default=4)
    parser.add_argument('--document',action='store_true',help='Also build Word/PDF using installed
Pandoc/LibreOffice')
    args=parser.parse_args()
    if args.workers<1:parser.error('workers must be positive')
    data=args.data.resolve();out=args.out.resolve();out.mkdir(parents=True,exist_ok=True)
    manifest={'started_at':dt.datetime.now().isoformat(),'complete':False,'stages':[],
            'data_dir':str(data),'output_dir':str(out),'python':sys.version,
            'cache_policy':'Reuse completed numerical runs only with identical input
bytes/core/config/numerical-library signatures.'}
    start=time.perf_counter()
    def stage(name,fn):
        print('Starting:',name,flush=True);t=time.perf_counter()
        result=fn()
        manifest['stages'].append({'name':name,'complete':True,'elapsed_seconds':time.perf_counter()-t})
        dump(out/'run_all_manifest.json',manifest)
        return result
    try:
        stage('price_inputs',lambda:build_price_inputs(data,min(3,args.workers)))
        stage('price_audit',lambda:audit_prices(data))
        info=stage('power_and_time_inputs',lambda:load_inputs(data))
        dump(out/'input_audit.json',info['metadata'])
        stage('tests',lambda:subprocess.run([sys.executable,'-m','unittest','discover',
            '-s',str(ROOT/'code'),'--p','test*.py'],check=True))
        stage('calibration',lambda:run_jobs(data,out/'calibration',jobs_for_calibration(),args.workers))
        selection=stage('freeze_selection',lambda:freeze(out,data))
        stage('main',lambda:run_jobs(data,out/'main',main_jobs(selection),args.workers))

        stage('sensitivity',lambda:run_jobs(data,out/'sensitivity',sensitivity_jobs(selection),args.workers))
        stage('mature_history_window_check',lambda:run_history_check(data,out,min(2,args.workers)))
    def audit():
        report=audit_all(data,out)
        dump(out/'independent_audit.json',report)

```

```

        if not report.get('passed', False): raise RuntimeError('Independent audit failed; see
independent_audit.json')
        return report
    stage('independent_audit', audit)
    stage('solver_precision_check', lambda: run_solver_check(data, out))
    stage('causal_execution', lambda: run_causal_study(out, data))
    stage('causal_audit', lambda: audit_causal_study(out, data))

stage('template_and_four_days', lambda: build_reports(data, out, selection['selected_q42'], selection['select
ed_q43']))
    stage('independent_four_day_review', lambda: review_four_days(out))
    stage('readable_price_versions', lambda: build_price_report(data, out))
    stage('figures', lambda: build_figures(out))
    from q4_analysis import build_analysis
    stage('analysis_and_boundaries', lambda: build_analysis(data, out))
    from q4_conclusions import build_conclusions
    stage('conclusions', lambda: build_conclusions(out))
    if args.document:
        from build_document import build_document
        stage('word_and_pdf', lambda: build_document(out))
    manifest['complete']=True; manifest['ended_at']=dt.datetime.now().isoformat()
    manifest['elapsed_seconds']=time.perf_counter()-start
    dump(out/'run_all_manifest.json', manifest)
    print('Complete:', out, flush=True)
except Exception as error:
    manifest['failure']=repr(error); manifest['elapsed_seconds']=time.perf_counter()-start
    dump(out/'run_all_manifest.json', manifest)
    raise

if __name__=='__main__': main()

```

88. question4/code/test_price_inputs.py

源文件 SHA-256: bdbea82da1588218c7d359a92defa8d275f664c95d33903e7e1d834fe4c9c04a ; 代码行数: 92。

```

import json
from pathlib import Path
import unittest
import numpy as np

from xgb_intraday import features_at_issue, training_arrays_at_issue, forecast_intraday
from price_inputs import assemble_arrays, extrapolate_last_node, read_attachment4, GATE_SLOTS

ROOT=Path(__file__).resolve().parents[1]

class TestIntradayPrices(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.raw=read_attachment4(ROOT/'data/attachment4.xlsx')
        cls.flat=cls.raw.ravel()
        cls.cfg=json.loads((ROOT/'data/xgboost_midnight_summary.json').read_text())['config']

    def test_features_cannot_see_future_after_release(self):
        for hour in (6,12,18):
            day=31;cutoff=day*144+hour*6
            expected,_,targets=features_at_issue(self.flat,day,hour)
            changed=self.flat.copy();changed[cutoff:]+=1000
            actual,_,_=features_at_issue(changed,day,hour)
            np.testing.assert_array_equal(expected,actual)
            self.assertEqual(targets[0],cutoff)
            self.assertEqual(targets[-1],(day+1)*144)
            self.assertEqual(expected[0,0],hour*6)
            self.assertEqual(expected[-1,0],0)
            self.assertEqual(expected[-1,2],6) # Feb2, 2025 is Sunday

    def test_actual_xgboost_prediction_future_perturbation(self):
        day=31;hour=6;cutoff=day*144+hour*6
        expected,_=forecast_intraday(self.flat,day,hour,self.cfg)
        changed=self.flat.copy();changed[cutoff:]=-1000
        actual,_=forecast_intraday(changed,day,hour,self.cfg)
        np.testing.assert_array_equal(expected,actual)

    def test_past_issue_features_and_label_release_cutoff(self):
        for hour in (6,12,18):
            day=31;cutoff=day*144+hour*6
            X,y,a,diag=training_arrays_at_issue(self.flat,day,hour,28)
            self.assertTrue((diag['label_indices']<cutoff).all())
            self.assertTrue((diag['origin_days']<day).all())
            self.assertEqual(diag['first_training_origin_day'],7)
            self.assertEqual(diag['same_issue_hour'],hour)
            np.testing.assert_array_equal(y,self.flat[diag['label_indices']])
            changed=self.flat.copy();changed[cutoff:]=np.nan
            X2,y2,_,_=training_arrays_at_issue(changed,day,hour,28)
            np.testing.assert_array_equal(X,X2);np.testing.assert_array_equal(y,y2)
            # Historical issue-date price is its own known end point, not the
            # current day's latest actual injected into all past feature rows.
            issue_nodes=diag['origin_days']*144+hour*6-1
            np.testing.assert_array_equal(X[:,17],self.flat[issue_nodes])

    def test_zero_nodes_inherited_exactly_and_intervals_not_divided_by_six(self):
        with np.load(ROOT/'data/xgboost_midnight_predictions.npz') as z:
            midnight=z['predictions'].copy()
            updates={h:np.full((334,145-h*6),.5) for h in (6,12,18)}
            arrays=assemble_arrays(self.raw,midnight,updates)
            np.testing.assert_array_equal(arrays['nodes'][:,0],midnight)
            np.testing.assert_array_equal(arrays['predicted'][:,0],(midnight[:,:-1]+midnight[:,1:])/2)
            self.assertAlmostEqual(arrays['forecast_bridge'][0],(self.raw[30,143]+midnight[0,0])/2)
            self.assertAlmostEqual(arrays['actual_bridge'][0],(self.raw[30,143]+self.raw[31,0])/2)

```

```

def test_release_node_and_old_intervals_not_available(self):
    midnight=np.full((334,145),.4)
    updates={h:np.full((334,145-h*6),.8) for h in (6,12,18)}
    arrays=assemble_arrays(self.raw,midnight,updates)
    for v,slot in enumerate(GATE_SLOTS[1:],1):
        self.assertTrue(np.isnan(arrays['predicted'][:,v,:slot]).all())
        np.testing.assert_array_equal(arrays['nodes'][:,v,slot],self.raw[31,:slot])
        np.testing.assert_allclose(arrays['predicted'][:,v,slot],(self.raw[31,:slot]+.8)/2)
    self.assertTrue(np.isnan(arrays['raw_actual_nodes'][-1,-1]))
    self.assertEqual(arrays['actual_estimated'].sum(),1)
    self.assertTrue(arrays['actual_estimated'][-1,-1])

def test_boundary_methods_and_no_price_clipping(self):
    y=np.linspace(.1,.6,6)
    self.assertAlmostEqual(extrapolate_last_node(y,'hold'),.6)
    self.assertAlmostEqual(extrapolate_last_node(y,'linear2'),.7)
    self.assertAlmostEqual(extrapolate_last_node(y,'ols6'),.7)
    self.assertAlmostEqual(extrapolate_last_node(np.array([1,.8,.6,.4,.2,0]),'linear2'),-.2)
    midnight=np.full((334,145),-.1)
    updates={h:np.full((334,145-h*6),.8) for h in (6,12,18)}
    arrays=assemble_arrays(self.raw,midnight,updates)
    self.assertTrue((arrays['predicted'][:,0]==-.1).all())

def test_midnight_cannot_be_refit_via_update_entry(self):
    with self.assertRaises(ValueError):forecast_intraday(self.flat,31,0,self.cfg)

if __name__=='__main__':unittest.main()

```

89. question4/code/test_q4_analysis.py

源文件 SHA-256: f46b67b024625a55183031c5e4d6157ce2e900bab58decc02fc4e914bfce4fd ; 代码行数: 129。

```

"""Analysis tests use synthetic arrays and a final-step analytical example."""
import unittest
from unittest.mock import patch

import numpy as np
import pandas as pd

import q4_analysis as analysis
from q4_history_check import history_jobs, START_DAY

class TestBlockComparison(unittest.TestCase):
    def test_constant_difference_and_full_sample_block(self):
        constant=np.full(310,-12.5)
        for block in (3,7,14):
            r=analysis.moving_block_mean_interval(constant,block)
            self.assertEqual(r['mean_difference_ci_lower_yuan'],-12.5)
            self.assertEqual(r['mean_difference_ci_upper_yuan'],-12.5)
            self.assertEqual(r['total_difference_yuan'],-3875.)
        # A block equal to the full series must retain the observed mean in
        # every replicate; an IID bootstrap would fail this counterexample.
        r=analysis.moving_block_mean_interval(np.arange(30.),30)
        self.assertEqual(r['mean_difference_ci_lower_yuan'],14.5)
        self.assertEqual(r['mean_difference_ci_upper_yuan'],14.5)

    def test_seed_repeatability_and_translation_equivariance(self):
        y=np.arange(310.)%17+np.linspace(-2,3,310)
        a=analysis.moving_block_mean_interval(y,7)
        b=analysis.moving_block_mean_interval(y,7)
        c=analysis.moving_block_mean_interval(y+100,7)
        self.assertEqual(a,b)
        for key in
('mean_daily_difference_yuan','mean_difference_ci_lower_yuan','mean_difference_ci_upper_yuan'):
            self.assertAlmostEqual(c[key]-a[key],100.,places=10)
            self.assertIn('not a prediction interval',a['interpretation'])

    def test_daily_pairing_sign_and_missing_date_rejection(self):
        dates=pd.date_range('2025-02-01',periods=334).strftime('%Y-%m-%d')
        left=pd.DataFrame({'window_date':dates,'cash_cost_yuan':np.full(334,90.)})
        right=pd.DataFrame({'window_date':dates,'cash_cost_yuan':np.full(334,100.)})
        diff,seen=analysis.paired_daily_difference(left,right)
        self.assertEqual(len(seen),310);np.testing.assert_array_equal(diff,np.full(310,-10.))
        with self.assertRaisesRegex(ValueError,'310 dates'):
            analysis.paired_daily_difference(left,right.drop(25))
        with self.assertRaisesRegex(ValueError,'310 dates'):
            analysis.paired_daily_difference(left,right.iloc[::-1])

class TestEndpointStudy(unittest.TestCase):
    def setUp(self):
        self.row={'interval_start':'2026-01-01T00:00:00','interval_end':'2026-01-01T00:10:00',
            'g_kwh':100.,'q_kwh':100.,'soc_start_kwh':1200.,
            'price_forecast_yuan_per_kwh':.7,'price_yuan_per_kwh':.5}
        base=analysis.execute_tail(self.row,1200.,0.,terminal_weight=.5)
        self.row.update(base)
        self.row['cash_cost_yuan']=analysis.settle_tail(self.row,base,.5)
        self.summary={'common_inventory_value':.5,'terminal_weight':.5,'config':{'controller':'mpc'},

'cash_cost_yuan':12345.,'inventory_adjusted_cost_yuan':12345.,'soc_start_kwh':1200.,'soc_end_kwh':1200.}
        self.power={m:{'mean_kw':v} for m,v in zip(analysis.METHODS,(1200.,1260.,1140.))},
            'pv':{m:{'mean_kw':0.} for m in analysis.METHODS}
        self.prices={m:{'last_interval_price':v} for m,v in zip(analysis.METHODS,(.5,.4,.6))}

    def test_price_only_changes_bill_not_action_or_controller_coefficient(self):

```

```

changed={**self.row, 'price_yuan_per_kwh':100000., 'cash_cost_yuan':float('nan')}}
with patch.object(analysis, 'solve_fixed_control', wraps=analysis.solve_fixed_control) as spy:
    normal=analysis.execute_tail(self.row, 1200., 0., terminal_weight=.5)
    altered=analysis.execute_tail(changed, 1200., 0., terminal_weight=.5)
self.assertEqual(normal, altered)
self.assertEqual(spy.call_args_list[0].args[2], [.7])
self.assertEqual(spy.call_args_list[1].args[2], [.7])
self.assertEqual(normal['emergency_kwh'], 100.)
self.assertEqual(analysis.settle_tail(self.row, normal, 1.), 600.)
self.assertEqual(analysis.settle_tail(self.row, normal, 3.), 1800.)

def test_nine_combinations_reproduce_baseline_and_preserve_price_independence(self):
    rows=analysis.endpoint_variants('analytical', self.summary, self.row, self.power, self.prices)
    self.assertEqual(len(rows), 9)
    base=next(r for r in rows if r['power_endpoint_method']==r['price_endpoint_method']=='hold')
    self.assertEqual(base['full_cash_change_yuan'], 0.)
    self.assertTrue(all(r['estimated_power_right_endpoint'] and r['estimated_price_right_endpoint']
for r in rows))
    self.assertTrue(all(r['actual_price_used_for_control'] is False for r in rows))
    for method in analysis.METHODS:
        part=[r for r in rows if r['power_endpoint_method']==method]

self.assertEqual(len({r['charge_kwh'], r['discharge_kwh'], r['emergency_kwh'], r['soc_end_kwh']} for r in
part}), 1)
        self.assertEqual(len({r['full_cash_cost_yuan'] for r in part}), 3)
        self.assertEqual(next(r for r in rows if r['power_endpoint_method']=='linear2' and
r['price_endpoint_method']=='hold')['full_cash_change_yuan'], 25.)

def test_saturated_surplus_respects_capacity_and_discards_rest(self):
    row={**self.row, 'soc_start_kwh':10799.}
    step=analysis.execute_tail(row, 0., 6000., terminal_weight=.5)
    self.assertEqual(step['charge_kwh'], 1/.9)
    self.assertEqual(step['soc_end_kwh'], 10800.)
    self.assertEqual(step['discharge_kwh'], 0.)
    self.assertEqual(step['emergency_kwh'], 0.)
    self.assertEqual(step['unused_paid_kwh']+step['curtailed_pv_kwh'], 1000.)

class TestMatureHistoryCheck(unittest.TestCase):
    def test_jobs_keep_frozen_modes_schedules_and_use_common_august_horizon(self):
        selected={'branch_selections':{'4-2':{'price_mode':'point', 'trade_hours':[0]},
            '4-3':{'price_mode':'joint', 'trade_hours':[0, 6, 18]}}}
        jobs=history_jobs(selected)
        self.assertEqual(START_DAY, 181); self.assertEqual(len(jobs), 6)
        for branch in ('4-2', '4-3'):
            part=[j for j in jobs if j['branch']==branch]
            self.assertEqual([j['config']['history_days'] for j in part], [14, 28, 56])
            for job in part:
                self.assertEqual(job['start_day'], 181); self.assertEqual(job['days'], 14)
                self.assertEqual(job['config']['initial_soc'], 6000.)
                self.assertEqual(job['config']['price_mode'], selected['branch_selections'][branch]
['price_mode'])
            self.assertEqual(list(job['trade_hours']), selected['branch_selections'][branch]
['trade_hours'])

    def test_mature_history_summary_refuses_unfilled_56_cap(self):
        runs={}
        for branch in ('4-2', '4-3'):
            for cap in (14, 28, 56):
                name='Q'+branch.replace('-', '')+f'_aug_history{cap}'
                runs[name]={ 'summary':{ 'windows':14, 'config':{ 'start_day':181, 'history_days':cap,
                    'soc_start_kwh':6000., 'soc_end_kwh':6000., 'cash_cost_yuan':float(10000+cap),
                    'inventory_adjusted_cost_yuan':float(10000+cap)},
                    'actual_history_min':cap, 'actual_history_max':cap}
                result=analysis.history_check_summary(runs)
                self.assertEqual(len(result), 6)

```

```
self.assertTrue(all(r['all_events_fill_history_cap'] for r in result))
runs['Q42_aug_history56']['actual_history_min']=23
with self.assertRaisesRegex(ValueError, 'did not actually fill'):
    analysis.history_check_summary(runs)

if __name__=='__main__':unittest.main()
```

90. question4/code/test_q4_audit.py

源文件 SHA-256: fe81c0ba81f0e5ac9422e7be8eff60fabfd59b9711ecc935c3ab92433c091906 ; 代码行数: 88。


```

"""Tampering tests for the independent auditor using a freshly executed day."""
import copy
import json
from pathlib import Path
import shutil
import tempfile
import unittest
from unittest.mock import patch

import pandas as pd

import q4_runner as runner
from q4_audit import audit_all, audit_run, source_data, ROOT

class TestIndependentAudit(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.temp=tempfile.TemporaryDirectory()
        cls.work=Path(cls.temp.name)
        cls.source=source_data(ROOT/'data')
        # One real-data day is fast and gives a self-contained fixture, so the
        # tests do not depend on optional pilot or delivered result folders.
        runner.run_strategy(ROOT/'data', cls.work, (0,), branch='4-2', days=1,
            name='fixture', config={'scenarios':1, 'controller':'greedy'})
        cls.fixture=cls.work/'fixture'

    @classmethod
    def tearDownClass(cls):cls.temp.cleanup()

    def variant(self, name):
        path=self.work/name
        if path.exists():shutil.rmtree(path)
        shutil.copytree(self.fixture, path)
        return path

    def test_fresh_run_and_external_absolute_output_path_pass(self):
        result=audit_run(self.fixture, ROOT/'data', self.source)
        self.assertTrue(result['passed'])
        self.assertEqual(result['intervals'], 144)
        self.assertEqual(result['january_night_intervals'], 1115)
        self.assertEqual(result['path'], str(self.fixture.resolve()))

    def test_cash_column_tampering_is_detected(self):
        path=self.variant('cash')
        rows=pd.read_csv(path/'trace.csv');rows.loc[0, 'cash_cost_yuan']+=.1
        rows.to_csv(path/'trace.csv', index=False)
        with self.assertRaisesRegex(AssertionError, 'cash_cost_yuan'):
            audit_run(path, ROOT/'data', self.source)

    def test_unexecuted_or_altered_signed_order_is_detected(self):
        path=self.variant('order')
        doc=json.loads((path/'event_log.json').read_text())
        doc['events'][0][0]['signed_q'][0]+=1
        (path/'event_log.json').write_text(json.dumps(doc))
        with self.assertRaisesRegex(AssertionError, 'immutable original plan'):
            audit_run(path, ROOT/'data', self.source)

    def test_realized_price_cannot_be_relabeled_as_control_forecast(self):
        path=self.variant('future_price')
        rows=pd.read_csv(path/'trace.csv')
        rows.loc[0, 'price_forecast_yuan_per_kwh']+=.1
        rows.to_csv(path/'trace.csv', index=False)
        with self.assertRaisesRegex(AssertionError, 'known-left/predicted-right'):

```

```

        audit_run(path, ROOT/'data', self.source)

def test_changed_input_cannot_reuse_old_run_signature(self):
    path=self.variant('source')
    # The source field is reviewed before any physical values, so even an
    # apparently numerically identical old run may not claim provenance.
    doc=json.loads((path/'summary.json').read_text());doc['run_signature']='0'*64
    (path/'summary.json').write_text(json.dumps(doc))
    with self.assertRaisesRegex(AssertionError, 'signature mismatch'):
        audit_run(path, ROOT/'data', self.source)

def test_incomplete_declared_experiment_cannot_pass_as_full_audit(self):
    stage=self.work/'coverage';stage.mkdir(exist_ok=True)
    shutil.copytree(self.fixture, stage/'fixture', dirs_exist_ok=True)
    (stage/'design.json').write_text(json.dumps({'jobs': [{'name': 'fixture'}, {'name': 'missing'}]}))
    with patch('q4_audit.source_data', return_value=self.source):
        report=audit_all(ROOT/'data', stage)
    self.assertFalse(report['passed'])
    self.assertEqual(report['audited_runs'], 1)
    self.assertEqual(len(report['pending_runs']), 1)
    self.assertIn('missing', report['pending_runs'][0])

if __name__=='__main__':unittest.main()

```

91. question4/code/test_q4_causal_replay.py

源文件 SHA-256: c53900e8f2d91ce09428d40f586f4cbdef52b45731becc1cdb59f851190acb92 ; 代码行数: 205。

```

"""Independent causal/event/integration checks, using two complete toy windows."""
import copy
import csv
import datetime as dt
import json
from pathlib import Path
import shutil
import tempfile
import unittest
from unittest.mock import patch

import numpy as np
import pandas as pd

from q4_causal_replay import causal_price_vector, run_replay
from q4_causal_audit import audit_replay

def synthetic_case(folder, *, controller='mpc', trade=(0,6), mixed_supply=False):
    dates=[dt.date(2025,2,25)+dt.timedelta(days=i) for i in range(2)]
    starts=[dt.datetime.combine(d,dt.time(0,10)) for d in dates]
    ends=[d+dt.timedelta(days=1) for d in starts]
    x=np.arange(289,dtype=float)
    l=1800+x;g=300+.2*x;p=.5+.001*x
    nodes=lambda a:np.array([a[:145],a[144:]])
    load_nodes,pv_nodes,price_nodes=map(nodes,(l,g,p))
    mean=lambda a:(a[:,-1]+a[:,1:])/2
    load_a,pv_a,price_a=map(mean,(load_nodes,pv_nodes,price_nodes))
    price_nodes_f=np.repeat((price_nodes+.07)[:,:None,:],4,axis=1)
    price_f=mean(price_nodes)[:,:None,:]+np.array([.07,.06,.05,.04])[None,:,None]
    data=dict(dates=dates>window_start=starts>window_end=ends,
              load_nodes=load_nodes,pv_nodes=pv_nodes,price_nodes_a=price_nodes,
              load_a=load_a,pv_a=pv_a,price_a=price_a,load_f=load_a+400,
              q2_pv_f=pv_a*.8,price_nodes_f=price_nodes_f,price_f=price_f,
              bridge_price_forecast=np.array([.51,.63]),inventory_value=.4)
    events=[];rows=[]
    for day,date in enumerate(dates):
        base=np.full(144,100.+100*day)
        if mixed_supply:base+=300*(np.arange(144)//18%2)
        final=base.copy()
        if 6 in trade:final[35:]+=75.
        events.append(dict(window_date=str(date),kind='dayahead',
                          decision_at=dt.datetime.combine(date,dt.time()).isoformat()))
        if 6 in trade:
            events.append(dict(window_date=str(date),kind='adjust',commit_start=35,commit_end=144,
                              decision_at=dt.datetime.combine(date,dt.time(6)).isoformat()))
        for t in range(144):
            rows.append(dict(window_date=str(date),interval_start=
            (starts[day]+dt.timedelta(minutes=10*t)).isoformat(),
                          price_yuan_per_kwh=price_a[day,t],load_actual_kw=load_a[day,t],pv_actual_kw=pv_a[day,t],
                          load_forecast_kw=data['load_f']
            [day,t],g_kwh=base[t],q_kwh=final[t],soc_start_kwh=7654.))
        folder.mkdir()
        with (folder/'trace.csv').open('w',newline='') as f:
            writer=csv.DictWriter(f,fieldnames=list(rows[0]));writer.writeheader();writer.writerows(rows)
        cfg=dict(strategy='synthetic',branch='4-2',trade_hours=list(trade),forecast_hours=[0],
                  price_hours=[0,6,12,18],controller=controller)
        (folder/'event_log.json').write_text(json.dumps(dict(config=cfg,events=events)))
        (folder/'summary.json').write_text('{}')
    return data

class TestCausalReplay(unittest.TestCase):
    def test_current_actual_left_only_affects_current_price(self):

```

```

with tempfile.TemporaryDirectory() as temp:
    data=synthetic_case(Path(temp)/'source')
    original=causal_price_vector(data,0,35,6)
    changed=copy.deepcopy(data)
    changed['price_nodes_a'][0,36:]+=900
    changed['price_a'][0,35:]+=500
    np.testing.assert_array_equal(original,causal_price_vector(changed,0,35,6))
    changed['price_nodes_a'][0,35]+=.2
    altered=causal_price_vector(changed,0,35,6)
    self.assertAlmostEqual(altered[0]-original[0],.1)
    np.testing.assert_array_equal(altered[1:],original[1:])
    with self.assertRaises(ValueError):causal_price_vector(data,0,34,6)

def test_future_formal_q_is_hidden_until_its_gate_and_next_g_until_midnight(self):
    calls=[]
    def dispatch(L,G,p,state,**kw):
        booked=kw['booked_q'].copy();calls.append((len(p),booked,kw['future_adjust_mask'].copy()))
        deficit=np.maximum(L-G-booked,0)
        return dict(c=np.zeros_like(L),d=np.zeros_like(L),e=deficit,
                    optimality_proven=True,solve_seconds=0.)
    def fixed(L,G,p,booked,state,**kw):
        calls.append((len(p),booked.copy(),np.zeros(len(p),bool)))
        return dict(c=np.zeros_like(L),d=np.zeros_like(L),e=np.maximum(L-G-
booked,0),solve_seconds=0.)
    with tempfile.TemporaryDirectory() as temp:
        root=Path(temp);data=synthetic_case(root/'source')
        with
patch('q4_causal_replay.solve_dispatch',side_effect=dispatch),patch('q4_causal_replay.solve_fixed_contro
l',side_effect=fixed):
            result=run_replay(None,root/'source',root/'out',0,2,6000.,(1.,),data_override=data)
            # First window until 06:00: even the far future is the original100,
            # although the full source ledger already contains the signed175.
            for n,booked,future in calls[:35]:
                self.assertEqual(booked.max(),100.)
                self.assertTrue(future[-1])
            self.assertEqual(calls[35][0],109)
            self.assertEqual(calls[35][1].min(),175.)
            # The first bridge is the first145-step control call, after next
            # midnight, with old signed175 and new original200 (not future275).
            bridge=next(item for item in calls if item[0]==145)
            self.assertEqual(bridge[1][0],175.)
            np.testing.assert_array_equal(bridge[1][1:],np.full(144,200.))
            reveal=json.loads((root/'out/contract_reveals_1min_causal.json').read_text())
            self.assertEqual([r['kind'] for r in reveal],['dayahead','adjust','dayahead','adjust'])
            self.assertEqual(reveal[2]['at'],'2025-02-26T00:00:00')
            self.assertTrue(result['checks_passed'])
            self.assertEqual(result['main_trace_original_local_initial_soc'],7654.)
            self.assertEqual(result['summary']['1min_causal']['soc_start_kwh'],6000.)

def test_synthetic_two_window_physics_loss_and_refinement_integral(self):
    with tempfile.TemporaryDirectory() as temp:
        root=Path(temp);data=synthetic_case(root/'source',controller='greedy',trade=
(0,),mixed_supply=True)
        result=run_replay(None,root/'source',root/'out',0,2,6000.,(1.,.1),data_override=data)
        base=result['summary']['10min_mean']
        for label in ('10min_mean','1min_causal','0.1min_causal'):
            part=result['summary'][label]
            self.assertGreater(part['charge_kwh'],0)
            self.assertGreater(part['discharge_kwh'],0)
            # Independent aggregate battery balance, including conversionloss.
            self.assertAlmostEqual(part['soc_end_kwh']-6000.,
                part['charge_kwh']-part['discharge_kwh']-part['battery_loss_kwh'],places=7)
            self.assertEqual(part['ordinary_cost_yuan'],base['ordinary_cost_yuan'])
            with (root/'out'/f'trace_{label}.csv').open() as f: rows=list(csv.DictReader(f))
            for a,b in zip(rows,rows[1:]):
                self.assertEqual(a['interval_end'],b['interval_start'])

```

```

        self.assertAlmostEqual(float(a['soc_end_kwh']),float(b['soc_start_kwh']),places=9)
        self.assertEqual(rows[-1]['interval_end'],'2025-02-27T00:10:00')
        self.assertLess(part['solver']['max_violation_kwh'],1e-7)
    for key in ('load','pv'):
        coarse=result['signal_energy_integration_checks']['lmin_causal']['observed_bias_kwh']
[key]
        fine=result['signal_energy_integration_checks']['0.1min_causal']['observed_bias_kwh']
[key]
        self.assertAlmostEqual(coarse/10,fine,places=7)

def test_real_lp_path_runs_and_respects_battery_constraints(self):
    with tempfile.TemporaryDirectory() as temp:
        root=Path(temp);data=synthetic_case(root/'source',trade=(0,),mixed_supply=True)
        result=run_replay(None,root/'source',root/'out',0,1,6000.,(1.,),data_override=data)
        self.assertGreater(result['summary']['lmin_causal']['solver']['lp_solves'],0)
        self.assertTrue(result['checks_passed'])

def test_invalid_common_state_and_micro_step_are_rejected(self):
    with tempfile.TemporaryDirectory() as temp:
        root=Path(temp);data=synthetic_case(root/'source')
        for state,steps in ((1000.,(1.,)),(6000.,(3.,)),(6000.,(1.,1.))):
            with self.assertRaises(ValueError):
                run_replay(None,root/'source',root/'out',0,2,state,steps,data_override=data)

def test_independent_audit_rejects_future_price_and_cash_tampering(self):
    with tempfile.TemporaryDirectory() as temp:
        root=Path(temp);data=synthetic_case(root/'source',controller='greedy',trade=
(0,),mixed_supply=True)
        run_replay(None,root/'source',root/'out',0,2,6000.,(1.,),data_override=data)
        self.assertTrue(audit_replay(root/'out',data)['passed'])
        path=root/'out/trace_lmin_causal.csv'
        with path.open() as f:rows=list(csv.DictReader(f))
        original=rows[0]['control_price_forecast']
        rows[0]['control_price_forecast']=str(data['price_a'][0,0])
        def save():
            with path.open('w',newline='') as f:
                w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
        save()
        with self.assertRaisesRegex(ValueError,'unavailable price'):audit_replay(root/'out',data)
        rows[0]['control_price_forecast']=original
        rows[0]['cash_cost_yuan']=str(float(rows[0]['cash_cost_yuan'])+1)
        save()
        with self.assertRaises(AssertionError):audit_replay(root/'out',data)

def test_packaged_ledger_is_checked_even_when_old_original_still_exists(self):
    with tempfile.TemporaryDirectory() as temp:
        root=Path(temp);data=synthetic_case(root/'source',controller='greedy',trade=
(0,),mixed_supply=True)
        out=root/'final/causal_replay/synthetic'
        run_replay(None,root/'source',out,0,2,6000.,(1.,),data_override=data)
        parallel=root/'final/main/synthetic'
        shutil.copytree(root/'source',parallel)
        self.assertTrue(audit_replay(out,data)['passed'])
        # Keep the original unmodified and present; a path-based fallback
        # must not let that old file hide alteration in the package copy.
        with (parallel/'trace.csv').open('a') as stream:stream.write('\n')
        self.assertTrue((root/'source/trace.csv').exists())
        with self.assertRaisesRegex(ValueError,'Source digest mismatch'):audit_replay(out,data)

def test_summary_micro_state_and_reveal_record_tampering_are_rejected(self):
    with tempfile.TemporaryDirectory() as temp:
        root=Path(temp);data=synthetic_case(root/'source',controller='greedy',trade=(0,))
        out=root/'out';run_replay(None,root/'source',out,0,2,6000.,(1.,),data_override=data)
        self.assertTrue(audit_replay(out,data)['passed'])
        summary_path=out/'summary.json';original=summary_path.read_bytes()
        summary=json.loads(original);summary['summary']['lmin_causal']['cash_cost_yuan']+=1.

```

```
summary_path.write_text(json.dumps(summary))
with self.assertRaisesRegex(AssertionError, 'Summary'):audit_replay(out,data)
summary_path.write_bytes(original)
path=out/'micro_trace_1min-causal.csv.gz';original_micro=path.read_bytes()
micro=pd.read_csv(path)
# A constant SOC offset keeps every micro recursion unchanged and
# remains inside physical limits here, but breaks the actual link.
micro['soc_start_kwh']+=1.;micro['soc_end_kwh']+=1.
self.assertLessEqual(micro.soc_end_kwh.max(),10800.)
self.assertGreaterEqual(micro.soc_end_kwh.min(),1200.)
micro.to_csv(path,index=False,compression='gzip')
with self.assertRaises(AssertionError):audit_replay(out,data)
path.write_bytes(original_micro)
path=out/'contract_reveals_1min-causal.json'
reveal=json.loads(path.read_text());reveal[0]['at']='2025-02-
25T06:00:00';path.write_text(json.dumps(reveal))
with self.assertRaisesRegex(ValueError, 'Contract reveal'):audit_replay(out,data)

if __name__=='__main__':unittest.main()
```

92. question4/code/test_q4_figures.py

源文件 SHA-256: e0b0a1b54f48ebe9722f45081d7127f7dfba097f8c12bacbf964a6bc4c17bd1b ; 代码行数: 91。

```

"""Dynamic figure-table semantics: preserve freeze and match causal comparisons."""
from pathlib import Path
from copy import deepcopy
import tempfile
import unittest

import numpy as np
import pandas as pd

from q4_figures import
Run, Study, monthly_main, schedule_comparison, paired_price_modes, sensitivity_comparison

SCHEDULES=[(0,), (0,6), (0,12), (0,18), (0,6,12), (0,6,18), (0,12,18), (0,6,12,18)]

def fixture_run(name, branch, mode, schedule, base, days=334, start='2025-02-01', initial=6000.):
    values=np.full(days, float(base)); date=pd.date_range(start, periods=days).strftime('%Y-%m-%d')

    daily=pd.DataFrame(dict(window_date=date, cash_cost_yuan=values, inventory_adjusted_cost_yuan=values, soc_start_kwh=initial, soc_end_kwh=initial, emergency_kwh=np.zeros(days)))

    s=dict(config=dict(branch=branch, price_mode=mode, trade_hours=list(schedule), pv_source='supplied'), window_s=days, cash_cost_yuan=values.sum(), inventory_adjusted_cost_yuan=values.sum(), soc_start_kwh=initial, soc_end_kwh=initial, emergency_kwh=0.)
    return Run(name, Path('/fixture/no-reading'), s, daily)

def fixture_study():
    main={}; dev=[]
    for branch in ('4-2', '4-3'):
        for mode in ('point', 'joint'):
            for j, schedule in enumerate([(0,)] if branch=='4-2' else SCHEDULES):
                name='Q'+branch.replace('-', '')+'_'+mode+'_H'+str(j).join(map(str, schedule))
                # Joint is cheaper in this artificial test, but the freeze is point.
                main[name]=fixture_run(name, branch, mode, schedule, 1000+j*3-(10 if mode=='joint' else 0))
                if branch=='4-3':
                    dev.append(dict(strategy=name, branch=branch, price_mode=mode, trade_hours=list(schedule), score=7000-j*10, cash=7000-j*10))
            selection=dict(selected_q42='Q42_point_H0', selected_q43='Q43_point_H0_6_12_18', branch_selections={
                '4-2':{'price_mode':'point', 'trade_hours':[0]}, '4-3':{'price_mode':'point', 'trade_hours':[0,6,12,18]}}, development=dev)
            sens={}
            for branch in ('4-2', '4-3'):
                prefix='Q'+branch.replace('-', '')+'_'
                for label, base in [('reference', 1000), ('joint_reference', 900), ('joint_mean', 920), ('joint_shuffled', 910), ('history14', 1050), ('initial1200', 1200)]:
                    name=prefix+label; sens[name]=fixture_run(name, branch, 'joint' if label.startswith('joint') else 'point', (0,), base, 14, '2025-02-25', 1200. if label=='initial1200' else 6000.)
            return Study(Path('/fixture'), selection, main, sens, {})

class FigureTablesTests(unittest.TestCase):
    def test_holdout_rank_does_not_reselect_development_choice(self):
        study=fixture_study(); before=deepcopy(study.selection); table=schedule_comparison(study)
        self.assertEqual(len(table), 8); self.assertTrue(table.price_mode=='point').all()
        self.assertEqual(table[table.selected_before_test].strategy.iloc[0], 'Q43_point_H0_6_12_18')
        self.assertEqual(table[table.test_rank_retrospective==1].strategy.iloc[0], 'Q43_point_H0')
        self.assertEqual(study.selection, before)

    def test_seven_development_and_310_holdout_denominators_distinct(self):
        table=schedule_comparison(fixture_study())
        for row in table.itertuples():
            self.assertEqual(row.development_windows, 7); self.assertEqual(row.test_windows, 310)

```



```

self.assertAlmostEqual(row.development_daily_score_yuan*7, row.development_inventory_score_yuan)
    self.assertAlmostEqual(row.test_daily_score_yuan*310, row.test_inventory_score_yuan)

def test_pairing_uses_chosen_schedule_and_excludes_changed_pv_input(self):
    study=fixture_study(); confound=fixture_run('Q42_matched_Q43_PV', '4-2', 'joint', (0,), 1)
    confound.summary['config']['pv_source']='q3_all'; study.main[confound.name]=confound
    records, summary=paired_price_modes(study)
    self.assertEqual(len(records), 2*334)
    q42=summary[summary.branch=='4-2'].iloc[0]

self.assertEqual(q42.joint_strategy, 'Q42_joint_H0'); self.assertAlmostEqual(q42.joint_minus_point_cash_yuan, -3340.)
    q43=summary[summary.branch=='4-3'].iloc[0]
    self.assertEqual(q43.joint_strategy, 'Q43_joint_H0_6_12_18')

def test_unequal_pair_dates_fail_instead_of_aligning_by_row(self):
    study=fixture_study(); run=study.main['Q42_joint_H0']; run.daily.loc[0, 'window_date']='2025-01-31'
    with self.assertRaisesRegex(ValueError, 'different dates'): paired_price_modes(study)

def test_joint_layer_sensitivity_uses_its_own_conditional_reference(self):
    rows=sensitivity_comparison(fixture_study()); row=rows[rows.experiment=='Q42_joint_mean'].iloc[0]
    self.assertAlmostEqual(row.inventory_delta_yuan, (920-1000)*14)
    self.assertEqual(row.joint_layer_reference, 'Q42_joint_reference')
    self.assertAlmostEqual(row.joint_layer_inventory_delta_yuan, (920-900)*14)

def test_initial_stock_sensitivity_uses_inventory_score_not_raw_cash(self):
    study=fixture_study(); r=study.sensitivity['Q42_initial1200']
    r.summary['cash_cost_yuan']=10000.; r.summary['inventory_adjusted_cost_yuan']=15000.
    rows=sensitivity_comparison(study); row=rows[rows.experiment=='Q42_initial1200'].iloc[0]
    self.assertEqual(row.cash_delta_yuan, -4000.); self.assertEqual(row.inventory_delta_yuan, 1000.)
    self.assertAlmostEqual(row.inventory_delta_percent, 1000/14000*100)

def test_monthly_costs_sum_to_selected_contract_totals(self):
    study=fixture_study(); rows=monthly_main(study)
    self.assertEqual(len(rows), 22)
    for branch in ('4-2', '4-3'):
        group=rows[rows.branch==branch]
        self.assertEqual(group.windows.sum(), 334)

self.assertAlmostEqual(group.cash_cost_yuan.sum(), study.selected(branch).summary['cash_cost_yuan'])

if __name__=='__main__': unittest.main()

```

93. question4/code/test_q4_independent.py

源文件 SHA-256: 81949fd632beedcd06eb83db8412d64919eddf2833204642dba47f1abc0f7a5c ; 代码行数: 169。

```

"""Independent counterexamples for Q4 prices, contracts and execution.

These tests deliberately vary future information and scenario probabilities;
they do not merely repeat the implementation's own residual checks.
"""

import copy
import datetime as dt
import json
from pathlib import Path
import tempfile
import unittest
from unittest.mock import patch

import numpy as np
import pandas as pd

import q4_runner as runner
from q4_optimizer import solve_dispatch, solve_fixed_control, cash_components
from q4_risk import scenario_inputs

def synthetic_inputs(n=32):
    dates=[dt.date(2025,2,1)+dt.timedelta(days=i) for i in range(n)]
    starts=[dt.datetime.combine(d,dt.time(0,10)) for d in dates]
    power=np.full((n,144),1500.)
    pn=np.full((n,4,145),.5)
    data=dict(dates=dates,window_start=starts,window_end=[s+dt.timedelta(days=1) for s in starts],
        load_f=power.copy(),load_a=power.copy(),pv_a=np.zeros_like(power),
        q2_pv_f=np.zeros_like(power),pv_f=np.zeros((n,4,144)),
        pv_unused_first_mean_kw=np.zeros(n),price_f=(pn[:,:,:-1]+pn[:,:,:1])/2,
        price_nodes_f=pn,price_a=np.full_like(power,.5),price_nodes_a=np.full((n,145),.5),
        actual_estimated=np.zeros((n,144),bool),price_estimated=np.zeros((n,144),bool),
        inventory_value=.5,bridge_price_forecast=np.full(n,.5),
        first_unexecuted_interval={'load_mean_kw':1500.,'pv_mean_kw':0.},startup_price_actual=.5)
    data['policy_pv']=data['pv_f']
    return data

class TestIndependentOptimizer(unittest.TestCase):
    def test_two_dimensional_constant_prices_reduce_to_one_dimensional(self):
        L=np.array([[400,500,450],[450,480,520.]])
        G=np.array([[0,50,0],[10,0,20.]])
        p=np.array([.4,1.2,.8])
        kw=dict(probabilities=[.3,.7],terminal_weight=.6,time_limit=5,mip_gap=1e-9)
        a=solve_dispatch(L,G,p,1600.,**kw)
        b=solve_dispatch(L,G,np.tile(p,(2,1)),1600.,**kw)
        self.assertAlmostEqual(a['objective'],b['objective'],places=6)
        np.testing.assert_allclose(a['g'],b['g'],atol=1e-6)

    def test_shared_original_cost_uses_probability_weighted_price(self):
        # No PV, no usable initial inventory; identical 100 kWh demand. There
        # is no uncertain quantity to obscure this analytical cost check.
        sol=solve_dispatch(np.full((2,1),100.),np.zeros((2,1)),np.array([[1.],[3.]]),1200.,
            probabilities=[.75,.25],terminal_weight=0,time_limit=5,mip_gap=1e-9)
        self.assertAlmostEqual(sol['g'][0],100.,places=7)
        self.assertAlmostEqual(sol['original_cost_in_objective'],150.,places=7)
        self.assertAlmostEqual(sol['objective'],150.,places=7)

    def test_current_order_is_shared_despite_scenario_price_information(self):
        sol=solve_dispatch(np.array([[100.],[200.]]),np.zeros((2,1)),np.array([[1.],[3.]]),1200.,
            mode='adjust',base_g=[100.],booked_q=[100.],current_mask=[True],
            probabilities=[.75,.25],terminal_weight=0,time_limit=5,mip_gap=1e-9)
        np.testing.assert_allclose(sol['q_scenarios'][:,0],200.,atol=1e-6)
        self.assertAlmostEqual(sol['objective'],225.,places=6)

```

```

self.assertEqual(sol['signed_q'].shape, (1,))
with self.assertRaises(ValueError):
    solve_dispatch(np.ones((1,1)), np.zeros((1,1)), [1.], 1200., mode='control',
        base_g=[0.], booked_q=[0.], current_mask=[True], terminal_weight=0)

def test_lp_and_milp_have_same_fixed_contract_value(self):
    cases=[(np.array([100,850,900,400.]), np.array([400,0,0,0.]),
        np.array([0,100,100,600.]), 1500.),
        (np.array([700,200,800,400.]), np.zeros(4), np.array([800,100,100,0.]), 10500.)]
    p=np.array([.4,1.3,.9,.5])
    for L,G,q,s0 in cases:
        for weight in (0.,.5,2.):
            lp=solve_fixed_control(L,G,p,q,s0,terminal_weight=weight)
            mip=solve_dispatch(L[None,:],G[None,:],p,s0,mode='control',base_g=q,booked_q=q,
                terminal_weight=weight,time_limit=5,mip_gap=1e-9)
            self.assertAlmostEqual(lp['objective'],mip['objective'],places=5)

def test_refund_fee_and_positive_price_domain(self):
    self.assertEqual(cash_components([1],[100],[80],[0])['total_cash_cost'],90)
    self.assertEqual(cash_components([1],[100],[120],[0])['total_cash_cost'],130)
    for p in (0.,-1.):
        with self.assertRaises(ValueError):
            solve_dispatch(np.ones((1,1)), np.zeros((1,1)), [p], 1200., terminal_weight=0)

class TestIndependentRisk(unittest.TestCase):
    def test_joint_price_and_power_residual_dates_remain_paired(self):
        data=synthetic_inputs(32)
        for i in range(31):
            data['load_a'][i]=1500.+6*i
            data['pv_a'][i]=2*i
            data['price_a'][i]=.5*np.exp(i/100)
        cfg={**runner.DEFAULT, 'min_history':14, 'scenarios':8}
        L,G,P,meta=scenario_inputs(data,31,0,0,0,0,cfg)
        draw=np.array(meta['draw_indices'])
        np.testing.assert_allclose(L[:,0],250+draw)
        np.testing.assert_allclose(G[:,0],draw/3)
        np.testing.assert_allclose(P[:,0],.5*np.exp(draw/100))
        self.assertNotIn(30,meta['history_indices']) # previous tail unfinished
        self.assertTrue(all(data['window_end'][j]≤dt.datetime(2025,3,4) for j in draw))
        point=scenario_inputs(data,31,0,0,0,0,{**cfg, 'price_mode': 'point'})
        np.testing.assert_array_equal(point[0],L); np.testing.assert_array_equal(point[1],G)
        np.testing.assert_array_equal(point[2],np.full_like(P,.5))
        data['price_a'][meta['draw_indices'][0],0]=0
        with self.assertRaises(ValueError):scenario_inputs(data,31,0,0,0,0,cfg)

class TestIndependentRunner(unittest.TestCase):
    def run_synthetic(self, data, out, name, days=1, **kwargs):
        config=
        {**runner.DEFAULT, 'scenarios':1, 'controller':'mpc', 'terminal_multiplier':1., 'time_limit':5}
        config.update(kwargs.pop('config', {}))
        with patch.object(runner, 'load_inputs', return_value=data):
            result=runner.run_strategy(Path(out)/'empty_data', out, (0,), branch='4-2',
                name=name, days=days, config=config, **kwargs)
            path=Path(out)/name
            return result, pd.read_csv(path/'trace.csv'), json.loads((path/'event_log.json').read_text())

def test_future_actual_price_does_not_change_current_order_or_first_action(self):
    a=synthetic_inputs();b=copy.deepcopy(a)
    # First left node is unchanged; unknown right nodes and their resulting
    # settlement prices change. Supplied forecast versions remain frozen.
    b['price_nodes_a'][0,1:]+=4
    b['price_a'][0]=(b['price_nodes_a'][0,:-1]+b['price_nodes_a'][0,1:])/2
    with tempfile.TemporaryDirectory() as out:
        _,ra,ea=self.run_synthetic(a,out,'a')

```

```

        _,rb,eb=self.run_synthetic(b,out,'b')
        np.testing.assert_array_equal(ea['events'][0]['signed_q'],eb['events'][0]['signed_q'])
    for col in
('g_kwh','q_kwh','charge_kwh','discharge_kwh','emergency_kwh','price_forecast_yuan_per_kwh'):
        self.assertEqual(ra.iloc[0][col],rb.iloc[0][col],places=7)
        self.assertNotEqual(ra.iloc[0]['price_yuan_per_kwh'],rb.iloc[0]['price_yuan_per_kwh'])

def test_price_update_does_not_grant_trade_or_unused_pv_permission(self):
    a=synthetic_inputs();b=copy.deepcopy(a)
    b['pv_f'][:,1:]=1e8
    with tempfile.TemporaryDirectory() as out:
        _,ra,ea=self.run_synthetic(a,out,'a')
        _,rb,eb=self.run_synthetic(b,out,'b')
        self.assertEqual([e['kind'] for e in ea['events']],['dayahead'])
        self.assertEqual(set(ra['price_forecast_hour']),{0,6,12,18})
        np.testing.assert_array_equal(ea['events'][0]['signed_q'],eb['events'][0]['signed_q'])
        np.testing.assert_array_equal(ra['q_kwh'],rb['q_kwh'])

def test_midnight_bridge_uses_new_price_vintage_but_old_contract(self):
    data=synthetic_inputs()
    data['bridge_price_forecast'][1]=1.4
    seen=[]
    original=runner.solve_dispatch
    def spy(L,G,P,S,**kwargs):
        if kwargs.get('mode')=='dayahead' and L.shape[1]==145:
            seen.append(np.asarray(P).copy())
        return original(L,G,P,S,**kwargs)
    with tempfile.TemporaryDirectory() as out,patch.object(runner,'solve_dispatch',side_effect=spy):
        _,rows,events=self.run_synthetic(data,out,'bridge',days=2)
        self.assertEqual(len(seen),1)
        np.testing.assert_allclose(seen[0][:,0],1.4)
        bridge=rows.iloc[143]
        self.assertEqual(bridge['window_date'],'2025-02-01')
        self.assertEqual(bridge['interval_start'],'2025-02-02T00:00:00')
        self.assertEqual(bridge['price_forecast_issue_time'],'2025-02-02T00:00:00')
        self.assertEqual(bridge['price_forecast_yuan_per_kwh'],1.4)
        self.assertEqual(bridge['q_kwh'],events['events'][0]['signed_q'][-1])
        self.assertEqual(len(events['events'][1]['signed_q']),144)

if __name__=='__main__':unittest.main()

```

94. question4/code/test_q4_reports.py

源文件 SHA-256: 73c0f4d079bd7eed0c5f96cce1f2b444f20d86559317ebe36624a19c1647ec39 ; 代码行数: 130。

```

"""Meaningful Q4 reporting checks with physically balanced toy ledgers."""
from pathlib import Path
from datetime import datetime, timedelta
from copy import deepcopy
import hashlib
import tempfile
import unittest

from openpyxl import load_workbook
from q4_reports import (EPS, audit_trace, day_groups, episodes, export_result, paper_tables,
                        summarize, total, _contract_groups, DATES)

ROOT=Path(__file__).resolve().parents[1]

def toy_ledger(variant='Q4-3', dates=2):
    allrows=[]; s=6000.; start=datetime(2025, 2, 1)
    for n in range(dates*144+1):
        a=start+timedelta(minutes=10*n); b=a+timedelta(minutes=10)
        # Contract D starts at D00:10; natural-day midnight belongs to D-1.
        contract=(a-timedelta(minutes=10)).date().isoformat()
        c=d=0.; q=70.+n%20; e=100.-q
        if n==6: c=50.; q=150.; e=0.
        if n==9: d=40.5; q=59.5; e=0.
        g=q+(10. if variant=='Q4-3' else 0.)
        down=g-q; p=.2+(n%6)*.03
        end=s+.9*c-d/.9

    row=dict(strategy='test', window_date=contract, interval_start=a.isoformat(), interval_end=b.isoformat(),

    price_yuan_per_kwh=p, price_forecast_yuan_per_kwh=1.7, load_forecast_kw=590., pv_forecast_kw=0., load_actual
    _kw=600., pv_actual_kw=0.,

    g_kwh=g, q_kwh=q, increase_kwh=0., decrease_kwh=down, charge_kwh=c, discharge_kwh=d, emergency_kwh=e,
        unused_paid_kwh=0., curtailed_pv_kwh=0., soc_start_kwh=s, soc_end_kwh=end,
        plan_cost_yuan=p*g, increase_cost_yuan=0., refund_yuan=p*down, penalty_yuan=.5*p*down,

    adjustment_net_cost_yuan=-.5*p*down, emergency_cost_yuan=5*p*e, cash_cost_yuan=p*g-.5*p*down+5*p*e,
        actual_estimated=False, price_estimated=n==dates*144)
    allrows.append(row); s=end
    return allrows[1:], allrows[:1]

class Q4ReportTests(unittest.TestCase):
    def setUp(self): self.temp=tempfile.TemporaryDirectory(); self.out=Path(self.temp.name)
    def tearDown(self): self.temp.cleanup()

    def test_actual_not_forecast_price_sets_all_costs(self):
        rows, startup=toy_ledger(); audit=audit_trace(startup+rows, label='fixture')
        self.assertLess(audit['max_cash_error_yuan'], 1e-9)
        changed=deepcopy(rows); changed[0]['plan_cost_yuan']=changed[0]
        ['price_forecast_yuan_per_kwh']*changed[0]['g_kwh']
        with self.assertRaisesRegex(AssertionError, 'original cost'): audit_trace(changed, label='incorrect
        forecast settlement')

    def test_cancellation_refund_is_subtracted_once(self):
        rows, _=toy_ledger(); r=rows[0]; p=r['price_yuan_per_kwh']
        self.assertAlmostEqual(r['refund_yuan'], 10*p); self.assertAlmostEqual(r['penalty_yuan'], 5*p)
        self.assertAlmostEqual(r['cash_cost_yuan'], p*r['g_kwh']-5*p+5*p*r['emergency_kwh'])
        changed=deepcopy(rows); changed[0]['cash_cost_yuan']+=changed[0]['refund_yuan']
        with self.assertRaisesRegex(AssertionError, 'cash'): audit_trace(changed, label='no-refund wrong
        cost')

    def test_natural_day_uses_previous_contract_and_real_soc(self):

```

```

        rows, startup = toy_ledger(); groups = day_groups(startup + rows)
        self.assertEqual(len(groups), 2)
        day = groups['2025-02-02']; self.assertEqual(len(day), 144)
        self.assertEqual(day[0]['window_date'], '2025-02-01'); self.assertEqual(day[1]
['window_date'], '2025-02-02')
        first = groups['2025-02-01']; self.assertEqual(total(first[:24], 'charge_kwh'), 50.)
        self.assertEqual(total(first[:24], 'discharge_kwh'), 40.5)
        self.assertEqual(first[0]['soc_start_kwh'], 6000.); self.assertEqual(first[-1]
['soc_end_kwh'], 6000.)

    def test_contract_natural_fee_bridge_identity(self):
        rows, startup = toy_ledger(); groups = day_groups(startup + rows)
        natural = sum(total(day, 'cash_cost_yuan') for day in groups.values()); tail = rows[-1]
['cash_cost_yuan']
        self.assertEqual(natural, total(rows, 'cash_cost_yuan') + total(startup, 'cash_cost_yuan') -
tail)
        self.assertEqual(rows[-1]['interval_start'], '2025-02-03T00:00:00')

    def test_emergency_episodes_split_at_midnight_and_conserve_energy(self):
        rows, startup = toy_ledger(); events = episodes(startup + rows)
        self.assertTrue(any(e['label'].endswith('24:00') for e in events))
        self.assertTrue(any(e['date'] == '2025-02-03' and e['label'] == '00:00-00:10' for e in events))
        self.assertEqual(sum(e['kwh'] for e in events), total(startup + rows, 'emergency_kwh'))

    def test_shifted_contract_or_missing_startup_refused(self):
        rows, startup = toy_ledger(); changed = deepcopy(rows); changed[0]['interval_start'] = '2025-02-
01T00:00:00'
        with self.assertRaisesRegex(AssertionError, 'Shifted/missing'): _contract_groups(changed)
        with self.assertRaisesRegex(AssertionError, 'all 144'):
            export_result(ROOT/'data/result4-3_template.xlsx', self.out/'bad.xlsx', rows, [], 'test', 'Q4-3',
{})

    def test_q42_template_fee_includes_emergency_and_preserves_headers(self):
        rows, startup = toy_ledger('Q4-2'); template = ROOT/'data/result4-
2_template.xlsx'; before = hashlib.sha256(template.read_bytes()).hexdigest()
        checked, natural = export_result(template, self.out/'result4-2.xlsx', rows, startup, 'test', 'Q4-2', {})

self.assertEqual(checked['storage_blocks'], 12); self.assertEqual(hashlib.sha256(template.read_bytes()).he
xdigest(), before)
        book = load_workbook(self.out/'result4-2.xlsx', data_only=True)
        try:
            first = list(_contract_groups(rows).values())[0]
            self.assertEqual(book['计划购电量'].cell(1, 145).value, '0:00-0:10+1')
            self.assertEqual(book['计划购电量'].cell(2, 147).value, total(first, 'cash_cost_yuan'))
            self.assertEqual(book['计划购电量'].cell(2, 146).value, total(first, 'g_kwh'))
            self.assertNotIn('调整购电量', book.sheetnames)
            self.assertEqual(book['充放电量'].max_row, 13)
        finally: book.close()

    def test_q43_template_separates_original_and_final_quantities(self):
        rows, startup = toy_ledger(); checked, natural = export_result(ROOT/'data/result4-
3_template.xlsx', self.out/'result4-3.xlsx', rows, startup, 'test', 'Q4-3', {})
        book = load_workbook(self.out/'result4-3.xlsx', data_only=True)
        try:
            first = list(_contract_groups(rows).values())[0]
            self.assertEqual(book['计划购电量'].cell(2, 147).value, total(first, 'plan_cost_yuan'))
            self.assertEqual(book['调整购电量'].cell(2, 147).value, total(first, 'cash_cost_yuan'))
            self.assertEqual(book['调整购电量'].cell(2, 2).value, rows[0]['q_kwh'])
            self.assertNotEqual(book['调整购电量'].cell(2, 2).value, rows[0]['q_kwh'] - rows[0]['g_kwh'])
            dates = [book['紧急购电量'].cell(i, 1).value for i in range(2, book['紧急购电量'].max_row + 1)]
            self.assertNotIn(datetime(2025, 2, 3), dates)
            self.assertEqual(book['跨年合同末段'].max_row, 2)
        finally: book.close()

    def test_four_required_days_refuse_partial_and_use_specific_slots(self):
        rows, startup = toy_ledger(); natural = day_groups(startup + rows)

```

```
with self.assertRaisesRegex(AssertionError, 'Missing required'):
    paper_tables(self.out/'bad.xlsx', self.out/'bad.md', natural, 'test', 'Q4-3')
fixture={}
source=natural['2025-02-01']
for stamp in DATES:
    shift=datetime.fromisoformat(stamp)-datetime(2025, 2, 1); copy=deepcopy(source)
    for row in copy:
        row['interval_start']=(datetime.fromisoformat(row['interval_start'])+shift).isoformat()
        row['interval_end']=(datetime.fromisoformat(row['interval_end'])+shift).isoformat()
    fixture[stamp]=copy
checked, summaries=paper_tables(self.out/'four.xlsx', self.out/'four.md', fixture, 'test', 'Q4-3')
self.assertEqual(len(summaries), 4)
book=load_workbook(self.out/'four.xlsx', data_only=True)
try: self.assertEqual(book['表1_指定购电'].cell(3, 2).value, fixture[DATES[0]][60]['q_kwh'])
finally: book.close()

if __name__=='__main__': unittest.main()
```

95. question4/code/test_q4_solver_check.py

源文件 SHA-256: 0355d496b16a992ca1c6d006205f629cfd73e2378a7bcd496520637bb43f8374 ; 代码行数: 48。


```

import unittest
import numpy as np
from q4_solver_check import available_contract

class ReconstructionTests(unittest.TestCase):
    def test_unrevealed_future_order_hidden(self):
        np.testing.assert_array_equal(available_contract([10,20,30],[11,22,999],[False,False,True]),
[11,22,30])
    def test_future_order_perturbation_invariant(self):
        g=[1,2,3];mask=[False,True,True]
        np.testing.assert_array_equal(available_contract(g,[4,5,6],mask),available_contract(g,
[4,999,888],mask))
    def test_all_original_before_first_gate(self):
        np.testing.assert_array_equal(available_contract([1,2],[8,9],[True,True]),[1,2])
    def test_shape_mismatch_rejected(self):
        with self.assertRaises(ValueError):available_contract([1,2],[3],[False,True])

class FullInputReconstructionTests(unittest.TestCase):
    def fixture(self):
        import datetime as dt
        import pandas as pd
        date=dt.date(2025,2,2);start=dt.datetime(2025,2,2,0,10)
        data={'dates':[date],'window_start':[start],'load_f':np.full((1,144),12.),
            'load_a':np.full((1,144),12.),'pv_a':np.zeros((1,144)),
            'policy_pv':np.zeros((1,4,144)), 'price_f':np.full((1,4,144),2.),
            'price_nodes_a':np.ones((1,145)), 'price_nodes_f':np.full((1,4,145),3.)}
        frame=pd.DataFrame({'window_date':[str(date)]*144,
            'interval_start':[str(start+dt.timedelta(minutes=10*t)) for t in range(144)],
            'g_kwh':np.ones(144), 'q_kwh':np.r_[np.ones(107),np.full(37,999.)],
            'soc_start_kwh':np.full(144,6000.), 'price_forecast_yuan_per_kwh':np.full(144,2.),
            'pv_forecast_kw':np.zeros(144), 'charge_kwh':np.zeros(144),
            'discharge_kwh':np.ones(144), 'emergency_kwh':np.zeros(144)})
        config={'forecast_hours':[0,18], 'price_hours':[0,6,12,18], 'trade_hours':[0,18]}
        return data,frame,config
    def test_full_reconstruction_hides_future_contracts(self):
        from q4_solver_check import reconstruct_control
        data,frame,config=self.fixture();r=reconstruct_control(data,frame,config,0,0)
        np.testing.assert_array_equal(r['booked_q'],np.ones(144))
    def test_unknown_actual_right_price_not_used(self):
        from q4_solver_check import reconstruct_control
        data,frame,config=self.fixture();first=reconstruct_control(data,frame,config,0,0)
        data['price_nodes_a'][0,1:]=100000
        other=reconstruct_control(data,frame,config,0,0)
        np.testing.assert_array_equal(first['price'],other['price'])
    def test_bridge_step_rejected(self):
        from q4_solver_check import reconstruct_control
        data,frame,config=self.fixture()
        with self.assertRaises(ValueError):reconstruct_control(data,frame,config,0,143)

if __name__=='__main__':unittest.main()

```

96. question4/code/verify_document.py

源文件 SHA-256: 136b62e3ce4db20fbe0507b6046697c6ec7795029562451e72485388ab4982fd ; 代码行数: 75。

```

"""Verify native formulas, table/media objects, PDF bounds and result values."""
from pathlib import Path
import argparse, hashlib, json, re, subprocess, shutil, zipfile
from xml.etree import ElementTree as ET

ROOT=Path(__file__).resolve().parents[1]
def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()

def verify_document(results_dir=ROOT/'results/final', render=False, visual_reviewed=False):
    import pdfplumber
    result=Path(results_dir).resolve();reports=result/'reports'
    docx=reports/'第四问_模型与结果说明.docx';pdf=docx.with_suffix('.pdf')
    build=result/'document_build';origin=json.loads((reports/'文稿生成记录.json').read_text())
    if sha(docx)≠origin['docx_sha256'] or sha(pdf)≠origin['pdf_sha256']:raise ValueError('Document
source chain changed')
    for filename,expected in origin['sources'].items():
        source=Path(filename)
        if not source.is_absolute():source=ROOT/source
        if not source.exists() or sha(source)≠expected:raise ValueError('Changed narrative or figure
source: '+filename)
        with zipfile.ZipFile(docx) as z:
            if z.testzip() is not None:raise ValueError('DOCX CRC error')
            root=ET.fromstring(z.read('word/document.xml'))
            ns=
{'m':'http://schemas.openxmlformats.org/officeDocument/2006/math','w':'http://schemas.openxmlformats.org
/wordprocessingml/2006/main'}
            text=''.join(x.text or '' for x in root.findall('.//w:t',ns))
            formulas=len(root.findall('.//m:oMath',ns));tables=len(root.findall('.//w:tbl',ns))
            if formulas<6 or 'EQ_' in text or r'\_' in text or '$$' in text:raise ValueError('Formula
objects or placeholders invalid')
            texts=[];outside=[]
            with pdfplumber.open(pdf) as book:
                count=len(book.pages)
                for i,p in enumerate(book.pages):
                    texts.append(p.extract_text() or '')
                    for ch in p.chars:
                        if ch['x0']<-.5 or ch['x1']>p.width+.5 or ch['top']<-.5 or ch['bottom']>p.height+.5:
                            outside.append({'page':i+1,'text':ch['text']})
            combined=''.join(texts).replace(' ','')
            if outside:raise ValueError(f'Characters outside PDF pages: {outside[:8]}')
            chinese=len(re.findall('[\u4e00-\u9fff]',combined))
            if chinese<3500 or count<6:raise ValueError('Possible missing text/font or truncation')
            a=json.loads((result/'analysis/analysis.json').read_text());selected={a['selection']
['selected_q42'],a['selection']['selected_q43']}
            required=[f'{r["full_cash_yuan"]:.2f}' for r in a['main_summary'] if r['strategy'] in selected]
            for number in required:
                if number not in combined:raise ValueError('Missing main result in PDF: '+number)
            if '/Users/' in combined:raise ValueError('Machine path leaked into reader document')
            if render:
                executable=shutil.which('pdftoppm')
                if not executable:raise RuntimeError('Install pdftoppm for page rendering')
                folder=build/'rendered_pages';folder.mkdir(parents=True,exist_ok=True)
                for pattern in ('page-*.png','contact-*.png'):
                    for previous in folder.glob(pattern):previous.unlink()
                subprocess.run([executable,'-r','100','-png',str(pdf),str(folder/'page')],check=True)
                from PIL import Image,ImageDraw
                files=sorted(folder.glob('page-*.png'),key=lambda p:int(p.stem.split('-')[1]))
                if len(files)≠count:raise ValueError('Rendered page count differs')
                for batch in range(0,count,6):
                    canvas=Image.new('RGB',(1200,1740),'#d4d4d4');draw=ImageDraw.Draw(canvas)
                    for j,p in enumerate(files[batch:batch+6]):
                        im=Image.open(p).convert('RGB');im.thumbnail((390,825))
                        x=j%3*400+(400-im.width)//2;y=j//3*870+30

```

```

        canvas.paste(im,(x,y));draw.text((x,y-20),p.stem,fill='black')
        canvas.save(folder/f'contact-{batch//6+1}.png')
    record={'passed':True,'visual_reviewed':bool(visual_reviewed),'pages':count,
        'visual_pages_reviewed':list(range(1,count+1)) if visual_reviewed else [],
        'native_formulas':formulas,'tables':tables,'figures':origin['counts']['figures'],
        'chinese_characters':chinese,'out_of_page_characters':outside,
        'main_cash_values_verified':required,'source_chain_verified':True,
        'scope':'Q4 stage explanation, not full contest paper; no full-paper cover/abstract/length
assertion',
        'files':{'docx_sha256':sha(docx),'pdf_sha256':sha(pdf)}}
    (reports/'文稿验证.json').write_text(json.dumps(record,ensure_ascii=False,indent=2),encoding='utf-8')
    return record

if __name__=='__main__':
    p=argparse.ArgumentParser();p.add_argument('--results',type=Path,default=ROOT/'results/final')
    p.add_argument('--render',action='store_true');p.add_argument('--visual-
reviewed',action='store_true')

args=p.parse_args();print(json.dumps(verify_document(args.results,args.render,args.visual_reviewed),ensu
re_ascii=False,indent=2))

```

97. question4/code/xgb_intraday.py

源文件 SHA-256: b00aeb846fd5143f2e241075f3ae9a671d9344feb05c3ce4f8d7d3f9d90972fa ; 代码行数: 112。

```

"""Price-only XGBoost updates at 06/12/18 with frozen Q4 hyperparameters.

Each update learns from previous forecast-origin days at the SAME issue hour.
Every historical feature is reconstructed from its own information prefix.
The target is only the remaining nodes through next-day 00:10. This is a
declared extension of the midnight forecaster, not a newly tuned model.
"""

from __future__ import annotations

import time
import numpy as np
import pandas as pd

FEATURE_NAMES = ['slot', 'horizon', 'target_weekday', 'sin1', 'cos1', 'sin2', 'cos2', 'sin3', 'cos3',
                  'same_clock_latest', 'target_lag2d', 'target_lag3d', 'target_lag7d', 'mean_last3_same_clock',
                  'mean_last7_same_clock', 'std_last7_same_clock', 'latest_minus_mean7', 'known_issue_price',
                  'last6_mean', 'last18_mean', 'previous_24h_mean', 'previous_24h_std', 'previous_24h_min',
                  'previous_24h_max', 'last3day_mean', 'last7day_mean', 'issue_change_1d', 'lag1_unavailable']

def features_at_issue(history: np.ndarray, day: int, hour: int,
                     start_date: str = '2025-01-01') → tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Features for future right nodes; history beyond the issue is ignored.

    Raw observation index k is at Jan1 00:10 + k*10min. Therefore a release
    at day*24h+hour has exactly day*144+hour*6 available observations.
    For midnight only, this reduces numerically to the previous Q4 feature
    contract. Midnight predictions themselves must still be inherited.
    """
    if type(day) is not int or day < 7 or hour not in (0, 6, 12, 18):
        raise ValueError('Need ≥7 historical days and a 0/6/12/18 issue hour')
    cutoff = day*144 + hour*6
    hist = np.asarray(history, dtype=float).reshape(-1)[:cutoff]
    if len(hist) ≠ cutoff or not np.isfinite(hist).all():
        raise ValueError('History does not contain the complete known issue-time prefix')
    steps = 145-hour*6
    h = np.arange(1, steps+1)
    targets = cutoff+h-1
    slot = targets%144
    # Even lead145 at midnight uses a two-day-lag node if its one-day-lag
    # target is not observed yet. No negative or future indexing is allowed.
    cycles = (h+143)//144
    latest = targets-cycles*144
    previous_indices = latest[:, None]-np.arange(7)[None, :]*144
    assert previous_indices.min() ≥ 0 and previous_indices.max() < cutoff
    previous = hist[previous_indices]
    lag2 = hist[targets-288]; lag3=hist[targets-432]; lag7=hist[targets-1008]
    phase = 2*np.pi*((targets+1)%144)/144
    target_dates = pd.Timestamp(start_date)+pd.to_timedelta((targets+1)*10, unit='min')
    repeated=lambd value=np.full(steps, float(value))
    features=np.column_stack([slot, h, np.asarray(target_dates.dayofweek), np.sin(phase), np.cos(phase),
        np.sin(2*phase), np.cos(2*phase), np.sin(3*phase), np.cos(3*phase),
        previous[:, 0], lag2, lag3, lag7, previous[:, :3].mean(axis=1), previous.mean(axis=1),
        previous.std(axis=1), previous[:, 0]-previous.mean(axis=1), repeated(hist[-1]),
        repeated(hist[-6:].mean()), repeated(hist[-18:].mean()), repeated(hist[-144:].mean()),
        repeated(hist[-144:].std()), repeated(hist[-144:].min()), repeated(hist[-144:].max()),
        repeated(hist[-432:].mean()), repeated(hist[-1008:].mean()), repeated(hist[-1]-hist[-145]),
        targets-144 ≥ cutoff])
    assert features.shape==(steps, len(FEATURE_NAMES))
    return features, lag7, targets

def training_arrays_at_issue(history: np.ndarray, day: int, hour: int,
                             window_days: int | None = 28):

```

```

    if hour not in (0,6,12,18): raise ValueError('Invalid issue hour')
    cutoff=day*144+hour*6
    hist=np.asarray(history,float).reshape(-1)[:cutoff]
    if len(hist)!=cutoff or not np.isfinite(hist).all():raise ValueError('Invalid current history
cutoff')
    if window_days is not None and (type(window_days) is not int or window_days<7):
        raise ValueError('Training window must be ≥7 origin days')
    first=max(7,0 if window_days is None else day-window_days)
    X=[];y=[];anchors=[];label_indices=[];origin_days=[]
    for origin in range(first,day):
        xx,aa,tt=features_at_issue(hist,origin,hour)
        keep=tt<cutoff

X.append(xx[keep]);y.append(hist[tt[keep]]);anchors.append(aa[keep]);label_indices.extend(tt[keep]);orig
in_days.extend([origin]*int(keep.sum()))
    if not X:raise ValueError('No completed historical same-hour training labels')
    diagnostics={'training_rows':sum(len(a) for a in y),'first_training_origin_day':first,
        'last_training_origin_day':day-1,'same_issue_hour':hour,
        'max_label_index':int(max(label_indices)),'cutoff_last_index':cutoff-1,
        'last_training_feature_source_index':(day-1)*144+hour*6-1,
        'window_interpretation':'previous W same-hour forecast-origin days; each historical feature uses
its own prefix',
        'label_indices':np.array(label_indices,dtype=int),'origin_days':np.array(origin_days,dtype=int)}
    return np.vstack(X),np.concatenate(y),np.concatenate(anchors),diagnostics

def forecast_intraday(history: np.ndarray, day: int, hour: int, config: dict):
    """Return 109/73/37 predicted future right nodes; never clip prices."""
    import xgboost as xgb
    if hour not in (6,12,18):raise ValueError('Midnight must use the frozen supplied forecasts')
    cfg=dict(config)
    if cfg.get('family')!='XGBoost':raise ValueError('Frozen XGBoost configuration required')
    t0=time.perf_counter()
    X,y,anchor,diag=training_arrays_at_issue(history,day,hour,cfg.get('window_days',28))
    Xp,ap,_=features_at_issue(history,day,hour)
    target=cfg.get('target','level')
    if target not in ('level','weekly_residual'):raise ValueError('Unknown frozen target transform')
    fit_y=y-anchor if target=='weekly_residual' else y

    model=xgb.XGBRegressor(n_estimators=cfg.get('n_estimators',200),learning_rate=cfg.get('learning_rate',.0
4),

    reg_lambda=cfg.get('reg_lambda',5.),reg_alpha=cfg.get('reg_alpha',0.),random_state=cfg.get('seed',202609
12),n_jobs=1,
        objective='reg:squarederror',tree_method='hist',max_bin=128,max_depth=cfg.get('max_depth',3),
        min_child_weight=cfg.get('min_child_weight',20),subsample=.9,colsample_bytree=.9)
    model.fit(X,fit_y)
    prediction=np.asarray(model.predict(Xp),dtype=float)
    if target=='weekly_residual':prediction+=ap
    if not np.isfinite(prediction).all():raise RuntimeError('Nonfinite intraday price forecast')
    diag.pop('label_indices');diag.pop('origin_days')
    diag.update(config=cfg,day_index=day,issue_hour=hour,forecast_nodes=len(prediction),
        fit_seconds=time.perf_counter()-t0,negative_or_zero_predictions=int((prediction≤0).sum()),
        minimum_prediction=float(prediction.min()),clipped=False,hyperparameters_retuned=False,
        current_feature_source_last_index=day*144+hour*6-1)
    return prediction,diag

```